

Chapter 1 First acquaintance with ZYNQ

1.1 Chapter Introduction

This chapter will introduce the embedded system architecture of ZYNQ and show you step by step how to develop the PS side of ZYNQ.

1.2 Theoretical study

1.2.1 What is ZYNQ

ZYNQ can be divided into the following two concepts:

P.S. (Processing System): a processor based on dual ARM Cortex A9 systems.

PL (Programmable Logic): Programmable logic, ie FPGA. The PL side can be used to customize peripherals for the PS side.

When using the PL side alone, you can think of ZYNQ as a pure FPGA. When PL is used with PS, you can think of the PL side as peripherals of the PS side. At this time, you can think of the PL side as customizing peripherals for the PS side, such as HDMI interfaces, cameras, etc.

To put it simply, this ZYNQ is an ARM with an external FPGA, but it is different from the ordinary FPGA+MCU design on the same PCB. ZYNQ can achieve high-speed communication between FPGA and MCU, while the FPGA+MCU design on the market is not feasible and generally uses SPI communication. ZYNQ can realize high-speed communication between PL and PS through AXI.

1.2.2 ZYNQ's minimal embedded system

First, let's introduce what ZYNQ's minimum embedded system looks like, as shown in the figure below.

Normally, ZYNQ's DDR must be placed on the PS side, because the PS side program must be run in the DDR. Of course, you can also hang DDR on both the PL side and the PS side. In any case, the PS side must hang DDR, otherwise the PS side will not be able to run. And if there is no DDR on the PL side, then we will access the DDR on the PS side through the HP interface to achieve data interaction.

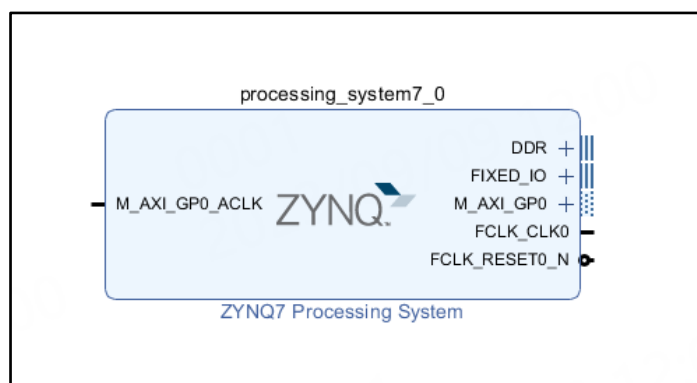


Figure 1-1 ZYNQ IP

1.2.3 ZYNQ development process

Next, we will introduce the general development process of the PS side of ZYNQ, which is generally divided into



Figure 1-2 Development process

As shown in the figure above, the development process can be roughly divided into 8 steps. The first to sixth steps are the hardware design part, which is carried out in Vivado. Steps seven to eight are the software design part, which is to create a new project in the SDK and write code. The eighth step is to go on the board to verify whether our program is correct. We may need to debug in subsequent complex projects.

1.3 Practical exercises

1.3.1 Experimental goals

Use the UART on the PS side to output data to the serial port assistant on the computer.

1.3.2 Hardware resources

The UART physical diagram and schematic diagram of the Haoyue development board are shown in Figure 1-3 and

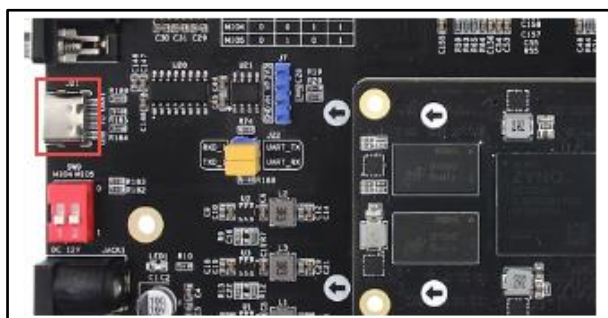


Figure 1-3 PS_UART



Figure 1-4 Development board UART0 pin

1.3.3 Programming

1. Overall description

In this project, we need to call DDR to store and run programs, and UART serial port to print information. The overall structure is shown in Figure 1-5.

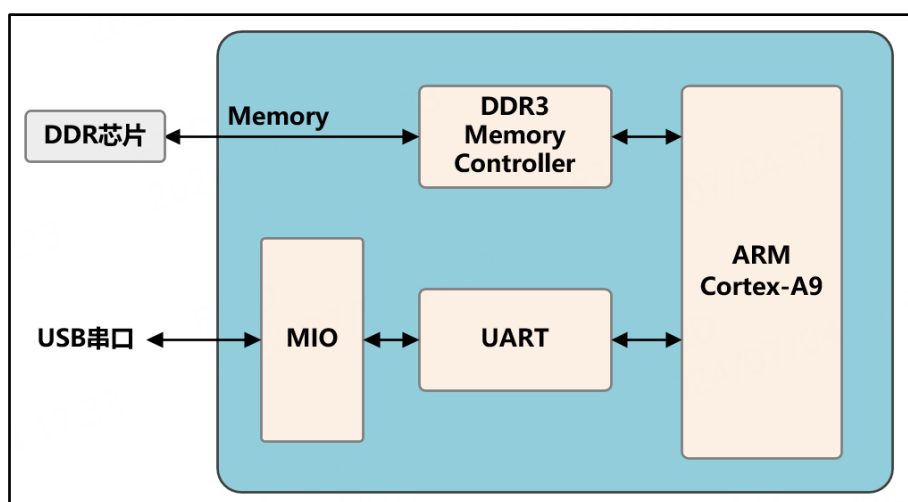


Figure 1-5 MIO driver UART block diagram

2. Hardware design

Below we will introduce the overall steps of establishing a vivado project on the PS side step by step.

Step1: Create a new vivado project

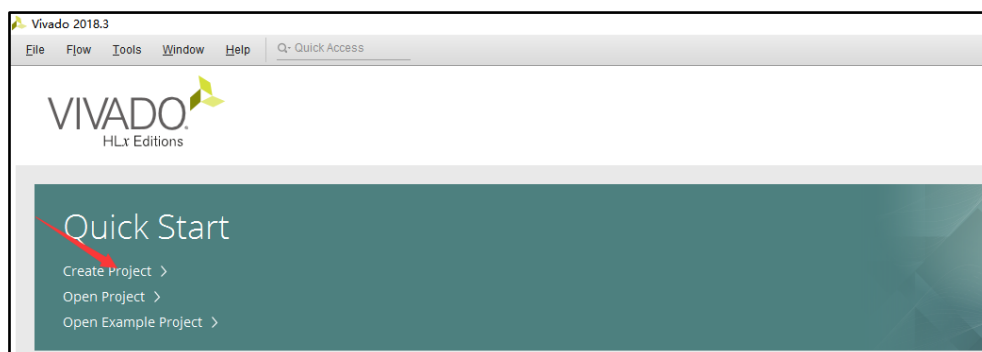


Figure 1-6 New project (1)

The following interface appears, click Next:

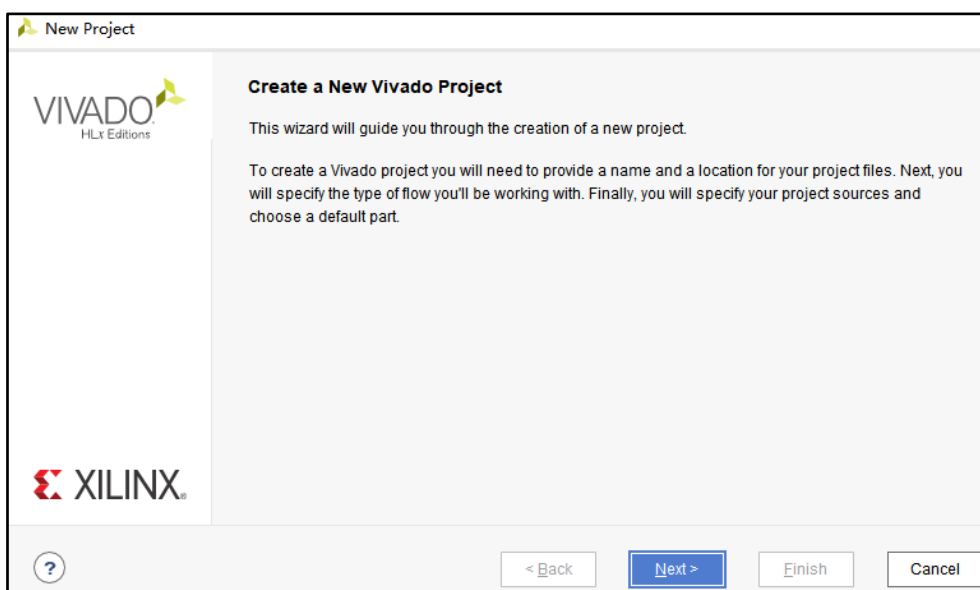


Figure 1-7 New construction (2)

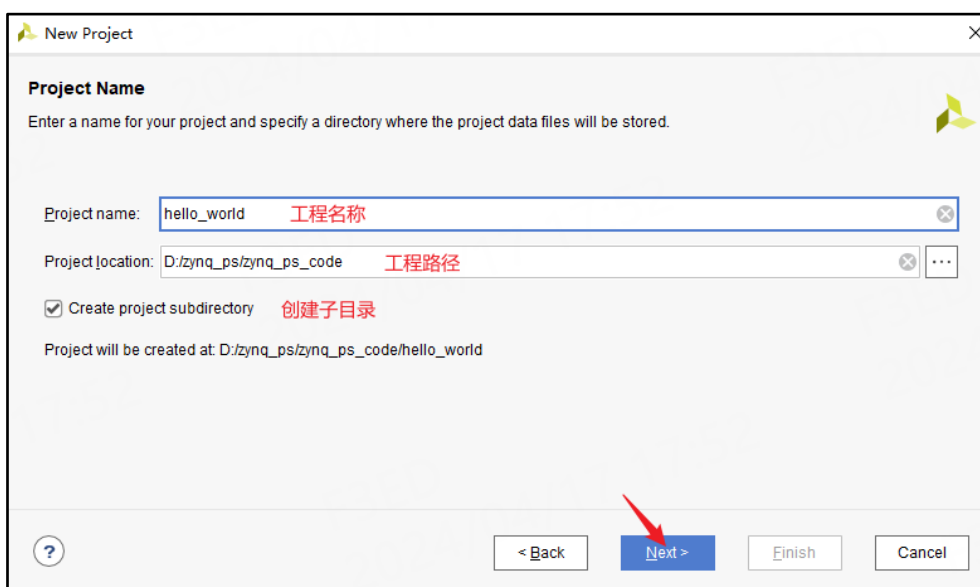


Figure 1-8 New construction (3)

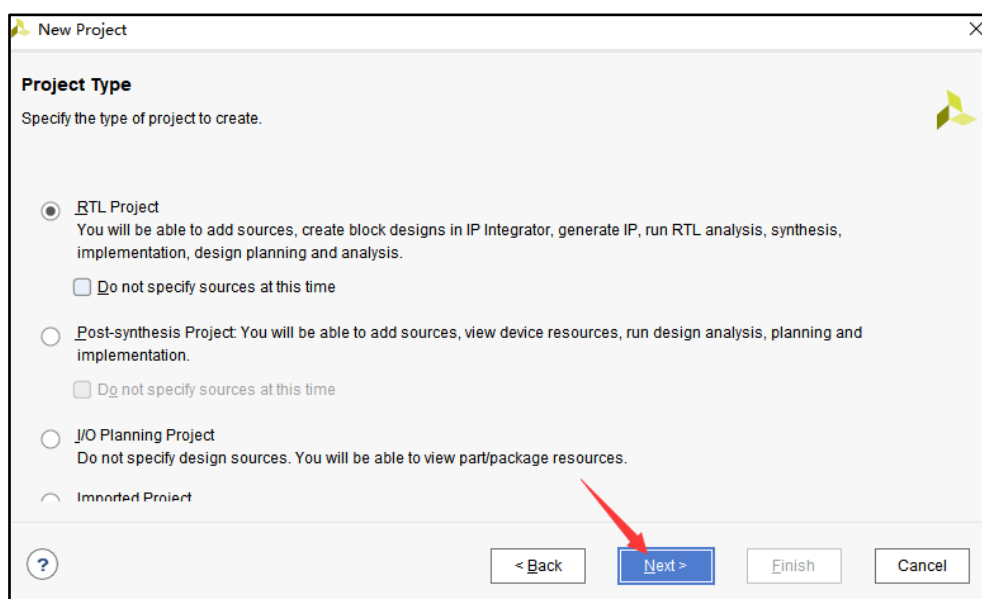


Figure 1-9 New construction (3)

Then keep the default and click NEXT until the following interface:

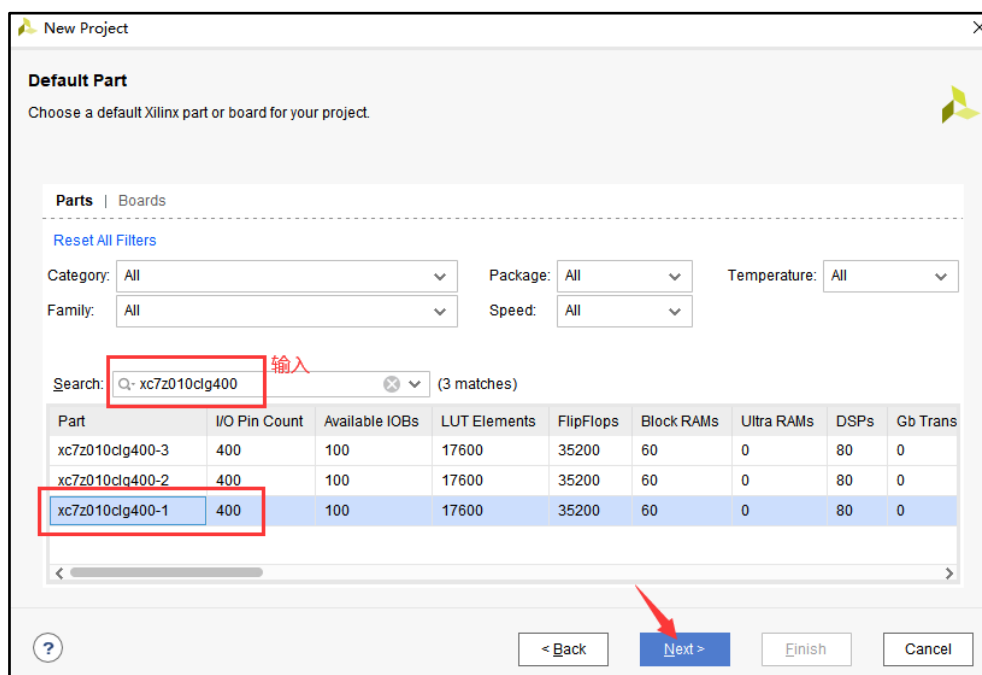


Figure 1-10 New construction (5)

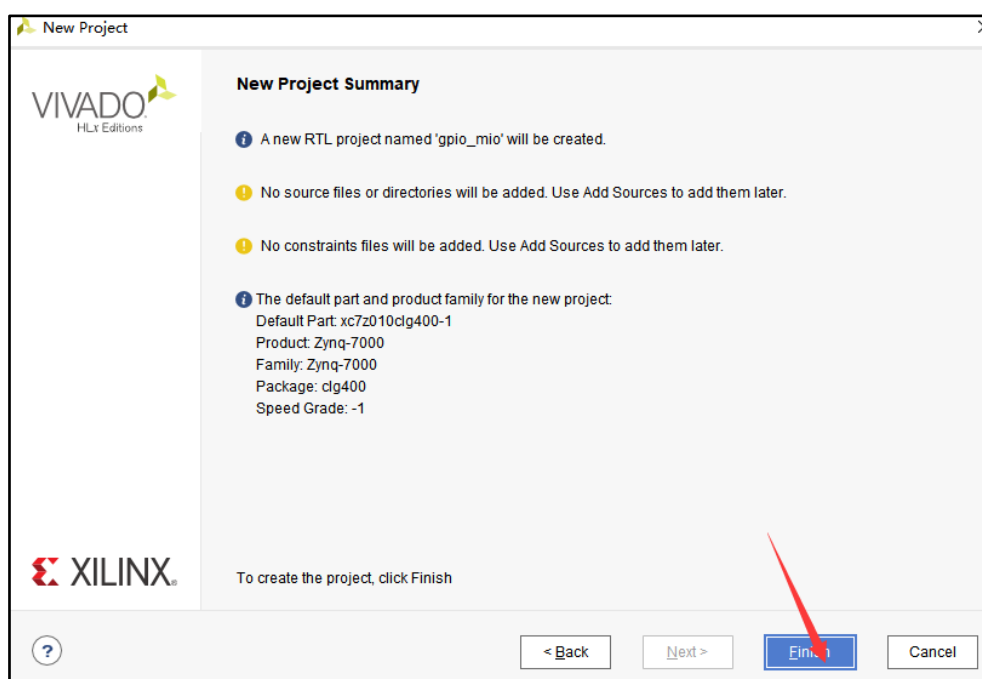


Figure 1-11 New construction (5)

At this point, we have completed the creation of the project.

Step2: Use IP Integrator to create a Processing System

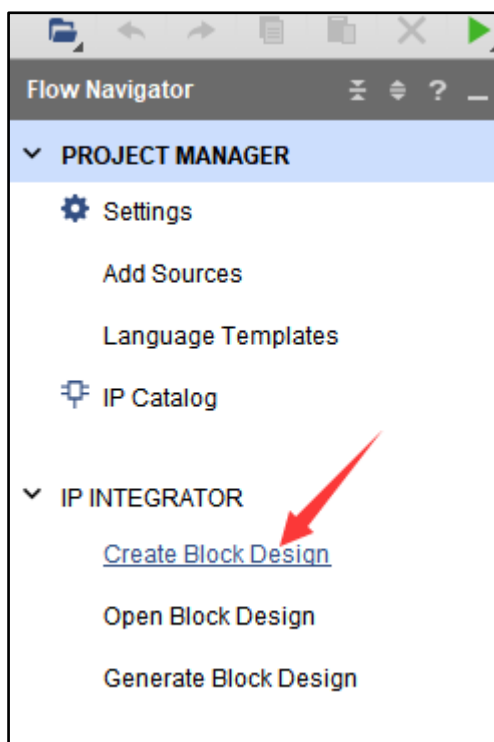


Figure 1-12 Creating a processing system (1)

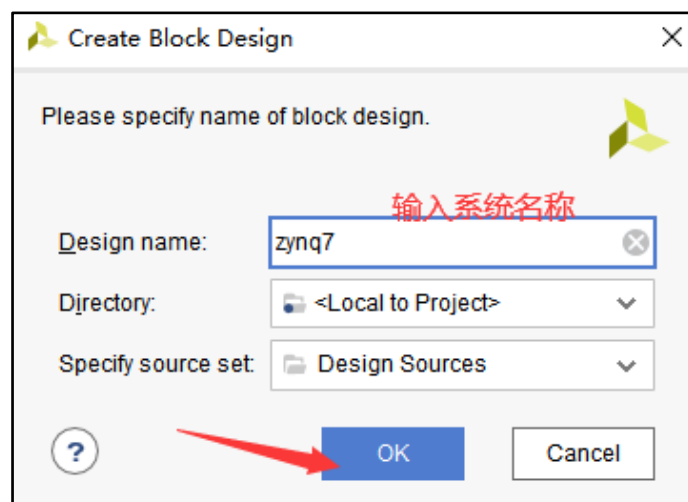


Figure 1-13 Creating a processing system (2)

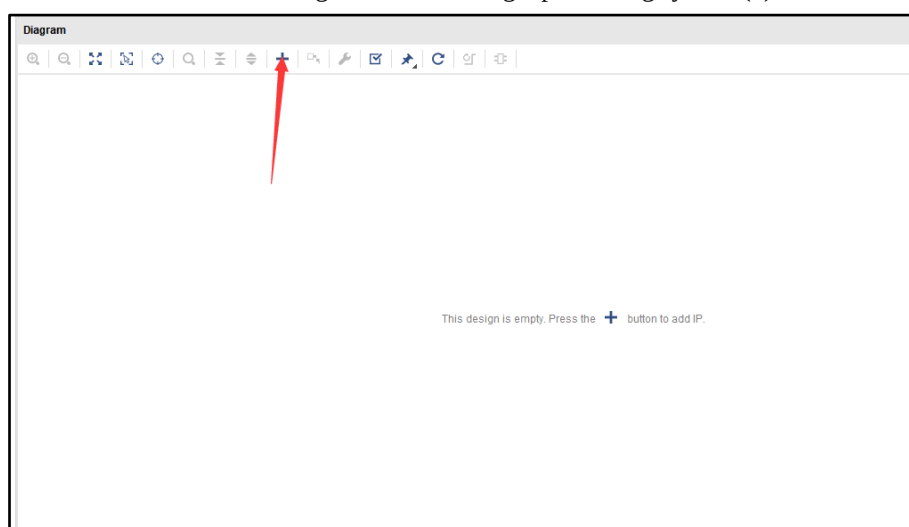


Figure 1-14 Add IP core

After opening the IP directory, type "zynq" in the search bar (or use ctrl+i), then find and double-click "ZYNQ7 Processing System" to add the ZYNQ7 processing system IP to the design.

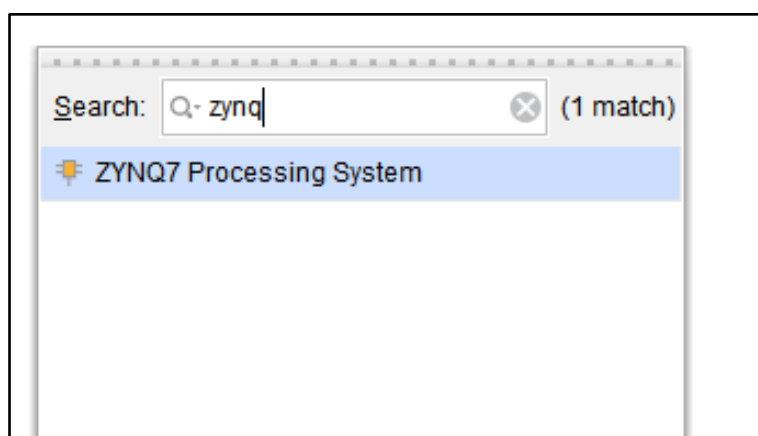


Figure 1-15 Add zynq processing system IP core

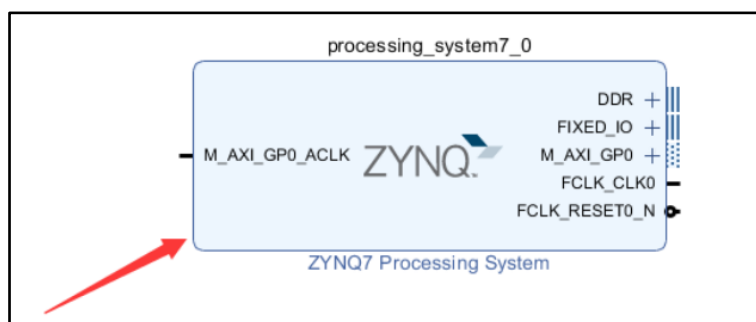


Figure 1-16 Double-click to enter configuration

Double-click the added ZYNQ7 Processing System module to enter the configuration interface of the ZYNQ7 processing system. The left side of the interface is the page navigation panel, and the right side is the configuration information panel. As shown below:

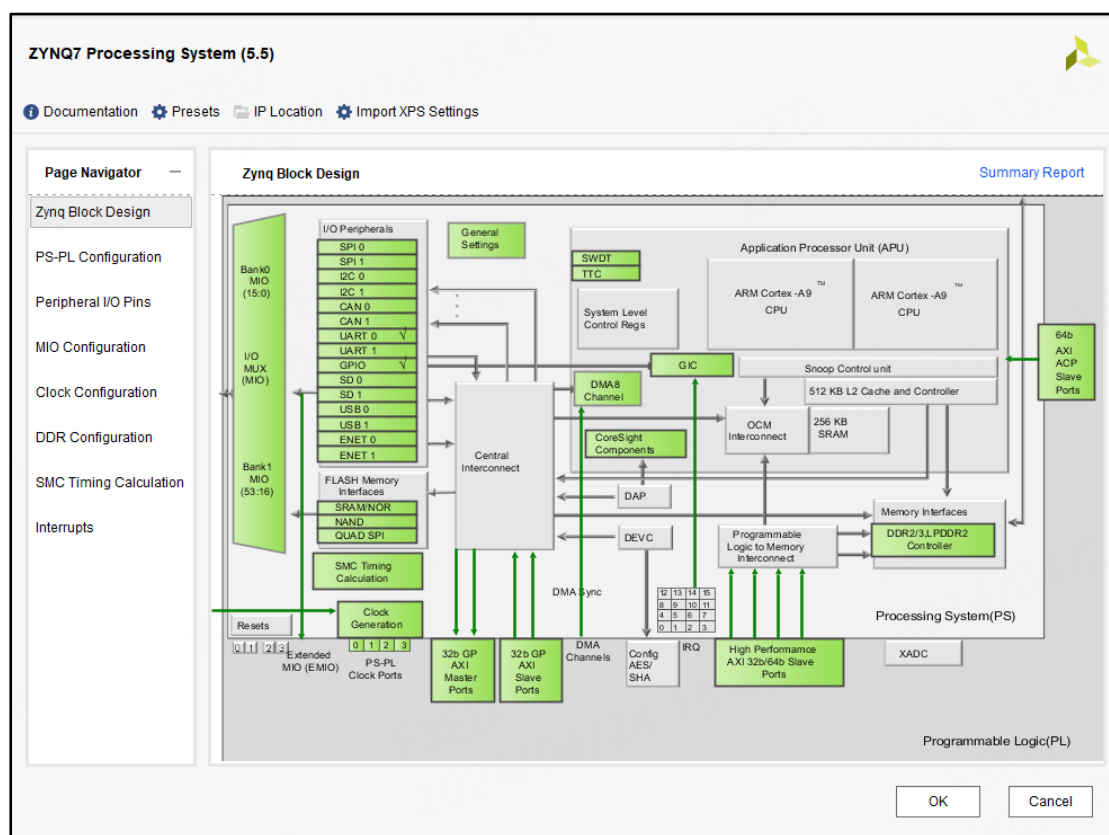


Figure 1-17 Double-click to enter configuration

The introduction of each interface is as follows:

The Zynq Block Design page displays various configurable blocks on the PS side. The gray parts are fixed and the green parts are configurable. Click a configurable block to enter the corresponding configuration page.

The PS-PL Configuration page is used to configure the PS-PL interface, including AXI, DMA controller and ACP bus interface.

The Peripheral IO Pins page is used to configure the corresponding MIO/EMIO configurations for different I/O pins.

The MIO Configuration page is used to configure the corresponding MIO/EMIO pin configurations for different I/O pins.

The Clock Configuration page is used to configure the peripheral clock, logical unit clock, DDR and CPU clock of Processing System 7.

The DDRConfiguration page is used to configure user DDR controller information.

The SMC Timing Calculation page is used to perform SMC timing calculations.

The Interrupts page is used to configure PS-PL interrupt ports.

According to the overall system block diagram, we need to configure UART and DDR.

1. Configure PS UART. Click the Peripheral I/O Pins page, and the following IO pin configuration interface appears:

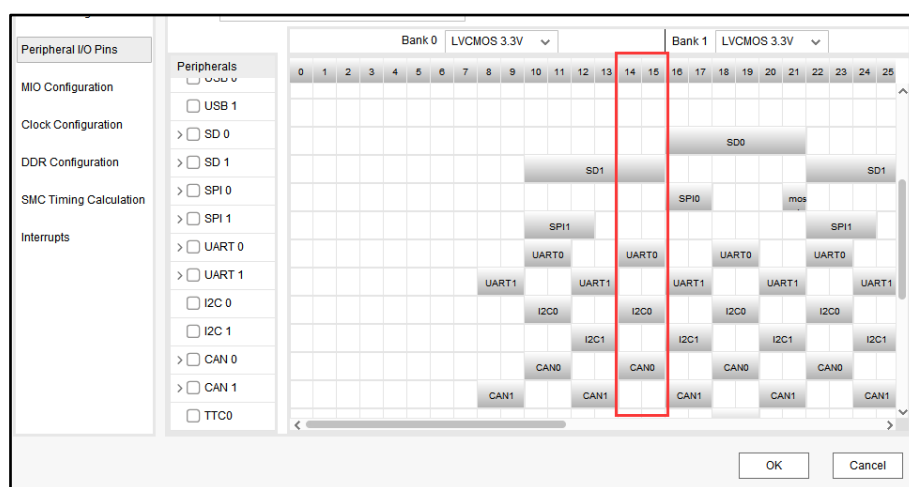


Figure 1-18 MIO14, MIO15 multiplexing

The connection between PS and external devices is mainly realized through multiplexed input/output (MIO). The 54 MIO pins of PS can be used to connect different peripheral interfaces, such as MIO14 and MIO15 in Figure 1-18, which can be configured as a UART0 pin interface or an I2C0 or CAN0 pin interface. The final selected configuration needs to correspond to the schematic diagram of the Haoyue development board.

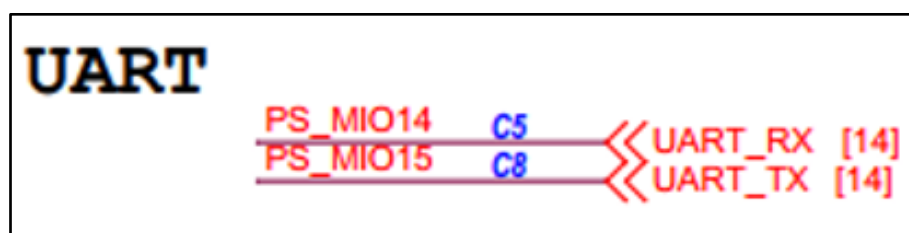


Figure 1-19 UART pin schematic diagram

According to the schematic diagram, what we need to configure is MIO14 and 15. Click on the Peripheral IO Pins

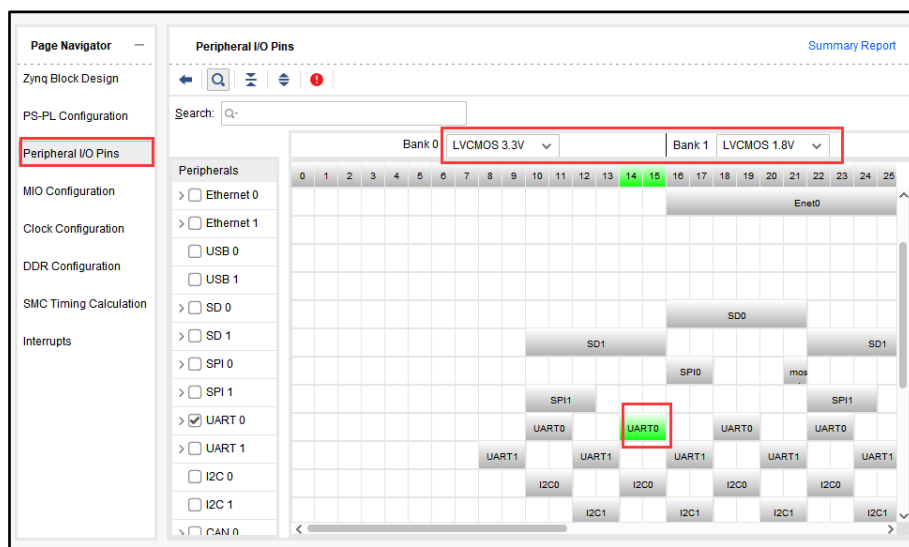


Figure 1-20 Configure UART serial port

It can also be configured on the MIO Configuration page:

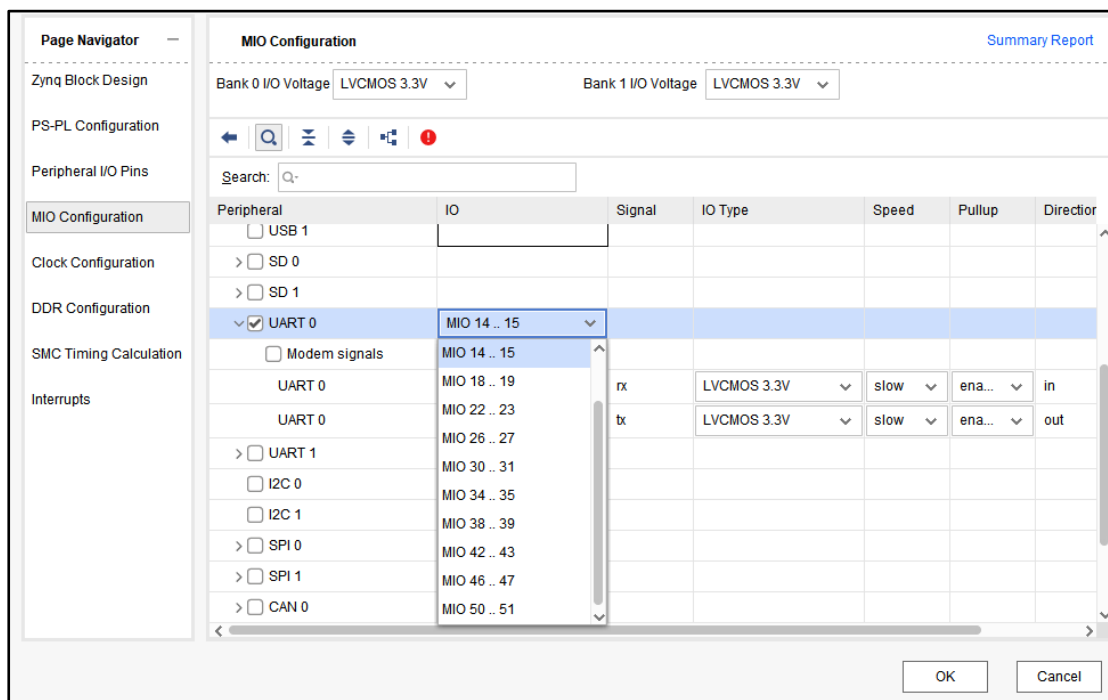


Figure 1-21 Configure UART serial port

Click on the PS-PL Configuration page on the left, where we can set the baud rate of UART0 serial communication. Under the General directory, you can see the baud rate of UART0 is 115200. Other baud rates can be selected through the drop-down button, generally keep the default settings.

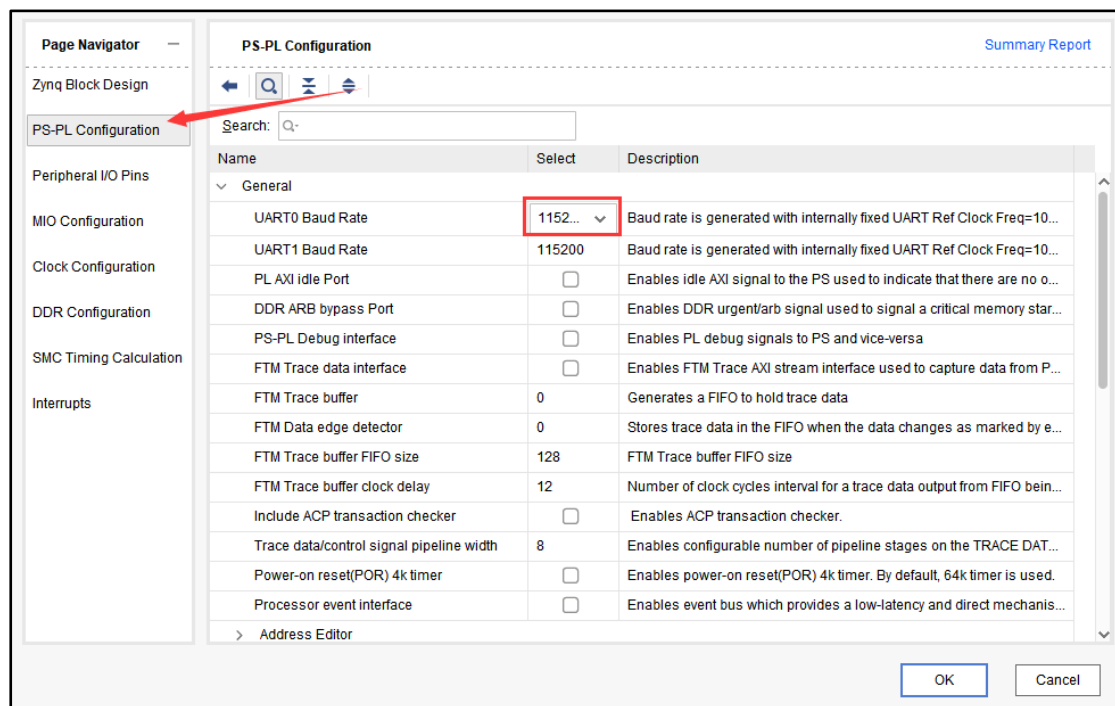


Figure 1-22 Configure UART baud rate

2. Configure DDR3 registers

Click the DDR Configuration page on the left, select the DDR device in the "Memory Part" column under DDR Controller Configuration on the right, and select MT41K256M16 RE-125 as the core board of XC7Z010. It should be noted that the model we choose here is not the DDR3 model on the Haoyue development board, but a model with similar parameters, or a compatible model. Other configuration options can be left as default.

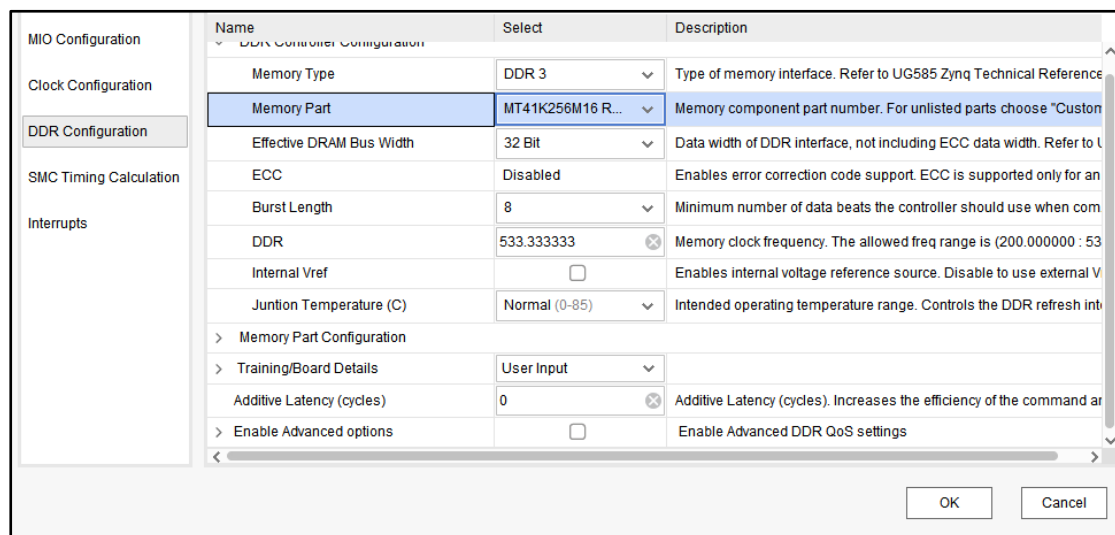


Figure 1-23 DDR3 configuration interface

3. Configure the clock of the PS.

Click the Clock Configuration page on the left. This interface is mainly used to configure the clock frequency in ZYNQ PS. For example, the default input clock is 33.33333Mhz, which is the same as the PS input clock frequency of our Haoyue core board. for

For the CPU clock, DDR clock and other peripheral clocks, we can just keep the default settings. As shown below:

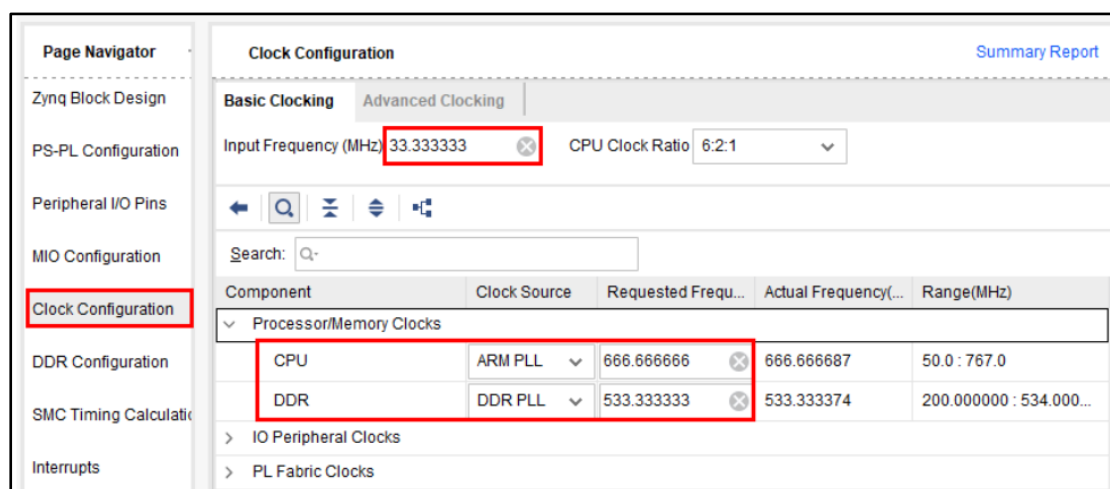


Figure 1-24 Configuring PS clock

Because this experiment is to build a minimal embedded system of ZYNQ, you only need to use the PS terminal in ZYNQ. Therefore, we removed the interface signals in PS that interact with the PL side. Of course, it doesn't matter if you don't remove it. Also on the Clock Configuration page, expand PL Fabric Clocks and uncheck FCLK_CLK0, as shown in the figure below:

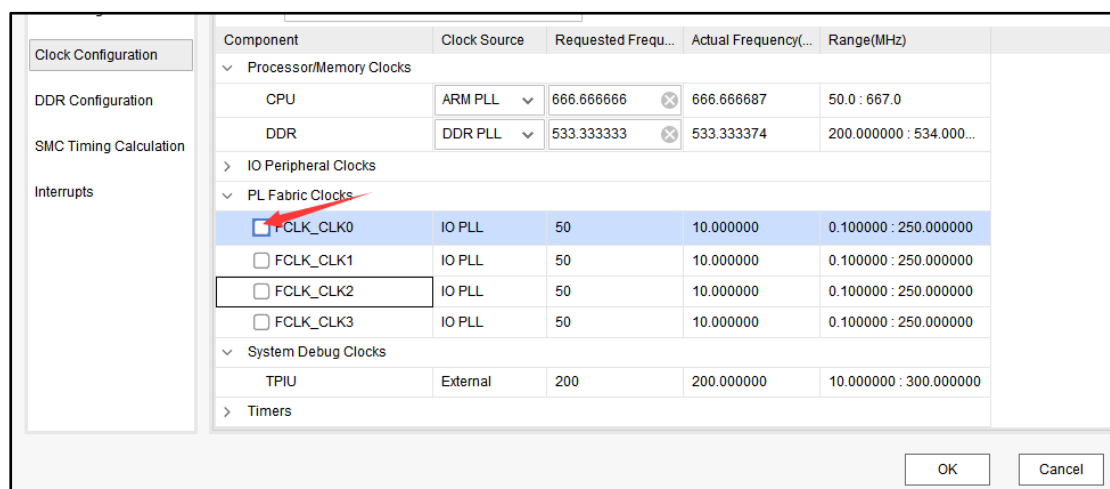


Figure 1-25 Uncheck FCLK_CLK0

Click the PS-PL Configuration page on the left, then expand Enable Clock Resets under General on the right, and uncheck FCLK_RESET0_N. In addition, expand the GP Master AXI Interface under AXI Non Secure Enablement in the current interface and uncheck the MAXI GP0 interface. As shown below:

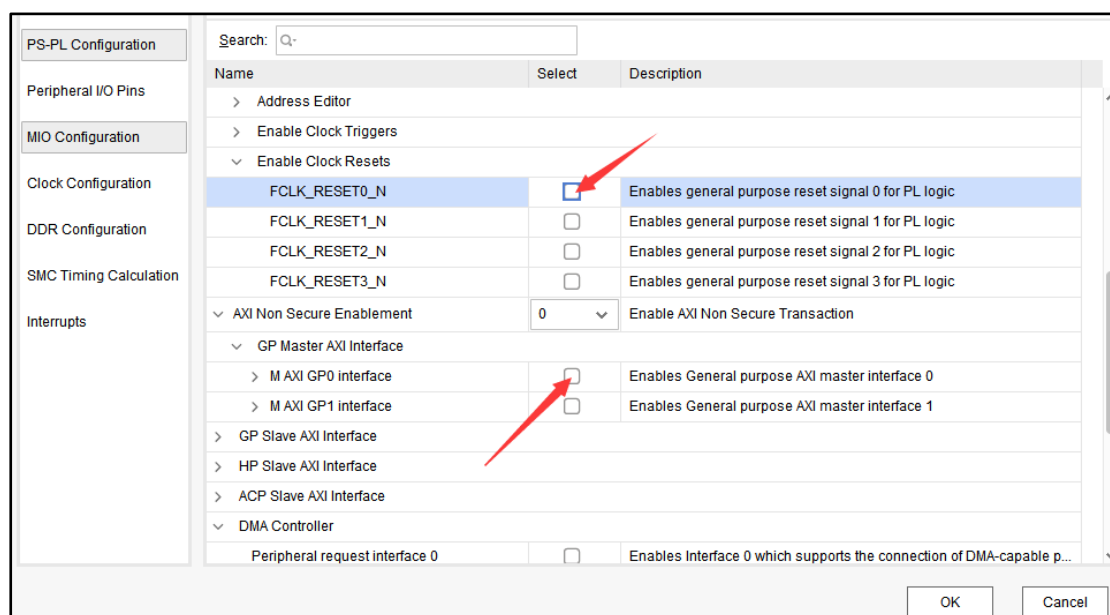


Figure 1-26 Uncheck FCLK_RESET0_N and M AXI GP0 interface

After configuring the ZYNQ7 Processing System IP module, click "Run Block Automation" at the location indicated by the arrow in Figure 1-27, and the dialog box shown in Figure 1-28 will pop up.

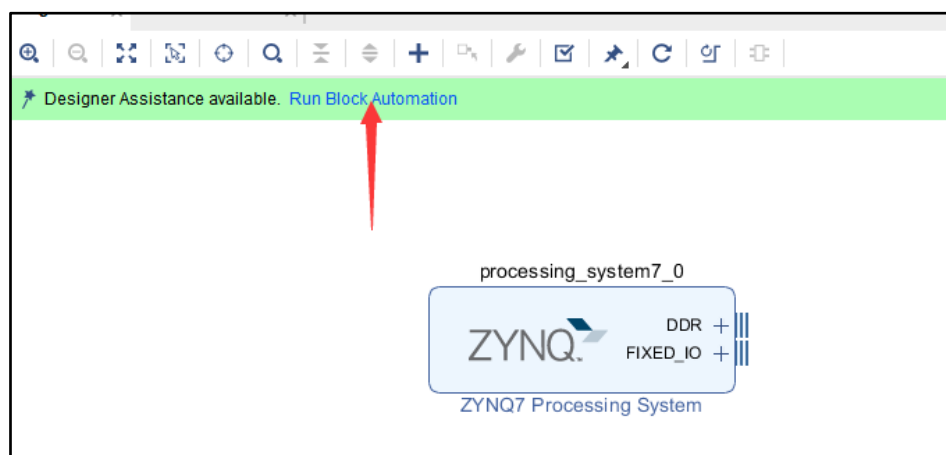


Figure 1-27 Click Run Block Automation

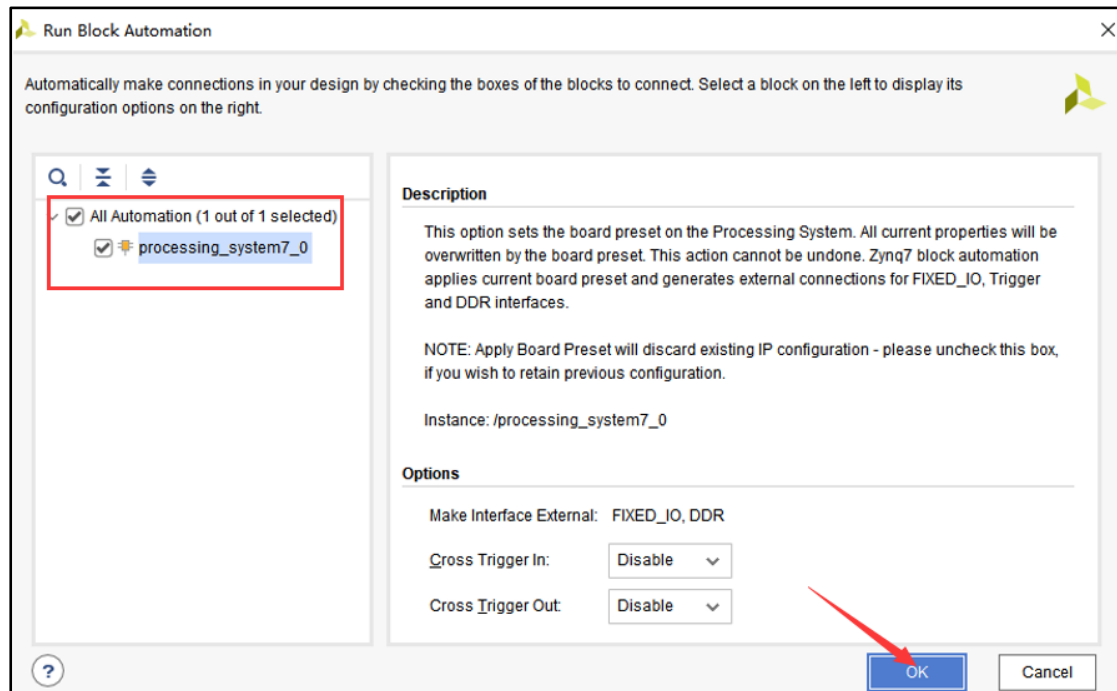


Figure 1-28 Run Block Automation

At this time, the ZYNQ7 PS module leads to two sets of external interfaces, namely DDR and FIXED_IO. The derived interfaces will be assigned to specific pins of the ZYNQ device. You can also click the plus sign "+" at the ZYNQ7 PS module interface to expand these two sets of interfaces and observe the signals in them. Follow the arrow operations shown in Figure 1-29.

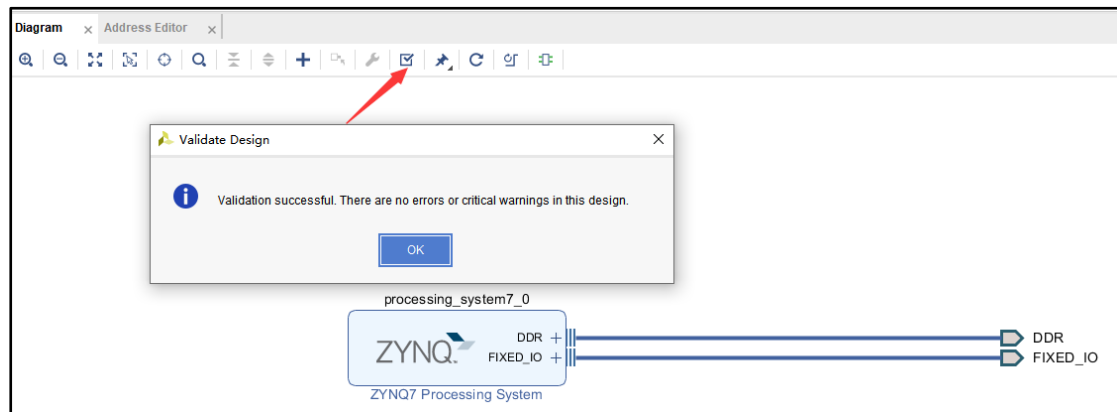


Figure 1-29 Verify design

After the verification is completed, a dialog box will pop up indicating that there are no error or critical warnings. Click "OK". If an error message or warning appears, you need to check your design.

Step3: Generate top-level HDL module

There is no need to add other IPs in this experiment. Just press the shortcut key Ctrl+S to save the current design. Or save it as shown in Figure 1-30.

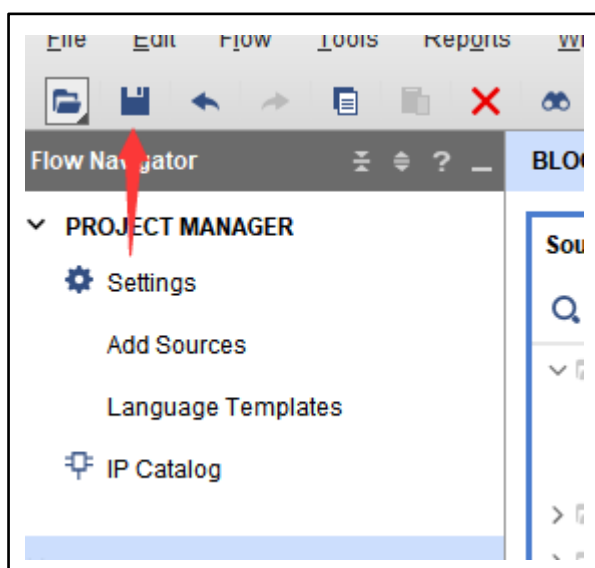


Figure 1-30 Save design

Then proceed as shown in Figure 1-31:

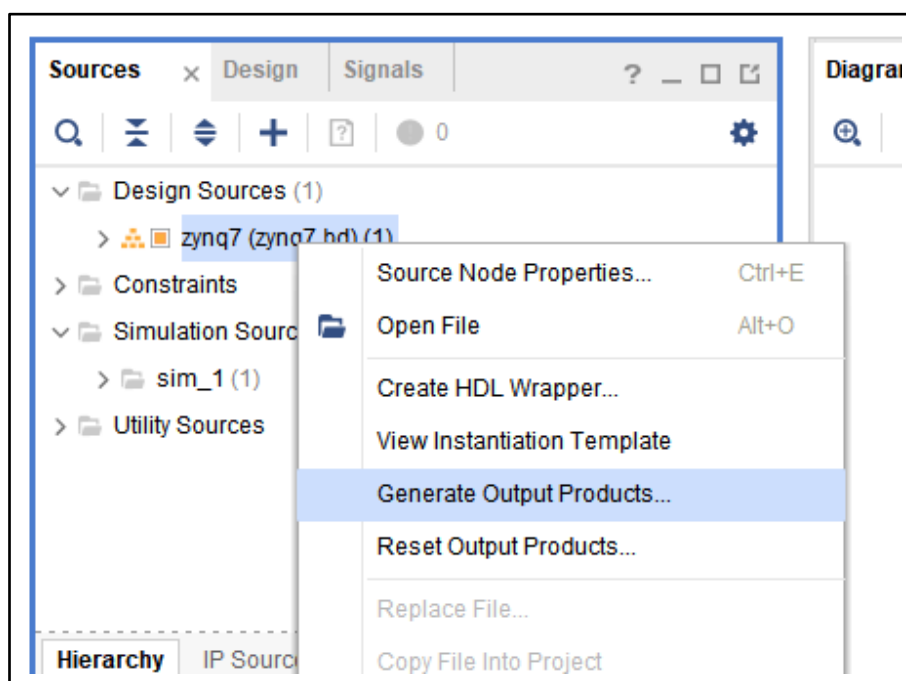


Figure 1-31 Select Generate Output Products

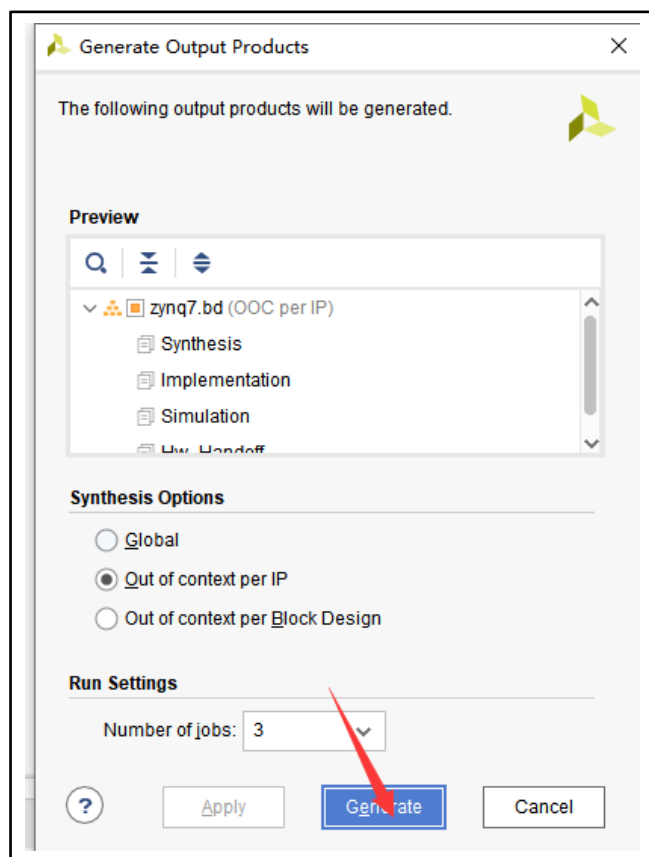


Figure 1-32 Generate Output Products

Then right-click `zynq7.bd` again on the "Hierarchy" tab and select "Create HDL Wrapper".

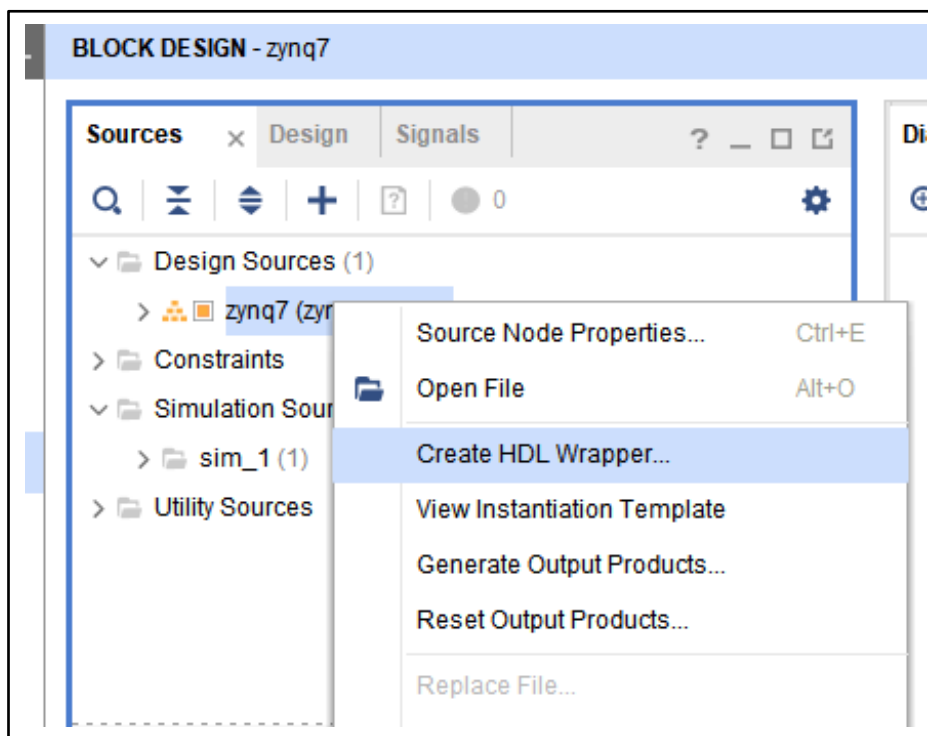


Figure 1-33 Select Create HDL Wrapper

Confirm to check "Let Vivado manage wrapper and auto-update" in the pop-up dialog box, so that we do not need to regenerate the top-level module after modifying BlockDesign, and the Vivado tool will automatically update the file. Then click "OK" to create the top-level module.

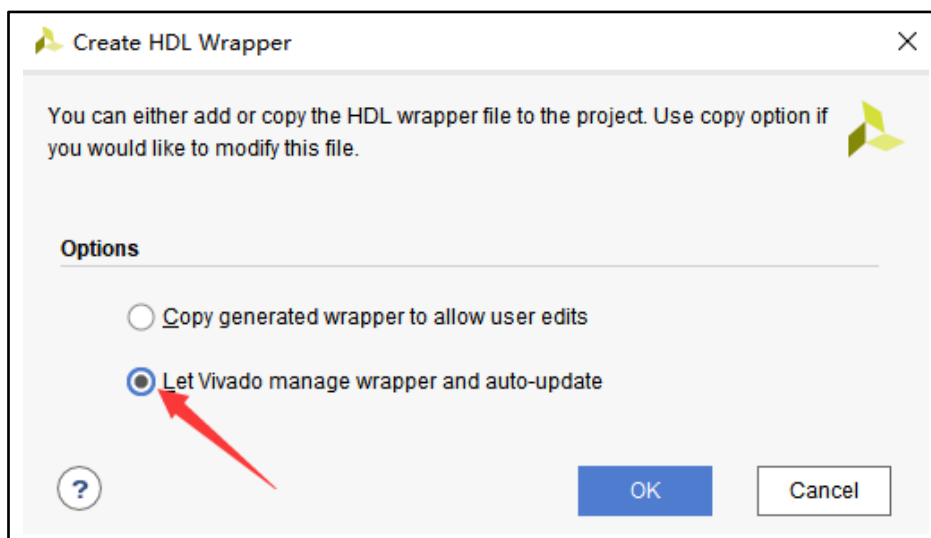


Figure 1-34 Create top-level module

After creation, the Design Sources structure is as shown below:

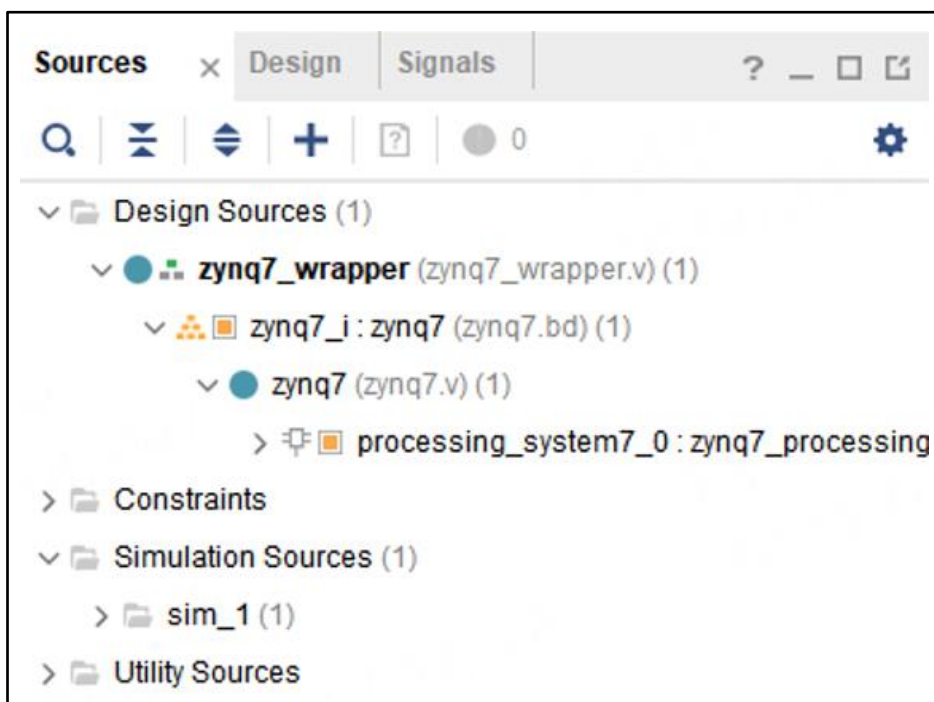


Figure 1-35 Design Sources structure

Step4: Generate Bitstream file

If PL resources are used in the design, pin constraints need to be added and the design needs to be synthesized, implemented, and a Bitstream file generated. Since the PL part is not used in this experiment, there is no need to generate Bitstream files, just export the hardware to the SDK.

Step5: Export SDK

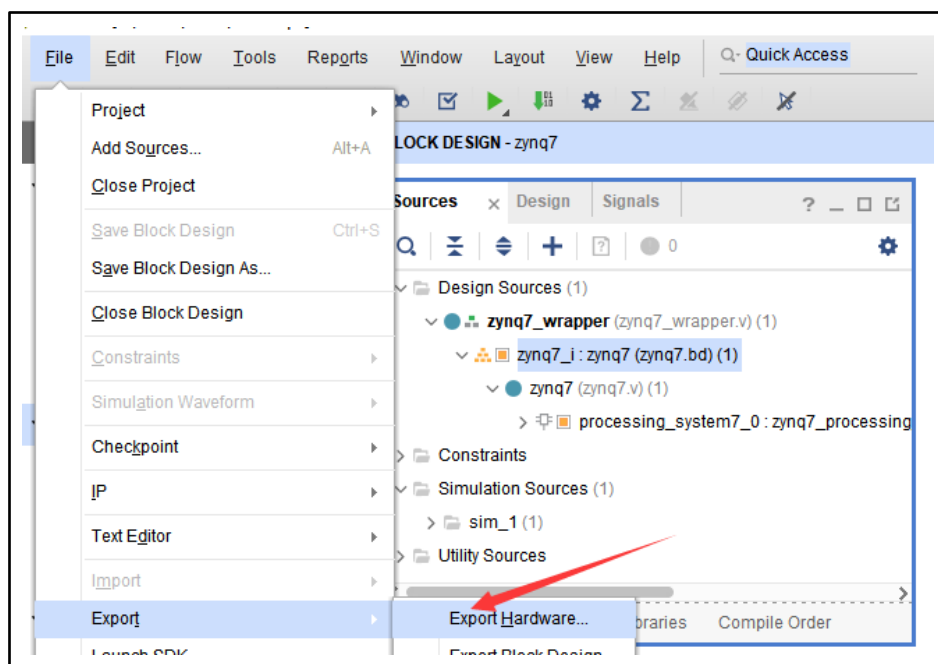


Figure 1-36 Select Export Hardware

In the pop-up dialog box, since no bitstream file is generated, there is no need to check "Include bitstream" and click the "OK" button directly.

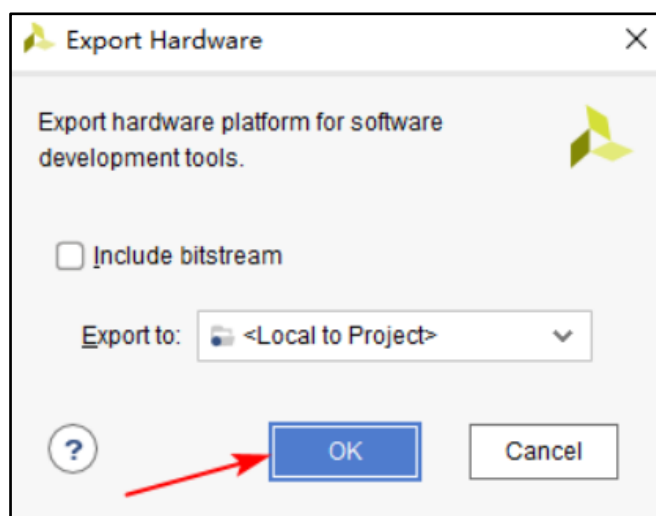


Figure 1-37 Export Hardware

"Export to <Local to Project>" is selected here, and the Vivado tool will create a new folder in the current project directory named "hello_world.sdk", which is our workspace for subsequent software development. During the Export Hardware process, the tool will export the hardware to the workspace in the form of a ZIP compressed file with the file name "zynq7_wrapper.hdf". This file contains the configuration information of the hardware platform we built earlier, and its suffix name .hdf means "Hardware Definition File", which is the hardware definition file.

Select File > Launch SDK in the menu bar to start the SDK development environment.

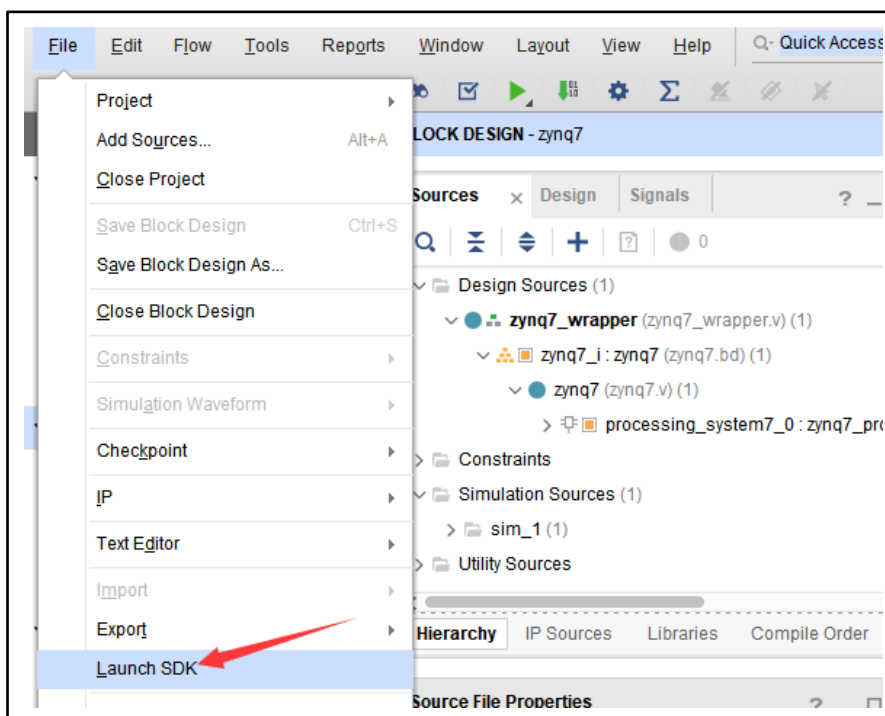


Figure 1-38 Select export SDK

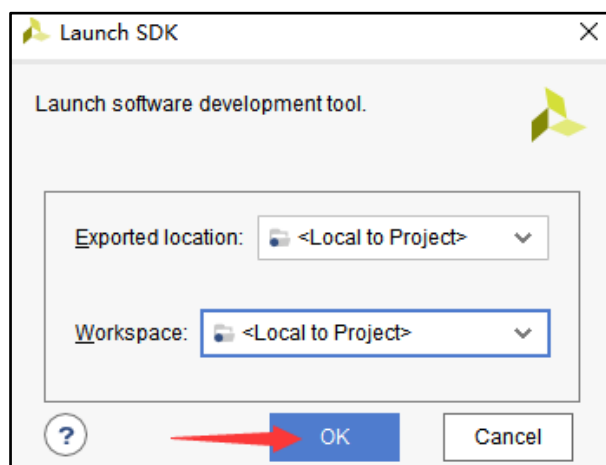


Figure 1-39 Export SDK

At this point, we have completed the hardware design part of the ZYNQ embedded system. Next, you need to carry out application development in the SDK software, which is the software design part.

3. Software design

In the hardware design part, we started the SDK development environment.

Step1: Create a new application project

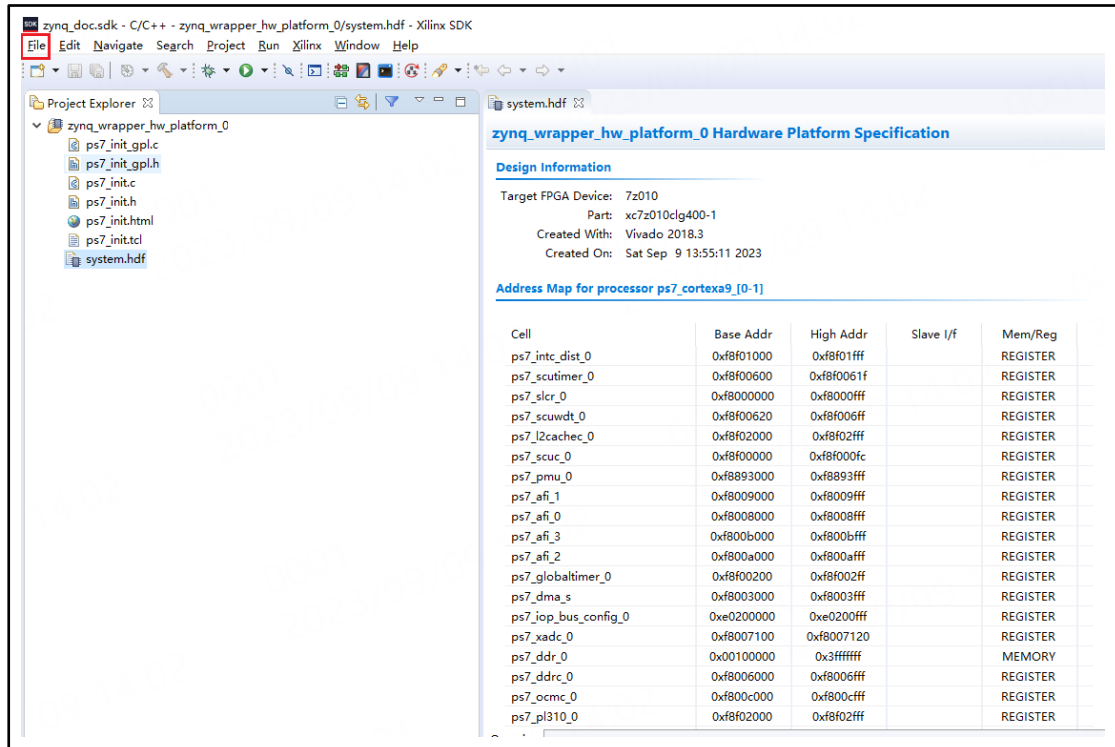


Figure 1-40 Project interface after exporting SDK

After clicking Launch SDK, wait for loading. When the above page appears, click File in the upper left corner.

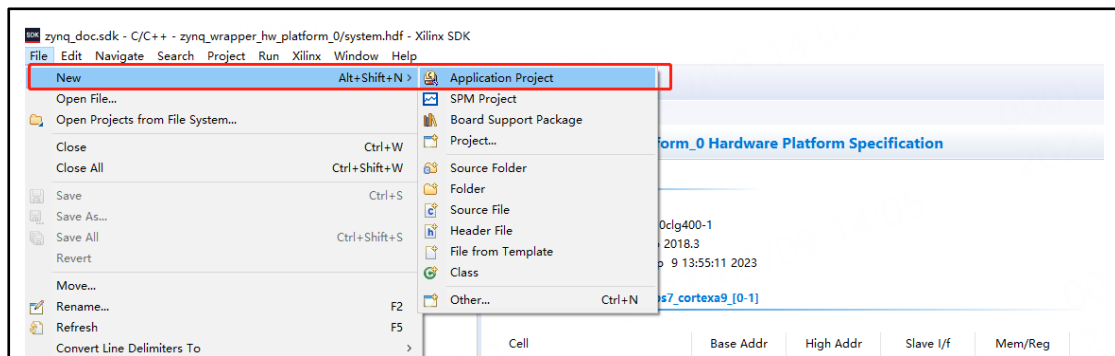


Figure 1-41 New project interface

After that, after the tab that pops up, select New->Application Project to create a new application project.

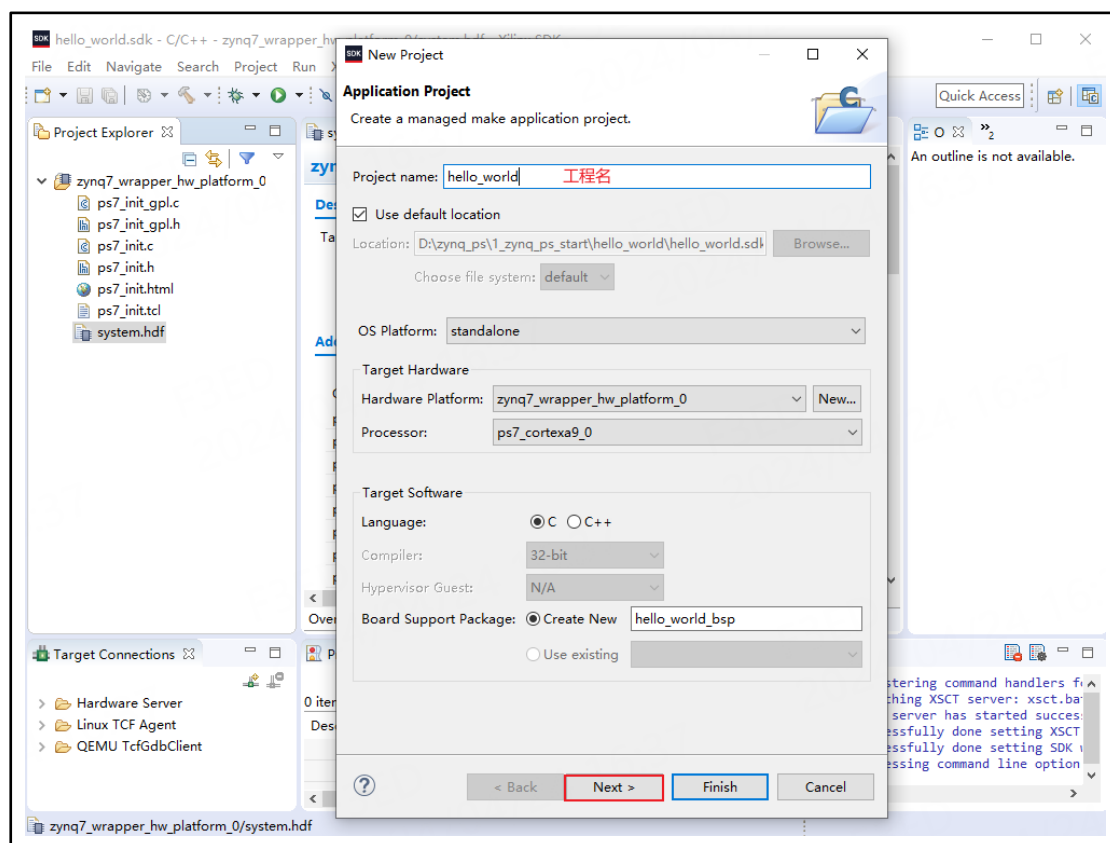


Figure 1-42 Enter project name

In the tab that pops up later, enter the name in Project name, keep the rest of the configuration as default, and then select Next, do not select Finish.

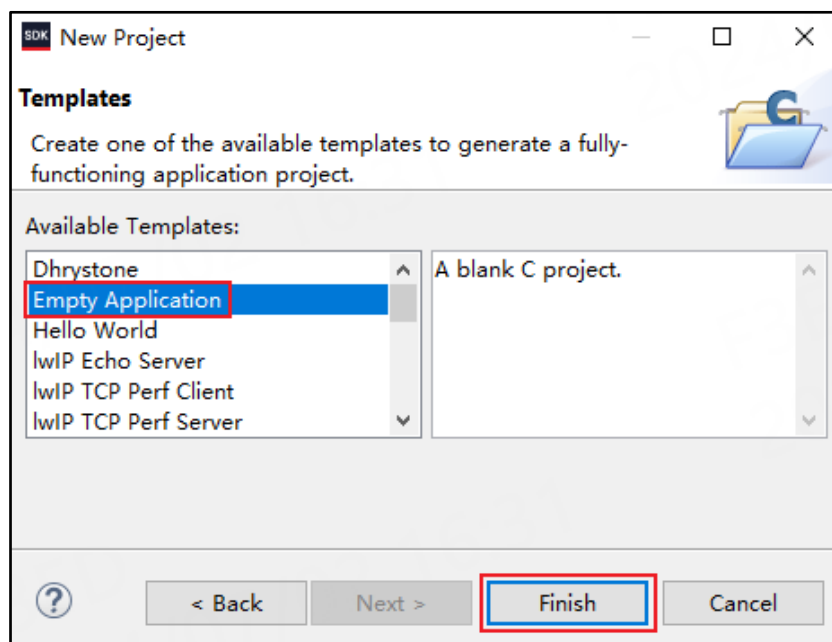


Figure 1-43 Template selection

After that, some official templates will pop up to choose from. The Hello World template will be selected by default, but this template has a lot of redundant things. For the sake of simplicity, we choose Empty Application to create a blank project.

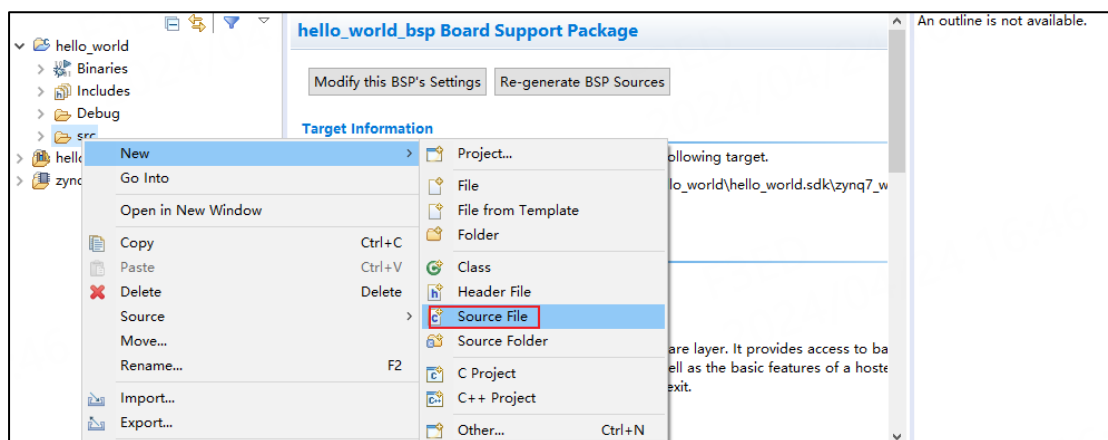


Figure 1-44 Create a new C file

Step2: Create a new source file

Expand the zynq_test folder, then right-click the src folder and select New->Source File to create a new C file.

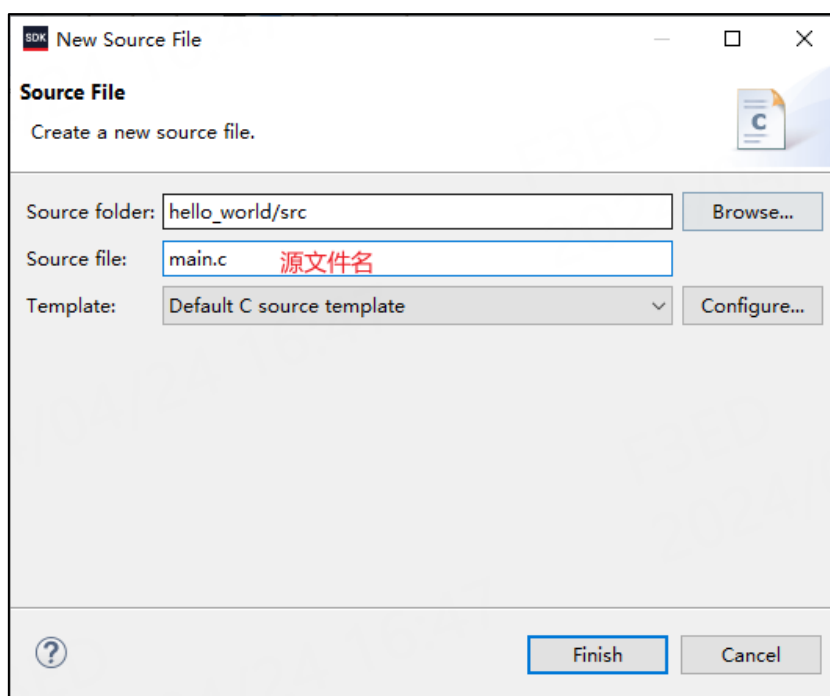


Figure 1-45 Enter file name

Enter the file name. Here I named it main.c. You can name it whatever you want, but it is best to standardize the naming. Next start writing code.

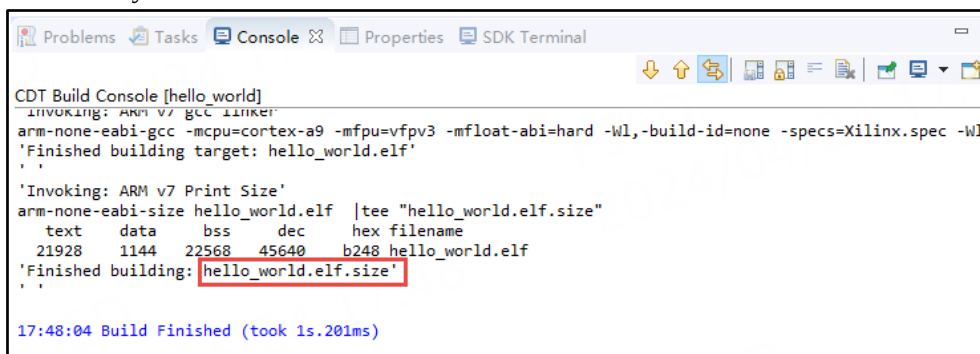
Step3: Write software code

Code List 1-1 Serial port printing reference code (main.c)

```
1 #include "stdio.h"
2 #include "xil_printf.h"
```

```
3 int main(void)
4 {
5     xil_printf("Hello World!\n");
6     return 0;
7 }
```

We introduce the header file `xil_printf.h`, which is officially provided by xilinx, so that we can subsequently use the function of printing strings, that is, the `xil_printf()` function. After we modify the code and save it. The SDK will automatically compile the project, and we do not need to compile it manually.



```
CDT Build Console [hello_world]
Invoking: ARM v7 GCC Linker
arm-none-eabi-gcc -mcpu=cortex-a9 -mfpv=vfpv3 -mfloat-abi=hard -Wl,-build-id=none -specs=Xilinx.spec -Wl
'Finished building target: hello_world.elf'

'Invoking: ARM v7 Print Size'
arm-none-eabi-size hello_world.elf |tee "hello_world.elf.size"
text      data      bss      dec      hex filename
21928     1144     22568    45640    b2488 hello_world.elf
'Finished building: hello_world.elf.size'

17:48:04 Build Finished (took 1s.201ms)
```

Figure 1-46 Compilation completed

At this point, the software code writing is completed, and then it is verified on the board.

1.3.4 Board verification

First, we connect the downloader to the JTAG interface on the Haomo board, and connect the other end of the downloader to the computer. Then use a USB cable to connect the USB_UART interface on the left side of the development board to the computer for serial communication. Next, turn both the boot mode switch `BOOT_CFG` on the Haomo board to the upper position (both are set to 0), that is, set to JTAG mode. Finally, connect the power supply of the development board and turn on the power switch. As shown below:

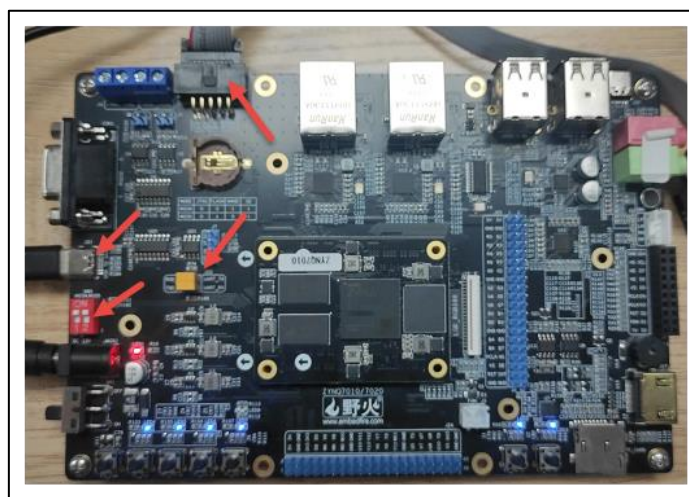


Figure 1-47 Download connection diagram

Under the SDK software, you can see an SDK Terminal. We click the plus sign to open the serial port. If there is no such tab, you can select `Window > Show View > Other` in the menu bar and search to add SDK Terminal in the Show View window.

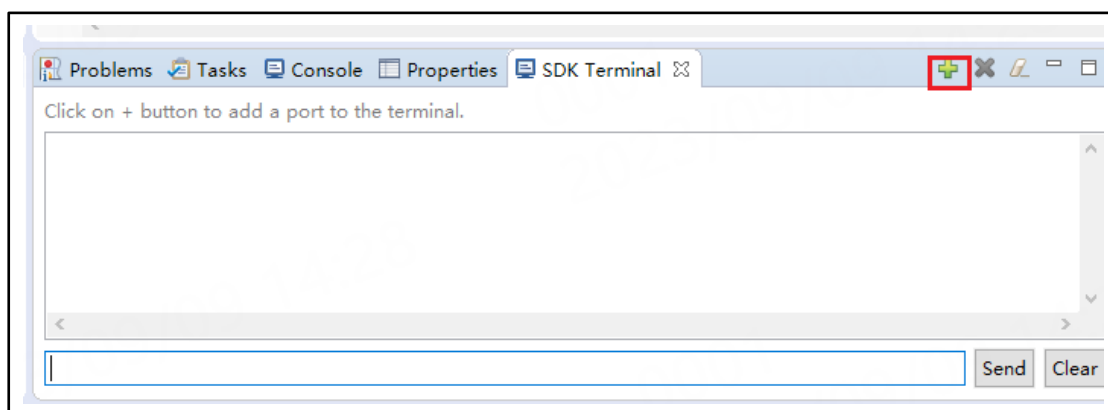


Figure 1-48 Add SDK Terminal

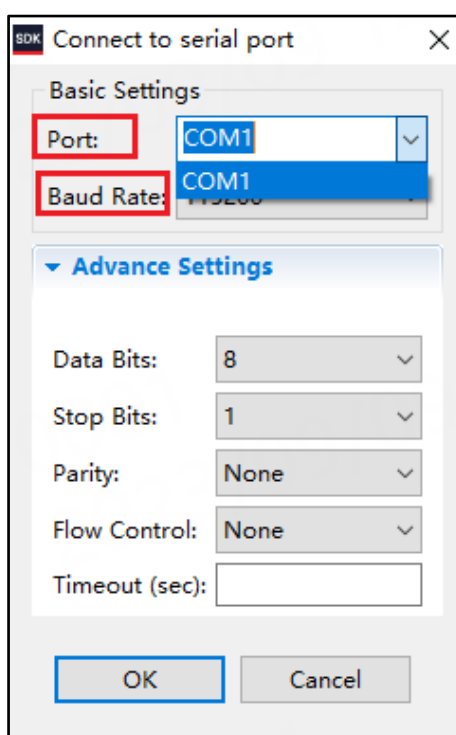


Figure 1-49 Select the serial port connected to the development board

In the tab that pops up: Port: is the device port, we can check the serial port number of our board through the device manager to select; Baud Rate: is the baud rate, the default is 115200; other configurations remain default, such as 8-bit data, no parity, 1-bit stop bit, etc.

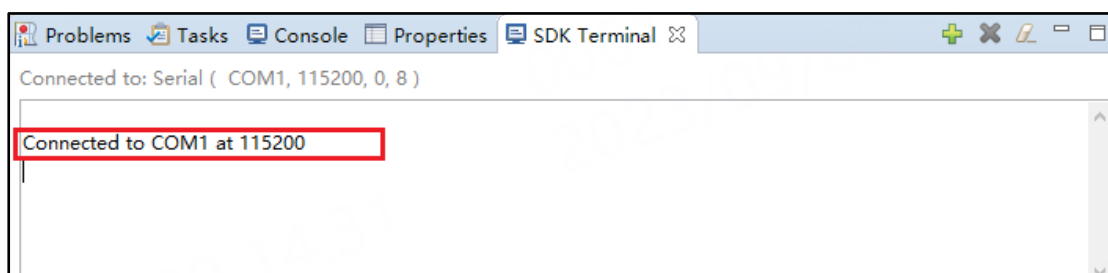


Figure 1-50 Diagram of successful connection

See The window displays Connector to COM1 at 115200, which means the connection is

Then right-click the application project and select Run as > Launch on Hardware (system Debugger) in the pop-up menu bar, as shown in the following figure:

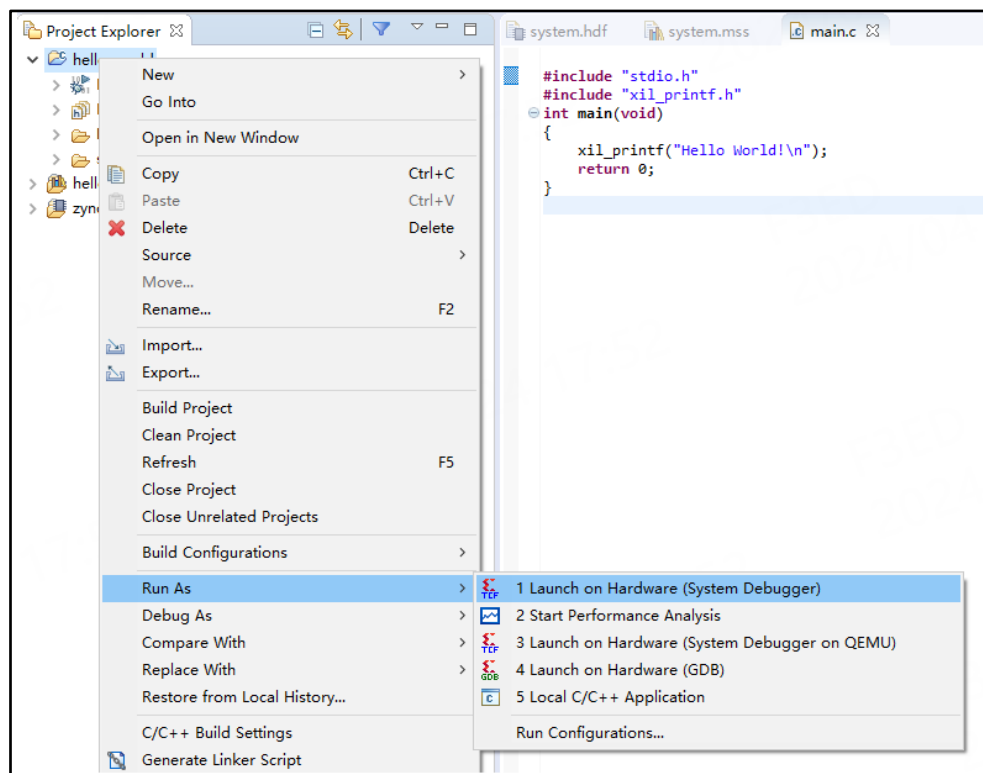


Figure 1-51 Download program

If the "Hello world!" string is successfully printed in the SDK Terminal window, it means that this experiment was successfully downloaded and verified on the development board.

1.4 End of chapter summary

In this chapter, we introduce how to build a ZYNQ minimal embedded hardware system through vivado, without involving the PL side. Finally, we export the hardware to the SDK for software development, and successfully print out the classic "Hello world!".

1.5 Development training

After studying this chapter, I believe everyone is familiar with the process of creating hardware projects and software projects. In the routines in this chapter, we use the xil_printf function provided by Xilinx instead of the familiar printf function in the C library, both of which can print strings. You can try to use printf to print and understand the differences between them.

Chapter 2 MIO Controlling LEDs

2.1 Chapter Introduction

ZYNQ PS contains a rich set of peripherals, such as CAN controller, UART controller, I2C controller, GPIO, etc., providing various industry standard interfaces for data communication with external devices. This chapter introduces the operation of MIO (multiplexed I/O) on the PS side. MIO is the basic peripheral IO. Through the VIVADO software setting, the software can export the signal through MIO, and can also connect the signal to the pin on the PL side through EMIO (extended multiplexed I/O). Among them, MIO is tri-state and can be set as output to control some simple peripherals and control the on and off of LED lights; it can also be set as input to observe the status of some simple peripherals, such as detecting whether a button is pressed.

In this chapter we will realize controlling the LED on the PS side through MIO, and achieve alternate flashing.

2.2 Theoretical study

2.2.1 Introduction

GPIO peripheral, full name general purpose I/O peripheral. It is a general-purpose input and output peripheral, and the status of the pin can be observed and controlled through software. GPIO signals can be routed to MIO or to pins on the PL side through the EMIO interface. If the IO is not enough, you can expand more IO through EMIO. ZYNQ supports up to 64 EMIO expansion. And each GPIO can be independently and dynamically programmed as input, output, or interrupt detection.

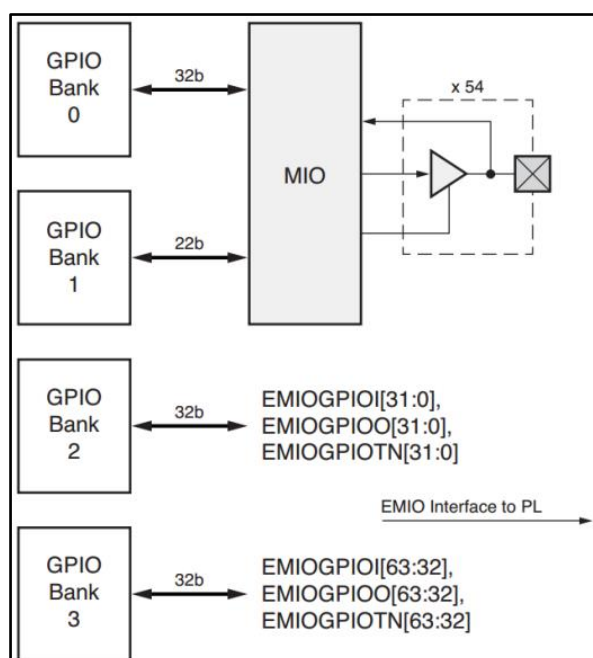


Figure 2-1 GPIO block diagram

As shown in Figure 2-1, the GPIO module is divided into 4 Banks, Bank0 and Bank1 are connected to MIO; and Bank2 and Bank3 are connected to EMIO.

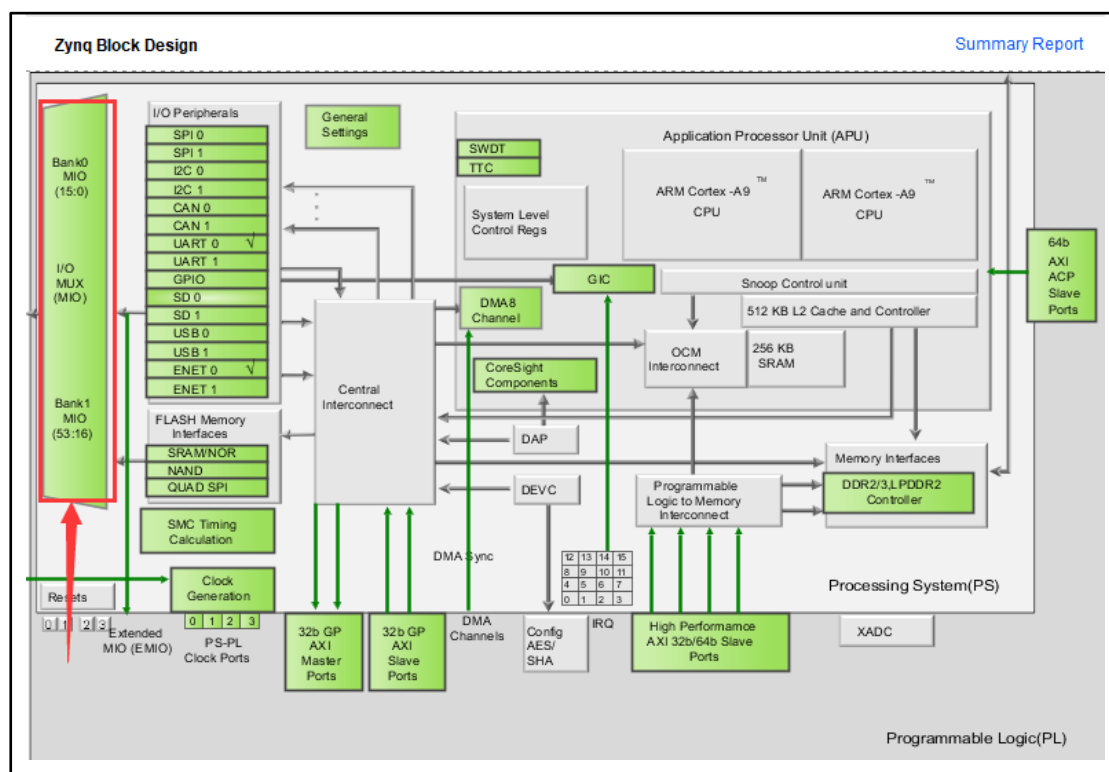
Bank0: Control MIO pins[31:0].

Bank1: Control MIO pins[53:32]. Bank
2: Control EMIO signals[31:0]. Bank3:
Controls EMIO signals[63:32].

Because the Zynq-7000 series chips have only 54 MIOs in total, only the lower 22 bits of Bank1 are valid.

The PS MIO I/O buffer is divided into two voltage domains. Within each domain, each MIO is programmed independently. MIO is divided into two Banks. Bank0 consists of 16 pins of MIO[0~15], and Bank 1 consists of 38 pins of MIO[16~53]. Our Haoyue development board has BANK500, BANK501, and BANK502 on the PS side. Among them, domestic BANK500 and BANK501 are used to connect to MIO, BANK500 corresponds to BANK0, BANK501 corresponds to BANK 1; and BANK502 is used exclusively to connect to DDR. Note: Bank0 and Bank1 here are for MIO, while the previously mentioned Bank0~Bank4 is for all GPIOs. Pay attention to the distinction and don't confuse them.

In addition, MIO7 and MIO8 can only be configured as outputs. The mode used to control the voltage during reset (as input) can only be used as output after reset. MIO7 MIO8 is configured as 0, and the voltage mode supports 2.5 and 3.3V. If configured as 1 only supports 1.8V. On the Hao yue development board, MIO7 is configured as 0, and the corresponding voltage of BANK500 is 3.3V; MIO8 is configured as 1, and the corresponding voltage of BANK501 is 1.8V.



pins, so when we design PS peripherals, we must allocate MIO reasonably. Once the MIO is selected, its pins are already fixed and do not require pin constraints like the pins on the PL side.

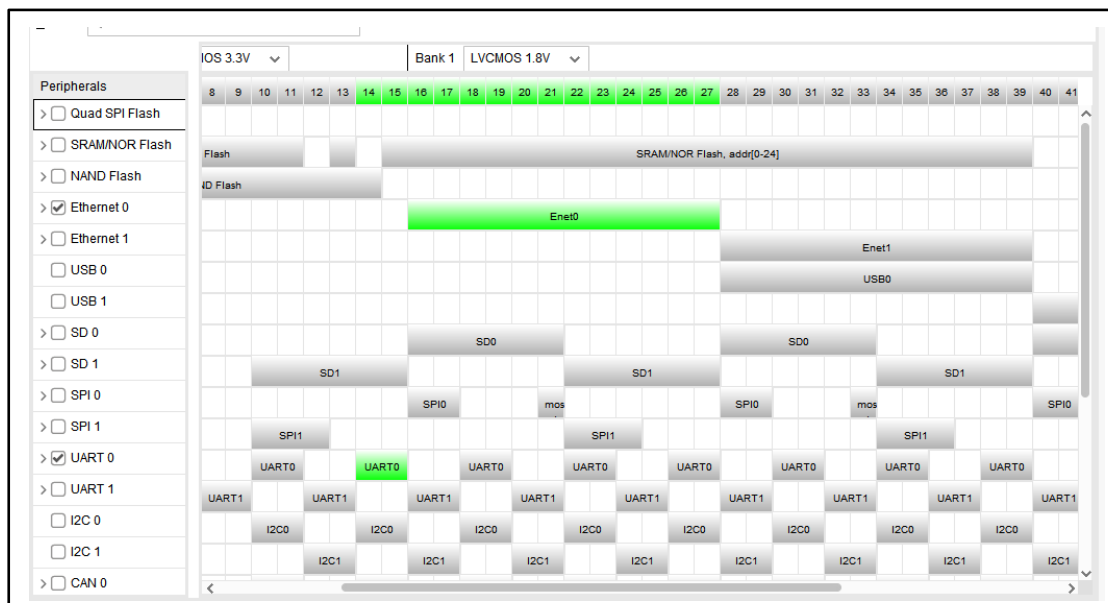


Figure 2-3 List of peripherals that MIO can connect to

Let's analyze the internal structure of MIO:

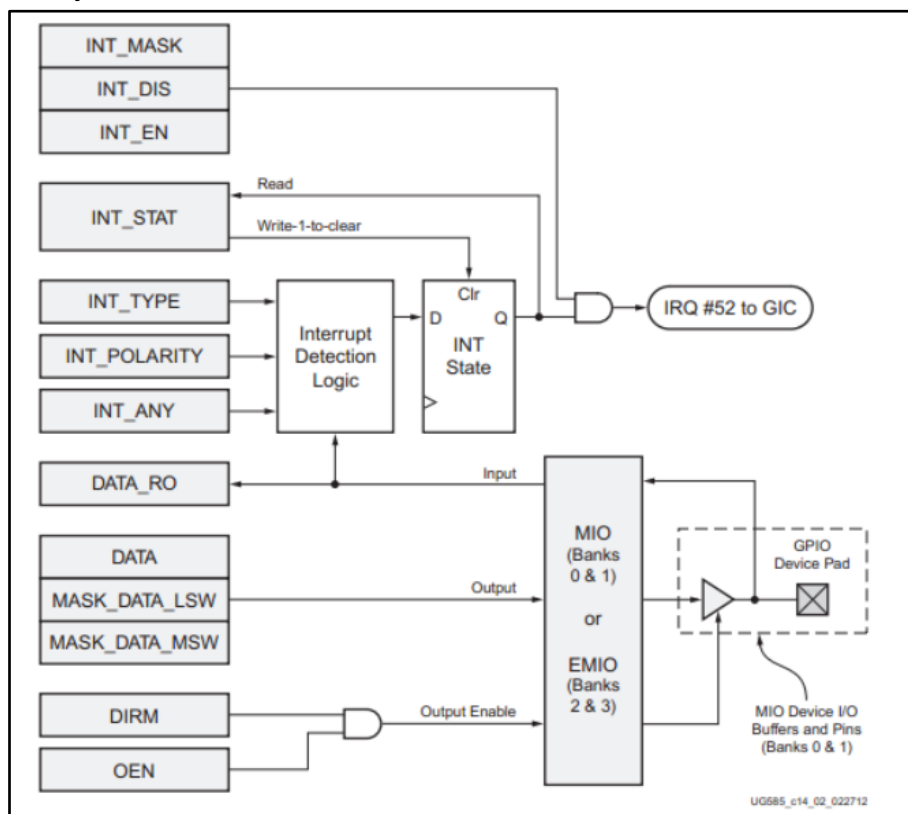


Figure 2-4 MIO internal structure

DATA_RO: This register can read the current GPIO pin value. Regardless of whether it is configured as input or output, the pin value can be read correctly. It should be noted that when this pin is not configured as a GPIO pin, reading this register will result in an uncertain value, because this register can only read the value of the GPIO pin.

DATA: When the GPIO pin is configured as an output, the value of the DATA register is the value to be output to the GPIO pin. Reading the value of this register will return the last value written to the DATA or MASK_DATA_{LSW, MSW} register, not the value currently on the GPIO pin.

MASK_DATA_LSW/ MASK_DATA_MSW: It is a combination of traditional data register and mask register. This register is 32 bits. The high 16 bits are used for traditional MASK and the low 16 bits are used for traditional DATA. Therefore, when the upper 16 bits are masked, even if the lower 16 bits are modified, the output of the pin will not be affected. MASK_DATA_LSW controls the lower 16 bits of BANK. MASK_DATA_MSW controls the high 16 bits of BANK.

DIRM: This register is a direction control register that controls input or output. This register only affects the output, because the input is always enabled. When DIRM[x]==0, output is prohibited.

OEN: Output enable. When the GPIO pin is configured as an output pin, this register controls whether the pin enables output. When configured as output disabled, this pin is tri-stated.

Here is only a brief introduction. For details, you can refer to the technical manual ug585-Zynq-7000-TRM.pdf.

2.3 Practical exercises

2.3.1 Experimental goals

Use GPIO to control the on and off of the PS-side LED through MIO to achieve the effect of LED flashing alternately.

2.3.2 Hardware resources

Haoyue Development has provided the physical picture and schematic diagram of the board, as shown in Figure 2-5, Figure 2-6, Figure 2-7.



Figure 2-5 LED physical picture

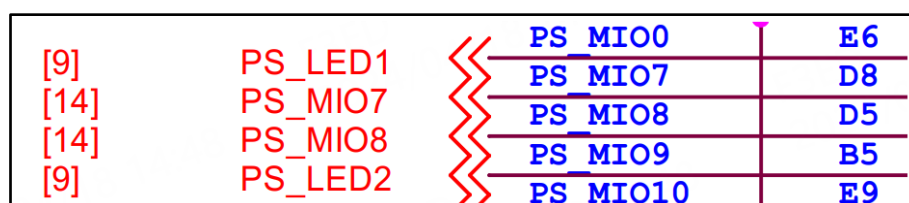


Figure 2-6 PS_LED pin schematic diagram

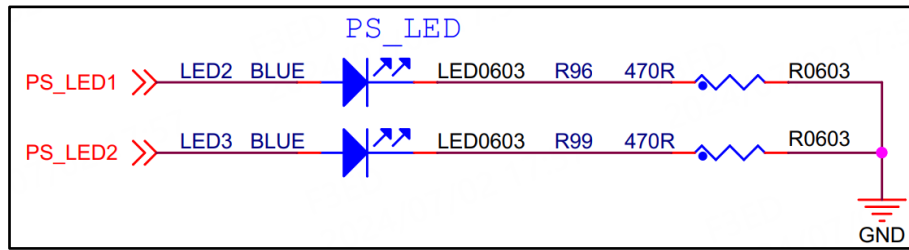


Figure 2-7 PS_LED schematic diagram

As can be seen from the schematic diagram, the pins we used in this experiment are PS_MIO0 and PS_MIO9. When the pin outputs high level, the LED lights up; when the input low level, the LED goes out.

2.3.3 Programming

1. Overall description

In this project, we need to call DDR to store and run programs, UART serial port to print information, and MIO to drive LED peripherals. The overall structure is shown in the figure below.

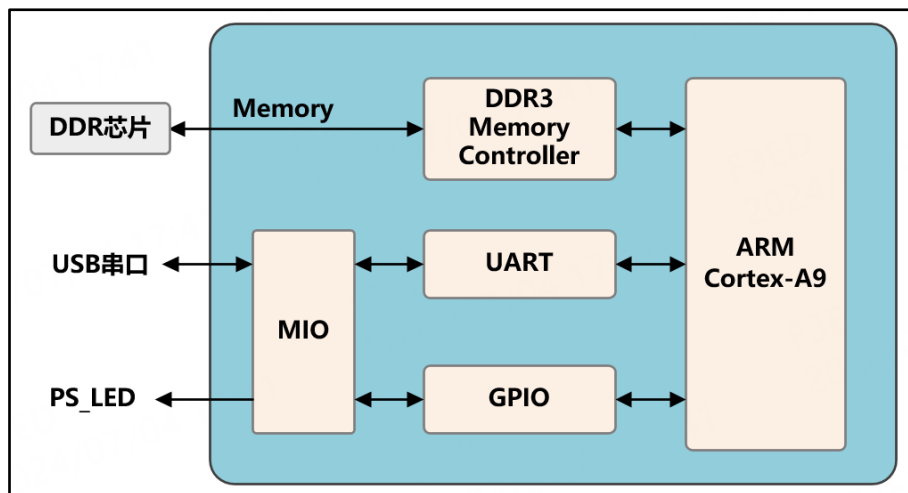


Figure 2-8 Overall block diagram of MIO driving LED

2. Hardware design

Below we briefly introduce the overall steps for establishing a vivado project in ZYNQ.

Step1: Create a new vivado project

In the Hello world experiment in the "First Acquaintance with ZYNQ" chapter, we have already created the ZYNQ PS processing system IP. We can change it based on the previous project, or re-create the project according to the steps in the "First Acquaintance with ZYNQ" chapter. Here we introduce the modifications based on the original project.

We open the Vivado project of "hello_world" in "First Introduction to ZYNQ", as shown in the figure below:

hello_world.runs	2024/4/18 9:00	文件夹	
hello_world.sdk	2024/4/18 9:49	文件夹	
hello_world.sim	2024/4/17 17:56	文件夹	
hello_world.srscs	2024/4/17 17:56	文件夹	
hello_world.xpr	2024/4/18 9:01	Vivado Project Fi...	12 KB

Figure 2-9 Open the project

After opening, select File > Project > Save As in the menu bar, as shown in the following figure:

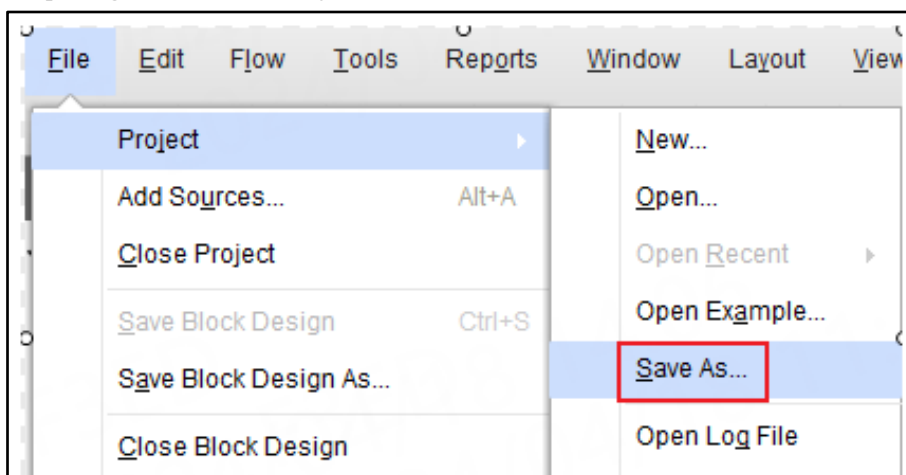


Figure 2-10 Save as new project



Figure 2-11 Enter project name and path

Then click OK to close the original project and open the newly created project. At this time, if we open the path where the project is located, we can see the file directory shown in Figure 2-12.

名称	修改日期	类型	大小
gpio_mio.cache	2024/4/25 9:47	文件夹	
gpio_mio.hw	2024/4/25 9:47	文件夹	
gpio_mio.ip_user_files	2024/4/25 9:47	文件夹	
gpio_mio.runs	2024/4/25 9:47	文件夹	
gpio_mio.sdk	2024/4/25 9:47	文件夹	
gpio_mio.sim	2024/4/25 9:47	文件夹	
gpio_mio.srcs	2024/4/25 9:47	文件夹	
gpio_mio.xpr	2024/4/25 9:47	Vivado Project Fi...	12 KB

Figure 2-12 Directory of the path where the project is located

At this point we have a new project gpio_mio based on the hello_world project, which saves the process of rebuilding the hardware. Since we do not need the software project in the original project, delete the "gpio_mio.sdk" folder indicated by the red box in the picture above.

Step2: Use IP Integrator to create a Processing System

Since we are creating a project based on the chapter "First Introduction to ZYNQ", here we do not create a block design but open a block design.

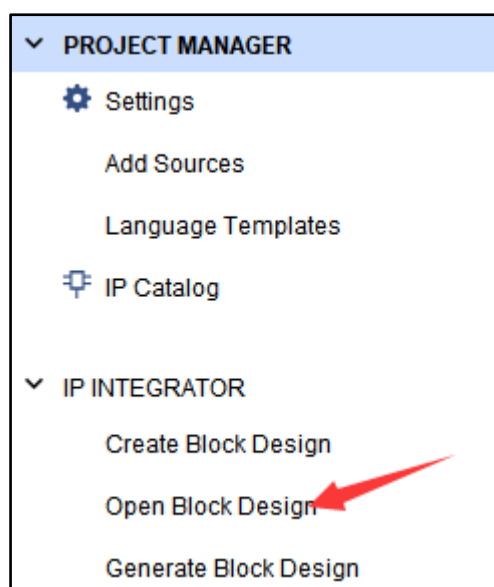


Figure 2-13 Open block design

In the Diagram window, double-click to open the ZYNQ7 Processing System redefinition window.

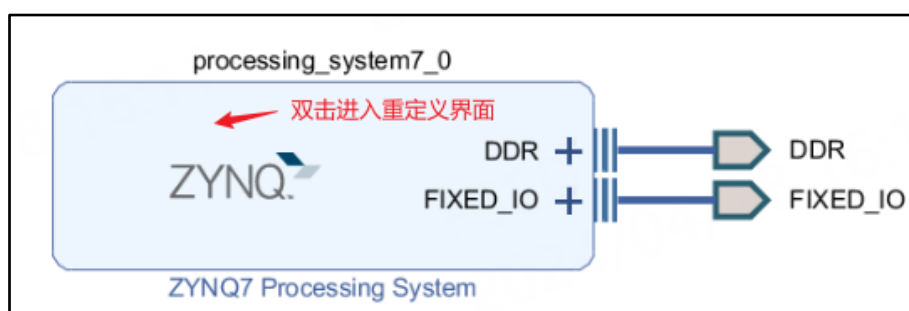


Figure 2-14 Double-click to open the processing system

Add MIO configuration in Peripheral I/O Pins, and after checking GPIO_MIO, you will find that GPIO_MIO14 and 15 are gray in the GPIO_MIO column. This is because these two GPIO_MIO are used as the interface pins of UART0. And after we check GPIO_MIO, except for the ones that have been configured, they are all highlighted in gray, but the ones actually used need to be based on the schematic diagram.

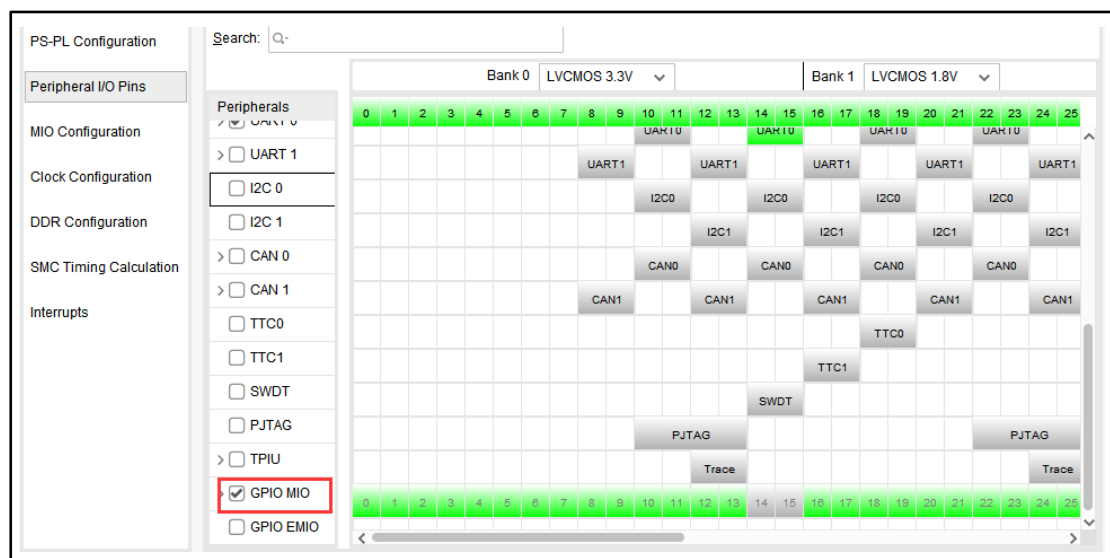


Figure 2-15 Check GPIO MIO

According to the schematic diagram, what we need to operate is GPIO MIO0 and MIO9. After completing the configuration, click the "OK" button in the lower right corner.

Step3: Generate top-level HDL module

Follow the steps shown in Figure 2-16 and Figure 2-17:

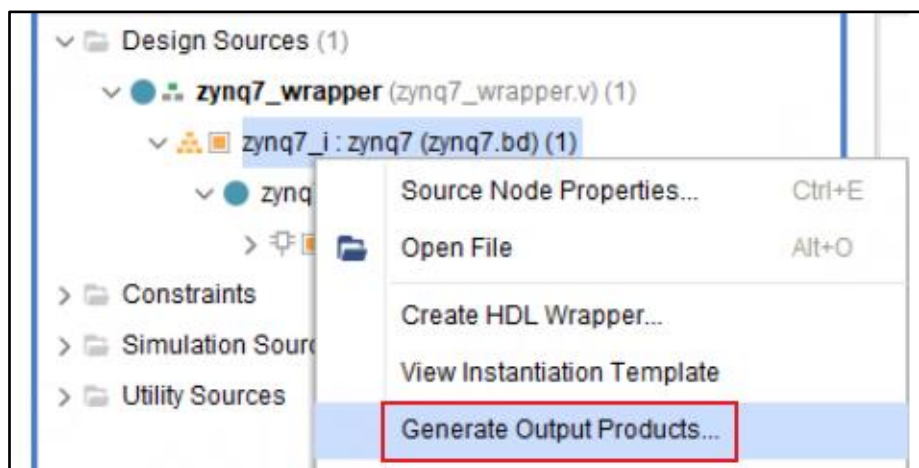


Figure 2-16 Select Generate Output Products

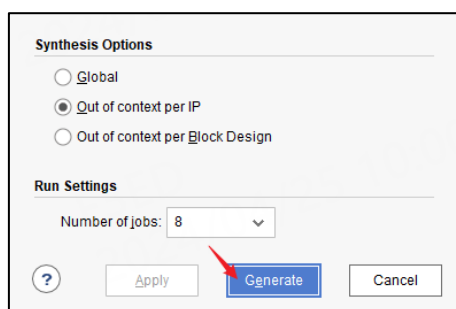


Figure 2-17 Generate

Since we are creating on the original project, we have selected the "Let Vivado manage wrapper and auto-update" option when creating the top-level file on the original project to allow the top-level module to be updated automatically. Therefore, although we have modified the block design here, there is no need to create HDL Wrapper. The software will automatically update the top-level module for us.

Step4: Generate Bitstream file

If PL resources are used in the design, pin constraints need to be added and the design needs to be synthesized, implemented, and a Bitstream file generated. Since the PL part is not used in this experiment, there is no need to generate Bitstream files, just export the hardware to the SDK.

Step5: Export SDK

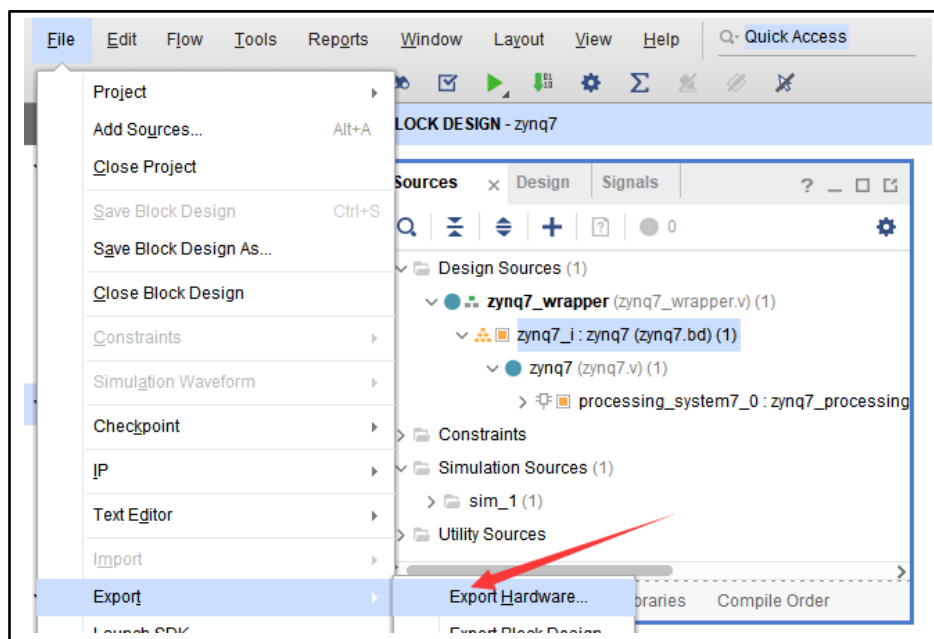


Figure 2-18 Select Export Hardware

In the pop-up dialog box, since no bitstream file is generated, there is no need to check "Include bitstream" and click the "OK" button directly.

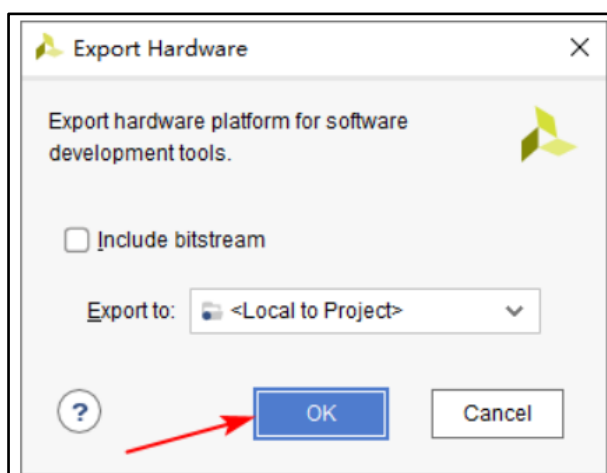


Figure 2-19 Export Hardware

If "Export to <Local to Project>" is selected here, the Vivado tool will create a new folder in the current project directory named "gpio_mio.sdk", which is our workspace for subsequent software development. During the Export Hardware process, the tool will export the hardware to the workspace in the form of a ZIP compressed file with the file name "zynq7_wrapper.hdf". This file contains the configuration information of the hardware platform we built earlier, and its suffix name .hdf means "Hardware Definition File", which is the hardware definition file.

Select File > Launch SDK in the menu bar to start the SDK development environment.

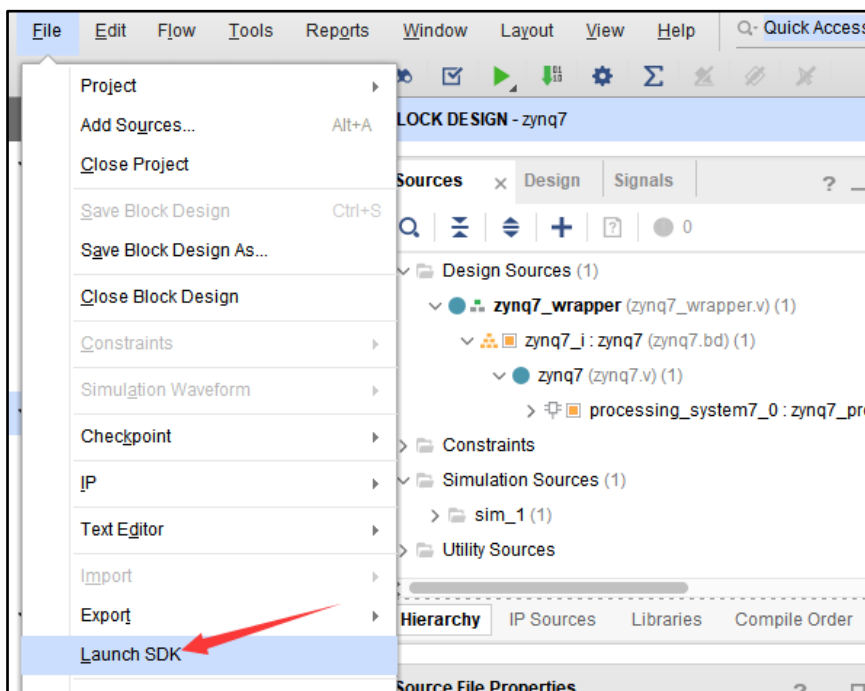


Figure 2-20 Export SDK

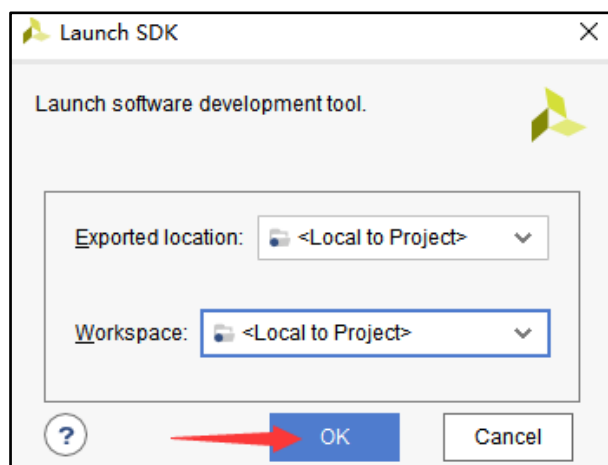


Figure 2-21 Setting up the workspace

At this point, we have completed the hardware design part of the ZYNQ embedded system. Next, you need to carry out application development in the SDK software, which is the software design part.

3. Software design

In the hardware design part, we started the SDK development environment. Next, start writing the software code.

Step1: Create a new application project

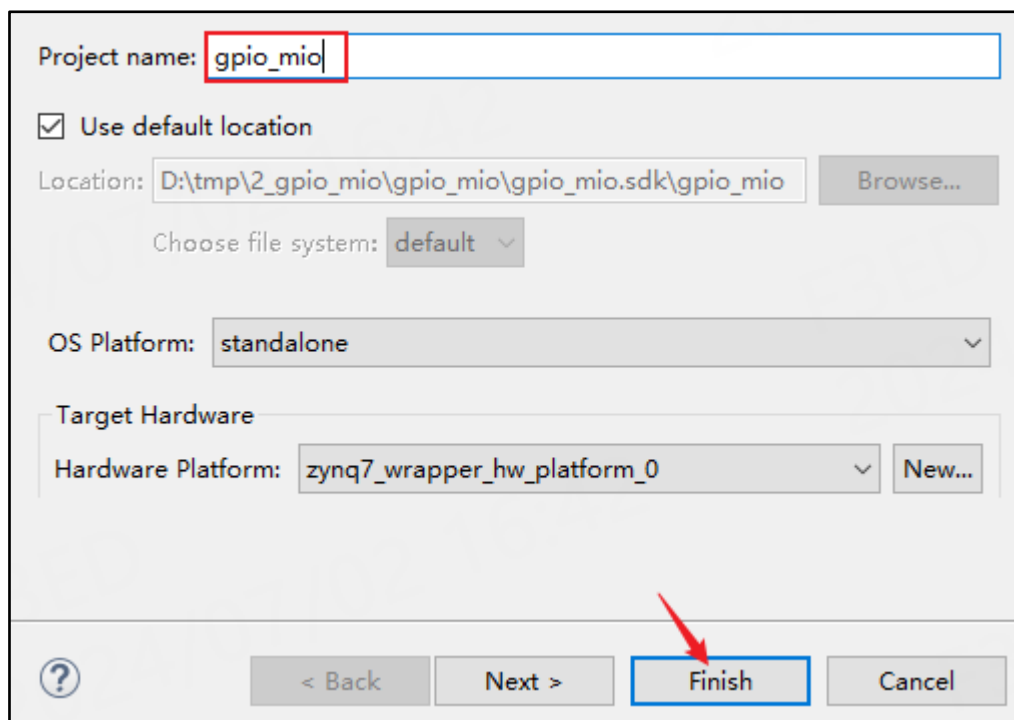


Figure 2-22 Fill in the project name

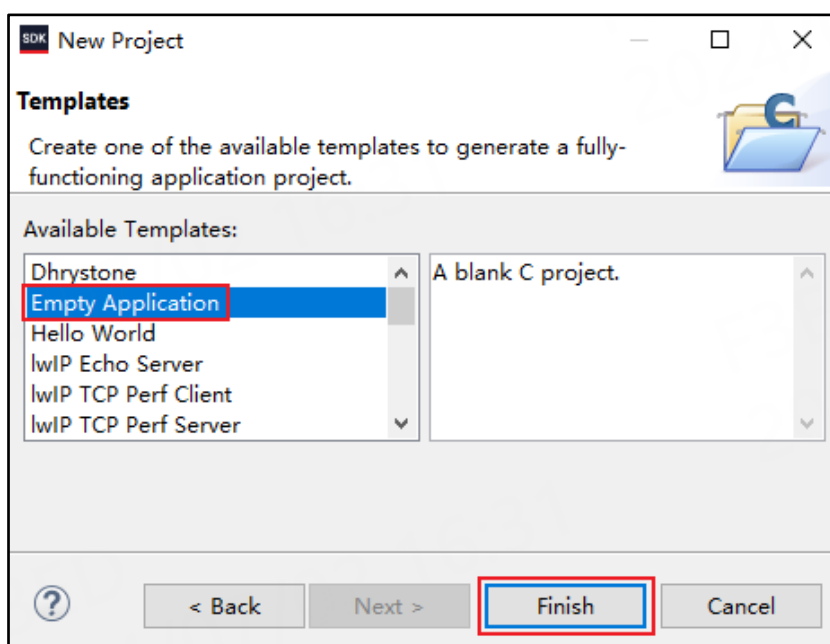


Figure 2-23 Create a new empty project template

Step2: Create a new source file

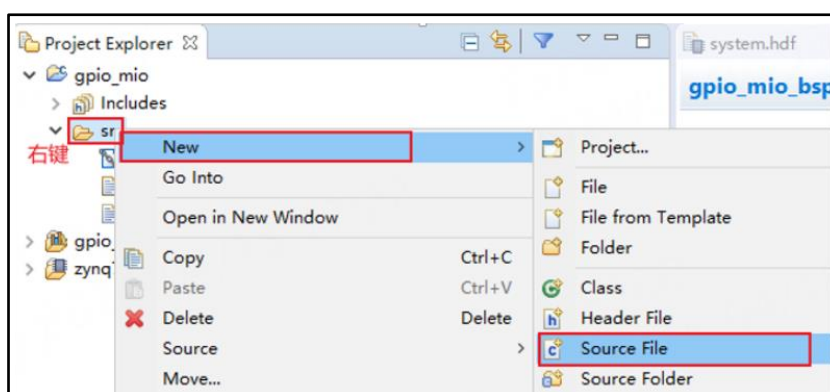


Figure 2-24 Select Source File

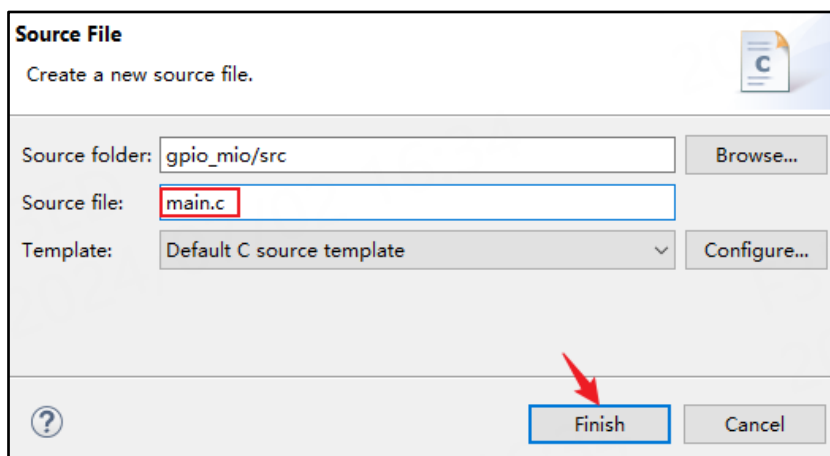


Figure 2-25 Fill in the file name (readers can choose any name and ensure the suffix is .c)

Step3: Write software code

Code List 2-1 MIO control LED reference code (main.c)

```
1 #include <stdio.h>
2 #include "xparameters.h"
3 #include "xgpiops.h"
4 #include "sleep.h"
5 #define PS_LED1 0    //ps_led1 固定为 mio0
6 #define PS_LED2 9    //ps_led2 固定为 mio9
7 int main(void)
8 {
9     int flag = 0;    //点灯标志
10    XGpioPs Psgpio_target;    //实例化 GpioPs 目标
11    XGpioPs_Config *Psgpio_ptr;    //配置 GpioPs 指针
12    Psgpio_ptr = XGpioPs_LookupConfig(XPAR_XGPIOPS_0_DEVICE_ID);
13    //查找使能 GPIOPS
14    XGpioPs_CfgInitialize(&Psgpio_target, Psgpio_ptr, Psgpio_ptr->
15    BaseAddr);    //初始化 GPIOPS
16
17    /*设置 PS_LED 引脚为输出模式*/
18    XGpioPs_SetDirectionPin(&Psgpio_target, PS_LED1, 1);
19    XGpioPs_SetDirectionPin(&Psgpio_target, PS_LED2, 1);
20
21    /*使能 PS_LED 引脚输出*/
22    XGpioPs_SetOutputEnablePin(&Psgpio_target, PS_LED1, 1);
23    XGpioPs_SetOutputEnablePin(&Psgpio_target, PS_LED2, 1);
24
25    /*无限循环检查按键状态*/
26    while (1) {
27
28        flag=~flag;
29        sleep(1);    //1s 延时
30        /*LED 交替闪烁*/
31        XGpioPs_WritePin(&Psgpio_target, PS_LED1,flag);
32        XGpioPs_WritePin(&Psgpio_target, PS_LED2,~flag);
33
34    }
35 }
```

Let me introduce the code. The second line, xparameters.h, is a header file containing device parameter information, which stores the device ID and related address of GPIO_PS. Xgpiops.h contains the declaration of PS_GPIO functions, such as the XGpioPs_WritenPin function on line 31.

Line 10 XGpioPS is the driver instance that defines the GPIO device. XGpioPs_Config defines the configuration information of the GPIO device. Then use XGpioPs_LookupConfig to find device configuration information. Then call XGpioPS_CfgInitialize to initialize GPIOPS.

The next step is to write the main function code. In order to realize that the lights keep flashing alternately, we use a flag flag in the while and assign its value to PS_LED1, and another one to invert the flag and assign it to PS_LED2. Sleep(1) delays 1s to achieve 1 second alternate flashing of the LED.

After writing, press ctrl+s to save the code, and the project will be automatically compiled. If not, you can compile the project as follows.

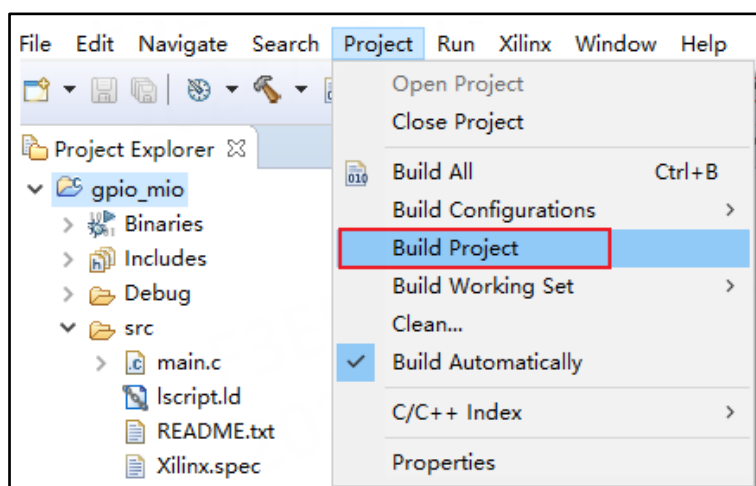


Figure 2-26 Compile project

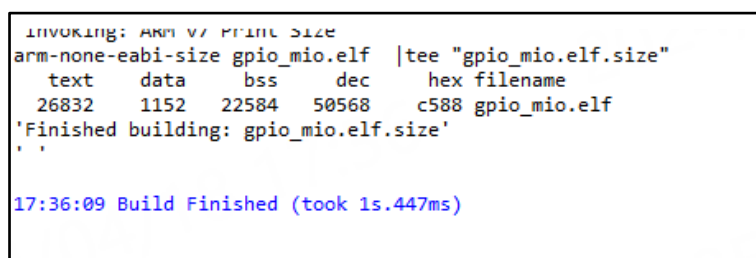


Figure 2-27 Compilation results

2.3.4 Board verification

1. Pin constraints

The pin constraints on the PS side will be automatically bound when Generate Output Products, so there is no need to pay attention to it. Since we do not use the PL side pins here, no pin constraints are needed in this chapter.

2. Result verification

As shown in the figure below, the development board is connected. After the lines are connected correctly, turn on

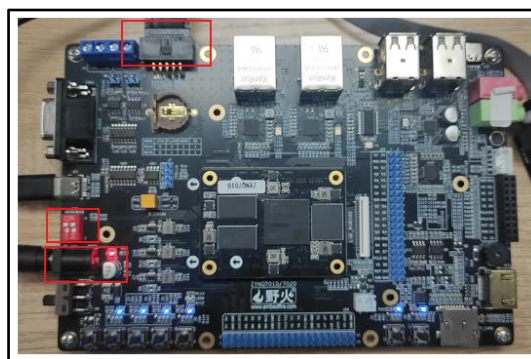


Figure 2-28 Program download connection diagram

As shown in the figure below, download the program for the development board.

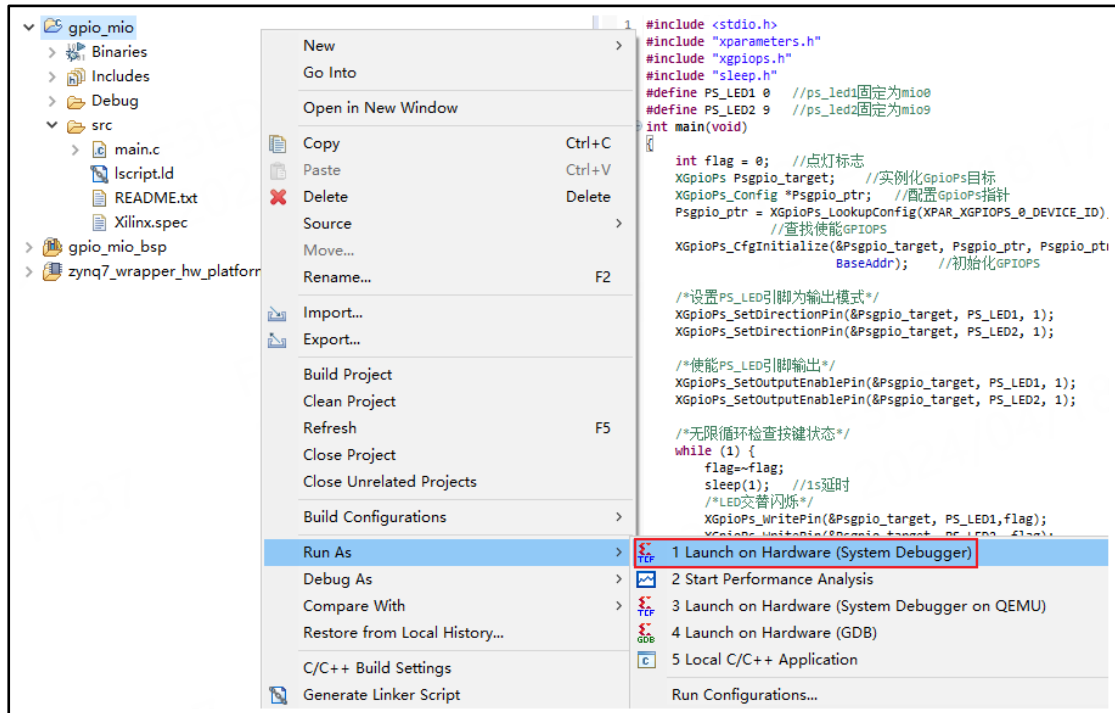


Figure 2-29 Download and run the program

After running the program, you can see that the two PS LEDs on the development board are flashing alternately.

2.4 Summary at the end of chapter

The main content of this chapter is to introduce how to use ZYNQ PS to connect GPIO. First, the control of MIO multiplexer and GPIO is introduced. Then in the vivado software, the GPIO is routed to MIO. Finally, by exporting the SDK, I programmed and controlled the GPIO to realize the alternating on and off of the LED. I also understood the basic steps of general initialization of the GPIO and learned how to use ZYNQ's GPIO.

2.5 Development training

You can try to modify it so that every time PS_LED1 flashes for 5 seconds, PS_LED2 stays on for 2 seconds.

Chapter 3 EMIO Control LED

3.1 Chapter Introduction

In the previous chapter, we introduced how to control ZYNQ external devices through MIO, but MIO cannot be connected to the I/O resources on the PL side, and the MIO on the PS side only has 54 pins. At this time, we can use extended MIO (Extended MIO, EMIO) to connect to external devices or IP modules that have been implemented on the PL side.

In this chapter we will use ZYNQ to control the LED on the PS side through EMIO.

3.2 Theoretical study

The block diagram of GPIO is shown in the figure below, from which we can see that GPIO is divided into 4 Banks, of which Bank0 and Bank1 are connected to MIO; while Bank2 and Bank3 are connected to EMIO, a total of 64 EMIOs.

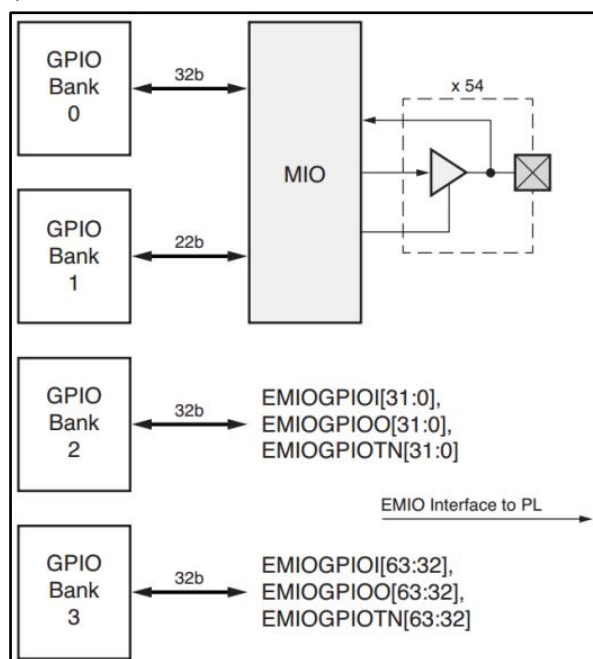


Figure 3-1 GPIO block diagram

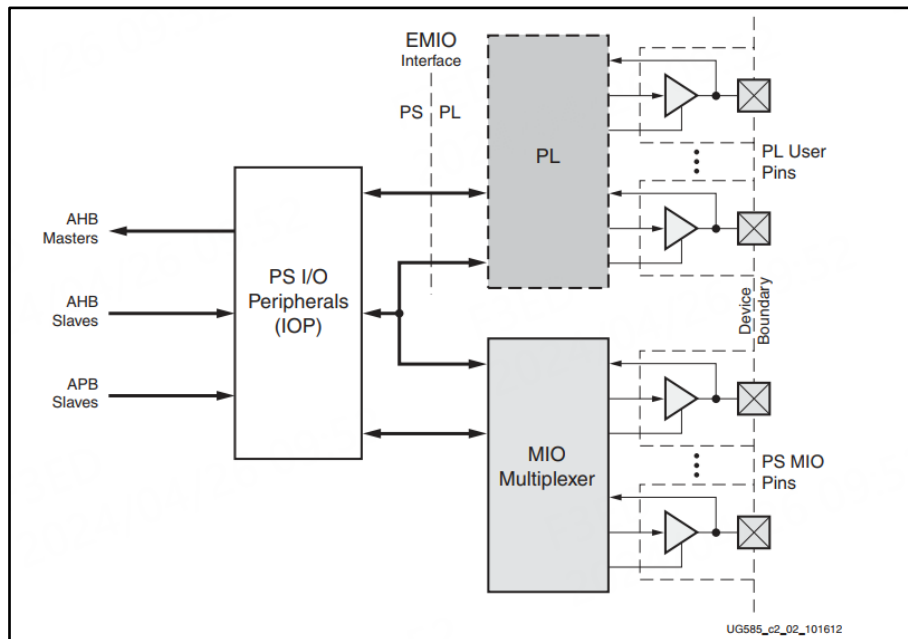


Figure 3-2 MIO-EMIO overview

As shown in Figure 3-2, MIO is the basis for I/O peripheral connection. Software programs the routing of I/O signals to MIO pins. I/O peripheral signals can also be routed to the PL (including PL device pins) through the EMIO interface. The PS side can directly connect to the I/O resources of the PL side through EMIO, or connect to the IP core that has been implemented on the PL side, as shown in the figure below.

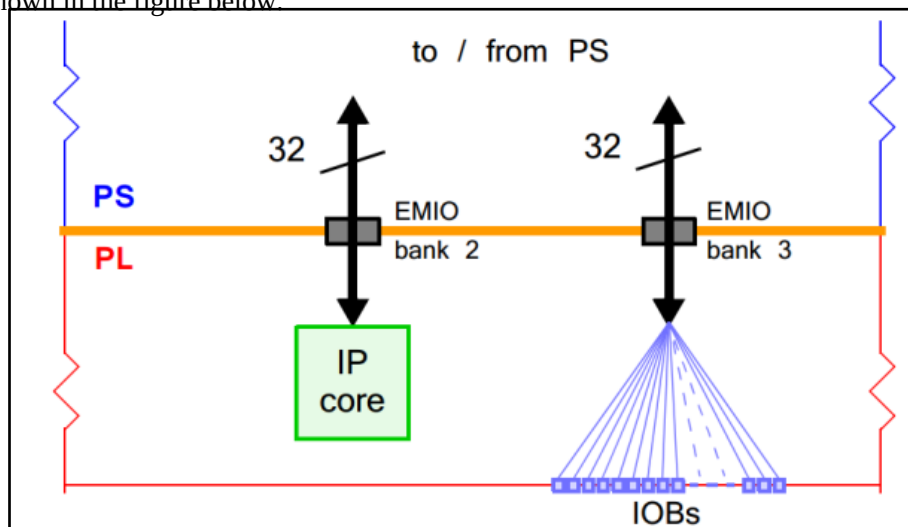


Figure 3-3 The role of EMIO

MIO is the basic pin of peripherals and is fixedly matched with peripherals. For example, serial ports and Ethernet have fixed circuits, so we do not need pin binding.

EMIO is an arbitrary configurable multi-function pin on the PL side. The pin is flexible and can be bound at will, so using EMIO requires pin binding. Although EMIO can flexibly bind pins, it does not mean that all peripherals can be connected to EMIO. For the specific peripheral routing table, please refer to the official manual ug585.

The register interface of the EMIO Bank is almost identical to the MIO Bank described previously. However, EMIO is just the connection interface between PS and PL, so there are some slight differences:

(1) The input comes from the PL end and has nothing to do with the current output value and the OEN register. When DIRM is configured as 0, it is input. You can use the DATA_RO register to read the current value.

(2) The output is related to the programming of DATA_MASK_DATA_LSW_MASK_DATA_MSW. The output does not support tri-state and has nothing to do with the value of OEN.

(3) The output enable comes directly from the output enable of the PS terminal, that is, EMIOGPIOTN is controlled

$$\text{EMIOGPIOTN}[x] = \text{DIRM}[x] \& \text{OEN}[x].$$

The difference mentioned above refers to the signal between PS and PL, not the signal from PL to peripheral pin. The pin signals from PL to peripherals support tri-state, as shown in Figure 3-4.

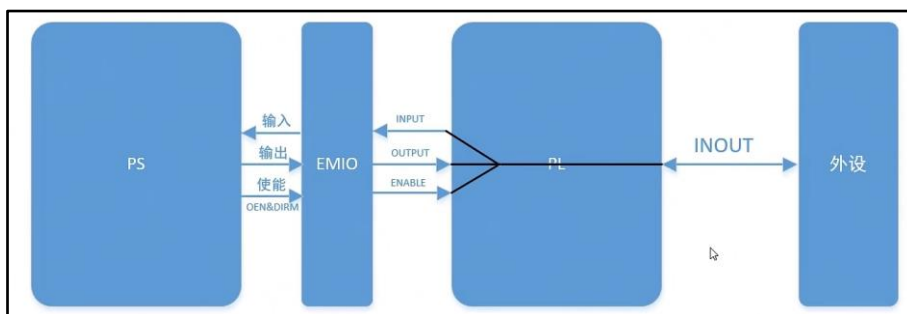


Figure 3-4 PS is connected to the PL terminal through EMIO

3.3 Practical exercises

3.3.1 Experimental goals

Use GPIO to control the PS-side LED on and off through EMIO to achieve the LED flashing effect.

3.3.2 Hardware resources

The physical diagram and schematic diagram of the buttons on the PL side and the LED on the PS side of the Haoyang



Figure 3-5 PL_KEY and PS_LED physical picture

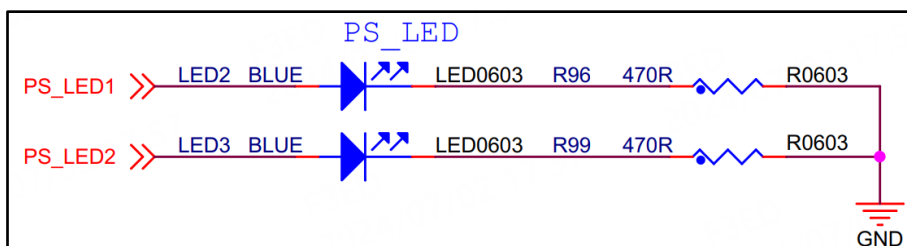


Figure 3-6 PS_LED schematic diagram

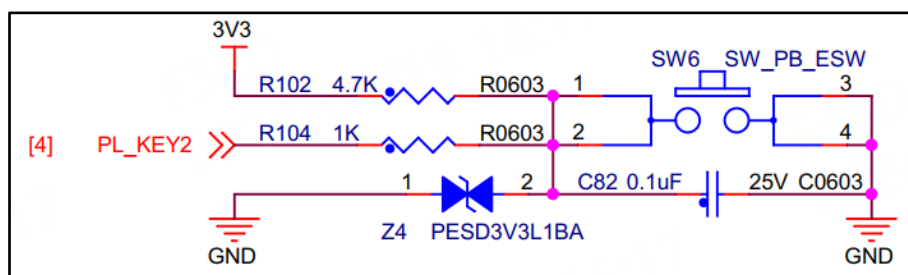


Figure 3-7 PL_KEY schematic diagram

3.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in the figure below.

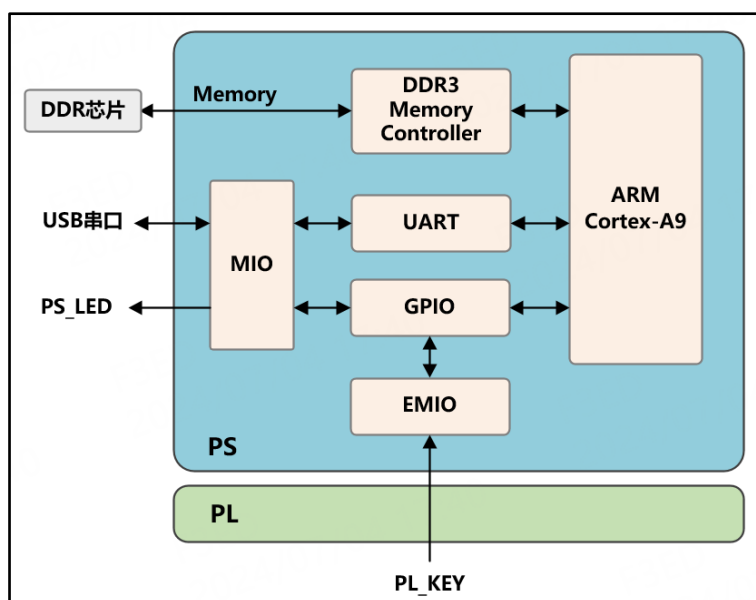


Figure 3-8 Overall block diagram of EMIO driving LED

2. Hardware design

Below we briefly introduce the overall steps for establishing a vivado project in ZYNQ.

Step1: Create a new vivado project

In the "gpio mio" experiment in the "MIO Control LED" chapter, we have already created the ZYNQ PS processing system IP. We can change it based on the previous project, or re-create the project according to the steps in the "First Understanding of ZYNQ" chapter. Here we introduce the modifications based on the original project.

Open the "MIO Control LED" chapter "gpio mio" to experiment with the vivado project. After opening, select File > Project > Save As in the menu bar, click Select, and in the pop-up dialog box, change the project name to gpio_emio, and then click "OK" as shown in the figure below:

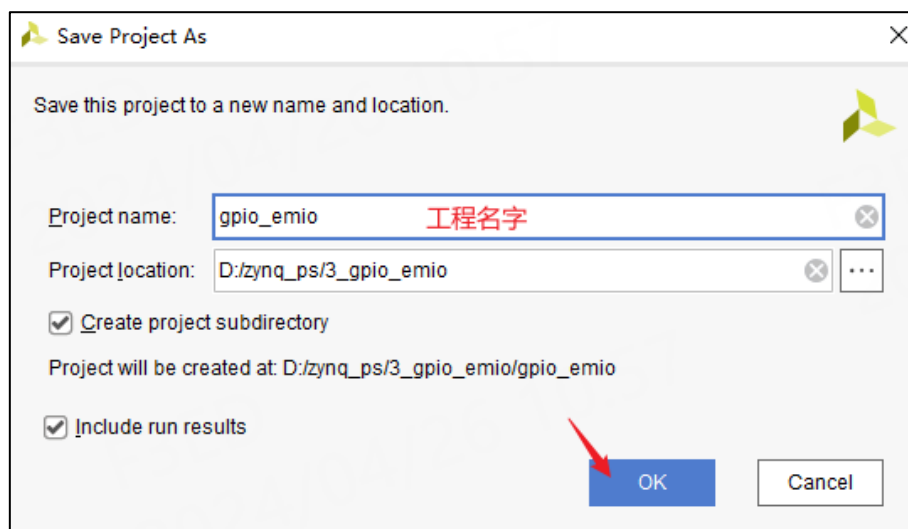


Figure 3-9 Save Project As

Then click OK to close the original project and open the newly created project. At this time, if we open the path where the project is located, the file directory shown is as shown below:

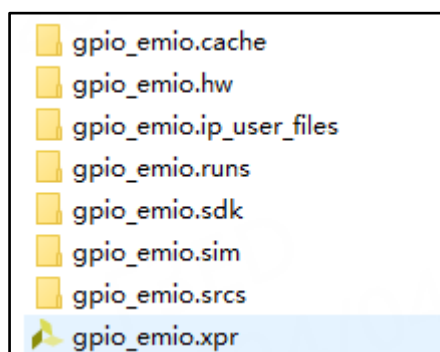


Figure 3-10 Directory of the path where the project is located

At this time, we have a new project gpio_emio based on the gpio_mio project, which saves the process of rebuilding the hardware. Since we do not need the software project in the original project, delete the "gpio_emio.sdk" folder in the file directory.

Step2: Use IP Integrator to create a Processing System

Since we are creating a project based on the "MIO Control LED" chapter, here we do not create a block design but open a block design.

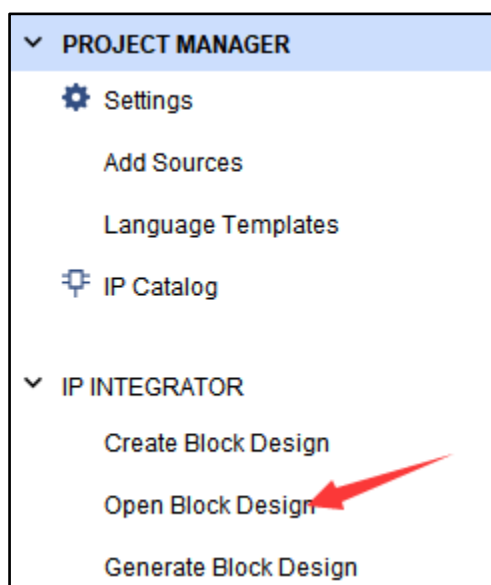


Figure 3-11 Open Block Design

In the Diagram window, double-click the added ZYNQ7 Processing System module to enter the ZYNQ7 processing system configuration interface. The configuration of this chapter is almost the same as the previous chapter, except that EMIO GPIO is added to the MIO Configuration. Since this experiment only uses one EMIO, the bit width is selected as 1, as shown in the figure below.

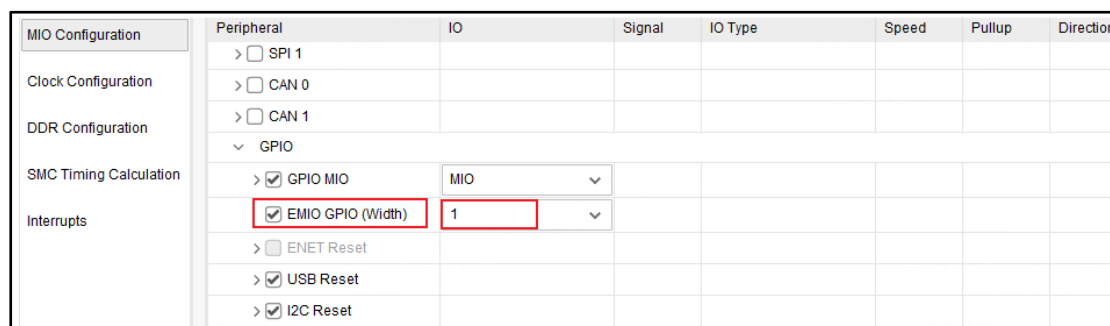


Figure 3-12 Check EMIO GPIO

After completing the configuration, click the "OK" button in the lower right corner. Then in the Diagram window, you can see that ZYNQ7 Processing System has one more GPIO_0 port than the one in the previous chapter, as shown in the figure below:

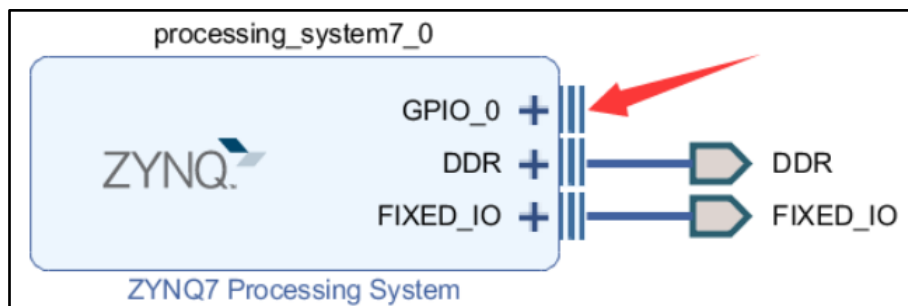


Figure 3-13 GPIO interface through EMIO

The EMIO port needs to be extracted manually:

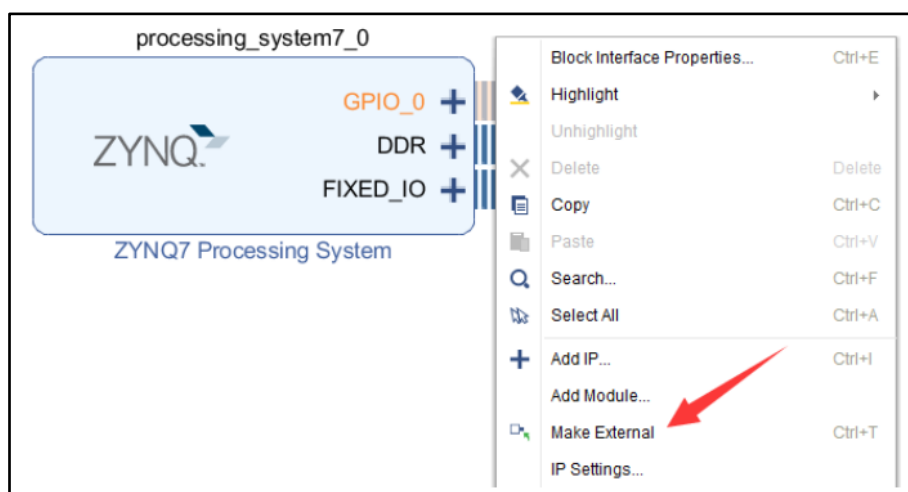


Figure 3-14 Configuring UART serial port

You can see that ZYNQ7 Processing System leads to an interface named GPIO_0_0, as shown in the figure below:

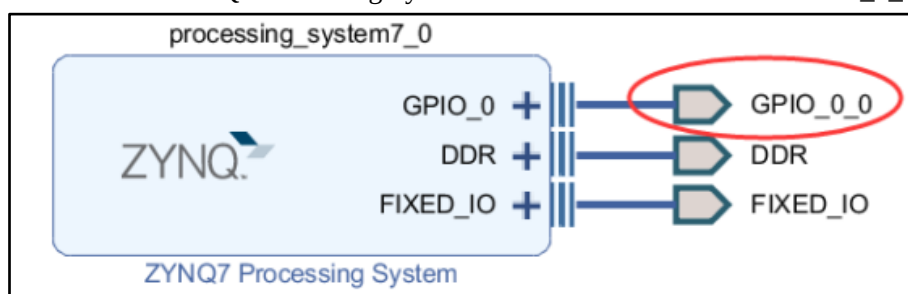


Figure 3-15 Leading GPIO interface

Here we click to select the interface, and change the name of the interface to GPIO_EMIO_KEY in the External Interface Properties column on the left. Of course, there is no problem if you do not modify it.

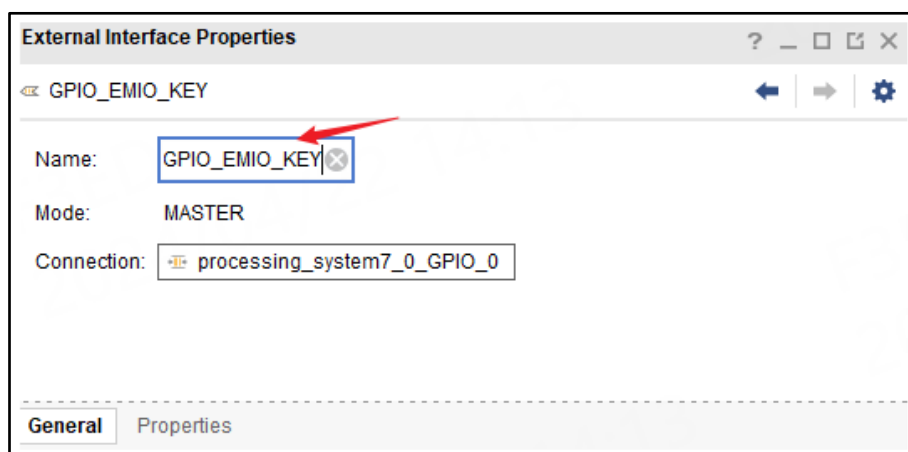


Figure 3-16 Modify GPIO name

Then press the arrow as shown in the picture.

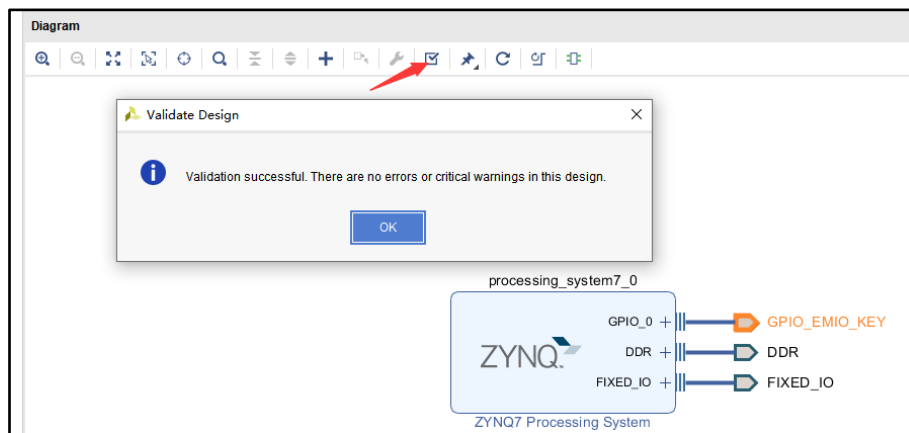


Figure 3-17 Verify design

After the verification is completed, a dialog box will pop up indicating that there are no errors or critical warnings. Click "OK". If an error message or warning appears, you need to check your design.

The Block design for this experiment is completed, press Ctrl + S to save.

Step3: Generate top-level HDL module

Right-click `zynq7.bd` in the "Hierarchy" tab of Sources, and click "Generate Output Products" in the pop-up dialog box, as shown in the figure below:

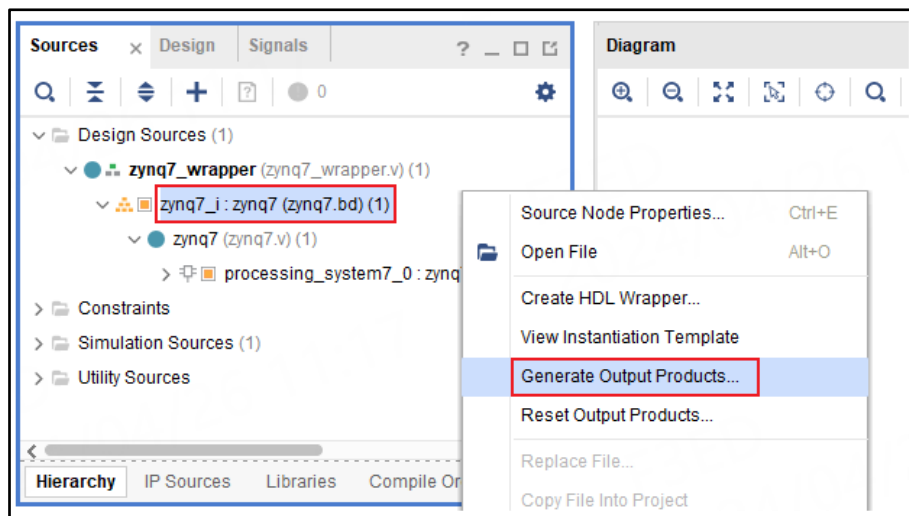


Figure 3-18 Select Generate Output Products

In the pop-up dialog box, click "Generate" to start the generation. You need to wait for a while and click "OK" after completion.

Since we created it on the original project, we have selected the "Let Vivado manage wrapper and auto-update" option when creating the top-level file on the original project to allow the top-level module to update automatically. Therefore, although we have modified the block design here, there is no need to create HDL Wrapper. The software will automatically update the top-level module for us.

Step4: Generate Bitstream file

This design uses PL's IO resources, so you need to add pin constraints, synthesize, implement and generate Bitstream files for the design.

Find SYNTHESIS in the Flow Navigator navigation bar on the left, and click "Run Synthesis" in this option. Then click "OK" in the pop-up dialog box. As shown in Figure 3-19 and Figure 3-20:

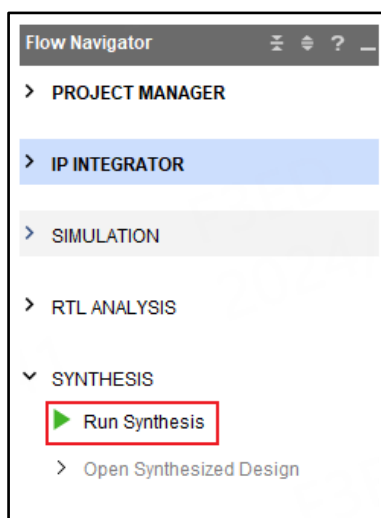


Figure 3-19 Run Synthesis

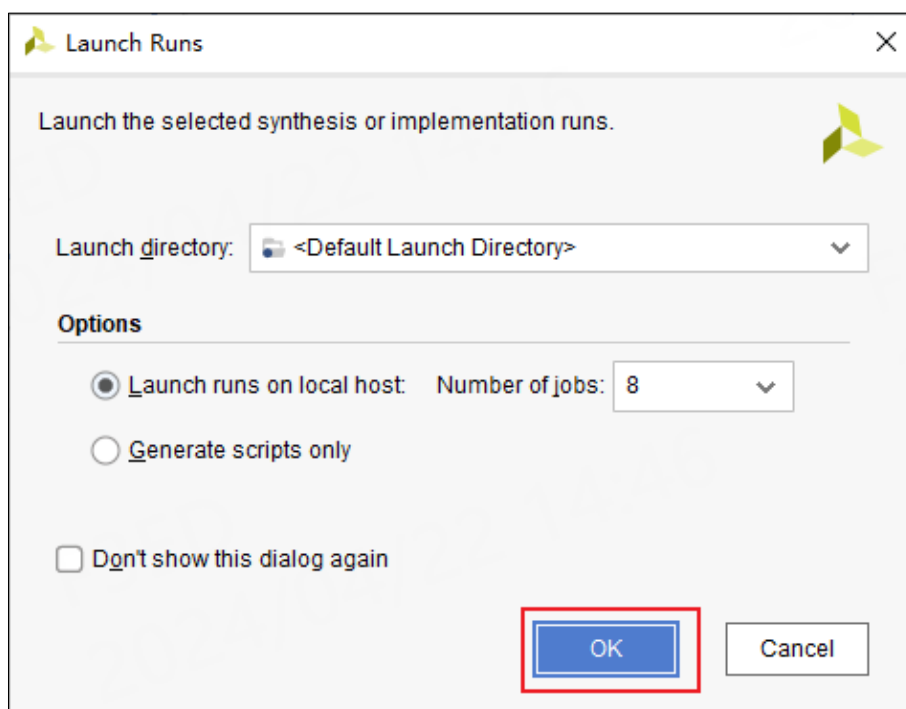


Figure 3-20 Elaborate Design dialog box

Wait for the synthesis to complete. After completion, click and select "Cancel" in the pop-up dialog box, as shown

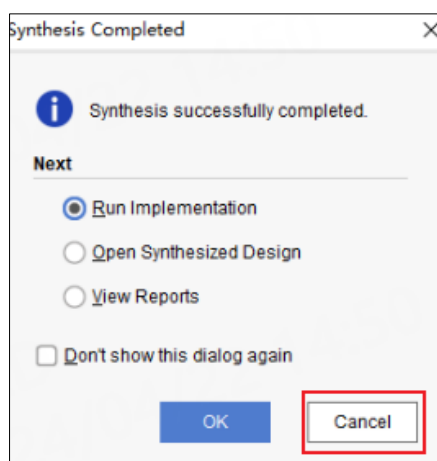


Figure 3-21 Synthesis Completed

After synthesis is completed, you can find SYNTHESIS in the Flow Navigator navigation bar on the left, click on the selected "Open Synthesized Design", and select "Schematic" in the pop-up drop-down list to open the schematic diagram of this design, as shown in the figure below:

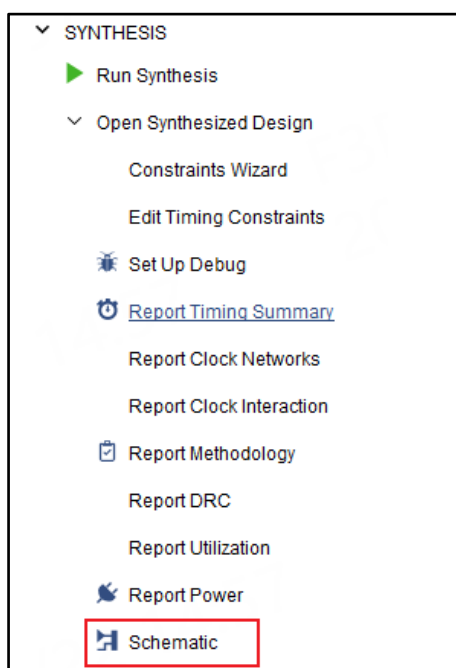
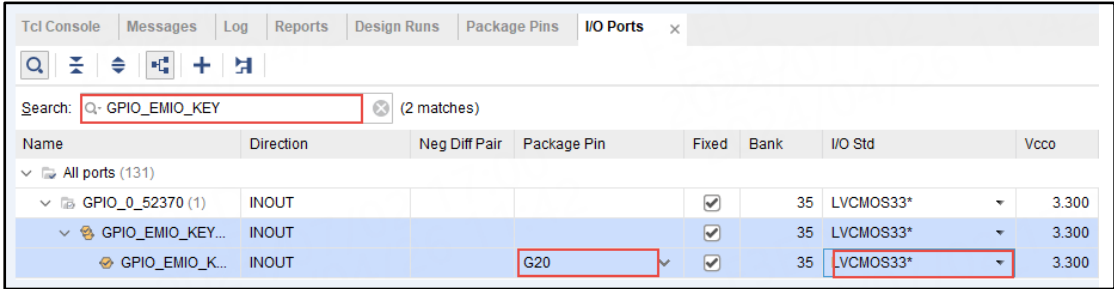


Figure 3-22 Open the design schematic

Find the I/O Ports window below the SYNTHESIZED DESIGN interface. If not, click Layout in the menu bar and select I/O Planning from the drop-down list. Then according to the schematic diagram of the development board, configure the pins of the PL resources used, mainly involving the pin name and the standard voltage of the bank where the IO is located. This experiment used G20, which corresponds to Bank35 and the standard voltage is 3.3V, as shown in the following figure:



Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco
▼ All ports (131)							
▼ GPIO_0_52370 (1)	INOUT			✓	35	LVC MOS33*	3.300
▼ GPIO_EMIO_KEY...	INOUT			✓	35	LVC MOS33*	3.300
▼ GPIO_EMIO_K...	INOUT		G20	✓	35	LVC MOS33*	3.300

Figure 3-23 Pin binding

After the setting is completed, press the shortcut Ctrl+S to save the pin constraints, enter the file name "emio" in the pop-up dialog box, and finally click "OK", as shown in the following figure:

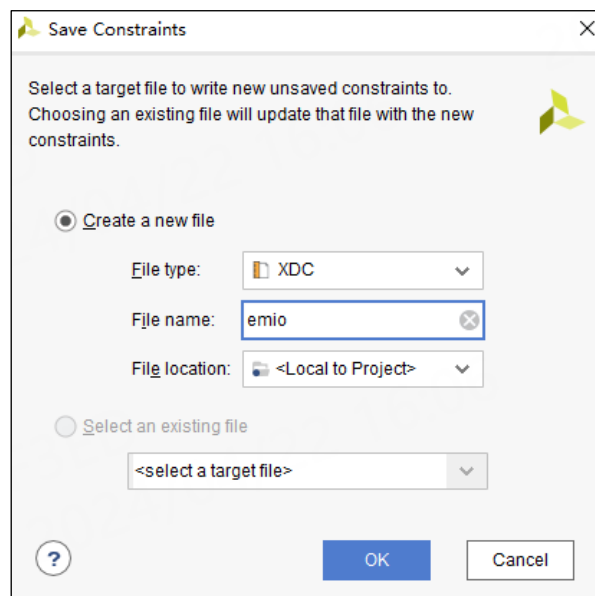


Figure 3-24 Save pin constraints

Then find PROGRAM AND DEBUG in the Flow Navigator navigation bar on the left, click "Generate Bitstream" in this option, and click "YES" and "OK" in sequence according to the prompts in the dialogue that pops up later. After completion, a dialog box will pop up prompting "Bitstream Generation successfully completed". At this time, the Bitstream file has been generated. Click "Cancel" to close the popup.

Step5: Export SDK

Since we are using the logical resources or IO resources on the PL side, we need to check "Include bitstream" and then click the "OK" button.

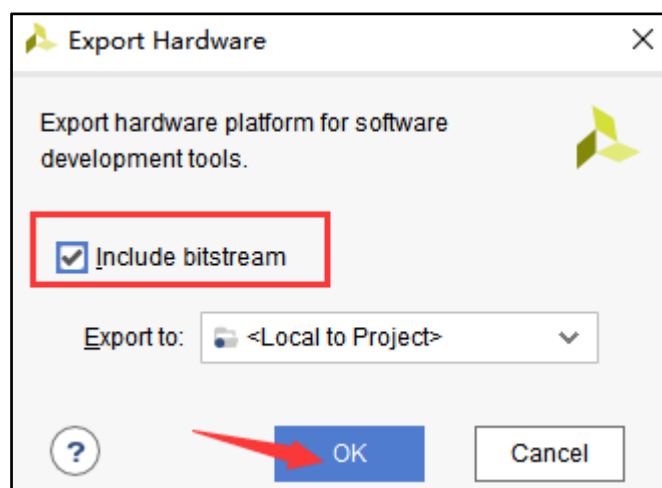


Figure 3-25 Export hardware

Select File -> Launch SDK -> OK in the menu bar to start the SDK development environment.

At this point, we have completed the hardware design part of the ZYNQ embedded system. Next, you need to carry out application development in the SDK software, which is the software design part.

3. Software design

In the hardware design part, we started the SDK development environment, and then we created a new application project and prepared to write software code.

Step1: Create a new application project

Select File->New->Application Project in the upper right corner tab.

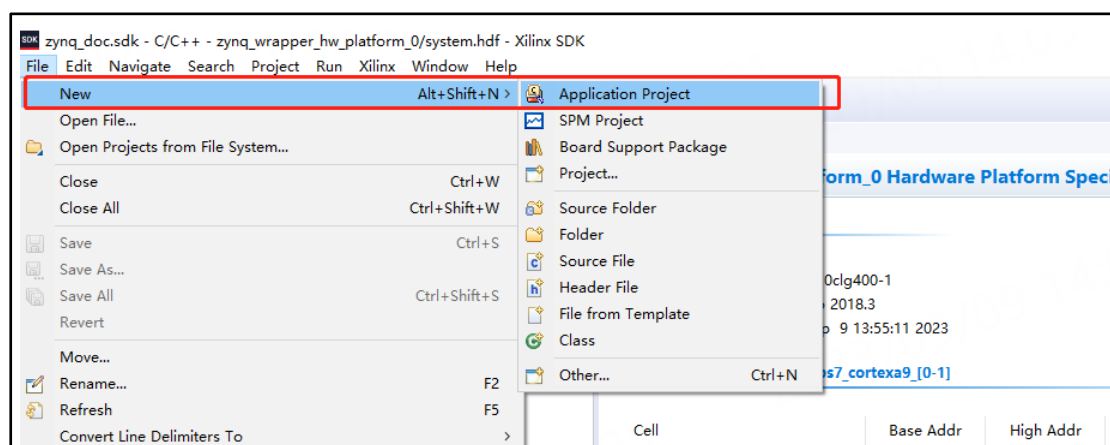


Figure 3-26 Create a new application project

Fill in the project name in Project name and click next.

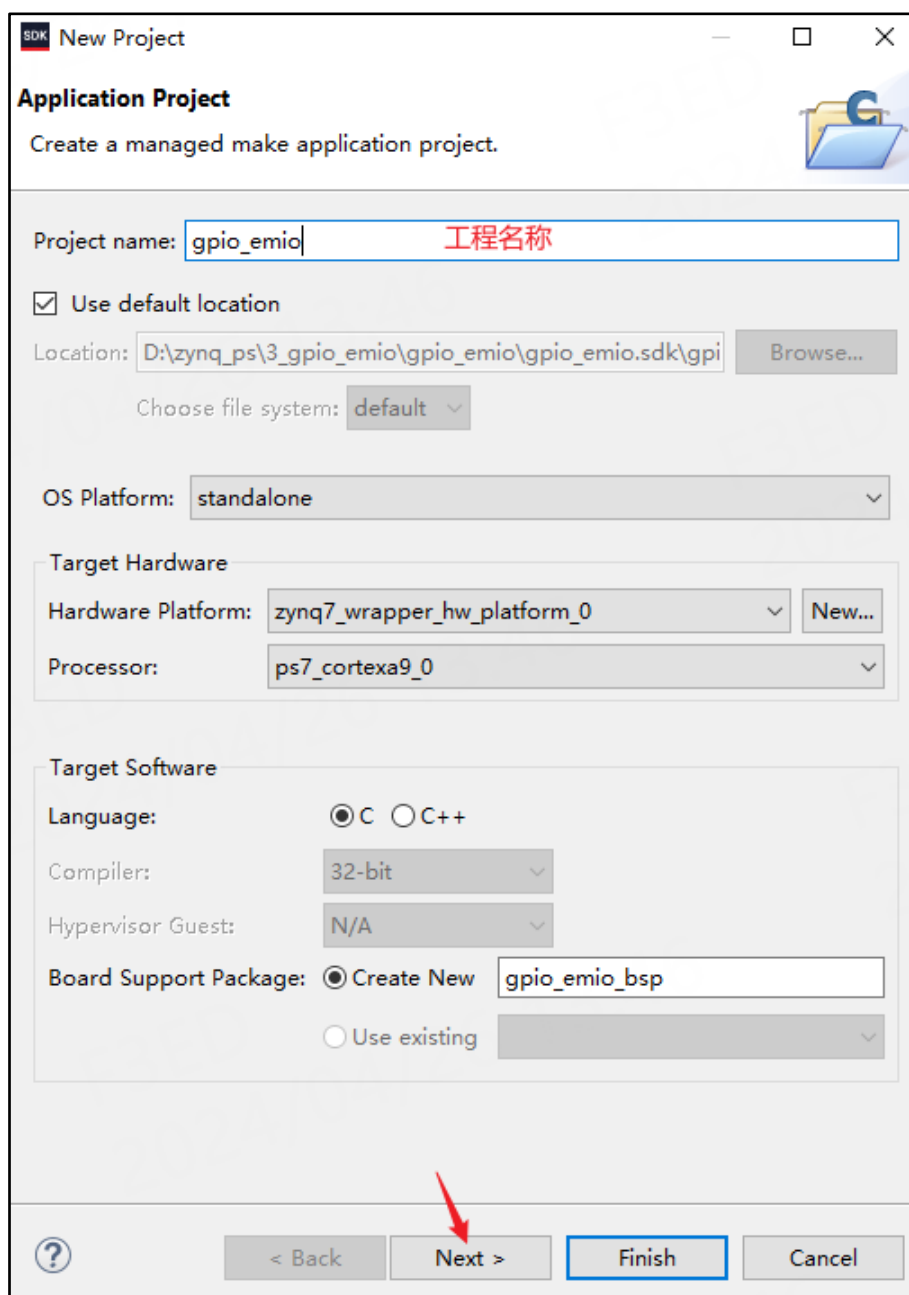


Figure 3-27 Fill in the project name

Select Empty Application in the next pop-up window and click Finish to complete the creation of the application p
Step2: Create a new source file

Next, right-click the src folder and create a new source file. src->New->Source File.

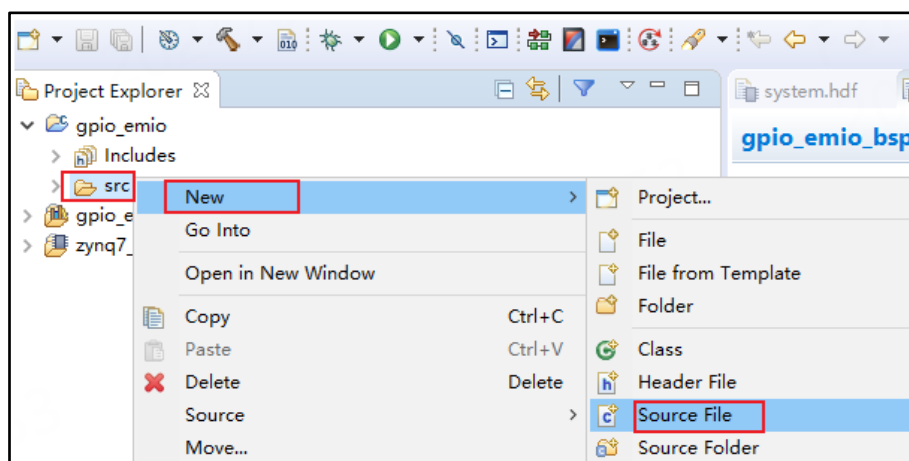


Figure 3-28 Select Source File

Then in the tab that pops up, fill in the file name for Source file, which I chose as main.c. Readers can choose whatever they want, as long as the suffix is .c.

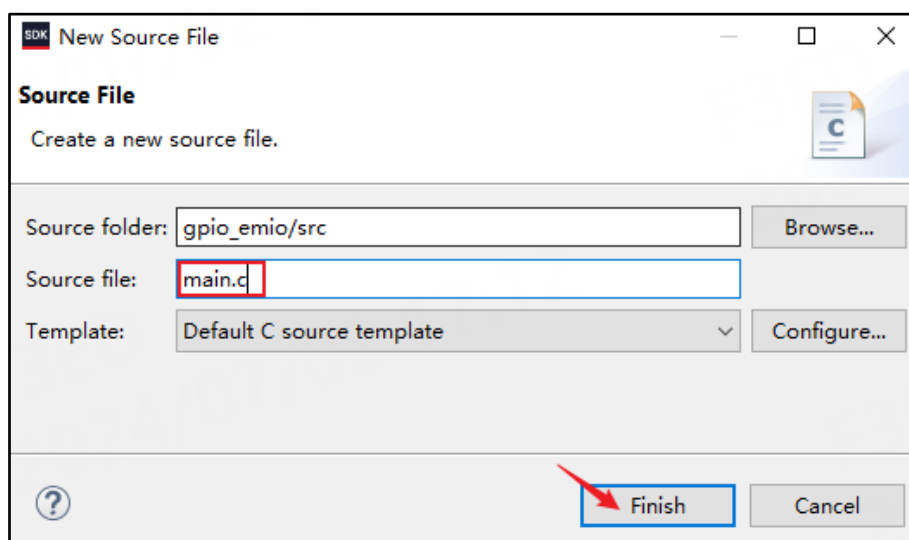


Figure 3-29 New Source File

Step3: Write code

Code List 3-1 EMIO control LED reference code (main.c)

```
1 #include <stdio.h>
2 #include "xparameters.h"
3 #include "xgpiops.h"
4 #include "sleep.h"
5 #define PS_LED1 0    /* ps_led1 固定为 mio0 */
6 #define PS_LED2 9    /* ps_led2 固定为 mio9 */
7 #define PL_KEY2 54   /* pl_key2 绑定到 emio0 */
8 int main(void)
9 {
10     int flag = 0;    /* 点灯标志 */
11
12     XGpioPs Psgpio_target;    /* 实例化 GpioPs 目标 */
13     XGpioPs_Config *Psgpio_ptr; /* 配置 GpioPs 指针 */
14
15     /* 查找使能 GPIOPS */
```

```

16 Psgpio_ptr = XGpioPs_LookupConfig( 17 XPAR_XGPIOPS_0_DEVICE_ID); 18 19 /* Init
ialize GPIOPS */ 20 Psgpio_ptr->BaseAddr); 22 23 /* Set the PS_LED pin to output mode */ 2
4 XGpioPs_SetDirectionPin(&Psgpio_target, PS_LED1, 1); 25 Enable PS_LED pin output */ 2
8 XGpioPs_SetOutputEnablePin(&Psgpio_target, PS_LED1, 1); 29 XGpioPs_SetDirectionPin(
&Psgpio_target, PL_KEY2, 0); 33 34 /* Infinite loop to check key status */ 35 while (1) { 36 if
(XGpioPs_ReadPin(&Psgpio_target, PL_KEY2) == 0) { 37 /* If the key is pressed, the flag pos
ition is reversed */ 38 flag=~flag; 39 /*LED flashes alternately*/ 40 XGpioPs_WritePin(&Psgp
io_target, PS_LED1,flag); 41 PL_KEY2) 43 == 0) { 44 /* Wait for the key to be released */ 45
} 46 } 47 } 48 }

```

The introduced library function is actually exactly the same as the previous chapter, except that one more EMIO pin is defined. You can see that the first EMIO is 54. Here we can actually calculate it. Because our EMIO is located in Bank2, and GPIO's Bank0 and Bank1 have 32 and 22 signals respectively. That is to say, the number is 0-53, and PL_KEY2 is bound to EMIO0 during vivado software configuration, so the corresponding pin of PL_KEY2 is 54, which can also be found in xgpiops.h. As shown below:

```

/*
#define XGPIOPS_DEVICE_MAX_PIN_NUM (u32)118 /**< Max pins in the Zynq GPIO device
* 0 - 31, Bank 0
* 32 - 53, Bank 1
* 54 - 85, Bank 2
* 86 - 117, Bank 3
*/

```

Figure 3-30 GPIO diagram

In the GPIO initialization section, the two LED lights continue to be configured as output mode and the EMIO is configured as input mode. After that, the key input signal is continuously detected in the while loop. Our key is high level by default. When pressed, it will become low level. The flag is inverted every time it is pressed.

Although this can simply realize the button lighting, if readers analyze it carefully, they will find that this code still has many flaws. For example, when the button is pressed all the time, if there are other things to process in the while loop, then these things will not be processed. We will use interrupts to solve this problem later.

Step4: Compile the project

After writing the code, we press ctrl+s and the project will be compiled automatically. If not, we can compile it manually through the tab. Project->Build Project is as follows:

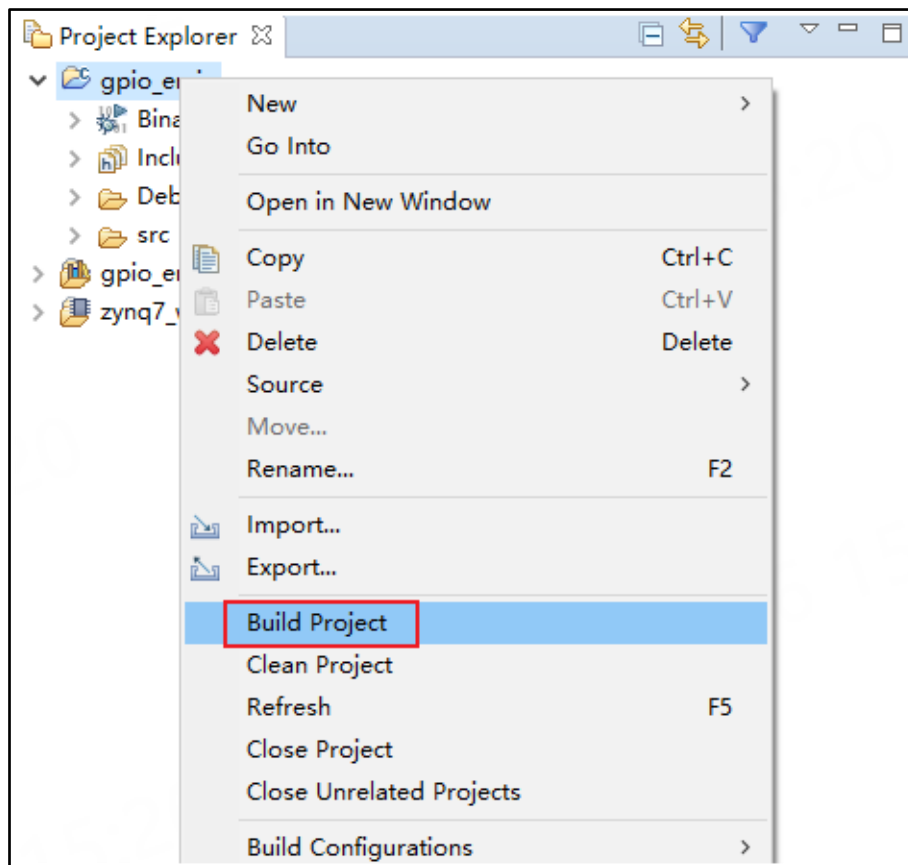


Figure 3-31 Manual compilation

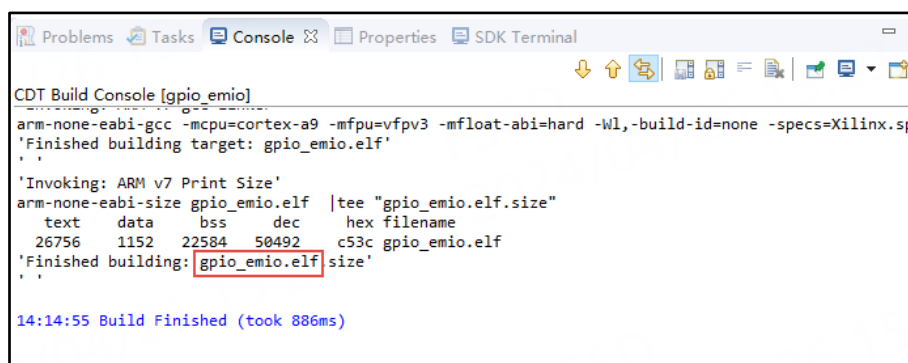


Figure 3-32 Compilation completed

3.3.4 Board verification

Development board connection: Connect the programmer to the JTAG interface on the Haoyue development board, and then use a USB cable to connect the USB serial port of the development board to the computer. Turn the DIP switches to "ON", finally connect the power cord, and turn on the power switch, as shown in the figure below:

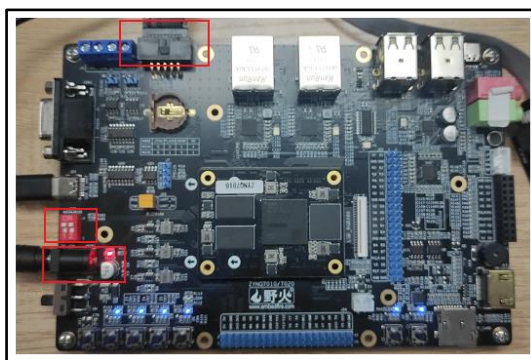


Figure 3-33 Program download connection diagram

Download the program for the development board, as shown in the figure below:

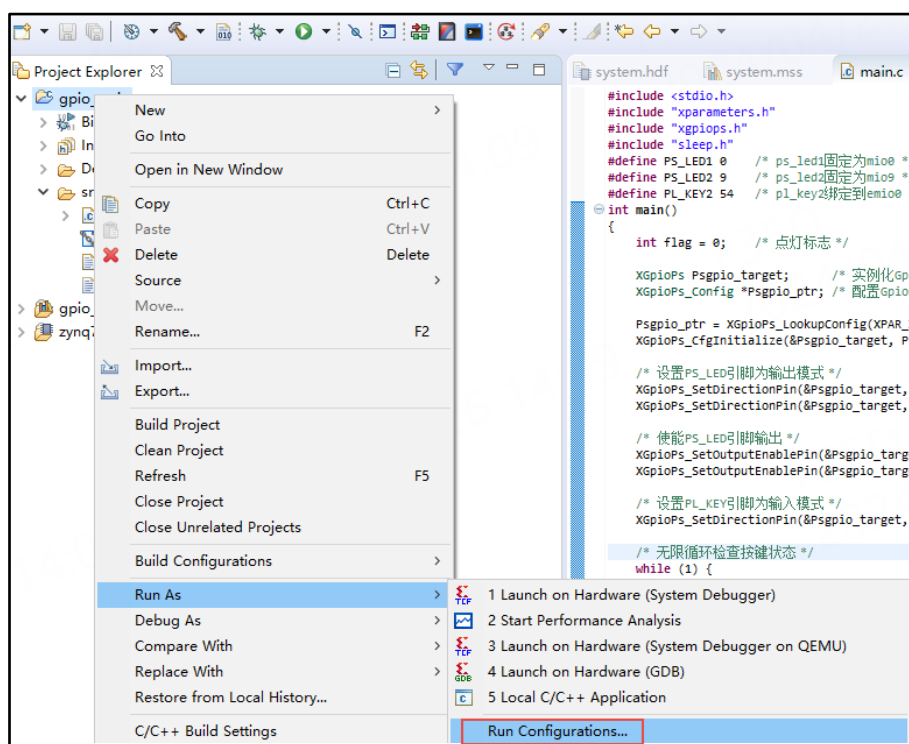


Figure 3-34 Select Run Configuration

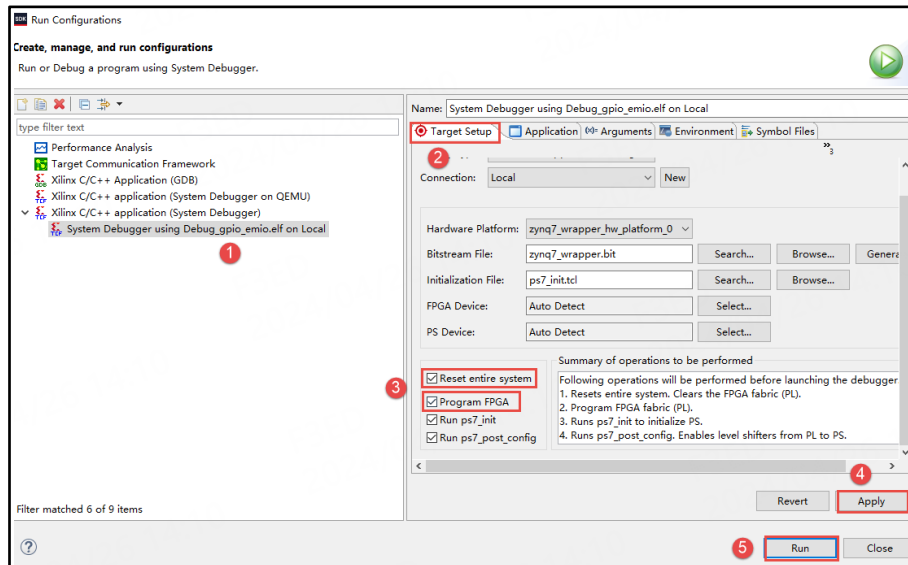


Figure 3-35 Run Configuration

1. If there is no System Debugger using Debug_gpio_emio.elf on local, please double-click Xilinx c/c++ application (System Debugger). 2. Select Target Setup. 3. Check the Reset entire system and Program FPGA checkboxes. Anyone who uses PL resources must select them. 4. Apply. 5. Run the download program.

After the program is run, the PS end PS_LED0 and PS_LED1 can be controlled to turn on and off alternately through the PL_KEY2 button on the development board.

3.4 Summary at the end of chapter

This chapter introduces how the PS side accesses the I/O resources of the PL side through EMIO. EMIO is routed to PL_KEY2 on the PL side on the development board, and the value of PL_KEY2 is obtained through query to control the alternating on and off of PS_LED0 and PS_LED1 on the PS side.

3.5 Development training

This chapter only uses the input function of the PL side I/O. Based on this chapter, you can try to add a PL LED and control the on and off by PL_KEY2 to deepen your understanding of the content of this chapter.

Chapter 4 MIO key interrupt experiment

4.1 Chapter Introduction

Interrupts are signals that notify the processor to process when faced with emergencies. Flexible use of interrupts can reduce system resource usage and is especially suitable for communication between the CPU and external devices.

In this chapter we will learn about interrupt signal control through MIO.

4.2 Theoretical study

Interrupt sources can be divided into hardware interrupts and software interrupts.

Hardware interrupt: generated by peripherals and introduced from outside through the INTR pin or NMI pin of the CPU, also called external interrupt. Including shieldable and unblockable. The CPU responds by inserting "INT N" between the two instructions. Can be divided into the following types:

Maskable Interrupts (IRQ). Interrupts that can be disabled or ignored by CPU instructions are called maskable interrupts, such as GPIO interrupts, timers, and UART interrupts. Non-Maskable Interrupts (NMI). Interrupts that cannot be disabled or ignored by CPU instructions are called non-maskable interrupts. Non-maskable interrupts are typically used when response time is critical or when interrupts should not be disabled during normal system operation. Hardware failure, power failure, or other urgent problem requiring immediate system attention. Inter-Processor Interrupts (IPI). Refers to the mechanism by which one processor sends an interrupt signal to another or multiple processors in a multi-processor system. This interrupt mechanism allows communication and cooperation between processors to perform operations such as synchronization, data transfer, etc. Software interrupt: An interrupt generated by an instruction under certain execution results.

Let's take a look at ZYNQ's interrupt block diagram:

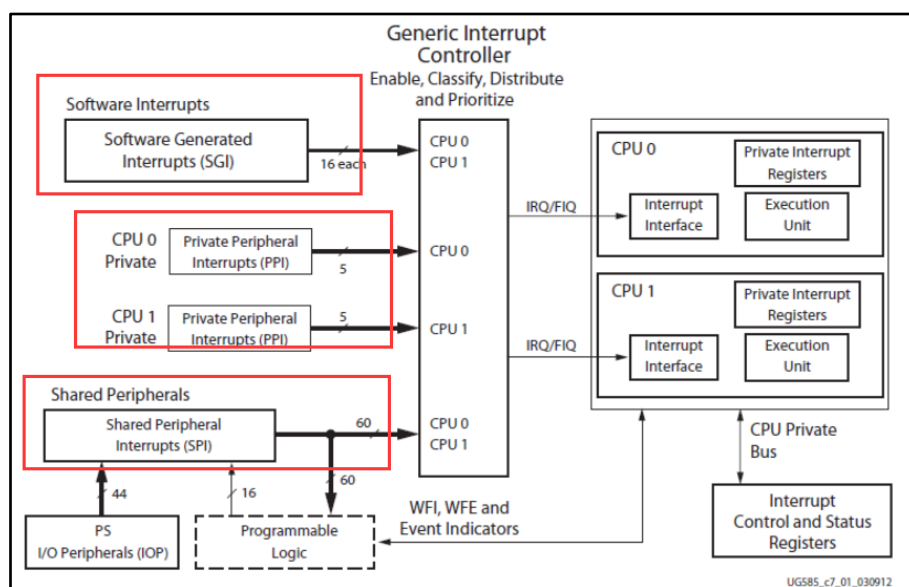


Figure 4-1 ZYNQ interrupt

It can be seen that there are three sources of CPU interrupts: private peripheral interrupts (PPI), software generated interrupts (SGI) and shared peripheral interrupts (SPI).

4.2.1 ZYNQ CPU Software Interrupt (SGI)

ZYNQ has two CPUs, each with its own 16 software interrupts. SGI is generated by writing to the ICDSGIR register, which is edge triggered (rising edge)

IRQ ID#	Name	SGI#	Type	Description
0	Software 0	0	Rising edge	A set of 16 interrupt sources that are private to each CPU that can be routed to up to 16 common interrupt destinations where each destination can be one or more CPUs.
1	Software 1	1	Rising edge	
~	...	~	...	
15	Software 15	15	Rising edge	

Figure 4-2 Software interrupt

4.2.2 ZYNQ CPU private peripheral interrupt

Each CPU has a set of PPIs (interrupt numbers 27-31), including global timers, private watchdog timers, private timers, and FIQ/IRQs from the PL.

IRQ ID#	Name	PPI#	Type	Description
26:16	Reserved	~	~	Reserved
27	Global Timer	0	Rising edge	Global timer
28	nFIQ	1	Active Low level (active High at PS-PL interface)	Fast interrupt signal from the PL: CPU0: IRQF2P[18] CPU1: IRQF2P[19]
29	CPU Private Timer	2	Rising edge	Interrupt from private CPU timer
30	AWDT[0, 1]	3	Rising edge	Private watchdog timer for each CPU
31	nIRQ	4	Active Low level (active High at PS-PL interface)	Interrupt signal from the PL: CPU0: IRQF2P[16] CPU1: IRQF2P[17]

Figure 4-3 Private peripheral interrupt

4.2.3 ZYNQ PS and PL shared interrupts

Generated by various I/O controllers and memory controllers on the PS and PL, these interrupt signals are routed to the appropriate CPU. 64 interrupt numbers: represented by spi_status_0[31:0], spi_status_1[31:0], of which the PL part has 16 shared interrupts and the PS has 44 (36, 93-95 are reserved).

Table 7-3: PS and PL Shared Peripheral Interrupts (SPI)

Source	Interrupt Name	IRQ ID#		Type	PS-PL Signal Name	I/O
		Decimal	Hex			
APU	CPU 1, 0 (L2, TLB, BTAC)	33:32	21h:20h	Rising edge	~	~
	L2 Cache	34	22h	High level	~	~
	OCM	35	23h	High level	~	~
Reserved	~	36	24h	~	~	~
PMU	PMU [1,0]	38, 37	25h:26h	High level	~	~
XADC	XADC	39	27h	High level	~	~
DVI	DVI	40	28h	High level	~	~
SWDT	SWDT	41	29h	High level	~	~
Timer	TTC 0	43:42	2B:2Ah	High level	~	~
Reserved	~	44	2Ch	~	~	~
DMAC	DMAC Abort	45	2Dh	High level	IRQP2F[28]	Output
	DMAC [3:0]	49:46	31h:2Eh	High level	IRQP2F[23:20]	Output
Memory	SMC	50	32h	High level	IRQP2F[19]	Output
	Quad SPI	51	33h	High level	IRQP2F[18]	Output
Debug	CTI	~	~	High level	IRQP2F[17]	Output

Figure 4-4 PS and PL shared interrupts

IOP	GPIO	52	34h	High level	IRQP2F[16]	Output
	USB 0	53	35h	High level	IRQP2F[15]	Output
	Ethernet 0	54	36h	High level	IRQP2F[14]	Output
	Ethernet 0 Wakeup	55	37h	High level	IRQP2F[13]	Output
	SDIO 0	56	38h	High level	IRQP2F[12]	Output
	I2C 0	57	39h	High level	IRQP2F[11]	Output
	SPI 0	58	3Ah	High level	IRQP2F[10]	Output
	UART 0	59	3Bh	High level	IRQP2F[9]	Output
	CAN 0	60	3Ch	High level	IRQP2F[8]	Output
PL	FPGA [7:0]	68:61	44h:3Dh	High level	IRQF2P[7:0]	Input
Timer	TTC 1	71:69	47h:45h	High level	~	~
DMAC	DMAC[7:4]	75:72	4Bh:48h	High level	IRQP2F[27:24]	Output
IOP	USB 1	76	4Ch	High level	IRQP2F[7]	Output
	Ethernet 1	77	4Dh	High level	IRQP2F[6]	Output
	Ethernet 1 Wakeup	78	4Eh	High level	IRQP2F[5]	Output
	SDIO 1	79	4Fh	High level	IRQP2F[4]	Output
	I2C 1	80	50h	High level	IRQP2F[3]	Output
	SPI 1	81	51h	High level	IRQP2F[2]	Output
	UART 1	82	52h	High level	IRQP2F[1]	Output
	CAN 1	83	53h	High level	IRQP2F[0]	Output
PL	FPGA [15:8]	91:84	5Bh:54h	High level	IRQF2P[15:8]	Input
SCU	Parity	92	5Ch	Rising edge	~	~
Reserved	~	95:93	5Fh:5Dh	~	~	~

Figure 4-5 PS_PL shared interrupt

All interrupt requests, whether PPI, SGI or SPI, are assigned a unique ID number for arbitration by the interrupt controller GIC. The interrupt dispatcher keeps a list of pending interrupts for each CPU, selects the highest priority interrupt, and sends it to the CPU interface. Interrupt priority is divided into 32 levels, 0 represents the highest priority, 0xF8 (decimal number 248) represents the lowest priority, and the step value between each priority level is 8.

If two interrupts with the same priority arrive at the same time, the interrupt with the smallest interrupt ID will be sent first. Each CPU has priority sequencing logic, so the selection of the highest priority interrupt is made independently by each CPU.

The interrupt dispatcher has a central list of interrupt, processor and active information and is responsible for triggering software interrupts to the CPU. To provide each processor with a separate copy, the SGI and PPI dispatcher registers are grouped. The hardware ensures that interrupts for multiple CPUs can only be accessed by one CPU at a time, so there is no need to use locks in the interrupt service routine. After sending the highest-priority pending interrupt to the CPU interface, the interrupt dispatcher will receive a message from the CPU that the interrupt has been acknowledged, so that it can change the status of the corresponding interrupt. Only the CPU that acknowledged the interrupt can end the interrupt.

4.2.4 Interrupt register GIC related registers

GIC-related registers, in the UG585 document, are all integrated under the mpcore domain, and are mainly divided into several types of registers: ICC and ICD:

Name	Register Description	Write Protection Lockdown
Interrupt Controller CPU (ICC)		
ICCICR	CPU interface control	Yes, except EnableNS
ICCPMR	Interrupt priority mask	~
ICCBPR	Binary point for interrupt priority	~
ICCIAR	Interrupt acknowledge	~
ICCEOIR	End of interrupt	~
ICCRPR	Running priority	~
ICCHPIR	Highest pending interrupt	~
ICCABPR	Aliased non-secure binary point	~

Figure 4-6 ICC

ICCICR: Enable/disable the CPU's first-level interrupt.

ICCPMR: The priority mask register of the CPU Interface. Only when an interrupt with a priority (judged by the lower 8 bits) higher than this value comes (compared with the specific interrupt priority) can the interrupt be processed.

ICCBPR: Register that controls the priority group. [2:0] of this field controls the 8-bit interrupt priority field which is split into a group priority field and a sub-priority field. The group priority field is used to determine interrupt preemption.

ICCIAR: Interrupt confirmation register, [12:10] is the CPU number that issued the interrupt request, [9:0] is the interrupt ID number.

ICCEOIR: Interrupt end register, [12:10] contains the CPUID value from the corresponding ICCIAR access after completing the SGI processing, [9:0] is the response value corresponding to the ICCIAR access.

ICCRPR: The [7:0] value is used to give the priority value of the highest priority interrupt currently active on the CPU.

ICCHPIR: Highest pending interrupt register, [12:10] means that if the PENDINTID field returns the ID of the SGI, then this field contains the CPUID value of the interrupt, which identifies the processor that generated the interrupt. [9:0] indicates the highest interrupt ID waiting for an interrupt.

ICCABPR: Insecure ICCBPR.

Interrupt Controller Distributor (ICD)		
ICDDCR	Secure/non-secure mode select	Yes
ICDICTR, ICDIIDR	Controller implementation	~
ICDISR [2:0]	Interrupt security	Yes
ICDISER [2:0], ICDICER [2:0]	Interrupt set-enable and clear-enable	Yes
ICDISPR [2:0], ICDICPR [2:0]	Interrupt set-pending and clear-pending	Yes
ICDABR [2:0]	Interrupt active	~
ICDIPR [23:0]	Interrupt priority, 8-bit fields	Yes
ICDIPTTR [23:0]	Interrupt processor targets, 8-bit fields	Yes
ICDICFR [5:0]	Interrupt sensitivity type, 2-bit field (level/edge, pos/neg)	Yes

Figure 4-7 ICD

ICDDCR: Allocate control register, safety or non-safety control to enable or disable interrupt configuration.

ICDICFR: Configuration register, used to configure the trigger mode, level trigger or edge trigger, there are 6 in total, each register has 32 bits, each two bits represents an interrupt, $32*6/2=96$ interrupt numbers, which can cover all interrupts.

ICDIPR: Interrupt priority register, 24 registers in total, ICDIPR 0 ~ ICDIPR 23, each 8 bits represents an interrupt priority configuration.

ICDIPTTR: CPU selection register, 24 registers, each 8 bits represents an interrupt number, a total of 96 0bxxxxxxx1 represents CPU interface 0, 0bxxxxxxx1x: represents CPU interface 1. Note that not all are configurable, some are fixed and CPU bound.

ICDICER: Interrupt mask register, a total of 3 registers, ICDICER 0-ICDICER 2, writing 1 to the corresponding bit means masking the corresponding interrupt number.

ICDISER: Interrupt enable register, a total of 3 registers, ICDISER 0-ICDISER 2, the function is opposite to ICDICER, writing 1 means enabling interrupt.

ICDISPR: Pull up the interrupt Pending register, there are 3 registers in total, ICDISPR 0-ICDISPR 2, set the pending register, write 1 to take effect.

ICDICPR: Clear the interrupt pending register, there are 3 registers in total, ICDICPR 0-ICDICPR 2, clear the pending register, write 1 to take effect.

PPI and SPI Status		
PPI_STATUS	PPI status: Corresponds to ICD*0 registers (security, enable, pending and active).	~
SPI_STATUS [2:1]	SPI status: Corresponds to ICD*(2:1) registers (security, enable, pending and active).	~
Software Generated Interrupts (SGI)		
ICDSGIR	Software-generated interrupts	~
Disable Write Accesses (SLCR register)		
APU_CTRL	CFGSDISABLE bit disables some write accesses	~

Figure 4-8 Other registers

The above is a brief introduction to ZYNQ's interrupt controller. For more details, you can refer to the Interrupts chapter in the official manual ug585.

The experiments in this chapter use GPIO interrupts, which are one of the shared interrupts. However, not every GPIO pin will have an interrupt number, but all GPIO pin interrupts in the 4 banks are combined together to use an interrupt number of 52

Sent to GIC uniformly. As for which pin caused the interrupt, you need to further judge through INT_STAT and INT_MASK.

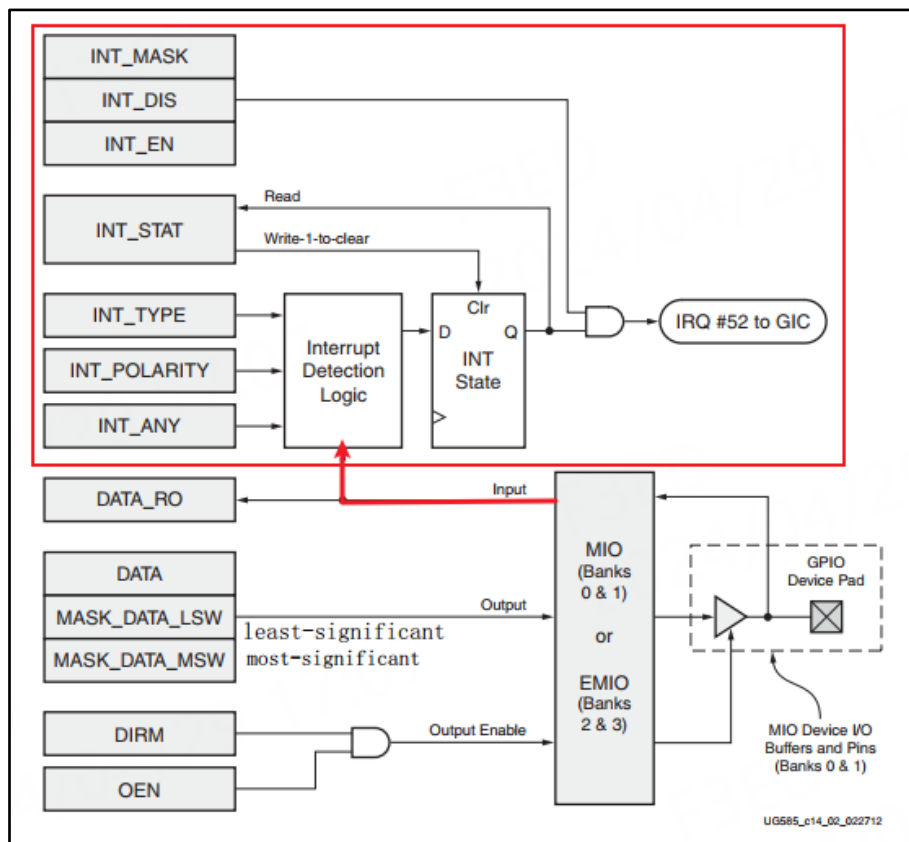


Figure 4-9 MIO internal structure

As shown in the figure above, when GPIO uses interrupts, 7 registers are involved:

INT_MASK: Read-only register used to display which bit of interrupt is masked and which bit of interrupt is enabled.

INT_EN: Writing 1 means turning on the interrupt enable of this bit.

INT_DIS: Writing 1 means to mask the interrupt of this bit.

INT_STAT: Write 1 to clear the interrupt flag. Writing 0 has no effect, the interrupt flag can be read, and the user can use it to determine whether an interrupt occurs. If 1, an interrupt is coming.

INT_TYPE: Set the trigger type, level trigger or edge trigger. 1 is edge trigger, 0 is level trigger.

INT_POLARITY: 1 means high level trigger (rising edge trigger), 0 means low level trigger (falling edge trigger).

INT_ON_ANY: When set to level trigger, this register is ignored. When set to edge trigger, setting this register to 1 can support rising edge and falling edge trigger interrupts.

4.3 Practical exercises

4.3.1 Experimental goals

In the previous chapter, we realized the button-controlled LED on and off through polling. This method has many shortcomings. Therefore, in this chapter, we will use the MIO interrupt method to realize the button-controlled LED on and off to achieve the effect of two LEDs alternately on and off.

4.3.2 Hardware resources

The physical diagram and schematic diagram of the buttons on the PS side and the LED on the PS side of the Hao

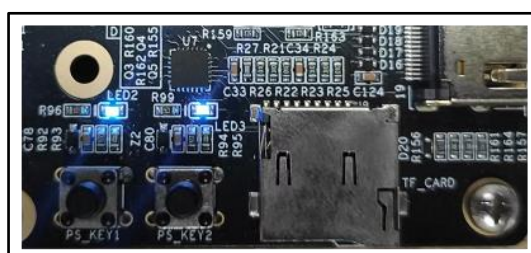


Figure 4-10 PS_KEY and PS_LED physical picture

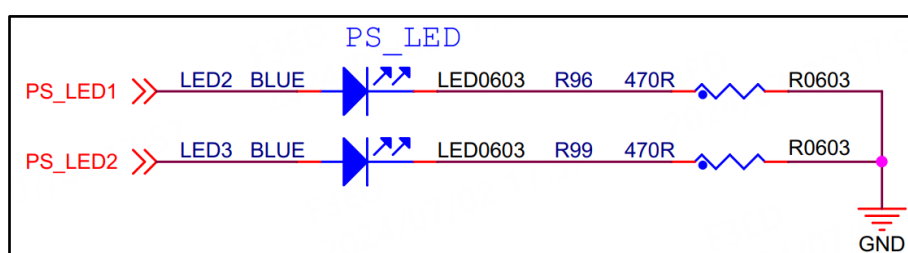


Figure 4-11 PS_LED schematic diagram

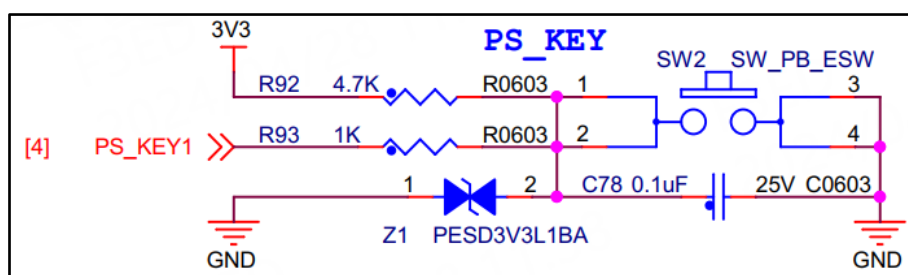


Figure 4-12 PS_KEY schematic diagram

4.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in the figure below.

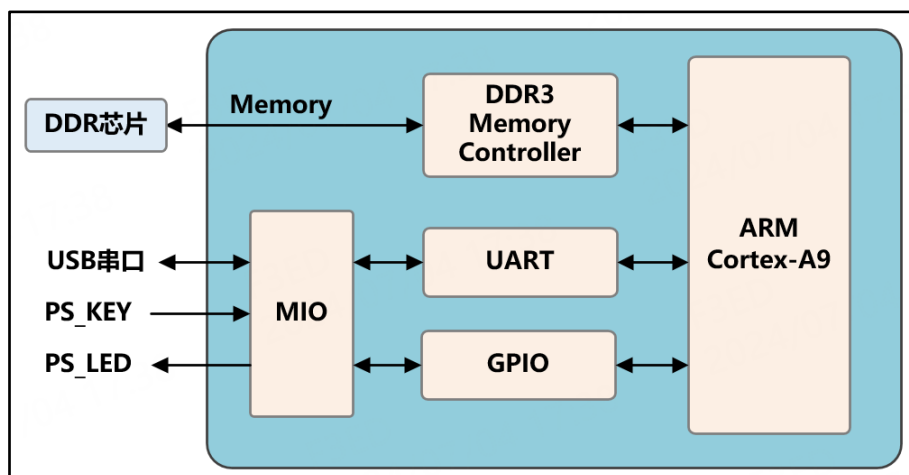


Figure 4-13 Overall block diagram of MIO key interrupt

2. Hardware design

Below we briefly introduce the overall steps for establishing a vivado project in ZYNQ.

Step1: Create a new vivado project

In the experiment in the "MIO Control LED" chapter, we have created the ZYNQ PS processing system IP. Here we modify it based on the original project.

We open the Vivado project "gpio_mio" in "MIO Control LED". After opening, select File > Project > Save As in the menu bar, as shown in the following figure:

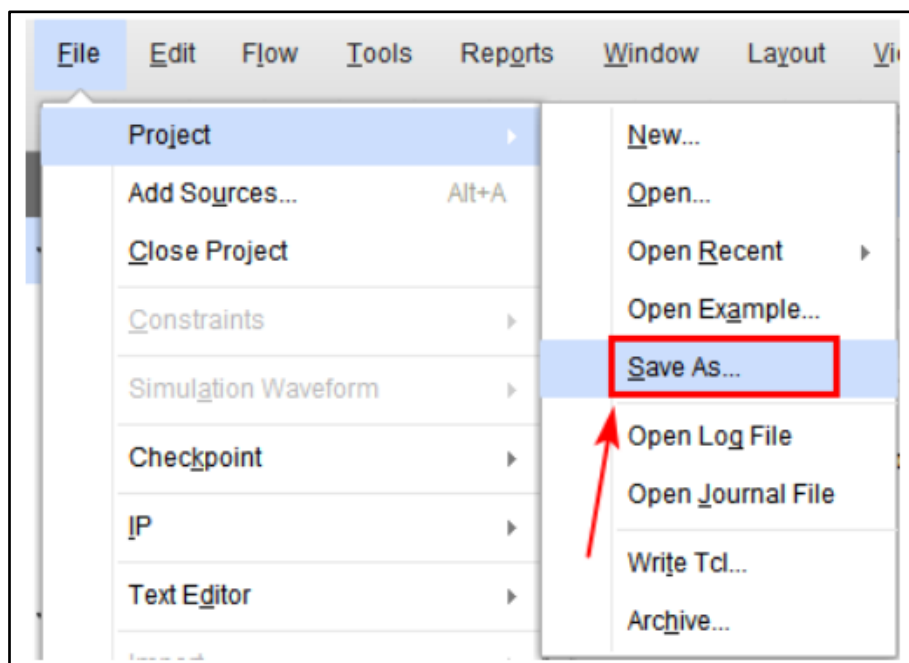


Figure 4-14 Save as new project

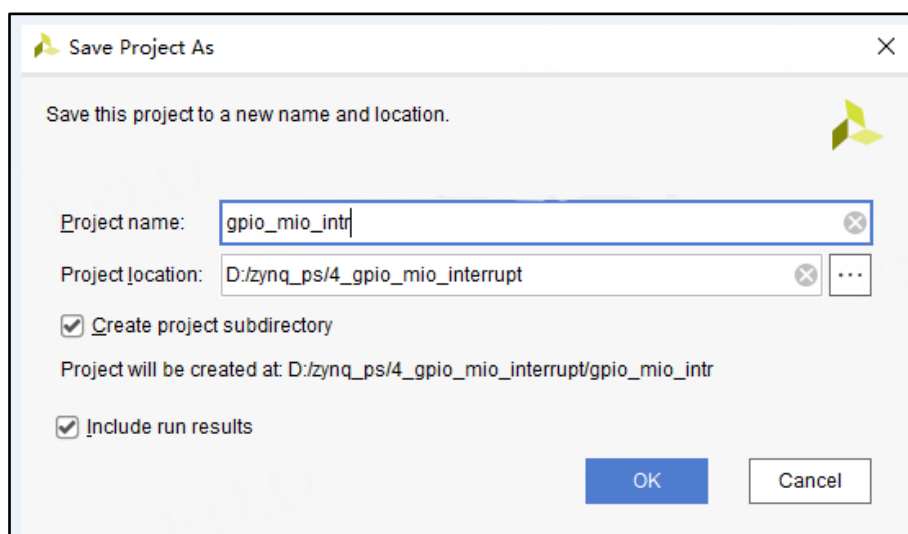


Figure 4-15 Save Project As

Then click OK to close the original project and open the newly created project. Then clear the contents of the gpio_mio_intr.sdk folder in the project directory, leaving only the zynq7_wrapper.hdf file. After completion, as shown in the figure below:



Figure 4-16 Only keep the zynq7_wrapper.hdf file

Step2: Use IP Integrator to create a Processing System

Since we created the project based on the "MIO Control LED" chapter, we do not need to modify it here and proceed to the next step.

Step3: Generate top-level HDL module

No modification is required.

Step4: Export SDK

Because the hardware has not been changed, we do not need to Export Hardware and can directly launch the Launch SDK. SDK environment.

In the **File** > **Launch SDK** -> **OK** in the single column to start SDK development

At this point, we have completed the hardware design part of the ZYNQ embedded system. Next, you need to carry out application development in the SDK software, which is the software design part.

3. Software design

In the hardware design part, we started the SDK development environment, and then we wrote the software code.

Step1: Create a new application project

Click File > New > Application Project in the upper right corner to create a new application project, and then click "Next", as shown in the figure below:

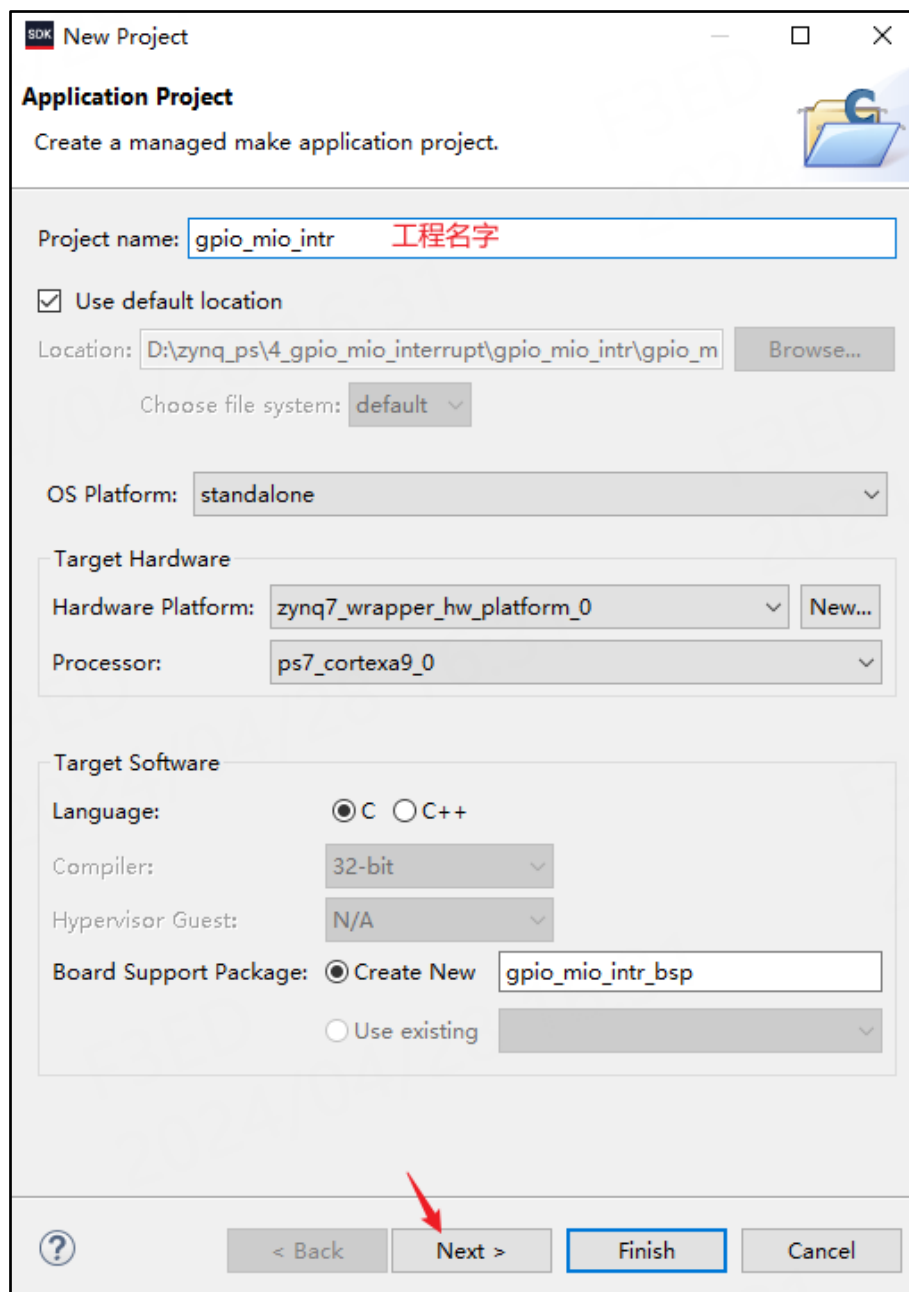


Figure 4-17 Fill in the project name

On the next tab, we choose to create a new blank template, and then click Finish to complete the creation of the application project.

Step2: Create a new source file

After completing the previous step, you can see that several new folders have been added to the left. After expanding, select src and right-click to create the source file main.c.

Step3: Write code

After the creation is completed, we double-click main.c and write the following code.

Code List 4-1 MIO key interrupt reference code (main.c)

```
1 #include "xparameters.h" 2 #include "xgpiops.h" 3 #include "xscugic.h" 4 #include "xil_exception.h" 5 #include "xplatform_info.h" 6 #include <xil_printf.h> 7 #include "sleep.h" 8 #include <stdio.h> 9 10 #define PS_LED1 0 /* PS_LED1 is fixed to mio0 */ 11 #define PS_LED2 9 /* ps_led2 is fixed to mio9 */ 12 #define PS_KEY1 10 /* PS_KEY1 is fixed to mio10 */ 13 14 /* Define ID for convenient operation */ 15 #define GPIO_ID XPAR_XGPIOPS_0_DEVICE_ID 16 #define INTC_ID XPAR_SCUGIC_SINGLE_DEVICE_ID 17 #define GPIO_INT_ID *gpio, u16 GpioIntrId)
```

```
21 ; 22 23 u8 intr_flag; /* Interrupt flag */ 24 XGpioPs_Config *Psgpio_ptr; 32 /* Find GPIOPS configuration information */ 33 Psgpio_ptr = XGpioPs_LookupConfig(GPIO_ID); 34 if (Psgpio_ptr == NULL) { 35 return GPIOPS */ 39 43 /* Set the PS_LED pin to output mode */ 44 XGpioPs_SetDirectionPin(&Psgpio_target, PS_LED1, 1); 45 XGpioPs_SetDirectionPin(&Psgpio_target, PS_LED2, 1); 46 PS_LED1, 1); 47 XGpioPs_SetOutputEnablePin(&Psgpio_target, PS_LED2, 1); 48 /* Initialization pin default state: high level */ 49 XGpioPs_WritePin(&Psgpio_target, PS_LED2, 1); 51 52 /* Initialize interrupt processing function */ 53 status = init_interrupt(&Cur_int, &Psgpio_target, GPIO_INT_ID); 54 if (status != XST_SUCCESS) { 55 xil_printf("Interrupt system init failed! \r\n"); 56 return
```

```
63 if (XGpioPs_ReadPin(&Psgpio_target, PS_KEY1) == 0) { 64 key_flag = ~key_flag; 65 XGpioPs_WritePin(
&Psgpio_target, PS_LED1, key_flag); 66 XGpioPs_WritePin(&Psgpio_target, PS_LED2, ~key_flag); 67 } 68 i
ntr_flag = FALSE; 69 } 70 } 71 return XST_SUCCESS; 72 } 73 /* Interrupt handling function */ 74 static void
intr_handler(void *callback_ref) 75 { 76 XGpioPs *gpio = (XGpioPs *) callback_ref; 77 /* Check pin status */
78 if (XGpioPs_IntrGetStatusPin(gpio, PS_KEY1) 79 && XGpioPs_IntrGetEnabledPin(gpio, PS_KEY1)) { 80
intr_flag = TRUE; 81 /* Clear key interrupt */ 82 XGpioPs_IntrClearPin(&Psgpio_target, PS_KEY1); 83 } 84 }
85 86 /* Interrupt initialization function */ 87 int init_interrupt(XScuGic *gic_ins_ptr, XGpioPs *gpio, u16 Gpi
oIntrId) 88 { 89 int status; 90 XScuGic_Config *Int_config; 91 92 /* Find interrupt configuration information */
93 Int_config = 97 98 /* Initialization interrupt */ 99 status = XScuGic_CfgInitialize(gic_ins_ptr, Int_config, 10
0 Int_config->CpuBaseAddress); 101 if (status != /* Interrupt exception enable */ 106 Xil_ExceptionRegisterHa
ndler( 107 XIL_EXCEPTION_ID_INT, 108 (Xil_ExceptionHandler) XScuGic_InterruptHandler, 109 gic_ins_p
tr); 110 111 = 124 XScuGic_Enable(gic_ins_ptr, GpioIntrId); 125 126 /* Set the interrupt type to falling edge tri
gger */ 127
```

```
128             XGPIOPS_IRQ_TYPE_EDGE_FALLING);
129
130     /* 正式开启中断*/
131     XGpioPs_IntrEnablePin(gpio, PS_KEY1);
132
133     return XST_SUCCESS;
134 }
```

In terms of introducing header files, compared to the previous chapter about button lighting, we have introduced the header file `xscugic.h`, which contains many interrupt-related functions.

After that, our GPIO initialization is almost the same as in the previous chapter. In order to use interrupts, we must add two new functions. One is to initialize the interrupt controller function and the interrupt service function. Lines 87-134 of the above code are to initialize our interrupt function. Next, we will focus on this function.

First, we define the configuration information `XScuGic_Config` of the interrupt controller, then use the `XScuGic_LookupConfig` function to find the configuration information, initialize it, and then enable interrupt exceptions. After that, we need to associate the interrupt service function. Use `XScuGic_Connect` to set the interrupt service function to be associated. After that, we need to set the interrupt type and use the `XScuGic_SetIntrTypePin` function. We can check the definition of this function to know what types there are. We can jump by clicking this function with the left mouse button of `ctrl +`. As shown below:

```
void XGpioPs_SetIntrTypePin(XGpioPs *InstancePtr, u32 Pin, u8 IrqType)
{
    u32 IntrTypeReg;
    u32 IntrPolReg;
    u32 IntrOnAnyReg;
    u8 Bank;
    u8 PinNumber;

    Xil_AssertVoid(InstancePtr != NULL);
    Xil_AssertVoid(InstancePtr->IsReady == XIL_COMPONENT_IS_READY);
    Xil_AssertVoid(Pin < InstancePtr->MaxPinNum);
    Xil_AssertVoid(IrqType <= XGPIOPS_IRQ_TYPE_LEVEL_LOW);

    /* Get the Bank number and Pin number within the bank. */
    XGpioPs_GetBankPin((u8)Pin, &Bank, &PinNumber);

    IntrTypeReg = XGpioPs_ReadReg(InstancePtr->GpioConfig.BaseAddr,
                                  ((u32)(Bank) * XGPIOPS_REG_MASK_OFFSET) +
                                  XGPIOPS_INTTYPE_OFFSET);

    IntrPolReg = XGpioPs_ReadReg(InstancePtr->GpioConfig.BaseAddr,
                                  ((u32)(Bank) * XGPIOPS_REG_MASK_OFFSET) +
                                  XGPIOPS_INTPOL_OFFSET);
}
```

Figure 4-18 Jump to function definition

When we search down, we can see that there are a total of five types, as shown in Figure 4-19, which are rising edge, falling edge, double edge, high level and low level. According to our code, we choose the falling edge interrupt.

```

switch (IrqType) {
    case XGPIOPS_IRQ_TYPE_EDGE_RISING:
        IntrTypeReg |= ((u32)1 << (u32)PinNumber);
        IntrPolReg |= ((u32)1 << (u32)PinNumber);
        IntrOnAnyReg &= ~((u32)1 << (u32)PinNumber);
        break;
    case XGPIOPS_IRQ_TYPE_EDGE_FALLING:
        IntrTypeReg |= ((u32)1 << (u32)PinNumber);
        IntrPolReg &= ~((u32)1 << (u32)PinNumber);
        IntrOnAnyReg &= ~((u32)1 << (u32)PinNumber);
        break;
    case XGPIOPS_IRQ_TYPE_EDGE_BOTH:
        IntrTypeReg |= ((u32)1 << (u32)PinNumber);
        IntrOnAnyReg |= ((u32)1 << (u32)PinNumber);
        break;
    case XGPIOPS_IRQ_TYPE_LEVEL_HIGH:
        IntrTypeReg &= ~((u32)1 << (u32)PinNumber);
        IntrPolReg |= ((u32)1 << (u32)PinNumber);
        break;
    case XGPIOPS_IRQ_TYPE_LEVEL_LOW:
        IntrTypeReg &= ~((u32)1 << (u32)PinNumber);
        IntrPolReg &= ~((u32)1 << (u32)PinNumber);
        break;
    default:
        /**< Default statement is added for MISRA C compliance. */
        break;
}

```

Figure 4-19 Interrupt type

After that, we will write the content of the interrupt service function, that is, what operation the CPU will perform each time the interrupt is triggered. When we press the button, the interrupt is triggered, enter the function in line 74 of the code, and detect the interrupt status of the pin at this time. If it has been triggered, copy the `intr_flag` to `TRUE` and clear the interrupt flag. At the same time, our main function will read the status of the button at this time, and then perform software delay debounce (on the Haoyue development board, we use capacitors to debounce the button) Hardware debounce, the debounce effect is still very ideal, software debounce is not required), read the value of the button pin after the delay, if it is 0, the button is pressed, and the LED light status is reversed.

Step4: Compile the project

After writing, we use `ctrl + s` to automatically compile. If not, use `Project->Build Project` above to manually compile the project, as shown in the following figure:

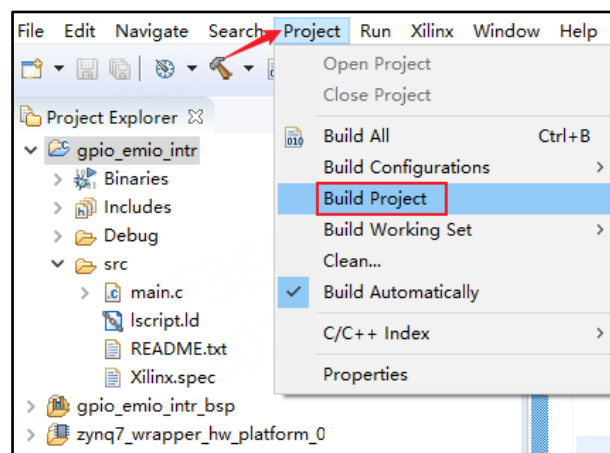


Figure 4-20 Compile project

4.3.4 Board verification

As shown in the figure below, the development board is connected. After the lines are connected correctly, turn on

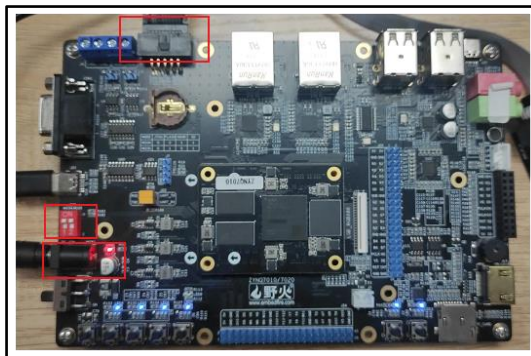


Figure 4-21 Program download connection diagram

Download the program to the development board. Right-click `gpio_mio_intr` in the project and select Run As > Launch on Hardware (system Debugger).

After the program is run, the PS_LED0 and PS_LED1 on the PS side can be controlled to turn on and off alternately through the PS_KEY1 button on the development board.

4.4 End of chapter summary

This chapter introduces ZYNQ's interrupt controller and GPIO interrupts. Using the interrupt method to read the status of the button solves the shortcomings of using the polling method in the previous chapter.

4.5 Development training

Through studying this chapter, I believe you have mastered the concepts related to interrupts and how to use code to implement the key interrupt function. However, in the experiment of this chapter, `XScuGic_Connect` was used to register a custom interrupt processing function. In fact, for GPIO, Xilinx provides a unified interrupt processing function `XGpioPs_IntrHandler`. This function identifies which pin the interrupt is caused by, and clears the corresponding interrupt flag. For the final callback, we use the callback function registered with `XGpioPs_SetCallbackHandler`. You can also try this method.

5.3.2 Hardware resources

The same as the experiment in the chapter "EMIO Controlling LED".

5.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in the figure below.

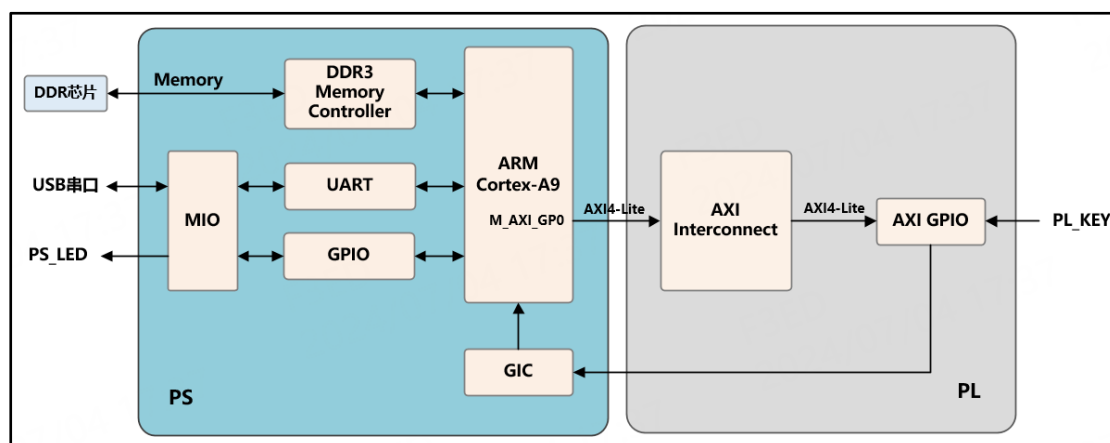


Figure 5-2 AXI GPIO interrupt overall block diagram

2. Hardware design

Below we briefly introduce the overall steps for establishing a vivado project in ZYNQ.

Step1: Create a new vivado project

In the experiment in the "MIO Control LED" chapter, we have created the ZYNQ PS processing system IP. Here we modify it based on the original project.

We open the Vivado project "gpio_mio" in "MIO Control LED", and after opening it, select File-> Project -> Save As in the menu bar, as shown in the following figure:

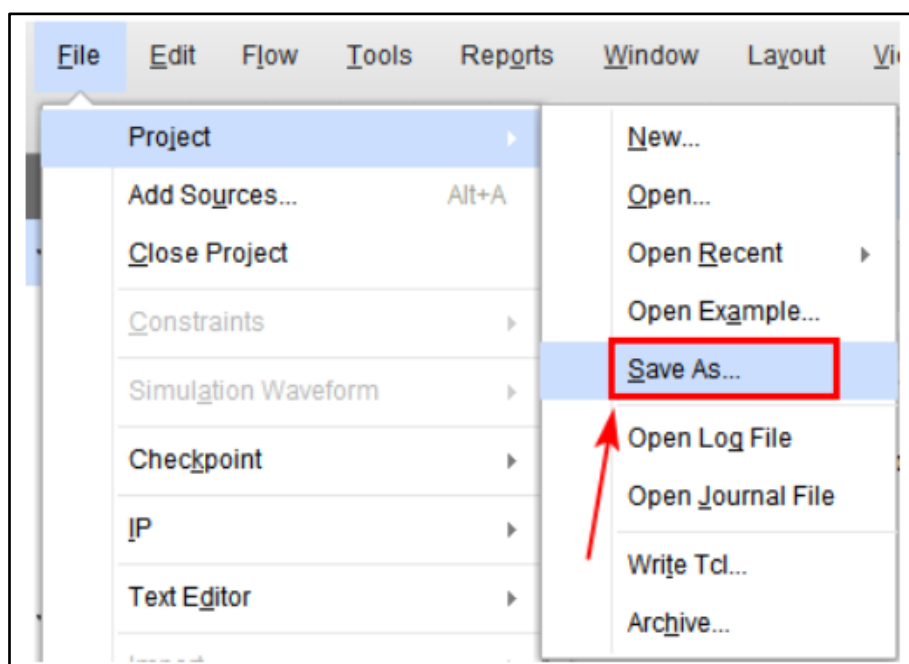


Figure 5-3 Save as new project

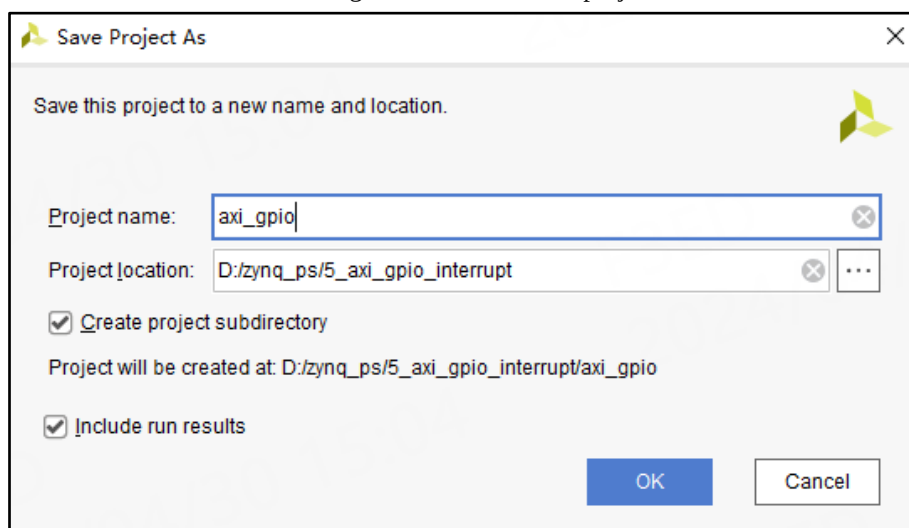


Figure 5-4 Save Project As

Then click OK to close the original project and open the newly created project.

Step2: Use IP Integrator to create a Processing System

We created the project based on the "MIO Control LED" chapter, and here we modify it on the ZYNQ system IP.

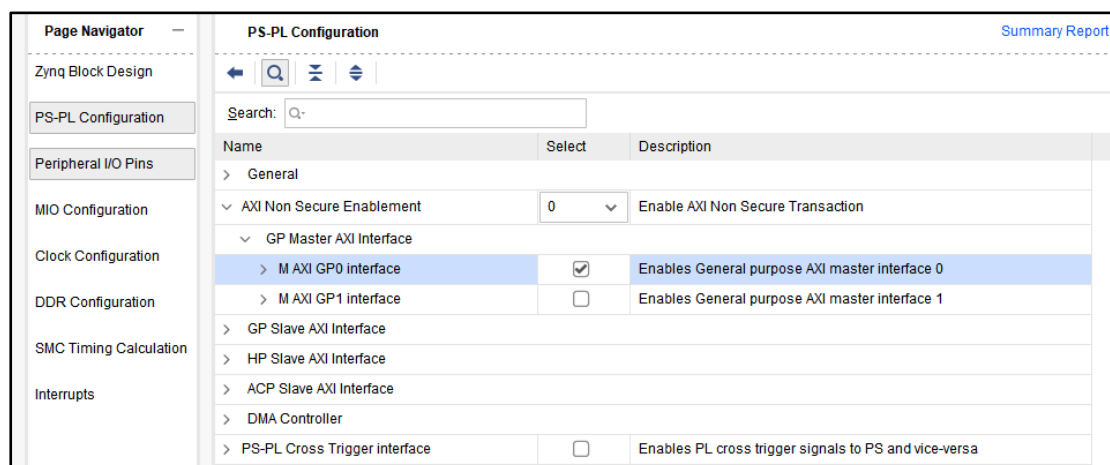


Figure 5-5 Turn on M AXI GP0

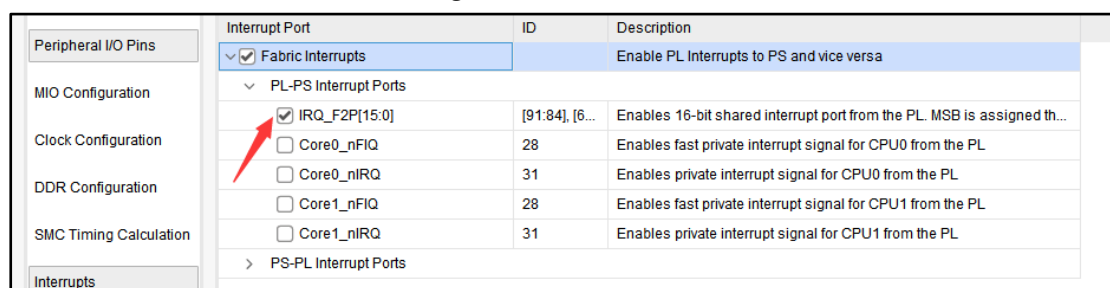


Figure 5-6 Enable PS-PL interrupt

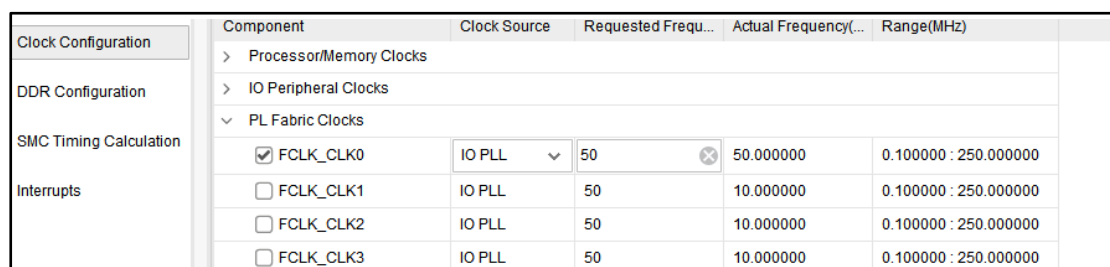


Figure 5-7 Turn on the clock provided to the PL side

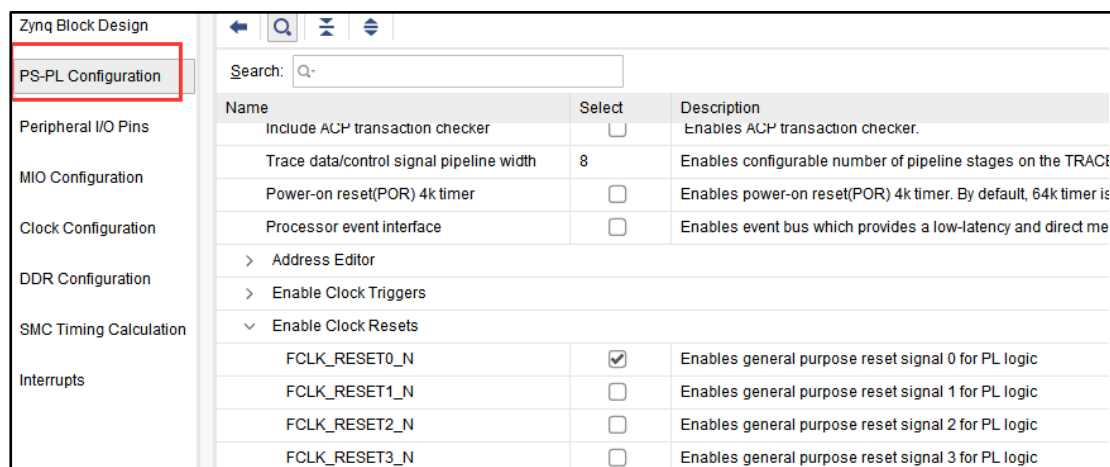


Figure 5-8 Turn on the reset signal of FCLK

Finally, click "OK" in the lower right corner, and the ZYNQ processing system for this experiment is configured.

After the ZYNQ7 PS configuration is completed, its interface is shown in Figure 5-9. M_AXI_GP0 is the General Purpose AXI interface, and M represents the PS as the master. The M_AXI_GP0_ACLK on the left is the global clock signal of this interface. All signals of the M_AXI_GP0 interface are sampled on the rising edge of this global clock. FCLK_CLK0 is generated by the IO PLL in the PS and serves as the clock signal output by the PS to provide a clock source for the peripherals in the PL. The frequency range is 0.1~250MHz. In the ZYNQ7 PS configuration, the default is 50MHz; FCLK_RESET0_N is the FPGA hardware reset port provided by the PS to the PL, and is active at low level; IRQ_F2P[0:0] It is the interrupt signal output from PL terminal to PS terminal (GIC).

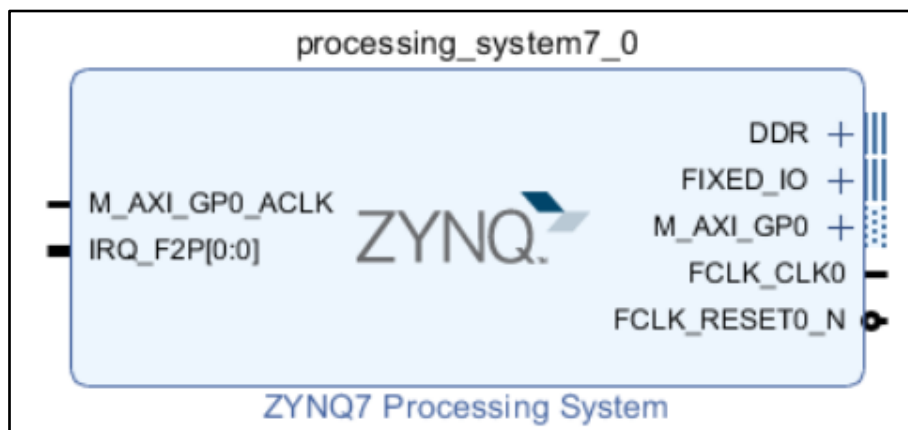


Figure 5-9 ZYNQ processing system after configuration

Next we add the AXI_GPIO module.

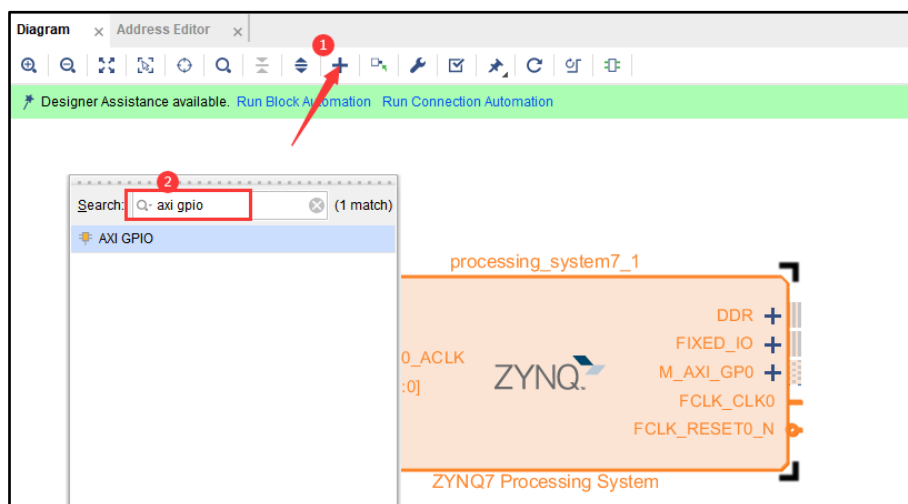


Figure 5-10 Adding AXI GPIO module

Double-click the added AXI GPIO module and modify it as shown in Figure 5-11:

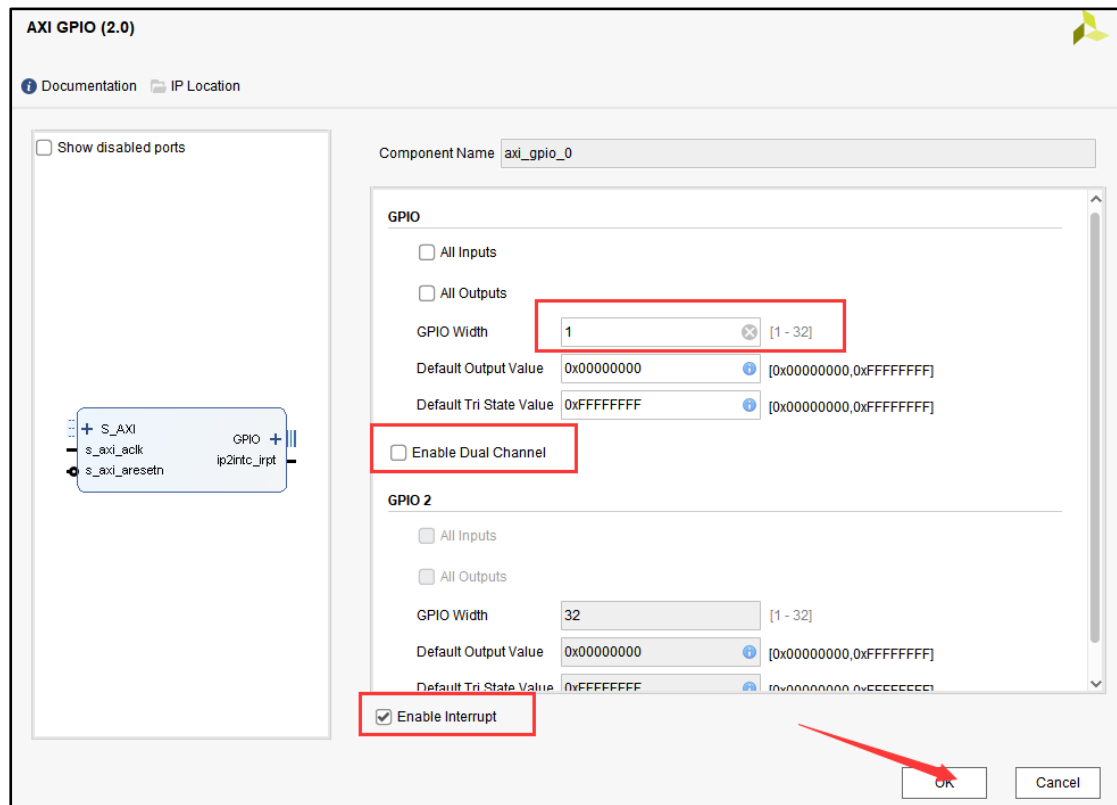


Figure 5-11 Modify AXI GPIO module parameters

Here we only use one PL-side button, so the GPIO bit width is 1, and there is no need to enable dual channels. Since this experiment requires PL to initiate an interrupt to GIC, "Enable Interrupt" needs to be checked.

Then we click Run Connection Automation.

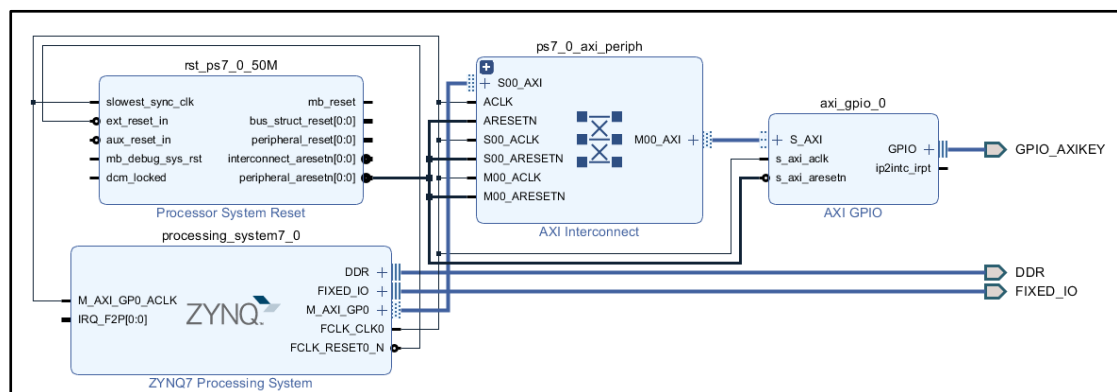


Figure 5-12 Automatic connection

It can be seen that the interrupt interface `IRQ_F2P` of the ZYNQ7 PS module is not automatically connected, and we need to connect it manually: place the mouse on the interface, and after the mouse turns into a pencil, press and hold the left button to connect it to "RQ_F2P".

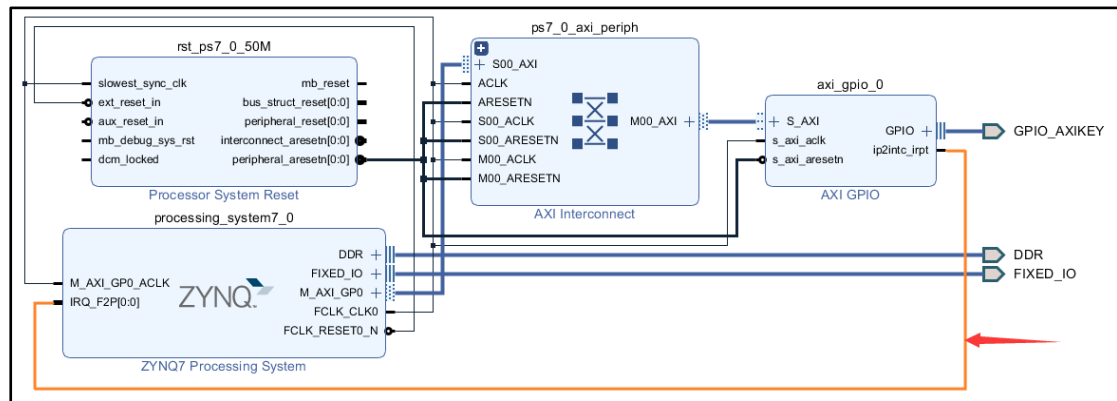


Figure 5-13 Manual connection interruption

Step3: Generate top-level HDL module

We have modified the block design. Here we need to verify whether the design is correct, save the design, and re-execute "Generate Output Products". If you are not sure, please refer to the "MIO Control LED" chapter.

Step4: Generate bit stream file

Here we use the pins on the PL side and need to perform pin constraints. Find RTL ANALYSIS in the Flow Navigator navigation bar on the left and click "Open Elaborated Design" in this option. Then click Layout in the menu bar and select I/O Planning in the drop-down list to open the I/O Ports window. We will assign pins to the interface AXI_GPIO_KEY derived from AXI GPIO in the I/O Ports window and press Ctrl + S to save.

GPIO_AXIKEY_12642 (1)	INOUT			✓	35	LVC MOS33*	3.300
GPIO_AXIKEY_tri_io ...	INOUT			✓	35	LVC MOS33*	3.300
GPIO_AXIKEY_tri_...	INOUT	G20	✓	✓	35	LVC MOS33*	3.300

Figure 5-14 Pin binding

Then find PROGRAM AND DEBUG in the Flow Navigator navigation bar on the left, click "Generate Bitstream" in this option, click "YES" and "OK" in sequence according to the prompts in the dialogue that pops up later, and then wait for the Bitstream file to be generated.

Step5: Export SDK

We export Hardware and check "Include bitstream" in the pop-up dialog box. Then Launch SDK starts the SDK environment.

Select File > Launch SDK in the menu bar to start the SDK development environment.

At this point, we have completed the hardware design part of the ZYNQ embedded system. Next, you need to carry out application development in the SDK software, which is the software design part.

3. Software design

In the hardware design part, we started the SDK development environment, then created a new application project

At this point, I believe that all readers already know how to create a new application project. I will reduce the length here and no longer repeat the process of creating a new application project, and directly enter the code writing and analysis.

Code List 5-1 AXI GPIO key interrupt reference code (main.c)

```

1 #include "stdio.h" 2 #include "xgpio.h" 3 #include "sleep.h" 4 #include "xgpiops.h" 5
#include "xscugic.h" 6 #include "xil_printf.h" 7 #include "xparameters.h" 8 #include "xi
l_exception.h" 9 10 /* ID redefinition */ 11 #define SCUGIC_ID XPAR_SCUGIC_0_D
EVICE_ID 12 #define GPIOPS_ID XPAR_XGPIOPS_0_DEVICE_ID 13 #define AXI
_GPIO_ID #define PS_LED1 0 // PS_LED1 is fixed to mio0 17 #define PS_LED2 9 // p
s_led2 is fixed to mio9 18 #define GPIO_CHANNEL1 1 // Use GPIO channel 1 19 20 v
oid axi_gpio_handler(void *CallbackRef); // Interrupt service function 21 22 XScuGic C
ur_int; // Interrupt controller Driver instance

23 XScuGic_Config * Scugiconfig_ptr; //Interrupt configuration pointer
24 XGpioPs Psgpio_target; // Instantiate gpio
25 XGpioPs_Config * Psgpio_ptr; // PS-side GPIO configuration information 26 XScuGic_Lookup
Config(SCUGIC_ID); 34 XGpioPs_CfgInitialize(&Psgpio_target, Psgpio_ptr, Psgpio_ptr-> 38 BaseAddr); 39
XGpio_Initialize(&axi_gpio, AXI_GPIO_ID); 40 41 // Polarity set to output 42 XGpioPs_SetDirectionPin(&P
sgpio_target, PS_LED1, 1); 43 XGpioPs_SetDirectionPin(&Psgpio_target, PS_LED2, 1); 44 // Output enable
45 XGpioPs_SetOutputEnablePin(&Psgpio_target, PS_LED2, 1); 47 //Output default level 48 XGpioPs_Write
Pin(&Psgpio_target, PS_LED1, LED_flage); 49 50 51 // Set AXI GPIO input mode 52 XGpio_SetDataDirecti
on(&axi_gpio, GPIO_CHANNEL1, 1); 53 // Enable channel interrupt 54 XGpio_InterruptEnable(&axi_gpio,
XGPIO_IR_CH1_MASK); 55 // Enable AXI GPIO interrupt 56

XGpio_InterruptGlobalEnable(&axi_gpio);
57 58 // Configure interrupt priority and trigger mode 59 XScuGic_SetPriorityTriggerType(&Cur_int, FA
BRIC_GPIO_ID, 0xA0, 60 0x01); 61 // Set interrupt service function

```

```

62 XScuGic_Connect(&Cur_int, FABRIC_GPIO_ID, axi_gpio_handler, 63 &axi_gpio); 64 // Enable
AXI GPIO interrupt 65 XScuGic_Enable(&Cur_int, FABRIC_GPIO_ID); 66 67 /* Set interrupt excep
tion handling */ 68 Xil_ExceptionInit(); 69 Xil_ExceptionRegisterHandler( 70 XIL_EXCEPTION_ID
_INT, 71 (Xil_ExceptionHandler)XScuGic_IRQHandler, 72 &Cur_int); 73 Interrupt service handl
er function */ 80 void axi_gpio_handler(void *CallbackRef) 81 { 82 int KEY_flag = 1; 83 XGpio *G
pioPtr = (XGpio *)CallbackRef; 84 85 // Turn off AXI GPIO interrupt 86 XGPIO_IR_CH1_MASK); 8
7 // Read key data 88 KEY_flag = XGpio_DiscreteRead(GpioPtr, GPIO_CHANNEL1); 89 if (KEY_f
lage == 0) { 90 LED_flag = ~LED_flag; XGpioPs_WritePin(&Psgpio_target, PS_LED1, LED_flag
e); 92 XGPIO_IR_CH1_MASK); // Clear interrupt 96 XGpio_InterruptEnable(GpioPtr, XGPIO_IR_C
H1_MASK); // Enable interrupt 97 }

```

Compared with before, we have introduced the header file `xgpio.h`, which contains some configuration functions of AXI_GPIO. For example, line 39 of the code `XGpio_Initialize` initializes AXI_GPIO. We configure the GPIO in input mode and turn on its interrupt enable, channel enable and global interrupt in lines 52-56 of code.

After that, it is time to configure the related functions of the interrupt. In lines 59-73 of code we begin to configure the GIC. Each interrupt has a specific ID, which can be found in `xparameters.h` as shown below.

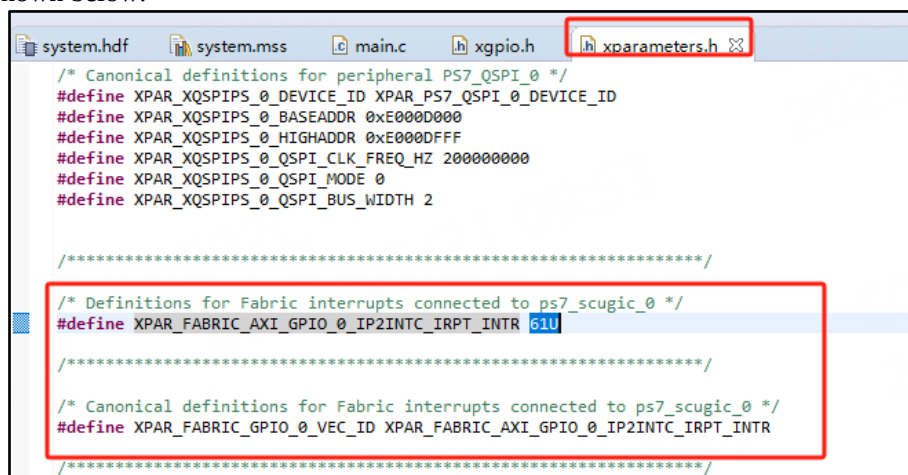


Figure 5-15 Interrupt ID

Here you can see that the interrupt ID number is 61. In fact, you can also see it when configuring interrupts in Block Design, as shown below:

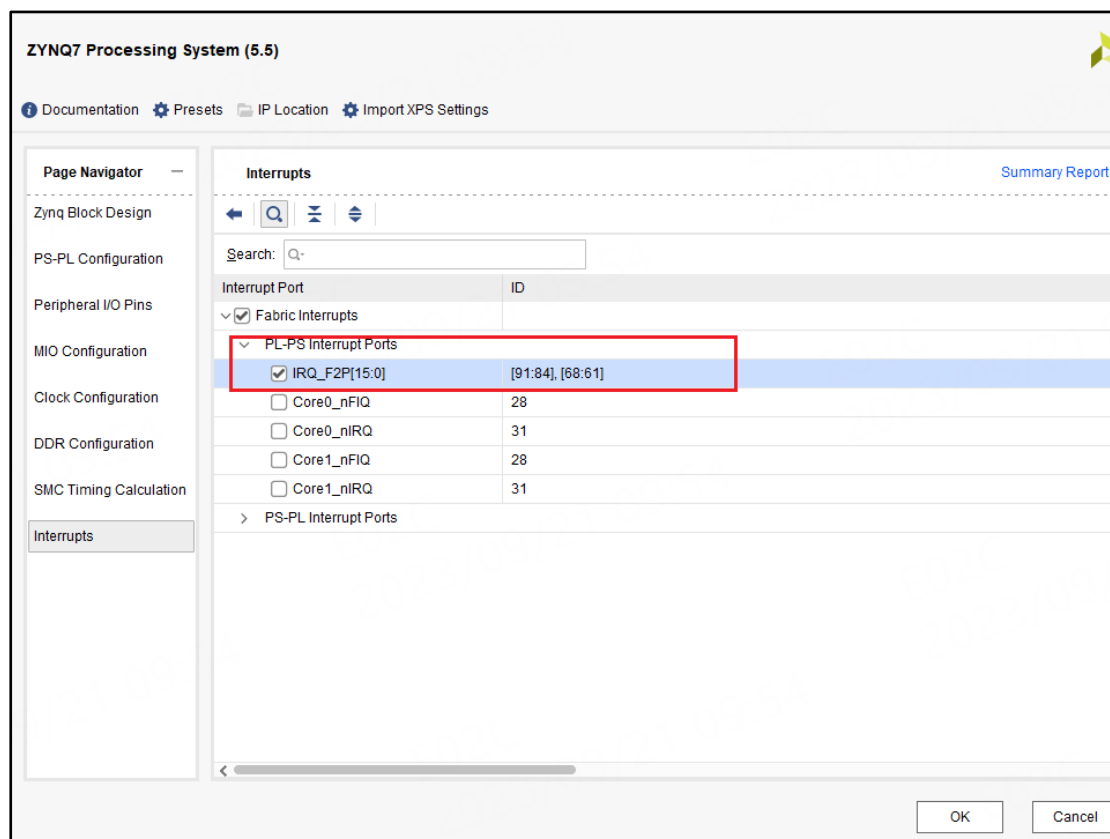


Figure 5-16 Interrupt number

You can see that the interrupt numbers range from 61 to 68 and 84 to 91, so our interrupt number here is 61. Next, you need to configure the interrupt priority and interrupt type. There are 32 priorities in total, and the step value is 8, which is 0, 8, 16 and so on. Interrupt trigger types are divided into high-level sensitive types and rising edge sensitive types. Here we configure a high level trigger. When our AXI_GPIO detects a change in the input signal, there will be an active high level interrupt request. So we set up a high level trigger.

Next, in line 62 of code, we associate the interrupt service function, and then in line 65 of code, we use XScuGic_Enable to enable the interrupt source corresponding to the ID, and then turn on interrupt exception handling. At this point, the interrupt configuration is also good. Next write the interrupt service function.

When our CPU detects that an interrupt is triggered, it will enter the interrupt service function. In lines 80-96 of the code, after entering the service function, we first turn off the interrupt enable of AXI_GPIO to avoid repeated triggering. Read the value of the AXI_GPIO channel using

At this point, the code is basically written, and then the compilation project begins. After compilation is completed, the corresponding elf file is generated. Next, board verification.

5.3.4 Board verification

As shown in the figure below, the development board is connected. After the lines are connected correctly, turn on

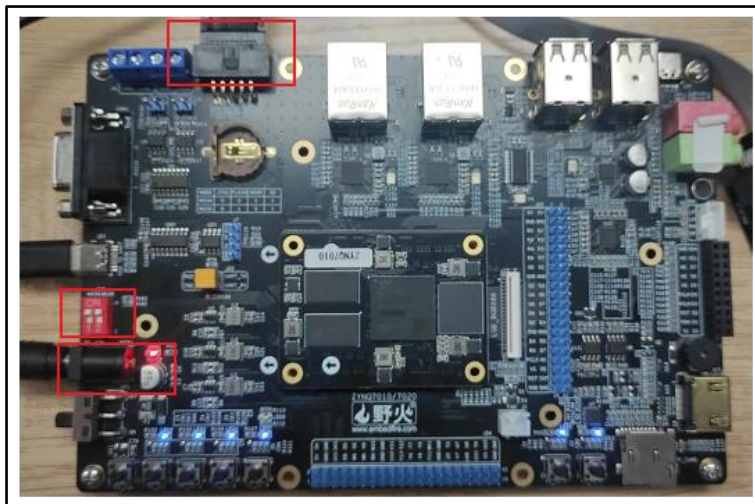


Figure 5-17 Program download connection diagram

As shown in Figure 5-18, download the program for the development board.

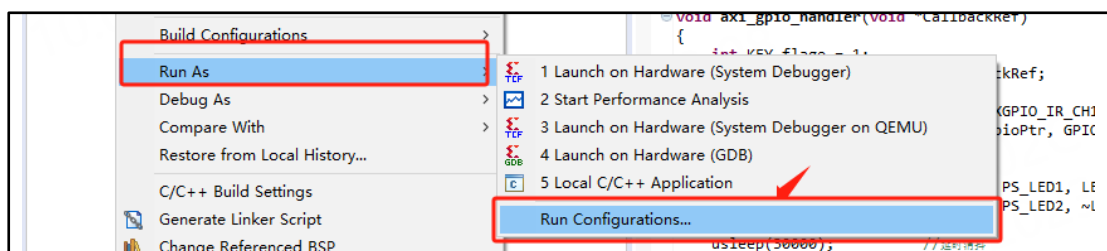


Figure 5-18 Program download diagram

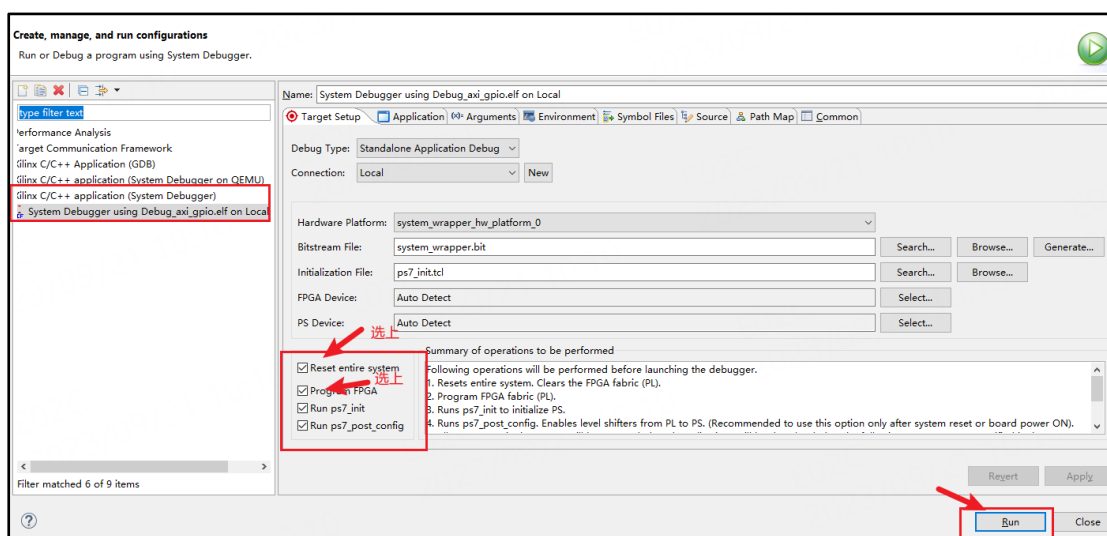


Figure 5-19 Download diagram

After the program is run, the PS end PS_LED0 and PS_LED1 can be controlled to turn on and off alternately through the PL_KEY2 button on the development board.

5.4 End of chapter summary

This chapter uses the AXI GPIO ip core located on the PL side. The PS side accesses AXI GPIO through the AXI4-Lite interface, accepts interrupt requests from AXI GPIO, and completes the detection of the buttons. Different from the key interrupt in the "EMIO Controlling LED" chapter, the interrupt source of this experiment is the AXI GPIO on the PL side, while the interrupt source of the "EMIO Controlling LED" experiment is the GPIO on the PS side.

5.5 Development training

In this chapter's experiment, the pin direction of AXI GPIO is configured as the input direction. You can try to change the "GPIO Width" of channel 1 to 2, bind one of the pins to the LED pin on the PL side of the Haoyue development board, and programmatically control the LED to turn on and off.

Chapter 6 UART serial port interrupt experiment

6.1 Chapter Introduction

Since its birth, the serial port has been widely used due to its simple and reliable transmission method. In today's era full of high-speed communication equipment, the serial port is still not outdated. Desktop computers and laptops long ago all had serial ports. Although many desktop computers no longer have serial ports, and laptops no longer have serial ports, the serial port has never been discarded. It just exists in another form, which is the USB to serial port chip. The serial port only needs 2 wires to achieve one receive and one send. It is simple to use, reliable and convenient, and can be widely used in low-speed situations. For example, some industrial control human-machine interfaces and the USB downloaders of some microcontrollers we use are all implemented using USB to serial port.

When we use the PS side, we print information and debug code by adding a UART controller. This chapter further analyzes the implementation process of UART interrupt in detail, and develops UART interrupt based on PL_PS interrupt.

6.2 Theoretical study

ZYNQ SoC contains two UART controllers, namely UART0 and UART1. They are full-duplex Asynchronous Receiver and Transceiver controllers that support a wide range of programmable baud rates and can accommodate automatic parity generation and multi-master detection modes. The UART controller has independent RX and TX data paths, each path includes a 64-byte FIFO, data will be serialized and deserialized in the TX and RX FIFO, and includes an I/O mode switch to support various loopback configurations of RxD and TxD signals. The FIFO interrupt status bit supports polling or interrupt-driven handlers. Software can read and write data bytes through the RX and TX data port registers. At the same time, parameters such as data bit length, stop bits, and verification methods can be configured according to needs.

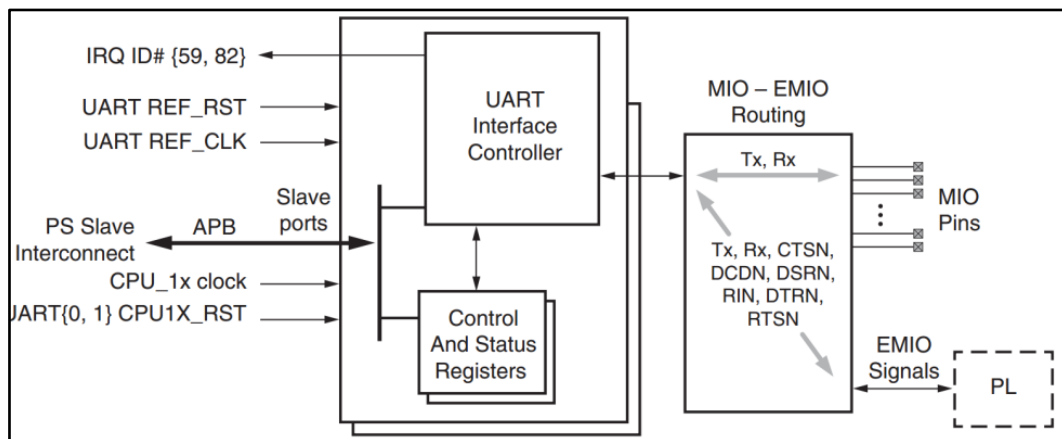


Figure 6-1 UART controller system block diagram

As can be seen from the above figure, the controller and I/O interface are driven by the reference clock (UART_REF_CLK). The interconnection of the controller also requires the APB interface clock (CPU_1x). These two clocks always come from the PS clock subsystem. The configuration of the UART controller and status acquisition are completed by the Control and Status Registers. The UART RxD and TxD signals can be routed to one of many sets of MIO pins or to the EMIO interface. All modem flow control messages

Numbers (CTSN, DSDN, DSRN, etc.) are always routed to the EMIO interface and are not available on the MIO pins. So if you need to use the flow control function, the signal must be routed to the EMIO interface.

By switching the I/O mode, the loopback mode of the signal can be configured. ZYNQ UART has four loopback modes, namely: Normal Mode (Normal Mode), Automatic Echo Mode (Automatic Echo Mode), Local Loopback Mode (Local Loopback Mode) and Remote Loopback Mode (Remote Loopback Mode).

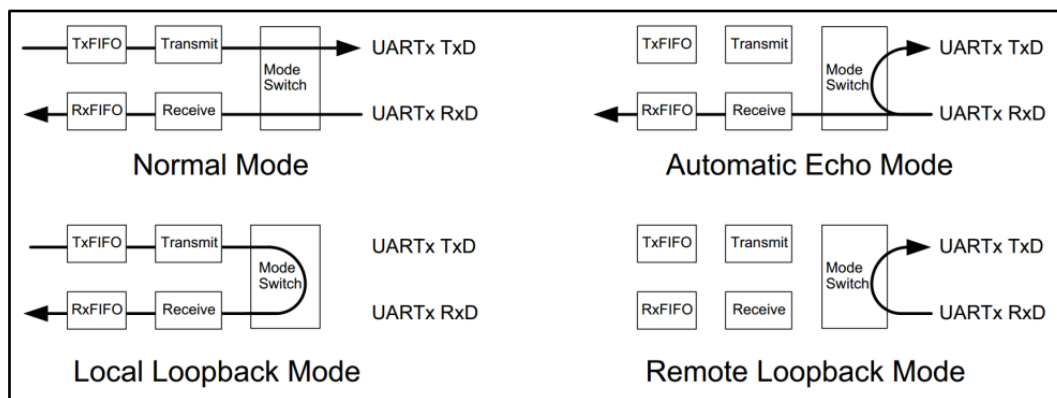


Figure 6-2 Four modes

These four modes are controlled by the CHMODE bit of the `uart.mode_reg0` register. The most common one in practical applications is the UART normal mode.

6.3 Practical exercises

6.3.1 Experimental objectives

Use the UART controller to complete the serial port interrupt data loopback function. And after receiving the corresponding data, the LED is lit.

6.3.2 Hardware resources

The physical diagram and schematic diagram of the USB to serial port of Haoyue development board are as follows.

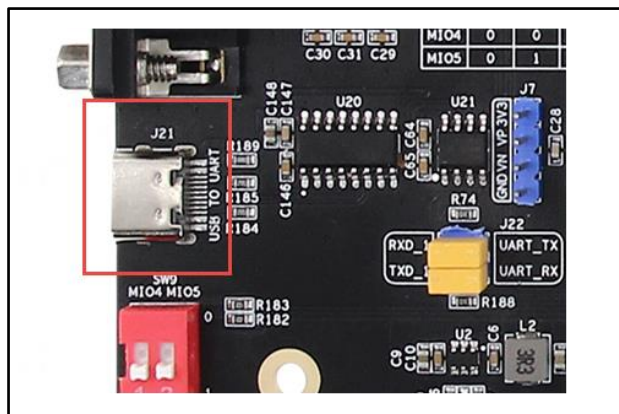


Figure 6-3 USB to serial port physical diagram

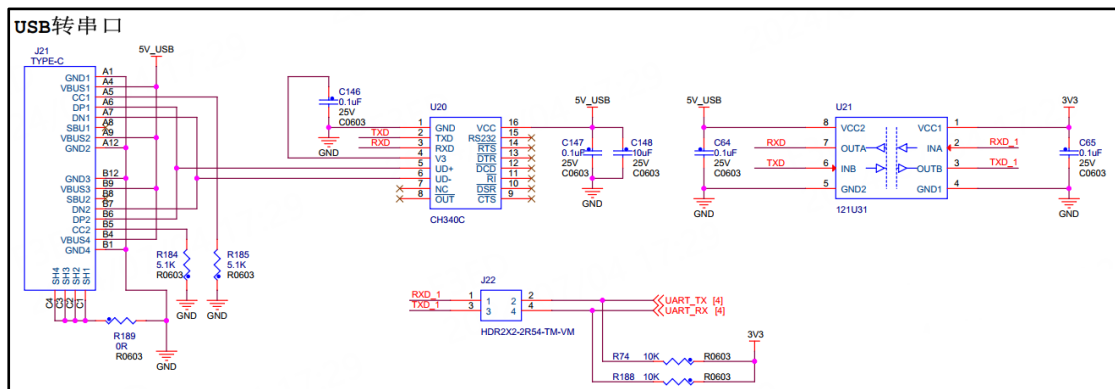


Figure 6-4 USB to serial port schematic diagram

6.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in the figure below.

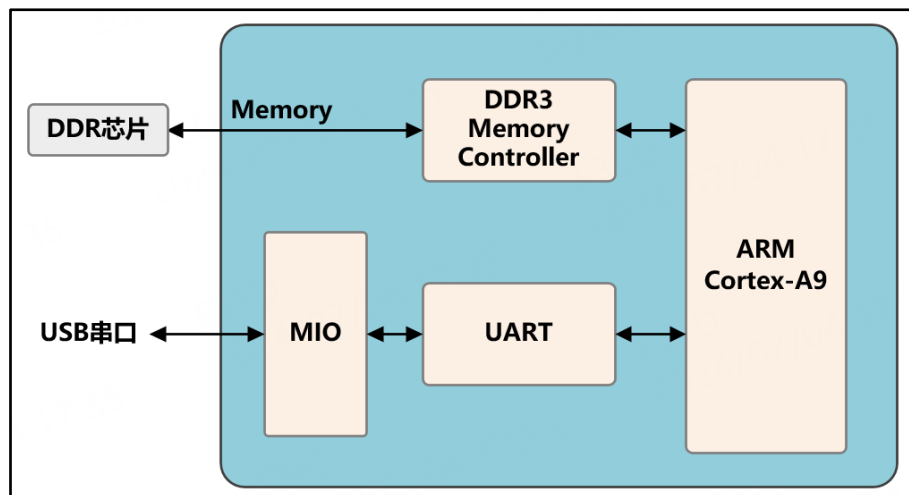


Figure 6-5 Overall block diagram

2. Hardware design

The Vivado part is basically the same as that in Chapter 1, except that in order to light up the lights, you need to configure more MIO, as shown below:

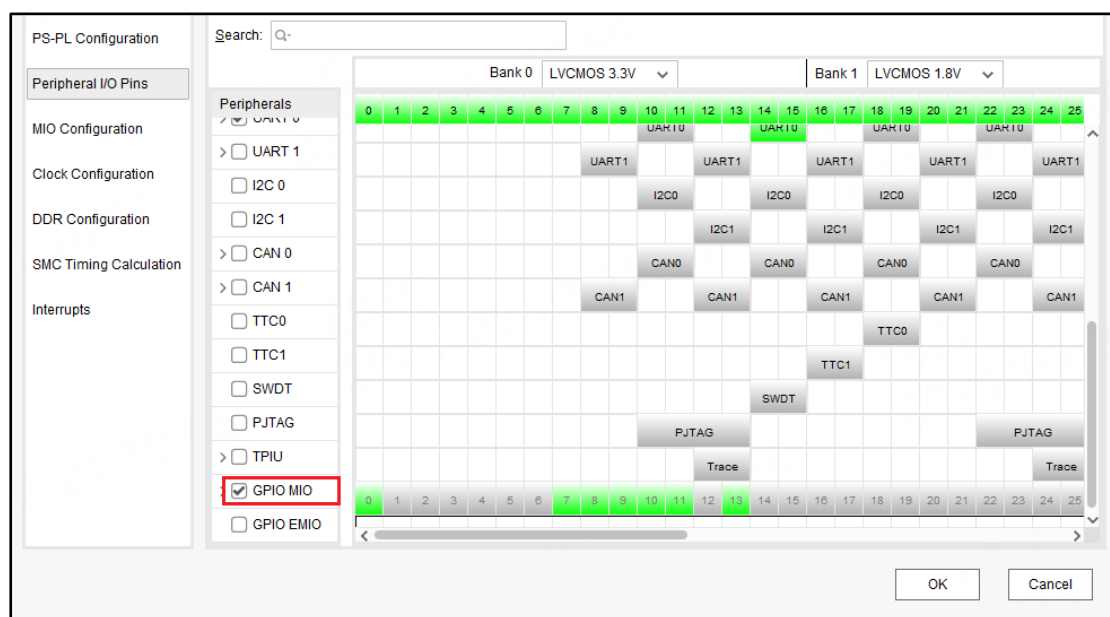


Figure 6-6 Configuration diagram

Next, export the hardware design and start the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and Code List 6-1 Serial port interrupt reference code (main.c)

```
1 #include "stdio.h" 2 #include "xuartps.h" 3 #include "xscugic.h" 4 #include "xil_printf.h"
5 #include "xparameters.h" 6 #include "xgpiops.h" 7 8 /* Redefine the ID number */ 9 #def
ine UART_ID XPAR_PS7_UART_0_DEVICE_ID // Serial port device ID 10 #define INT
C_ID XPAR_SCUGIC_SINGLE_DEVICE_ID // Interrupt ID 11 #define UART_INT_ID
XPAR_XUARTPS_0_INTR // Serial port interrupt ID 12 #define GPIOPS_ID mio0 15 #de
fine PS_LED2 9 // ps_led2 is fixed to mio9 16 17 XScuGic Cur_int; 18 23 24 XGpioPs Psg
pio_target; // Instantiate gpio 25 31 int main(void)
```

```
32 {33 XUartPs_Config *Uart_ptr; 34
```

```
    Uart_ptr = XUartPs_LookupConfig(UART_ID);  
42 // Initialize serial port 43 XUartPs_CfgInitialize(&Psuart_target, Uart_ptr, Uart_ptr-> 44 BaseAddress); 45  
// Function self-test 46 if (XUartPs_SelfTest(&Psuart_target) != XST_SUCCESS) 47 xil_printf("Initialization  
failed !!\r\n"); 48 // Set the normal working mode 49 XUartPs_SetOperMode(&Psuart_target, XUARTPS_OP  
ER_MODE_NORMAL); 50 // Set the baud rate 51 XUartPs_SetFifoThreshold(&Psuart_target, 1); 54 55 Scug  
iconfig_ptr = XScuGic_LookupConfig(INTC_ID); 56 Xil_ExceptionInit(); 59 // Set the interrupt exception ha  
ndling function 60 Xil_ExceptionRegisterHandler( 61 XIL_EXCEPTION_ID_INT, 62 (Xil_ExceptionHandler  
)XScuGic_InterruptHandler, 63 (void *)&Cur_int); 64 67 XScuGic_Connect(&Cur_int, 68 UART_INT_ID, 6  
9 uart_intr_handler, 70 (void *)&Psuart_target); 71 // Set the interrupt trigger mode 72 XUartPs_SetInterruptM  
ask(&Psuart_target, XUARTPS_IXR_RXOVR); 73 XScuGic_Enable(&Cur_int, UART_INT_ID); // Enable s  
erial port interrupt 74 75 while (1) { 76 if (ReceivedFlag) 77 ReceivedFlag = 0; 78 } 79 return 0; 80 } 81 82 83  
/* Serial port transceiver function */ 84 void uart_intr_handler(void *call_back_ref) 85 { 86 XUartPs *Uart_re  
ceive_ptr = (XUartPs *) call_back_ref; 87 int Buffer = 0; 88 int Int_flag; 89 90 /* Detect interrupt flag */ 91  
Int_flag = 94 95 if (Int_flag & XUARTPS_IXR_RXOVR) { // Determine if an interrupt occurs
```

```
96 // Receive data into the buffer 97 Buffer = XUartPs_RecvByte(XPAR_PS7_UART_0_BASEADDR);
98 // Clear the interrupt flag 99 XUARTPS_IXR_RXOVR); 101 } 102 103 if (Buffer==0x31) { // '1' corre
sponds to 0x31 104 LED_flage1=~LED_flage1; 105 XGpioPs_WritePin(&Psgpio_target,PS_LED1,LED
_flage1); 106 } else if (Buffer== 0x32) { // '2' corresponds to 0x32 107 LED_flage2=~LED_flage2; 108
XGpioPs_WritePin(&Psgpio_target,PS_LED2,LED_flage2); 109 } 110 // Send buffered data 111 XUartP
s_SendByte(XPAR_PS7_UART_0_BASEADDR,Buffer); 112 } 113 114 /* gpio initialization */ 115 void
gpio_init(void) 116 { 117 Psgpio_ptr = XGpioPs_LookupConfig(GPIOPS_ID); 118 XGpioPs_CfgInitializ
e(&Psgpio_target, Psgpio_ptr, Psgpio_ptr-> 119 BaseAddr); 120 121 // Set the polarity to output 122 XGp
ioPs_SetDirectionPin(&Psgpio_target, PS_LED1, 1); 123 XGpioPs_SetDirectionPin(&Psgpio_target, PS_
LED2, 1); 124 // Output enable 125 XGpioPs_SetOutputEnablePin(&Psgpio_target, PS_LED1, 1); 126 12
7 // Output high level 128 XGpioPs_WritePin(&Psgpio_target, PS_LED1, LED_flage1); 129
```

This time we also introduced the header file `xuartps.h`, which contains some related functions for configuring the UART controller.

Next is the familiar configuration of interrupts. The steps are basically the same: first enable the UART controller, then configure the corresponding mode and priority, and then configure the GIC to enable the UART interrupt source and associated interrupt service functions.

In lines 41-53 of code, the UART controller is initialized and configured accordingly. First, find the device information and initialize it, then use `XUartPs_SetOperMode` to configure the working mode of the serial port, here it is configured as normal mode, and then use `XUartPs_SetBaudRate` to configure the baud rate, here I use 115200. Finally, set the amount of data that triggers the interrupt, that is, set the interrupt trigger level of RXFIFO. Here we set it to 1, so that every time the serial port sends a piece of data, an interrupt will be triggered.

Then we start configuring the interrupt. In code 58-73, we first configure the GIC controller. After the configuration is completed, we associate the UART interrupt service function. The `XUartPs_SetInterruptMask` function is used to configure the UART trigger mode. `XUARTPS_IXR_RXOVR` We use this mode. This mode triggers an interrupt when the amount of data in the RXFIFO is reached. That is, when it reaches one, we trigger an interrupt. Finally, disable the UART interrupt in the GIC.

Next, write the interrupt service function. In lines 89-119 of the code, first use the `XUartPs_ReadReg` function to read the interrupt ID register, and then compare it with our interrupt mode to determine whether it is the interrupt we want. If so, use the `XUartPs_RecvByte` function to read the received data and clear the corresponding interrupt flag bit. Make another judgment

If the received data is '1' or '2' (hexadecimal is 0x31, 0x32 respectively), the LED status will be flipped. Finally, use the XUartPs_SendByte function to send the received data to implement the serial port loopback function.

Lines 117-129 of the code are to initialize MIO for lighting. The operation here is the same as when MIO was lighting before. Initialize GPIO and then configure the mode to output and enable it.

After writing the code, compile the project and prepare for board verification

6.3.4 Board verification

As shown in Figure 6-7, the development board is connected. After the USB cable is correctly connected, turn on the switch

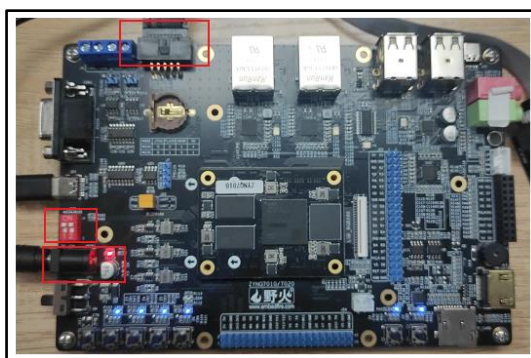


Figure 6-7 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project and select Run As > Launch on Hardware (system Debugger).

After the program download is completed, open the serial port assistant. As shown in the figure below, when the serial port assistant sends the string "12" to the Haoyue development board, the serial port assistant will also receive the same string "12". In addition, the string "12" contains the information '1' and '2' for controlling the LED, so the status of the LED on the Haoyue development board will also change accordingly.

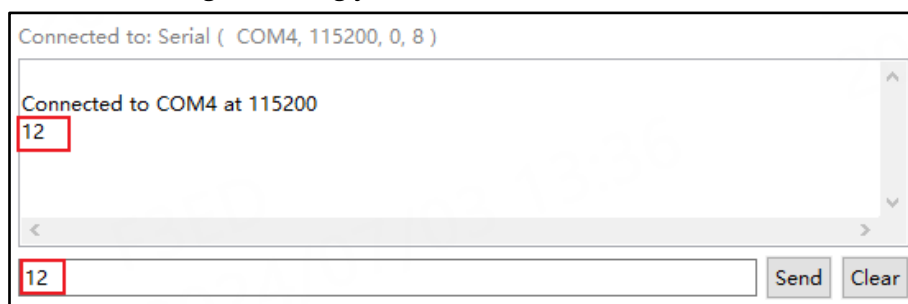


Figure 6-8 Send string

6.4 End of chapter summary

This chapter introduces the UART controller. There are four loopback modes for ZYNQ UART. And analyzed the implementation process of UART interrupt. We receive data from the computer serial port assistant through the interrupt, analyze the serial port data and control the LED status, and send the received data back to the serial port assistant.

6.5 Development training

In the experiment of this chapter, the received data is sent back to the serial port assistant through the XUartPs_SendByte() function in the interrupt. However, if it is configured in automatic echo mode or remote loopback mode, the data will be automatically sent back to the serial port assistant. You can try it based on the experiments in this chapter.

Chapter 7 Timer Interrupt Experiment

7.1 Chapter Introduction

Timers play a vital role in an embedded operating system. Each operating system has at least one timer that is used to generate interrupts, perform task scheduling and other operations. The operating system needs to know the time. For example, the system reads the status of a certain port every once in a while. The timer regularly informs the operating system of the current clock, software timers by triggering a certain message, and hardware timers by interrupts.

7.2 Theoretical study

According to Figure 7-1, the middle part is the GIC, which is the interrupt controller, and is connected to other timers for enabling, turning off, masking, setting priorities, etc. for interrupts. The others are a 24-bit system watchdog timer (SWDT), two triple timer counters (TTC), a 64-bit global timer (GT), and each Cortex-A9 processor has its own independent 32-bit private timer and 32-bit watchdog timer.

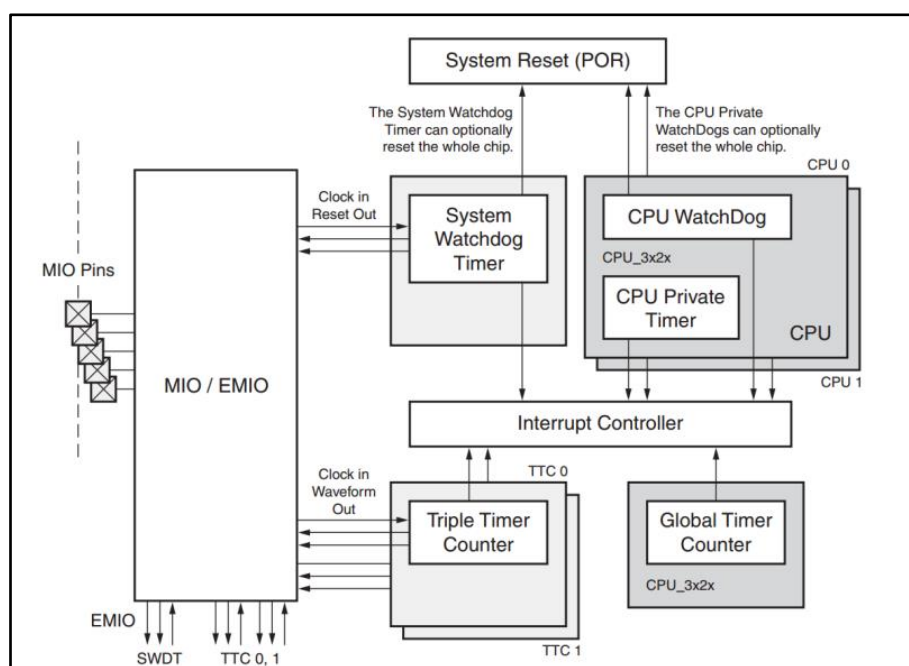


Figure 7-1 Timer system block diagram

Each ARM core of ZYNQ has its own private timer, and the working frequency of the private timer is half of the CPU. The ARM operating frequency is 666MHz, so the frequency of the private timer is 333MHz. The characteristics of private timers are as follows:

- (1) 32-bit counter, which generates an interrupt when the counter counts down to zero;
- (2) 8-bit prescaler for better control of interrupt cycles;
- (3) Can be configured as single timing or automatic reload mode;
- (4) Configuring the starting value of the counter can set the timing time, timing time = $1/\text{timer frequency} * (\text{preload value} + 1)$;

7.3 Practical exercises

7.3.1 Experimental objectives

Control the PS LED light to turn on and off through the timer interrupt.

7.3.2 Hardware resources

See the "MIO Control LED" chapter.

7.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in the figure below.

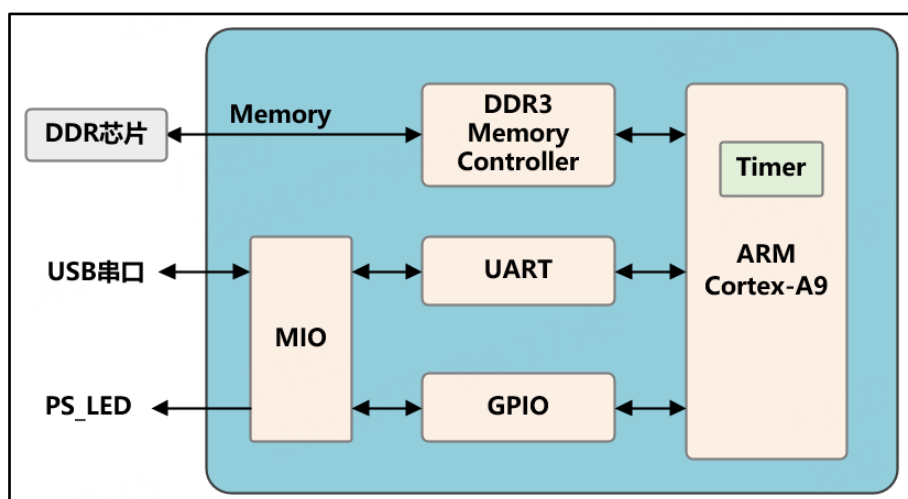


Figure 7-2 Overall block diagram

2. Hardware design

This project only includes the PS part. We can save it as a new project on the "MIO Control LED" project "time_intr" to complete the hardware design part.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project and v
Code List 7-1 Timer interrupt reference code (main.c)

```
1 #include "xscugic.h"  
2 #include "xgpiops.h"  
3 #include "xparameters.h"  
4 #include "xscutimer.h"  
5 #include "xil_exception.h"  
6 #include "xil_printf.h"
```

7

```
8 /* Rename ID */ 9 #define TIMER_ID XPAR_XSCUTIMER_0_DEVICE_ID 10 #define INTC_ID X
PAR_SCUGIC_SINGLE_DEVICE_ID 11 #define TIMER_INT_ID PS_LED1 0 14 #define PS_LED2 9
15 16 /* Timer frequency = TIMER_LOAD_VALUE 0x9EC969F 20 21 22 XScuGic Cur_int; 23 XScu
Timer Timer; 24 XGpioPs_Config *Psgpio_ptr; 31 32 Psgpio_ptr = XGpioPs_LookupConfig(XPAR_X
GPIOPS_0_DEVICE_ID); 33 BaseAddr; 35 /* Set the PS_LED pin to output mode */ 36 XGpioPs_Set
DirectionPin(&Psgpio_target, PS_LED1, 1); 37 XGpioPs_SetOutputEnablePin(&Psgpio_target, PS_LE
D1, 1); 40 XScuTimer_Config *Timer_ptr; 47 // ID lookup enable 48 Timer_ptr = XScuTimer_Lookup
Config(TIMER_ID); 49 // Initialization 50 XScuTimer_LoadTimer(time, TIMER_LOAD_VALUE); 53
// Enable automatic loading 54 XScuGic_Config * Scugiconfig_ptr; 61 Scugiconfig_ptr = XScuGic_Loo
kupConfig(INTC_ID); 62 65 XIL_EXCEPTION_ID_INT, 66 (Xil_ExceptionHandler)XScuGic_Interrup
tHandler, 67 Intc); 68 Xil_ExceptionEnable(); // Turn on interrupt exception handling 69 70 // Associate
d interrupt service function 71 XScuGic_Connect(Intc,
```

```

72             TIMER_INT_ID,
73             timer_intr_handler,
74             (void *)time);
75
76     XScuGic_Enable(Intc, TIMER_INT_ID);
77     XScuTimer_EnableInterrupt(time); //使能定时器中断
78 }
79
80 int main(void)
81 {
82     LED_Init(); //LED 初始化
83     TIMER_Init(&Timer); //定时器初始化
84     TIMER_Intr_init(&Cur_int,&Timer); //定时器中断初始化
85     XScuTimer_Start(&Timer); //启动定时器
86     while (1);
87 }
88
89 /* 定时器中断服务函数 */
90 void timer_intr_handler(void *CallBackRef)
91 {
92     static int LED_Flage = 0;
93     XScuTimer *Timer_ptr = (XScuTimer *) CallBackRef;
94     // LED 状态翻转
95     LED_Flage = ~LED_Flage;
96     XGpioPs_WritePin(&Psgpio_target, PS_LED1,LED_Flage);
97     XGpioPs_WritePin(&Psgpio_target, PS_LED2,~LED_Flage);
98     // 清中断
99     XScuTimer_ClearInterruptStatus(Timer_ptr);
100 }

```

Compared with the previous design, this design introduces the header file xscutimer.h, which contains related func

Let's look at lines 46-54 of the code first. Here we initialize the timer. In addition to searching for configuration information and initializing, we use XScuTimer_LoadTimer to set the reload value and XScuTimer_EnableAutoReload to enable automatic reloading. Here is an explanation of how to calculate the reload value. As we can see from the above, the frequency of the private timer is 333MHZ, and it needs to be timed for 0.5s. That is, $1/0.5$ is 2HZ. So to calculate the reload value, 333MHZ divided by the reload value will equal 2HZ, so the reload value is 333MHZ/2HZ, which is equal to 166500000. Because the timer count is reduced to 0, it actually needs to be reduced by one, that is, 166499999 converted to hexadecimal is 0x9EC969F.

Next, let's look at lines 60-77 of the code. Here we begin to configure the timer interrupt. First, configure the GIC. After initializing the GIC, associate the timer interrupt service function, turn on the GIC enable, and then turn on the timer interrupt in the GIC. At this point, the configuration of the interrupt is over.

Next write the timer interrupt service function. After the program is running, the interrupt service function will be entered every 0.5s, and the LED_flag will be flipped every time the interrupt is triggered to realize the LED status flip. At the end of each time, XScuTimer_ClearInterruptStatus must be used to clear the timer flag.

As for the initialization of MIO, I believe that readers will basically know it here, so I will not go into details here. If you forget, you can read the MIO chapter.

Next, compile the project and run it on the board for verification.

7.3.4 Board verification

As shown in the figure below, the development board is connected. After the lines are connected correctly, turn on

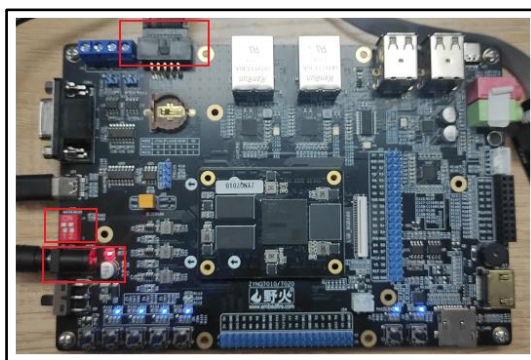


Figure 7-3 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project and select Run As > Launch on Hardware (system Debugger).

After the program download is completed, you can see that the status of the two LEDs of the PS on the Haoyue development board changes every 500ms.

7.4 End of chapter summary

This chapter introduces the timer resources on the ZYNQ PS side, and through experiments, the private timer generates an interrupt every 500ms.

7.5 Development training

In the "MIO Key Interrupt Experiment", a simple software delay is used to eliminate the jitter of the button. This kind of jitter elimination consumes CPU resources. After learning the timers in this chapter, you can try to use interrupts and timers to eliminate jitter. The timer delay jitter elimination process does not consume CPU resources and can save CPU resources.

Chapter 8 Custom IP Core

8.1 Chapter Introduction

During the ZYNQ development process, the PL side often needs to interact with the PS side. At this time, if the required PL side resources are encapsulated into an IP core with an AXI bus, it will be very convenient to call it on the PS side. This can greatly simplify system design, shorten design time, and enable design reuse.

In this chapter we will introduce how to customize the IP core.

8.2 Theoretical study

In Vivado software, we can customize IP cores through the creation and packaging IP wizard. This function allows users to encapsulate existing projects, specific project modules or specific file directories into IP cores. In addition, users can also create an IP core with an AXI 4 interface, which is suitable for data exchange between PS and PL. In this experiment, we adopted a common method, that is, building an IP core with an AXI interface to implement data transmission between PS and PL through the AXI protocol.

8.3 Practical exercises

8.3.1 Experimental objectives

Customize the breathing light IP core. The PS side communicates with the IP core through the AXI bus to control the breathing light.

8.3.2 Hardware resources

The actual LED picture and schematic diagram of the Haoyue development board are shown in Figure 8-1 and Figure 8-2.

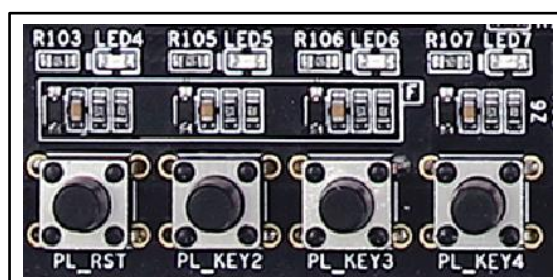


Figure 8-1 PL_LED physical picture

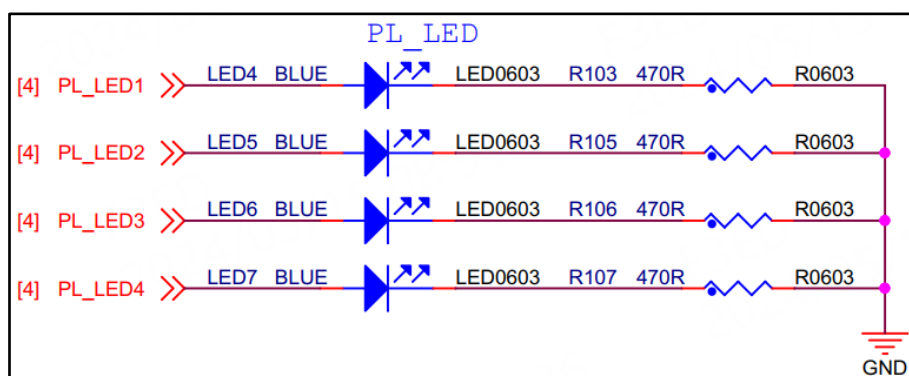


Figure 8-2 PL_LED schematic diagram

8.3.3 Programming

1. Overall description

The overall structure is shown in Figure 8-3.

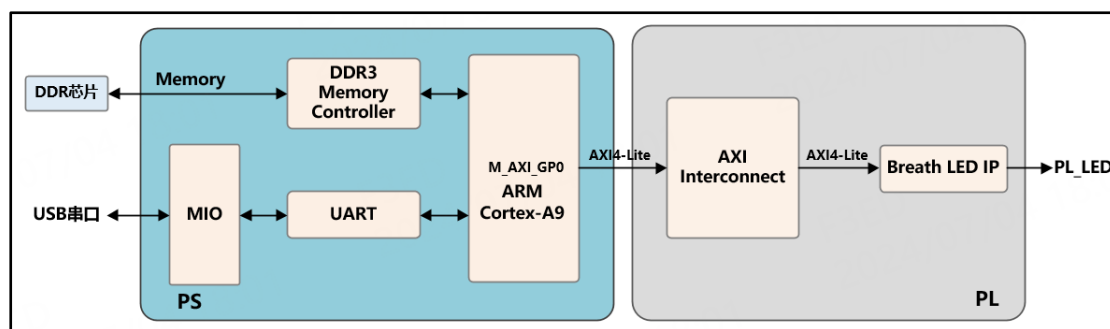


Figure 8-3 Overall block diagram

2. Hardware design

Below we introduce step by step how to customize the IP core and use it.

Step1 Create a new IP core

Open Vivado, enter the Vivado interface, and click "Manage IP" in the "Tasks" column. Select "New IP Location..." from the pop-up options. As shown below:

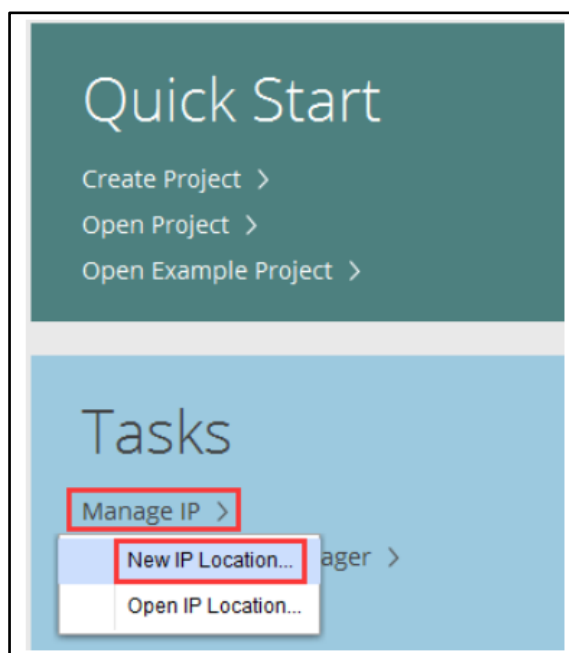


Figure 8-4 Click Manage IP

The following interface appears, click NEXT:

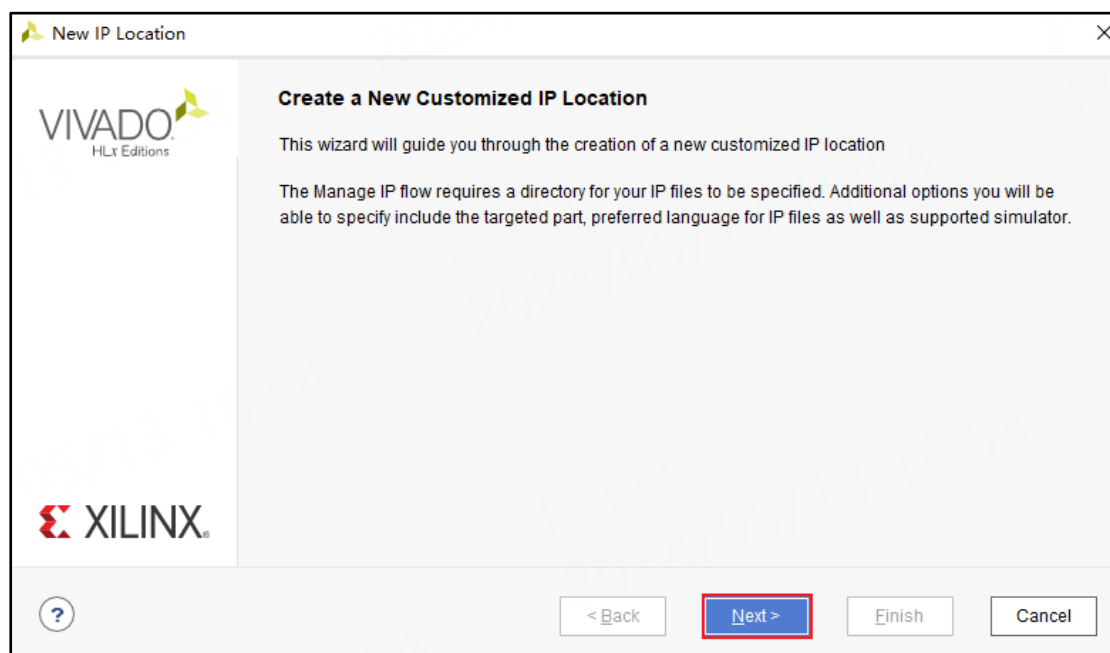


Figure 8-5 Custom IP core interface

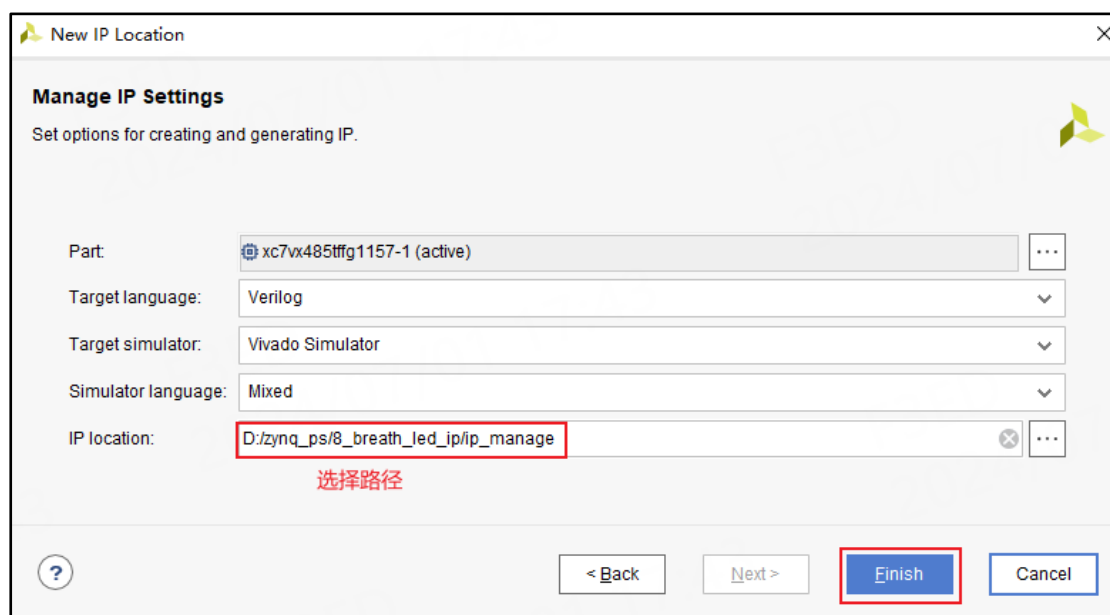


Figure 8-6 Select the path where the IP core is generated

After the project is created, run the Create and Pack IP wizard. Click "Tools" on the menu bar, select "Create and Package New IP", and click "NEXT" in the pop-up interface.

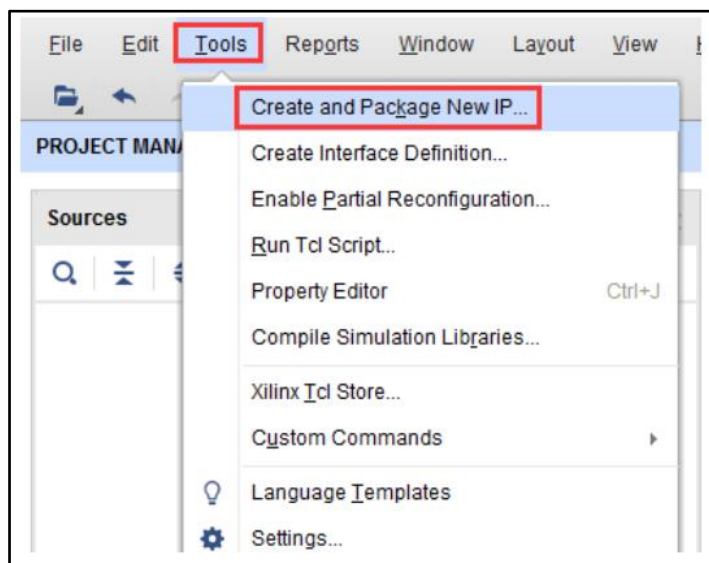


Figure 8-7 Select to create and encapsulate ip core

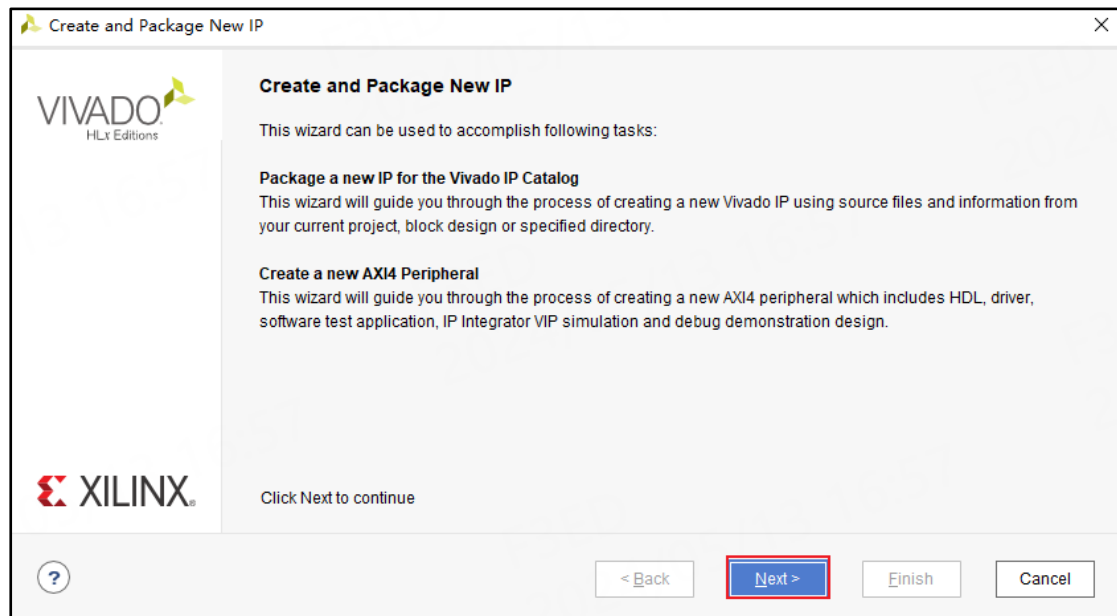


Figure 8-8 Click NEXT

Next, choose to encapsulate the IP or create an IP core with an AXI4 interface. Here we choose to create an IP core with an AXI interface, select "Create a new AXI4 peripheral", and click the "NEXT" button.

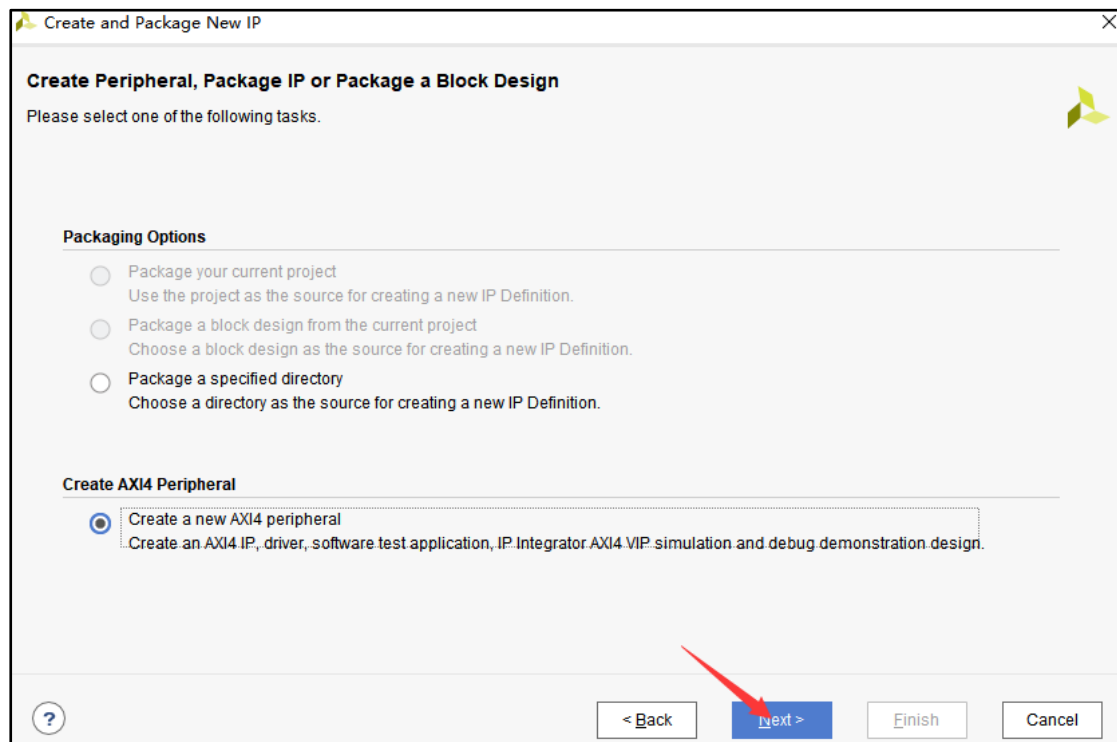


Figure 8-9 Select with AXI interface

Next, set the IP core name (Name), version number (Version), display name (Display name), description (Description) and path (IP location) respectively. Generally, only the name is modified and the others remain as default.

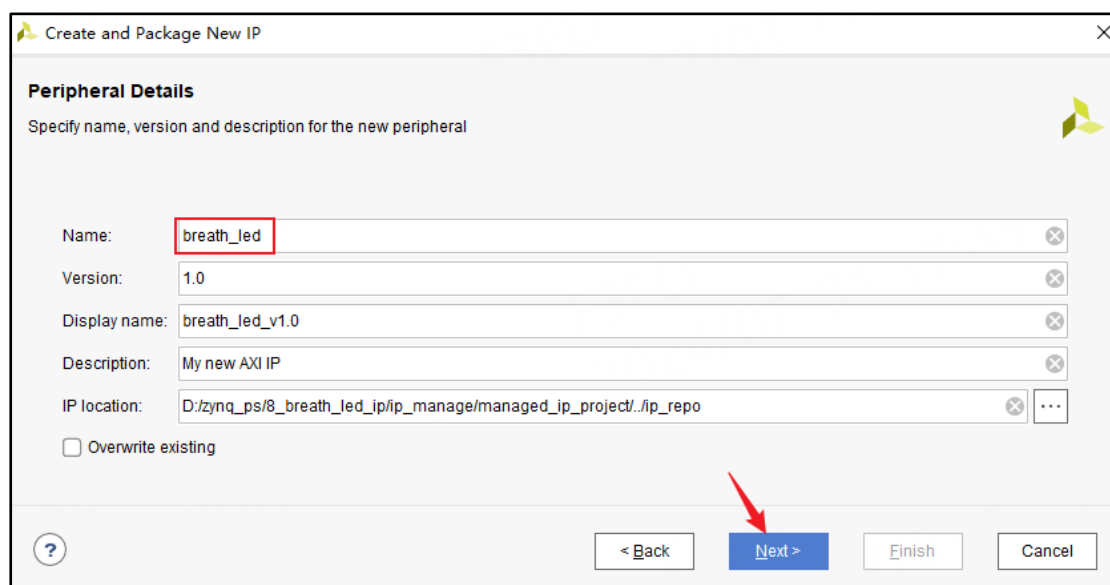


Figure 8-10 IP parameter settings

Next is the AXI interface settings.

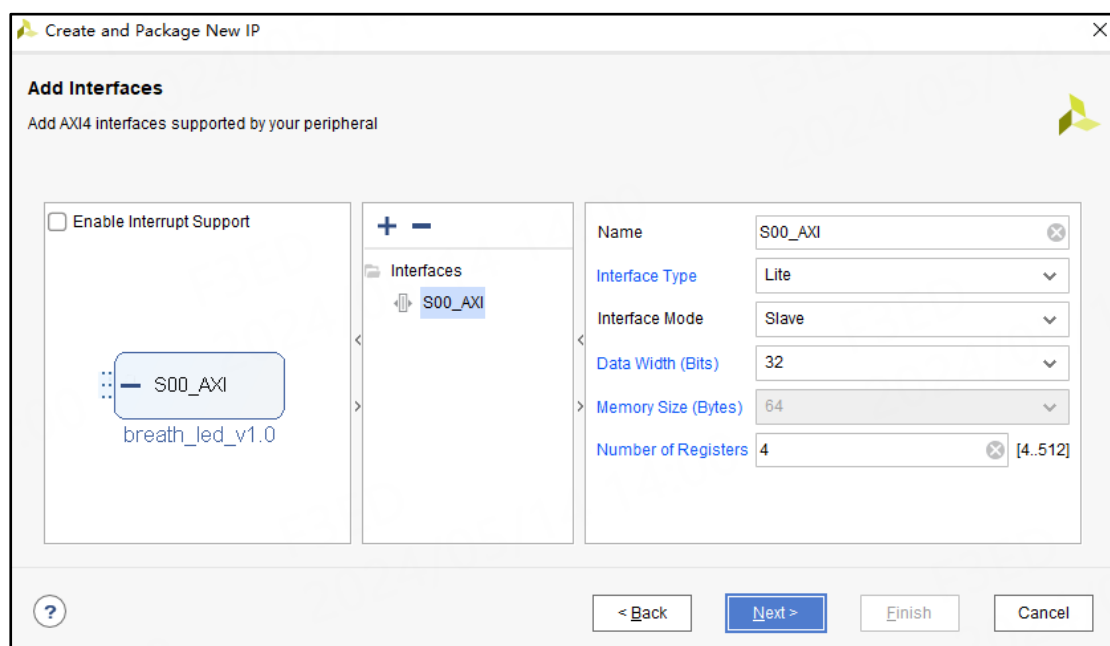


Figure 8-11 AXI interface settings

Interface Type is divided into three types in AXI: Lite, Full and Stream. AXI4-Lite is a simplified interface mainly used for storage mapping communication of small amounts of data; AXI4-Full is a high-performance storage mapping interface suitable for communication of large amounts of data; and AXI4-Stream is specially used for high-speed data stream transmission, which is a non-storage mapping interface. Considering that this experiment only needs to process the communication of a small amount of data, it is appropriate to choose to use the default Lite interface.

The interface mode (Interface Mode) can be selected as Slave (slave) or Master (host) in the AXI protocol. The connection is established through the "handshake" process between the slave and the host. In this experimental setup, we use the Slave interface mode by default.

The data width (Data Width) remains at the default value here, which is 32 bits.

The configuration option of Memory Size is not available in XI4-Lite interface mode, so it is not set.

Number of Registers is used to configure the number of registers for devices such as LED breathing lights in PL. Here we continue to use the default value.

After clicking "Next", an interface will appear summarizing the details of the packaging interface and the next operation options. In this step, maintain the default settings, which add the newly created IP core to the IP library. After completing these settings, click the "Finish" button to complete the creation and packaging of the IP core.

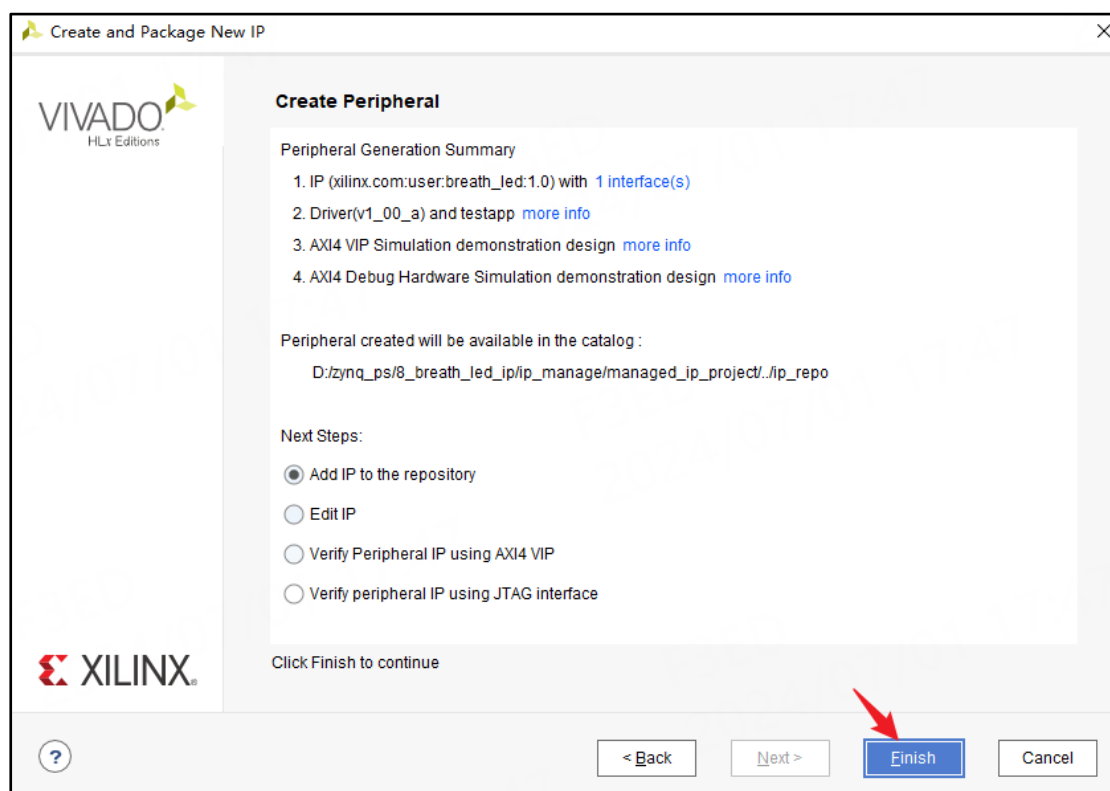


Figure 8-12 Create and package IP core summary interface

In the IP Catalog interface, the newly added IP core will be displayed in "AXI Peripheral" under "User Repository", and its name is identified as "breath_led_v1.0". As shown below:

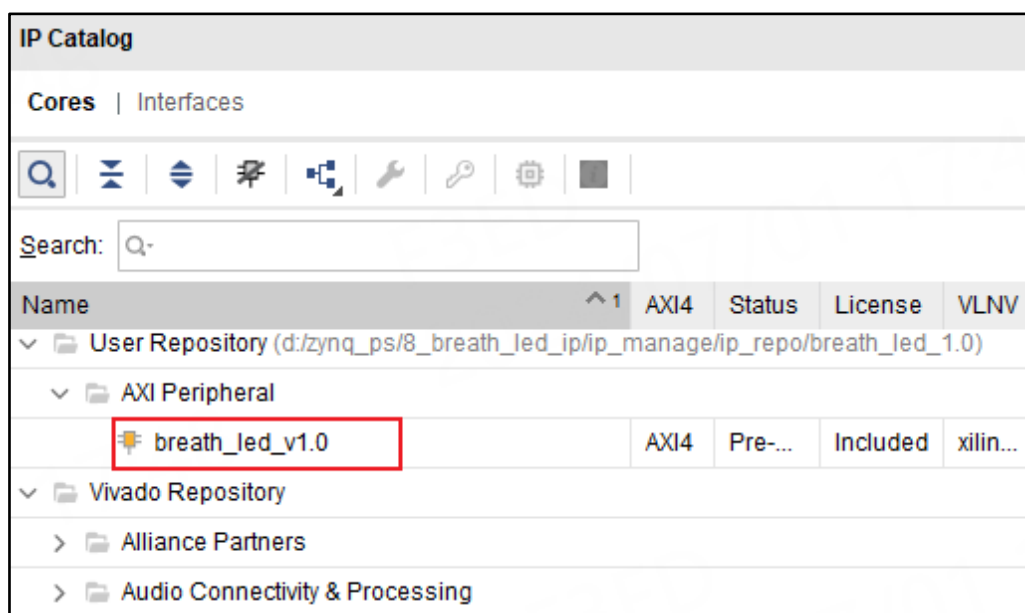


Figure 8-13 Newly created IP

Step2: Edit the ip core just created

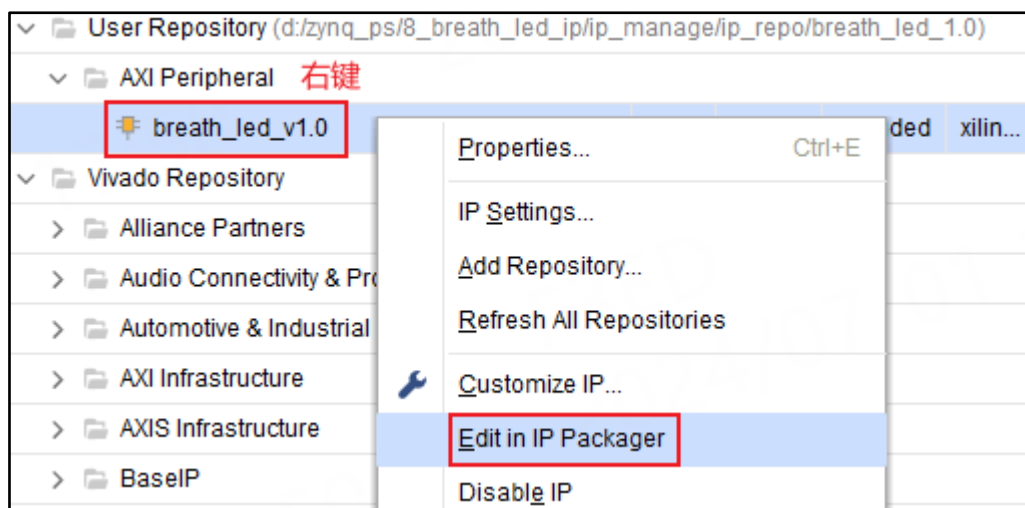


Figure 8-14 Edit customized ip core

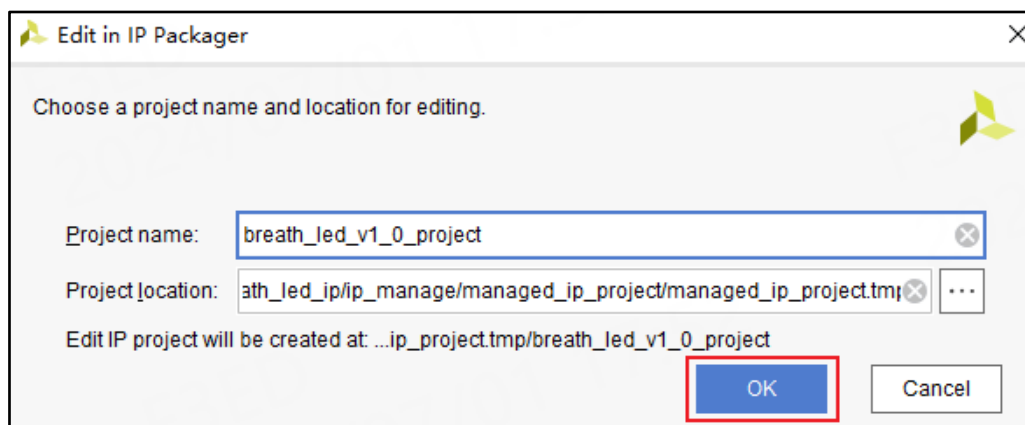


Figure 8-15 Generating a project for editing IP cores

After completing the above steps, a new project will be opened.

Since the software has already designed the corresponding code for the AXI bus for us, we only need to add the corresponding port under the top-level file of the newly created project and instantiate our breathing light module as a submodule.

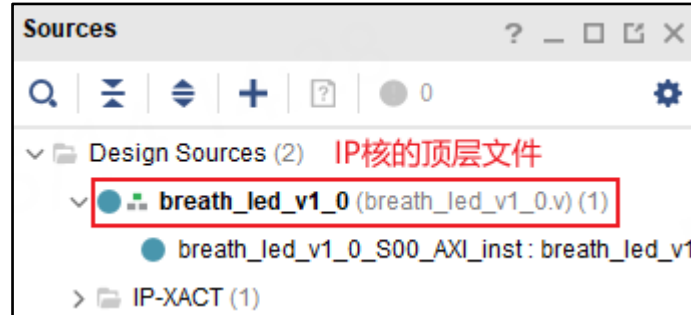


Figure 8-16 IP core top-level file

Double-click the `breath_led_v1_0.v` file, and add the following code in line 7 of the code: "parameter `BREATH_FREQ = 10'd100`," this parameter is used to set the default breathing frequency of the breathing light. In line 18 of the code, add the following code: "output wire `led`,".

```

4      module breath_led_v1_0 #
5      (
6          // Users to add parameters here
7          parameter BREATH_FREQ = 10'd100,
8          // User parameters ends
9          // Do not modify the parameters beyond this line
10
11
12         // Parameters of Axi Slave Bus Interface S00_AXI
13         parameter integer C_S00_AXI_DATA_WIDTH  = 32,
14         parameter integer C_S00_AXI_ADDR_WIDTH  = 4
15     )
16     (
17         // Users to add ports here
18         output wire led,

```

Figure 8-17 Edit `breath_led_v1_0 (1)`

Add the previously added instantiation parameters and port name to the `breath_led_v1_0_S00_AXI` instantiation place, as shown in Figure 8-18, and then save the file.

```

46     // Instantiation of Axi Bus Interface S00_AXI
47     breath_led_v1_0_S00_AXI # (
48         .BREATH_FREQ (BREATH_FREQ),
49         .C_S_AXI_DATA_WIDTH(C_S00_AXI_DATA_WIDTH),
50         .C_S_AXI_ADDR_WIDTH(C_S00_AXI_ADDR_WIDTH)
51     ) breath_led_v1_0_S00_AXI_inst (
52         .led(led),
53         .S_AXI_ACLK(s00_axi_aclk),

```

Figure 8-18 Edit `breath_led_v1_0 (2)`

Then open the `breath_led_v1_0_S00_AXI.v` file, and also add the following code to the 7th line of the code: "parameter BREATH_FREQ = 10'd100," and the following code to the 18th line of the code: "output wire led,".

```

4      module breath_led_v1_0_S00_AXI #
5      (
6          // Users to add parameters here
7          parameter BREATH_FREQ = 10'd100,
8          // User parameters ends
9          // Do not modify the parameters beyond this line
10
11         // Width of S_AXI data bus
12         parameter integer C_S_AXI_DATA_WIDTH    = 32,
13         // Width of S_AXI address bus
14         parameter integer C_S_AXI_ADDR_WIDTH    = 4
15     )
16     (
17         // Users to add ports here
18         output wire led,
19         // User ports ends
20         // Do not modify the ports beyond this line

```

Figure 8-19 Edit `breath_led_v1_0_S00_AXI` (1)

Add the `breath_led_top.v` file code that instantiates the breathing light function at the end of `breath_led_v1_0_S00_AXI.v`.

```

400     // Add user logic here
401     breath_led_top#(
402         .BREATH_FREQ    ( BREATH_FREQ    )
403     )u_breath_led_top(
404         .sys_clk         ( S_AXI_ACLK     ),
405         .rst_n           ( S_AXI_ARESETN ),
406         .breath_led_en   ( slv_reg0[0] ), //寄存器0的第一位
407         .freq_set        ( slv_reg1[9:0] ), //寄存器1的 前10位
408         .led              ( led           )
409     );

```

Figure 8-20 Edit `breath_led_v1_0_S00_AXI` (2)

Save the file. At this time, a `breath_led_top.v` is also needed. We add `breath_led_top.v` to the project according to the operation. First, select `breath_led_v1_0.v` under Design Sources, and then click the "+" number above.

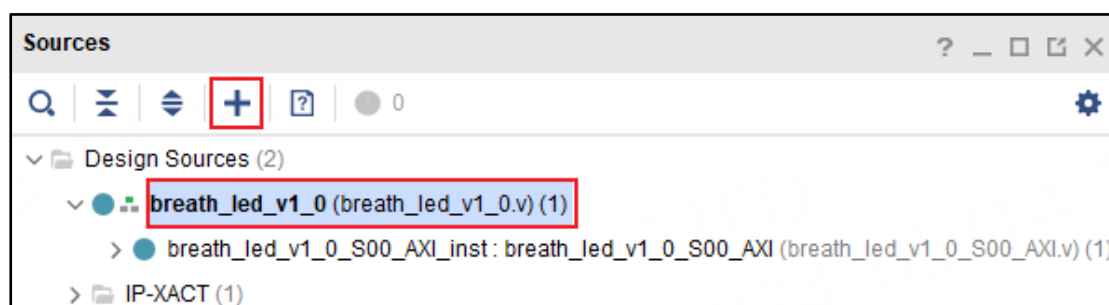


Figure 8-21 Add Sources

In the pop-up interface, select "add or create design sources" and click "NEXT".

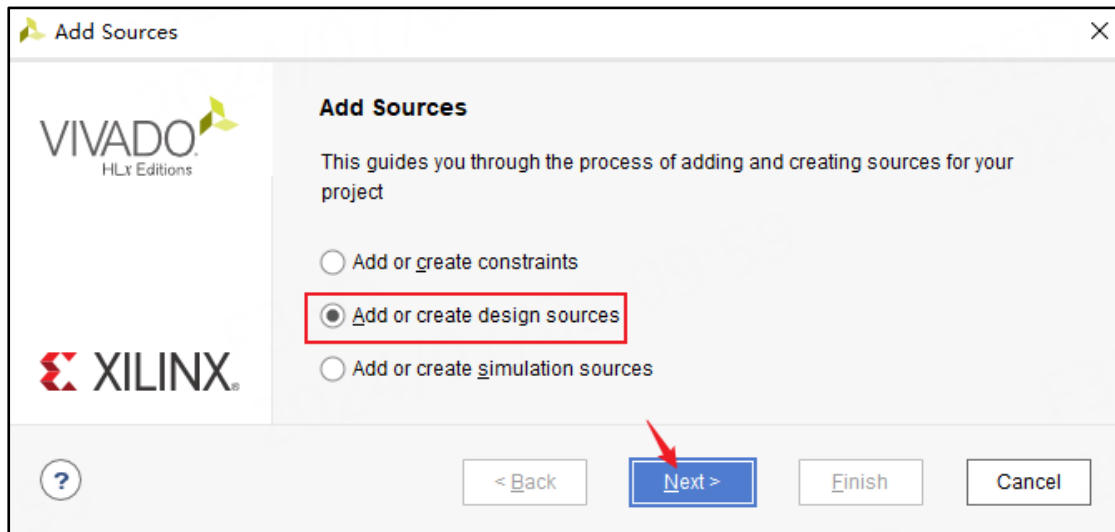


Figure 8-22 Select Add or create design sources

Click "Create File".

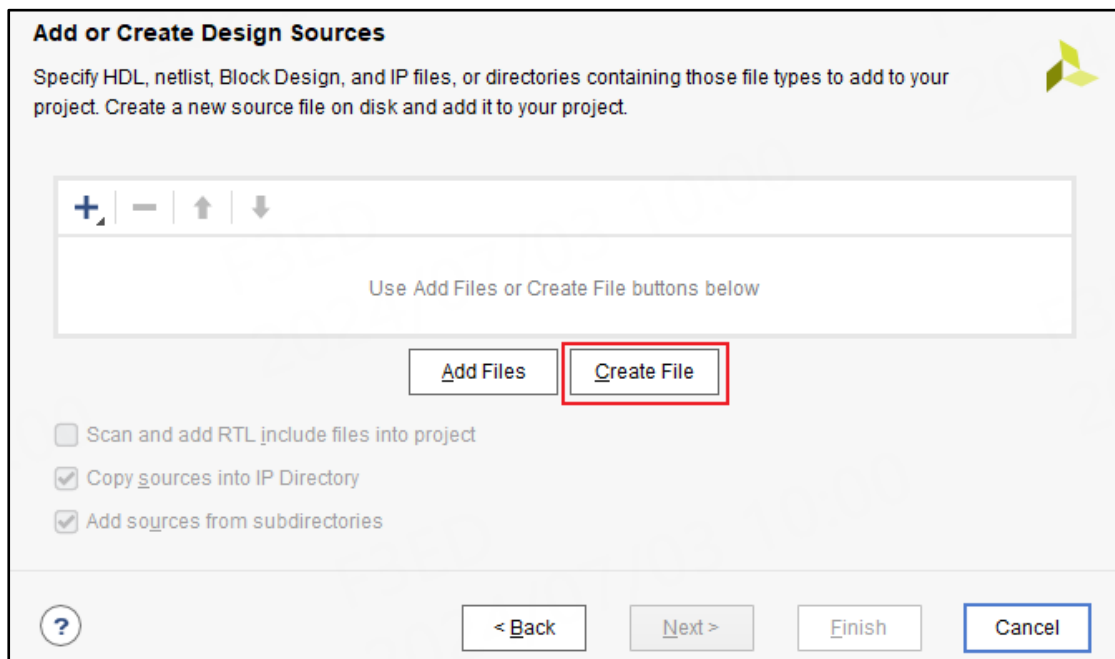


Figure 8-23 Create File

In the pop-up dialog box, enter the file name "breath_led_top", and select the path as...\\ip_manage\\ip_repo\\breath_led_1.0\\hdl directory, and then click "OK", as shown in the figure below.

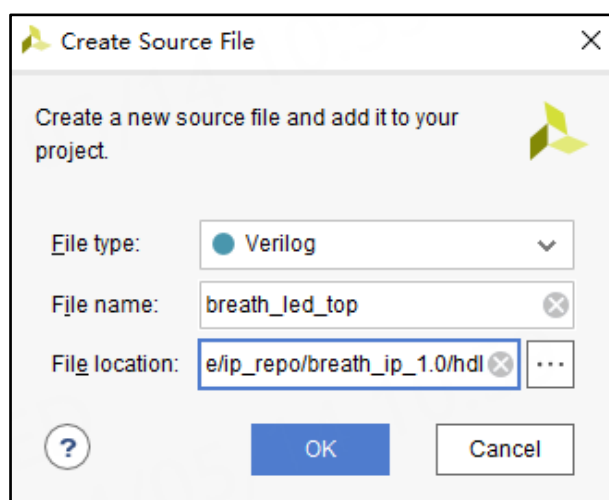


Figure 8-24 Create Source File

Then click "Finish", and then select "OK" and "YES" in the dialog boxes that pop up successively to complete the creation of `breath_led_top.v`. The following is the directory structure under Design Sources after generation.

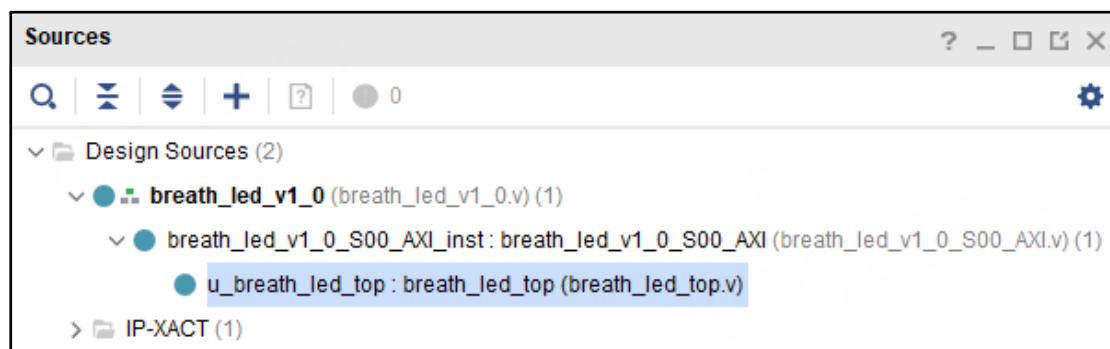


Figure 8-25 Design Sources structure

Click to open `breath_led_top.v` and enter the following code.

Code List 8-1 Breathing light module reference code (`breath_led_top.v`)

```

1 module breath_led_top
2 #(
3     parameter    BREATH_FREQ =    10'd100
4 )
5 (
6     input    wire        sys_clk        , //系统时钟
7     input    wire        rst_n          , //复位
8
9     //呼吸灯控制参数
10    input    wire        breath_led_en, //呼吸灯使能信号 1:打开 0:关闭
11    input    wire    [9:0]    freq_set    , //呼吸频率设置 10-1000
12
13    output    wire        led            //输出 led
14 );
15
16 //*****
17 /**                                reg define
18 //*****
19 reg [15:0]    cnt_1ms            ; //1ms 计数器
20 reg [9:0]     freq_set_reg        ; //呼吸灯频率寄存
21 reg [15:0]    high_duty_cnt        ; //高电平计数

```

```
22 reg duy_t_flag; //1 means the duty cycle is reduced, 0 means the duty cycle is increased 23 //*****
***** 24 //** assign code 25 //*****
***** 26 //Compare with the duty cycle only when the
breathing light is enabled, output pwm, otherwise output directly 0 27 assign led = (!breath_led_en)?1'b0
:(cnt_1ms <= high_duty_cnt)?1'b1
:1'b0;

28 //***** 29 //** always code 30
//***** 31 //Period count Generate
s the period of pwm wave 1ms 50_000 32 always @ (posedge sys_clk or negedge rst_n) begin 33 if (!rst_n) 34 cnt_1ms <= 16'd0; 35 else if (!breath_led_en) 36 cnt_1ms <= 16'd0; 37 else if (cnt_1ms == (50_000-1)) 38 cnt_1ms <= 16'd0; 39 else 40 cnt_1ms <= cnt_1ms + 16'd1; 41 end 42 43 //Change breathing light frequency minimum 10 maximum 1000 44 always @ (posedge sys_clk or negedge rst_n) begin 45 if (!rst_n) 46 freq_set_reg <= BREATH_FREQ; 47 else if (freq_set >= 1000) 48 freq_set_reg <= 1000; 49 else if (freq_set <= 10) 50 freq_set_reg <= 10; 51 else 52 freq_set_reg <= freq_set; 53 end 54 55 //Generate pwm wave 56 always @ (posedge sys_clk or negedge rst_n) begin 57 if (!rst_n) 58 begin 59 high_duty_cnt <= 16'd0; 60 duy_t_flag <= 1'b0; 61 end 62 else if (breath_led_en == 0) //The breathing light is not enabled. Set all to 0 63 begin 64 high_duty_cnt <= 16'd0; 65 duy_t_flag <= 1'b0; 66 end 67 else if (cnt_1ms == 16'd50_000-1) //Counting completes one cycle 68 begin 69 if (duy_t_flag) //Adjust the value and direction of the duty cycle. 1 means minus 70 begin 71 if (high_duty_cnt == 16'd0) 72 duy_t_flag <= 1'b0; 73 else if (high_duty_cnt < freq_set_reg) 74 high_duty_cnt <= 16'd0; 75 else 76 high_duty_cnt <= high_duty_cnt - freq_set_reg; 77 end 78 else //Duty cycle increases 79 begin 80 if (high_duty_cnt >= 16'd50_000-1) 81 duy_t_flag <= 1'b1; 82 else 83 high_duty_cnt <= high_duty_cnt + freq_set_reg; 84 end 85 end 86 else //Keep the duty cycle unchanged until one cycle is reached
```

```
87 high_duty_cnt <= high_duty_cnt; 88 end 89 90 91 endmodule
```

Find SYNTHESIS in the Flow Navigator navigation bar on the left, click "Run Synthesis" in this option, and verify whether the code has syntax errors.

After successful compilation, start setting the IP package and switch the interface to Package IP. If it is closed accidentally, you can reopen it through component.xml under the IP-XACT interface.

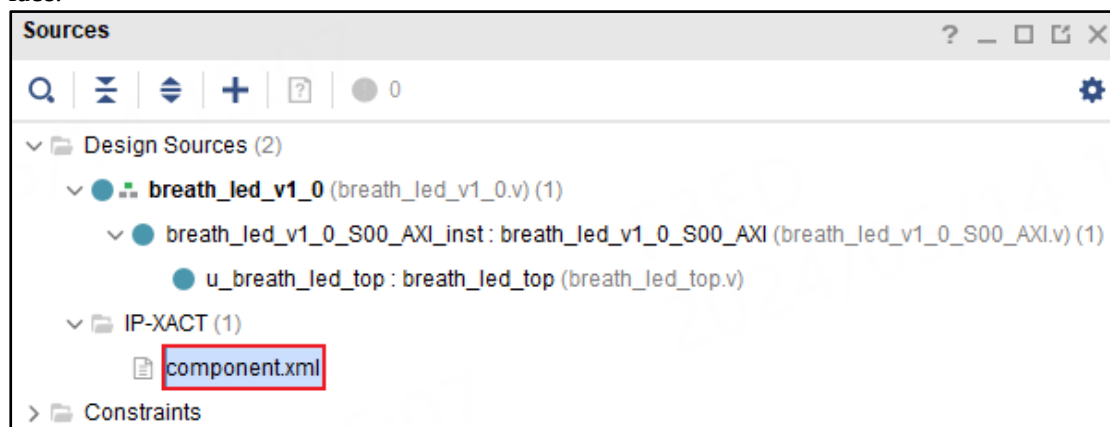


Figure 8-26 Open Package IP interface

Click Compatibility to modify the devices supported by this IP core. Click the "+" icon under the Family column and select "Add Family Explicitly...", as shown in Figure 8-27:



Figure 8-27 Compatibility interface

Check "zynq(zynq-7000)" here to indicate that the IP core supports zynq devices. Life-cycle indicates the current product life cycle of the IP core. Here, select "Pre-Production".

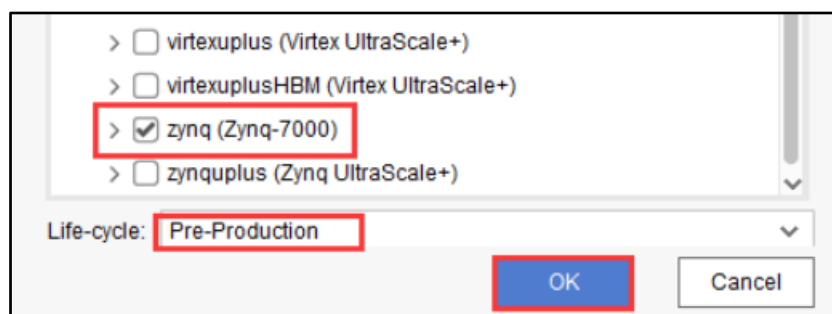


Figure 8-28 Select supported devices

Click on File Groups and then click on "Merge Changes from File Groups Wizard" on the interface.

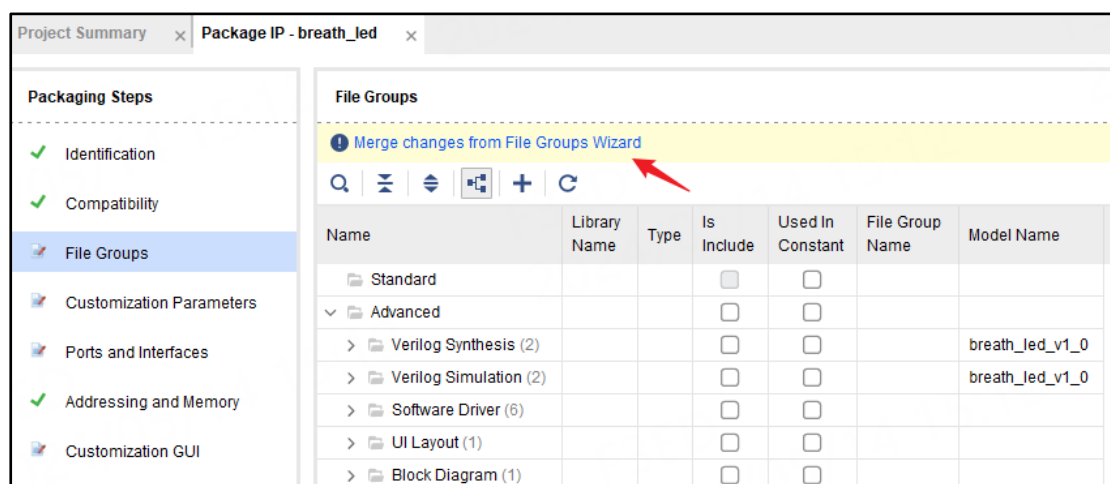


Figure 8-29 File Groups interface

Click Customization Parameters, click "Merge Changes from Customization Parameters Wizard".

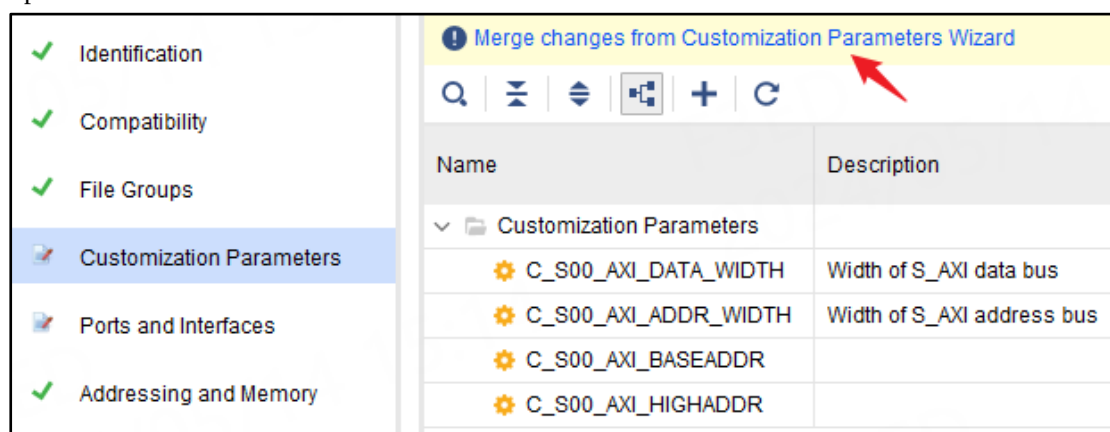


Figure 8-30 Customization Parameters interface

At this time, there is a Hidden Parameters column. Expand this interface and you can see the customized parameter BREATH_FREQ in the program. Right-click this parameter and select "Edit Parameter..." to pop up the parameter editing interface.

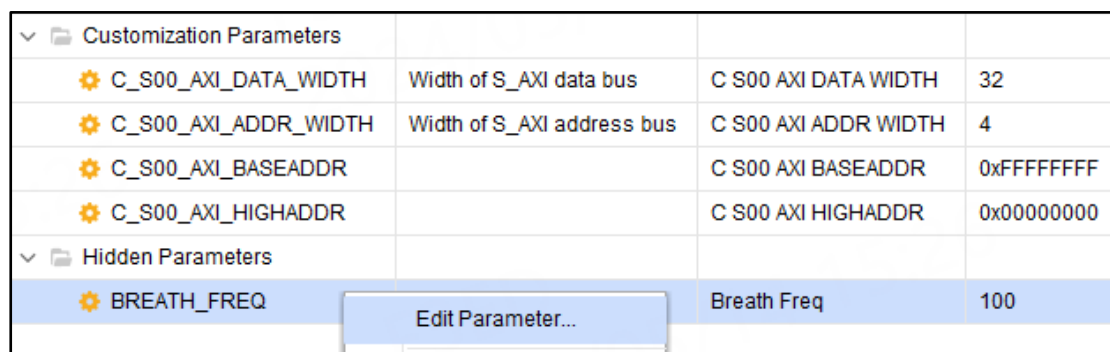
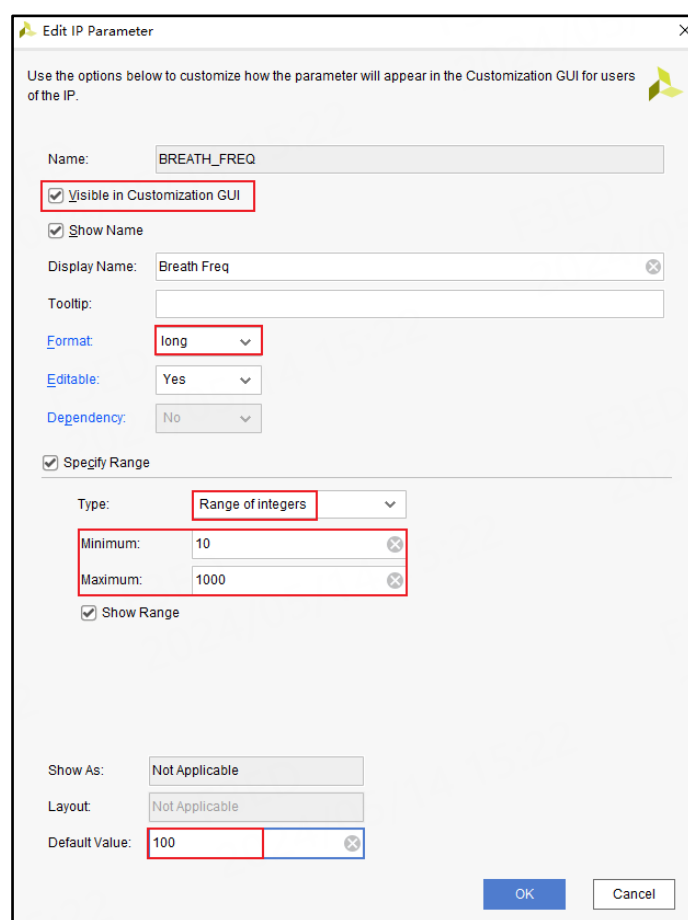


Figure 8-31 Open the parameter editing interface



The 'Edit IP Parameter' dialog box is used to customize the appearance of an IP parameter in the Customization GUI. It includes fields for Name, Display Name, Tooltip, Format, Editable, Dependency, and a Range specification section. The 'Visible in Customization GUI' checkbox is checked. The 'Specify Range' section is also checked, with 'Range of integers' selected as the Type, and Minimum and Maximum values set to 10 and 1000 respectively. The 'Default Value' is set to 100. The 'Show Range' checkbox is also checked. The 'Show As' and 'Layout' fields are set to 'Not Applicable'. The 'OK' and 'Cancel' buttons are at the bottom right.

Use the options below to customize how the parameter will appear in the Customization GUI for users of the IP.

Name: BREATH_FREQ

☒ Visible in Customization GUI

☒ Show Name

Display Name: Breath Freq

Tooltip:

Format: long

Editable: Yes

Dependency: No

☒ Specify Range

Type: Range of integers

Minimum: 10

Maximum: 1000

☒ Show Range

Show As: Not Applicable

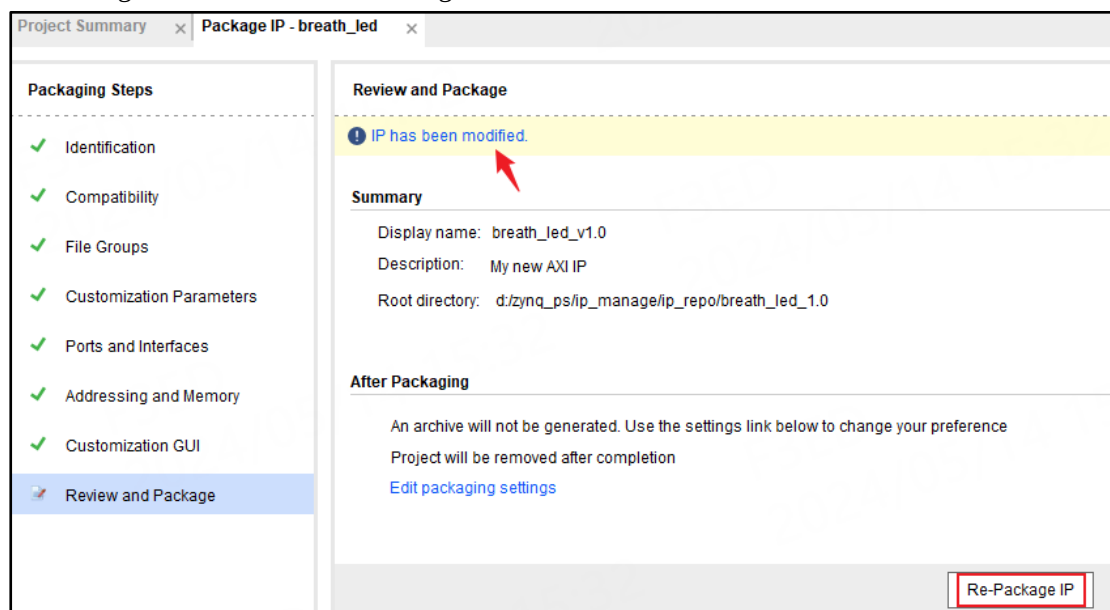
Layout: Not Applicable

Default Value: 100

OK Cancel

Figure 8-32 Parameter modification

Click "Review and Package", then click "IP has been modified" to update the summary interface, and finally click "Re-Package IP". Click "Yes" in the pop-up interface box to close the project of editing the IP core, as shown in Figure 8-33:



The 'Review and Package' interface shows the 'Package IP - breath_led' project. The 'Packaging Steps' list on the left includes Identification, Compatibility, File Groups, Customization Parameters, Ports and Interfaces, Addressing and Memory, Customization GUI, and Review and Package. The 'Review and Package' section on the right shows a summary of the IP core, including the display name, description, and root directory. A yellow banner at the top indicates 'IP has been modified'. The 'After Packaging' section provides information about the archive generation and project removal. The 'Re-Package IP' button is highlighted at the bottom right.

Project Summary x Package IP - breath_led x

Packaging Steps

- ✓ Identification
- ✓ Compatibility
- ✓ File Groups
- ✓ Customization Parameters
- ✓ Ports and Interfaces
- ✓ Addressing and Memory
- ✓ Customization GUI
- Review and Package

Review and Package

IP has been modified.

Summary

Display name: breath_led_v1.0

Description: My new AXI IP

Root directory: d:/zynq_ps/ip_manage/ip_repo/breath_led_1.0

After Packaging

An archive will not be generated. Use the settings link below to change your preference

Project will be removed after completion

[Edit packaging settings](#)

Re-Package IP

Figure 8-33 Review and Package interface

After the IP core encapsulation is completed, the Vivado software automatically generates .c and .h files to facilitate the operation of the IP core in the SDK software. The file is in the IP core path ...ip_manage\ip_repo\breath_led_1.0\drivers\breath_led_v1_0\src.

名称	类型	大小
 breath_led.c	C 源文件	1 KB
 breath_led.h	C Header 源文件	3 KB

Figure 8-34 IP file generated by the software

In the Manage IP project interface, click "File" on the menu bar, select "Close Project" to close the project, and the custom IP core is successfully generated.

Step3: Create vivado project

Previously, we only encapsulated the PL resources we need into the ip module. Next, we need to create a new project to call the IP core.

If readers are unfamiliar with how to create a project, please refer to the "MIO Control LED" chapter. After we create the project, we need to add the customized IP core to the IP library of this project. Click "Tools" on the menu bar and select "Setting".

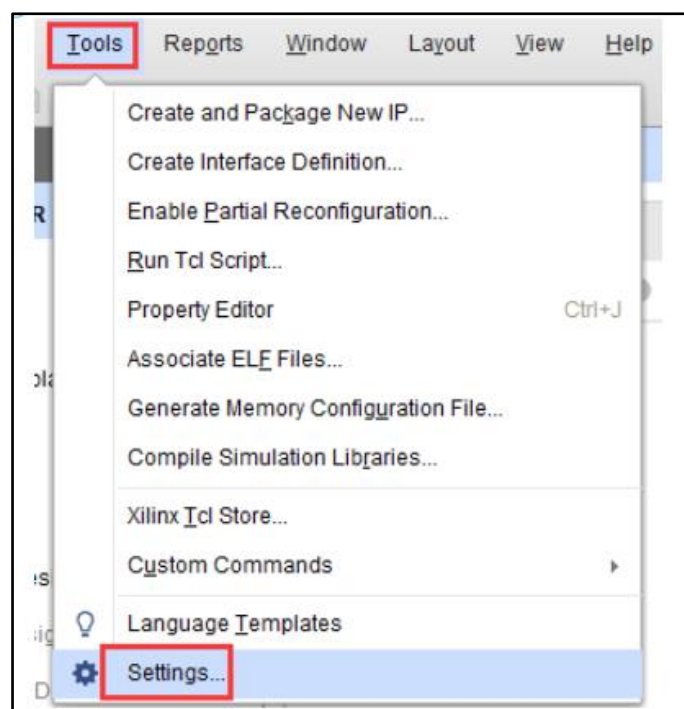


Figure 8-35 Open the setting interface

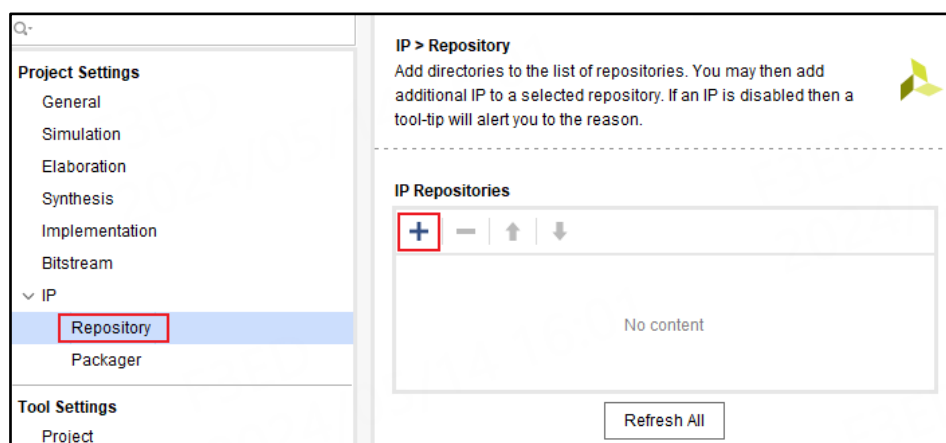


Figure 8-36 IP library interface

In the pop-up interface, select the path where the custom IP core is located "...ip_manage ip_repo\breath_led_1.0". The pop-up interface is shown in Figure 8-37:

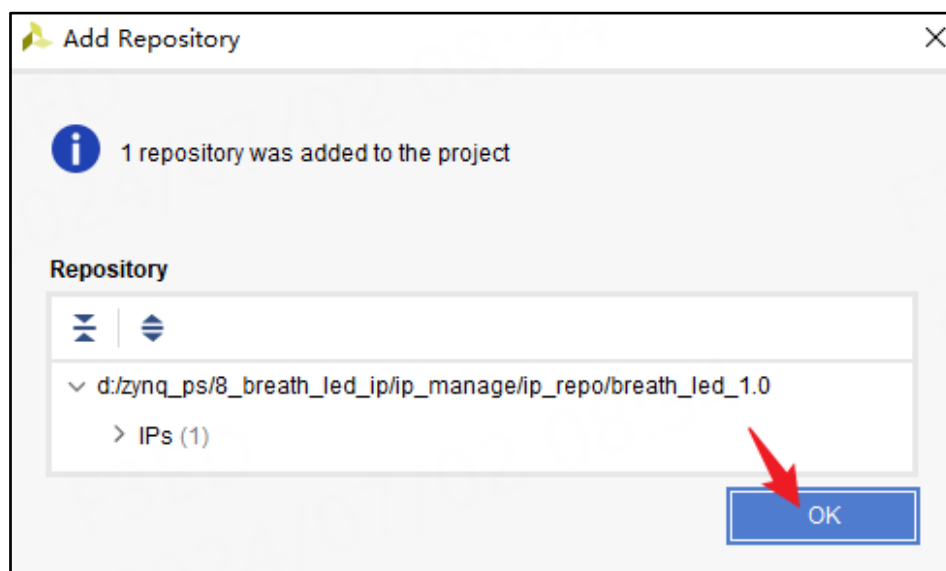


Figure 8-37 Add IP core

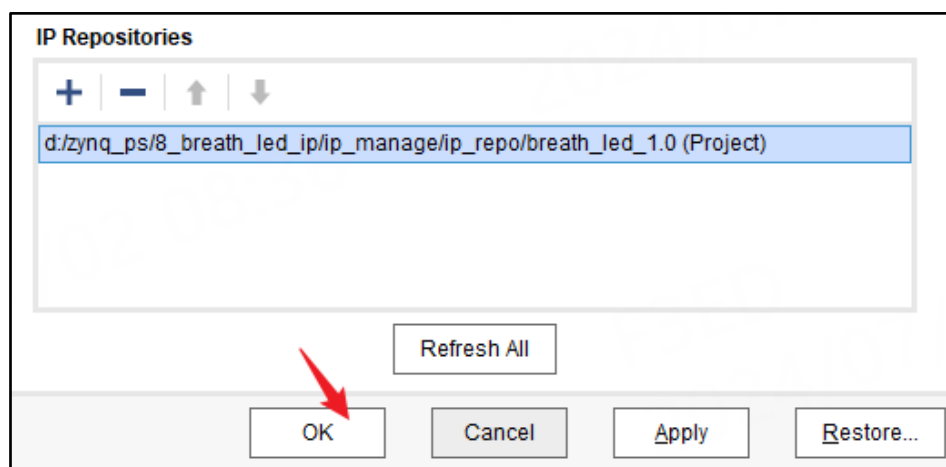


Figure 8-38 Complete IP core addition

Step4: Use IP Integrator to create a Processing System

To create a project, refer to the "MIO Control LED" chapter. However, unlike the previous construction of the embedded minimum system, we retain the FCLK_CLK0, FCLK_RESET0_N and M_AXI_GP0_ACLK interfaces, which is equivalent to only adding the UART serial port and modifying the DDR model after creating a new ZYNQ7 processing system, and keeping the others as default.

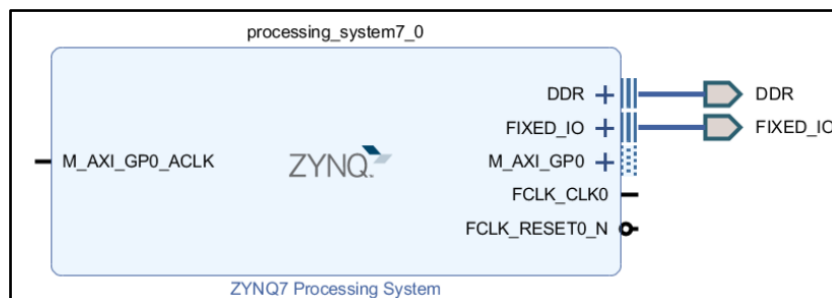


Figure 8-39 ZYNQ7 block diagram

Next, add the breath_led IP core, click the "+" icon, enter "breath" in the search box, double-click to select it and add it. Right-click the LED pin of the module, select "Make External", and manually extract the LED output pin.

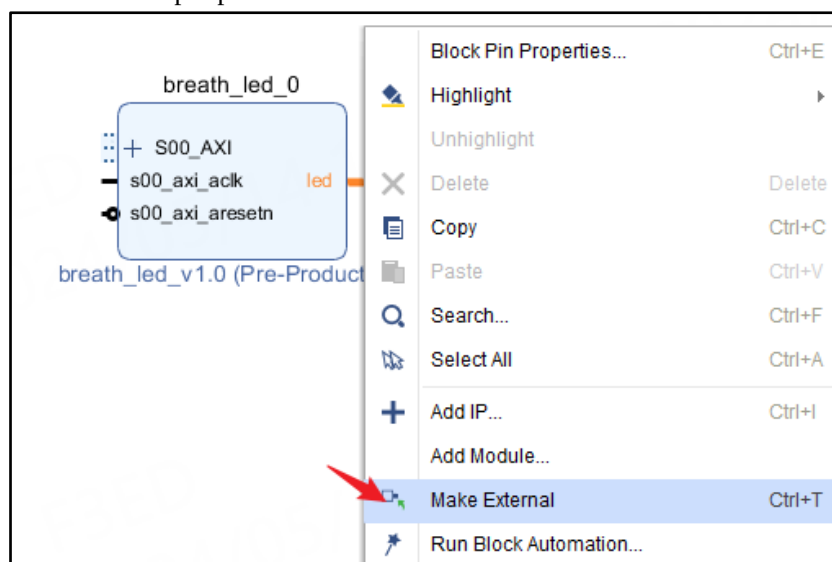


Figure 8-40 Leading out the LED pins

Next, click "Run Connection Automation" to automatically connect the wiring. If the overall block diagram does not look good after wiring, you can right-click on the blank space, select Regenerate Layout, and re-layout the wiring, as shown in Figure 8-41:

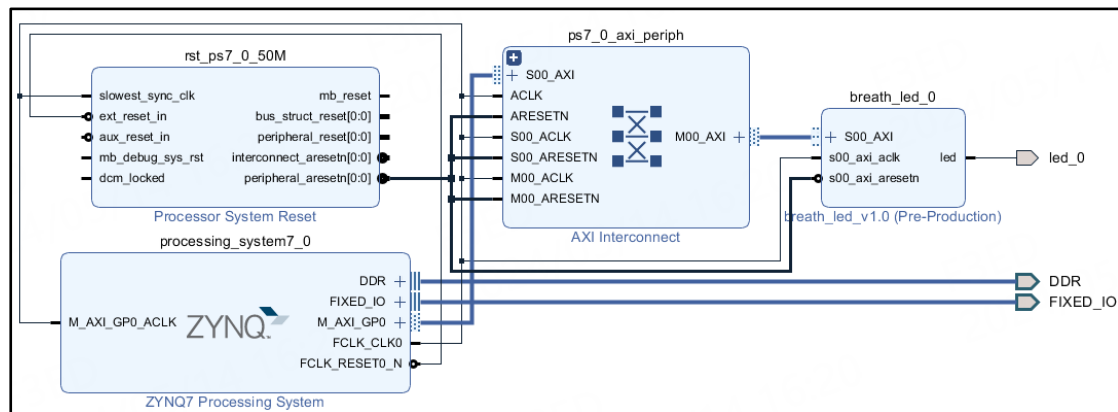


Figure 8-41 Overall block diagram after rearrangement

Step5: Generate top-level HDL module

After verifying that the design is correct, press ctrl + s to save the design, and then operate as shown in Figure 8-42

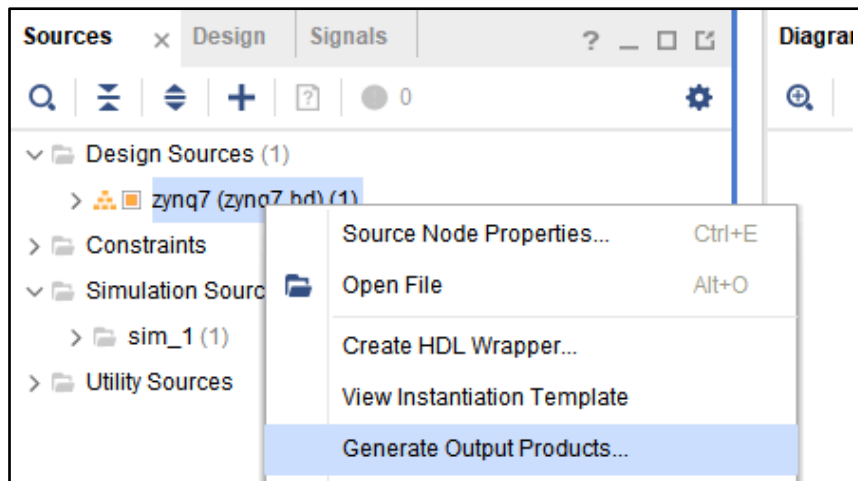


Figure 8-42 Select Generate Output Products

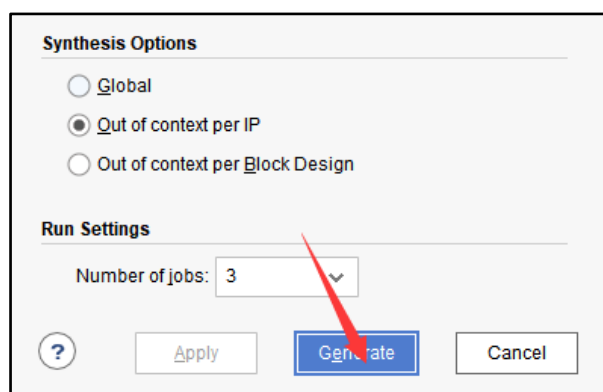


Figure 8-43 Generate

Then right-click zynq7.bd again in the "Hierarchy" tab and select "Create HDL Wrapper".

Step6: Generate Bitstream file

Pin binding is performed first, and then the bit stream file is generated.

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
All ports (131)						
> DDR_35690 (71)	INOUT			✓	502	(Multiple)*
> FIXED_IO_35690 (59)	INOUT			✓	(Multiple)	(Multiple)*
Scalar ports (1)						
breath_led	OUT		K16	✓	35	LVC MOS33*

Figure 8-44 Pin binding

Step7: Export SDK

Select File > Export > Export hardware in the menu bar. In the pop-up dialog box, check "Include bitstream" and click the "OK" button.

Then select File > Launch SDK in the menu bar to start the SDK development environment.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and

Code List 8-2 PS control breathing light reference code (main.c)

```
1 #include "stdio.h" 2 #include "xparameters.h" 3 #include "xil_printf.h" 4 #include "breath_led.h" 5 #include
"xil_io.h" 6 #include "sleep.h" 7 8 //led base address 9 #define LED_IP_BASEADDR XPAR_BREATH_LED
_0_S00_AXI_BASEADDR 10 //led register address 0 11 #define LED_IP_REG0 BREATH_LED_S00_AXI_
SLV_REG0_OFFSET 12 //led register address 1 13 #define LED_IP_REG1 BREATH_LED_S00_AXI_SLV_
REG1_OFFSET 14 15 //main function 16 int main() 17 { 18 xil_printf("This is breath led\r\n"); 19 //Turn on th
e breathing light and enable frequency control 20 BREATH_LED_mWriteReg(LED_IP_BASEADDR, LED_I
P_REG0, 0x00000001); 21 //Set the frequency 22 BREATH_LED_mWriteReg(LED_IP_BASEADDR, LED_I
P_REG1, (0x000003FF & 50)); 23 while (1) { 24 25 } 26 }
```

In line 4 of the code, you can see that we have introduced the header file `breath_led.h`. This is a header file generated based on our customized IP. It can be found in the generated board-level support package, that is, bsp, as shown in the figure below:

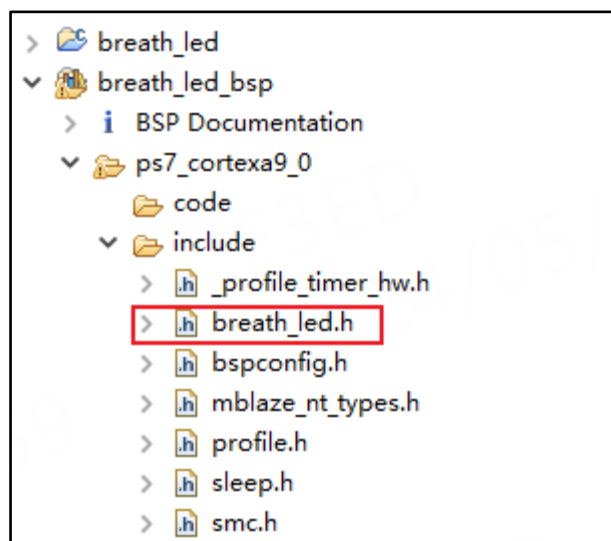


Figure 8-45 breath_led.h

Double-click to open the breath_led.h file, and you can see that four register addresses are declared. Because one is 32 bits, which is four bytes, the address deviation between each one is 4. It also provides a function for writing registers and a function for reading registers below. Readers can open the file to check it themselves.

```
/****** Include Files *****/
#include "xil_types.h"
#include "xstatus.h"

#define BREATH_LED_S00_AXI_SLV_REG0_OFFSET 0
#define BREATH_LED_S00_AXI_SLV_REG1_OFFSET 4
#define BREATH_LED_S00_AXI_SLV_REG2_OFFSET 8
#define BREATH_LED_S00_AXI_SLV_REG3_OFFSET 12

/****** Type Definitions *****/
/**
 *
 * Write a value to a BREATH_LED register. A 32 bit write is performed.
 * If the component is implemented in a smaller width, only the least
 * significant data is written.
 *
 * @param BaseAddress is the base address of the BREATH_LED device.
 * @param RegOffset is the register offset from the base to write to.
 * @param Data is the data written to the register.
 *
 * @return None.
 *
 * @note
 * C-style signature:
 * void BREATH_LED_mWriteReg(u32 BaseAddress, unsigned RegOffset, u32 Data)
 *
 */
#define BREATH_LED_mWriteReg(BaseAddress, RegOffset, Data) \
    Xil_Out32((BaseAddress) + (RegOffset), (u32)(Data))

/**
 *
 * Read a value from a BREATH_LED register. A 32 bit read is performed.
 */
```

Figure 8-46 Content diagram

Next, look at the eighth line of the code, which defines the base address of our IP. There are two ways to find this address. The first method can be found in xparameters.h, as shown below:

```
/* Definitions for driver BREATH_LED */
#define XPAR_BREATH_LED_NUM_INSTANCES 1

/* Definitions for peripheral BREATH_LED_0 */
#define XPAR_BREATH_LED_0_DEVICE_ID 0
#define XPAR_BREATH_LED_0_S00_AXI_BASEADDR 0x43C00000
#define XPAR_BREATH_LED_0_S00_AXI_HIGHADDR 0x43C0FFFF
```

Figure 8-47 IP related definitions

The XPAR_BREATH_LED_0_S00_AXI_BASEADDR here is our base address, which we can also find in Vivado. This is the second method, as shown below:

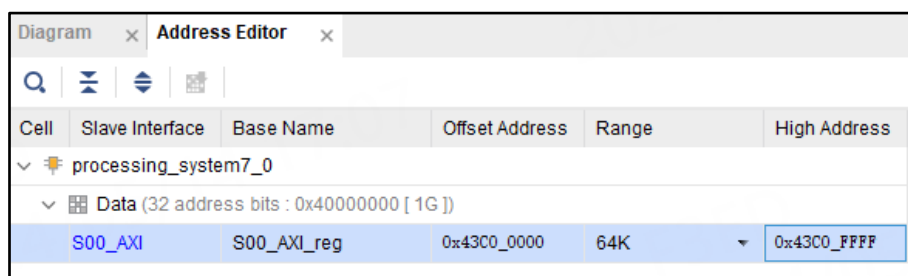


Figure 8-48 address editor

It can be seen that it corresponds to the SDK. Note here that the addresses of Vivado and SDK must correspond. Next we look at the main function. It is actually very simple. It uses the provided write operation function to start the IP and set the frequency of the breathing light. The higher the frequency, the faster the breathing rate.

After writing the code, start compiling the project and prepare for board verification.

8.3.4 Board verification

After correctly connecting the Haoyue development board to the computer, right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system and Program FPGA -> Run.

After the program is downloaded, PL LED1 on the development board switches states in a loop with a breathing effect.

8.4 End of chapter summary

This chapter introduces how to package PL's breathing lamp module into an IP core and add it to our project. PS can access the IP core through the AXI interface and configure the breathing light IP core, such as enabling the breathing light and the frequency of the breathing light.

8.5 Development training

On the Haoyue development board, PL has LED1~LED4. Through pin constraints, they can all be used as breathing lights. You can use your imagination to control these LEDs through programs to create better effects.

Chapter 9 Program Solidification Experiment

9.1 Chapter Introduction

In the previous experiments, we downloaded configuration files and applications to the ZYNQ chip through the JTAG interface. In this way, once the development board is powered off, the program will disappear and the program needs to be downloaded again to implement the corresponding functions. In this chapter, we learn to store the program in non-volatile memory so that it can be automatically loaded and executed at startup. This process requires the participation of the boot loader.

9.2 Theoretical study

During the startup and configuration process of ZYNQ SoC, the processor system (PS) is responsible for starting and initializing the entire system, including the dynamic configuration of programmable logic (PL). By loading the PL configuration bitstream file, the logic of the PL can be dynamically configured according to application requirements, thereby achieving system flexibility and function customization. In some cases, you can choose to power up the PS side but not the PL side to achieve power optimization.

The PS side first executes the startup code on the BootROM. This code is directly executed by XILINX in the On Chip ROM. It cannot be seen or modified, so the BootROM is the first piece of code executed by the APU after power on. This is the part of the BootROM. ZYNQ-7000 is a dual-core Cortex-A9 with two CPU cores. The BootROM is executed by CPU_0. At this moment, CPU_1 is in the WFE waiting state; BootROM The main task is to configure the system, copy the boot image FSBL from the boot device to OCM, then jump to FSBL, transfer control, and complete its mission; (of course, in non-safe mode, on-chip XIP operation is also supported, that is, running code directly on QSPI Flash or NOR Flash).

The boot device (non-volatile memory) can also store the BIT file that configures the FPGA and the software code ELF file of the PS-side program, but these cannot be accessed by the BootROM code. The non-volatile memory used by ZYNQ PS for booting supports Quad-SPI, NAND, NOR, or SD cards. Haoyue ZYNQ development board hardware supports SD card and QSPI as configuration devices.

FSBL is generated by compiling our code. After the compilation is completed, the bin startup file is generated and this bin is placed in our startup device.

The FSBL stage has four operations:

- (1) Use PS7 Init data initialization PS (MIO, DD) generated by Vivado; (2) If there is a configuration file on the PL side, the bit stream will be loaded to complete the configuration of the PL side; (3) SSBL (second-stage boot download program), or load the ARM (bare metal) program into the DDR memory ;
- (4) Jump to SSBL to perform operations, or jump to DDR to run code;

PL 的配置是通过处理器配置访问接口 (Processor Configuration Access Port, PCAP) 进行的。The PCAP interface allows the processor system (PS side) to load the configuration bitstream (bitstream) to the FPGA logic (PL side), allowing partial or complete configuration of the PL side to realize the function of dynamically configuring the PL. This dynamic configuration capability enables ZYNQ SoC to reconfigure FPGA logic at runtime, enabling flexible processor programmable logic co-design.

When ZYNQ is powered on, the PS_POR_B signal will be set. At this moment, the ZYNQ hardware immediately samples a group of pins MIO[8:2] called Boot Strap Pins. This group of pins tells the hardware how to start (QSPI Flash, NAND Flash, NOR Flash, SD Card), and whether to turn on the PLL, etc. (the high and low levels of each pin represent the 0/1 signal).

Chapter 6: Boot and Configuration

Table 6-4: Boot Mode MIO Strapping Pins

Pin-signal / Mode	MIO[8]	MIO[7]	MIO[6]	MIO[5]	MIO[4]	MIO[3]	MIO[2]
	VMODE[1]	VMODE[0]	BOOT_MODE[4]	BOOT_MODE[0]	BOOT_MODE[2]	BOOT_MODE[1]	BOOT_MODE[3]
Boot Devices							
JTAG Boot Mode; cascaded is most common ⁽¹⁾				0	0	0	JTAG Chain Routing ⁽²⁾ 0: Cascade mode 1: Independent mode
NOR Boot ⁽³⁾				0	0	1	
NAND				0	1	0	
Quad-SPI ⁽³⁾				1	0	0	
SD Card				1	1	0	
Mode for all 3 PLLs							
PLL Enabled			0	Hardware waits for PLL to lock, then executes BootROM.			
PLL Bypassed			1	Allows for a wide PS_CLK frequency range.			
MIO Bank Voltage ⁽⁴⁾							
	Bank 1	Bank 0	Voltage Bank 0 includes MIO pins 0 thru 15. Voltage Bank 1 includes MIO pins 16 thru 53.				
2.5 V, 3.3 V	0	0					
1.8 V	1	1					

Notes:

- JTAG cascaded mode is most common and is the assumed mode in all the references to JTAG mode except where noted.
- For secure mode, JTAG is not enabled and MIO[2] is ignored.
- The Quad-SPI and NOR boot modes support execute-in-place (this support is always non-secure)
- Voltage Banks 0 and 1 must be set to the same value when an interface spans across these voltage banks. Examples include NOR, 16-bit NAND, and a wide TPIU test port. Other interface configuration may also span the two banks.

Figure 9-1 Startup mode

The startup process of the entire system is shown in the figure below:

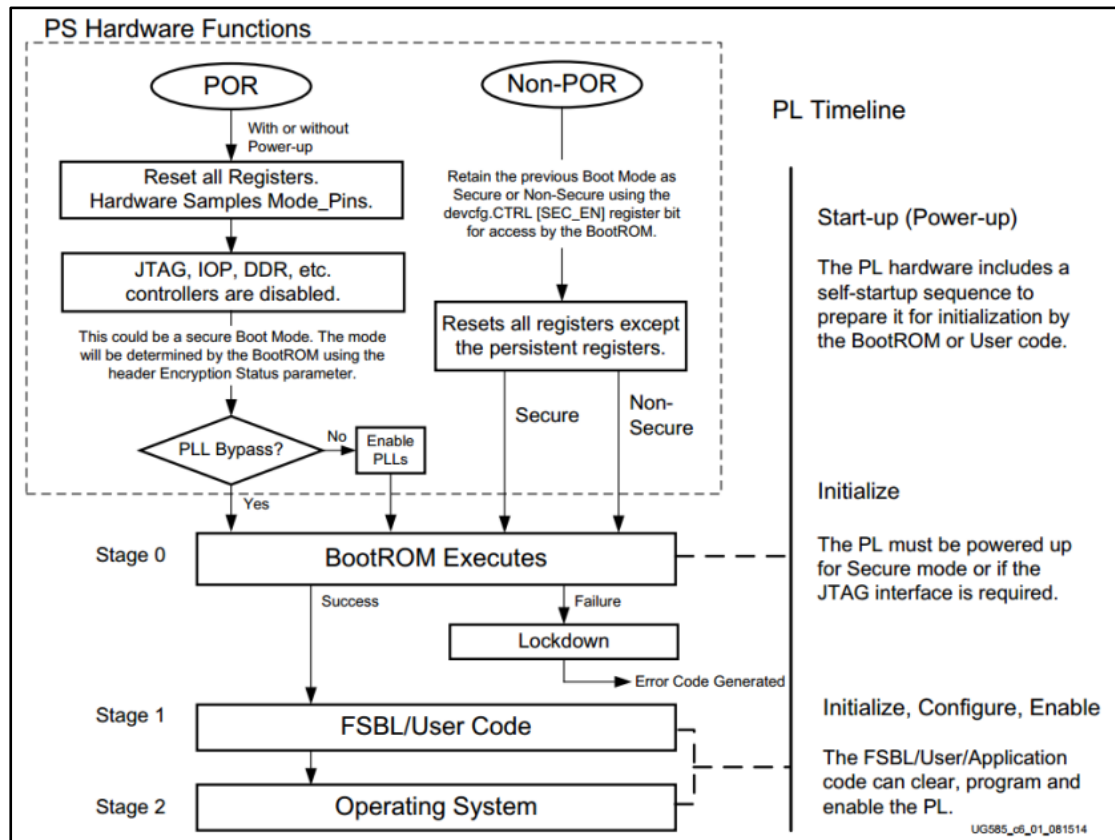


Figure 9-2 ZYNQ system startup process

9.3 Practical exercises

9.3.1 Experimental objectives

The experimental task of this chapter is to create FSBL based on the "AXI GPIO key interrupt control LED" experiment to realize the program self-starting after power-on, including starting from the SD card and starting from the QSPI Flash.

9.3.2 Hardware resources

Refer to the chapter "AXI GPIO Button Interrupt Control LED".

9.3.3 Programming

1. Hardware design

This experiment is carried out on the basis of "AXI GPIO button interrupt control LED", so we first open the project of this experiment. We want to realize the power-on self-start of the program, so enable QSPI and SD controller peripherals.

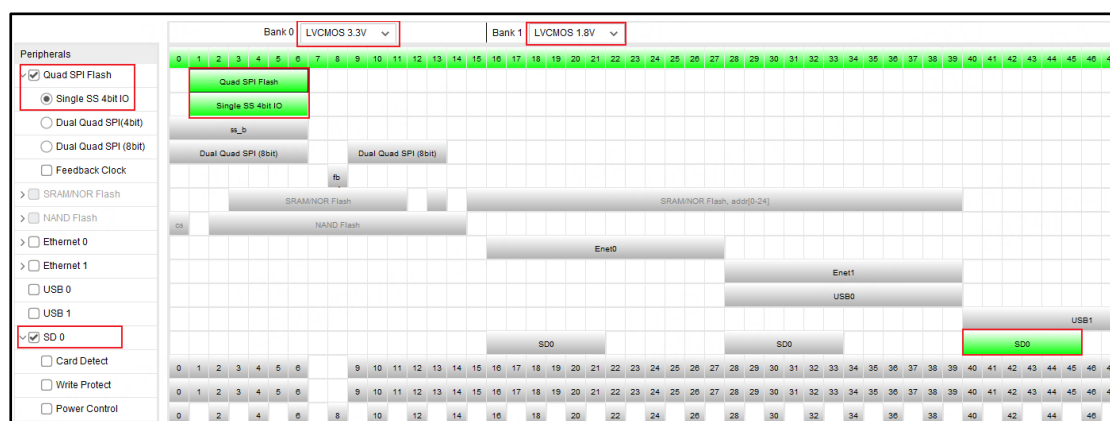


Figure 9-3 Enabling QSPI-Flash and SD controller

The specific MIO pins and bank voltages connected to each peripheral in Figure 9-3 can be known by viewing the schematic diagram of the Haoyue development board. Then save the design, follow the steps related to the previous project, generate the bit flow file, and export the hardware.

2. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and then directly transplanted the previous AXI_GPIO key interrupt control LED code. After compilation, you can download it to the board first to see if the phenomenon is normal. Then we open the board-level support package configuration, as shown below:

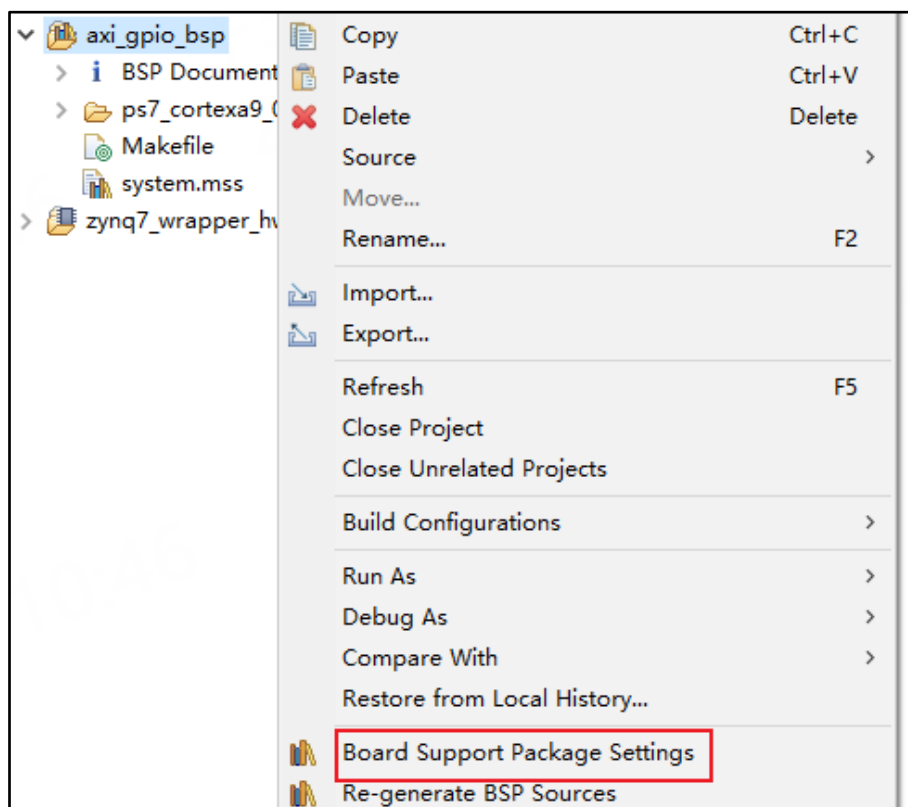


Figure 9-4 Open board-level support package configuration

Then check xilffs to read and write SD card and FLASH.

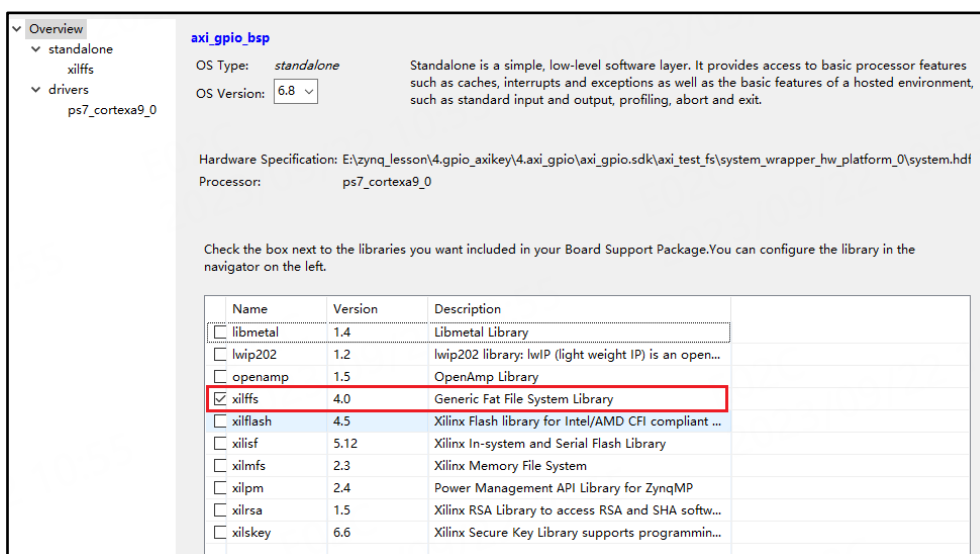


Figure 9-5 Check xilffs

Next, we create a new fsbl project to configure the PL side and load the software code. Xilinx provides us with a template for this project. We can use it directly. Make sure to check use existing to use the current support package.

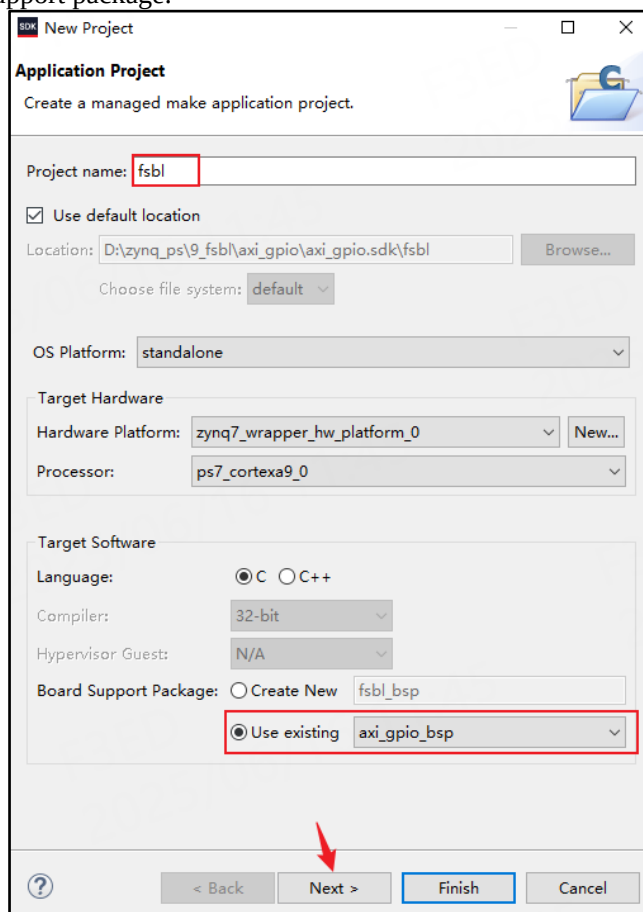


Figure 9-6 Create a new fsbl project

Then select ZYNQ FSBL.

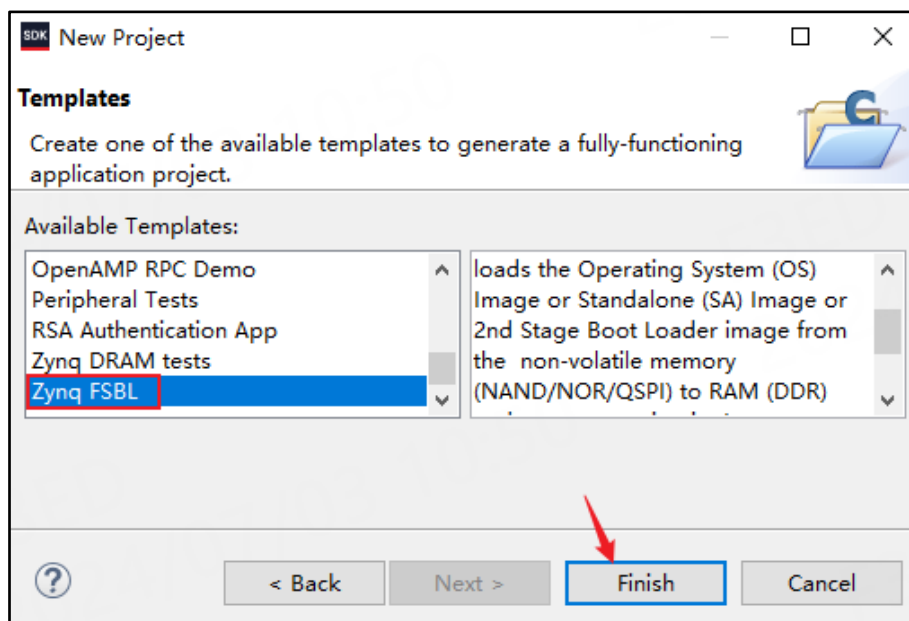


Figure 9-7 Select ZYNQ FSBL

After waiting for compilation, right-click the axi_gpio project, and then click to select "Creat Boot Image".

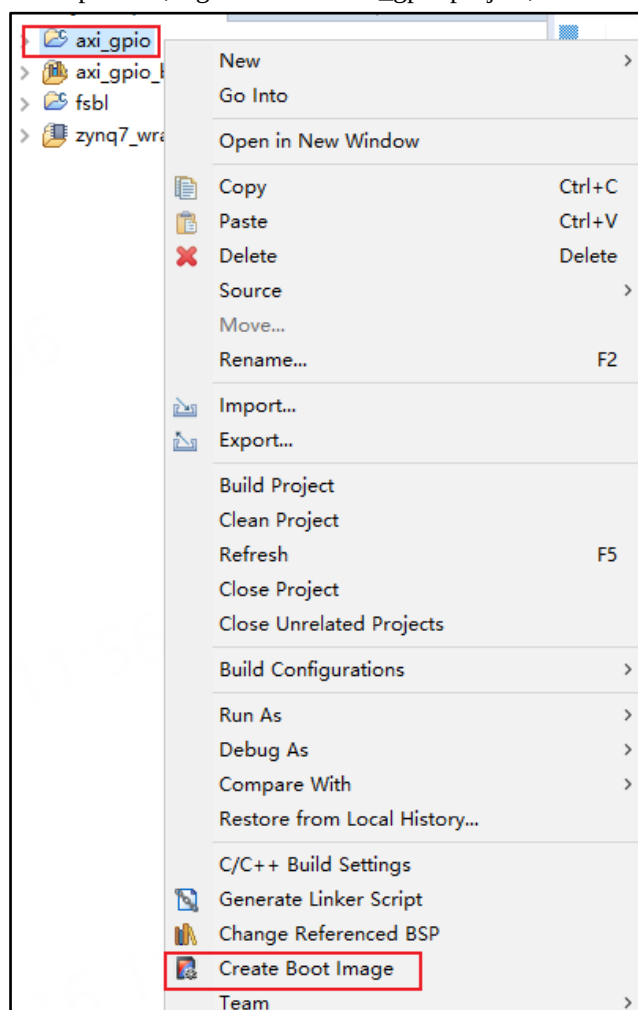


Figure 9-8 Select Create Boot Image

Click "Create Image" in the pop-up Create Boot Image interface.

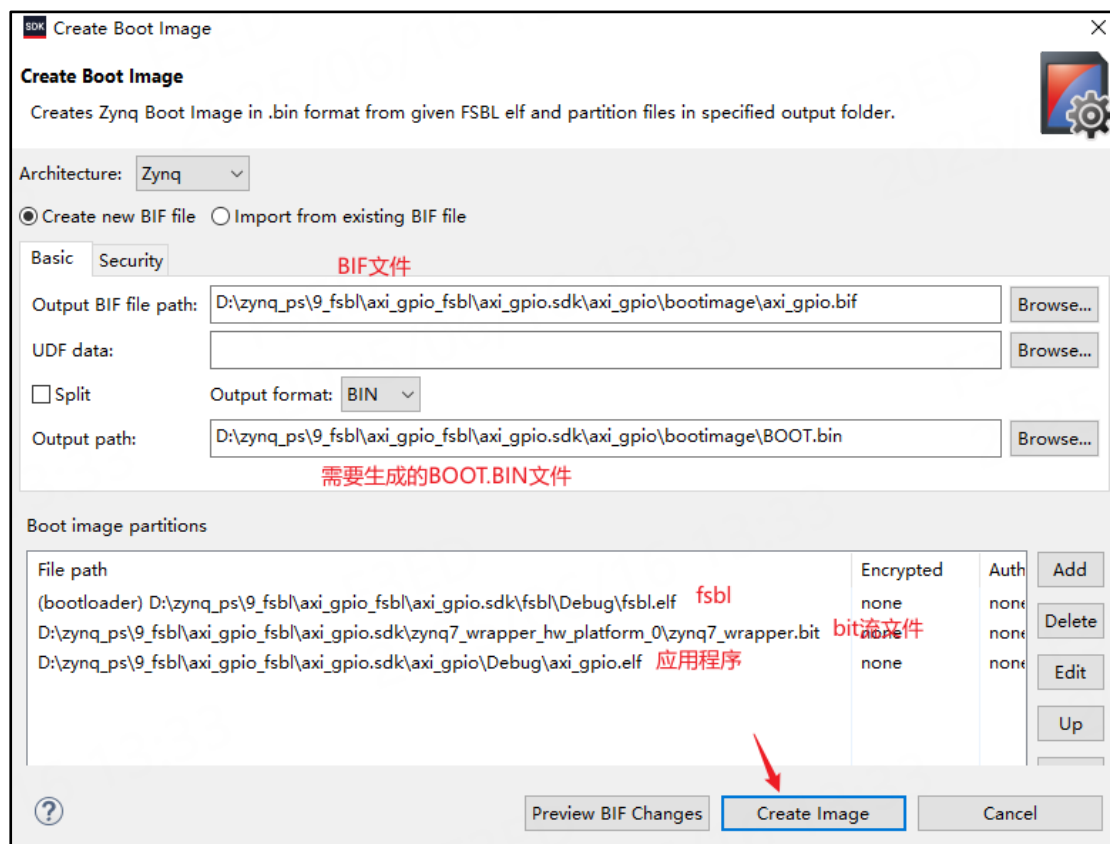


Figure 9-9 Create Boot Image

Select Create Image and the operation is completed. We go to the folder to find the corresponding file. BOOT.bin is the file we need.



Figure 9-10 Generated file

9.3.4 Board verification

Method one:

Boot via SD card. We can prepare an SD card (prerequisite is the FAT 16/32 file system), then copy the generated BOOT.bin file to the SD card, set the boot mode switches MIO4 and MIO5 of the development board to 1, 1, and select the SD card to boot. And insert the SD card into the SD card slot of the development board. Then power on.

Method two:

Solidify into QSPI Flash, the startup mode here remains unchanged, continue to use JTAG mode, select Xilinx->Program Flash in the upper menu bar.

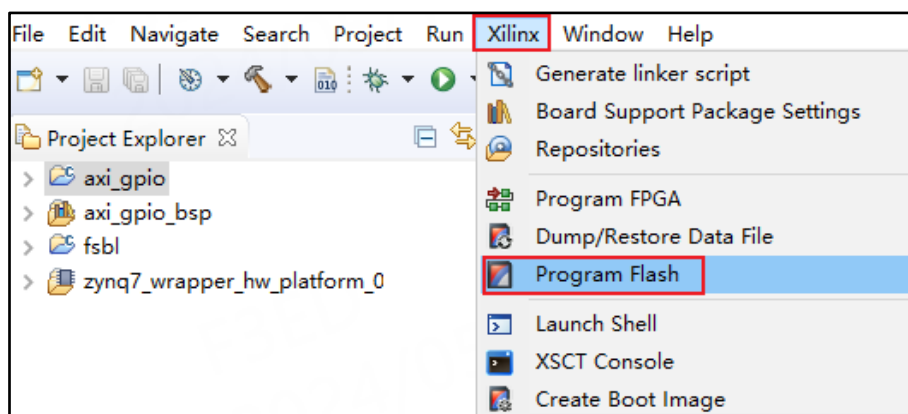


Figure 9-11 Program Flash

Then in the pop-up tab, select the BOOT.bin file just generated under Image File, and select the FSBL.elf file of the FSBL project under FSBL File. Then click program and wait for a while.

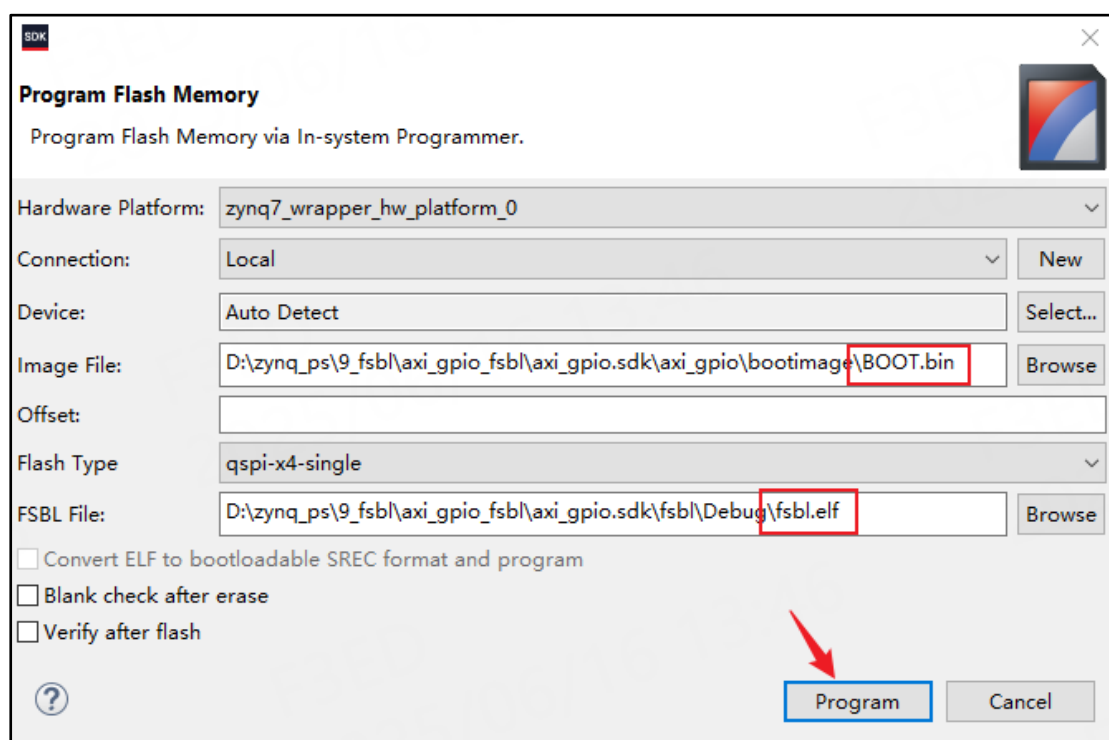
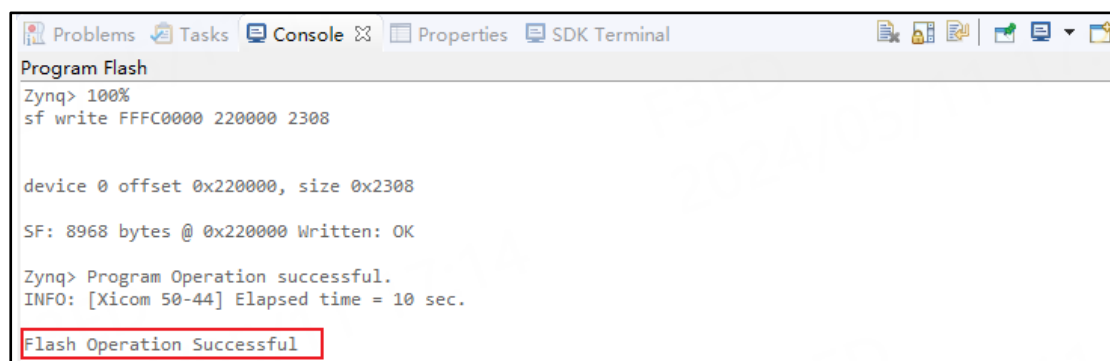


Figure 9-12 File selection

Curing is complete, as shown in the picture below:



```
Problems Tasks Console Properties SDK Terminal
Program Flash
Zynq> 100%
sf write FFFC0000 220000 2308

device 0 offset 0x220000, size 0x2308

SF: 8968 bytes @ 0x220000 Written: OK

Zynq> Program Operation successful.
INFO: [Xicom 50-44] Elapsed time = 10 sec.
Flash Operation Successful
```

Figure 9-13 Complete program

As shown in the picture above, the download has been completed. Turn off the power, adjust the startup mode, select MIO4 as 0, MIO5 as 1, and boot from FLASH. You can refer to the experiment in the chapter "AXI GPIO Button Interrupt Control LED" to test whether the program is cured successfully. If the experimental phenomena are consistent, it is successful.

9.4 End of chapter summary

This chapter briefly introduces the ZYNQ processor system startup process, and solidifies the program of the "AXI GPIO button interrupt control LED" experiment to FLASH and SD cards, realizing automatic loading and execution of the program during power-on and reset. In terms of hardware design, we added QSPI and SD controller; in the SDK, we created a new application project and transplanted the code. We also added an fsbl project and finally generated the BOOT.bin file.

9.5 Development training

Through the study of this chapter, you can solidify the experiments in the previous chapters into FLASH or SD card. You can try it and become familiar with the solidification process of the ZYNQ program.

Chapter 10 PS Reading XADC Experiment

10.1 Chapter Introduction

XADC, digital-analog hybrid module, is the hard core located on the PL side. XADC has two analog-to-digital converters with 12-bit resolution and a sampling rate of 1 MSPS (1 Mega sample per second). An analog multiplexer includes dedicated analog input VP/VN and 16 groups of multiplexed analog signal inputs VAUX[15:0] (analog input voltage unipolar 0-1V; bipolar -0.5-0.5V). It has built-in voltage and temperature sensors to monitor the voltage and temperature information of the chip.

10.2 Theoretical study

Let's learn about the internal structure of XADC.

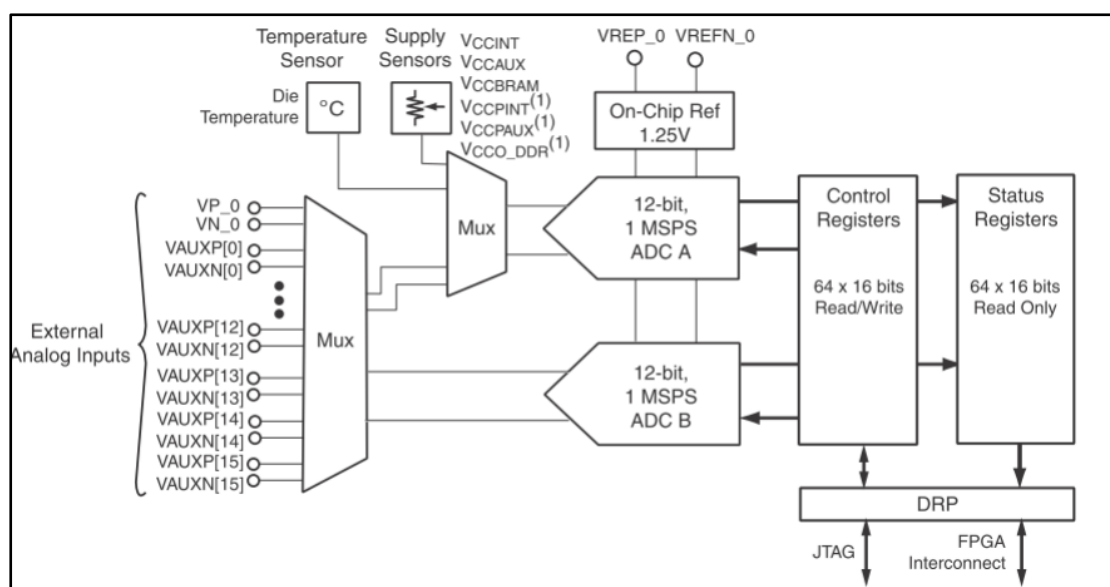


Figure 10-1 XADC block diagram

It can be seen from the figure that the ADC is divided into two 12-bit A and B, so it is called double 12-bit. ADC A can sample the supply voltage, which includes VCCINT, VCCAUX, and VCCBRAM. The Zynq-7000 series of chips also supports sampling of VCCPINT, VCCPAUX, and VCCO_DDR, as well as temperature sampling and external analog input sampling. ADC B can only sample and convert external analog inputs.

It should be noted that although the XADC hard core has 17 external inputs, our Haoyue development board only leads to the dedicated analog input VP/VN. The XADC internally includes a 16-bit control register and a status register. The control register can be read and written through DRP (DynamicReconfiguration Port) to achieve the initial configuration of the XADC, while the status register can only be read. The ADC The sampled converted value is saved in the corresponding status register and can be read out through DRP, so that it can be used outside the FPGA. For specific operations on registers, please refer to the relevant manuals.

We can read sensor information values in three ways. The first is to connect the development board to the downloader, then turn on the power, and then open Vivado, as shown in the figure:

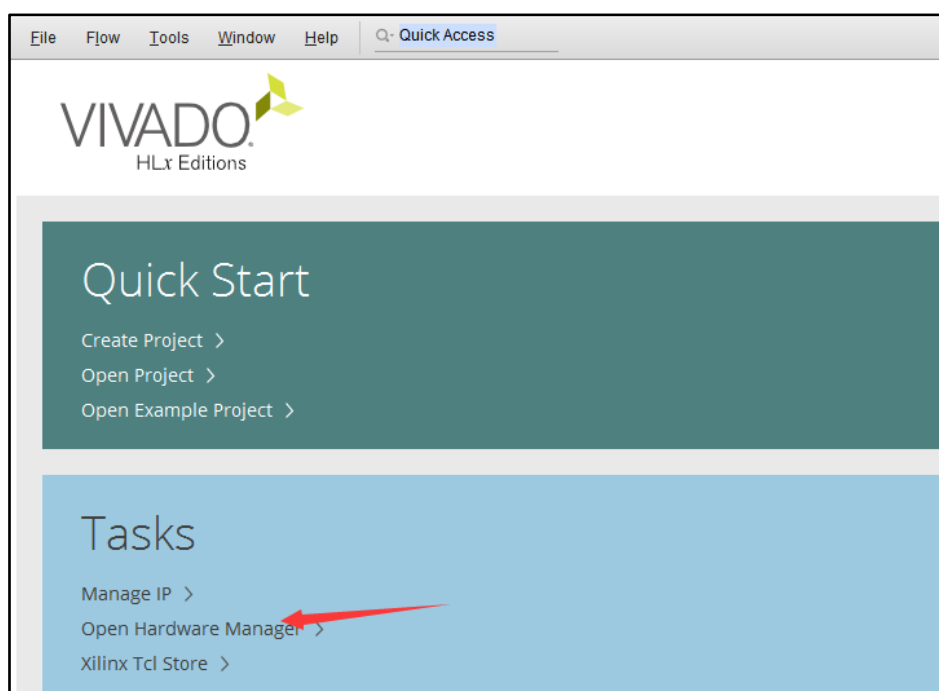


Figure 10-2 Open Hardware Manager

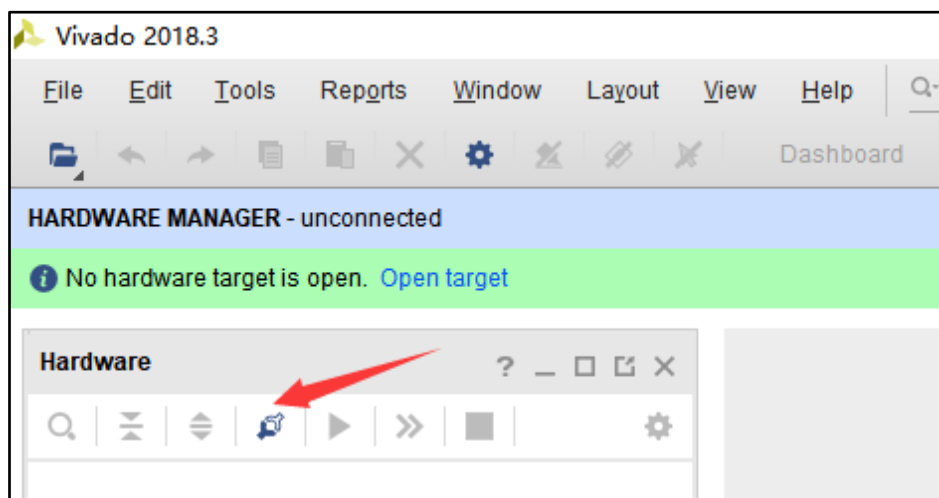


Figure 10-3 Automatic connection

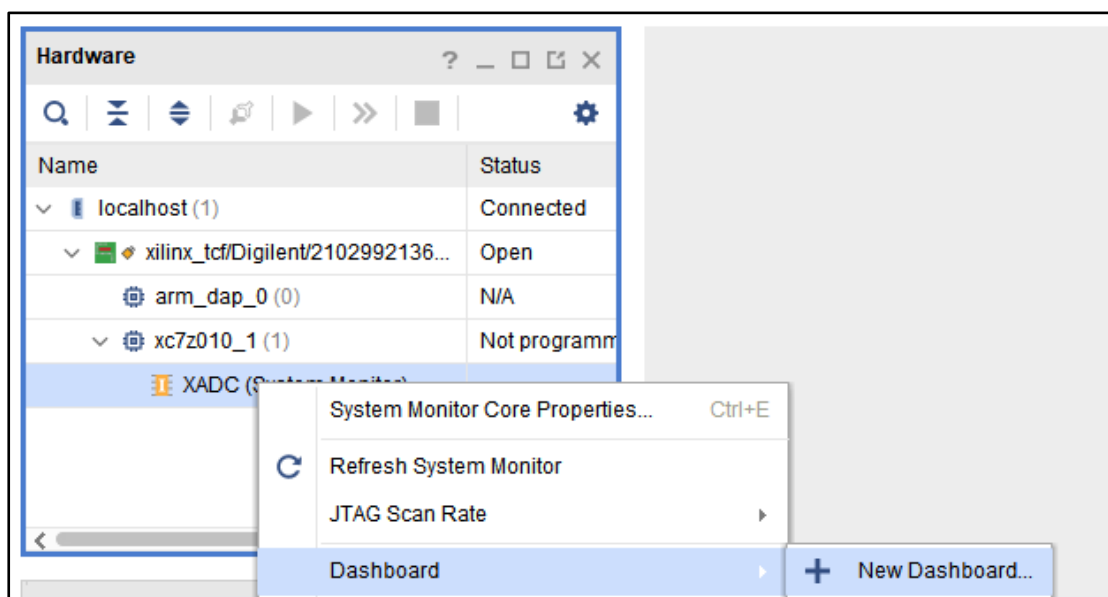


Figure 10-4 Create a new dashboard

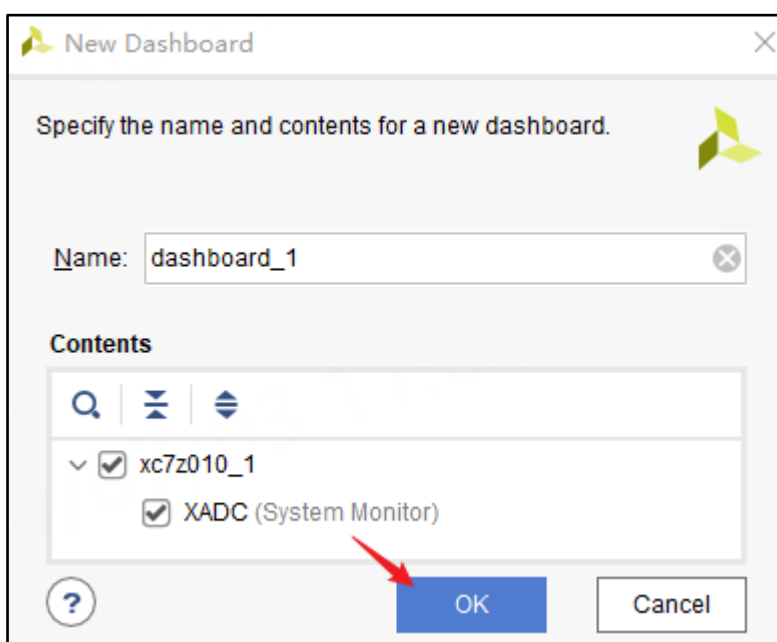


Figure 10-5 Click OK

After the dashboard interface appears, there is only temp at the beginning. We can add other signals manually:

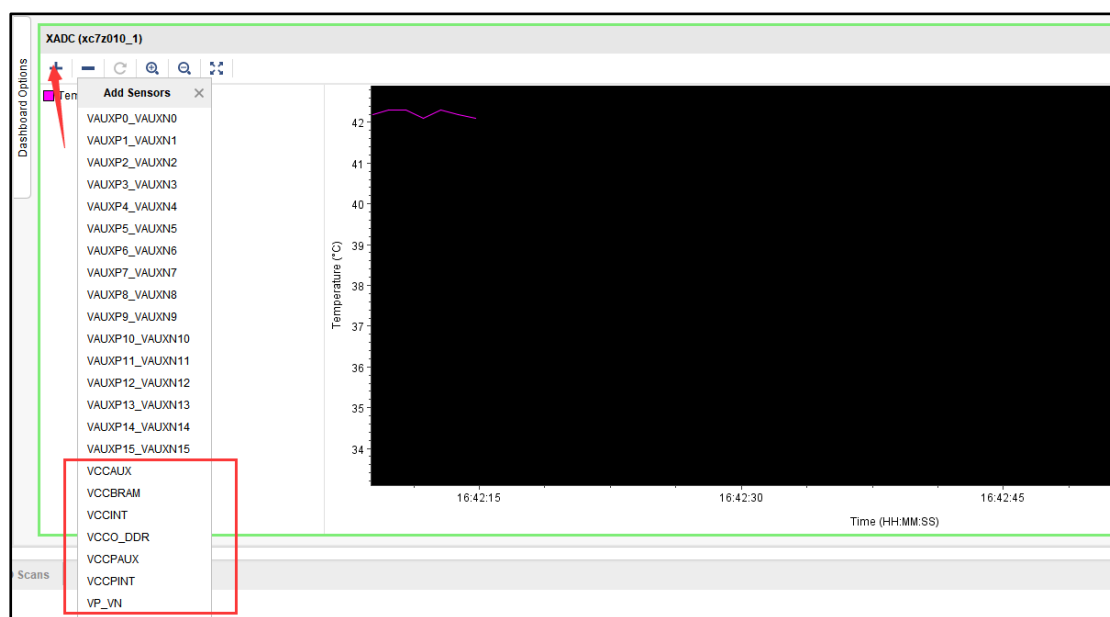


Figure 10-6 Adding other signals

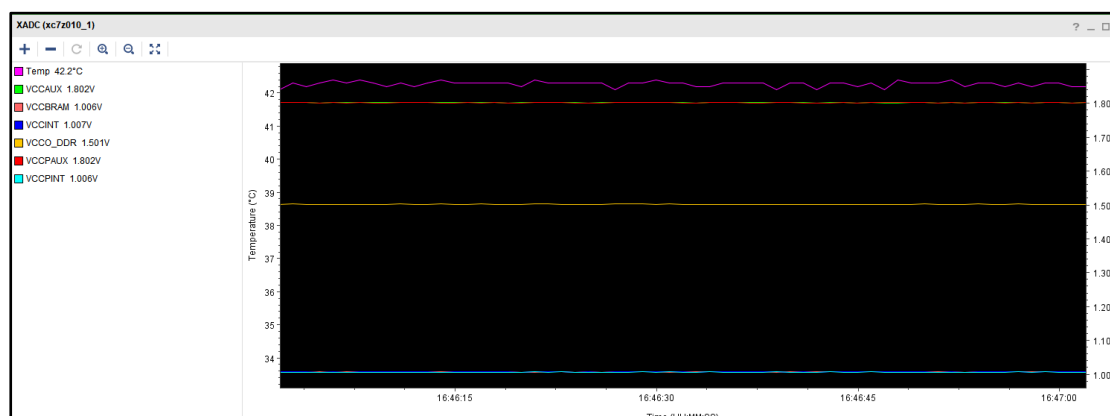


Figure 10-7 Display interface

The advantage of this method is graphical display, which is more intuitive, but the disadvantage is that the data value cannot be obtained. The following describes PS reading XADC information.

The second method is through the AXI Master interface (M_AXI_GP) from PS to PL, in which case the PL part needs to call the AXI XADC IP core. This method uses a parallel data transfer path (that is, the DRP interface).

The third method is implemented through the PS-XADC interface, which does not require any programming configuration of the PL. The PS-XADC interface is similar to the PL-JTAG interface, using a serial data transmission path and a relatively slow transmission speed. However, the advantage of using the PS-XADC interface is that it does not occupy any PL resources and does not require programming of the PL, only powering the PL part.

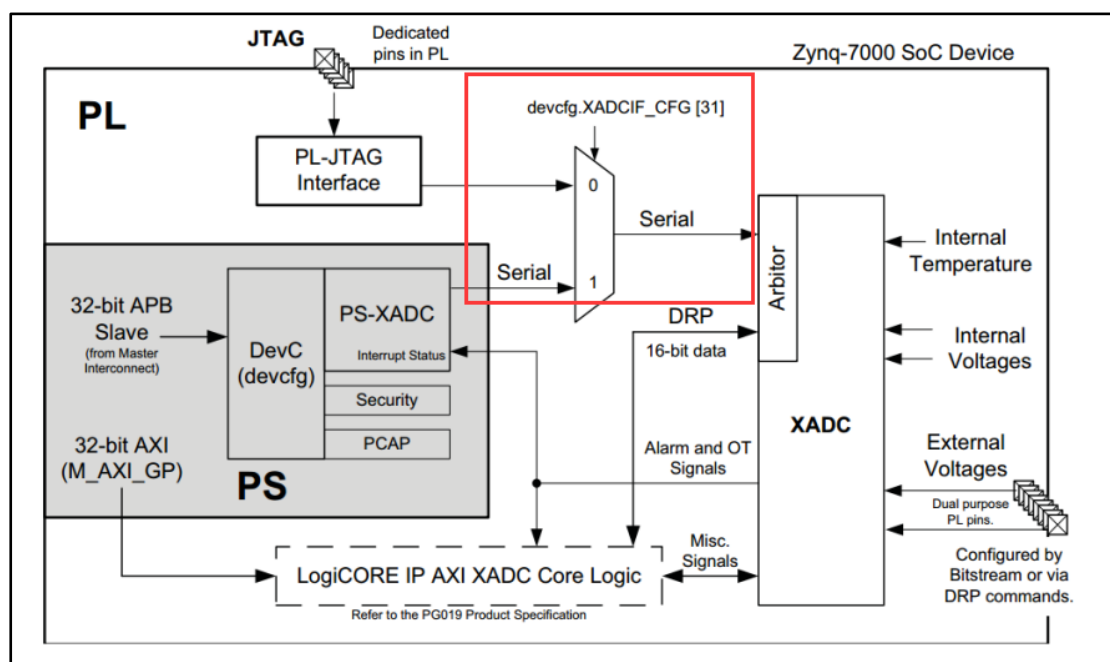


Figure 10-8 XADC system module block diagram

As can be seen from Figure 10-8, the PL-JTAG interface and PS-XADC interface cannot be used at the same time. XADC can arbitrate between the PL-JTAG interface (or PSXADC interface) and the DRP interface. This ensures that only one of the three methods mentioned above can be used at a time.

This chapter only introduces the last method, which is to read the chip temperature, power supply voltage and other information measured by the XADC through the PS-XADC interface and print it out through the serial port.

10.3 Practical exercises

10.3.1 Experimental objectives

Read the temperature, power supply voltage and other information of the chip measured by the XADC through the PS-XADC interface, and print it out through the serial port for display.

10.3.2 Hardware resources

XADC belongs to the hard core of the ZYNQ-7000 chip and is integrated within the chip. The design of Block Design here is exactly the same as in the first chapter. Directly export the hardware and then start the SDK

10.3.3 Programming

After starting the SDK, create a new application project and start writing software code to read relevant data.

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 10-9.

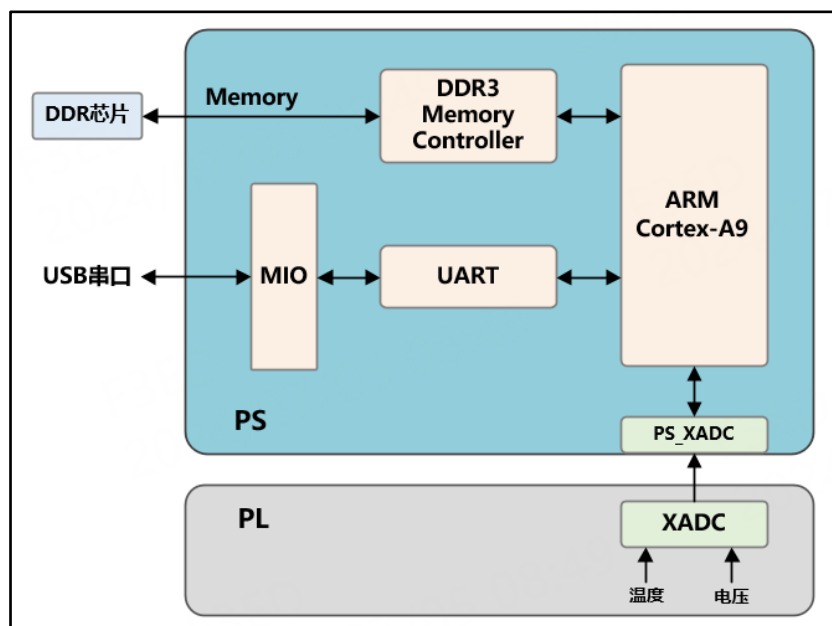


Figure 10-9 Overall block diagram

2. Hardware design

The CPU directly reads the temperature and voltage data collected by the XADC module in the PL through the PS-XADC interface on the PS side, and then prints it out through the serial port. It should be noted that although the resources in the PL are used, because it is a hard core module, there is no need to configure the PL. We can complete it on the ZYNQ minimal system, that is, using DDR and serial ports.

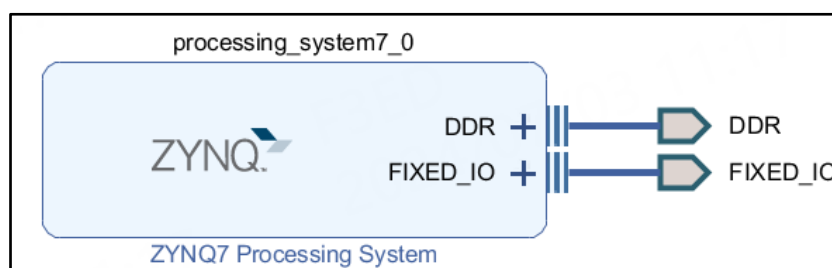


Figure 10-10 ZYNQ minimum system

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project and wrote relevant code to read data.

Write the following code in main.c.

Code Listing 10-1 Read xadc reference code (main.c)

```
1 #include "stdio.h"
2 #include "sleep.h"
3 #include "xadcps.h"
4 #include "xparameters.h"
5 #include "xil_printf.h"
6
7 #define XADC_ID XPAR_XADCPS_0_DEVICE_ID
8
```

```
9 static XAdcPs xadc_ps;          //实例化 XADC
10
11 int main(void)
12 {
13     XAdcPs_Config *Xadc_ptr; //Xadc 初始化指针
14     u32 original_data;        //原始数据
15     float Temp;               //温度转换值
16     float VCC_Pint_value;     //内核供电电压
17     float VCC_Paux_value;     //辅助供电电压
18     float VCC_Pddr_value;     //DDr 供电电压
19
20     //ID 查找使能
21     Xadc_ptr = XAdcPs_LookupConfig(XADC_ID);
22     //初始化
23     XAdcPs_CfgInitialize(&xadc_ps, Xadc_ptr, Xadc_ptr->BaseAddress);
24     //设置 xadc 模式为安全模式
25     XAdcPs_SetSequencerMode(&xadc_ps, XADCPS_SEQ_MODE_SAFE);
26
27     while (1) {
28         /*获取并转换温度值*/
29         original_data = XAdcPs_GetAdcData(&xadc_ps, XADCPS_CH_TEMP);
30         Temp = XAdcPs_RawToTemperature(original_data);
31         /*获取并转换内核供电电压*/
32         original_data = XAdcPs_GetAdcData(&xadc_ps, XADCPS_CH_VCCPINT);
33         VCC_Pint_value = XAdcPs_RawToVoltage(original_data);
34         /*获取并转换辅助供电电压*/
35         original_data = XAdcPs_GetAdcData(&xadc_ps, XADCPS_CH_VCCPAUX);
36         VCC_Paux_value = XAdcPs_RawToVoltage(original_data);
37         /*获取并转换 DDr 供电电压*/
38         original_data = XAdcPs_GetAdcData(&xadc_ps, XADCPS_CH_VCCPDRO);
39         VCC_Pddr_value = XAdcPs_RawToVoltage(original_data);
40
41         printf("The Temp is %f °C \n", Temp);
42         printf("The VCC_Pint_value is %f V \n", VCC_Pint_value);
43         printf("The VCC_Paux_value is %f V \n", VCC_Paux_value);
44         printf("The VCC_Pddr_value is %f V \n", VCC_Pddr_value);
45
46         sleep(1);
47     }
48     return 0;
49 }
```

In the third line of the code here, we introduce a new header file xadcps.h. This file contains some functions for obtaining adc parameters.

In lines 20-25 of code, Xadc is initialized and Xadc is set to safe mode using XAdc_SetSequencerMode.

Then in lines 27-47 of the code, we start to read the chip's temperature, voltage and other information. These functions are not difficult to use. Xilinx officials have thoughtfully packaged them for us. As long as we give the type to be obtained, we can get the corresponding original data. Then we can use the conversion functions provided by Xilinx officials to obtain temperature, voltage and other information. Very convenient. For example, using the XAdcPs_GetAdcData function, as long as XADCPS_CH_TEMP is given, the original data of the temperature is obtained, and then XAdcPs_RawToTemperature can be used to obtain the temperature. The following voltage acquisition method is the same.

After writing, compile the project.

10.3.4 Board verification

As shown in Figure 10-11, the development board is connected. After the lines are connected correctly, turn on the

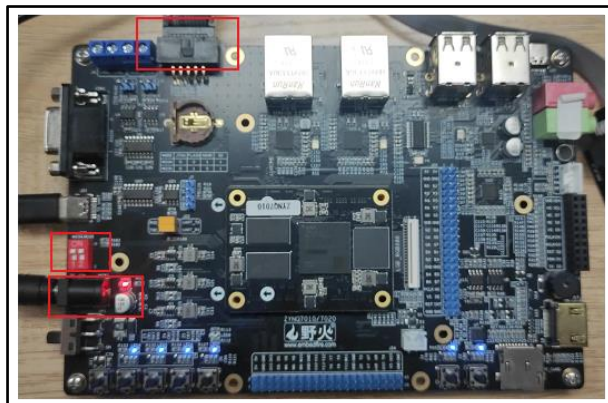


Figure 10-11 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project and select Run As > Launch on Hardware (system Debugger).

After the program download is completed, open the serial port assistant and you can see the voltage and temperature

Connected to: Serial (COM4, 115200, 0, 8)

```
The VCC_Paux_value is 1.796265 V
The VCC_Pddr_value is 1.483932 V
The Temp is 46.396027 °C
The VCC_Pint_value is 1.000671 V
The VCC_Paux_value is 1.796402 V
The VCC_Pddr_value is 1.484390 V
The Temp is 46.411407 °C
The VCC_Pint_value is 1.000351 V
The VCC_Paux_value is 1.796036 V
The VCC_Pddr_value is 1.483932 V
```

Figure 10-12 Serial port printing data

10.4 End of chapter summary

This chapter introduces the relevant knowledge of the XADC hard core located on the ZYNQ PL side, and obtains the voltage and temperature data measured by the XADC through the PL-JTAG interface and PS-XADC interface respectively, and finally obtains the real voltage and temperature values after conversion.

10.5 Development training

You can try to set a temperature threshold. When the temperature exceeds the threshold, a warning will be sent to the outside through LED flashing.

Chapter 11 QSPI Flash read and write test experiment

11.1 Chapter Introduction

I believe everyone is familiar with the Flash chip. It is a non-volatile storage chip. It is estimated that everyone has used it when learning MCUs such as microcontrollers and STM32. Then we will also use it when learning FPGA. Because it is a non-volatile memory, the data will not be lost after power failure. In the design of FPGA engineering, Flash is mainly used as an external chip to store FPGA programs, so that the FPGA The program we want to execute can be executed immediately after powering on. SPIFlash chip is a Flash chip that supports SPI communication protocol.

The Quad-SPI Flash controller is part of the PS internal input/output peripheral (IOP). Since the Flash of the Haoyue development board is connected to the PS side, we need to control the Flash chip through the Flash controller.

11.2 Theoretical study

ZYNQ's QSPI Flash controller provides three operating modes: I/O mode, linear address mode and traditional SPI mode.

In I/O mode, the communication protocol is managed by software, sending Flash commands and data through the controller's four TXD registers, and reading data from the RXD register. This mode uses the 32bit APB interface.

Linear address mode uses a set of device operations to reduce the software overhead of reading data from Flash. Linear mode uses hardware to implement commands to the Flash device and stream data from the Flash device to the AXI interface. For users, the process of reading QSPI Flash is similar to accessing ROM, simple and direct, but this mode only supports reading operations, not writing.

In the traditional SPI mode, the QSPI Flash controller behaves as an ordinary SPI controller, and this mode is relatively rarely used.

The QSPI Flash controller can only connect 1~2 Flash devices. The two devices can be connected in parallel to form an 8-bit wide combination for use, or they can be divided into two independent Flash devices, which is a stack mode.

The system block diagram of QSPI Flash controller is shown in Figure 11-1:

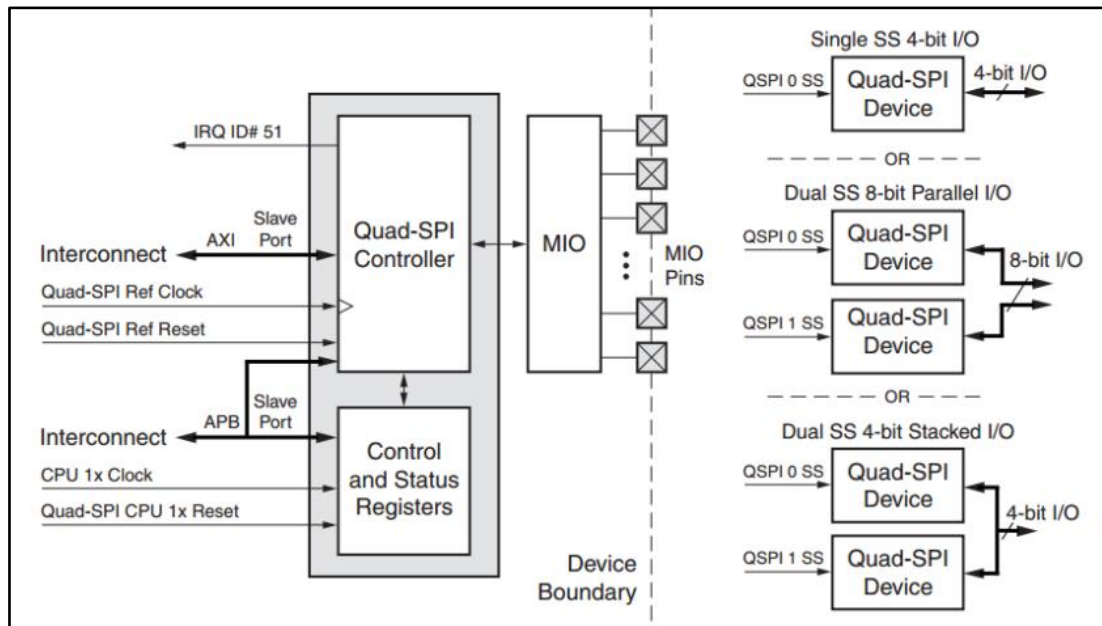


Figure 11-1 QSPI Flash system block diagram

As can be seen from Figure 11-1, the QSPI Flash controller uses MIO to connect to external Flash devices and provides three different connection methods: single slave device mode, dual slave device parallel mode and dual slave device stack mode. Using dual slave device mode can effectively expand the storage capacity of QSPI Flash.

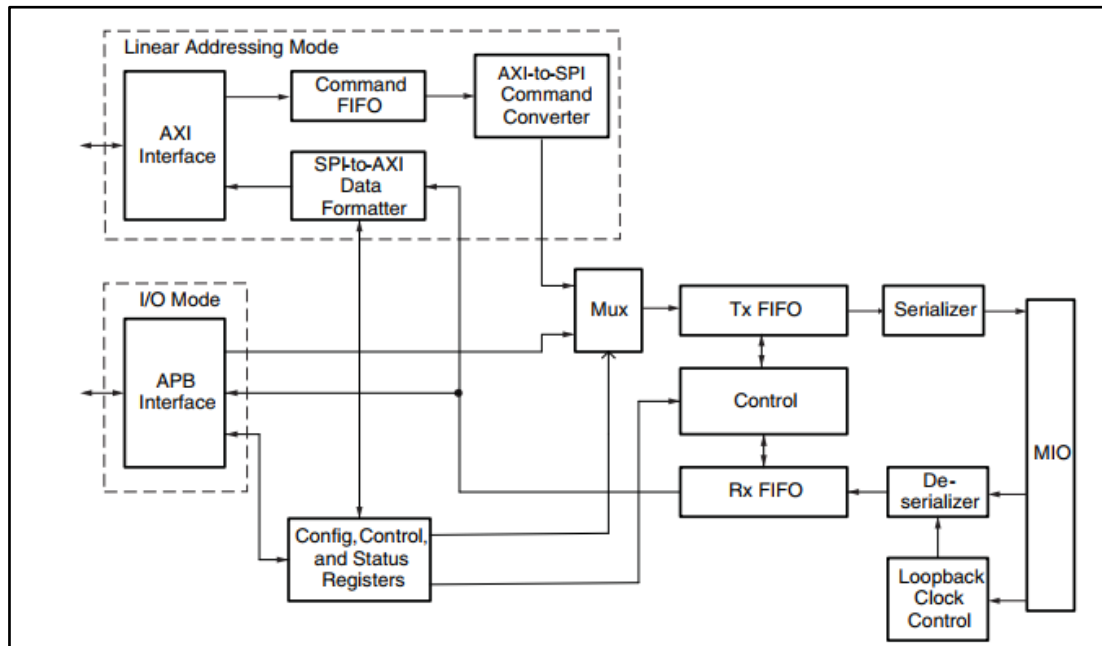


Figure 11-2 Internal structure of Flash controller

As can be seen from Figure 11-2, in the linear address mode, the PS end converts the instruction from AXI to SPI through the AXI interface, and sends it to the Tx FIFO through the two-way selector (up to 63 double words can be stored), and then outputs it from the MIO through parallel conversion to serial to control the Flash device. Then the data stream returned by the Flash device is transmitted to the Data Formatter through the Rx FIFO (up to 63 double words can be stored), thereby the SPI data stream is converted into an AXI data stream, and finally transmitted to the PS-side ARM core through the AXI interface to complete the read operation. It can also be seen here that the linear address mode does not support the write mode.

In I/O mode, the software is responsible for converting commands and data into instructions that comply with the QSPI Flash protocol and writing them into the transmission buffer (Tx FIFO). Next, the sending logic performs serial-to-parallel conversion of the contents in the Tx FIFO according to the QSPI interface standard, and sends it to the external Flash memory through MIO. At the same time, the receiving logic samples the sent serial data, converts it back to parallel format, and stores it in the receive buffer (Rx FIFO). If a read operation is performed, after sending the read command and address, the MIO switches from output mode to input mode under the control of the sending logic in order to receive data from Flash. This data will be stored in the Rx FIFO. Since the Rx FIFO also receives command data from the software, the data in the Rx FIFO needs to be filtered to extract valid data from Flash.

This experiment is to test the read and write of Flash, so we let the Flash controller work in I/O mode.

11.3 Practical exercises

11.3.1 Experimental objectives

In I/O mode, complete the read and write test on the Haoyue development board W25Q128J V, that is, compare the written data with the read data. If they are the same, it means success.

11.3.2 Hardware resources

This experiment is to test the read and write of the Flash chip. In addition to the minimum system, the development board also has a Flash chip connected to the PS side. The schematic diagram and physical hardware diagram are shown in Figure 11-3 and Figure 11-4.

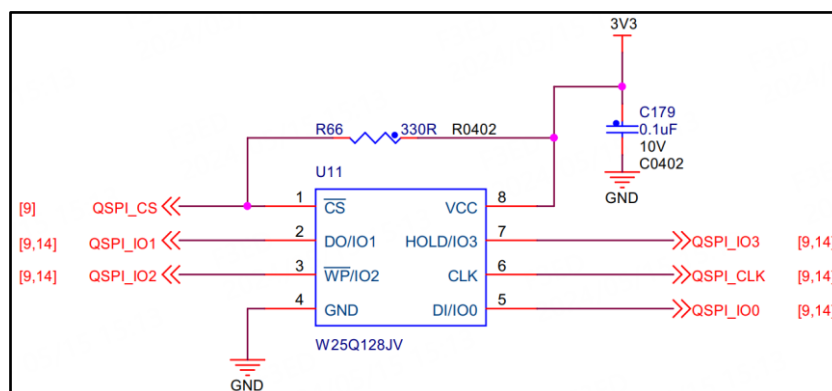


Figure 11-3 Flash chip pin diagram

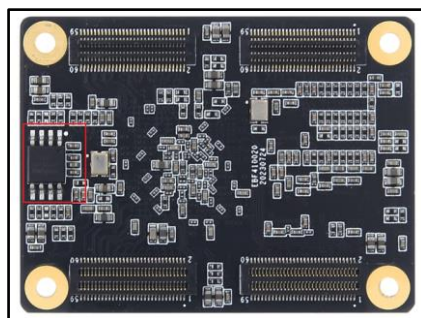


Figure 11-4 Flash actual picture

11.3.3 Programming

1. Overall description

According to the experimental goal, we can draw the overall structure as shown.

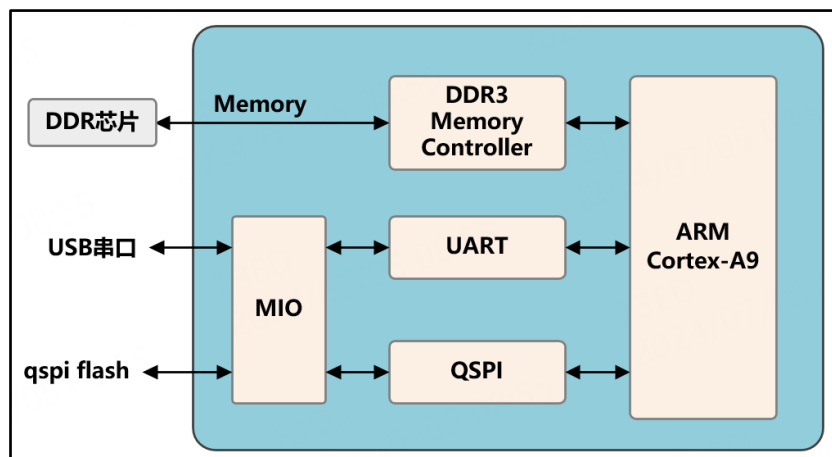


Figure 11-5 Overall block diagram

2. Hardware design

We can just add a QSPI_Flash controller on the basis of the minimal system.

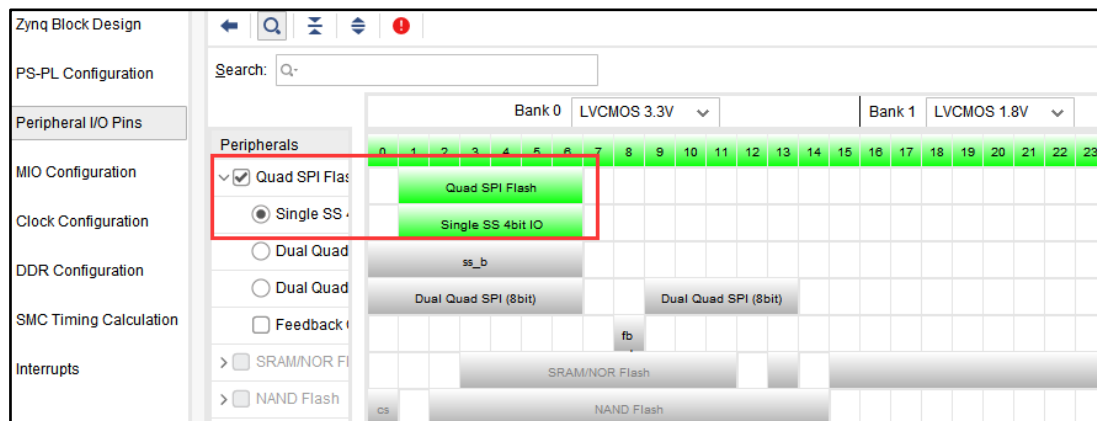


Figure 11-6 Add QSPI controller

The following operations are similar to the first chapter and will not be repeated here.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and wrote software code. Since there is too much code in this part, only the core part is introduced here, and the remaining readers can read the complete code.

Code List 11-1 Read and write qspi flash reference code (main.c)

```

1 #include "xparameters.h" /* SDK generated parameters */
2 #include "xqspips.h" /* QSPI device driver */

```

```

3 #include "xil_printf.h" 4 5 #define QSPI_DEVICE_ID XPAR_XQSPIPS_0_DEVICE_ID 6 7 /* Send to flash ins
instruction */ 8 #define WRITE_STATUS_CMD 0x01 9 #define WRITE_CMD 0x02 10 #define READ_CMD 0x03
11 #define WRITE_DISABLE_CMD 0x04 12 #define READ_STATUS_CMD 0x05 13 #define WRITE_ENABL
E_CMD 0x06 14 #define FAST_READ_CMD 0x0B 15 #define DUAL_READ_CMD 0x3B 16 #define QUAD_R
EAD_CMD 0x6B 17 #define BULK_ERASE_CMD 0xC7 18 #define SEC_ERASE_CMD 0xD8 19 #define REA
D_ID 0x9F 20 21 /* Define the offset within the FlashBuffer data type for each data */ 22 #define COMMAND_O
FFSET 0 /* FLASH instruction */ 23 #define ADDRESS_1_OFFSET 1 /* MSB byte of address to read or write */
24 #define ADDRESS_2_OFFSET 2 /* Middle byte of address to read or write */ 25 #define ADDRESS_3_OFFS
ET 3 /* LSB byte of address to read or write */ 26 #define DATA_OFFSET 4 /* Start of Data for Read/Write */ 27
#define DUMMY_OFFSET 4 /* Dummy byte offset for fast, dual and quad 28 reads*/ 29 #define DUMMY_SIZE
1 /* Number of dummy bytes for fast, dual and 30 quad reads*/ 31 32 #define RD_ID_SIZE 4 /* Read ID comman
d + 3 bytes ID response */ 33 #define BULK_ERASE_SIZE 1 /* Bulk Erase command size */ 34 #define SEC_ER
ASE_SIZE 4 /* Sector Erase command + Sector address */ 35 36 37 38 /* Specifies additional bytes sent to flash m
emory on the QSPI interface, 39 * Contains command and address control information */ 40 #define OVERHEAD
_SIZE 4 41 42 43 #define SECTOR_SIZE 0x10000 44 #define NUM_SECTORS 0x100 45 #define NUM_PAGES
0x10000 46 #define PAGE_SIZE 256 47 48 #define PAGE_COUNT 16 49 50 #define TEST_ADDRESS 0x00055
000 51 #define UNIQUE_VALUE 0x05 52 53 //Define buffer size 54 #define MAX_DATA (PAGE_COUNT * P
AGE_SIZE) 55 56 /* The following are official functions */

```

```

57 void FlashErase(XQspiPs *QspiPtr, u32 Address, u32 ByteCount);
58 void FlashWrite(XQspiPs *QspiPtr, u32 Address, u32 ByteCount, u8 Command) ;
59 void FlashRead(XQspiPs *QspiPtr, u32 Address, u32 ByteCount, u8 Command);
60 int FlashReadID(void);
61 void FlashQuadEnable(XQspiPs *QspiPtr);
62 int QspiFlashPolledExample(XQspiPs *QspiIns
tancePtr, u16 QspiDeviceId); 63 64 static XQspiPs QspiInstance; //instantiate qspi 65 66 int Test = 5; 67

```

```

68 u8 ReadBuffer[MAX_DATA + DATA_OFFSET + DUMMY_SIZE];
69 u8 WriteBuffer[PAGE_SIZE + DATA_OFFSET];
70
71 int main(void)
72 {
73     int Status;
74
75     xil_printf("QSPI FLASH Test ! \r\n");
76     Status = QspiFlashPolledExample(&QspiInstance, QSPI_DEVICE_ID);
77     if (Status == XST_SUCCESS)
78         xil_printf("Successfully ran QSPI FLASH Polled Example Test\r\n");
79     else
80         xil_printf("Failed to Polled Example Test\r\n");
81     return XST_SUCCESS;
82 }

```

This time, the header file xqspips.h is introduced. In fact, these codes are copied from the official routines, and made some modifications.

In lines 8-55 of the code, we define the Flash instruction, the offset of each data segment in the Flash Buffer, the number and size of PAGE, SECTOR and other information. The specific information depends on the Flash model and can be obtained by consulting the chip manual.

Then we declared several functions in lines 57-62 of the code. These functions are officially provided by Xilinx. We can directly use them after modifying them. For example, functions such as FlashErase(), FlashWrite(), Flash_Read(), etc. are all used directly from the official routine functions.

The main function calls our modified function to perform read and write tests on Flash. Now let's parse the test function.

Code List 11-2 Read and write qspi flash reference code (main.c)

```

1 /* Polling for reading and writing */ 2 int QspiFlashPolledExample(XQspiPs *QspiInstancePtr, u16 QspiDeviceId) 3 { 4 u8 *BufferPtr; 5 u8 UniqueValue; 6 int Count; 7 int Page; 8 XQspiPs_Config *QspiConfig; 9 10 QspiConfig = XQspiPs_LookupConfig(QspiDeviceId); // ID lookup enable 11 XQspiPs_SelfTest(QspiInstancePtr); // Self-test to check hardware build 15 16 /* Initialize read and write buffer */ 17 for (UniqueValue = UNIQUE_VALUE, Count = 0; Count < PAGE_SIZE; 18 Count++, UniqueValue++) { 19 WriteBuffer[DATA_OFFSET + Count] = (u8)(UniqueValue + Test); 20 } 21 memset(ReadBuffer, 0x00, sizeof(ReadBuffer)); // Clear 22 23 /* Set manual start and chip select operation */ 24 XQspiPs_SetOptions(QspiInstancePtr, XQSPIPS_MANUAL_START_OPTION | 25 XQSPIPS_FORCE_SSELECT_OPTION | 26 XQSPIPS_CLK_PRESCALE_8); 30 31 /* Determine chip selection */ 32 XQspiPs_SetSlaveSelect(QspiInstancePtr); 33 FlashReadID(); 34 FlashQuadEnable(QspiInstancePtr);

```

```

35 36 /* Erase flash.*/ 37 FlashErase(QspiInstancePtr, TEST_ADDRESS, MAX_DATA); 38 /* Write data by
page */ 39 for (Page = 0; Page < PAGE_COUNT; Page++) { 40 FlashWrite(QspiInstancePtr, (Page * PAGE_
SIZE) + TEST_ADDRESS, 41 PAGE_SIZE, WRITE_CMD); 42 } 43 44 FlashRead(QspiInstancePtr, TEST_
ADDRESS, MAX_DATA, READ_CMD); 45 /* Pointer records write address */ 46 BufferPtr = &ReadBuffer
[DATA_OFFSET]; 47 for (UniqueValue = UNIQUE_VALUE, Count = 0; Count < MAX_DATA; 48 Count
++, UniqueValue++) { 49 if (BufferPtr[Count] != (u8)(UniqueValue + Test)) { 50 return XST_FAILURE; 51
} 52 } 53 54 return XST_SUCCESS; 55 }

```

First, in lines 10-14 of the program, the Qspi Flash controller is initialized. The method is the same as before, first LookupConfig() and then CfgInitialize() for initialization, and then perform self-test.

In lines 17-32 of the program, we first clear all buffers to 0 and then manually turn it on and configure it to chip select mode. Generally, there are two modes: manual mode and automatic mode. This time we are in manual mode. In this manual mode, our users control the data transmission. When the chip select signal is valid, it means the transmission starts. The chip selection is generally divided into automatic and manual. While the amount of data automatically transmitted is limited by the depth of the TX FIFO, manual chip select does not have this limitation, so it is more suitable for batch data transmission.

In lines 33-44 of the code, we operate according to the program provided by Xilinx. We first use FlashReadID() to read the ID of the Flash, then enable the Flash Quad mode, then use FlashErase() to erase the Flash, then use FlashWrite() to write test data to a specific address in the Flash, and then use the FlashRead() function to read the data from the address. At the same time, record reading BUFFER.

After that, lines 46-52 compare whether the data in the read BUFFER and the written BUFFER are consistent to determine whether the read and write to the Flash are successful.

The rest of the code is officially packaged by Xilinx, that is, reading, writing, erasing, etc. of Flash. I will not post it here. You can use it directly for practical applications. Readers who are interested can study the relevant instructions, etc.

Compile the project and prepare for board testing.

11.3.4 Board verification

As shown in Figure 11-7, the development board is connected. After the line is correctly connected, turn on the switch.

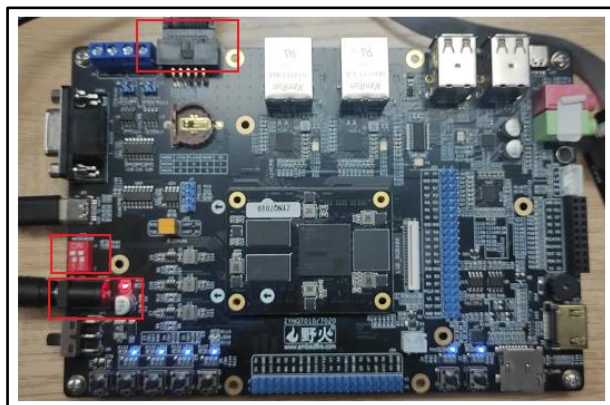


Figure 11-7 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project and select Run As > Launch on Hardware (system Debugger).

After the program download is completed, open the serial port assistant and you can see the relevant information p

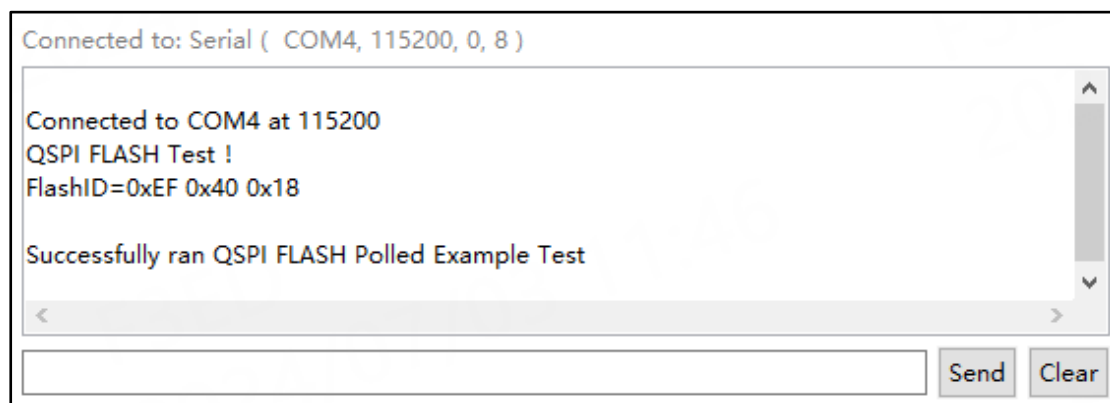


Figure 11-8 Serial port data printing

If the word "Successfully" is printed, it means that the qspi flash read and write test was successful.

11.4 End of chapter summary

This chapter introduces the relevant knowledge of ZYNQ's QSPI Flash controller. We learned that the controller supports three operating modes (I/O mode, linear address mode and traditional SPI mode) and three connection modes (single slave device mode, dual slave device parallel mode and dual slave device stacking mode), and used the I/O mode to complete the read and write test of W25Q128JV on the Haoyue development board.

11.5 Development training

This experiment uses serial port printing to determine whether the read and write test is successful, but you can also use the LED on the development board as an indicator, which makes it easier to observe the experimental results.

Chapter 12 SD Card Reading and Writing Test Exp

12.1 Chapter Introduction

SD memory card, as a new storage device based on semiconductor flash memory, is widely used in various portable devices, such as digital cameras, smartphones, and multimedia players, due to its compact size, fast data transmission capabilities, and hot-swappable functions.

In this chapter, we will introduce how to read and write TXT files on the SD card (Micro SD card, also known as TF card) by using the Haoyue development board.

12.2 Theoretical study

12.2.1 Introduction to SD card

To talk about SD cards, we must first introduce MMC cards. MMC (Multi-Media Card) was launched in 1997 by Siemens and SanDisk. Because of its advanced packaging technology, 7-pin pins, small size and light weight, it is very suitable for mobile storage needs. MMC supports 1bit mode, 20MHz clock, and adopts bus structure.

SD card is developed on the basis of MMC card. Its full name is Secure Digital Card, which is translated as secure digital card and is referred to as SD card. Jointly launched by Panasonic, Toshiba and SanDisk, it was released in August 1999. The data transfer and physical specifications of SD cards are similar to MMC, and the size is similar to MMC cards, measuring 32mm x 24mm x 2.1mm. The length and width are the same as the MMC card, but it is 0.7mm thicker than the MMC card to accommodate larger capacity storage units. SD cards and MMC cards have some compatibility: new SD devices can access MMC cards. However, SD cards are not designed to support device access from MMC cards.

Pictures of MMC cards and SD cards, see Figure 12-1 for details.

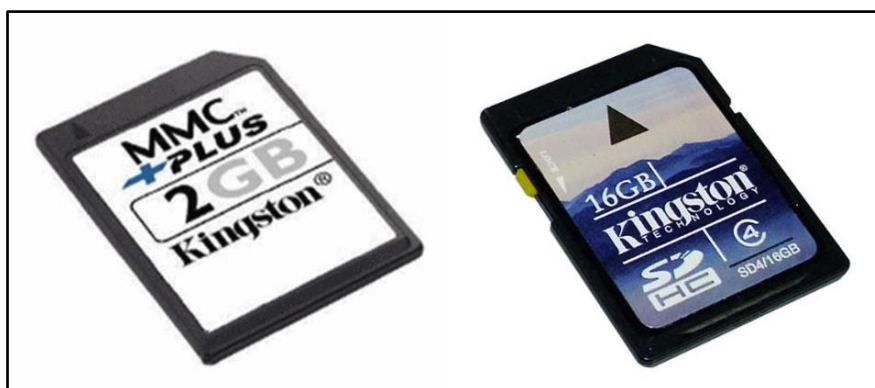


Figure 12-1 MMC card and SD card

The SD card in the picture above is just one type of SD card. According to the shape and size, SD cards can be divided into three categories: standard SD card, MiniSD card and MicroSD card.

Standard SD is the largest form of SD card. Most consumer-grade digital cameras and camcorders on the market today use this standard SD card. This card has a standard "cutaway" design.

Mini SD cards are not much different from standard SD cards in terms of performance, mainly smaller in size. Mini SD card features small size, stable performance, and is fully compatible with standard SD card slots when used with a dedicated adapter card. Moreover, the Mini SD card adopts a low power consumption design and is more suitable for mobile communication devices than the standard SD card. Therefore, it mainly attacks information terminals of mobile phones, PDAs, and handheld computers. Mini SD cards have been replaced by smaller Micro SD cards, which are no longer available on the market. Most standard SD cards and Mini SD cards have a physical write-protect switch on the card itself.

Micro SD card, formerly known as Trans-flash Card (TF card), was officially renamed Micro SD card in 2004. It was invented by SanDisk and is mainly used in mobile phones.

The overall dimensions of the three types of SD cards are shown in Figure 12-2.

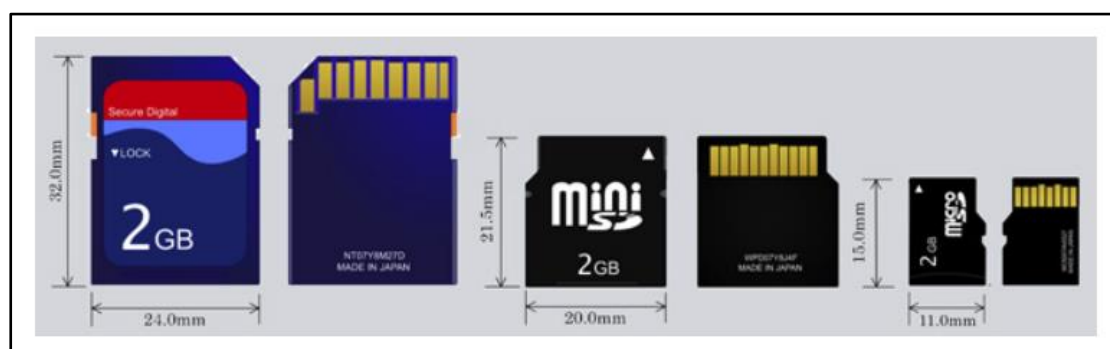


Figure 12-2 Dimensions of three types of SD cards

Regular SD memory cards (non-UHS-II cards) have only one row of pins and a voltage of 3.3V. They support default speed mode, high-speed mode and UHS-I mode in the bus interface. A standard size SD card has 9 pins (two VSS) while a microSD has 8 pins (one VSS).

Then a second row of pins was added to the first row of pins to support UHS-II and UHS-III bus interfaces. The 1.8V VDD is included in the second row of pins, allowing the UHS-II / UHS-III card to choose between two different voltage supplies (3.3V or 1.8V).

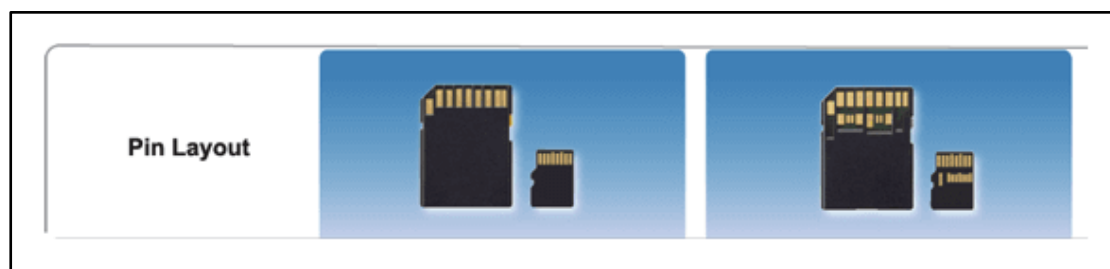


Figure 12-3 SD card appearance pin diagram

In fact, the SD card we often talk about or use is just part of the SD family. SD as a standard is developed by the SD Association. Its success and popularity have resulted in countless devices, applications, and services. The SD Association provides a simple classification system as shown in Figure 12-4 to identify the types, sizes, uses and characteristics of various SD standards that can be used with host devices.

It should be noted that the SD memory cards currently in use are classified as "removable random access memory" and have four capacity standards: SD, SDHC, SDXC and SDUC. The removable random access memory category also includes an optional content security function, allowing the SD memory card to be used to store copyright-protected sounds, images, and books.

The SD Association further extends the SD standard to define input/output (I/O) functions, so that the SD card slot commonly used in consumer electronics is not only used to insert memory cards to store data, but also uses the SDIO interface to expand the device to add functions such as network services, satellite navigation, and photography.

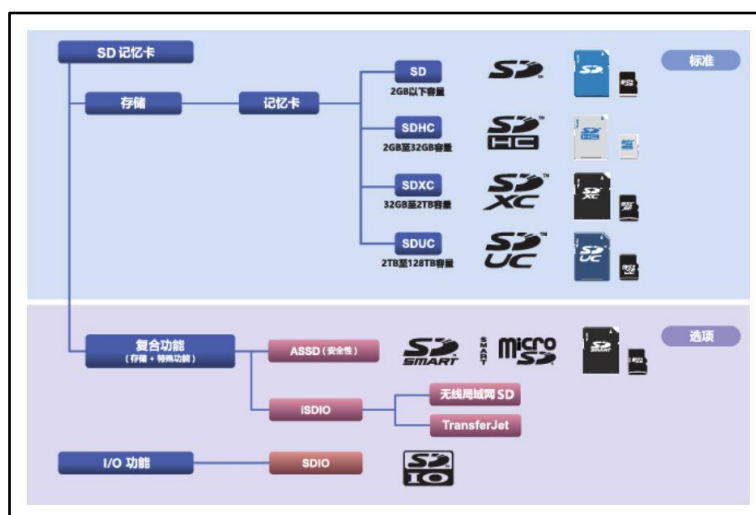


Figure 12-4 SD family diagram

12.2.2 SD card capacity standards and speed levels

In addition to the appearance size classification mentioned above, SD cards can also be classified according to capacity standards. If SD cards are classified according to capacity, SD cards can be divided into 4 levels: SD (Secure Digital Card, secure digital card) card, SDHC card (Secure Digital High Capacity, high-capacity secure digital card), SDXC card (SD eXtended Capacity, expanded capacity secure digital card) and SDUC (Secure Digital Ultra Capacity, ultra-capacity secure digital card). Today, the mainstream SD products in the market are SDHC and SDXC, two larger-capacity memory cards. SD cards have been gradually eliminated by the market because of their small capacity, while SDUC cards have too large a capacity and are expected to appear in the future market. There are four capacity standards for SD cards, see Figure 12-5 for details.

	SD - 容量上限为2GB，使用FAT 12与FAT 16档案系统。
	SDHC - 容量介于2GB至32GB间，使用FAT32档案系统。
	SDXC - 容量介于32GB至2TB间，使用exFAT档案系统。
	SDUC - 容量介于2TB至128TB间，使用exFAT档案系统。

Figure 12-5 Four capacity standards of SD cards

SD cards produced by different brands and manufacturers have different definitions of access speeds, which can cause confusion for users when choosing an SD card. Therefore, the SD Association divides different levels according to the minimum sustained speed of writing video to the SD card at a constant speed. The rate of each level is measured by how many MBs are transferred per second, and the unit is MB/S.

The SD Association defines three speed classes: Speed Class, UHS Speed Class, and Video Speed Class. The specific transmission speeds of the three speed levels are shown in Figure 12-6.

最低写入速度	速度等级	UHS速度等级	视频速度等级
2MB/S	Class 2 (C2)	——	——
4MB/S	Class 4 (C4)	——	——
6MB/S	Class 6 (C6)	——	6级(V6)
10MB/S	Class 10 (C10)	UHS-1	10级(V10)
30MB/S	——	UHS-3	30级(V30)
60MB/S	——	——	60级(V60)
90MB/S	——	——	90级(V90)



Figure 12-6 Specific transmission speed of SD card speed level

Speed level

Starting from the SD2.0 specification, the speed classification of SD ordinary cards and high-speed cards are: Class2, Class4, Class6 and Class 10.

UHS Speed Class

UHS (Ultra High Speed) is an SD card bus standard launched at the same time as SDXC. UHS speed grades are available for SDHC and SDXC.

The UHS speed classes defined by the SD Association are UHS Speed Class 1 (UHS-1) and UHS Speed Class 3 (UHS-3). U1 and U3 can be applied to the UHS bus IF product series (UHS-I, UHS-II and UHS-III). Manufacturers usually mark the speed directly on the SD card, which helps consumers choose the performance they need by reading and writing the mark.

Video speed rating

The video speed grade is defined to meet the needs of high-resolution and high-quality 4K/8K video recording, and also has the important feature of supporting next-generation flash memory types such as 3D NAND. In addition, the video speed rating covers the speed of HD high definition (2K) video. Video speed classes defined by the SD Association are V6, V10, V30, V60 and V90. V6 and V10 are available in high speed and UHS bus IF product families. V30 is available in the UHS bus IF product family. V60 and V90 are available in UHS-II / UHS-III product families.

12.2.3 SD card physical structure

From the physical structure, the SD card includes 5 parts, namely storage unit, storage unit interface, power detection, card and interface controller and interface driver. See Figure 12-7 for details. The storage unit is a component that stores data. The storage unit transmits data with the card control unit through the storage unit interface. The power detection unit ensures that the SD card operates at a suitable voltage. If a power-off or power-on state occurs, it will reset the control unit and the storage unit interface. The card and interface control unit controls the operating status of the SD card, which includes 8 registers. The interface driver controls the input and output of the SD card pins.

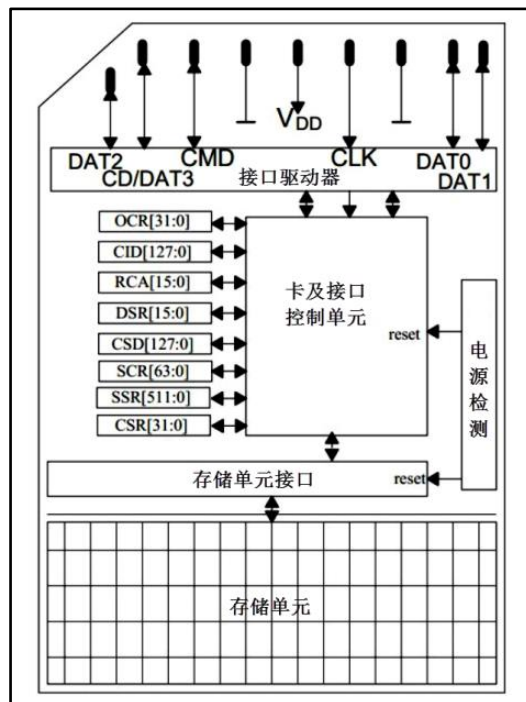


Figure 12-7 SD card physical structure diagram

1. SD card pins and connections

A conventional SD card has a total of 9 pin interfaces, including 3 power lines, 1 clock line, 1 command line and 4 data lines. There are two working modes: SDIO mode and SPI mode (mode 3). An introduction to each signal is as follows:

CLK: Synchronous clock line, generated by the SDIO host; when using SPI mode, this pin is connected to the SCK clock signal of the SPI bus. **CMD:** Command control line, the SDIO host sends commands through this line to control the SD card. If the command requires the SD card to provide a response (response), the SD card also transmits the response information through this line; when using the SPI mode, this pin is connected to the MOSI signal of the SPI bus, and the SPI host sends commands and data to the SD card through it. However, because the MOSI of the SPI bus is only used for the host to output signals to the slave, the SD card does not use this signal line when returning response information. **DAT0-3:** In SDIO mode, they are both data lines, transmitting read and write data. The SD card can pull D0 low to indicate busy status; in SPI mode, DAT0 is connected to the MISO signal of the SPI bus. The SD card sends data or response to the host through this signal line. DAT3 is connected to the CS signal of the bus. The SPI host selects the SD card to communicate through this signal line.

VDD, VSS1, VSS2: power and ground signals. The Micro SD card does not contain the VSS2 signal, that is, the Micro SD card has only 8 lines.

The SD card has two working modes. In the SDIO mode, the SD card uses a total of six signal lines: CLK, CMD, and DAT[3:0]; when the SDIO bus is connected to multiple SD cards, the CLK clock signal line can be shared. For the CMD and DAT[3:0] signal lines, each SD card must be connected independently. The connection method between SDIO bus and SD card, see Figure 12-8 for details.

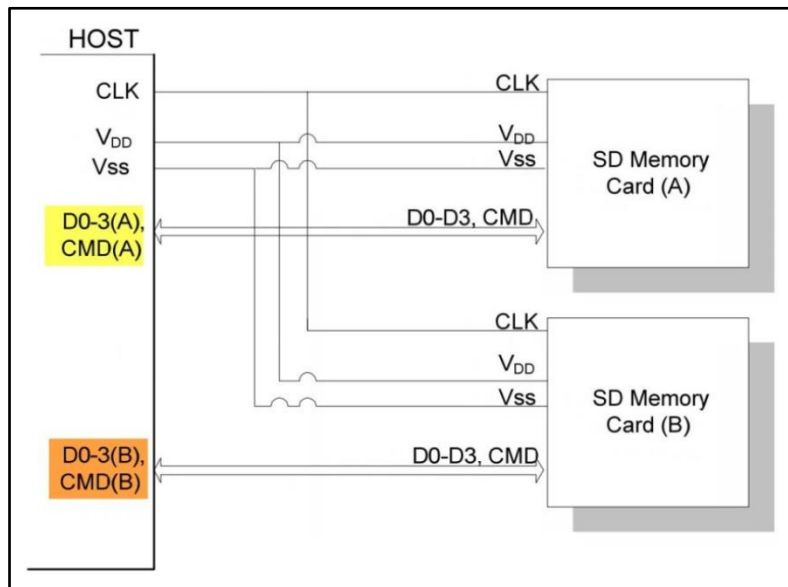


Figure 12-8 SDIO bus and SD card connection method

In SPI mode, the SD card uses a total of four signal lines: CS(DAT[3]), CLK, MISO(DAT[0]), and MOSI(CMD); when the SPI bus is connected to multiple SD cards, except for the CS chip select signal line, which cannot be shared, other signals can be shared. SPI bus and SD card connection method, see Figure 12-9 for details.

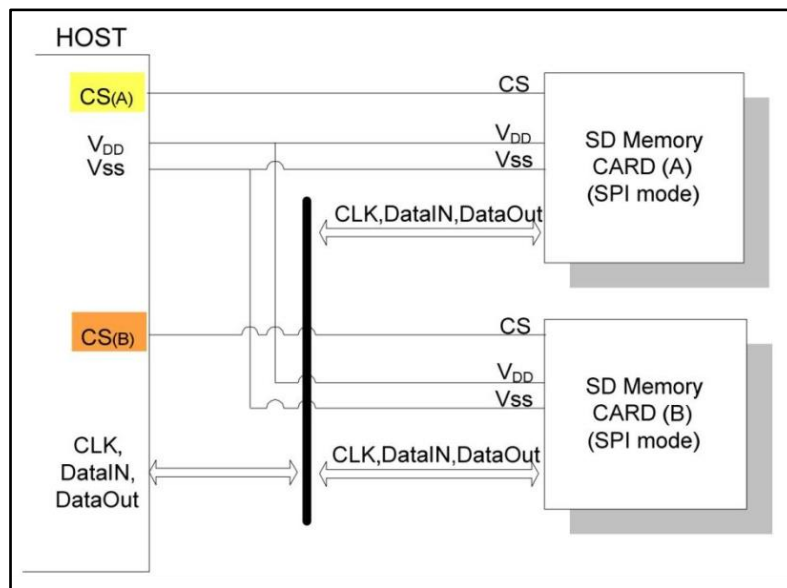


Figure 12-9 SPI bus and SD card connection method

2. SD card register

The SD card has a total of 8 registers, which are used to set or represent SD card information. See Figure 12-10 for the register description. These registers can only be accessed through the corresponding commands. The control operations on the SD card are performed through commands. The SD card defines 64 commands (some commands do not support SPI mode). Each command has a special meaning and can achieve a specific function. After the SD card receives the command, it will modify the internal registers of the SD card according to the command requirements. In program control, only the combined command is sent to realize the control of the SD card and the read and write operations.

名称	bit 宽度	描述
CID	128	卡识别号(Card identification number):用来识别的卡的个体号码(唯一的)
RCA	16	相对地址(Relative card address):卡的本地系统地址, 初始化时, 动态地由卡建议, 主机核准。
DSR	16	驱动级寄存器(Driver Stage Register):配置卡的输出驱动
CSD	128	卡的特定数据(Card Specific Data):卡的操作条件信息
SCR	64	SD 配置寄存器(SD Configuration Register):SD 卡特殊特性信息
OCR	32	操作条件寄存器(Operation conditions register)
SSR	512	SD 状态(SD Status):SD 卡专有特征的信息
CSR	32	卡状态(Card Status):卡状态信息

Figure 12-10 Brief description of SD card registers

For a detailed introduction to the SD card register, readers can refer to the SD simple specification file.

《Part1_Physical_Layer_Simplified_Specification_Ver7.10》.

ZYNQ integrates two SD card controllers internally, and the standalone of Xilinx SDK has been transplanted to the FATFS (called xilffs in the SDK software) file system. Therefore, after adding the xilffs library to the SDK, you can use the API functions in FATFS to operate the SD card in the program.

12.2.4 SD card controller (SD/SDIO Controller)

ZYNQ's built-in SD card controller follows the SD2.0 protocol specification and can operate at a maximum operating frequency of 50Mhz. The controller is compatible with multiple interfaces, including eMMC, MMC3.31, SDIO2.0, SD2.0 and SPI, and supports both SDHC and SDH S d evices. In addition, the SD card controller supports multiple DMA modes, including SDMA (single-operation DMA), ADMA1 (DMA with 4K boundary restrictions), and ADMA2 (allowing data t ransfers of arbitrary location and size on 32-bit systems). In order to improve the efficiency of dat a transmission, the SD controller adopts an independent read and write double buffering FIFO me chanism and is accessed by the ARM processor through the AHB bus, which improves the throug hput bandwidth.

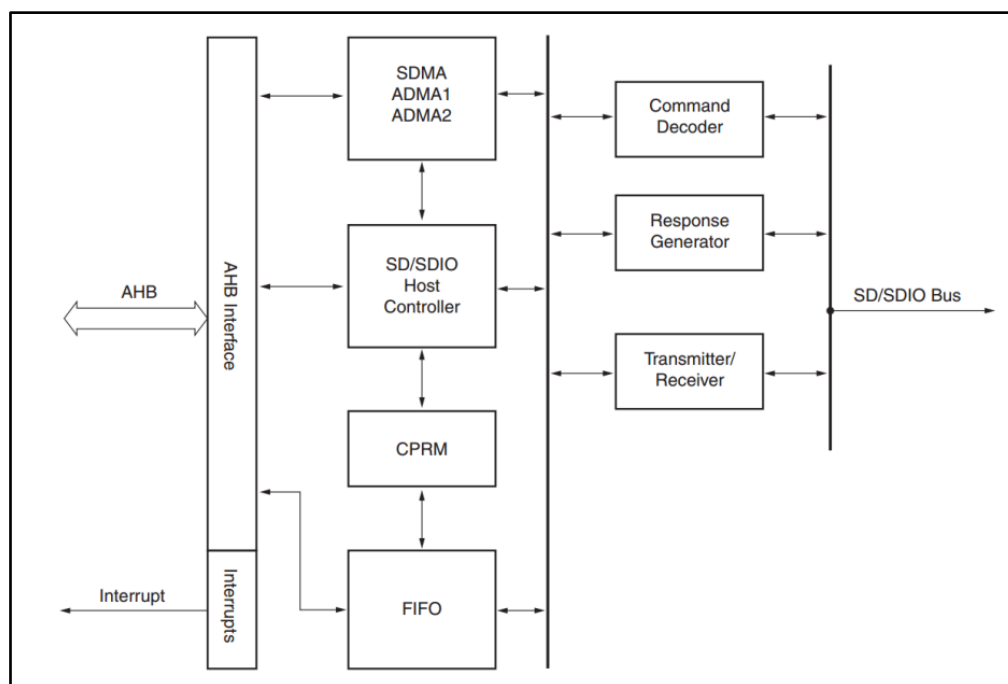


Figure 12-11 SD controller internal structure diagram

In the SD controller, the read and write channels utilize an independent 512-byte deep double -buffered FIFO to perform data read and write operations. Specifically, during a write operation, t he processor writes data to one of the FIFOs via the AHB bus while simultaneously transferring d ata from the other FIFO to the SD bus. In a read operation, data is written from the SD bus to one FIFO and the processor reads data from another FIFO via the AHB bus. This double-buffered FIF O design effectively ensures the continuity of data transmission and maximizes bandwidth, impro ving data processing efficiency.

12.2.5 FATFS file system

FatFs is a general-purpose file system module for implementing FAT file systems in small em bedded systems. FatFs is written in accordance with ANSI C, so it does not depend on the hardwa re platform and has good hardware platform independence. It can be embedded into various micro controllers, such as 8051, PIC, AVR, SH, Z80, H8, ARM, etc. without any modification. The stan dalone of Xilinx SDK has already ported the FATFS file system. Therefore, developers only need to introduce the xilffs library into the SDK to manage and operate the SD card through the API fun ctions provided by FATFS.

The characteristics of FATFS are as follows:

1. Clear structure and less code, the file system is separated from the IO bottom layer, very suitable for beginners;
2. Can support up to 10 logical drive letters and two-level folders;
3. Support FAT12/FAT16 and FAT32 format file systems;
4. Support the use of long file names.



Figure 12-12 FATFS hierarchy diagram

From Figure 12-12, we can see that the structure of the FATFS file system is divided into three levels

Application layer: At the top level, users perform file operations through a series of simple API interface functions, such as `f_open`, `f_read`, `f_write` and `f_close`. These operations are similar to how files are processed on a PC. Users do not need to have an in-depth understanding of the internal architecture of FATFS or the intricacies of the FAT protocol.

FATFS module layer: This middle layer is responsible for implementing the read and write protocols for FAT files. The core of the FATFS module is the `ff.c` and `ff.h` files. Normally, users do not need to modify these files, but can directly include the `ff.h` header file into their own projects.

Low-level interface: The FATFS module provides a low-level interface for interacting with storage hardware, including disk I/O (i.e., the read and write interface of the storage medium) and the real-time clock function that provides file creation and modification time.

Regarding the introduction and explanation of API functions, you can refer to the following website for a deeper understanding. This website provides instructions and examples for each API function. http://elm-chan.org/fsw/ff/00index_e.html

12.3 Practical exercises

12.3.1 Experimental objectives

Through the FATFS library that comes with Xilinx SDK, the function of reading and writing TXT text in the TF card is completed, and the reading and writing test results are printed out through the serial port.

12.3.2 Hardware resources

This experiment is to perform a read and write test on the SD card. The schematic diagram and hardware diagram of the connection between the SD card and the development board used this time are shown in Figure 11-3 and Figure 12-14.

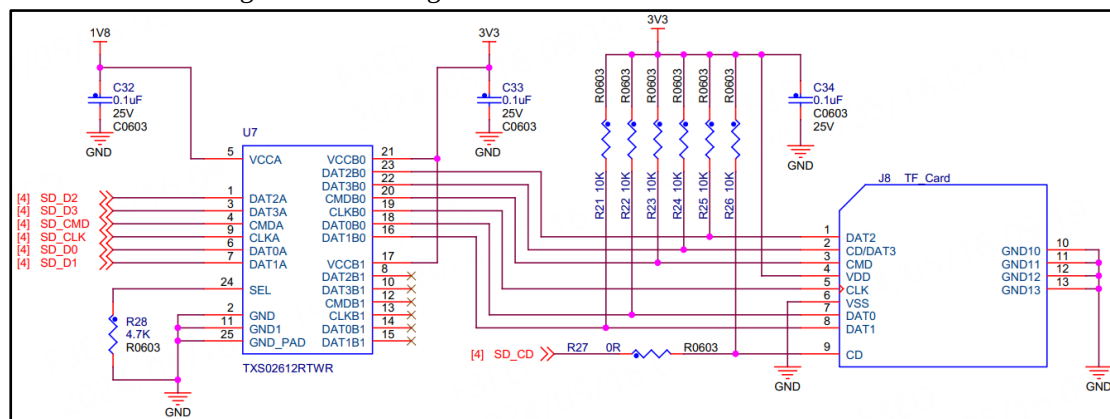


Figure 12-13 SD pin diagram

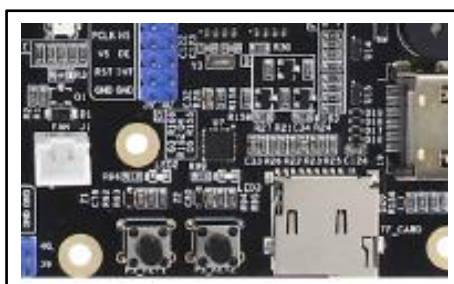


Figure 12-14 Actual picture of TF card holder

12.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 12-15.

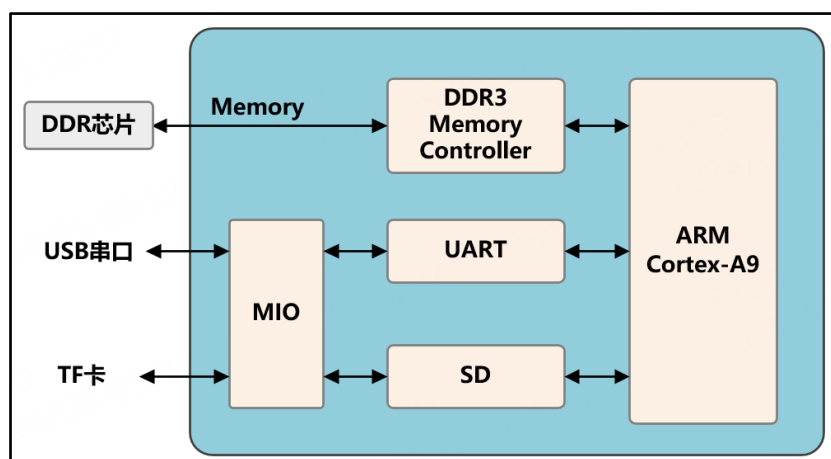


Figure 12-15 Overall block diagram

2. Hardware design

We just add an SD card controller to the minimal system.

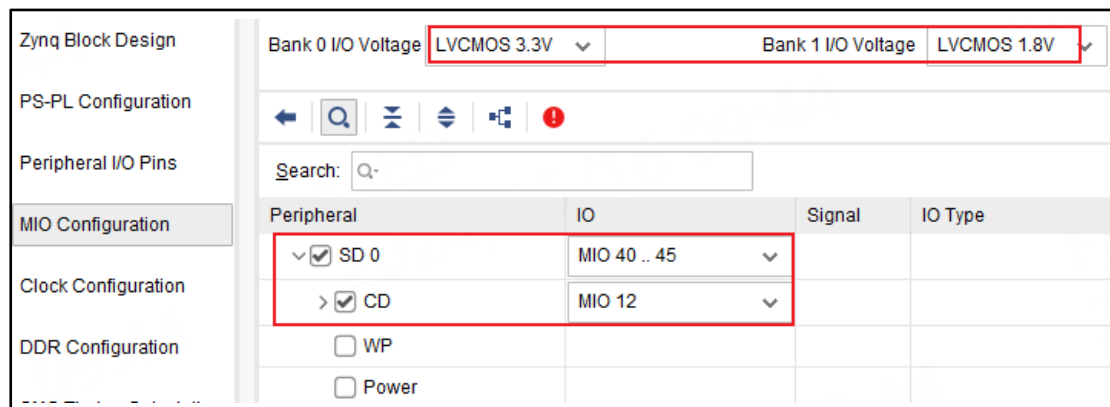


Figure 12-16 Add SD card controller

The following operations are similar to the first chapter and will not be repeated here.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, wrote software code, and implemented the corresponding functions.

This experiment requires opening the xilffs library in BSP and introducing the ff.h header file as follows:

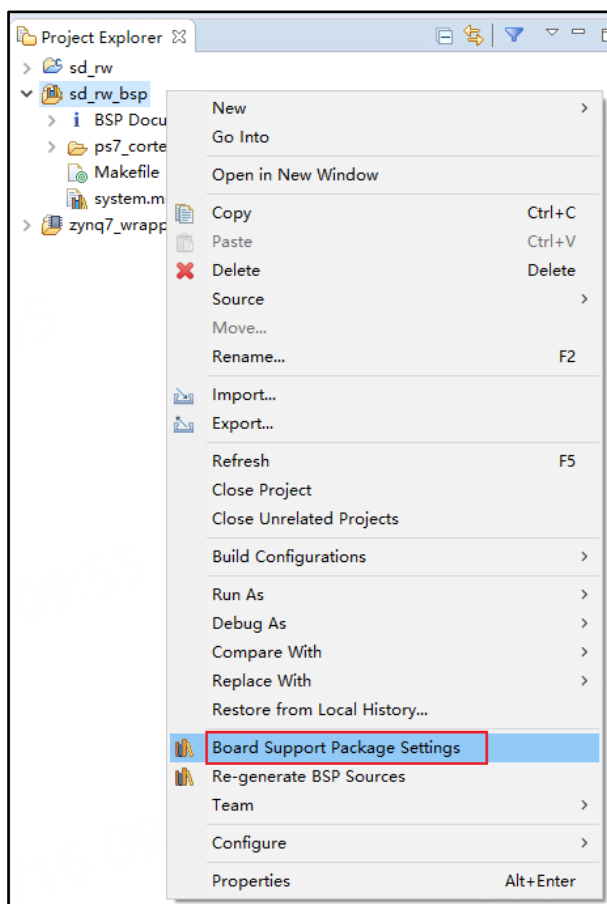


Figure 12-17 BSP opening diagram

Then in the pop-up tab, check xilffs and enter the configuration.

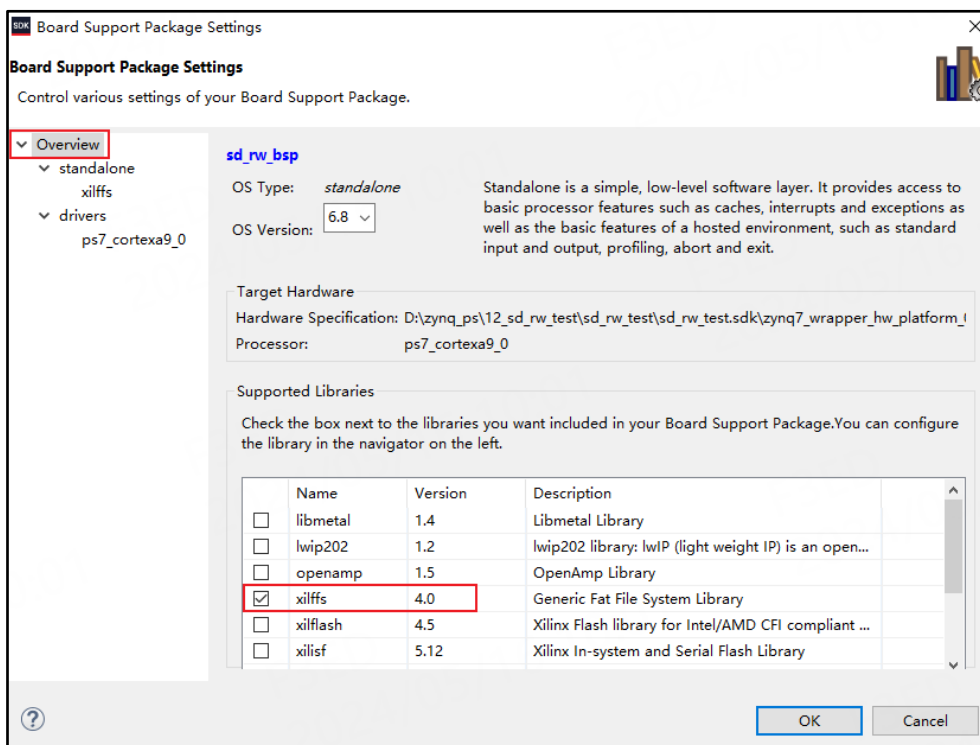


Figure 12-18 Add xilffs

To support long file names, change the value of use_lfn to true.

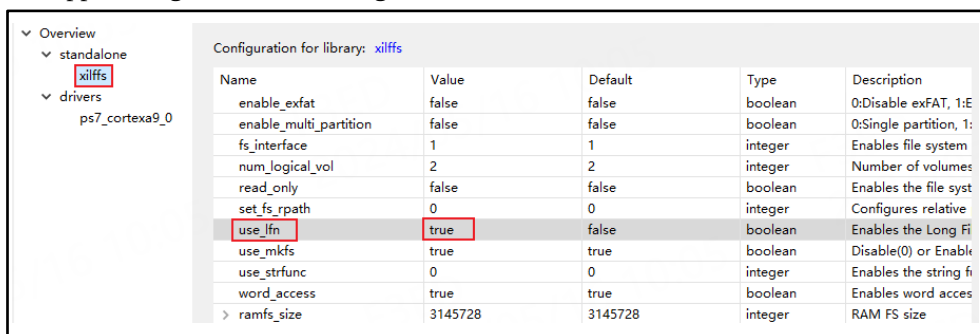


Figure 12-19 Support long file names

Next, start writing code.

Code List 12-1 SD card read and write test reference code (main.c)

```
1 #include "xparameters.h" 2 #include "xil_printf.h" 3 #include "xdevcfg.h" 4 #include "ff.h" 5 6 const char
TXT_Content[30] = "I'm LZR, how are you?"; 7 static FATFS fatfs; //File system 8 9 FILE Txt; //Define file
object 10 11 int Mount_Init() 12 { 13 TCHAR *path = "0:/"; 14 unsigned char work[512]; 15 16 if (f_mount(
&fatfs, path, 1) != FR_OK) { //Mount SD card 17 xil_printf("Mount failed, Maybe not fat format.\r\n"); 18 //
Format SD card 19 if (f_mkfs(path, FM_FAT32, 0, work, sizeof(work)) != FR_OK) { 20 xil_printf("Failed t
o format FAT\r\n"); 21 return -1; 22 } 23 //Remount 24 if (f_mount(&fatfs, path, 1) != FR_OK) { 25 xil_prin
tf("Failed to mount FAT\r\n"); 26 return -1; 27 } 28 } 29 xil_printf("SD card mount success!\n"); 30 return 0;
31 } 32 33 34 void FILE_Write(char *file_name, u32 src_addr, u32 byte_len) 35 { 36 UINT WRITE_Byte; 37
/*Open the file with write permission, and create it if there is no corresponding file*/ 38 f_open(&Txt, file_na
me, FA_CREATE_ALWAYS | FA_WRITE); 39 /*Move the file data pointer to the beginning*/ 40 f_lseek(
&Txt, 0); 41 /*Write the content and return the number of written bytes*/ 42 f_write(&Txt, (void*) src_addr, b
yte_len, &WRITE_Byte); 43 /*Close the file*/ 44 f_close(&Txt); 45 }
```

```
46 47 void FILE_Read(char *file_name,u32 src_addr,u32 byte_len) 48 { 49 UINT READ_BYTE; 50
/*Open the file with read permission*/ 51 f_open(&Txt,file_name,FA_READ); 52 /*Move the data p
ointer to the beginning*/ 53 f_lseek(&Txt,0); 54 /*Read file content*/ 55 f_read(&Txt,(void*)src_add
r,byte_len,&READ_BYTE); 56 /*Close file*/ 57 f_close(&Txt); 58 } 59 60 61 int main() 62 { 63 cha
r TXT_Buffer[30]; 64 65 if (Mount_Init() < 0) //Mount sd card 66 return -1; 67 68 //Write file FIRE.t
xt 69 FILE_Write("FIRE.txt",(u32)TXT_Content,strlen(TXT_Content)); 70 //Read file FIRE.txt 71 F
ILE_Read("FIRE.txt",(u32)TXT_Buffer,strlen(TXT_Content)); 72 73 //Compare read and write conte
nt 74 if (strcmp(TXT_Content, TXT_Buffer) == 0) 75 xil_printf("Success! write and read identical.\n
"); 76 else 77 xil_printf("fail ! write and read inequality.\n"); 78 79 return 0; 80 }
```

In lines 6-9 of code, we define the array of content to be written. The file system and file objects are defined again.

In lines 16-28 of the code, it is used to determine whether our development board has mounted the SD card. If it is not inserted, it will cause the SD card to fail to mount. If it is unsuccessful, the SD card will be formatted and the SD card will be remounted.

Then write the FILE_Write() function, in lines 36-44 of the code, to open a file in the SD card. If it does not exist, create it, and write the array defined at the beginning of the program into the file. Finally, use f_lseek to move the read-write pointer of the file object to the beginning of the file. 0 means the beginning of the file. Then use f_write() to write data to the SD card. After writing, use f_close() to close the file.

In the FILE_Read() function, in lines 49-57 of the code, the corresponding file is also opened, then the pointer is moved to the beginning, and then the data is read out.

Finally, we judge in main whether the contents of the two are the same. And print the result through the serial port.

After writing is completed, compile the project.

12.3.4 Board verification

As shown in Figure 12-20, the development board is connected. After the lines are connected correctly, turn on the

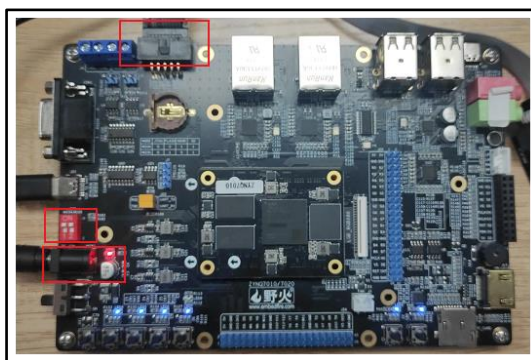


Figure 12-20 Program download connection diagram

Then download the program to the development board. Right-click the application project name in the project and select Run As > Launch on Hardware (system Debugger).

After the program download is completed, open the serial port assistant and you can see the relevant information p

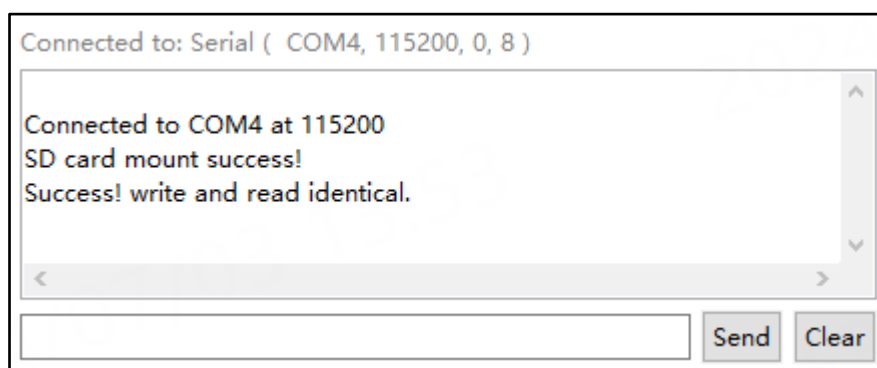


Figure 12-21 SD card read and write test

When the program is run, if the result shown in the picture above appears, it means that the SD card read and write

12.4 End of chapter summary

This chapter popularizes relevant knowledge about SD, and also introduces ZYNQ SD control and FATFS file system. Finally, the FATFS library that comes with Xilinx is added to the SD K, and the function of reading and writing TXT text in the TF card is realized with the help of this library.

12.5 Development training

In this experiment, we mainly introduced the use of basic API functions of FATFS, such as file opening, reading, writing, closing, etc. These functions are the basis for file operations, but FATFS is much more than that. The FATFS library provides a rich set of APIs to support more complex and advanced file system operations. If you are interested, you can learn more about the use of FATFS and related knowledge on your own.

Chapter 13 eMMC Reading and Writing Test Experiment

13.1 Chapter Introduction

eMMC, or Embedded Multi Media Card, provides a built-in storage solution for embedded systems, mobile devices and consumer electronics products. As a non-volatile memory that complies with MMC Association standards, eMMC provides a unified MMC interface. It not only increases storage space, but also supports complex operating systems and applications through high-speed data transmission capabilities, thereby improving device performance and user experience. Its compact design and high performance make eMMC the preferred built-in storage solution for many portable devices.

Since the interface standard of eMMC is the same as that of SD card, we can operate eMMC similarly to SD card. In this chapter, we will use the Haoyue development board to learn how to read and write eMMC.

13.2 Theoretical study

eMMC integrates high-density NAND Flash and device controller (Flash controller) into a single BGA chip through a unified MMC standard interface, effectively saving PCB motherboard space. The Flash controller in this design is responsible for managing NAND Flash. Compared with directly connecting NAND Flash to the host (such as the ZYNQ development board), eMMC can shield the physical characteristics of NAND Flash and reduce the complexity of host-side software processing, allowing the host to focus on application development without the need for special processing of NAND Flash. In addition, eMMC uses technologies such as cache and memory arrays to achieve significantly better read and write performance than traditional NAND Flash.

In comparison, SD cards and eMMC have similarities in interfaces and transmission protocols, but SD cards serve as external storage devices and are easy for users to plug in and out, while eMMC is directly welded to the motherboard, providing a more stable connection and higher reading and writing speeds. Therefore, eMMC can be thought of as an SD card fixed to the motherboard.

The eMMC chip model of the ZYNQ core development board is KLM8G1GETF, which has a storage capacity of 8GByte and is suitable for storing user data. If you look at the connection diagram between the development board and the eMMC chip, you will find that we only connected the 4 data lines of the EMMC, and the upper 4 bits were not connected. This is because the data bit width of the ZYNQ SDIO controller is 4 bits.

As the eMMC protocol is gradually upgraded, its bus speed continues to increase. To ensure compatibility with older eMMC devices, all devices first use Backward Compatible Mode by default after initialization or reset. Once the eMMC device initialization is completed, the host can switch the device to high-speed mode through a certain process. In compatibility mode, the eMMC clock frequency is 0 to 26MHz; after switching to high-speed mode, the clock frequency can reach up to 200MHz.

13.3 Practical exercises

13.3.1 Experimental objectives

Through the FATFS library that comes with Xilinx SDK, the function of reading and writing eMMC is completed, and the reading and writing test results are printed out through the serial port.

13.3.2 Hardware resources

This experiment is to perform a read and write test on the eMMC card. The schematic diagram and physical diagram of the wiring between the eMMC card and the development board used this time are shown in Figure 13-1 and Figure 13-2.

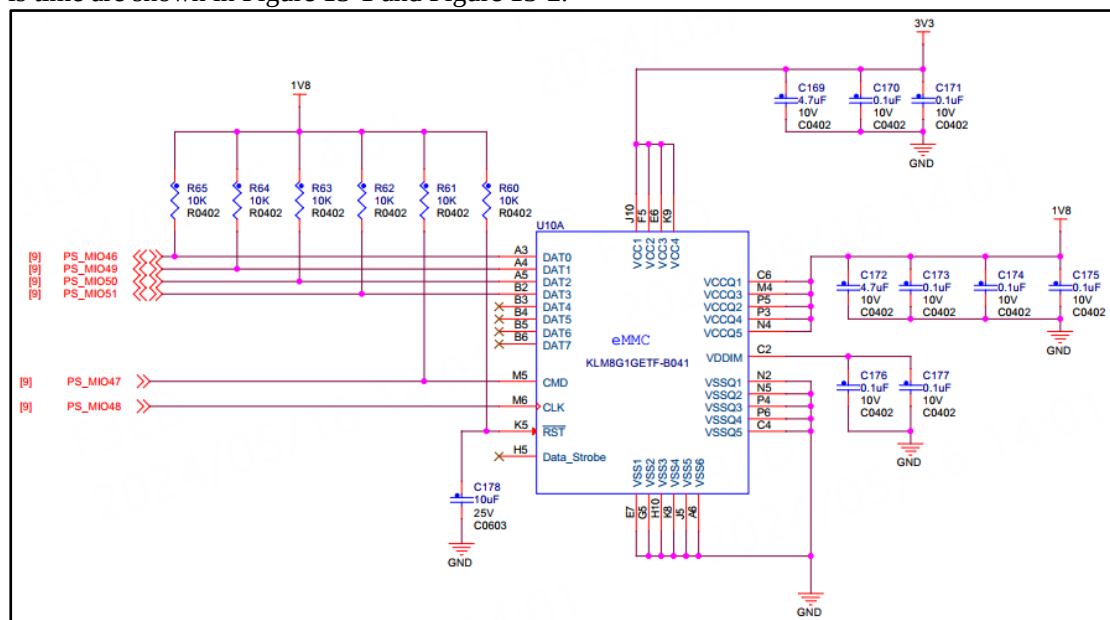


Figure 13-1 eMMC pin diagram

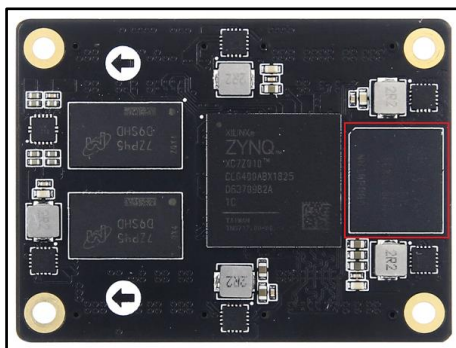


Figure 13-2 eMMC physical picture

13.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 13-3.

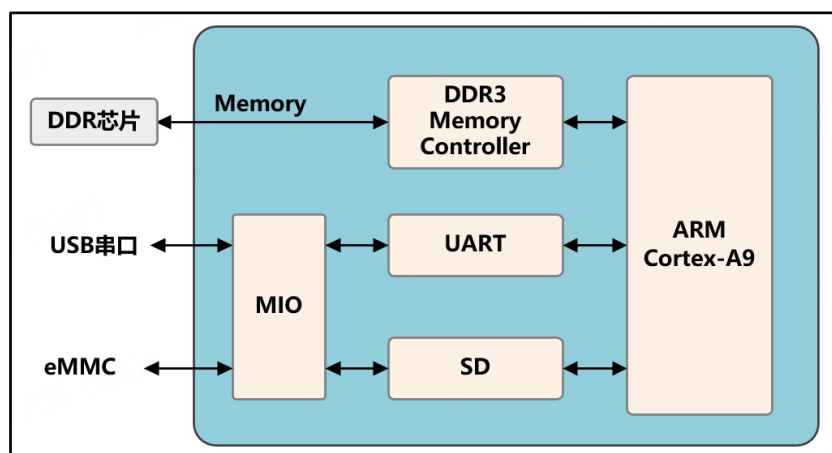


Figure 13-3 Overall block diagram

2. Hardware design

We just add an SD controller to the minimum system.

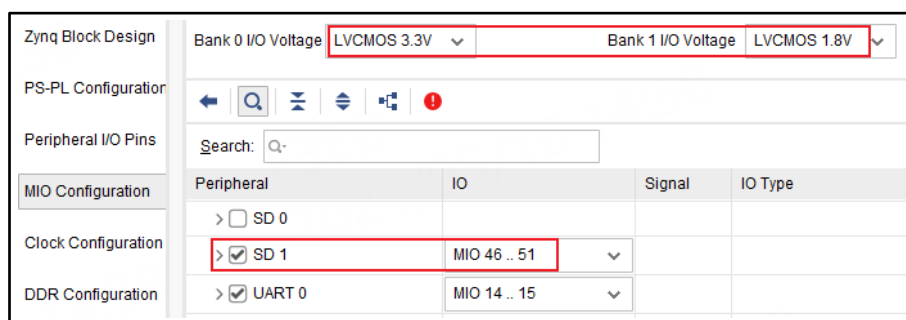


Figure 13-4 Add SD card controller

The following operations are similar to the first chapter and will not be repeated here.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project and wrote code. The code this time is exactly the same as the SD card read and write test in the previous section. Just copy the code over.

13.3.4 Board verification

As shown in Figure 13-5, the development board is connected. After the lines are connected correctly, turn on the s

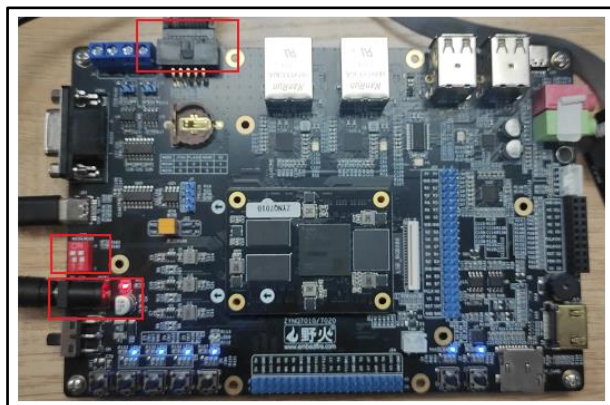


Figure 13-5 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system -> Run.

After the program download is completed, the serial port outputs information as shown in Figure 13-6, indicating t

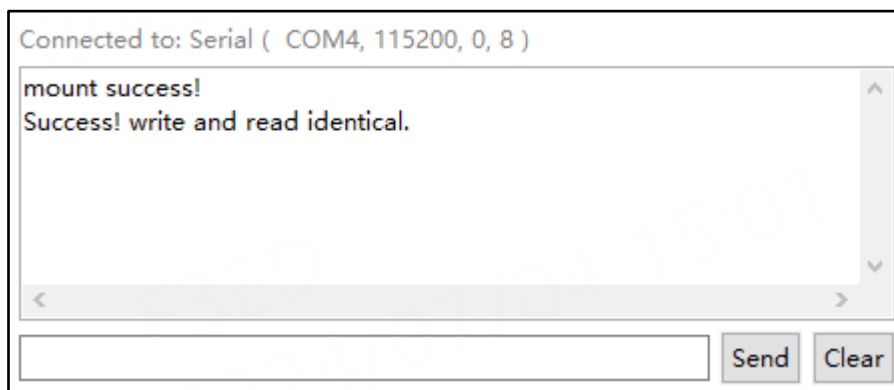


Figure 13-6 Serial port output

13.4 End of chapter summary

In this chapter, we introduced the relevant knowledge of eMMC, and completed the reading and writing test of eMMC based on the Fatfs library that comes with xilinx sdk.

13.5 Development training

Whether the eMMC read and write test is successful can also be indicated by LED lights, which makes it easier to

Chapter 14 Dual-Core AMP Experiment

14.1 Chapter Introduction

The previous routines all use a single-core CPU. In some cases, such as multi-tasking, parallel processing, etc., a dual-core CPU is required. The ZYNQ 7000 SOC chip integrates two independent dual-core Cortex-A9 core processors, which are just capable of complex tasks. In this chapter, we will use the Haoyue development board to learn dual-core AMP experiments.

14.2 Theoretical study

In the classification of multi-core processors, there are two main types: homogeneous multi-core architecture and heterogeneous multi-core architecture. Each core of the homogeneous multi-core architecture has the same structure, similar performance and functions, so it is relatively simple in design and implementation, and has high versatility. Heterogeneous multicore architectures contain cores of different architectures, often combining general-purpose processors with specialized cores optimized for specific uses. This configuration allows the heterogeneous multi-core architecture to show stronger performance advantages in specific areas, especially applications that rely on hardware acceleration.

Xilinx's ZYNQ SoC combines homogeneous and heterogeneous multi-core architectures. ZYNQ SoC contains two Cortex-A9 processor cores with the same structure and integrates a programmable logic unit (PL). This design makes the entire ZYNQ system a heterogeneous multi-core system, which not only has high versatility, but also provides stronger performance.

For multi-core processors, at the software level, its operating modes can be divided into AMP (asymmetric multi-processor) and SMP (symmetric multiprocessing). There are three operating modes: SMP (symmetric multiprocessing) and BMP (constrained multiprocessing).

AMP: refers to multiple processor cores running different tasks relatively independently. Each core is isolated from each other and can run different operating systems.

SMP: refers to multiple processors running an operating system, and this operating system manages multiple processors.

BMP: Similar to SMP mode, but developers can specify that a task be executed only on specified processor cores.

Typically, AMP run mode incurs almost no additional overhead, even when running bare metal applications, making it ideal for applications with high real-time requirements. However, achieving sharing of processor resources and communication between processors still requires a carefully designed software architecture. For example, in power control and protection equipment, complex human-machine interface communication and high real-time calculations are often required. For this reason, the AMP operating mode is usually used, in which one processor runs the Linux operating system and the other processor runs bare metal applications, thereby simultaneously meeting the complex functional and real-time requirements of power system control equipment.

SMP provides a unified operating system platform for advanced applications, allowing developers to build applications without worrying about resource sharing and process communication between different cores. However, the SMP system is not without cost, and the performance overhead it introduces may have a greater impact on applications with high real-time requirements. For example, multi-core processors in PC computers generally operate in SMP mode and need to process complex functions, but do not have high real-time requirements.

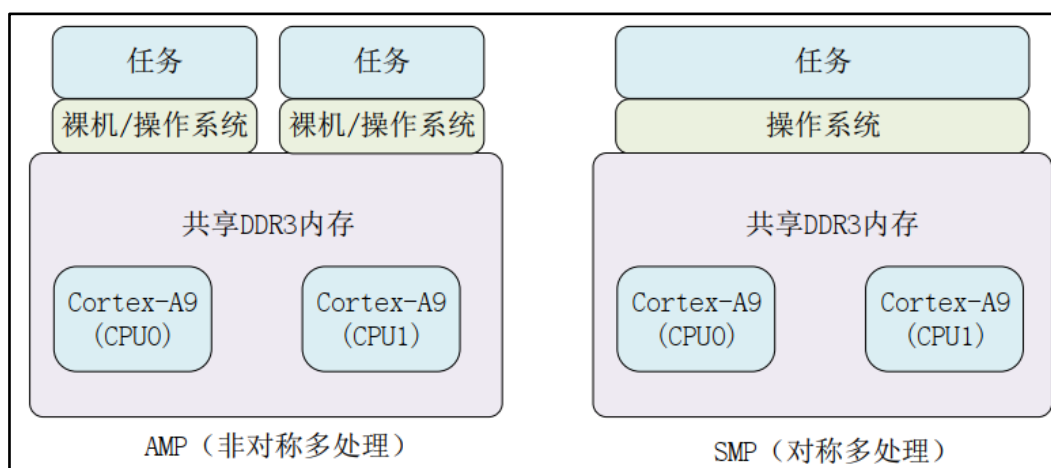


Figure 14-1 AMP and SMP operating modes

Since AMP is mostly used for bare metal development and can run different applications on two CPUs, this project mainly uses this method for dual-core communication. For AMP related designs, you can refer to the XAPP1079 document for design verification.

The ZYNQ-7000 AP SoC offers two Cortex-A9 processors that share common memory and peripherals. Asymmetric multiprocessing (AMP) is a mechanism that allows two processors to run their own operating systems or bare metal applications and loosely couple these applications through shared resources.

In AMP mode, each CPU can access its own independent program tasks, but care must be taken to prevent CPU conflicts on shared resources, otherwise errors will occur.

For this architecture, the CPU will have some private resources and shared resources, as follows:

(1) Private resources: L1 cache, Private peripheral interrupts (PPI), Memory management unit (MMU), Private timers. (2) Shared resources: Interrupt control distributor (ICD), DDR memory, On-chip memory (OCM), Global timer, Snoop control unit (SCU) and L2 cache, UART0 (serial port).

In AMP mode, the two processors run independently and there is no conflict between private resources. However, for shared resources, care must be taken to avoid simultaneous access to prevent conflicts and ensure the normal operation of the program. The two processors communicate through OCM. OCM provides extremely high performance and low-latency dual-processor access compared to DDR memory. Access determinism is further ensured by disabling cache access to the OCM for both processors.

Measures taken to prevent problems with shared resources include:

DDR memory: CPU0 and CPU1 use memory with independent addresses.

L2 cache: Disable CPU1 from using the L2 cache. If CPU1 wants to use the L2 cache, it needs to request an L2 cache flush and invalidation from CPU0, which CPU0 will perform.

ICD: Interrupts from the PL core are routed to the PPI controller of CPU1. By using PPI, CPU1 is free to handle interrupts without accessing the ICD.

Timer: CPU1 uses a dedicated timer.

OCM: Each CPU handles access to the OCM very carefully to prevent contention. A single OCM address location is used as a flag to communicate between the two processors. CPU0 initializes the flag to 0 before starting CPU1. When the flag is zero, CPU0 owns the UART. When the flag is non-zero, CPU1 owns the UART. Only CPU0 sets the flag, only CPU1 clears it.

We have learned about interrupts before, and we can use software-generated interrupts (SGI, Software Generated Interrupts) to avoid access conflicts. In a ZYNQ system, each CPU can use SGI to interrupt itself, another CPU, or both CPUs simultaneously. To generate an SGI, simply write the interrupt number to the software-generated interrupt register (ICDSGIR) and specify the target CPU. Each CPU has a set of SGI registers that can generate 16 different software interrupts. Figure 14-2 shows the SGI interrupt ID numbers, which range from 0 to 15.

IRQ ID#	Name	SGI#	Type	Description
0	Software 0	0	Rising edge	A set of 16 interrupt sources that are private to each CPU that can be routed to up to 16 common interrupt destinations where each destination can be one or more CPUs.
1	Software 1	1	Rising edge	
~	...	~	...	
15	Software 15	15	Rising edge	

Figure 14-2 Software interrupt

SGI can trigger the CPU to generate an interrupt, but if data needs to be transferred, shared memory must be used for data interaction. Shared memory refers to reserving a memory area with a known address and length in the memory for CPU0 and CPU1. The two CPUs communicate by reading and writing data in the shared memory. Using software interrupts to trigger the CPU avoids simultaneous accesses to shared memory.

The OCM (on-chip memory) in ZYNQ includes 256KB RAM and 128KB ROM (ie BootRom), of which the part that can be rewritten by two CPUs is 256KB RAM. OCM is divided into four 64KB RAM blocks, which has a relatively small storage space, while external DDR3 memory usually has a larger storage capacity. When two CPUs need to interact with a large amount of data, DDR3 memory can be used as shared memory; when the amount of data interaction is small, OCM can be used as shared memory or DDR3 memory can be selected.

It is important to note that although both CPUs run on DDR3 memory, they use OCM as shared memory for communication rather than using DDR3 memory. It is worth mentioning that whether you choose OCM or DDR3 memory as shared memory, you need to disable the cache function of the storage space before accessing it. This ensures that the CPU can read the latest data in the memory address in time to avoid consistency problems when two CPUs access shared memory. Cache attributes can be disabled by calling the `Xil_SetTlbAttributes(INTPTR Addr, u32 attrib)` function. The first parameter of this function is the base address of the shared memory, and the second parameter is used to set memory attributes, including whether to disable cache, etc.

Xilinx officially provides two application notes on dual-core AMP: "XAPP1078" and "XAPP1079". Among them, "XAPP1078" describes the application situation of one core running the Linux operating system and the other core running bare metal applications; while "XAPP1079" describes the application situation of two cores running bare metal applications in their own independent environments.

The function of the experiment in this chapter is to allow the two cores to run independent bare metal applications, so the "XAPP1079" document is particularly worthy of reference. This document details the startup sequence and method of the two CPUs in the ZYNQ chip, as well as the configuration in the bare metal SDK environment.

14.1 Practical exercises

14.1.1 Experimental objectives

The goal of this experiment is ZYNQ dual-core communication. The specific idea is: CPU0 receives the data from the serial port, writes it to OCM, and then initiates a soft interrupt to CPU1. After receiving the interrupt, CPU1 reads the data in the OCM to control the frequency of the breathing light and initiates a soft interrupt to CPU0.

14.1.2 Hardware resources

This experiment used the internal resources of the ZYNQ chip.

14.1.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in the figure below.

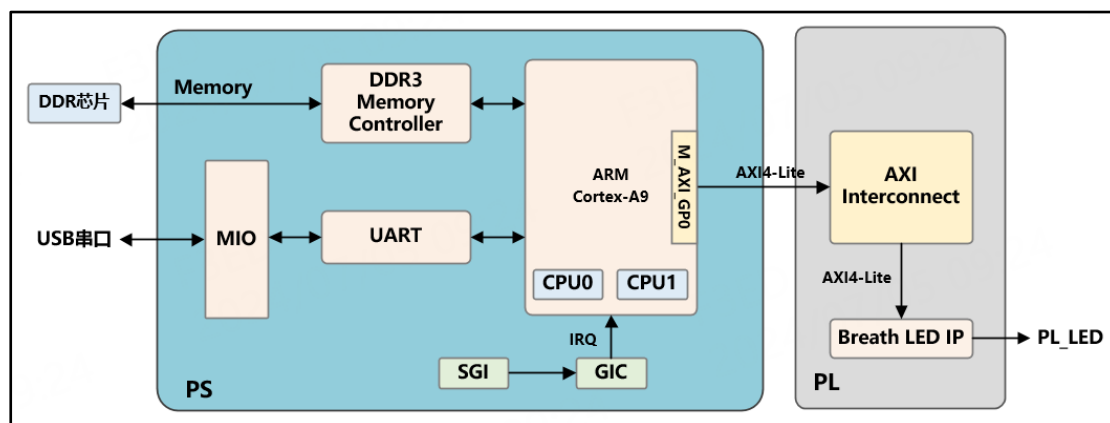


Figure 14-3 Overall block diagram

2. Hardware design

In this experiment, both CPUs will use the serial port. CPU0 will write to OCM and CPU1 will read from OCM. This requires that shared peripherals and memory cannot be accessed at the same time to avoid conflicts. We can use software generated interrupts (SGI) to avoid this conflict.

In the initial state, CPU0 uses the serial port first. After receiving the user data, it writes the data to OCM and generates an interrupt to trigger the CPU1 interrupt. At this time, CPU0 no longer accesses the serial port and OCM. After receiving the interrupt, CPU1 starts to access the serial port and OCM, first reads the data from the OCM, and outputs the information through the serial port. After completing these operations, CPU1 will generate an interrupt to trigger the CPU0 interrupt and stop accessing the serial port and OCM.

When CPU0 receives the interrupt from CPU1, it can reuse the serial port to receive user input data. In this way we avoid simultaneous accesses to shared peripherals and memory,

We used the IP core of the breathing lamp. Here we can directly call the IP core of the breathing lamp that we have customized before. The dual-core AMP experiment is consistent with the use of single-core CPU in setting up the hardware environment, so we will not repeat it here. Only the final overall hardware diagram is given.

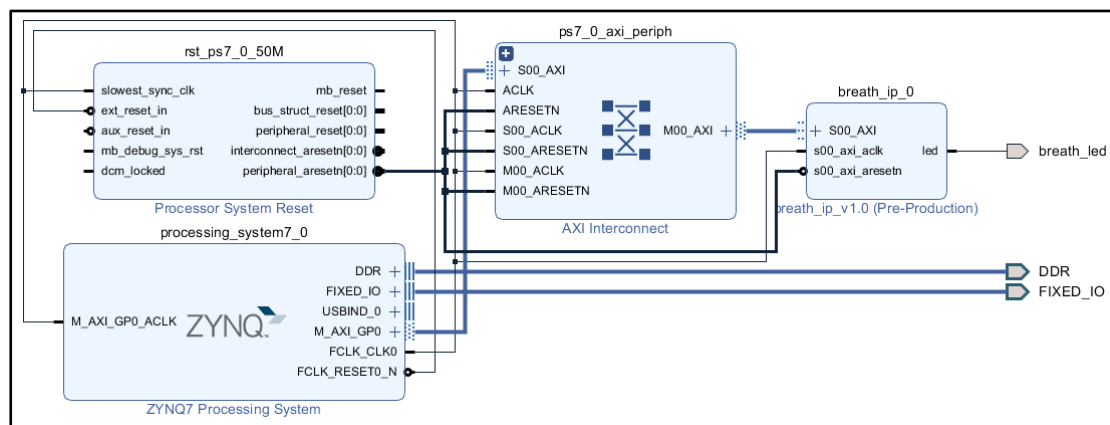


Figure 14-4 Dual-core AMP experimental block diagram

3. Software design

In the hardware design part, we started the SDK development environment. This time we need to create two different application projects. Compared with before, the operation is different. Create a new CPU0 project and select the Empty template, as shown below.

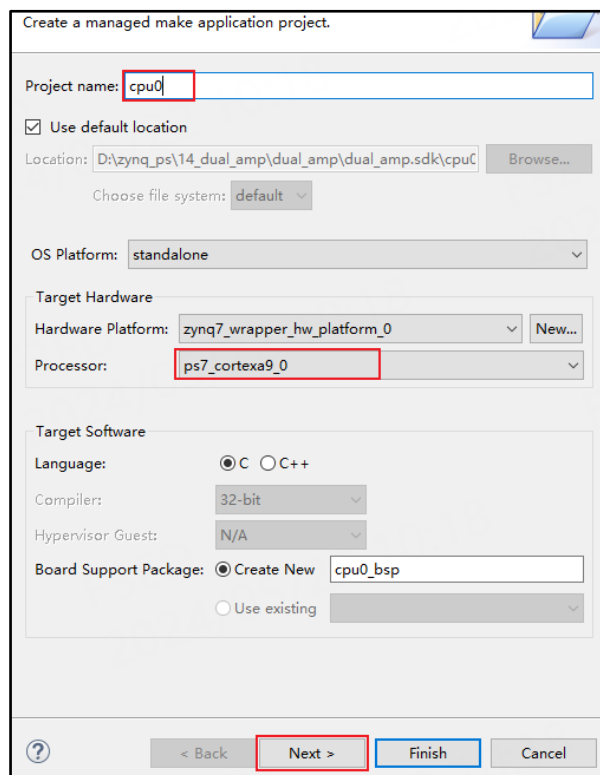


Figure 14-5 Create a new cpu0 project

At this time we need to modify the memory allocation, because CPU0 and CPU1 share the DDR3 memory space, so we need to allocate memory to them, as long as the allocation is reasonable. Configure 255Mbyte for CPU0, and give the rest to CPU1. We find the relevant configuration in `cpu0->src->lscrip.ld` and modify it, and `ps7_ram0` and `ps7_ram1` are OCM The base address and storage space size of the shared memory. `ctrl+s` save afterwards.

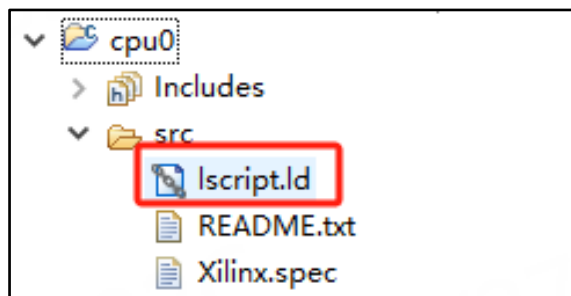


Figure 14-6 lscrip.ld diagram

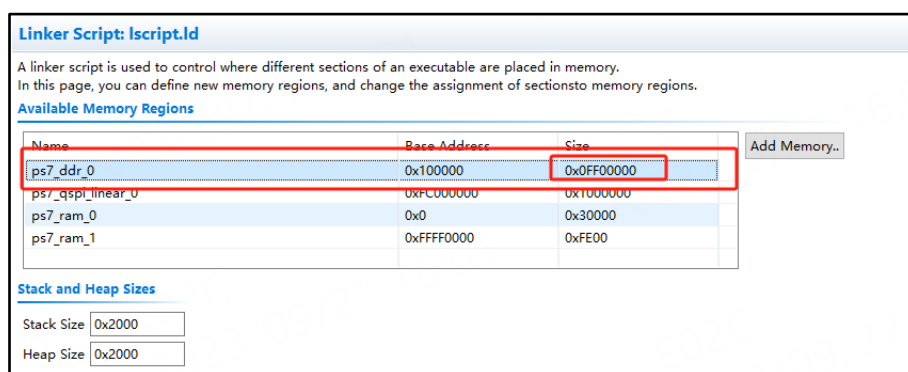


Figure 14-7 Setting the memory allocation of CPU0

Then create a new project for CPU1 and pay attention to modifying the Processon (processor) as follows:

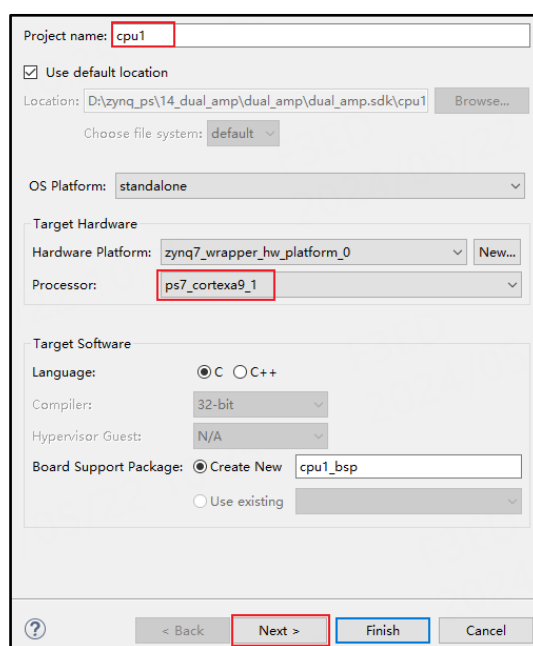


Figure 14-8 Create a new CPU1 project

Then continue to modify the memory allocation of CPU1, continue to open the corresponding lscript.ld file, and modify the memory allocation of DDR3. At this time, we need to change the starting address. Since the end of CPU0 is at 0X10000000, we start from 0X10000000 and the size is 0X30000000. As shown below:

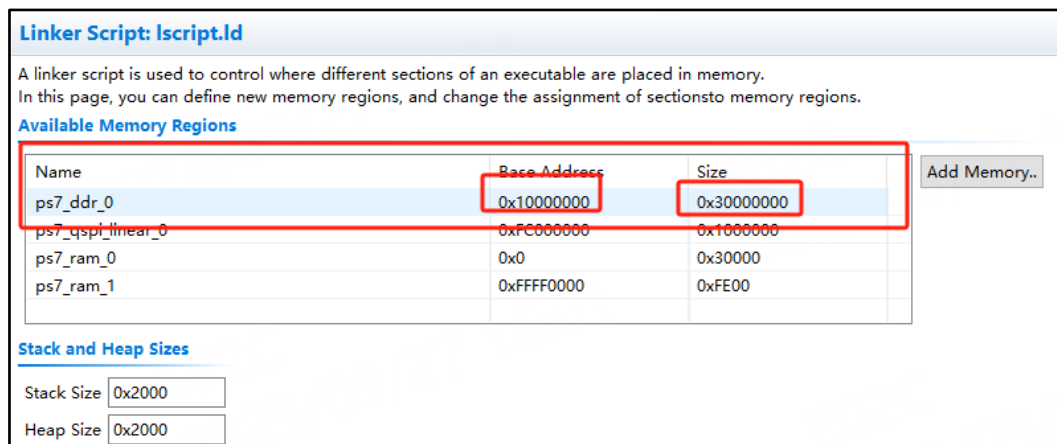


Figure 14-9 Modify CPU1 memory allocation

After that, you need to make some settings for the board-level support package of CPU1 to make the dual-core work.

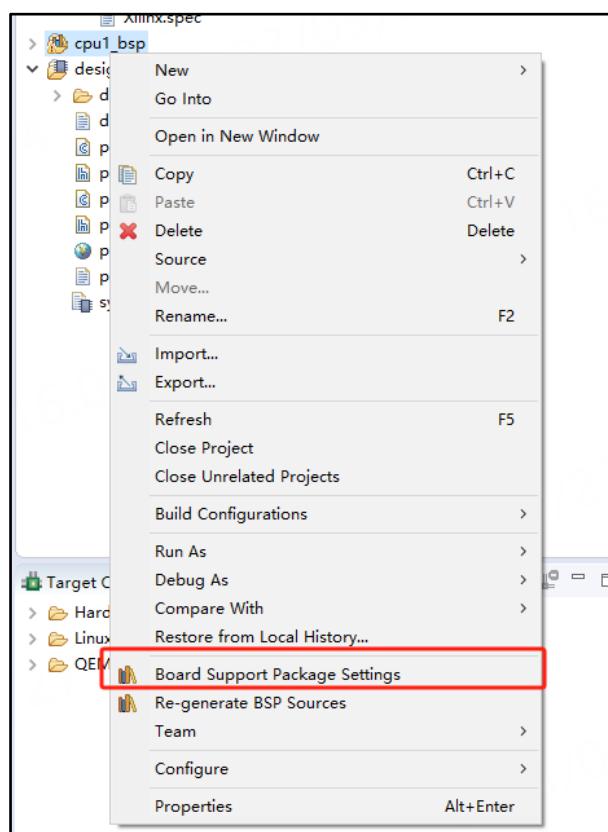


Figure 14-10 Opening the CPU1 board support package

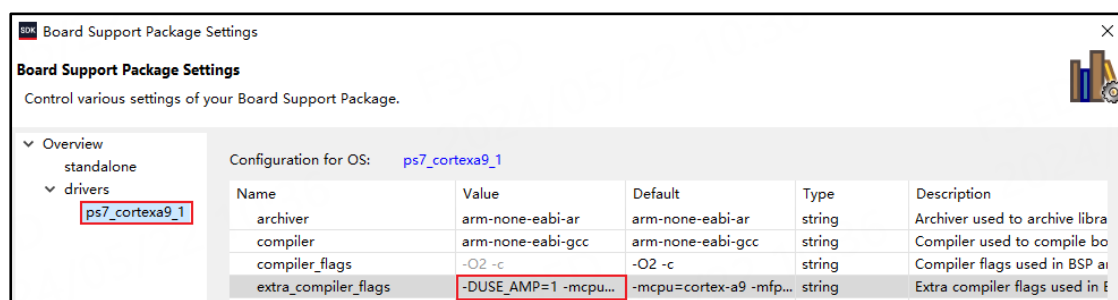


Figure 14-11 Setting up the CPU1 board support package

We add `-DUSE_AMP=1` in the Value column to turn on dual core. This configuration is actually to modify the cross-compilation tool chain of the project. In fact, it is equivalent to defining a macro `#define USE_AMP1`, but it is an operation that can be used in the entire project without including it.

This macro is related to L2 Cache. CPU1 and CPU0 share L2 Cache. In order to avoid unnecessary problems, we define `USE_AMP1` to make CPU1 use L2 Cache. Next, start writing code, create a new `cpu0.c`, `cpu1.c` file under each project, and write the following code:

Code Listing 14-1 CPU0 reference code (cpu0.c)

```

1 #include "xparameters.h"
2 #include "xscugic.h"
3 #include "xil_printf.h"
4 #include "xil_exception.h"
5 #include "xil_mmu.h"
6 #include "stdio.h"
7
8 //宏定义
9 #define INTC_DEVICE_ID      XPAR_SCUGIC_SINGLE_DEVICE_ID //中断 ID
10 #define SHARE_OCM_ADDR     0xffff0000 //共享 OCM 首地址
11 #define CPU1_BASE_ADDR     0xffffffff //CPU1 地址
12 #define CPU1_PRO_ADDR      0x10000000 //CPU1 程序读取地址
13
14 #define CPU1_ID             XSCUGIC_SPI_CPU1_MASK
15 #define CPU0SOFT_INTR_ID   0 //cpu0 的软件中断号
16 #define CPU1SOFT_INTR_ID   1 //cpu1 的软件中断号
17
18 // "SEV"指令唤醒 CPU1 并跳转至相应的程序
19 #define sev()               __asm__("sev")
20
21 void START_Cpu1();
22 void CPU0_Intr_init(XScuGic *intc_ptr);
23 void INTR_Handler(void *CallbackRef);
24
25 XScuGic Intc; //中断控制器驱动程序实例
26 int rec_freq_flag = 0; //接收到呼吸灯频率设置的标志
27 int freq; //频率设置
28
29 int main()
30 {
31     //S=b1 TEX=b100 AP=b11, Domain=b1111, C=b0, B=b0
32     Xil_SetTlbAttributes(SHARE_OCM_ADDR, 0x14de2); //禁用缓存 Cache
33
34     START_Cpu1(); //启动 CPU1
35     CPU0_Intr_init(&Intc); //CPU0 中断初始化
36     while(1){
37         if(rec_freq_flag == 0){

```

```
38 xil_printf("This is CPU0,Please input the numbers " 39 "0~3 to change breath led frequency\r\n"); 40 scanf(
"%d",&freq); 41 if(freq >= 0 && freq <=3){ 42 xil_printf("Your input number is %d\r\n",freq); 43 //Write to
OCM In shared memory 44 Xil_Out8(SHARE_OCM_ADDR,freq); 45 //Send software interrupt to cpu1 46 X
ScuGic_SoftwareIntr(&Intc,CPU1SOFT_INTR_ID,CPU1_ID); 47 rec_freq_flag = 1; 48 } 49 else{ 50 should
enter 0~3\r\n"); 51 xil_printf("\r\n"); 52 } 53 } 54 } 55 return 0 ; 56 } 57 58 /*Start cpu1*/ 59 void START_Cp
u1() 60 { 61 /* Send to CPU1_BASE_ADDR(0Xffffff0) 62 * The address is written to the access memory ba
se address of CPU1 */ 63 *intc_ptr) 70 { 71 //Initialize interrupt controller 72 XScuGic_Config *intc_cfg_ptr;
73 intc_cfg_ptr = XScuGic_LookupConfig(INTC_DEVICE_ID); 74 intc_cfg_ptr->CpuBaseAddress); 76 77 //
Set and turn on the interrupt exception handling function 78 Xil_ExceptionRegisterHandler(XIL_EXCEPTIO
N_ID_INT, 79 (Xil_ExceptionHandler)XScuGic_InterruptHandler, intc_ptr); 80 Xil_ExceptionEnable(); 81 82
//Set the interrupt service function 83 XScuGic_Connect(intc_ptr, CPU0SOFT_INTR_ID, 84 (Xil_ExceptionH
andler)INTR_Handler, (void *)intc_ptr); 85 86 XScuGic_Enable(intc_ptr, CPU0SOFT_INTR_ID); //CPU0 sof
tware interrupt 87 } 88 89 /*Interrupt service function*/ 90 void INTR_Handler(void *CallbackRef) 91 { 92 xi
l_printf("This is CPU0,Soft Interrupt from CPU1\r\n"); 93 xil_printf("\r\n"); 94 rec_freq_flag = 0; 95 }
```

In lines 10-16 of the code, we define SHARE_OCM_ADDR, which represents the base address of OCM's shared memory. We can see in lscript.ld that it is actually the address of ps7_ram0 or ps7_ram1. Here we use ps7_ram1 as the shared memory of CPU1 and CPU0. Then the starting address of the stored program of CPU1 is defined.

The address CPU1_BASE_ADDR is needed when CPU0 starts CPU1, and CPU1_PRO_ADDR is the application starting address of CPU1. Then the interrupt numbers of CPU0 and CPU1 are defined.

In the main function, use the Xil_SetTlbAttributes() function to disable the cache to maintain the consistency of OCM access between two CPUs.

Then there is a START_Cpu1 function. If it is not solidified, this function is not necessary. Direct Jtag download does not require this function.

CPU0_Intr_init() initializes the interrupt of CPU0 and is used to respond to the interrupt of CPU1. It is better to initialize the Gic controller first, and then associate the interrupt service function of CPU0. When receiving an interrupt from CPU1, set rec_freq_flag to 0 and receive user information.

In the while loop, if rec_freq_flag is 0, a string is output on the serial port to prompt the user whether this is CPU0. Press the number 0~3 to set different frequencies. The Xil_Out8() function writes the data to the OCM shared memory, then uses the XScuGic_SoftwareIntr() function to send an interrupt to CPU1, and then raises the rec_freq_flag to stop receiving user input and wait for CPU1 to respond. If the entered number is not 0~3, the user will be prompted to re-enter.

Code Listing 14-2 CPU1 reference code (cpu1.c)

```
1 #include "xparameters.h" 2 #include "xscugic.h" 3 #include "xil_printf.h" 4 #include "xil_exception
.h" 5 #include "xil_mmu.h" 6 #include "stdio.h" 7 #include "breath_led.h" 8 9 //Macro definition 10 #
define INTC_DEVICE_ID XPAR_SCUGIC_SINGLE_DEVICE_ID 11 #define SHARE_OCM_AD
DR 0xffff0000 //Shared OCM first address 12 13 #define CPU0_ID //Software interrupt number of cp
u1 16 17 //led base address 18 #define LED_IP_BASEADDR XPAR_BREATH_LED_0_S00_AXI_
BASEADDR 19 //led register address 0 20 #define LED_IP_REG0 BREATH_LED_S00_AXI_SLV_
REG0_OFFSET 21 //led register address 1 22 #define LED_IP_REG1 BREATH_LED_S00_AXI_SL
V_REG1_OFFSET 23 24 void CPU1_Intr_init(XScuGic *intc_ptr); 25 void INTR_Handler(void *Ca
llbackRef); 26 27 XScuGic Intc; //Interrupt controller instance 28 int soft_intr_flag = 0; //Software int
errupt flag 29 int freq; //Frequency setting 30 31 int main() 32 { 33 int freq_step = 0; 34 //S=b1 TEX=
b100 AP=b11, Domain=b1111, C=b0, B=b0 35 Xil_SetTlbAttributes(SHARE_OCM_ADDR,0x14de
2); //Disable cache Cache 36 37 /*CPU1 interrupt initialization*/ 38 CPU1_Intr_init(&Intc);
```

```
39 /*Turn on the breathing light enable frequency switch*/ 40 BREATH_LED_mWriteReg(LED_IP_BASEADDR, LED_IP_REG0, 1); 41 while(1){ 42 if(soft_intr_flag){ 43 //Read data from the shared OCM 44 freq = Xil_In8(SHARE_OCM_ADDR); 45 xil_printf("CUP1 Received data is %d\r\n",freq); 46 switch(freq){ 47 case 0 : freq_step = 0; break; 48 case 1 : freq_step = 50; break; 49 case 2 : freq_step = 200; break; 50 case 3 : freq_step = 400; break; 51 default : freq_step = 50; break; 52 } 53 //Set the breathing light frequency 54 BREATH_LED_mWriteReg(LED_IP_BASEADDR, LED_IP_REG1, (0x00000000 | freq_step)); 56 //Send software interrupt to cpu0 57 XScuGic_SoftwareIntr(&Intc,CPU0SOFT_INTR_ID,CPU0_ID); 58 soft_intr_flag = 0; 59 } 60 } 61 return 0 ; 62 } 63 64 //CPU1 interrupt initialization 65 void CPU1_Intr_init(XScuGic *intc_ptr) 66 { 67 //Initialize interrupt controller 68 XScuGic_Config *intc_cfg_ptr; 69 intc_cfg_ptr = XScuGic_LookupConfig(INTC_DEVICE_ID); 70 intc_cfg_ptr->CpuBaseAddress); 72 73 //Set and turn on the interrupt exception handling function 74 Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT, 75 (Xil_ExceptionHandler)XScuGic_InterruptHandler, intc_ptr); 76 Xil_ExceptionEnable(); 77 78 //Set the interrupt service function 79 XScuGic_Connect(intc_ptr, CPU1SOFT_INTR_ID, 80 (Xil_ExceptionHandler)INTR_Handler, (void *)intc_ptr); 81 82 INTR_Handler(void *CallbackRef) 87 { 88 xil_printf("This is CUP1,Soft Interrupt from CPU0\r\n"); 89 soft_intr_flag = 1; 90 }
```

The code for CPU1 is basically the same, except for initializing the LEDs. There are also differences in the interrupt service function and the main function.

First, in the main function, use `Xil_SetTlbAttributes()` to disable Cache. `CPU1_Intr_init()` is used to initialize and enable the interrupt function of CPU1 and the associated interrupt service function. After the interrupt is initialized, we use the `BREATH_LED_mWriteReg()` function to enable the breathing light.

Next in the while loop, when we receive the interrupt from CPU0, we pull the `soft_intr_flag` high, and use `Xil_in8()` to read the data in the OCM, and then judge the read data to change the frequency of the breathing light.

There are 4 different frequencies in total. After processing, an interrupt is sent to CPU0 and waits for the next trigger. And pull soft_intr_flag low and wait for CPU0 interrupt.

Compile the project and prepare for boarding.

14.1.4 Board verification

As shown in Figure 14-12, the development board is connected. After the lines are connected correctly, turn on the

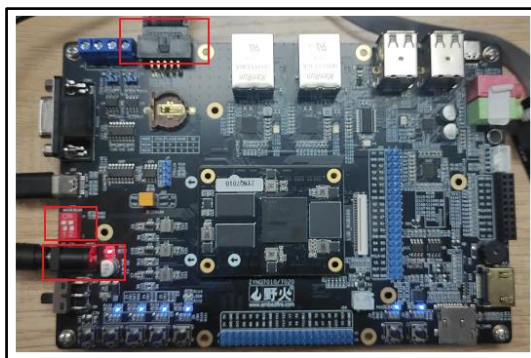


Figure 14-12 Program download connection diagram

As shown in Figure 14-13, download the program for the development board.

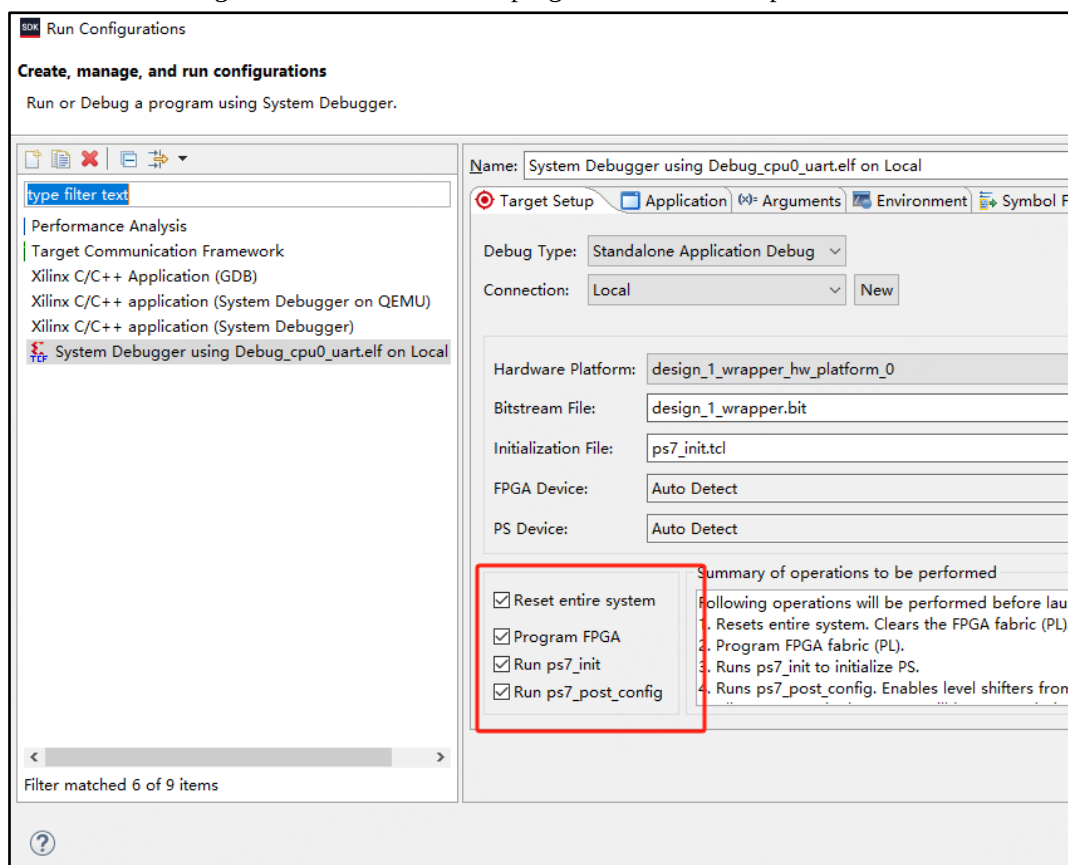


Figure 14-13 Program download diagram (1)

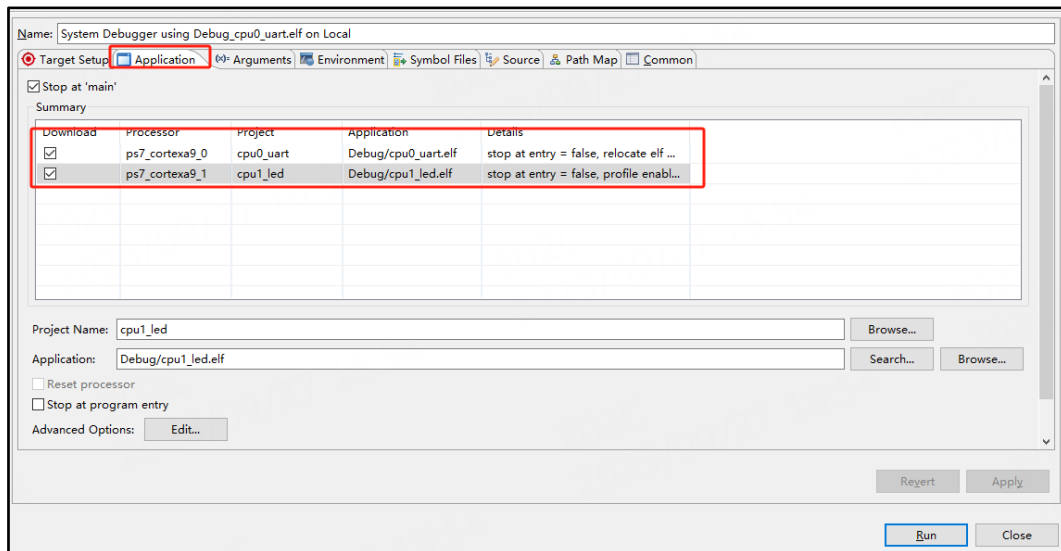


Figure 14-14 Download diagram (2)

When the program is first downloaded to the development board, the frequency switch of the breathing light is turned off and is always on. At this time, sending 0~3 through the serial port can control the frequency of the breathing light. The larger the value, the higher the frequency.

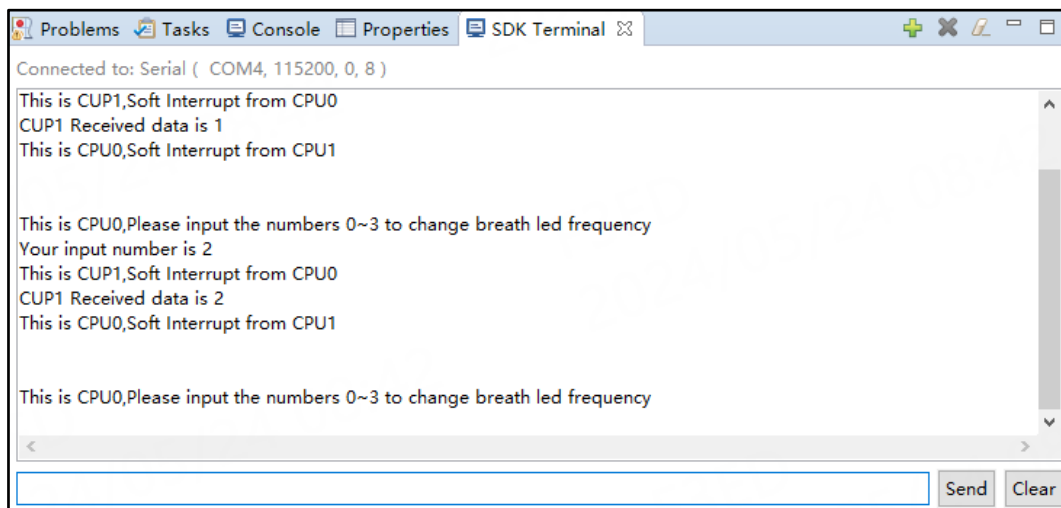


Figure 14-15 Serial port sending data

14.2 End of chapter summary

In the theoretical study of this chapter, we learned that there are two types of multi-core processors: heterogeneous and homogeneous, and there are AMP, SMP, and BMP operating modes in software. In AMP mode, different processor cores can run different operating systems or bare metal code, but conflicts must be avoided when accessing shared resources.

14.3 Development training

The PS LED is controlled through dual-core communication. CPU0 controls the LED to turn on and CPU1 controls the LED to turn off to achieve a flashing effect.

Chapter 15 Bram Interactive Experiment

15.1 Chapter Introduction

During the ZYNQ SoC development process, data interaction is often required between PL and PS. For those application scenarios with high transmission speed requirements, large data volume and continuous addresses, using AXI DMA is an effective solution. However, the applicability of AXI DMA is limited when the amount of data is small, the addresses are discontinuous, or the data length is irregular. To solve this situation, BRAM can be used for data exchange. In this chapter, we will explore how to implement data transfer between PS and PL via BRAM.

15.1 Practical drill

15.2 Theoretical study

BRAM is block RAM, located on the PL side. It is essentially the same as common RAM on the market. However, because it is embedded in the FPGA, its capacity is relatively small. However, for communication requirements with small data volumes and discontinuous addresses, BRAM has the advantage of being more flexible. We need to complete the data interaction between PS and PL through BRAM. In PL, we can read and write BRAM by outputting clock, address, read and write control and other signals (for BRAM operation timing and IP settings, please refer to the PL side tutorial "Magic Weapon for Rapid Development - IP Core" chapter). In PS, since PS is not directly connected to BRAM, we need to use the AXI BRAM controller to complete the read and write operations on BRAM. The AXI BRAM controller is a soft core integrated in the Vivado design software and can be configured into AXI4-lite interface mode or AXI4 interface mode. The block diagram of AXI4-Lite interface mode is shown in Figure 15-1.

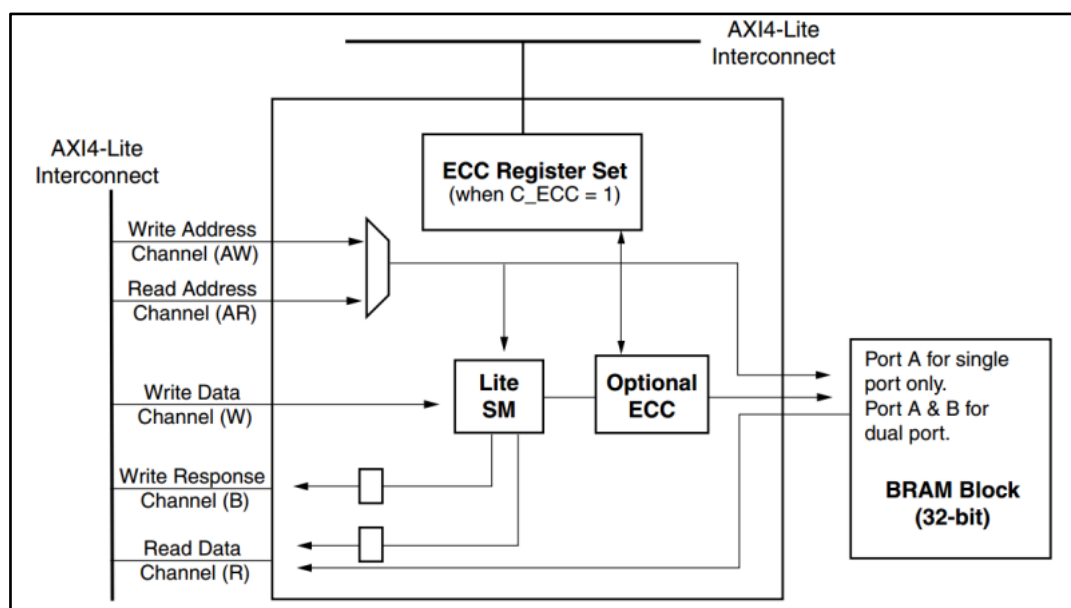


Figure 15-1 AXI-Lite interface block diagram

In AXI4 interface mode, the BRAM controller supports data widths of 32-bit, 64-bit, 128-bit, 512-bit and 1024-bit, while the AXI4-Lite interface is a lightweight interface and only supports 32-bit data width. As can be seen from the figure,

PS accesses BRAM through the AXI4-Lite interface, and when the ECC option is enabled, ECC allows the AXI master interface to detect and correct single-bit and double-bit errors in BRAM blocks. The AXI BRAM controller serves as the slave interface of the AXI bus and interconnects with the AXI master interface to perform read and write operations on BRAM. For different application scenarios, this IP core supports two modes: single transmission and burst transmission.

15.2.1 Experimental objectives

PS writes the data received by the serial port into BRAM, then reads the data from BRAM and prints it out through the serial port; at the same time, PL also reads the data from BRAM and uses ILA to observe whether the read data is consistent with the data printed by the serial port.

15.2.2 Hardware resources

This experiment used the internal resources of the ZYNQ chip.

15.2.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 15-2.

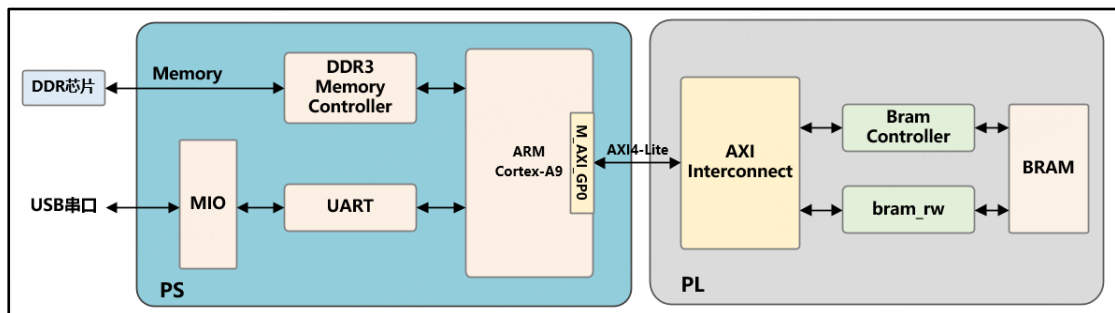


Figure 15-2 Overall block diagram

In the overall block diagram, M_AXI_GP0 on the PS side serves as the master port and is connected to the AXI BRAM controller IP core and PL read BRAM IP core (bram_rw) on the PL side through the AXI4 bus. Among them, AXI Interconnect IP (AXI Interconnect) is used to connect AXI memory-mapped (memory-mapped) master devices and slave devices. The AXI BRAM controller serves as the IP core for reading and writing BRAM on the PS side, while the PL reading BRAM IP core is our custom IP core, which is responsible for realizing the function of the PL side reading data from BRAM.

2. Hardware design

First create a Vivado project named "bram", then create a Block Design design (system.bd) and add the ZYNQ7 Processing System module. Then according to the overall system block diagram, add the AXI BRAM Controller IP core in Block Design.

Add AXI BRAM Controller

Right-click in an empty area of the Diagram window and select "Add IP." Search for "AXI BRAM" in the pop-up IP directory, and finally double-click "AXI BRAM Controller" in the search results to add it to the design.

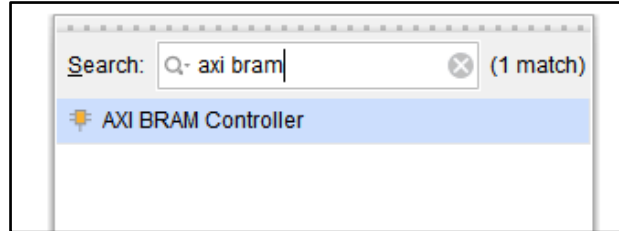


Figure 15-3 Adding AXI BRAM Controller

AXI BRAM Controller module is used to control BRAM on the PS side. Double-click to open the configuration and connect to the AXI bus. It can be used to read and write the BRAM module. The AXI Protocol is set to AXI4 and the data width is set to 32 bits. Since the AXI4 bus is byte addressing, when mapping to the BRAM address, it needs to be addressed in 4 bytes. Memory depth is not set here, it needs to be set in the Address Editor. The number of BRAM ports is set to 1, which is used to connect the PORTA of dual-port RAM. Disable ECC function.

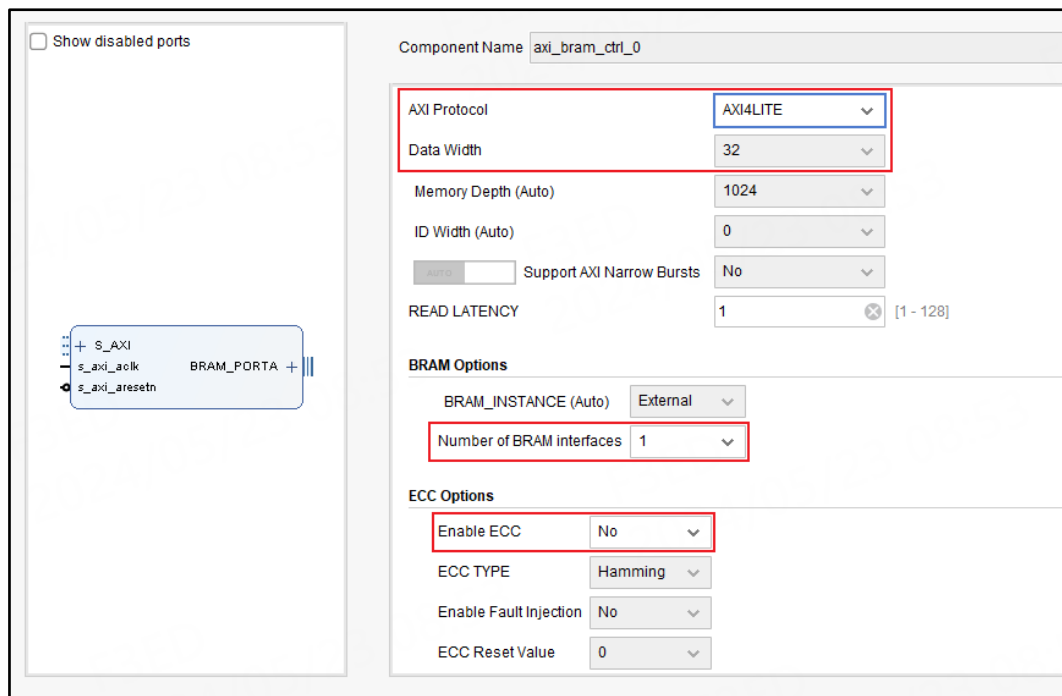


Figure 15-4 BRAM controller configuration interface

Table 3-1: BRAM Configuration for UltraScale, 7 Series, or Zynq-7000 Devices

Supported Memory Sizes / BRAM Memory Configuration	Number of BRAM Primitives (36k/each)	Size of BRAM_Addr (each port)	Typical BRAM_Addr Bit Usage with BRAM Configuration
32-bit Data BRAM Data Width			
4k / (1024 x 32)	1	10	BRAM_Addr [11:2]
8k / (2048 x 32)	2	11	BRAM_Addr [12:2]
16k / (4096 x 32)	4	12	BRAM_Addr [13:2]
32k / (8192 x 32)	8	13	BRAM_Addr [14:2]
64k / (16384 x 32)	16	14	BRAM_Addr [15:2]
128K / (32,768x32)	32	15	BRAM_Addr [16:2]

Figure 15-5 Mapping relationship between BRAM controller and BRAM.

It should be noted that Memory Depth cannot be set here. The memory depth of addressing B RAM is set in the Address Editor. Finally, click the "OK" button to complete the setting.

Add BRAM ip core

Follow the method of adding AXI BRAM Controller, this time search for "Block Memory", and then configure the ip core:

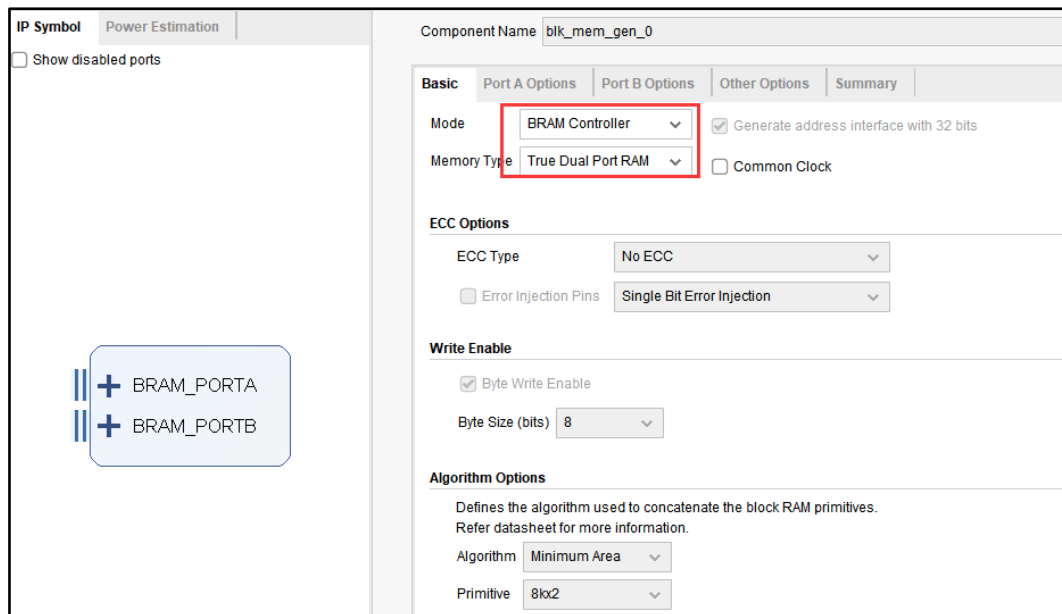


Figure 15-6 BRAM configuration interface (1)

The BRAM IP core supports two modes, one is the independent mode (Stand Alone), the data depth and width of the RAM can be freely configured; the other is the BRAM controller mode (B RAM Controller), the address and data are 32 bits by default. Since the BRAM controller IP core was added in this experiment, the BRAM mode selects the BRAM controller mode. Memory type selects dual-port RAM, one end is connected to the BRAM controller and the other end is connected to the PL RAM controller.

Next switch to the Other Options interface:

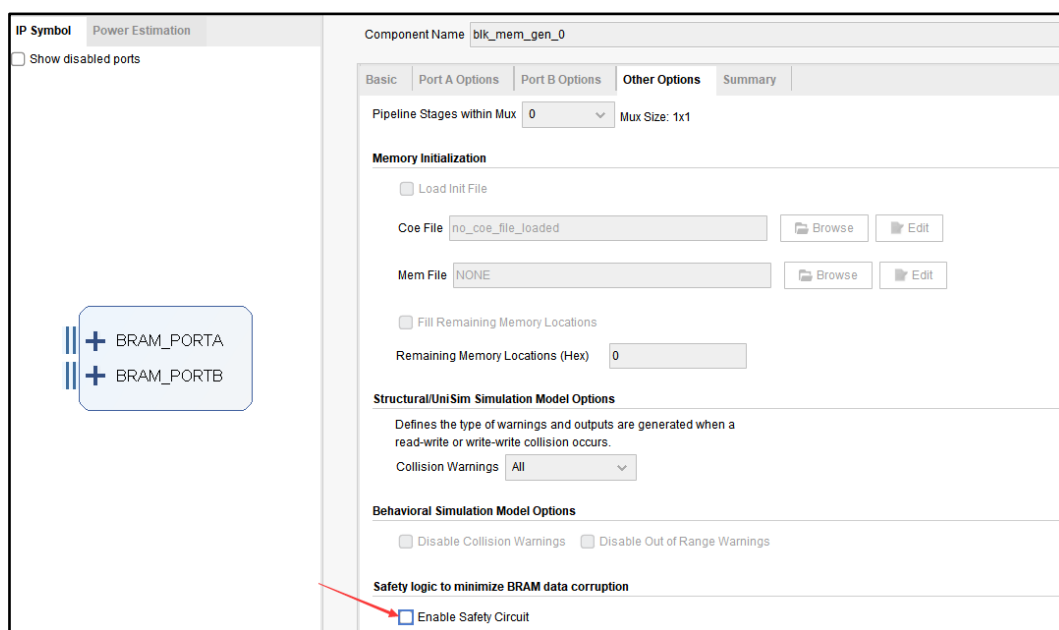


Figure 15-7 BRAM configuration interface (2)

A safety circuit is built into the BRAM IP core to reduce the probability of BRAM data errors. If this safety circuit is enabled, the BRAM ports will increment the `rsta_busy` and `rstb_busy` ports to indicate when the BRAM can be accessed. However, in this configuration we choose not to enable the safety circuit and finally click the "OK" button to complete the setup.

Customized PL BRAM read-write controller

Some settings of the custom IP core in this experiment are somewhat different from the "Custom IP core-breathing light experiment", so we will next demonstrate the creation of a new custom IP core based on this experimental project.

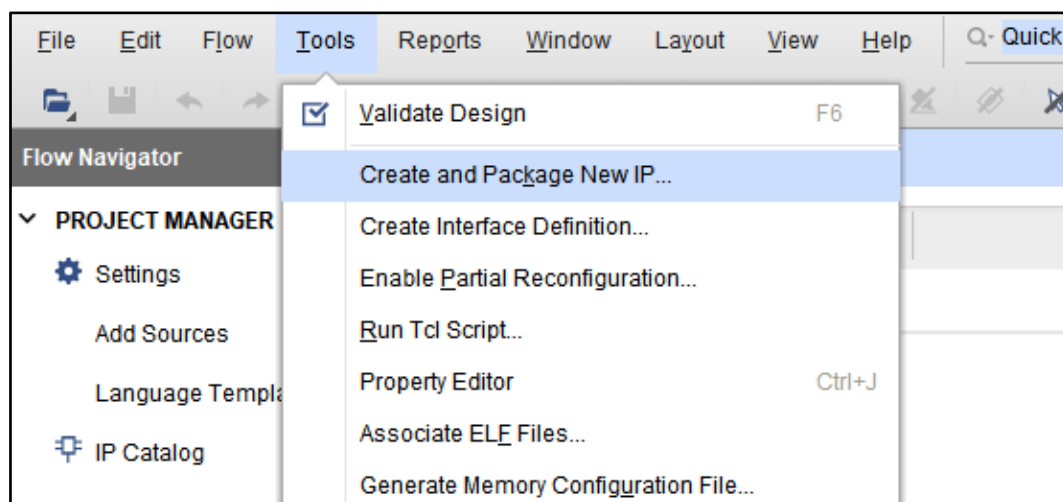


Figure 15-8 Create a new ip core

In the pop-up interface, click "Next", then select "Create a new AXI4 peripheral", then click "Next" to enter the following interface.

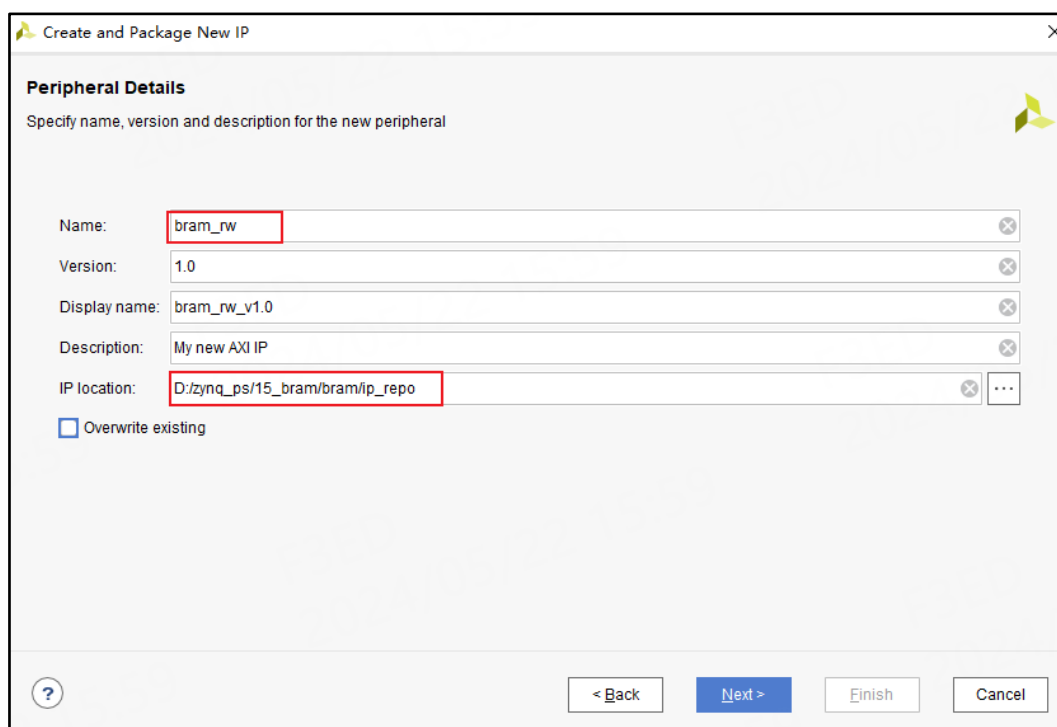


Figure 15-9 Custom IP name

Enter "bram_rw" in Name, IP location is the ip_repo folder under this project path, no other changes are needed, just "Next" until the creation of the custom ip core is completed. Then open the IP Catalog:

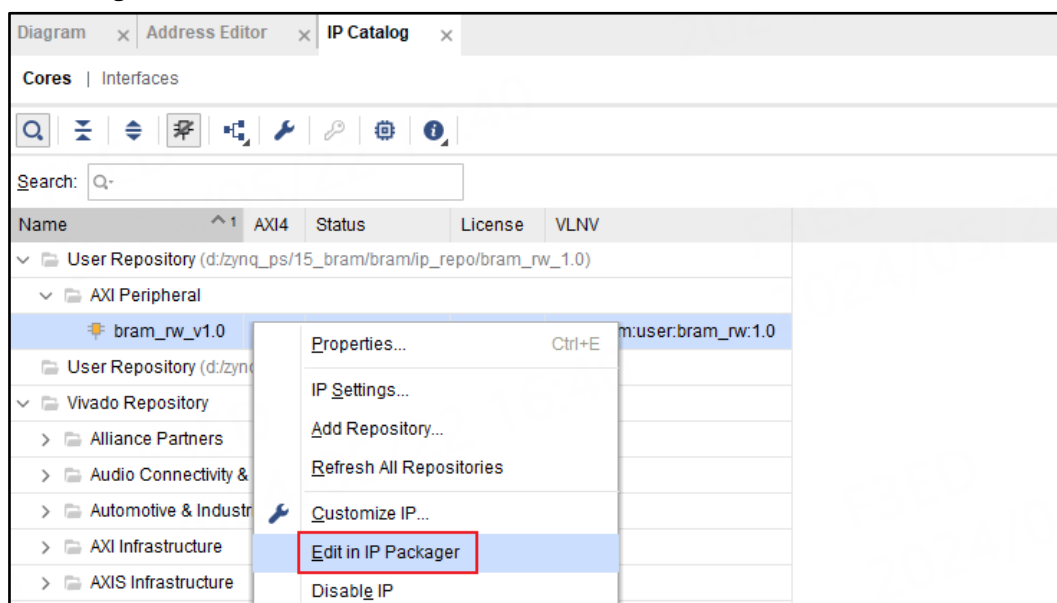


Figure 15-10 Edit customized ip core

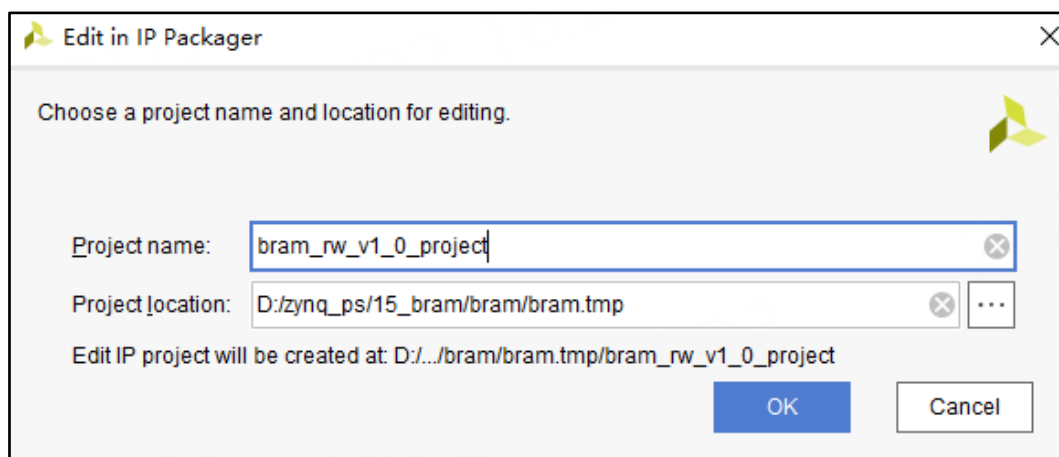


Figure 15-11 Click OK

You can see that a new project interface will pop up. Then open the bram_rw_v1_0.v file and add the following code in the middle line between Users to add ports here and User port ends. The se ports are used to connect the BRAM_PORTB of the BRAM port.

```
// Users to add ports here
output wire      ram_clk    , //RAM时钟
output wire      ram_rst    , //RAM复位信号,高电平有效
output wire      ram_en     , //RAM使能信号
output wire [31:0] ram_addr  , //RAM地址
output wire [3:0] ram_we     , //RAM读写控制信号
output wire [31:0] ram_wr_data, //RAM写数据

input wire [31:0] ram_rd_data, //RAM中读出的数据

// User ports ends
```

Figure 15-12 Adding a top-level port

Where the bram_rw_v1_0_S00_AXI module is instantiated, add the following code:

```
// Instantiation of Axi Bus Interface S00_AXI
bram_rw_v1_0_S00_AXI # (
    .C_S_AXI_DATA_WIDTH(C_S00_AXI_DATA_WIDTH),
    .C_S_AXI_ADDR_WIDTH(C_S00_AXI_ADDR_WIDTH)
) bram_rw_v1_0_S00_AXI_inst (
    .ram_clk      (ram_clk      ),
    .ram_rst      (ram_rst      ),
    .ram_rd_data  (ram_rd_data),
    .ram_en       (ram_en       ),
    .ram_addr     (ram_addr     ),
    .ram_we       (ram_we       ),
    .ram_wr_data  (ram_wr_data),

    .S_AXI_ACLK(s00_axi_aclk),
```

Figure 15-13 New signal in instantiated file

Open bram_rw_v1_0_S00_AXI.v, and also add the following code in the middle line between n Users to add ports here and User port ends:

```
// Users to add ports here
output wire      ram_clk      , //RAM时钟
output wire      ram_rst      , //RAM复位信号,高电平有效
output wire      ram_en       , //RAM使能信号
output wire [31:0] ram_addr    , //RAM地址
output wire [3:0] ram_we       , //RAM读写控制信号
output wire [31:0] ram_wr_data, //RAM写数据

input wire [31:0] ram_rd_data, //RAM中读出的数据
// User ports ends
```

Figure 15-14 Inserting signal

In the middle line between Add user logic here and User logic ends at the end of the program, add the following code:

```
// Add user logic here
bram_rw bram_wr_inst(
.clk      (S_AXI_ACLK),
.rst_n    (S_AXI_ARESETN),
.start_rd (slv_reg0[0]),
.start_addr (slv_reg1),
.rd_len   (slv_reg2),
//RAM端口
.ram_clk   (ram_clk      ),
.ram_rd_data (ram_rd_data),
.ram_en     (ram_en       ),
.ram_addr   (ram_addr     ),
.ram_we     (ram_we       ),
.ram_wr_data (ram_wr_data),
.ram_rst    (ram_rst      )
);
// User logic ends
```

Figure 15-15 Instantiating submodules

This code instantiates the bram_rw module, where the start_rd signal is the start signal to start reading BRAM, start_addr is to set the starting address of reading BRAM, rd_len is to set the number of BRAMs to be read, and is connected to the data corresponding to register address 0, address 1 and address 2 of the AXI4-Lite bus respectively.

The next step is to create a new module bram_rw under the project, the path is ../bram/ip_repo/bram_rw_1.0/hdl and write our code for reading BRAM:

Code Listing 15-1 Read BRAM module reference code (bram_rw.v)

```
01 module bram_rw(
02     input          clk          , //时钟信号
03     input          rst_n        , //复位信号
04     input          start_rd     , //读开始信号
05     input [31:0]    start_addr   , //读开始地址
06     input [31:0]    rd_len      , //读数据的长度
07     //RAM 端口
08     output         ram_clk      , //RAM 时钟
09     input [31:0]    ram_rd_data, //RAM 中读出的数据
10     output reg     ram_en       , //RAM 使能信号
11     output reg [31:0] ram_addr   , //RAM 地址
```

```
12 output reg [3:0] ram_we, //RAM read and write control signal 13 output reg [31:0] ram_wr_data, //RAM write data
14 output ram_rst //RAM reset signal, high level active 15); 16 17 //reg define 18 reg [1:0] flow_cnt; 19 reg start_rd_d0;
20 reg start_rd_d1; 21 22 //wire define 23 wire pos_start_rd; 24 25 //*****
***** 26 /** main code 27 //*****
***** 28 29 assign ram_rst = 1'b0; 30 assign ram_clk = clk ; 31 assign pos_start_rd = ~start_rd_d1 & start_rd_d0;
32 33 //Delay two beats, take the rising edge of start_rd signal 34 always @(posedge clk or negedge rst_n) begin
35 if(!rst_n) begin 36 start_rd_d0 <= 1'b0; 37 start_rd_d1 <= 1'b0; 38 end 39 else begin 40 start_rd_d0 <= start_rd;
41 start_rd_d1 <= start_rd_d0; 42 end 43 end 44 45 //Read data from RAM according to the read start signal 46 always @(posedge clk or negedge rst_n) begin
47 if(!rst_n) begin 48 flow_cnt <= 2'd0; 49 ram_en <= 1'b0; 50 ram_addr <= 32'd0; 51 ram_we <= 4'd0; 52 end 53 else begin 54 case(flow_cnt) 55 2'd0 : begin 56 if(pos_start_rd) begin 57 ram_en <= 1'b1; 58 ram_addr <= start_addr; 59 flow_cnt <= flow_cnt + 2'd1; 60 end 61 end 62 2'd1 : begin 63 if(ram_addr - start_addr == rd_len - 4) begin //The data is read 64 ram_en <= 1'b0; 65 flow_cnt <= flow_cnt + 2'd1; 66 end 67 else 68 ram_addr <= ram_addr + 32'd4; //Address accumulation 4 69 end 70 2'd2 : begin 71 ram_addr <= 32'd0; 72 flow_cnt <= 2'd0; 73 end 74 endcase 75 end 76 end
```

This completes the instantiation of the modules contained in the custom IP core. You can see that it is the same as the submodules instantiated under the top-level module before.

Next, double-click component.xml under the IP-XACT interface to enter the Packaging Steps interface. Click the "FileGroups" column, and then click "Merge changes from File Groups Wizard" on the interface; click the "Customization Parameters" column, and then click "Merge changes from Customization Parameters Wizard" on the interface. These operations are similar to the previous custom IP. The focus is on the "Ports and Interfaces" column. You can see the ports of the top-level module of the IP core. In order to facilitate the connection of the BRAM ports of the customized IP core in the Diagram interface, we need to define the BRAM-related ports as bus interfaces.

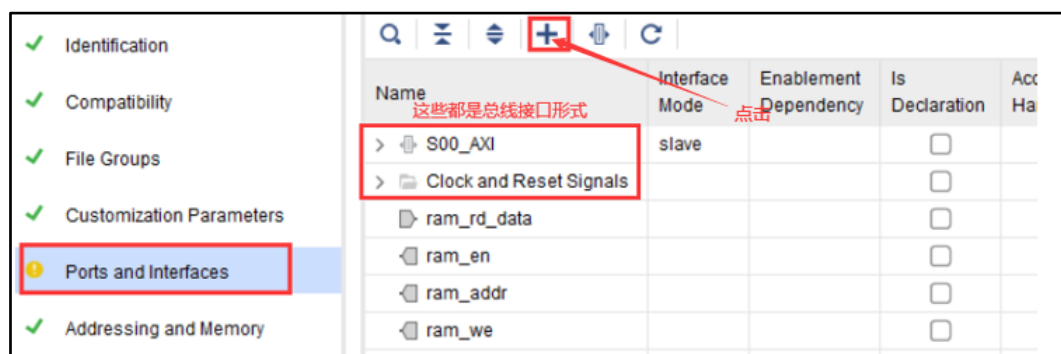


Figure 15-16 Adding bus interface

In the pop-up interface, first enter "bram_port" in Name, and then click the "..." icon on the right.

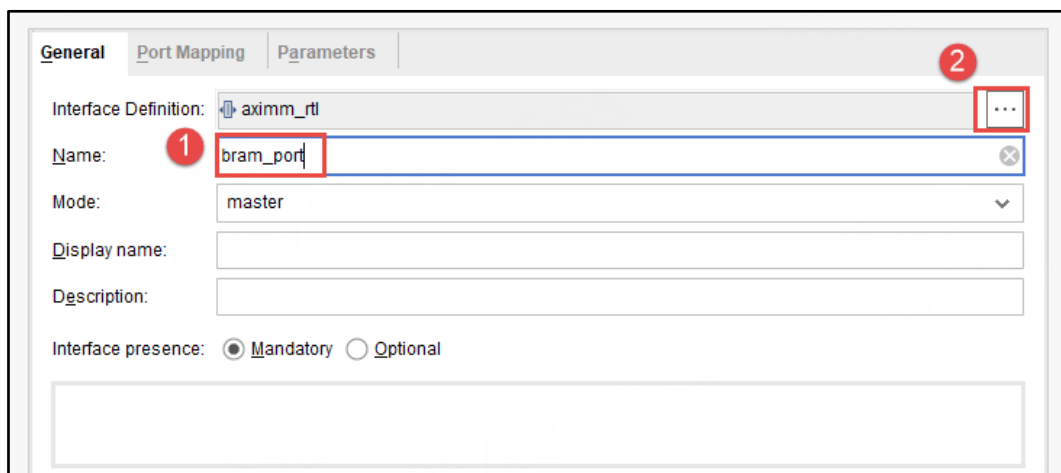


Figure 15-17 Modify name

After completing the above operation, the following interface will pop up.

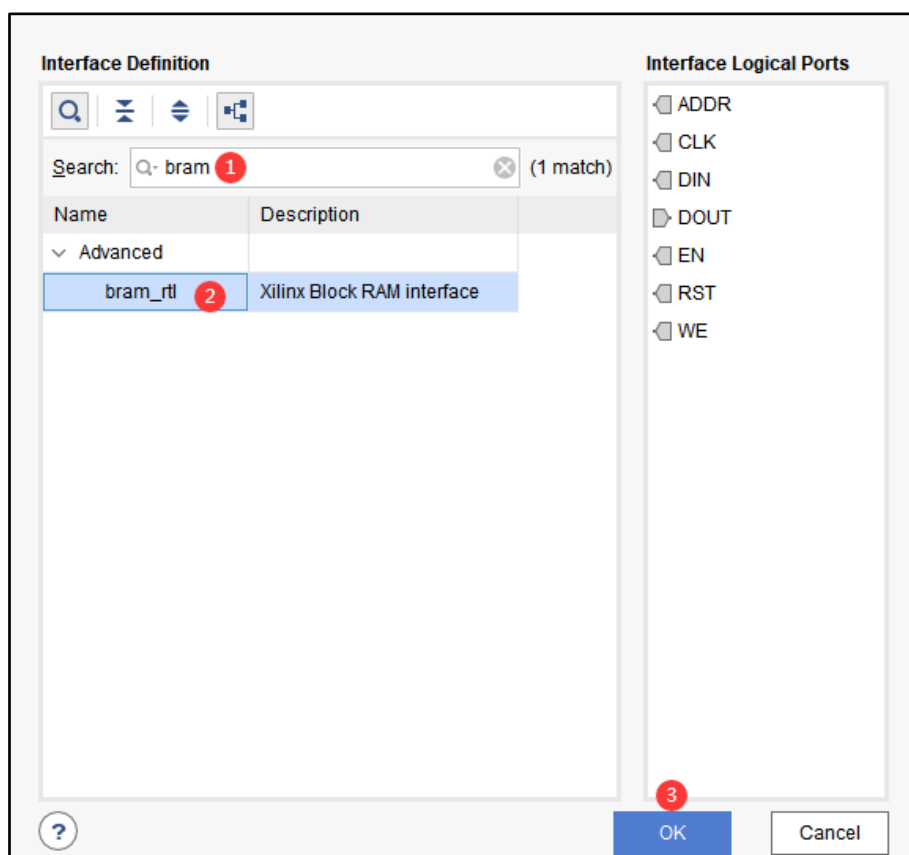


Figure 15-18 Introduction of Block RAM interface

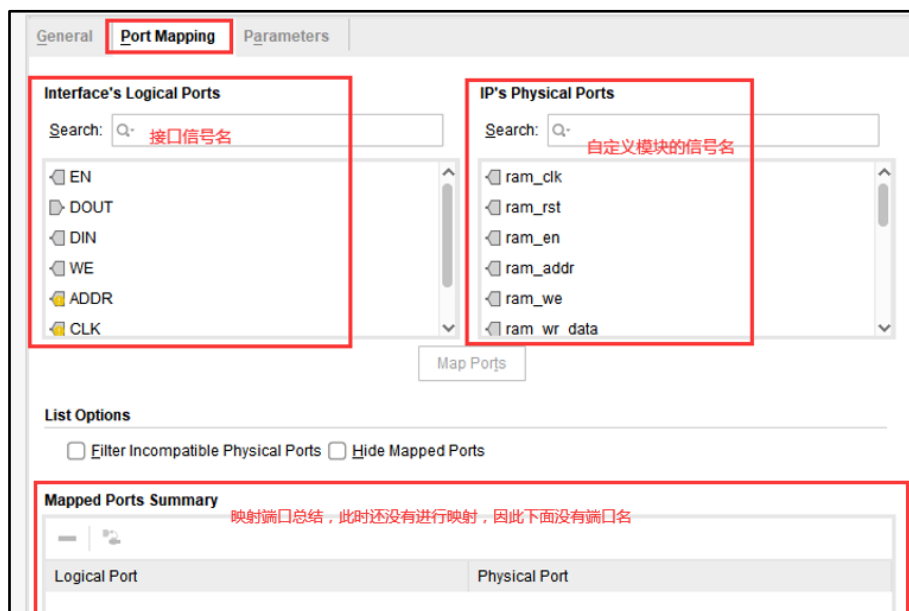


Figure 15-19 Port mapping interface

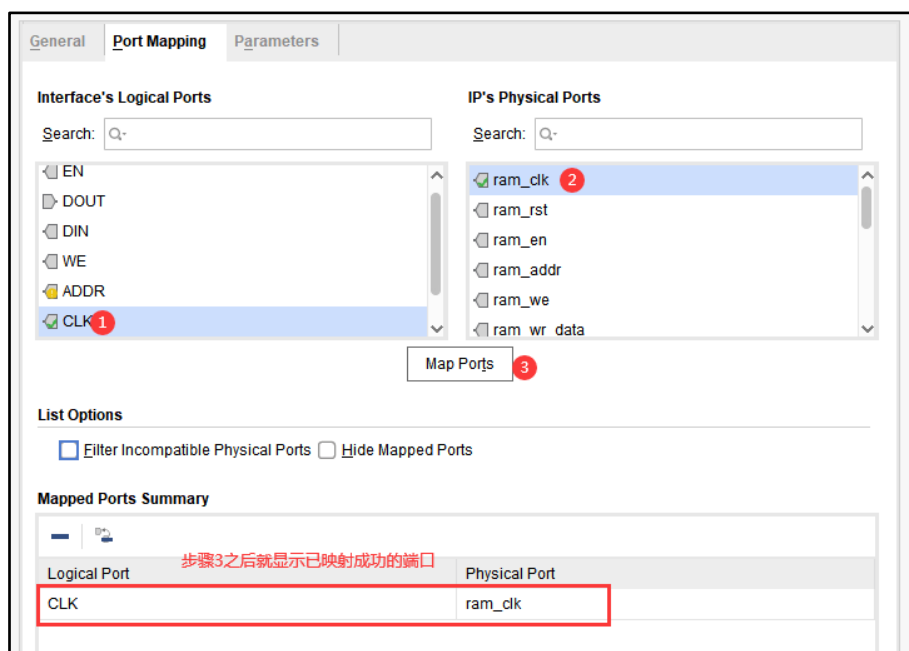


Figure 15-20 Port mapping

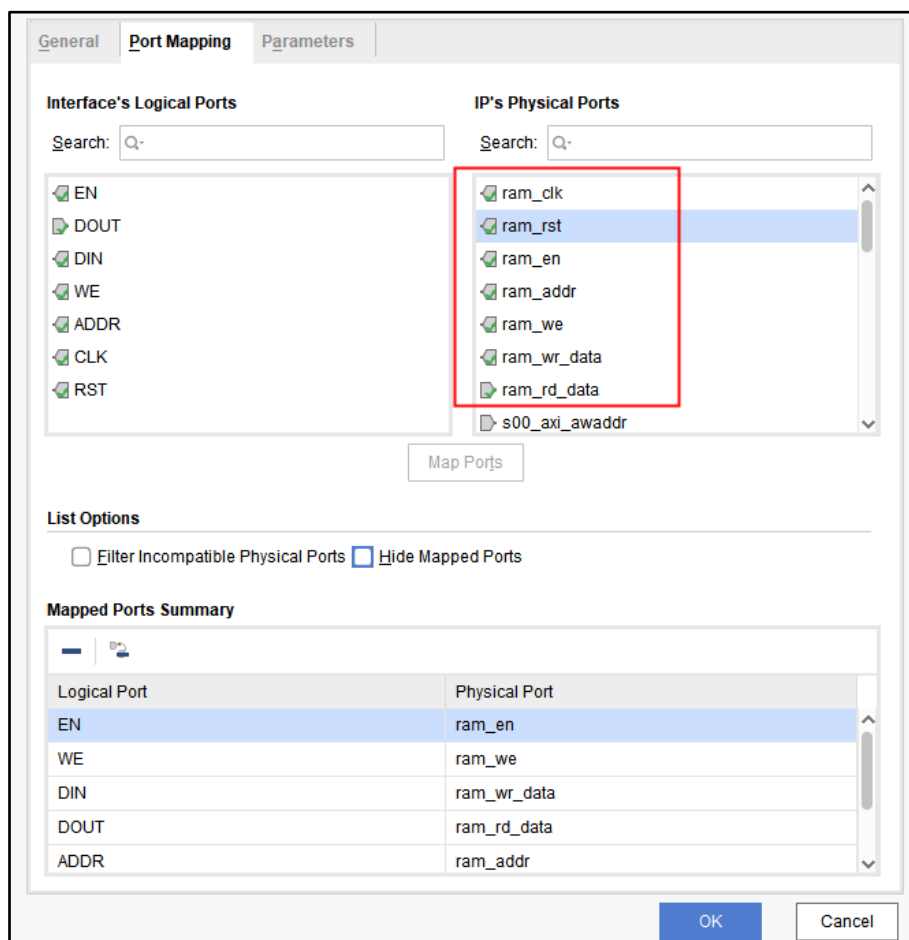


Figure 15-21 Port name to be mapped

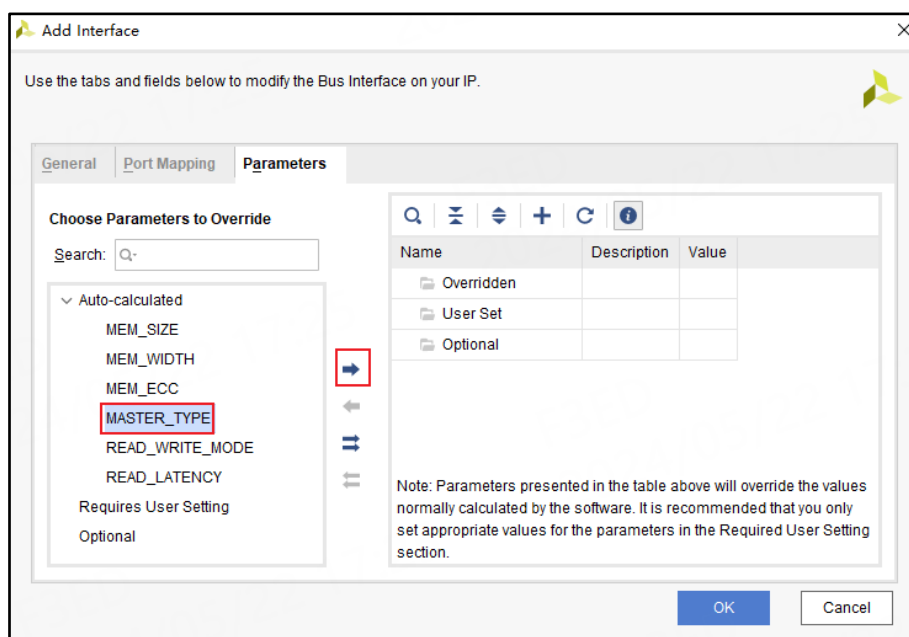


Figure 15-22 Add MASTER_TYPE

After that, "MASTER_TYPE" will appear in the Overridden directory on the right. Finally, click OK. You can see the newly created bus interface in the "Ports and Interfaces" column:

▼	bram_port	master		☐	
	ram_rst			☐	
	ram_clk			☐	
	ram_wr_data			☐	
	ram_en			☐	
	ram_rd_data			☐	
	ram_we			☐	
	ram_addr			☐	

Figure 15-23 bram_port bus interface

Next, switch to the "Review and Package" page, click "IP has been modified" on the upper side to update the IP, and finally click "Re-Package IP" to complete the packaging of the IP core. At this time, the packaging interface of the IP core will automatically close. The custom IP core is completed.

Return to the project interface of this experiment, and add the IP core just encapsulated in the Diagram interface just like adding other IP cores.

After the addition is completed, click "Run Connection Automation" at the top of the interface and check "All Automation" on the left side of the pop-up page. The automatically connected modules and their interfaces are listed below. Click "OK", and the tool will automatically connect the bram_port interface of the PL reading BRAM IP core and the BRAM_PORTB interface of the BRAM IP core.

Next, switch to the "Address Editor" page, expand "Data" under "processing_system7_0", and set the range to "4K", as shown in Figure 15-24. Since the data bit width of BRAM is 32 bits, the storage depth of BRAM is 1K.

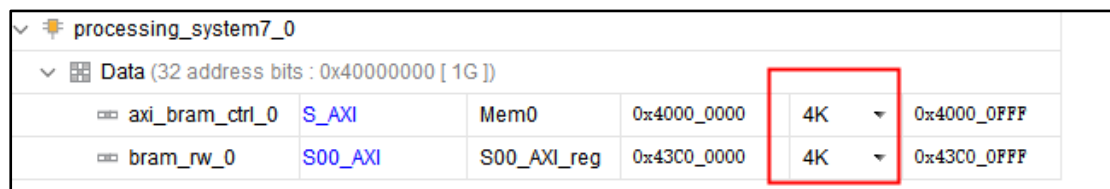


Figure 15-24 Allocating address space size

At this point our Block Design is completed. Right-click in the blank area of the Diagram window and select "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally "Ctrl + S" saves the design.

Next, right-click the Block Design design file "zynq7.bd" in the Source window, and then execute "Generate OutputProducts" and "Create HDL Wrapper" in sequence.

Then "Run Synthesis" analyzes and synthesizes the synthesis file, and then "Set Up Debug" to add signals to the ILA interface:

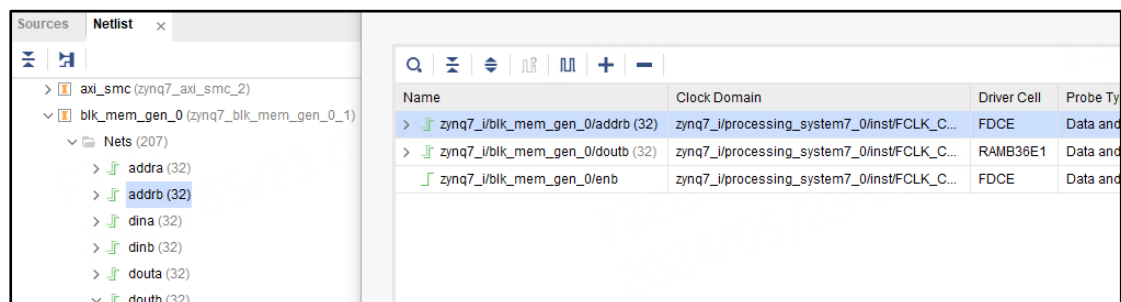


Figure 15-25 Add signal to be observed

Click "Next" and set the sampling depth to 1024.

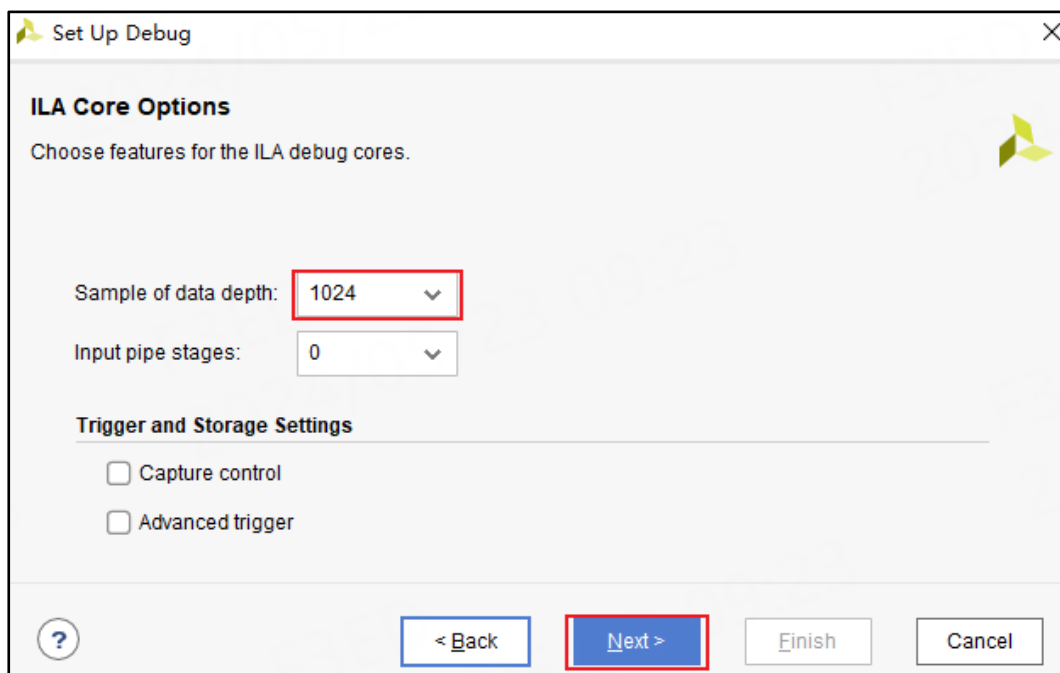


Figure 15-26 Set sampling depth

Next, click "Next" and "Finish" in order to complete the addition of signals, and press "Ctrl + S" to save the Synthesized Design.

Finally, the bit stream file is generated, the hardware is exported, and the SDK is started.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and

Code Listing 15-2 Bram cross-reference code (main.c)

```
1 #include "xil_printf.h" 2 #include "stdio.h" 3 #include "bram_rw.h" 4 #include "xbram.h" 5 6 //Base address 7 #define BRAM_BASE XPAR_BRAM_RW_0_S00_AXI_BASEADDR 8 //ram read address 9 #define BRAM_START BRAM_RW_S00_AXI_SLV_REG0_OFFSET 10 //Register base address 11 #define BRAM_REG_ADDR BRAM_RW_S00_AXI_SLV_REG1_OFFSET 12 //Read length 13 #define BRAM_DEPTH BRAM_RW_S00_AXI_SLV_REG2_OFFSET 14 15 #define START_ADDR 0 //ram starting address 16 #define BRAM_DATA_BYTE 4 //number of bytes occupied 17 18 char buffer[256]; 19 int LENGTH; 20 21 /*write function*/ 22 void BRAM_Write( 23 {
```

```
24 int i = 0, count = 0; 25 26 /*Write characters in a loop*/ 27 for (i = BRAM_DATA_BYTE * START_ADDR; 28 i < BRAM_DATA_BYTE * (START_ADDR + LENGTH); 29 i += BRAM_DATA_BYTE) { 30 XBram_WriteReg(XPAR_BRAM_0_BASEADDR, i, buffer[count]);
```

```
31 count++; 32 } 33 //Set the write length 34 BRAM_RW_mWriteReg(BRAM_BASE, BRAM_DEPTH, 35 BRAM_DATA_BYTE*LENGTH); 36 //Set the ram starting address 37 BRAM_RW_mWriteReg(BRAM_BASE, BRAM_REG_ADDR, 38 BRAM_DATA_BYTE*START_ADDR); 39 40 /*Operation signal line writing*/ 41 BRAM_RW_mWriteReg(BRAM_BASE, BRAM_START, 1); 42 BRAM_RW_mWriteReg(BRAM_BASE, BRAM_START, 0); 43 } 44 45 /*Read function*/ 46 void BRAM_Read() 47 { 48 int READ_Buffer = 0, i = 0; 49 /*Loop to read characters*/ 50 for (i = BRAM_DATA_BYTE * START_ADDR; 51 i < BRAM_DATA_BYTE * (START_ADDR + LENGTH); 52 i += BRAM_DATA_BYTE) { 53 READ_Buffer = 61 while (1) { 62 printf("Please write some string!\n"); 63 scanf("%256s", buffer); //Read the input character 64 LENGTH = strlen(buffer); //Calculate the length 65 if (LENGTH > 256) { 66 printf("The input character is too long !\n"); 67 return 0; 68 } 69 BRAM_Write(); 70 BRAM_Read(); 71 } 72 return 0; 73 }
```

In lines 7-13 of the code, we define the base address of the BRAM and the addresses of the three registers of the IP we designed ourselves. These addresses can be found in xparameters.h. Since the data we write at one time is 32bit, which is 4 bytes, we define BRAM_DATA_BYTE as 4 bytes. Define the starting address of ram as 0.

In the BRAM_Write() function, the data is written into bram through X Bram_WriteReg(), the address and length are written to different registers, and a pulse signal is given to start writing. In the BRAM_Read() function, the data is also read out through X Bram_ReadReg() and then printed out through the serial port.

In the while loop of the main function, the program receives characters input by the user through the serial port. The 256 here means that a maximum of 256 characters can be received. Once 256 is exceeded, the user will be prompted to enter too many characters and re-enter. Then call the written BRAM_Write() and BRAM_Read() to read and write bram.

After the writing is completed, compile the project and prepare for board verification.

15.2.4 Board verification

As shown in Figure 15-27, the development board is connected. After the lines are connected correctly, turn on the

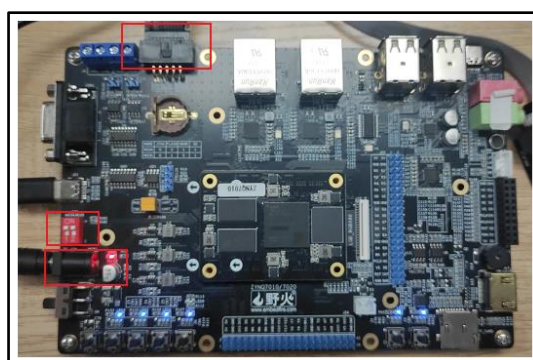


Figure 15-27 Program download connection diagram

As shown in Figure 15-28, download the program for the development board.

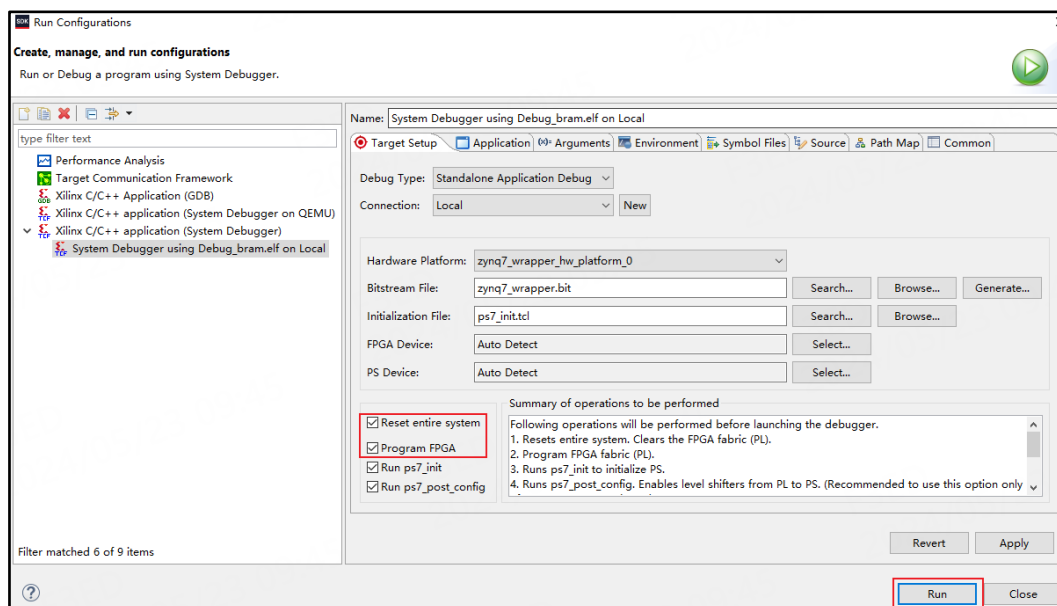


Figure 15-28 Program download diagram

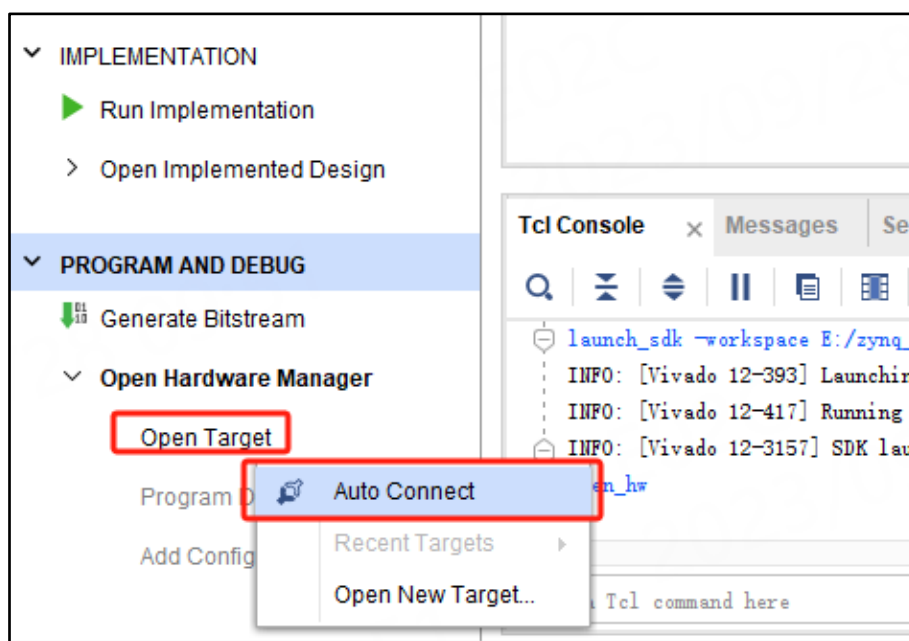


Figure 15-29 Auto Connect

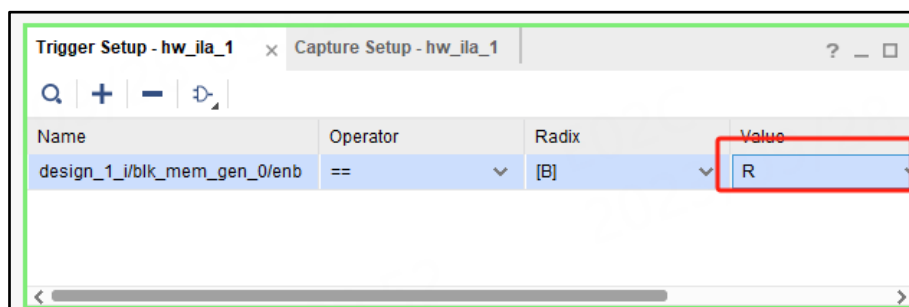


Figure 15-30 Check rising edge trigger

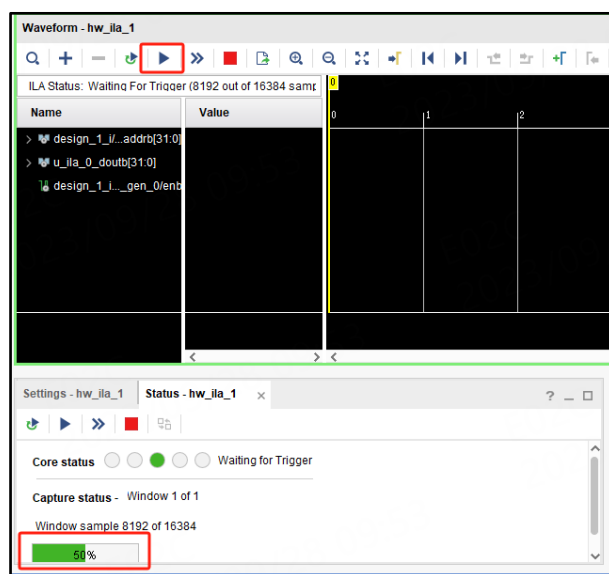


Figure 15-31 Then wait for the trigger

Open the serial port assistant and send a string with a length less than 256 characters. (note sending new line)

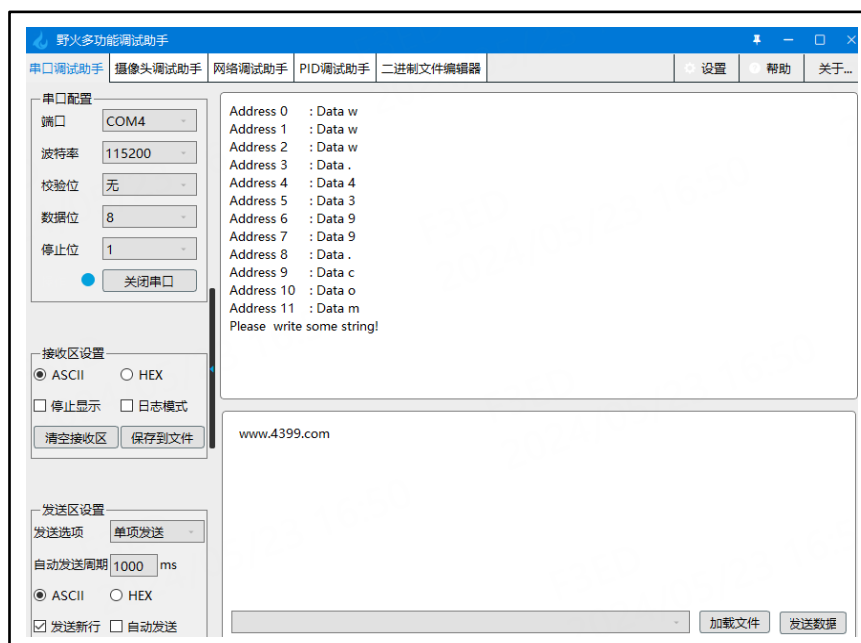


Figure 15-32 Data printed by the PS serial port

At the same time, in the Vivado interface and ILA debugging interface, change the display of doutb to ASCII display.

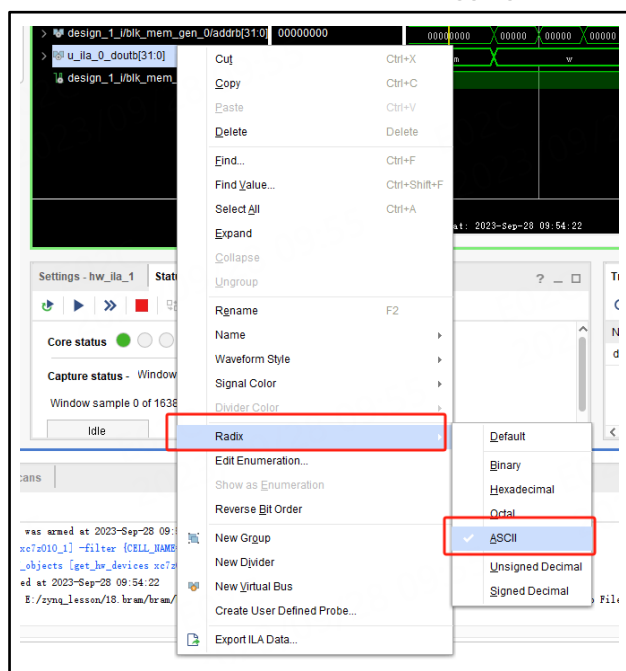


Figure 15-33 Change Radix to ASCII

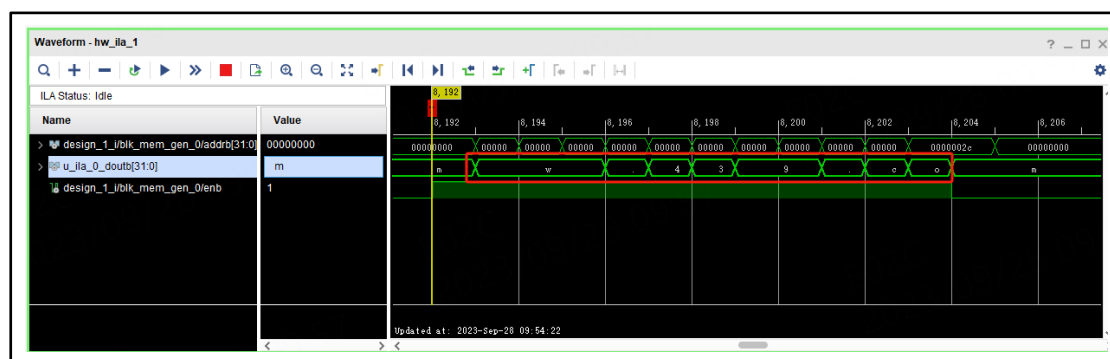


Figure 15-34 Data received by PL end

After comparison, we can know that the data printed by the PL and PS serial ports are consistent, so our bram data interaction experiment was successful.

15.3 End of chapter summary

Both BRAM and AXI can be used for data interaction between PS and PL, but BRAM is often used in situations where the amount of data is small, the addresses are discontinuous, and the length is irregular. In the hardware design, we also customized the IP core for reading BRAM. In the experiment, ILA was used to observe whether the read data was consistent with the data printed by the serial port.

15.4 Development training

It can be changed to PL to output the received data +1.

Chapter 16 AXI Interface Reading and Writing DDR

16.1 Chapter Introduction

During the development process of ZYNQ SOC, data exchange between PL and PS is often required. As part of the smallest system of ZYNQ SOC, DDR plays an extremely important role in caching data. DDR has a larger capacity than BRAM. Compared with using DMA, you don't need to understand various protocols in detail. In this chapter, we introduce how to realize the interaction of PS and PL data through the AXI bus.

16.2 Theoretical study

ZYNQ tightly integrates high-performance ARM Cortex-A series processors and high-performance FPGAs into a single chip, which brings many advantages to the design, such as reducing size and power consumption, reducing design risks, and increasing design flexibility. After integrating processors and FPGAs with different process characteristics on one chip, the interconnection path between the on-chip processor and FPGA has become a core element of ZYNQ chip design. If the data interaction between Cortex-A9 and FPGA becomes a bottleneck, the performance advantages of the combination of processor and FPGA will not be fully utilized. In order to solve this problem, the ZYNQ chip uses the AXI bus to connect the processing system (PS) and programmable logic (PL) to achieve high-speed data transmission between the ARM core and FPGA.

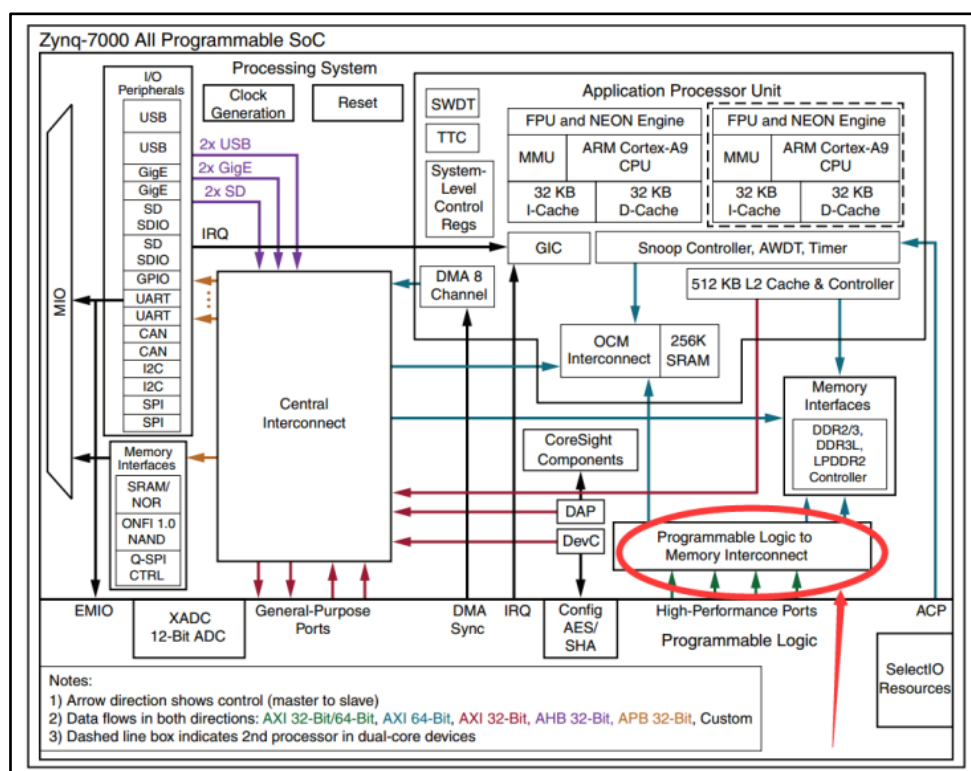


Figure 16-1 AXI interface block diagram

16.2.1 Introduction to AXI protocol

AXI (Advanced eXtensible Interface), or Advanced Extensible Interface, is a bus protocol that is an important part of the AMBA (Advanced Microcontroller Bus Architecture) protocol proposed by ARM. The reason why the AXI4 interface is used to read and write ddr is because Xilinx's migen ddr IP core, whether it is the 6 series, 7 series or the latest FPGA, integrates the AXI4 interface and uses the AXI4 interface for reading and writing. It can be compatible with other xilinx FPGAs in the future and has greater reusability.

AXI is a An on-chip bus for high performance, high bandwidth, and low latency, with the following features

- (1) The address/control and data channels of the bus are separated.
- (2) Support unaligned data transmission.
- (3) Supports burst transmission and only needs to provide the first address.
- (4) Independent read and write channels can provide low resource consumption DMA.
- (5) Supports sending and receiving data packets.
- (6) Easily added to the pipeline stages for timing closure.

In digital circuits, the transmission of information relies on the transmission of binary numbers 0 and 1, so a set of signals is required to form an interface. The AXI4 protocol supports the following three types of interfaces:

- (1) AXI4 : Provides a high-performance storage mapping interface, suitable for the transmission of large amounts of data.
- (2) AXI4-Lite: A simplified version of the AXI4 interface, suitable for storage mapping communication with a small amount of data.
- (3) AXI4-Stream: used for high-speed data stream transmission, non-storage mapping interface.

Here we explain the concept of memory map. If a protocol is memory mapped, then sessions issued by the host (either read or write operations) will be tagged with an address. This address corresponds to an address in the system storage space, indicating that the session is a read and write operation for this storage space.

The AXI4 protocol supports burst transmission and is mainly used in high-speed data transmission scenarios that require specified addresses such as processor access to memory. AXI4-Lite provides a single data transfer for peripherals, mainly used to access registers in some low-speed peripherals. The AXI4-Stream interface is similar to FIFO. No address is required for data transmission. Data is read and written continuously between master and slave devices. It is mainly used in scenarios that require high-speed data transmission, such as video processing, high-speed AD, PCIe and DMA interfaces.

16.2.2 AXI interface composition

AXI4 interface, which consists of five independent channels:

- (1) Read address
- (2) Read data
- (3) Write address
- (4) Write data
- (5) Write response

The following is a schematic diagram of the read transfer process using read addresses and read data channels:

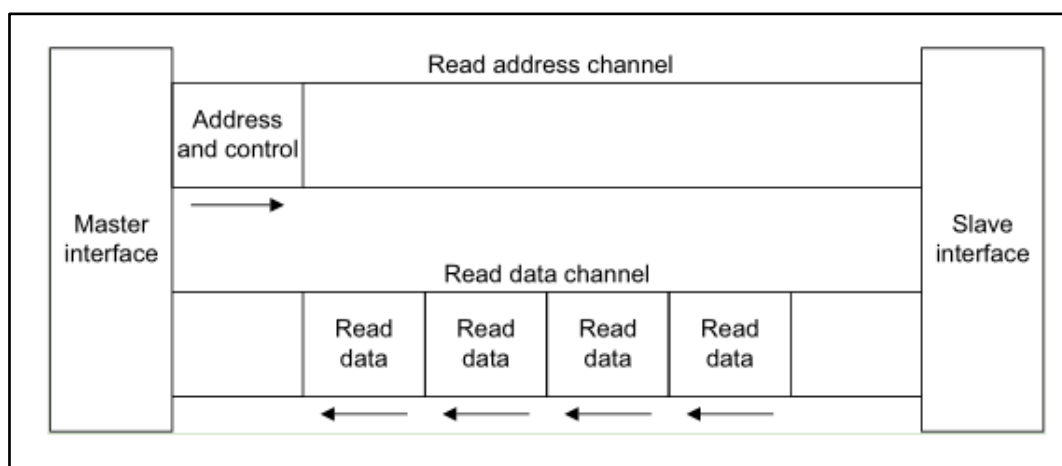


Figure 16-2 Schematic diagram of AXI4 reading data

As shown in Figure 16-2, during the read transmission process, the host first sends the read address and control signal through the read address channel, and then the slave returns the read data through the read data channel. It should be noted that this is a burst read operation. The host only provides a starting address, and four data are read continuously from this address.

The write transfer process is shown in Figure 16-3, involving three channels: write address, write data and write response. The host first sends the write address and control signals through the write address channel, and then writes four consecutive data bursts on the write data channel. Subsequently, after receiving the data, the slave returns a response signal through the write response channel.

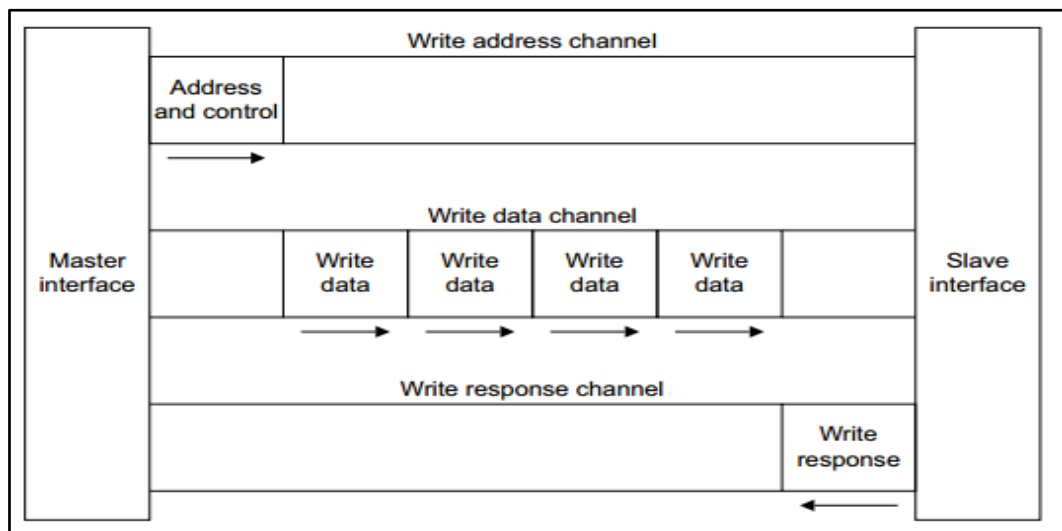


Figure 16-3 Schematic diagram of AXI4 writing data

Table 16-1 AXI4 interface description

通道	信号	源	信号描述
全局信号	Aclk	主机	全局时钟
	Aresetn	主机	全局复位，低有效
写通道地址与控制信号	M_AXI_WR_awid	主机	写地址 ID，用来标志一组写信号
	M_AXI_WR_awaddr	主机	写地址，给出一次写突发传输的写地址
	M_AXI_WR_awlen	主机	突发长度，给出突发传输的次数
	M_AXI_WR_awsz	主机	突发大小，给出每次突发传输的字节数
	M_AXI_WR_awburst	主机	突发类型

通道	M_AXI_WR_awlock	主机	总线锁信号，可提供操作的原子性
	M_AXI_WR_awcache	主机	内存类型，表明一次传输是怎样通过系统的
	M_AXI_WR_awprot	主机	保护类型，表明一次传输的特权级及安全等级
	M_AXI_WR_awqos	主机	质量服务 QoS
	M_AXI_WR_awvalid	主机	有效信号，表明此通道的地址控制信号有效
	M_AXI_WR_awready	从机	表明“从”可以接收地址和对应的控制信号
写通道数据通道	M_AXI_WR_wdata	主机	写数据
	M_AXI_WR_wstrb	主机	写数据有效的字节线，用来表明哪 8bits 数据是有效的
	M_AXI_WR_wlast	主机	表明此次传输是最后一个突发传输
	M_AXI_WR_wvalid	主机	写有效，表明此次写有效
	M_AXI_WR_wready	从机	表明从机可以接收写数据
写通道响应通道	M_AXI_WR_bid;	从机	写响应 IDTAG
	M_AXI_WR_bresp	从机	写响应，表明写传输的状态
	M_AXI_WR_bvalid	从机	写响应有效
	M_AXI_WR_bready	主机	表明主机能够接收写响应
读通道地址与控制信号通道	M_AXI_RD_arid	主机	读地址 ID，用来标志一组写信号
	M_AXI_RD_araddr	主机	读地址，给出一次写突发传输的读地址
	M_AXI_RD_arlen	主机	突发长度，给出突发传输的次数
	M_AXI_RD_arsize	主机	突发大小，给出每次突发传输的字节数
	M_AXI_RD_arburst	主机	突发类型
	M_AXI_RD_arlock	主机	总线锁信号，可提供操作的原子性
	M_AXI_RD_arcache	主机	内存类型，表明一次传输是怎样通过系统的
	M_AXI_RD_arprot	主机	保护类型，表明一次传输的特权级及安全等级
	M_AXI_RD_arqos	主机	质量服务 QOS
	M_AXI_RD_arvalid	主机	有效信号，表明此通道的地址控制信号有效
	M_AXI_RD_arready	从机	表明“从”可以接收地址和对应的控制信号
读通道数据通道	M_AXI_RD_rid	从机	读 IDtag
	M_AXI_RD_rdata	从机	读数据
	M_AXI_RD_rresp	从机	读响应，表明读传输的状态
	M_AXI_RD_rlast	从机	表明读突发的最后一次传输
	M_AXI_RD_rvalid	从机	表明此通道信号有效
	M_AXI_RD_rready	主机	表明主机能够接收读数据和响应信息

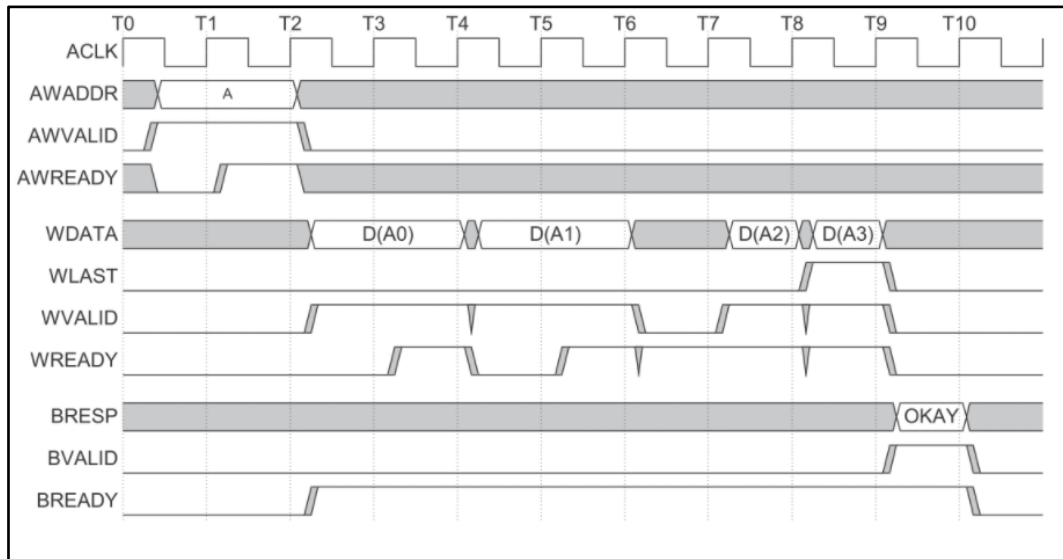


Figure 16-4 AXI4 write data timing diagram

Burst write timing:

AXI4 burst write can be divided into 7 states: write idle, write channel write address wait, write channel write address, write data wait, write data cycle, write response acceptance, write completion. The reason why it is divided into 7 states is to prepare the state machine for subsequent writing of programs.

Write idle: waiting for a burst signal to be triggered.

Write channel write address wait: prepare write address AWADDR, then pull AWVALID high.

Write channel write address: The slave receives AWVALID and issues AWREADY.

Waiting for writing data: Prepare data WDATA, pull WVALID high.

Write data loop: The slave accepts WVALID, confirms that the data WDATA is valid and accepted, sends WREADY, AXI is a burst transfer: loop this operation until the last data flag bit of WLAST is received.

Accept write response: BVALID sent by the slave is received, and the master sends BREADY.

Write end: pull down the signal that is not pulled down to enter write idle.

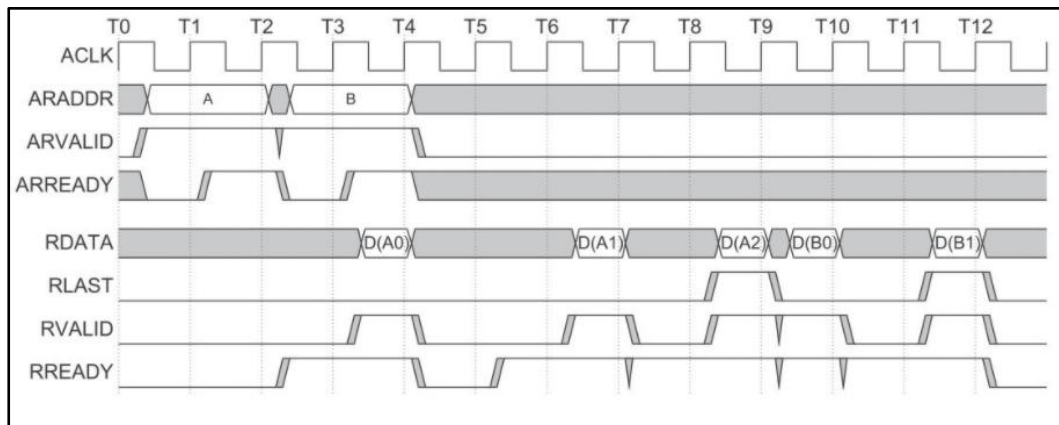


Figure 16-5 AXI4 read data timing diagram

Burst read timing:

AXI4 burst read can be divided into 6 states: read idle, read channel write address waiting, read channel write address, read data wait, read data loop, read end. The reason why it is divided into 6 states is to prepare the state machine for subsequent writing of programs.

Read idle: waiting for a burst signal to be triggered.

Read channel write address wait: prepare write address ARADDR, then pull ARVALID high.

Read channel write address: The slave receives ARVALID and issues ARREADY.

Read data waiting: the slave machine prepares data WDATA, and the slave machine pulls RVALID high.

Read data loop: The host accepts RVALID, confirms that the data RDATA is valid and accepted, sends RREADY to the slave, AXI is a burst transfer: loop this operation until the last data flag bit of RLAST is received.

End of reading: Pull down the signal that is not pulled down to enter read idle state.

From the above timing diagram and timing analysis, we can see that two relationships must be maintained:

- (1) Read data must always follow the address associated with its data.
- (2) A write response must always occur last after the write transaction with which it is associated.

16.2.3 AXI4 handshake signal

Each channel in the AXI bus contains a set of information signals, as well as a VALID and a READY signal. From the above read and write timing diagram and timing analysis, it can be seen that the reading and writing of the AXI interface depends on the handshake signals VALID and READY.

The AXI bus is divided into five channels:

The read address channel includes ARVALID, ARADDR, and ARREADY signals; the write address channel includes AWVALID, AWADDR, and AWREADY signals; the read data channel includes RVALID, RDATA, RREADY, and RRESP signals; the write data channel includes WVALID, WDATA,WSTRB, and WREADY signals;

Write response channel, including BVALID, BRESP, and BREADY signals;

The VALID signal is generated by the source and indicates that the information on the current address or data line is valid; while the READY signal is generated by the destination and indicates that it is ready to receive address, data and control information. The VALID and READY signals provide three handshake mechanisms in the AXI bus, as shown in the following figure:

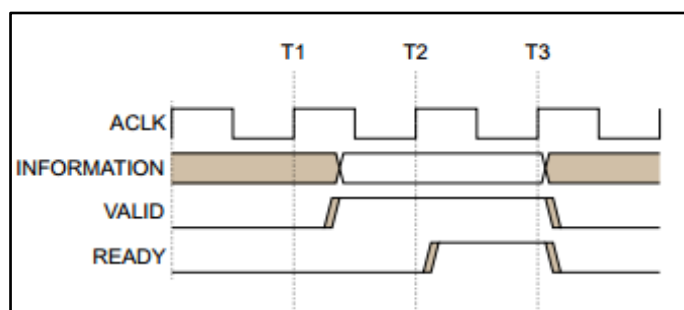


Figure 16-6 VALIDbeforeREADY handshake

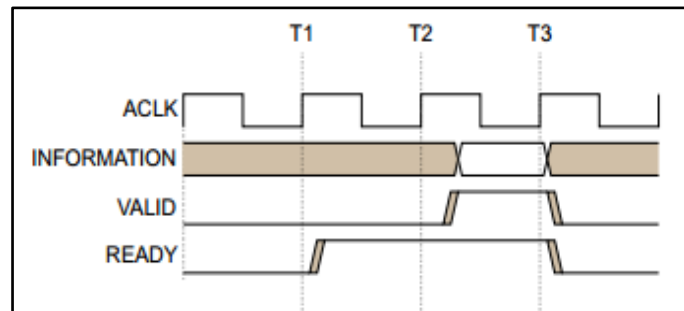


Figure 16-7 READY before VALID handshake

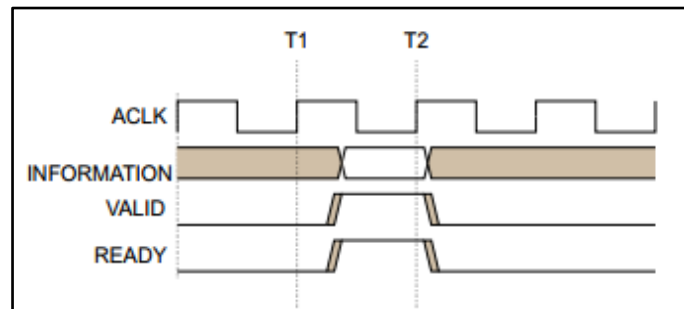


Figure 16-8 VALID with READY handshake

ACLK is a clock signal. In the AXI protocol, all input signals are sampled on the rising edge of ACLK, and all output signals must change after the rising edge of ACLK. After T1, the source pulls VALID high, indicating that valid address, data or control information is transmitted on the INFORMATION signal line. The destination pulls READY high after T2, indicating that it is ready to receive data. At this time, the source must keep the INFORMATION data stable until data transmission at T3. It should be noted that the source end is not allowed to wait for the READY signal of the destination end to be pulled high before setting the VALID signal to a valid state. Furthermore, once VALID is asserted high, the source must keep it asserted until a successful handshake occurs (detection of both VALID and READY being asserted on the rising edge of the clock). Generally speaking, VALID must continue to hold until the READY signal before it can be pulled low.

Write address channel: When the host drives valid address and control signals, the host can assert AWVALID. Once asserted, AWVALID needs to remain asserted until the slave's AWREADY is sampled on the rising edge of the clock. The default value of AWREADY can be high or low, but high is recommended (if low, a transfer requires at least two cycles, one to assert AWVALID and one to assert AWREADY); when AWREADY is high, the slave must be able to accept the valid address provided to it.

Write data channel: During the write burst transfer process, the host can only assert WVALID when it provides valid write data. Once asserted, it needs to remain asserted until the rising edge of the clock samples WREADY of the slave. The default value of WREADY can be high, which requires that the slave can always accept write data in a single cycle. The host needs to assert the WLAST signal when driving the last write burst transfer.

Write response channel: The slave can only assert BVALID when it drives a valid write response. Once asserted, it needs to be held until the BREADY signal of the master is sampled on the rising edge of the clock. The default value of BREADY can be set high when the host always accepts a write response signal within one cycle.

Read address channel: When the host drives valid address and control signals, the host can assert ARVALID. Once asserted, ARVALID needs to remain asserted until the slave's ARREADY is sampled on the rising edge of the clock. The default value of ARREADY can be high or low, but high is recommended (if it is low, a transfer requires at least two cycles, one to assert ARVALID and one to assert ARREADY); when ARREADY is high, the slave must be able to accept the effective address provided to it.

Read data channel: The slave can assert RVALID only when the slave driver is effectively reading data. Once asserted, it needs to be maintained until the rising edge of the clock samples the BREADY of the master. The default value of BREADY can be high, which requires the host to accept read data immediately as soon as it starts reading transmission. The slave needs to assert RLAST when the last burst read transfer occurs.

Dependencies between channel handshake signals

The signal pointed by the single arrow can be asserted before or after the start signal of the arrow; the signal pointed by the double arrow must be asserted after the start signal of the arrow.

The read transaction handshake dependencies are as follows:

- (1) The device can give the ARREADY signal when ARVALID appears, or it can give the ARREADY signal first and then wait for the ARVALID signal.
- (2) But the device must wait for both ARVALID and ARREADY signals to be valid before giving the RVALID signal and starting data transmission.

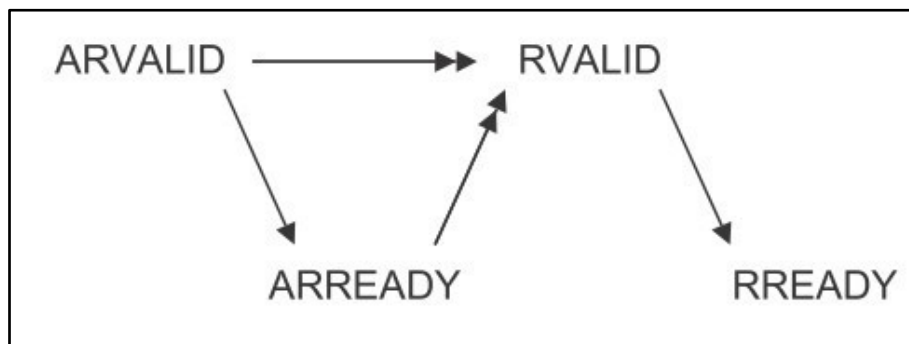


Figure 16-9 Read handshake dependencies

The write transaction handshake dependencies are as follows:

- (1) The host does not necessarily have to wait for the device to give the AWREADY or WREADY signal before giving the AWVALID or WVALID signal. (2) The device can wait for the signal AWVALID or WVALID to be valid or both before giving the AWREADY signal. (3) The device can wait for the AWVALID or WVALID signal to be valid or both signals to be valid before giving the WREADY signal.

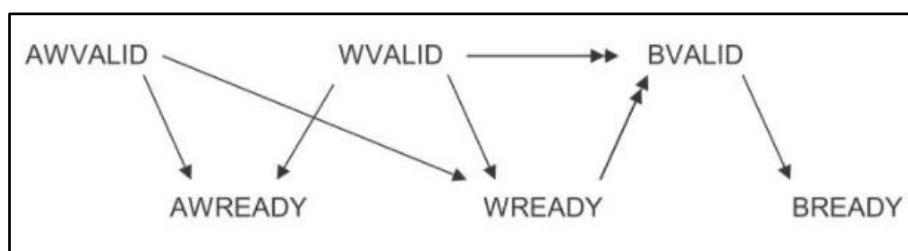


Figure 16-10 Writing handshake dependencies

16.2.4 Control signals and address description of address channel

1. Address ID

AWID[3:0] and ARID[3:0]: For only one master-slave device, the value can be set to any value.

2. Address structure

AWADDR[31:0] and ARADDR[31:0]: The AXI protocol is based on burst. The host only gives the address of the first byte of the burst transfer, and the slave must calculate the subsequent addresses of the burst transfer. Burst transmission cannot cross the 4KB boundary (preventing bursts from crossing the boundary between two slaves and also limiting the number of address increments that the slave needs to support).

3. Burst length

AWLEN[7:0] and ARLEN[7:0]: ARLEN[7:0] determines the burst length of read transmission, and AWLEN[7:0] determines the burst length of write transmission. AXI4 extended burst length supports INCR burst type 1~256 transmissions, and for other transmission types it still maintains 1~16 burst transmissions ($\text{Burst_Length} = \text{AxLEN}[7:0] + 1$).

Burst transmission rules: wrappingburst, burst length must be 2, 4, 8, 16; burst cannot cross 4 KB boundary; early termination of burst transmission is not supported.

None of the components can terminate a burst transfer prematurely. However, the host can reduce the number of write transfers by deasserting all write strobes so that not all write bytes are written. In a read burst, the host can ignore subsequent read data to reduce the number of reads. In other words, no matter what, all burst transfers must be completed.

4. Burst size

ARSIZE[2:0], read burst transfer; AWSIZE[2:0], write burst transfer.

Table 16-2 Burst byte size description

AxSIZE[2:0]	传输字节大小
3'b000	1
3'b001	2
3'b010	4
3'b011	8
3'b100	16
3'b101	32
3'b110	64
3'b111	128

If it is a 16-bit data width, 16 bits equals 2 bytes, then the AxSIZE will be 3'b001; if it is a 32-bit width, it will be 4 bytes, and the AxSIZE will be 3'b010; if it is a 64-bit data width, the AxSIZE will be 3'b011; for the AXI interface, the more commonly used data widths are 32, 64, and 128 bits.

5. Burst type

AWBURST[1:0] and ARBURST[1:0]:

FIXED: Address fixed during burst transmission, used for FIFO access

INCR: Incremental burst, the address is incremented during transmission. The amount of increment depends on the

WRAP: Loopback burst, similar to incremental burst, but will return to a low address at the boundary of a specific high address. The length of the loopback burst can only be 2, 4, 8, or 16 transmissions, and the first address of the transmission is aligned with the size of each transmission. The lowest address aligns the entire transferred data size. The loopback boundary is equal to $(AxSIZE * AxLEN)$. Commonly used in applications are INCR incremental bursts.

Table 16-3 Burst type description

AxBURST[1:0]	突发类型
2'b00	FIXED
2'b01	INCR
2'b10	WRAP
2'b11	Reserved

16.2.5 Data channel signal description

WDATA and RDATA: write and read data line signals.

WSTRB: Valid bytes, WSTRB[n:0] corresponds to the corresponding write byte, WSTRB[n] corresponds to WDATA[8n+7:8n], that is, whether the number of bytes of the data width is valid. When WVALID is low, WSTRB can be any value. When WVALID is high, the byte line with WSTRB high must indicate valid data. For general applications, just set all WSTRB to 1 to ensure that all data is valid. The read channel does not have this signal.

WLAST and RLAST: writing and reading the last byte. Pulling high indicates the transmission of the last byte, which also means the end of the transmission.

16.3 Practical exercises

16.3.1 Experimental objectives

The experimental task of this chapter is to customize an IP core for the AXI4 interface and read and write the PS-side DDR3 through the AXI_HP interface. Then use the LED light to determine whether it is correct.

16.3.2 Hardware resources

This experiment uses the button PL_KEY1 and LED light PL_LED3 on the PL side of the ZYNQ development board. See Figure 16-11.

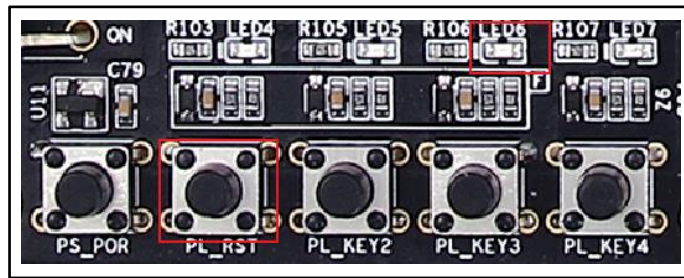


Figure 16-11 Hardware resources

16.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure diagram as shown in Figure 16-12.

This time we conduct a large amount of data exchange between PS and PL, so we use the S_AXI_HP0 port on the PS side, which has a higher rate than the AXI_GP port. The input button controls the DDR_test module to perform a read and write test on the DDR3 on the PS side. After the read and write test is completed, the LED indicates read and write completion and read and write errors. AXI Smartconnect is used to connect AXI memory-mapped (memory-mapped) master devices and slave devices.

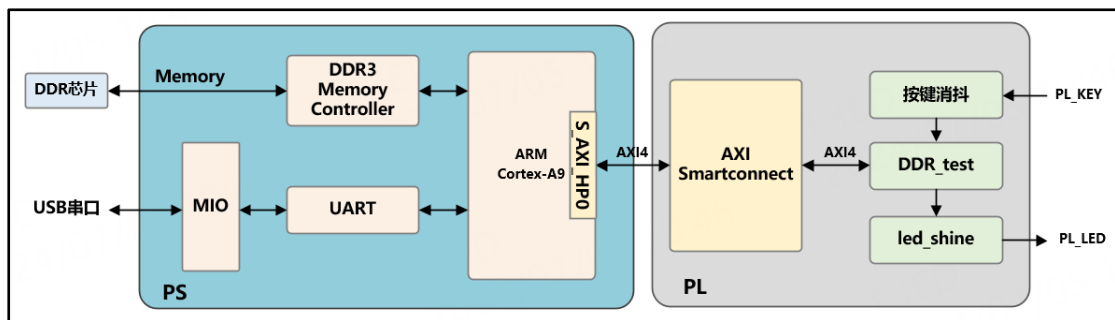


Figure 16-12 Overall block diagram

2. Hardware design

For the creation of the project, please refer to the previous chapter. Below we give the configured ZYNQ system II

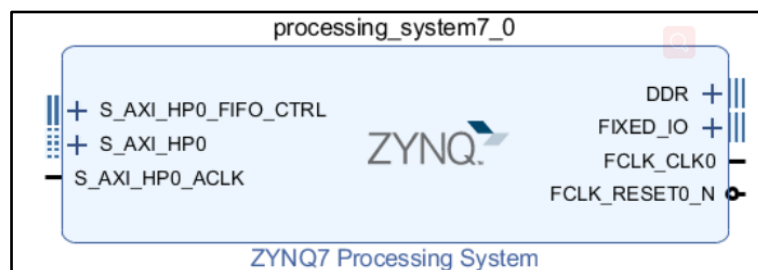


Figure 16-13 ZYNQ7 system module

The next step is to create our DDR_test module ip core and name it axi4_rw. For specific operations, please refer to the "Customized ip core" chapter. The difference is that the IP created this time is used as the master device of the AXI bus, so the modified configuration is as shown in Figure 16-14:

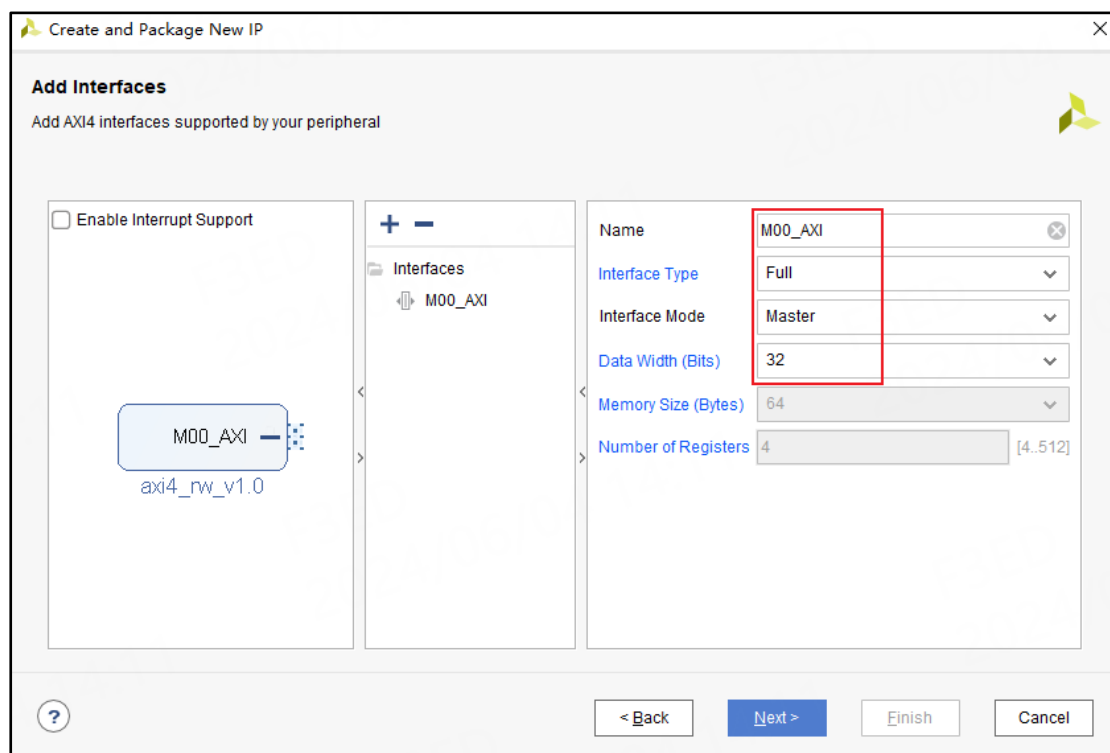


Figure 16-14 Custom IP interface configuration

The AXI4 interface has a total of five independent channels, and each channel has at least a few to as many as a dozen signals. It would be quite complicated for us to implement such an interface ourselves. But don't worry, when creating the IP of the AXI4 interface, the IP packaging tool provided by Vivado has automatically helped us implement such an interface and provided a sample program.

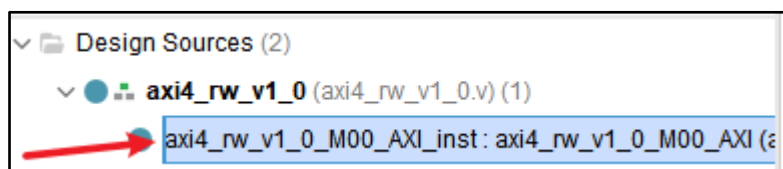


Figure 16-15 Sample program

Although the code in this file looks relatively long (more than 900 lines), it is mostly comments and very detailed. You can learn the implementation of the AXI4 protocol host by reading the code and comments.

After completing the creation of the DDR_test module IP, the IP core has been automatically added to the project's IP library. Find the ip_repo folder in the project directory. Only keep the axi4_rw_1.0 folder, and the others can be deleted directly, as shown in Figure 16-16.

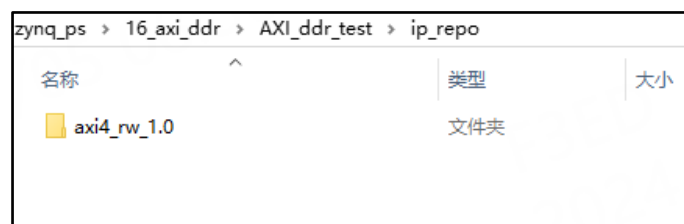


Figure 16-16 ip_repo folder

Add the customized IP core axi4_rw in the Diagram window. After adding, double-click to configure the IP. Modify the value of the variable C M00 AXI TARGET SLAVE BASE ADDR to 0x1000_0000, which is located in the address space of the DDR3 memory and is the starting address of the axi4_rw IP core for read and write operations. We reserve the storage space before this address for running software programs in PS.

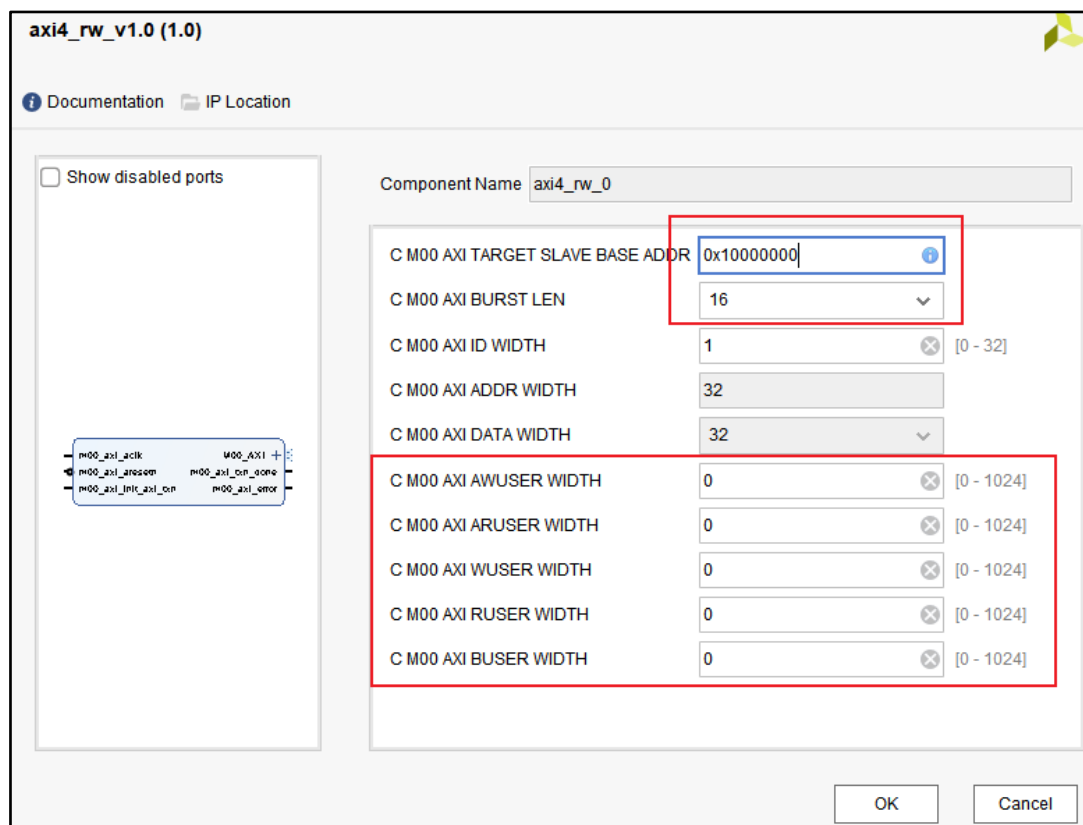


Figure 16-17 Configure custom ip core

Next, we add the button debounce module. The program file is consistent with the button debounce tutorial of our PL tutorial. Next, we add the debounce module to our project.

Create a new AXI_ddr_test.srcs/sources_1/imports/new folder in the project directory, and then copy key_filter.v to the folder.

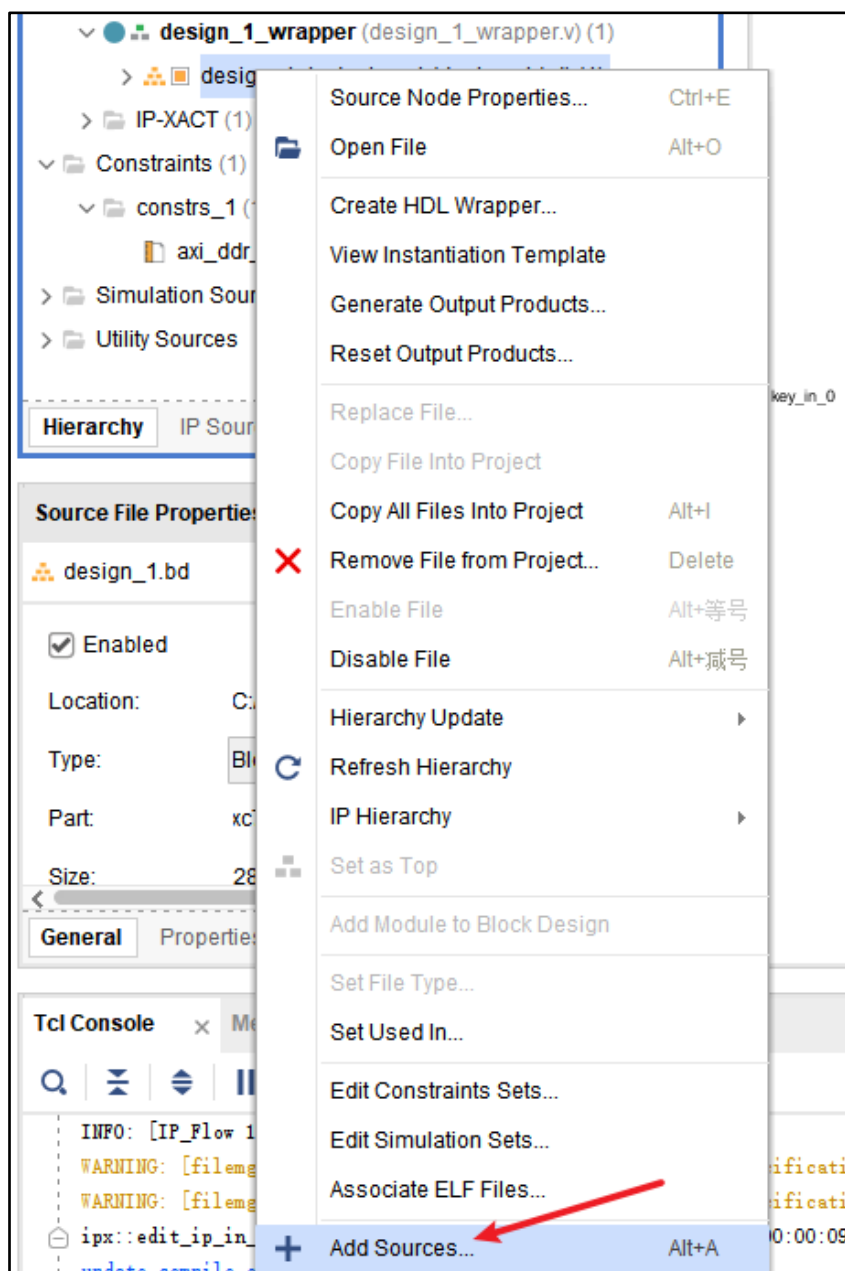


Figure 16-18 Add source files

Find the path where key_filter.v is located, select it and add it to this project.

Then add the led_shine module. When there is an error, the LED light is always on. When the reading and writing is completed without errors, the LED light flashes for a period of time. The code is as follows:

Code List 16-1 led_shine module reference code (led_shine.v)

```
01 module led_shine( 02 input sys_clk,
03 input rst_n, 04 input error, 05 input done, 06 output reg led
```

```
07 ); 08 09 wire flag ; 10 reg [3:0] num;
```



```
11 reg [25:0] cnt;
12 reg shine;
13
14 reg flag1,flag2;
15 always@(posedge sys_clk or negedge rst_n)
16     if(rst_n == 1'b0) begin
17         flag1 <= 1'b0;
18         flag2 <= 1'b0;
19     end
20     else begin
21         flag1 <= flag;
22         flag2 <= flag1;
23     end
24
25 assign flag = done & (~error);
26 assign flag3 = flag1 & (~flag2);
27
28 always@(posedge sys_clk or negedge rst_n)
29     if(rst_n == 1'b0)
30         num <= 4'd0;
31     else if(flag3 == 1'b1)
32         num <= 4'd10;
33     else if(num == 4'd0)
34         num <= num;
35     else if(shine == 1'b1 && num != 4'd0)
36         num <= num - 1'b1;
37
38 always@(posedge sys_clk or negedge rst_n)
39     if(rst_n == 1'b0)
40         shine <= 1'b0;
41     else if(cnt == 26'd49_999_999)
42         shine <= 1'b1;
43     else
44         shine <= 1'b0;
45
46 always@(posedge sys_clk or negedge rst_n)
47     if(rst_n == 1'b0)
48         cnt <= 26'd0;
49     else if(num != 4'd0)
50         if(cnt == 26'd49_999_999)
51             cnt <= 26'd0;
52     else
53         cnt <= cnt + 1'd1;
54
55 always@(posedge sys_clk or negedge rst_n)
56     if(rst_n == 1'b0)
57         led <= 1'b1;
58     else if(error == 1'b1)
59         led <= 1'b0;
60     else if(shine == 1'b1)
61         led <= ~led;
62
63 endmodule
```

Add the led_shine.v file in the same way.

Next, right-click in the blank space of the Diagram interface, select "Add Module..." and add the key_filter module and led_shine module.

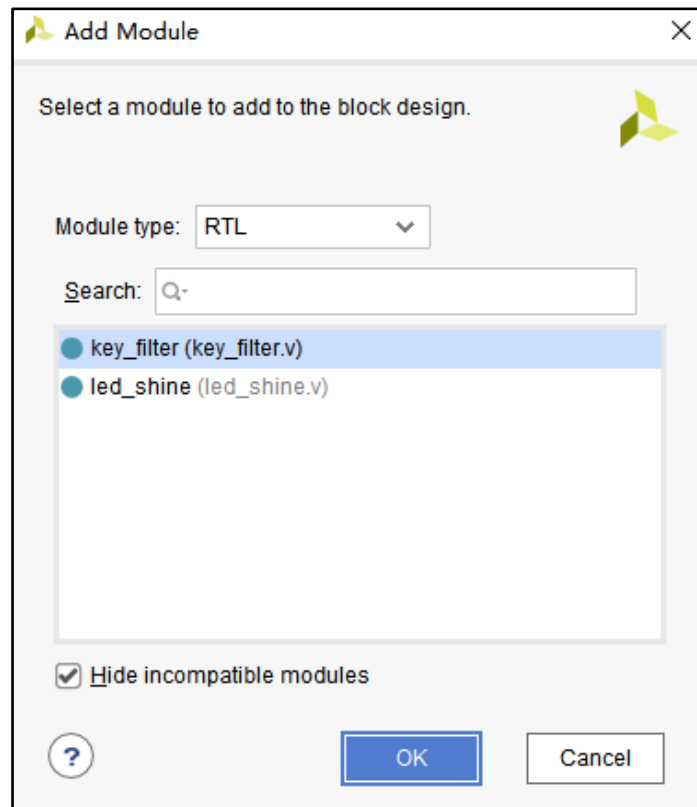


Figure 16-19 Add module

After the IP core is added, use the tool's automatic connection function to connect the design. Then manually connect the button to the debounce module, AXI4_RW IP core, and led_shine. After the connection is completed, right-click in the blank area of the Diagram window and select "Regenerate Layout" to re-layout the design. The layout interface is shown in Figure 16-20:

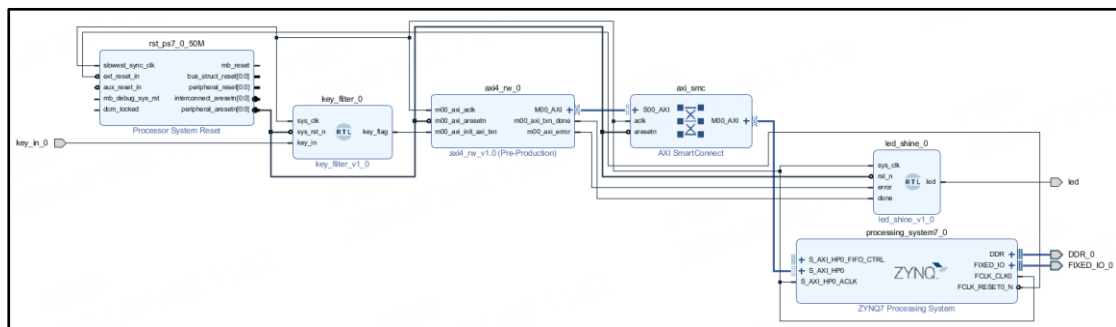


Figure 16-20 Overall design block diagram

As can be seen from Figure 16-20, after executing the automatic connection, the tool automatically adds two IP cores, namely AXI Smartconnect and Processor System Reset. The M_AXI of the axi4_rw IP core serves as the host interface and is connected to the S_AXI slave interface of the ZYNQ7 system through the intermediate IP core AXI_SMC. The function of AXI Smartconnect is similar to that of the AXI Interconnect IP core, which is used to connect the AXI memory-mapped master device to the memory-mapped slave device. Among them, key_in_0 and led pin are extracted manually.

At this point our Block Design is completed. Right-click in the blank area of the Diagram window and select "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. "Ctrl + S" saves the design and executes "Generate Output Products" and "Create HDL Wrapper" in sequence.

Next, perform pin constraints, click Open Elaborated Design in RTL ANALYSIS in the Flow Navigator navigation bar, and then click Layout -> I/O Planning in the menu bar. The pin assignment is shown in Figure 16-21.

Name	Direction	Neg Di...	Package ...	Fixed	Bank	I/O Std
All ports (132)						
DDR_35690 (71)	INOUT			✓	502	(Multiple)*
FIXED_IO_35690 (59)	INOUT			✓	(Multiple)	(Multiple)*
Scalar ports (2)						
key_in_0	IN		G19	✓	35	LVC MOS33*
led	OUT		K14	✓	35	LVC MOS33*

Figure 16-21 Pin assignment

Finally, generate the bitstream file, export the hardware, and start the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and

Code Listing 16-2 Read DDR reference code (main.c)

```
1 #include "stdio.h" 2 #include "xil_cache.h" 3 #include "xil_printf.h" 4 #include "xil_io.h" 5 6 int main() 7 { 8     int i; 9     char chr; 10     Xil_DCacheDisable(); //Close data cache 12     while(1) 13 { 14         printf("Please enter one character at random !\n"); 15         scanf("%c",&chr); 16         for(i=0;i<256;i=i+4) 17 { 18             printf("addr %d : date %d\n",i,(int)(Xil_In32(0x10000000+i))); 19         } 20     } 21     return 0; 22 }
```

The code is very small, 11 lines of code. Turn off the data cache to ensure that the data in the DDR is read every time, not the data in the cache. Line 12~20 is a while loop. Whenever we input any character from the serial port, we will read the value of the 0x10000000 memory segment, which is 256 bytes long, and print it out through the serial port. It should be noted that Xil_In32 reads 32-bit data each time, so i must accumulate 4 each time in the for loop;

16.3.4 Board verification

As shown in Figure 16-22, the development board is connected. After the lines are connected correctly, turn on the

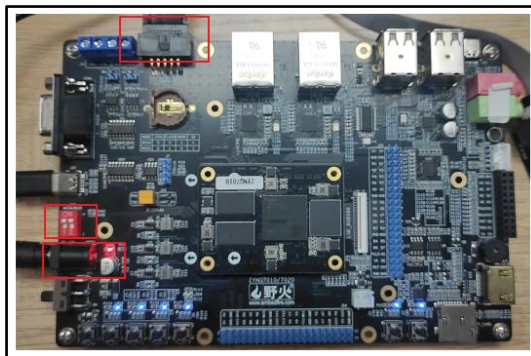


Figure 16-22 Program download connection diagram

As shown in Figure 16-23, download the program for the development board.

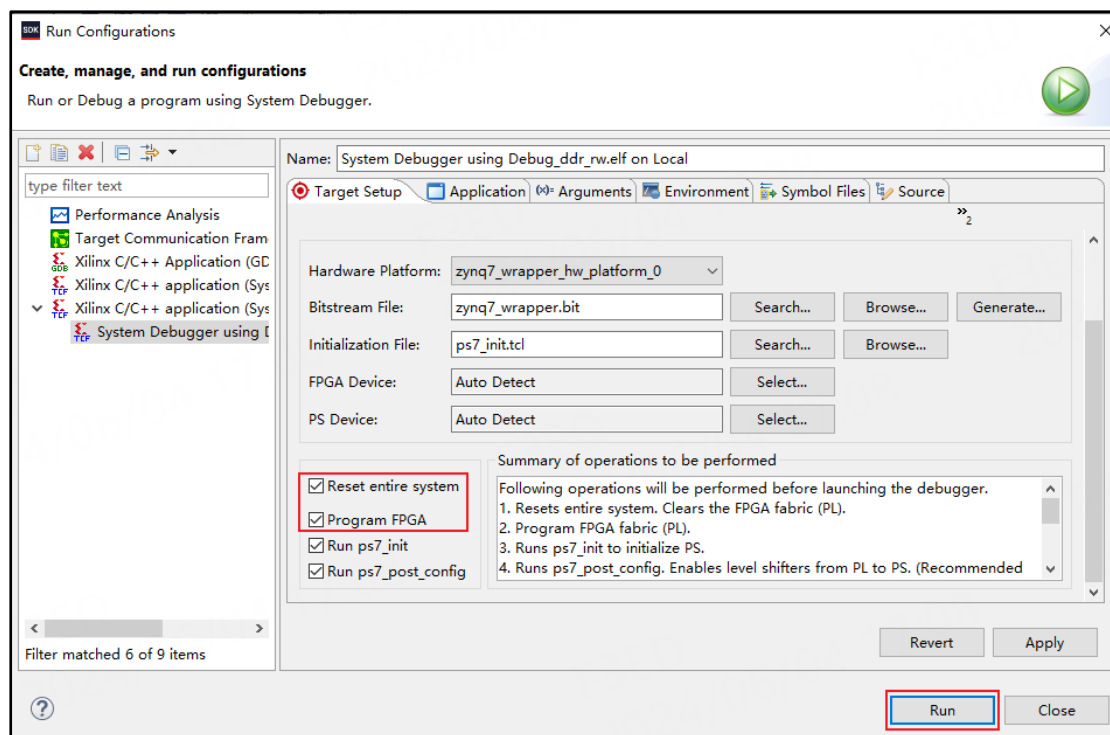


Figure 16-23 Program download

Implementation phenomenon: Before pressing the button PL_KEY1, any character is sent through the serial port, and the data read is a random value, as shown in Figure 16-24. After pressing the button PL_KEY1, send any character again, and the read data is 1~64, as shown in Figure 16-25. At the same time, PL_LED3 on the development board will flash for a period of time, indicating that the read and write test is successful.

```
addr 0 : date 71323649
addr 4 : date 71438336
addr 8 : date -1968700920
addr 12 : date 1704195
addr 16 : date 1206272
addr 20 : date 1241531744
addr 24 : date -1039136120
.....
addr 228 : date 76578882
addr 232 : date 33563555
addr 236 : date 527116
addr 240 : date 1074040902
addr 244 : date 18780236
addr 248 : date 805308448
addr 252 : date 6324624
```

Figure 16-24 Random values

```
addr 0 : date 1
addr 4 : date 2
addr 8 : date 3
addr 12 : date 4
addr 16 : date 5
addr 20 : date 6
addr 24 : date 7
.....
addr 232 : date 59
addr 236 : date 60
addr 240 : date 61
addr 244 : date 62
addr 248 : date 63
addr 252 : date 64
```

Figure 16-25 Determine the value

16.4 End of chapter summary

Data interaction is often required between PL and PS, and DDR can meet the needs of large amounts of data storage on the PL side. In this experiment, by customizing an AXI4 IP core, PL can access the DDR on the PS side through the HP interface.

16.5 Development training

This experiment only writes and reads 64 pieces of data to DDR3. You can modify it on the basis of this experiment to write and read 1024 pieces of data (that is, 4K address range) to DDR3.

Chapter 17 AXI DMA Reading and Writing Test

17.1 Chapter Introduction

In the previous chapter, we said that the data interaction between PL and PS needs to be processed through the ARM core on the PS side, that is, our hardware subsystem indirectly reads and writes the system memory. In this chapter, we introduce another data reading and writing method—DMA (Direct Memory Access) method, which realizes high-speed data interaction between peripherals and DDR.

17.2 Theoretical study

DMA is a memory access technology in computer science. It is often used in high-speed peripherals. By requesting the CPU to give up bus control, it can realize direct high-speed data transfer between the control peripherals and the storage unit. Under normal circumstances, reading peripheral data into memory or transferring memory to peripherals is generally done through CPU control, such as using query or interrupt methods. Although the interrupt method can improve the utilization of the CPU, it will also have efficiency problems. For batch transfer of data, the DMA method can solve the problem of efficiency and speed. The CPU only needs to provide the address and length to the DMA, and the DMA can take over the bus and access the memory. After the DMA completes the work, it notifies the CPU and hands over the bus control.

DMA is a fast data transfer method, usually used to transfer data blocks with large amounts of data. Many hardware systems will use DMA, including hard disk controllers, graphics cards, network cards and sound cards. Using DMA is also a good choice when using high-speed AD/DA.

DMA uses hardware to implement high-speed data transfer directly between memory and memory or between memory and I/O devices. When using DMA, the CPU sends a storage transfer request to the DMA controller, so that while the DMA controller is transferring, the CPU performs other operations. When the transfer operation is completed, the DMA notifies the CPU in the form of an interrupt. In order to initiate a transfer transaction, the DMA controller must obtain the following data:

- (1) Source address - The address from which data is read.
- (2) Destination address - the address where the data is written.
- (3) Transfer length - the number of bytes that should be transferred.

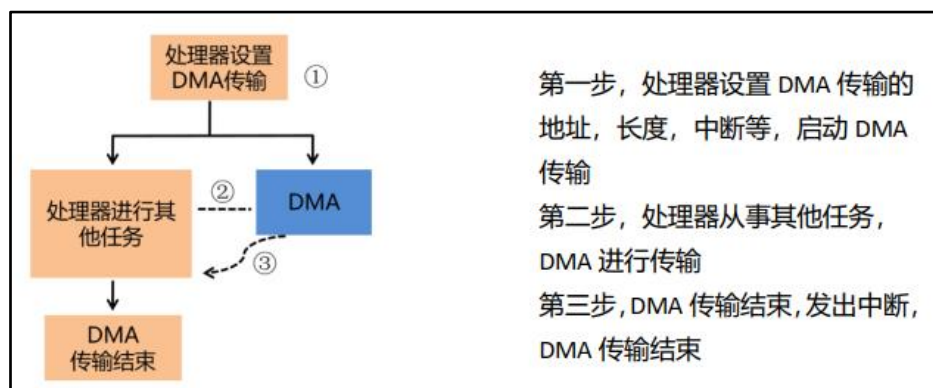


Figure 17-1 DMA transfer process

ZYNQ provides two types of DMA, one is the hard-core DMA integrated in the PS, and the other is the soft-core AXI DMA IP used in the PL.

Let's talk about hard-core DMA first. During the design process of ARM CPU, a large amount of data movement has been taken into consideration, so the CPU comes with a DMA controller DMAC. This DMAC resides in the PS and must be programmed through DMA instructions reside in the memory. These programs are often prepared by the CPU, so the CPU operation consent is required to start DMA. DMAC supports up to 8 channels, so cores with multiple DMA structures can implement multiple DMA transfers in the same DMAC. The connection between DMAC and PL is through the AXI_GP interface. This interface supports up to 32-bit width, which also limits the transmission rate in this mode. The theoretical maximum rate is 600MB/s. This mode does not occupy PL resources, but requires programming of DMA instructions, which will increase the complexity of the software.

Another way is to use the DMA IP core on the PL side, which uses the AXI_HP interface to complete high-speed data transmission, with a rate of up to 1200MB/s. However, the logic design of this method is more complex and takes up more resources.

This chapter focuses on PL's DMA, the AXI DMA IP core.

The AXI Direct Memory Access (AXI DMA) IP core provides high-bandwidth direct memory access between the AXI4 memory map and the AXI4-Stream IP interface. Its optional scatter gather feature also offloads data movement tasks from the central processing unit (CPU) in processor-based systems. Initialization, status and management registers are accessed through the AXI4-Lite slave interface. The core functional composition is shown in Figure 17-2:

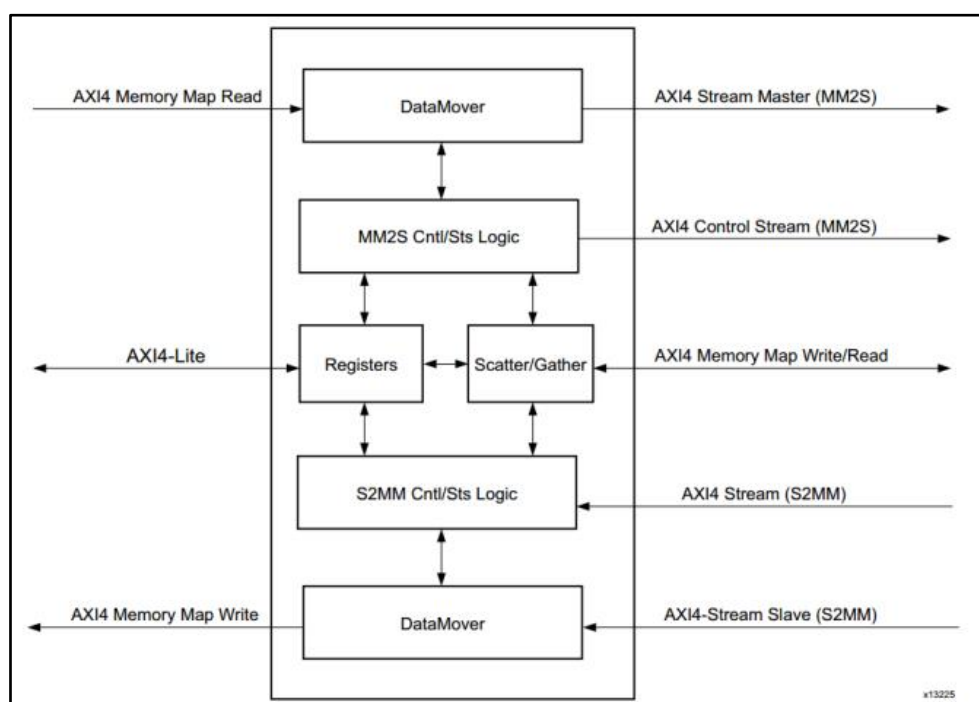


Figure 17-2 AXI DMA block diagram

As can be seen from the figure, the IP core uses three buses. AXI-Lite is a lightweight AXI bus and is often used to configure registers. AXI Memory Map is used to interact with memory. This project is DDR. The AXI4 Stream interface is used to read and write peripherals. It can send and receive continuous data streams without an address. Among them, AXI4 Stream Master (MM2S, Memory Map to Stream) is used to write to peripherals, and AXI4-Stream Slave (S2MM, Stream to Memory Map) is used to read peripherals. The peripherals of this project can be considered as the PL side.

AXI DMA provides 3 modes, namely Direct Register mode, Scatter/Gather mode and Cyclic DMA mode. Here we briefly introduce the commonly used Direct Register mode and Scatter/Gather mode.

Direct Register DMA direct register mode is Simple DMA. Direct Register mode provides a configuration for MM2S and S2MM channels to perform simple DMA transfers, which consumes less FPGA resources. Simple DMA allows applications to define a single transaction between the DMA and the Device. This mode contains two channels: one is responsible for the transfer from DMA to Device, and the other is from Device to DMA. To initiate a transfer, the application must set the buffer address and length fields.

Scatter/Gather DMA mode allows data to be transferred to and from multiple storage areas in a single DMA transaction. This is equivalent to chaining multiple Simple DMA requests together. SGDMA allows an application to define a sequence of transactions in memory, and the hardware handles these transactions automatically without further intervention from the application. During this process, the application can continue to add more tasks to keep the hardware working. Users can detect the completion status of transactions through polling or interrupting. SGDMA processes the entire packet (that is, a sequence of data bytes representing the message) and allows the packet to be broken down into one or more transactions. For example, an Ethernet IP packet consists of a 14-byte header and one or more bytes of payload. Using SGDMA, an application can point a BD (Buffer Descriptor, an object that describes a transaction) to the header, another BD to the payload, and transmit them as a single message. This strategy can make the TCP/IP stack more efficient by allowing the packet's header and payload to be stored in different memory areas without having to assemble the packet into contiguous blocks of memory.

This project does not need to use the Scatter/Gather DMA mode, because this function consumes more resources and the software control is more complex. However, this project can be implemented using the direct register mode.

In addition, as mentioned before, the AXI bus accesses the PS side DDR through the HP port, which is a way of PS and PL data interaction, and the DMA in this chapter is another way of PS and PL data interaction. Essentially, the two methods are the same, both access the PS side DDR. The difference is that it is implemented in a PL side code, which is more flexible and controllable for users. The disadvantage is that it requires writing code, which is difficult for people who are not familiar with FPGA; DMA The control right is mainly on the PS side, and the PS configures the reading and writing of DMA. The advantage is that it is relatively intuitive, but it requires good software skills.

17.3 Practical exercises

17.3.1 Experimental objectives

PS writes a piece of data through DMA, then reads the data from DMA and compares whether the read data is consistent with the written data.

17.3.2 Hardware resources

This experiment used the internal resources of the ZYNQ chip.

17.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 17-3.

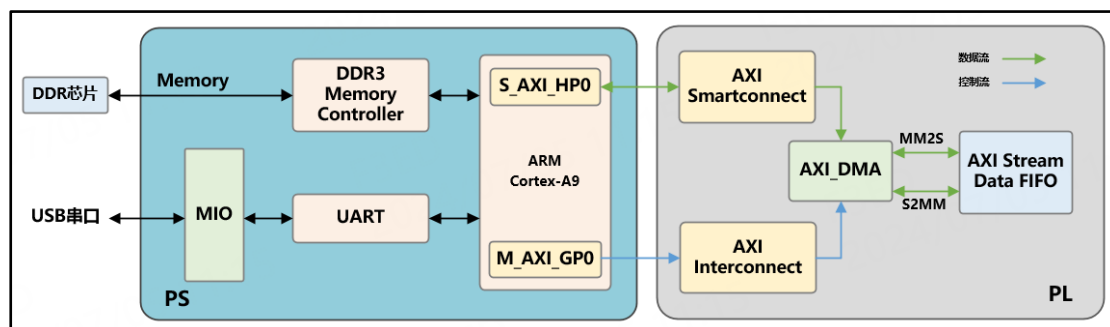


Figure 17-3 Overall block diagram

In practical applications, DMA is generally connected to an IP core that generates data or requires data. The IP core can be a high-speed AD or DA IP core with a Stream interface. Without loss of generality, in this experiment, we use the AXI4 Stream Data FIFO IP core to act as this type of IP for DMA loopback experiments.

PS opens a GP0 interface for setting, starting and monitoring data transmission of AXI DMA, and opens an HP0 interface for high-speed data transmission.

First, the PS side ARM sends data to DDR.

Then the PS controls the DMA to read the data in the DDR through the data read channel; the DMA passes the read data to the FIFO.

Then the FIFO transfers the data to the DMA; the PS controls the DMA so that the DMA writes the data into the DDR through the data write channel.

Final transmission verification compares the received data with the sent data to see if they are consistent.

2. Hardware design

Detailed steps for project creation can be found in the previous chapters. Below we only list the configuration process.

ZYNQ system ip

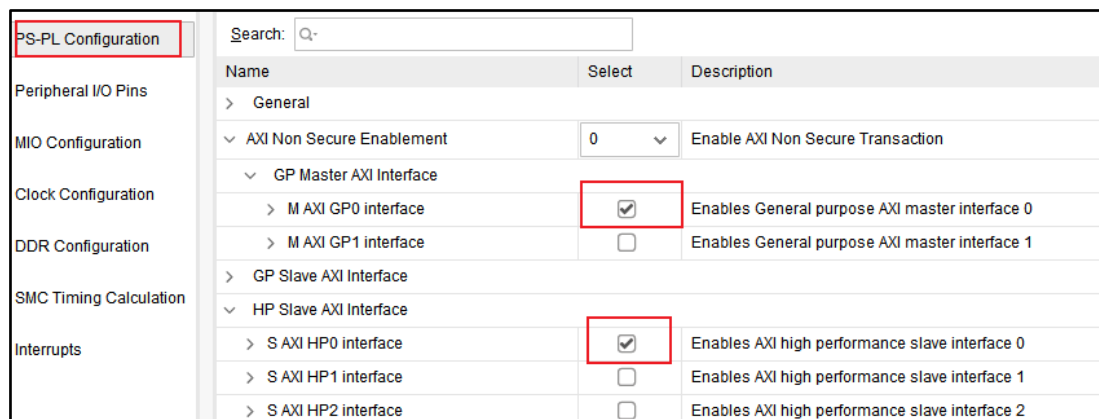


Figure 17-4 Adding GP0 and HP0 interfaces

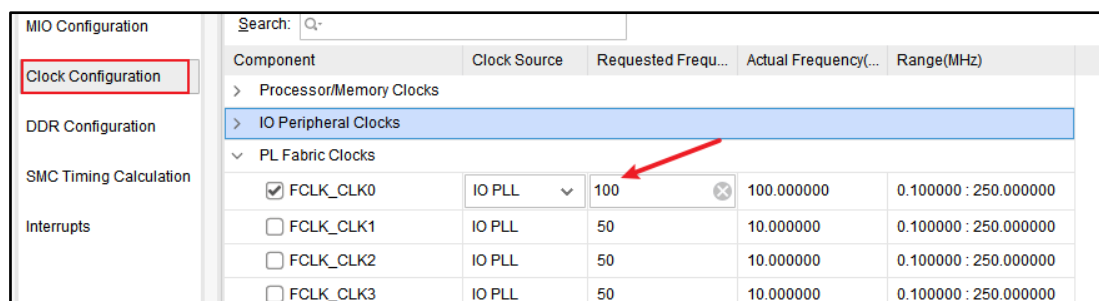


Figure 17-5 Clock supplied to PL side

From the above theoretical study, we can know that DMA notifies the CPU by sending an interrupt after the transfer is completed, so we need to enable the interrupt from PL to PS.

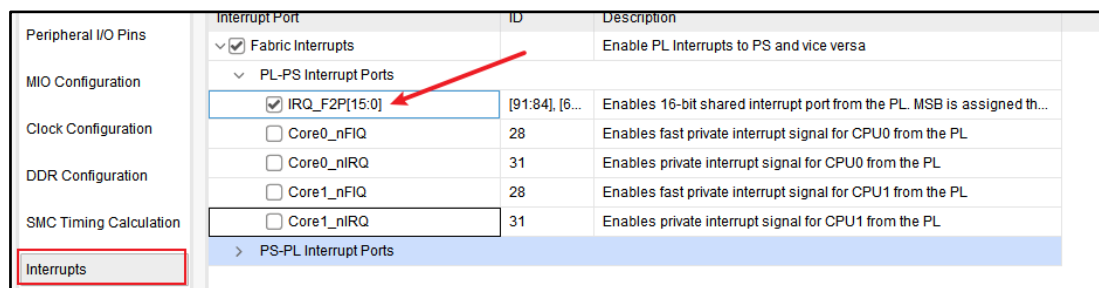


Figure 17-6 Enable PL to PS interrupt

This completes the configuration of the ZYNQ system IP. Click OK on the configuration interface to generate the

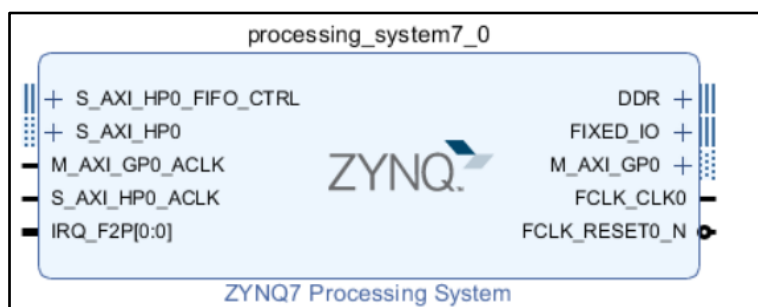


Figure 17-7 ZYNQ7 system module

DMA IP

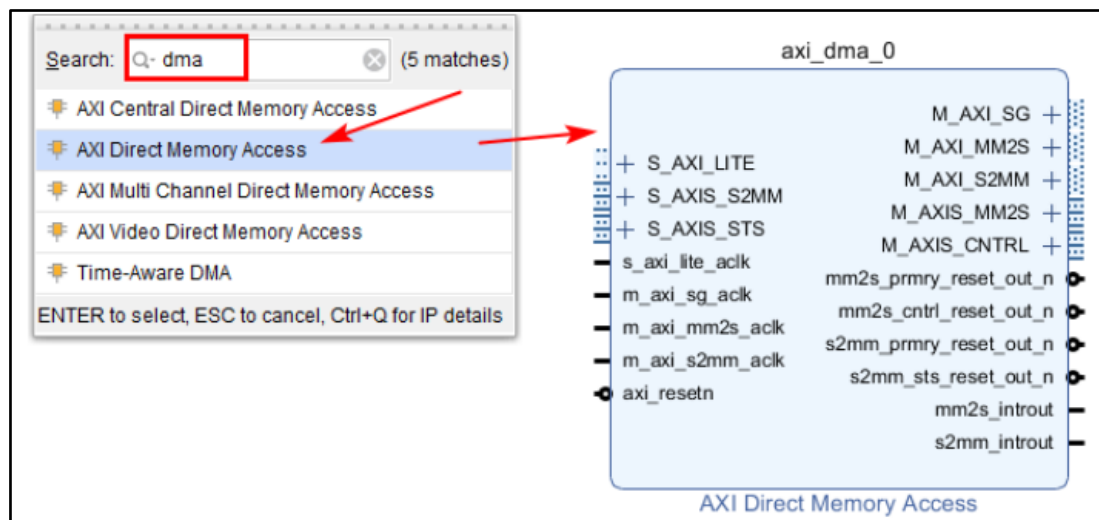


Figure 17-8 Adding DMA module

Double-click the IP core to display the configuration interface:

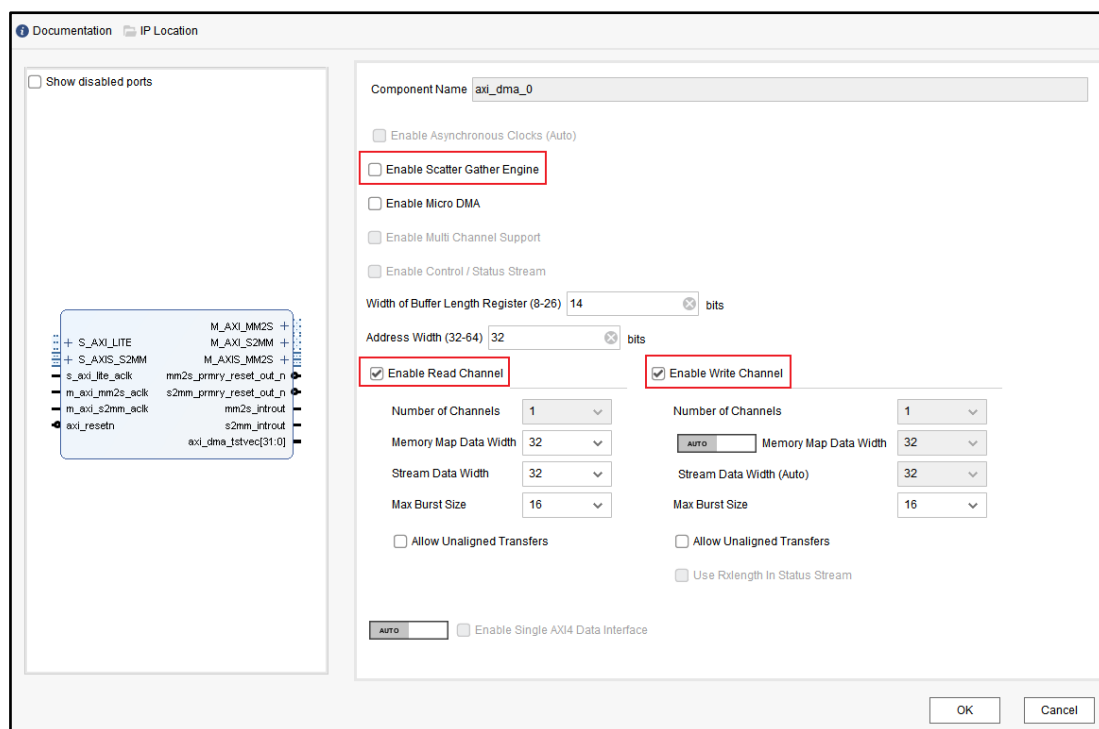


Figure 17-9 AXI DMA configuration interface

Enable Scatter Gather Engine: Check this option to enable Scatter Gather mode operation and include the Scatter Gather Engine in the AXI DMA. Uncheck this option to enable Direct Register mode operation, excluding the Scatter Gather Engine in AXI DMA. Disabling the Scatter Gather Engine causes all output ports of the Scatter/Gather Engine to be bound to zero, and all input ports remain open. Here we uncheck Enable Scatter Gather Engine.

Enable Micro DMA: Selecting this option will generate a highly optimized DMA that consumes less resources and is suitable for application scenarios where small amounts of data are transferred. However, we leave it unchecked here.

Width of Buffer Length Register: This integer value specifies the number of significant bits used to control the length of the field buffer and the number of bytes of the Status field transferred in the Scatter/Gather descriptor. The number of bytes is equal to 2^{Width} . The maximum length here is 26, and the number of bytes is $226 = 67,108,864$ bytes, which is 64MByte. Here we keep the default setting of 14.

Address Width (32 - 64): Specifies the width of the address space, which can be any value between 32 and 64. Here keep the default value of 32.

Enable Read Channel: Enable the AXI DMA read channel MM2S. The relevant options are as follows:

(1) **Number of Channels:** Specify the number of channels. Keep the default value of 1. (2) **Memory Map Data Width:** The data bit width of the AXI MM2S memory map read data bus. Valid values are 32, 64, 128, 256, 512 and 1024. Keep the default value of 32 here. (3) **Stream Data Width:** The data bit width of the AXI MM2S AXI4-Stream data bus. This value must be less than or equal to Memory MapData Width. Valid values are 8, 16, 32, 64, 128, 512, and 1024. Here keep the default value of 32. (4) **Max Burst Size:** Burst partition granularity setting. This setting specifies the maximum value for the burst period on the AXI4-Memory Map side of MM2S, valid values are 2, 4, 8, 16, 32, 64, 128 and 256. Keep the default value of 16 here. (5) **Allow Unaligned Transfers:** Enable or disable the MM2S Data Realignment Engine (DRE). When selected, DRE is enabled, allowing data to be realigned to the 8-bit byte level on the MM2S data path. For the MM2S channel, its role is to read data from memory. When DRE is enabled, data reading can start from any byte offset of the buffer address, and the data read is aligned so that the first byte read is the first valid byte output on the AXI4-Stream. Note that unaligned buffers, source addresses, or destination addresses are not supported if the corresponding channel's DRE is disabled. Using unaligned addresses with DRE disabled can lead to undefined results. DRE supports AXI4-Stream data width up to 512 bits.

Enable Write Channel: Enable the write channel S2MM of AXI DMA. For related options, please refer to the read

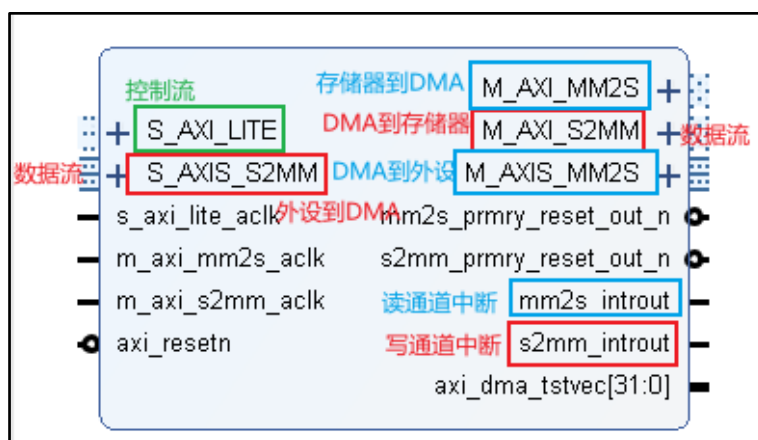


Figure 17-10 Generate AXI DMA IP core

AXI-Stream Data FIFO IP

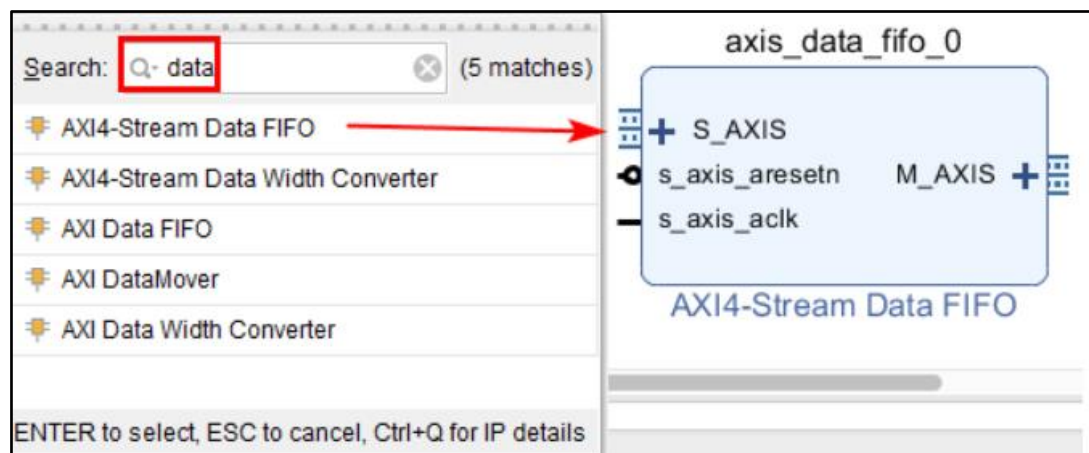


Figure 17-11 Adding AXI-Stream Data FIFO ip core

The IP core can be kept as default. We only need to know that the IP core implements the conversion between the AXI-Full bus and the AXI-Stream bus.

Concat IP

This IP core integrates multiple scattered signals into bus signals. Here, the read and write channels each have an interrupt, which can be combined and connected to the interrupt signal of ZYNQ IP.

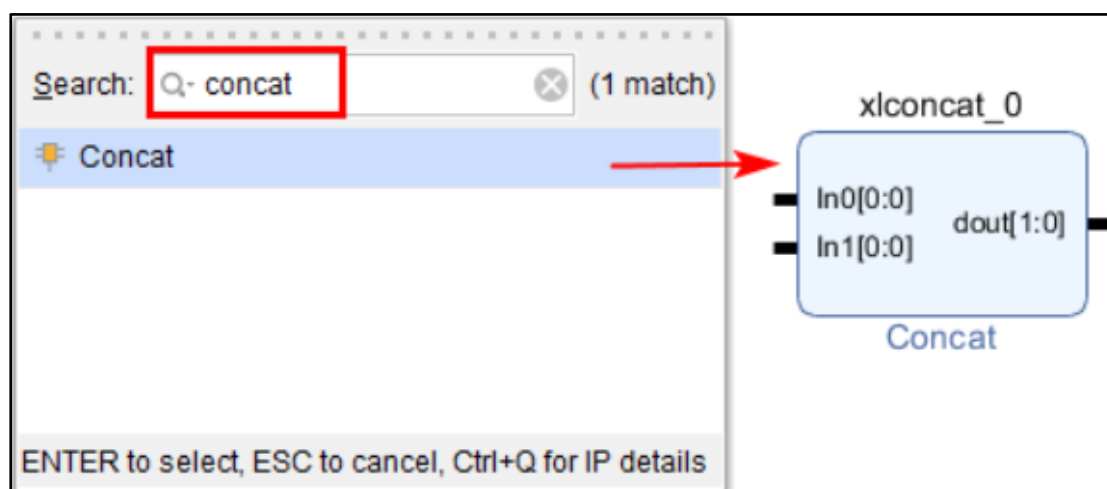


Figure 17-12 Concat IP

The creation and configuration of all IP cores is completed here. If there is a Run Block Automation prompt in the Diagram, click Automatically configure ZYNQ IP. If not, ignore it. Then click "Run connection Automation" to connect automatically, check "All Automation" in the pop-up interface, and then click OK.

After the automatic connection was completed, it was found that axis_data_fifo IP and Concat IP were not connected. We connected manually. The final overall IP module connection diagram is shown in Figure 17-13:

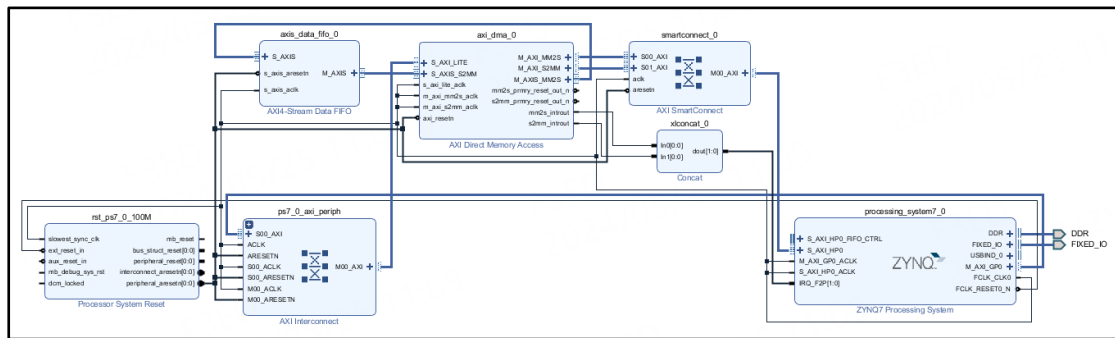


Figure 17-13 Overall IP block diagram

At this point our Block Design is completed. Press the "F6" key to perform "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally press the shortcut key "Ctrl + S" to save the design.

Then execute "Generate Output Products" and "Create HDL Wrapper" in sequence. Then generate a bit stream file and export the hardware (check "Include bitstream" in the pop-up dialog box). Finally launch the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and

Code List 17-1 AXI DMA read and write test reference code (main.c)

```
1 #include "xaxidma.h" 2 #include "xparameters.h" 3 #include "xil_printf.h" 4 #include "xscugic.h" 5 6 #define
RX_INTR_ID XPAR_FABRIC_AXIDMA_0_S2MM_INTROUT_VEC_ID 7 #define TX_INTR_ID XPAR_F
ABRIC_AXIDMA_0_MM2S_INTROUT_VEC_ID 8 #define MEM_BASE_ADDR (XPAR_PS7_DDR_0_S
AXI_BASEADDR + 0x1000000) 9 #define TX_BUFFER_BASE (MEM_BASE_ADDR + 0x00100000) 10 #d
efine RX_BUFFER_BASE (MEM_BASE_ADDR + 0x00300000) 11 #define MAX_PKT_LEN 0x100 12 #defi
ne RESET_TIMEOUT_COUNTER 10000 13 14 static int check_data(int Length, u8 StartValue); 15 static void
TxIntr_Handler(void *Callback); 16 static void RxIntr_Handler(void *Callback); 17 static int SetupIntr_System
(XScuGic * IntcInstancePtr, 18 20 u16 TxIntrId, u16 RxIntrId); 21 22 static int Index; 33 u8 *TxBuffer_Ptr; 34
u8 *RxBuffer_Ptr; 35 XAxiDma_Config *Config; 36
```

```
37 TxBuffer_Ptr = (u8 *) TX_BUFFER_BASE; 38 RxBuffer_Ptr = (u8 *) RX_BUFFER_BASE; 39 40
Config = XAxiDma_LookupConfig(XPAR_AXIDMA_0_DEVICE_ID); 41 XAxiDma_CfgInitialize(
&AxiDma, Config); 42 43 if (XAxiDma_HasSg(&AxiDma)) { 44 xil_printf("Device configured as SG
mode \r\n"); 45 return XST_FAILURE; 46 } 47 48 SetupIntr_System(&Intc, &AxiDma, TX_INTR_ID
, RX_INTR_ID); 49 50 XAxiDma_IntrEnable(&AxiDma, XAXIDMA_IRQ_ALL_MASK, 51 XAXID
MA_DMA_TO_DEVICE); 52 XAXIDMA_DEVICE_TO_DMA); 54 55 /*Judgment flag*/ 56 Error =
0; 57 Tx_Done = 0; 58 Rx_Done = 0; 59 60 Value = 0x0; //Start writing at address offset 0 61 for (Inde
x = 0; Index < MAX_PKT_LEN; Index++) { 62 TxBuffer_Ptr[Index] = Value; 63 Value = (Value + 2)
& 0xFF; //The data is incremented by 64 } 65 66 Xil_DCacheFlushRange((UINTPTR) TxBuffer_Ptr,
MAX_PKT_LEN); 67 XAxiDma_SimpleTransfer(&AxiDma, (UINTPTR) TxBuffer_Ptr, 68 MAX_P
KT_LEN, XAXIDMA_DMA_TO_DEVICE); 69 70 XAXIDMA_DEVICE_TO_DMA); 72 73 Xil_DC
acheFlushRange((UINTPTR) RxBuffer_Ptr, MAX_PKT_LEN); 74 while (!Tx_Done && !Rx_Done &
& !Error); 75 76 /*Judge whether sending and receiving are normal*/ 77 if (Error) { 78 test transmit%s
done, " 79 "receive%s done\r\n", Tx_Done ? "" : " not", 80 Rx_Done ? "" : " not"); 81 goto Done; 82 }
83 84 /*Check whether the data is accurate*/ 85 if (check_data(MAX_PKT_LEN, 0x0) != XST_SUCC
ESS) { 86 xil_printf("Data check failed\r\n"); 87 goto Done; 88 }; 89 90 xil_printf("Successfully ran A
XI DMA interrupt Example\r\n"); 91 DisableIntr_System(&Intc, TX_INTR_ID, RX_INTR_ID); 92 93
Done: 94 95 return 0; 96 } 97 98 /*Check the read and write data value*/ 99 static int check_data(int L
ength, u8 StartValue) 100 { 101 int Index = 0;
```

```
102 u8 Value = StartValue; 103 u8 *RxPacket = (u8 *) RX_BUFFER_BASE; 104
105 for(Index = 0; Index < Length; Index++) { 106 if (RxPacket[Index] != Value)
{ 107 xil_printf("Data error %d: %x/%x\r\n", 108 Index, RxPacket[Index], Value)
; 109 return XST_FAILURE; 110 } 111 Value = (Value + 2) & 0xFF; 112 } 113 r
eturn
```

In lines 40-53 of the code, AXI DMA is initialized and the AXI DMA interrupt system is established. When the DMA transfer is completed, the CPU will be notified through an interrupt.

Lines 61-73 of the code first write data to the TX_BUFFER_BASE address of the DDR, and then execute Xil_DCacheFlushRange before line 67 to refresh the cache in the TxBuffer_Ptr address range to avoid data inconsistency when the cache hits. Then line 73 refreshes the buffer of the RxBuffer_Ptr address location to ensure that the latest data can be read when the DMA transfer is completed. Finally wait for the DMA transfer to complete.

In line 85 of the code, after the DMA transfer is completed, the data of RX_BUFFER_BASE and TX_BUFFER_BASE are checked to see if the data is correct. The specific code is implemented in lines 99-114. Finally turn off DMA interrupts.

Next, we will introduce AXI DMA interrupt related.

Code List 17-2 AXI DMA read and write test part reference code (main.c)

```
116 /*Send interrupt handler function*/ 117 static void TxIntr_Handler(void *Callback) 118 { 119 int TimeOut;
120 u32 IrqStatus; 121 XAxiDma *AxiDmaInst = (XAxiDma *) Callback; 122 123 IrqStatus = XAxiDma_Intr
GetIrq(AxiDmaInst, XAXIDMA_DMA_TO_DEVICE); 124 XAXIDMA_IRQ_ERROR_MASK) { 128 Error
= 1; 129 134 TimeOut -= 1; 135 } 136 return; 137 } 138 139 if ((IrqStatus & XAXIDMA_IRQ_IOC_MASK)) 1
40 Tx_Done = 1; //Complete, flag position one 141 } 142 143 /*Receive interrupt processing function*/ 144 stat
ic void RxIntr_Handler(void *Callback) 145 { 146 int TimeOut; 147 u32 IrqStatus; 148 XAxiDma *AxiDmaIns
t = (XAxiDma *) Callback; 149 150 IrqStatus = XAXIDMA_DEVICE_TO_DMA);
```



```
151 XAxiDma_Reset(AxiDmaInst); 157 TimeOut = RESET_TIMEOUT_COUNTER; 158 while (TimeOut) { 159 if (
XAxiDma_ResetIsDone(AxiDmaInst)) 160 break; 161 TimeOut -= 1; 162 } 163 return; 164 } 165 166 167 if ((IrqStat
us & XAXIDMA_IRQ_IOC_MASK)) 168 Rx_Done = 1; //Complete, flag position one 169 } 170 171 /*Interrupt syst
em creation*/ 172 static int SetupIntr_System(XScuGic * IntcInstancePtr, XAxiDma * 173 AxiDmaPtr, 174 u16 TxIn
trId, u16 RxIntrId) 175 { 176 int Status; 177 XScuGic_Config *IntcConfig; 178 179 IntcConfig = (NULL == IntcCon
fig) { 181 return XST_FAILURE; 182 } 183 Status = 186 return XST_FAILURE; 187 } 188 189 192 Status = XScuG
ic_Connect(IntcInstancePtr, RxIntrId, 200 (Xil_InterruptHandler) RxIntr_Handler, AxiDmaPtr); 201 if (Status != XS
T_SUCCESS) { 202 return Status; 203 } 204 205 XScuGic_Enable(IntcInstancePtr, TxIntrId); 206 XScuGic_Enable(
IntcInstancePtr, RxIntrId); 207 208 Xil_ExceptionInit(); 209 IntcInstancePtr; 212 Xil_ExceptionEnable(); 213 214 re
turn XST_SUCCESS; 215 } 216
```

```
217 /*关闭 DMA 中断*/
218 static void DisableIntr_System(XScuGic * IntcInstancePtr, u16 TxIntrId,
219                               u16 RxIntrId)
220 {
221     XScuGic_Disconnect(IntcInstancePtr, TxIntrId);
222     XScuGic_Disconnect(IntcInstancePtr, RxIntrId);
223 }
```

Lines 117-169 of the code are the interrupt processing functions of DMA transfer, which are the sending interrupt processing function and the receiving interrupt processing function. When an interrupt occurs, `XAxiDma_IntrGetIrq()` will be used to obtain the current interrupt, and `XAxiDma_IntrAckIrq()` will be used to confirm the processing of the interrupt, and check whether an error occurs. If so, If an error occurs, the `XAxiDma_Reset()` function will reset the AXI DMA and wait to ensure that the AXI DMA reset is completed. Then determine whether the current interrupt is a transmission completion interrupt, and if so, set `Tx_Done` or `Rx_Done` to 1.

Lines 172-215 are the creation of the AXI DMA interrupt system. Start by calling the interrupt controller, and then set the interrupt priority and trigger type on lines 189 and 192. Then associate the interrupt service functions `TxIntr_Handler` and `RxIntr_Handler`, and finally register the interrupt exception service function and enable interrupts.

Lines 218-223, turn off DMA interrupt.

After the writing is completed, compile the project and prepare for board verification.

17.3.4 Board verification

As shown in Figure 17-14, the development board is connected. After the lines are connected correctly, turn on the

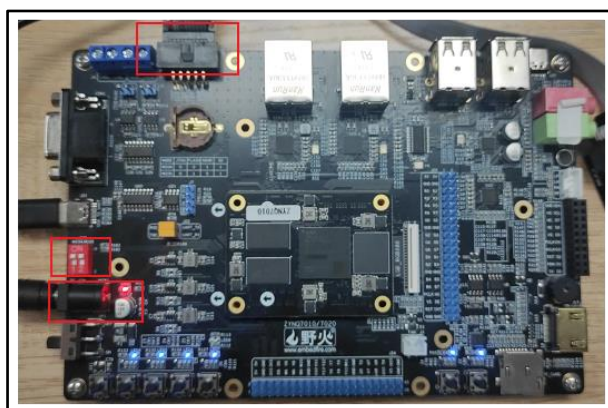


Figure 17-14 Program download connection diagram

As shown in Figure 17-15, download the program for the development board.

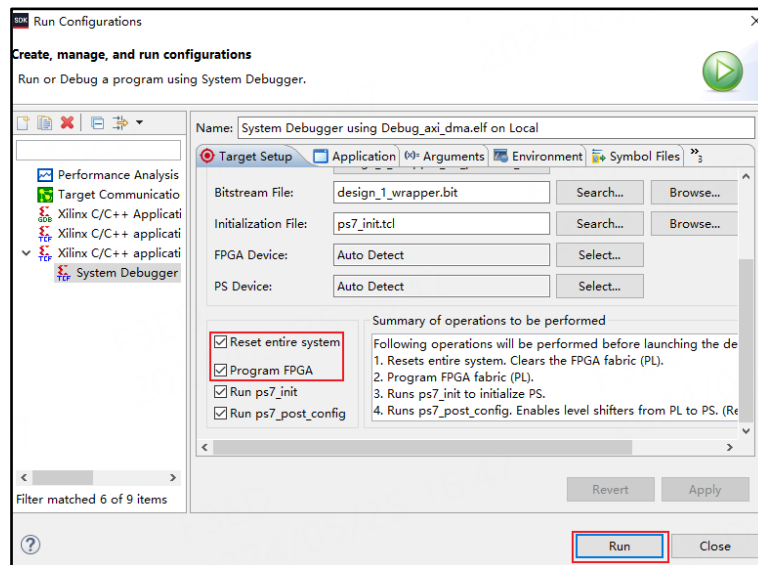


Figure 17-15 Program download diagram

After the program runs, the serial port prints out the following data, and AXI DMA successfully reads and writes.

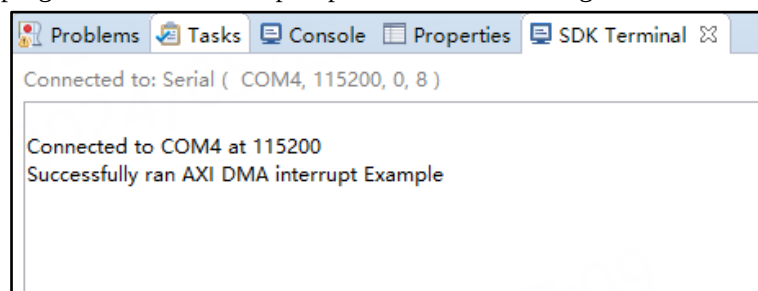


Figure 17-16 Output results

17.4 End of chapter summary

In this chapter of study, we have an in-depth exploration of the related knowledge of DMA. DMA is an efficient data transfer technology that can achieve high-speed data transfer between peripherals and memory without occupying CPU resources. This technology has a wide range of applications, especially in scenarios where large amounts of data need to be processed. Mastering DMA technology will help us optimize system performance.

17.5 Development training

You can try to let AXI DMA handle larger data, such as 2048 bytes.

Chapter 18 VDMA driving LCD display

18.1 Chapter Introduction

In the previous chapter, we learned that PS and PL directly realize the interaction of large amounts of data between PL and DDR through DMA. In this chapter, we mainly explain the application of AXI VDMA IP. It is a high-bandwidth solution provided by Xilinx specifically for video applications. It is designed to achieve high-bandwidth data transmission between the AXI-Stream video interface and the AXI-4 interface, and can easily implement double buffering and multi-buffering mechanisms.

This chapter mainly transmits the data to be displayed by the PS to the LCD interface of the PL side through VDMA.

18.2 Theoretical study

Our ZYNQ board has an LCD interface. The PL side tutorial also introduces how to implement LCD display, but how to use the PS side to control the LCD display is the focus of this chapter.

Since PS does not have an integrated display control system, it still needs to be implemented with the help of PL. How to transmit data to the display part of the PL side? Here you need to use the data interaction interface between PS and PL. Usually, the high-performance HP port is selected for high-speed data transmission between PS and PL. The maximum data width of the HP port is 64 bits. If the frequency reaches 150MHz, the bandwidth can reach 9.6Gbps, which can meet the transmission of image data.

There are many solutions to realize data transmission, but they are all inseparable from the DMA system. The DMA system can read the display data from ddr3 to the display and reduce the CPU overhead. VDMA is a special DMA developed by xilinx. It is specially used for video input and output. It is an important part of learning xilinx FPGA video processing.

We know that in order to play the video smoothly, one or several frames of data will usually be buffered first, and VDMA is mainly used for video stream access, so it can easily implement double buffering and multi-buffering mechanisms. In the VDMA IP core, the multi-buffering mechanism of the frame buffer can be implemented by configuring the genlock mode of VDMA.

18.2.1 Introduction to VDMA

The VDMA data interface is divided into read channel and write channel. Users can use the write channel to convert AXI-Stream type data stream into Memory Map format and write it to DDR3. Through the read channel, users can read data from DDR3 and convert Memory Map format data to AXI-Stream data stream output. It can be seen that VDMA is essentially a data handling IP, designed to provide video read and write transmission functions from the AXI4 domain to the AXI4-Stream domain, providing a convenient solution for data in and out of DDR3.

After the data is stored in DDR, the CPU can perform some processing (zooming, cropping, etc.) and then send it to the display device to achieve the desired application purpose. Of course, it is also possible to simply parse the captured video and store the data in the frame buffer for display.

VDMA can control up to 32 frame memories and can switch frame memories freely, so double buffering and multi-buffering operations can be easily implemented. This is also a very important feature. In subsequent system design, multi-buffering is usually used to implement display.

The key features & advantages of VDMA are as follows:

- (1) Enable video streams to be directly accessed into memory with high bandwidth.

- (2) Efficient 2D DMA operations.
- (3) Independent asynchronous read and write data frame operation.
- (4) Gen-Lock frame synchronization mechanism.
- (5) Supports up to 32 frame memories.
- (6) Support dynamic switching of video formats.
- (7) Burst length and line cache depth are adjustable.
- (8) The processor can control the IP's initialization, status, interrupt and management registers.
- (9) The basic AXI stream data bit width is an integer multiple of 8, such as 8, 16, 24, 32, etc., up to 1024.

The block diagram of VDMA is shown in Figure 18-1:

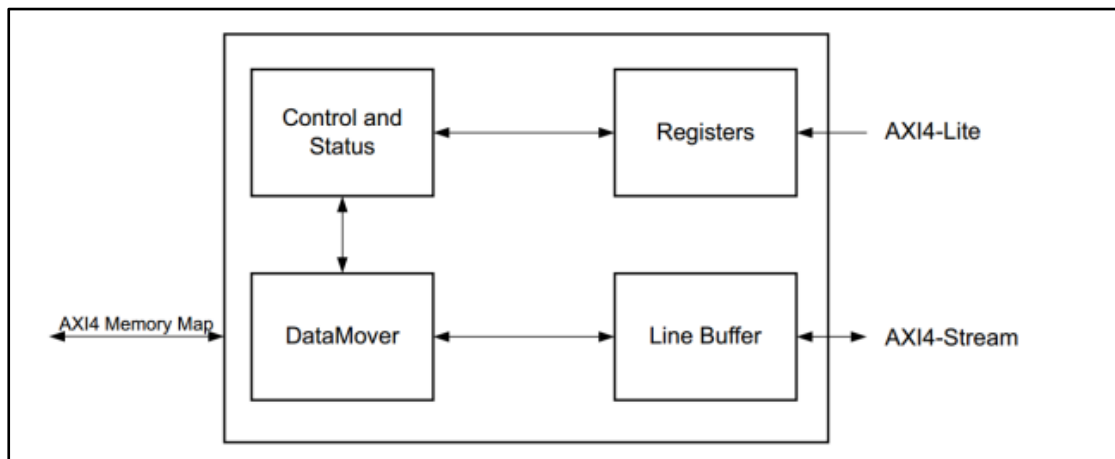


Figure 18-1 VDMA block diagram

The AXI4-Lite interface can be used to program registers to enable dynamic software configuration of the VDMA. After the registers are programmed through the AXI4-Lite interface, the control/status logic block generates the appropriate commands for the DataMover data handling module to initiate write and read operations on the AXI4 master interface. The configurable asynchronous line buffer module for temporarily saving pixel data temporarily saves pixel data before writing it to the AXI4-Memory Map interface or AXI4-Stream interface.

The VDMA IP core not only provides frame buffering functionality, but also integrates video-specific functions such as Gen-Lock and Frame Sync for fully synchronous frame DMA operations and 2D DMA transfers. In addition to synchronization capabilities, VDMA supports frame store numbering and scatter gather or register direct mode operation for central processor control. In this design, the scatter gather function of VDMA is not used because the system requirements can be implemented through the simple register direct mode of VDMA, thus avoiding the additional area cost caused by the scatter gather function. The scatter gather feature should only be enabled if the system requires complex software control of VDMA.

18.2.2 Frame buffer

Frame Buffer: referred to as frame buffer or video memory, it is a direct image of the picture displayed on the screen, also known as bit map (Bit Map) or raster. It is a storage area that provides data cache for display devices (such as HDMI monitors, LCD screens, etc.). Each storage unit of the frame buffer corresponds to a pixel on the screen, and the entire frame buffer corresponds to a frame of image.

Usually, the transmission rates of the image input source and the image display do not match (for example, the transmission speed of the image input source is faster or the transmission speed of the image display end is faster). In this case, a storage area is needed to cache the input data so that the display device can read the data and facilitate subsequent image processing of the video data.

When using frame buffer memory to cache image data, you can choose a single frame buffer or a multi-frame buffer scheme. Single frame buffer means that the input and display of images are completed by reading and writing the same storage area. Although this method can realize the image display function of the display device, there is a problem: as the input data source is continuously written, the single frame image read may be the superposition result of multiple frames of input image data. This condition may cause tearing in the image displayed by the display device.

In this RGB LCD screen color bar display experiment, since the color bar data only needs to be written once and only needs to be read after the writing is completed, the single frame cache solution can meet the needs. However, in subsequent learning camera image display experiments, due to the need for frequent writing and reading of image data, a VDMA multi-frame cache solution needs to be used at this time.

18.2.3 Genlock

In many video applications, the data rates at the image input and output often do not match. In order to avoid display problems caused by this rate mismatch, multiple frame buffers are usually allocated to avoid the image tearing problem that a single frame buffer may cause. In this multi-frame buffer scheme, while the image input end writes to one frame buffer, the output end reads data from another frame buffer. In the VDMA IP core, multi-frame buffering is implemented through synchronous phase locking.

The VDMA IP core has four genlock modes: Genlock Master (genlock master mode), Genlock Slave (genlock slave mode), Dynamic Genlock Master (dynamic genlock master mode) and Dynamic Genlock Slave (dynamic genlock slave mode).

VDMA has a write channel (S2MM) and a read channel (MM2S). Through the write channel, the data at the input end can be written to the frame buffer; through the read channel, the data can be read from the frame buffer. Each channel can select one of four synchronization modes. Next, we will introduce these four synchronization modes one by one.

Genlock Master

When the write channel (S2MM) or read channel (MM2S) is configured as Genlock Master, the channel will not skip or repeat any frame buffer area, but will read data in frame buffer order. In this mode, the current frame number will be output to the `mm2s_frame_ptr_out` port, and the frame number provided by the `mm2s_frame_ptr_in` port will not be detected. The channel configured as Genlock Slave should closely follow the changes of the Genlock Master channel, but with a certain delay. The size of the delay is configured in the register (`*frmdly_stride[28:24]`).

Genlock Slave

When the write channel (S2MM) or read channel (MM2S) is configured as a Genlock Slave, the channel will try to synchronize with the Genlock Master channel by skipping or repeating some frame buffer areas. The channel will sample the `mm2s_frame_ptr_in` port to obtain the frame number of the Genlock Master. In order to implement status feedback, the channel will output the current frame number to the `mm2s_frame_ptr_out` port.

Dynamic Genlock Master

When the write channel (S2MM) or read channel (MM2S) is configured as Dynamic Genlock Master, the channel will skip the frame buffer that the Dynamic Genlock Slave channel is operating and access the area where the next frame buffer of the frame that the Slave channel is operating is located.

Dynamic Genlock Slave

When the write channel (S2MM) or read channel (MM2S) is configured as a Dynamic Genlock Slave, the channel will operate the frame operated by the Dynamic Genlock Master channel in the previous cycle.

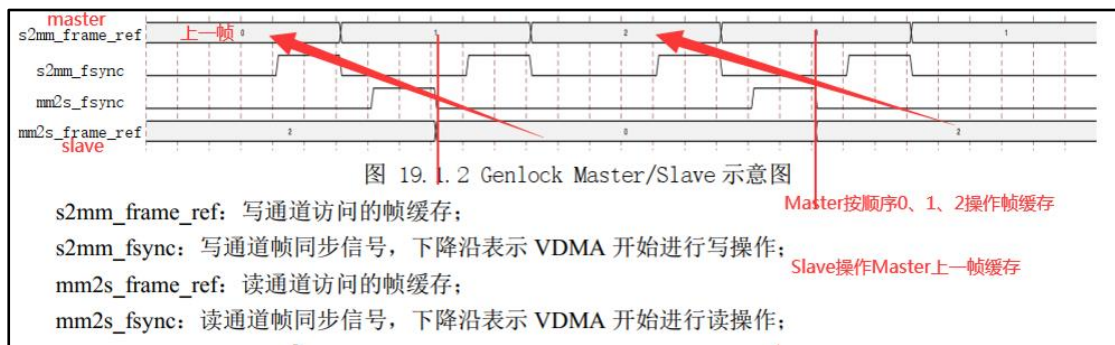


Figure 18-2 Genlock Master/Slave frame access diagram

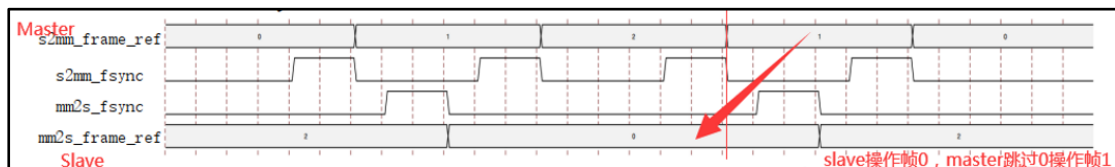


Figure 18-3 Dynamic Genlock Master/Slave frame access diagram

It can be seen from Figure 18-2 and Figure 18-3 that in order to prevent the same frame buffer from being accessed by the read channel and the write channel at the same time, the VDMA must be configured in dynamic genlock mode, and the number of frame buffers needs to be no less than 3. However, because too many frame buffers have minimal improvement in efficiency and will occupy more storage space and consume more CPU time, in camera image display applications, the number of frame buffers is usually set to 3 and dynamic genlock mode is used.

18.3 Practical exercises

18.3.1 Experimental objectives

The PS side writes the color bar data into DDR3, and then VMDA transfers the color bar data in DDR3 to the AXI4-Lite interface, and the final output is displayed on the screen.

18.3.2 Hardware resources

This experiment uses the LCD interface display on the PL side.

18.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 18-4.

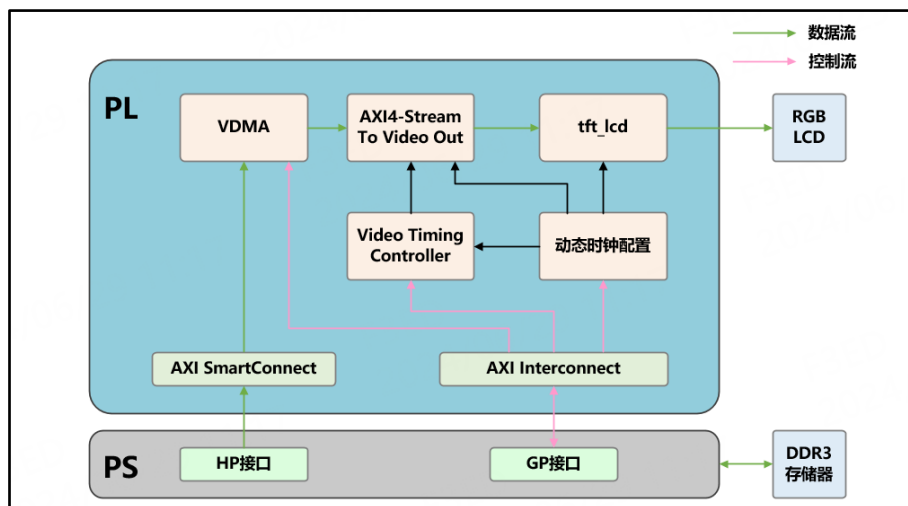


Figure 18-4 Overall block diagram

In practical applications, the VDMA IP core provides high-bandwidth direct memory access between memory and AXI4-Stream class target peripherals, and does not itself generate RGB LCD interface timing. Therefore, in addition to adding the VDMA IP core, this experiment also needs to add an IP core for generating RGB LCD interface timing. ZYNQ provides the AXI4-Stream to Video Out IP core, which can convert the AXI4-Stream data stream output by VDMA into the data stream of the video protocol (including parallel data, video synchronization signals, etc.). The converted data stream complies with the timing of the RGB LCD interface. In addition, after adding the AXI4-Stream to Video Out IP core, you also need to add the Video Timing Controller (VTC) IP core. This IP core provides a bridge for the AXI4-Stream interface and the video output interface to control the timing parameters of the video output. The dynamic clock configuration module can dynamically provide different clock frequencies, thereby realizing the display of different image resolutions and frame rates on the LCD.

Let's look at the pink control flow first. PS uses a GP interface to configure the VDMA IP core, Video Timing Controller IP core and dynamic clock, so that these modules work in a predetermined state.

Looking at the green data flow again, this time the color bar data is generated by the PS side, and the generated data is written into the system memory DDR3 memory; VDMA is connected to the AXI_HP port through the AXI Smartconnect IP core, thereby efficiently accessing DDR3. Afterwards, VDMA transmits the data obtained from the DDR to the AXI4-Stream to Video Out IP core. The AXI4-Stream to Video Out IP core, under the control of the VTC IP core, converts the data in the AXI4-Stream format into the video output data format (such as RGB888), and connects the output video data stream to the input end of the tft_lcd IP core. The tft_lcd IP core is a customized IP core for this experiment.

2. Hardware design

Detailed steps for project creation can be found in the previous chapters. Below we only list the configuration process.

ZYNQ system ip

The configuration of the ZYNQ system IP in the "AXI DMA Read and Write Test" chapter is the same, as shown in

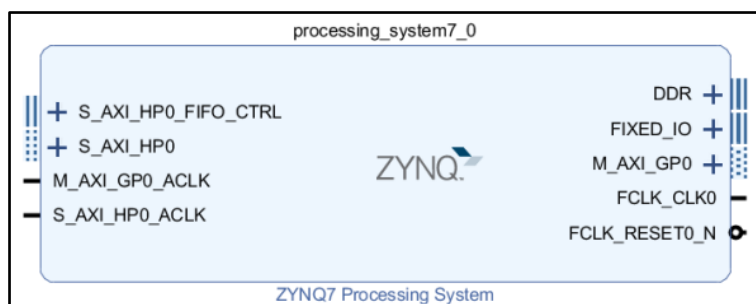


Figure 18-5 ZYNQ7 system module

VDMAIP

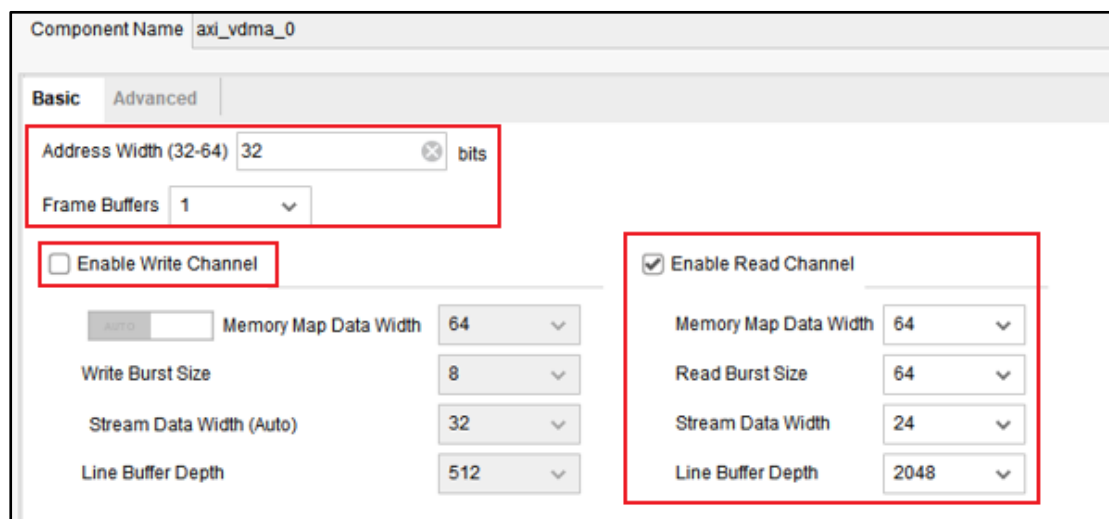


Figure 18-6 AXI VDMA configuration interface

Double-click to open the VDMA configuration interface. In the Basic interface, keep the Address Width at the default 32. The Frame Buffers option selects the number of frame buffers to be processed by AXI VDMA. For address spaces larger than 32 bits, up to 8 frame buffers are allowed. Since for this color bar display experiment, the data only needs to be written once, there is no need to set multiple frame buffer areas. Here we set it to 1. Because this experiment is to read data from DDR3 and output it to the LCD, you only need to check Enable Read Channel, and there is no need to check Enable Write Channel.

The Memory Map Data Width option selects the desired AXI4 data width for the MM2S channel. Valid values are 32, 64, 128, 256, 512, and 1024. Just keep the default 64 here.

Read Burst Size is used to specify the size of the burst read, here select 64.

The Stream Data Width option selects the AXI4-Stream data width of the MM2S channel. Valid values are multiples of 8, up to 1024. It must be noted that this value must be less than or equal to Memory Map Data Width. Here, because the output data format is RGB888, it is set to 24.

The Line Buffer Depth option can select the line buffer depth of the MM2S channel (the width of the line buffer is the size of the stream data), and it can be set to 2048 here.

The Advanced interface is used to configure the genlock mode of the read and write channels . Since this experiment only has one frame buffer and only the read channel is enabled, the genlock mode does not need to be configured. Just keep the default.

AXI4-Stream to Video Out IP

Component Name: v_axi4s_vid_out_0

Pixels Per Clock: 1

Video Format: RGB

AXI4S Video Input Component Width: 8

Native Video Output Component Width: 8

FIFO Depth: 1024

Clock Mode: ☐ Common ☒ Independent

Timing Mode: ☒ Slave ☐ Master

Hysteresis Level: 12 [0 - 1023]

Figure 18-7 Add AXI4-Stream to Video Out ip core

Configure the AXI streaming video output controller parameters, and select Independent for Clock Mode, which means that the clocks of the AXI4-Stream input stream and Video output stream are independent and asynchronous, while common is synchronous. In this experiment the two are asynchronous. The Slave mode of Timing Mode means that VTC is a timing Slave, and the Video Out module controls the timing output through clock enable. That is, at this time, the output vtg_ce of AXI4-Stream to Video Out should be connected to the input gen_clken of VTC. Master Mode means that VTC is the timing master and is not controlled by Video Out. Refer to module user manual pg044 for details.

Video Timing Controller IP

Configure the parameters of the video timing controller (VTC for short) and check Include AXI4-Lite Interface. This option will use the AXI4-Lite interface, which can dynamically access the program and change the processing parameters, which means that the Video Timing Controller can be dynamically configured. Enable Generation is the enable that generates output timing. After selection, the vtiming_out bus signal will appear. Enable Detetion is used to detect input timing signals. If enabled, the vtiming_in bus will appear. Since this experiment is image output, it is not enabled. Just keep other options as default.

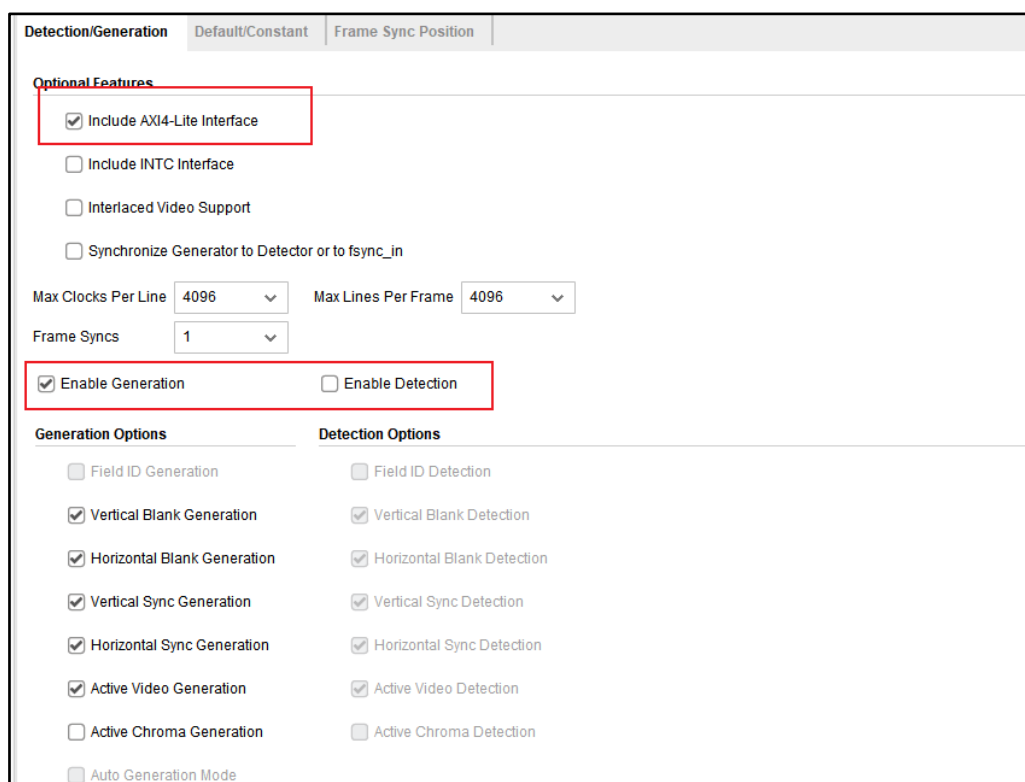


Figure 18-8 Video Timing Controller IP

Concat IP Core

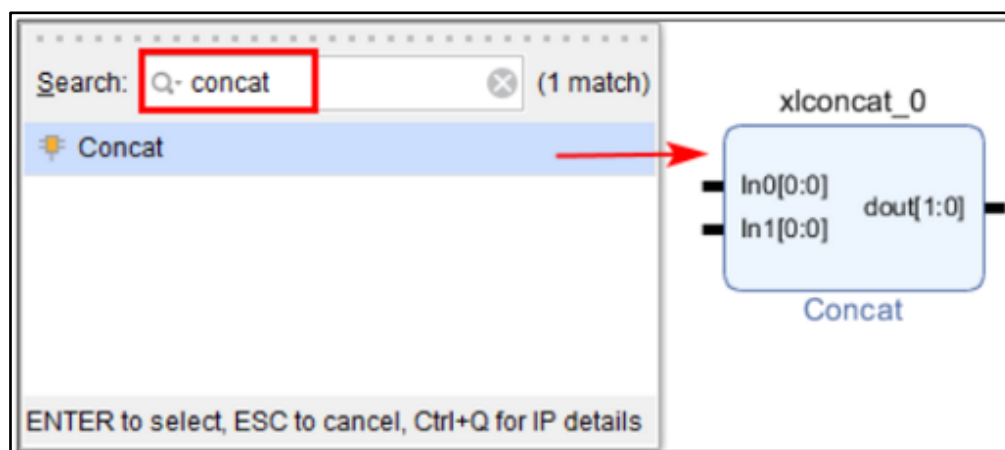


Figure 18-9 Add concat ip core

This IP core is used to integrate multiple scattered signals into bus signals.

Add custom tft_lcd IP core

Here we need to customize an IP core to implement LCD display. For specific principles, please refer to the PL-side LCD display tutorial. For the process of generating the entire IP, please refer to the "Customized IP Core" chapter. Only the instantiated code is given here.

Code Listing 18-1 tft_lcd module reference code (tft_lcd.v)

```

1 module tft_lcd #(
2     //parameter define
3     parameter    VID_IN_DATA_WIDTH = 24,
4     parameter    REDDEPTH    = 8,

```

5 parameter GREENDEPTH = 8, 6 parameter BULEDEPTH = 8, 7 parameter VID_OUT_DATA_WIDTH = REDDEPTH + GREENDEPTH

```
8 BULEDEPTH 9)( 10 //VID_OUT 11 input [VID_IN_DATA_WIDTH-1:0] rgb_data ,
12 input rgb_de , 13 input rgb_hsync, 14 input rgb_vsync, 15 16 input pixel_clk, 17 input vid_rst , 18 19 //RGB LCD 20 output lcd_pclk , 21 output lcd_rst , 22 output reg lcd_hs , 23 output reg lcd_vs , 24 output reg lcd_de , 25 output lcd_bl , 26 27 28 output [VID_OUT_DATA_WIDTH-1:0] rgb_tft 29 ); 30 31 //reg define 32 reg [VID_IN_DATA_WIDTH-1:0] int_pData ; 33 reg rd_id_flag=1'b0; 34 35 //wire define 36 wire [REDDEPTH-1:0] lcd_red ; 37 wire [GREENDEPTH-1:0] lcd_green ; 38 wire [BULEDEPTH-1:0] lcd_blue ; 39 40 //***** 41 //
** main code 42 //*****
** 43 44 //LCD data assignment 45 always @(posedge pixel_clk or negedge vid_rst) begin 46 if(!vid_rst) 47 int_pData <= 0; 48 else if (rgb_de == 1'b1) 49 int_pData <= rgb_data; 50 else 51 int_pData <= {VID_IN_DATA_WIDTH{1'b0}}; 52 end 53 54 //LCD control signal assignment 55 always @(posedge pixel_clk or negedge vid_rst) begin 56 if(!vid_rst) begin 57 lcd_de <= 1'b0; 58 lcd_hs <= 1'b0; 59 lcd_vs <= 1'b0; 60 end 61 else begin 62 lcd_de <= rgb_de; 63 lcd_hs <= rgb_hsync; lcd_vs <= rgb_vsync; 65 end 66 end 67 68 69 assign lcd_pclk = pixel_clk; 70 assign lcd_rst = vid_rst;
```

```

71 assign lcd_bl = 1'b1; 72 assign lcd_red = int_pData[VID_IN_DATA_WIDTH - 1:VID_IN_DATA_WI
DTH -
73 REDDEPTH]; 74 assign lcd_green = int_pData[VID_IN_DATA_WIDTH/3 * 2 - 1: 75
VID_IN_DATA_WIDTH/3 * 2 - GREENDEPTH]; 76 assign lcd_blue = int_pData[VID_I
N_DATA_WIDTH/3 - 1: 77 VID_IN_DATA_WIDTH/3 - BLUEDEPTH]; 78 79 assign rg
b_tft = {lcd_red,lcd_green,lcd_blue}; 80 81 endmodule

```

LCD screens with different resolutions have different driving clock frequencies, so a dynamic clock controller needs to be used to control the clock output. The axi_dynclk IP from open source software can realize this function. We put these two IPs in the ip_repo directory of the routine (lcd_show). You can copy the ip_repo folder to your own project directory, as shown in the figure below:



Figure 18-10 Customized IP core storage directory

Then open Settings and add a custom IP library as shown in Figure 18-11.

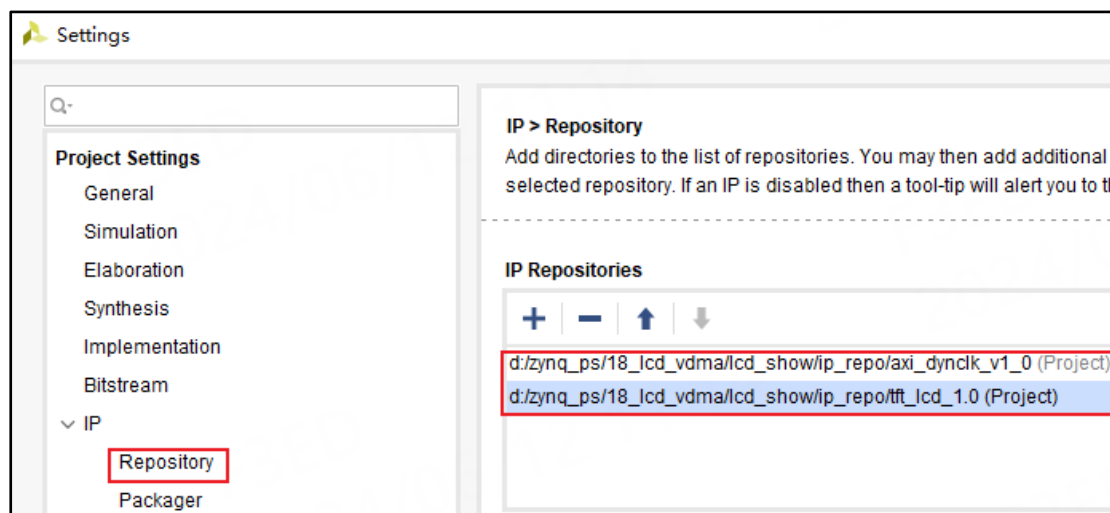


Figure 18-11 Adding a custom IP library

Then add tft_lcd IP and Dynamic Clock Generator IP (dynamic clock controller) in the Diagram interface, and then double-click the tft_lcd IP core to enter the configuration interface.



The diagram illustrates the ZYNQ7 Processing System architecture. Key components and their interconnections include:

- processing_system7_0**: The central ZYNQ7 Processing System, featuring the ZYNQ logo and various control signals like `S_AXI_HP0_FIFO_CTRL`, `S_AXI_HP0`, `M_AXI_GPO_ACLK`, `S_AXI_HP0_ACLK`, `IRQ_F2P[0:0]`, `DDR`, `FIXED_IO`, `M_AXI_GPO`, `FCLK_CLK0`, and `FCLK_RESET0_N`.
- axi_dynclk_0**: Dynamic Clock Generator (Pre-Production), which receives `s00_axi`, `REF_CLK_I`, `s00_axi_aclk`, and `s00_axi_aresetn` signals, and outputs `PXL_CLK_O`, `PXL_CLK_5X_O`, and `LOCKED_O`.
- v_tc_0**: Video Timing Controller, which receives `ctrl`, `clk`, `clken`, `s_axi_aclk`, `s_axi_ackn`, `gen_clken`, `resetn`, `s_axi_aresetn`, and `fsync_in` signals, and outputs `vtimeing_out` and `fsync_out[0:0]`.
- axi_vdma_0**: AXI Video Direct Memory Access, which receives `S_AXI_LITE`, `s_axi_lite_ack`, `m_axi_mm2s_ack`, `m_axis_mm2s_ack`, `axi_resetn`, and `mm2s_frame_ptr_in[5:0]` signals, and outputs `M_AXI_MM2S`, `M_AXIS_MM2S`, `mm2s_frame_ptr_out[5:0]`, and `mm2s_introut`.
- v_axi4s_vid_out_0**: AXI4-Stream to Video Out, which receives `video_in`, `vtimeing_in`, `ack`, `ackn`, `aresetn`, `fid`, `vid_io_out_cik`, `vid_io_out_ce`, and `vid_io_out_reset` signals, and outputs `vid_io_out`, `vlg_ce`, `locked`, `overflow`, `underflow`, `fifo_read_level[10:0]`, and `status[31:0]`.
- xlconcat_0**: Concatenation block, which receives `in0[0:0]` and `in1[0:0]` signals, and outputs `out[1:0]`.
- ttl_lcd_0**: TTL LCD controller, which receives `S00_AXI`, `rgb_data[23:0]`, `rgb_de`, `rgb_hsync`, `rgb_vsync`, `pixel_clk`, `vid_rst`, `s00_axi_aclk`, and `s00_axi_aresetn` signals, and outputs `ttl_lcd_v1.0` (Pre-Production).

Figure 18-13 Manual connection (1)

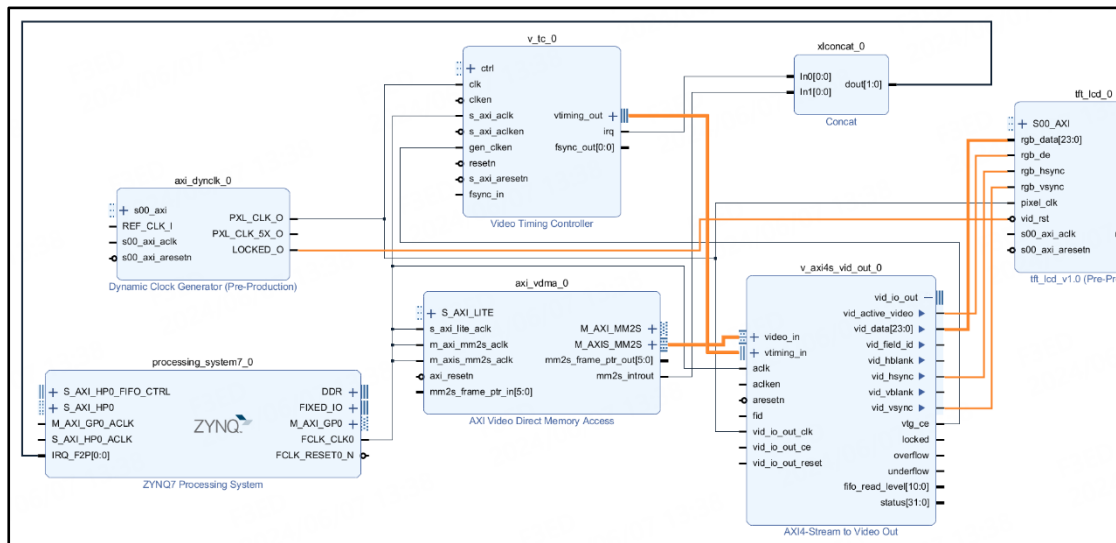


Figure 18-14 Manual connection (2)

Then it will connect automatically. If there is a Run Block Automation prompt in the Diagram, click Automatically configure ZYNQ IP. If not, ignore it. Then click "Run connection Automation" to connect automatically, check "All Automation" in the pop-up interface, and then click OK. We will also need to bring out the port pins later.

The final overall IP module connection diagram is shown in Figure 18-15:

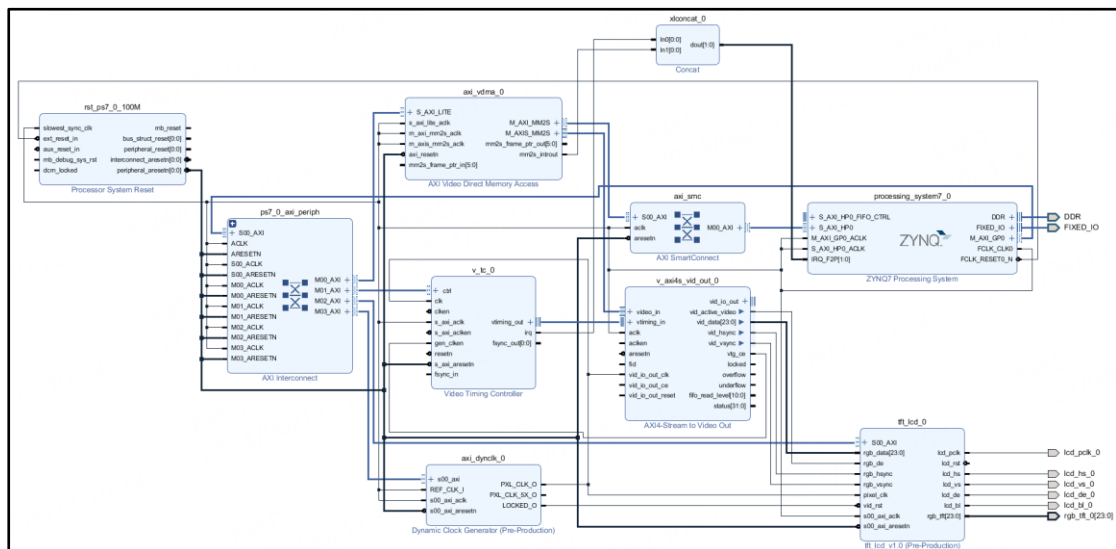


Figure 18-15 Overall IP block diagram

At this point our Block Design is completed. Press the "F6" key to perform "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally, press the shortcut key "Ctrl + S" to save the design.

Then execute "Generate Output Products" and "Create HDL Wrapper" in sequence. Then perform pin constraints on the used pins.

Code Listing 18-2 Pin Constraints (vdma_lcd.xdc)

```
1 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[23]}]
2 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[22]}]
3 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[21]}]
```

```

4 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[20]}]
5 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[19]}]
6 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[18]}]
7 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[17]}]
8 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[16]}]
9 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[15]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[14]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[13]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[12]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[11]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[10]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[9]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[8]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[7]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[6]}]
19 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[4]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[3]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[2]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[1]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {rgb_tft_0[0]}]
25 set_property PACKAGE_PIN T15 [get_ports {rgb_tft_0[23]}]
26 set_property PACKAGE_PIN T14 [get_ports {rgb_tft_0[22]}]
27 set_property PACKAGE_PIN V13 [get_ports {rgb_tft_0[21]}]
28 set_property PACKAGE_PIN U13 [get_ports {rgb_tft_0[20]}]
29 set_property PACKAGE_PIN U12 [get_ports {rgb_tft_0[19]}]
30 set_property PACKAGE_PIN T12 [get_ports {rgb_tft_0[18]}]
31 set_property PACKAGE_PIN T10 [get_ports {rgb_tft_0[17]}]
32 set_property PACKAGE_PIN T11 [get_ports {rgb_tft_0[16]}]
33 set_property PACKAGE_PIN R14 [get_ports {rgb_tft_0[15]}]
34 set_property PACKAGE_PIN P14 [get_ports {rgb_tft_0[14]}]
35 set_property PACKAGE_PIN P16 [get_ports {rgb_tft_0[13]}]
36 set_property PACKAGE_PIN P15 [get_ports {rgb_tft_0[12]}]
37 set_property PACKAGE_PIN U17 [get_ports {rgb_tft_0[11]}]
38 set_property PACKAGE_PIN T16 [get_ports {rgb_tft_0[10]}]
39 set_property PACKAGE_PIN V18 [get_ports {rgb_tft_0[9]}]
40 set_property PACKAGE_PIN V17 [get_ports {rgb_tft_0[8]}]
41 set_property PACKAGE_PIN R17 [get_ports {rgb_tft_0[7]}]
42 set_property PACKAGE_PIN R16 [get_ports {rgb_tft_0[6]}]
43 set_property PACKAGE_PIN U19 [get_ports {rgb_tft_0[5]}]
44 set_property PACKAGE_PIN U18 [get_ports {rgb_tft_0[4]}]
45 set_property PACKAGE_PIN W19 [get_ports {rgb_tft_0[3]}]
46 set_property PACKAGE_PIN W18 [get_ports {rgb_tft_0[2]}]
47 set_property PACKAGE_PIN W16 [get_ports {rgb_tft_0[1]}]
48 set_property PACKAGE_PIN V16 [get_ports {rgb_tft_0[0]}]
49 set_property PACKAGE_PIN T19 [get_ports lcd_b1_0]
50 set_property PACKAGE_PIN V15 [get_ports lcd_de_0]
51 set_property PACKAGE_PIN V12 [get_ports lcd_hs_0]
52 set_property PACKAGE_PIN W13 [get_ports lcd_vs_0]
53 set_property PACKAGE_PIN W15 [get_ports lcd_pclk_0]
54 set_property IOSTANDARD LVCMOS33 [get_ports lcd_b1_0]
55 set_property IOSTANDARD LVCMOS33 [get_ports lcd_de_0]
56 set_property IOSTANDARD LVCMOS33 [get_ports lcd_hs_0]
57 set_property IOSTANDARD LVCMOS33 [get_ports lcd_pclk_0]
58 set_property IOSTANDARD LVCMOS33 [get_ports lcd_vs_0]

```

Then generate a bit stream file and export the hardware (check "Include bitstream" in the pop-up dialog box). Finally launch the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and started writing code. Readers should copy the following files to their own projects when doing this experiment:

lcd_show > lcd_show.sdk > vdma_lcd > src >

名称	类型	大小
display_ctrl	文件夹	
dynclk	文件夹	
vdma_api	文件夹	
lscript.ld	LD 文件	7 KB
main.c	C 源文件	3 KB
README.txt	TXT 文件	1 KB

Figure 18-16 SDK related source code

Vdma_api stores related functions for configuring VDMA. This is an official routine provided by xilinx. It has also been modified here to support three functions: read-only, write-only, and read-write.

display_ctrl is also an open source file from Digilent, and dynclk is also an open source file from Digilent. It is used to control the dynamic clock configuration module to output different clocks to be compatible with more resolutions.

Code List 18-3 LCD display reference code (main.c)

```
1 #include <stdio.h> 2 #include <stdlib.h> 3 #include <string.h> 4 #include "sleep.h" 5 #include "xil_types.h" 6 #include "xil_cache.h" 7 #include "xparameters.h" 8 #include "xaxivdma.h" 9 #include "xaxivdma_i.h" 10 #include "vdma_api/vdma_api.h" 11 #include "display_ctrl/display_ctrl.h" 12 13 #define CLOCK_ADDR XPAR_AXI_DYNCLK_0_BASEADDR //Clock address 14 #define VDMA_DEVICE_ID XPAR_AXIVDMA_0_DEVICE_ID //VDMA device ID 15 #define XVTC_DEVICE_ID XPAR_VTC_0_DEVICE_ID //VTC device ID 16 17 void LCD_Show(u32 width, u32 height); 18 19 22 23 /*Buffer memory base address*/ 24 unsigned int const FB_Base = (XPAR_PS7_DDR_0_S_AXI_BASEADDR + 0x1000000); 25 26 int main(void) 27 { 28 VIDEO_Mode = VMODE_800x480; 29 30 run_vdma_frame_buffer( 31 &Vdma, 32 VDMA_DEVICE_ID, 33 VIDEO_Mode.width, 34 VIDEO_Mode.height, 35 FB_Base, 36 0, 37 0, 38 ONLY_READ); //Configure lcd parameters 39 40 //Initialize Display 41 DisplayInitialize(&Dispctrl, XVTC_DEVICE_ID, CLOCK_ADDR); 42 DisplaySetMode(&Dispctrl, &VIDEO_Mode); //Set lcd parameters
```

```
43 DisplayStart(&Dispctrl); //Turn on 44 45 /*LCD display color bar*/ 46 LCD_Show(VIDEO_Mode.wid
th,VIDEO_Mode.height); 47 return 0; 48 } 49 50 51 //Fill function 52 void LCD_Show(u32 width, u32 he
ight) 53 { 54 u32 count; 55 u8 *frame = (u8*)FB_Base; 56 57 for (count = 0; count < (width * height)*3;
count += 3) { 58 if (count >=0 && count < height*width) { 59 frame[count + 0] = 0x00; 60 frame[count
+ 1] = 0x00; 61 frame[count + 2] = 0xFF; 62 } 63 else if (count >= height*width && count < height*wid
th*2) { 64 frame[count + 0] = 0x00; 65 frame[count + 1] = 0xFF; 66 frame[count + 2] = 0x00; 67 } 68 el
se { 69 frame[count + 0] = 0xFF; 70 frame[count + 1] = 0x00; 71 frame[count + 2] = 0x00; 72 } 73 } 74
Xil_DCacheFlush(); //Update Cache 75 }
```

In lines 8-11 of the code we introduce several new header files. The vdma files and related functions refer to the official routines provided by xilinx and make corresponding modifications.

In lines 13-15 of code, we define the base address CLOCK_ADDR of the dynamic clock IP and the device base addresses of VDMA and VTC, which can be found in xparameters.h.

In line 24 of the code, we define the address FB_Base where the image data is stored in the memory. Generally, we add 0x1000000 to the base address of DDR3. This is not fixed and large enough.

In the main function, we use VIDEO_Mode to configure the resolution to be displayed. This is a custom structure that can be found in the lcd_mode.h header file, as shown below:

Code Listing 18-4 LCD configuration information (lcd_mode.h)

```
_1 typedef struct { 2 char label[64]; 3 u32 width; 4 u32 height; 5
u32 hps; 6 u32 hpe; 7 u32 hmax; 8 u32 hpol; 9 u32 vps; 15 16 stat
ic const VideoMode VMODE_480x272 = {
```

```
17 .label = "480x272@60Hz", 18 .width = 480, 19 .height = 272,
20 .hps = 482, 21 .hpe = 523, 22 .hmax = 525, 23 .hpol = 0, 24 .v
ps = 274, 25 .vpe = 284, 26 .vmax = 286, 27 .vpol = 0, 28 .freq =
9 29 }; 30 31 32 33 static const VideoMode VMODE_800x480 =
{ 34 .label = "800x480@60Hz", 35 .width = 800, 36 .height= 480
, 37 .hps = 840, 38 .hpe = 968, 39 .hmax = 1056, 40 .hpol = 0, 41
.vps = 490, 42 .vpe = 492, 43 .vmax = 525, 44 .vpol = 0, 45 .freq
= 33.0 46 };
```

Since all configurations are configured using variables, if readers want to modify the resolution, they only need to modify the value of VIDEO_Mode. Width represents the image width, and height represents the image height. The following hps, hpe, hmax, hpol, vps, vpe, vmax, vpol and freq can be found online, that is, VGA display timing or VESA timing can be found.

Then use the run_vdma_frame_buffer() function to configure the size, address and operation mode of VDMA to write to the memory. Read-only mode is used here because we only need to read the color bar data from the memory. The function run_vdma_frame_buffer() is actually a function officially provided by xilinx for configuring VDMA. It has been modified here to support read-only, write-only, and read-write. The author will not analyze it in detail here. The essence is to configure the registers of VDMA to implement the corresponding functions. Readers can study it in detail if they are interested.

Line 75 of code 52~ is a function that generates color bars. It directly replaces the value of the corresponding memory address with color bar data, so that the color bar data can be written into the memory. Since RGB888 is 3 bytes, the code is 57 lines, and our count needs +3 each time. At the end of the function, the data cached in the DataCache is flushed to DDR3 by calling the Xil_DCacheFlush() function so that VDMA can read it.

Compile the project and verify it on the board.

18.3.4 Board verification

As shown in Figure 18-17, the development board is connected to an LCD screen. This experiment uses a 5-inch 8

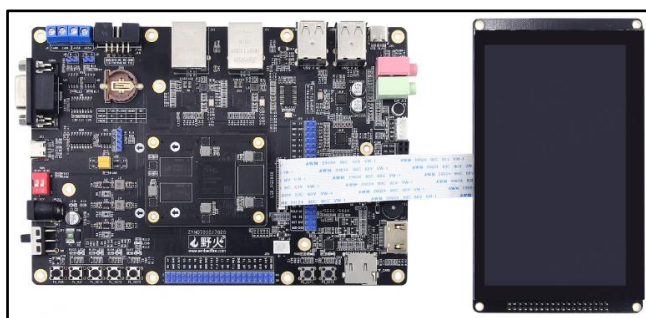


Figure 18-17 Connecting the LCD screen

Then connect the development board to the 12V DC power supply and the JTAG port of the XILINX downloader, and then turn on the power switch to power on.

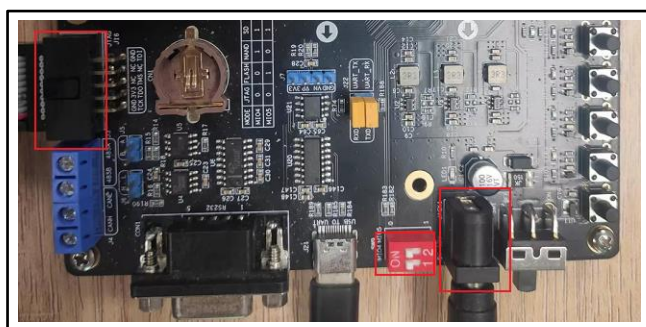


Figure 18-18 Program download connection diagram

As shown in Figure 18-19, download the program for the development board.

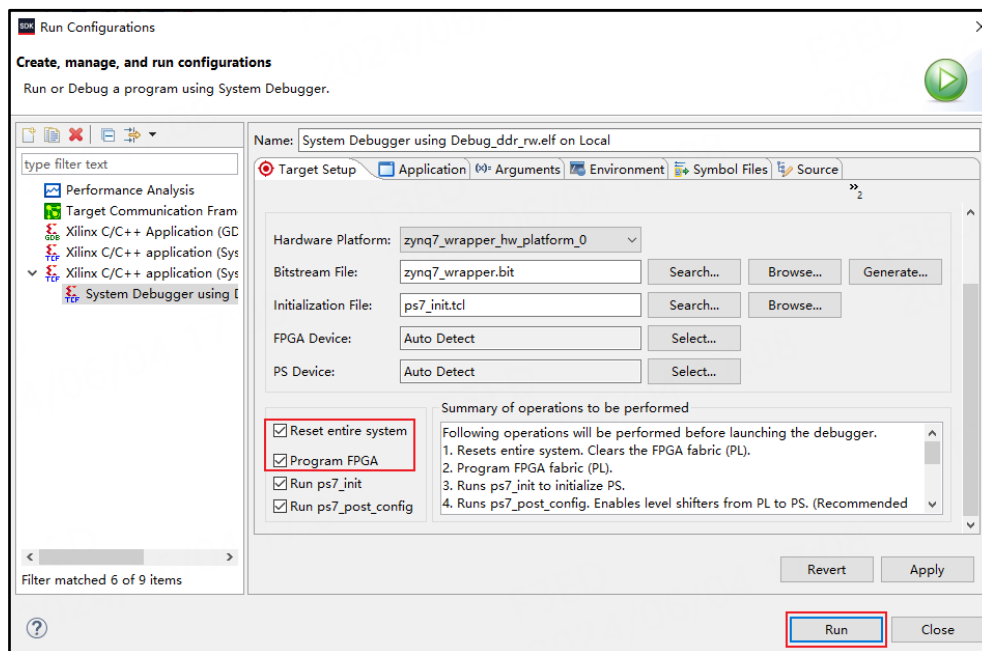


Figure 18-19 Program download diagram

After the program download is completed, color bars will be displayed on the LCD, as shown in Figure 18-20.



Figure 18-20 LCD color bar display

18.4 End of chapter summary

VDMA is an IP core dedicated to video input and output, which can achieve high-bandwidth data transmission between AXI-Stream and AXI4 interfaces. It provides multi-frame buffering and genlock mechanisms. The AXI4-Lite interface can be used to dynamically configure VDMA. VDMA transfers image data in DDR3 to the AXI-Stream interface.

In addition, we added an IP core with dynamic clock configuration to provide different clock frequencies for compatibility with more resolutions.

18.5 Development training

In this experiment, we divided the screen into three equal parts in the vertical direction, displaying red, green, and blue color bars respectively. You can try to make further improvements by dividing the screen into nine equal parts in a nine-square-grid format, displaying different colors respectively.

Chapter 19 VDMA drives HDMI display

19.1 Chapter Introduction

In the previous chapter, we learned about VDMA driving LCD display. This chapter mainly transmits the data to be displayed by the PS to the HDMI interface of the PL side through VDMA for display.

19.2 Theoretical study

The theoretical introduction to HDMI is explained in the PL side, so I won't go into too much detail.

19.3 Practical exercises

19.3.1 Experimental objectives

The PS side writes the color bar data into DDR3, and then VDMA transfers the color bar data in DDR3 to the AXI4-Lite interface, and the final output is displayed on the HDMI screen.

19.3.2 Hardware resources

This experiment uses the HDMI interface display on the PL side.

19.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 19-1.

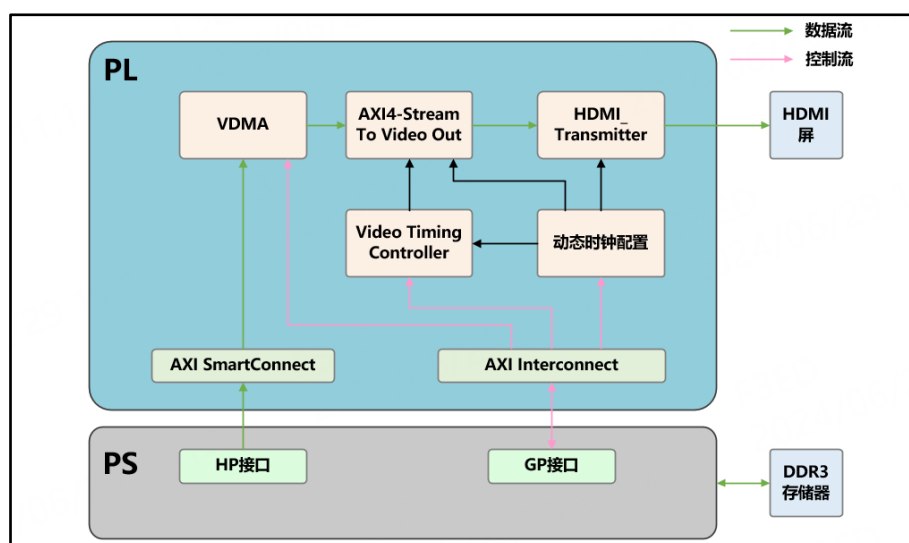


Figure 19-1 Overall block diagram

In practical applications, this block diagram is actually the same as the VDMA_LCD display, except that it is replaced by an HDMI display, and the tft_lcd IP core is replaced by an HDMI_Transmitter IP. There are no other changes.

2. Hardware design

Detailed steps for project creation can be found in the previous chapters. Below we only list the configuration process of the ZYNQ system IP.

The configuration of the ZYNQ system IP in the "AXI DMA Read and Write Test" chapter is the same, as shown in Figure 19-2.

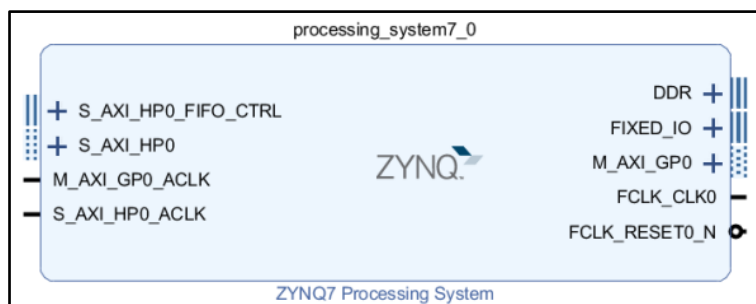


Figure 19-2 ZYNQ7 system module

Concat IP Core

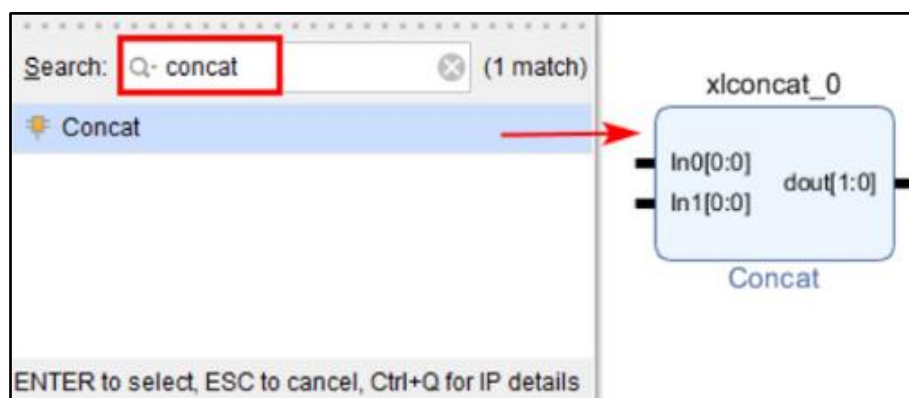


Figure 19-3 Add concat ip core

This IP core is used to integrate multiple scattered signals into bus signals.

Add custom HDMI_Transmitter IP core

Here we need to customize an IP core to implement HDMI display. The specific code is consistent with the PL side HDMI display. I will not repeat the description here.

HDMI displays with different resolutions have different driving clock frequencies, so a dynamic clock controller needs to be used to control the clock output. The axi_dynclk IP from open source software can implement this function. We put these two IPs in the ip_repo directory of the routine. You can copy the ip_repo folder to your own project directory, as shown in the following figure:



Figure 19-4 Customized IP core storage directory

Then open Settings and add a custom IP library as shown in Figure 19-5.



Figure 19-5 Adding a custom IP library

Then add HDMI_Transmitter IP and Dynamic Clock Generator IP (dynamic clock controller) in the Diagram interface, as shown in the following figure:

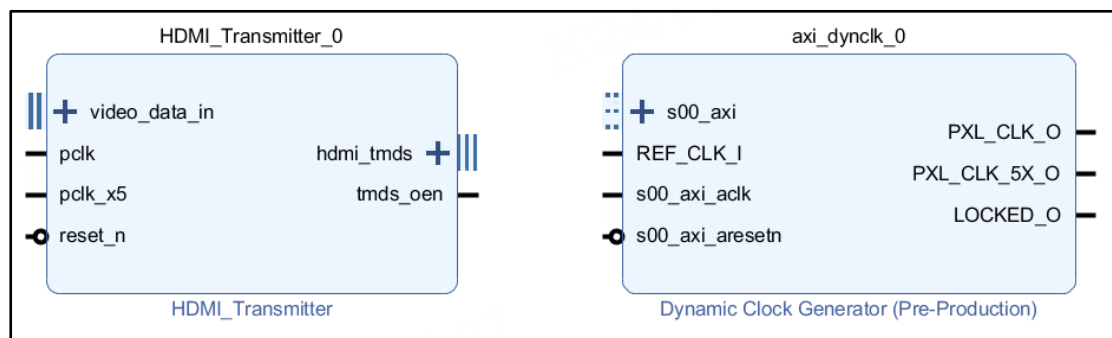


Figure 19-6 HDMI_Transmitter IP and Dynamic Clock Generator IP

The creation and configuration of all IP cores is completed here. Because Vivado's automatic connection function cannot completely connect according to our needs, we first manually connect some interfaces before automatic connection to avoid Vivado incorrect connection. The manual connection is as shown in the figure below:

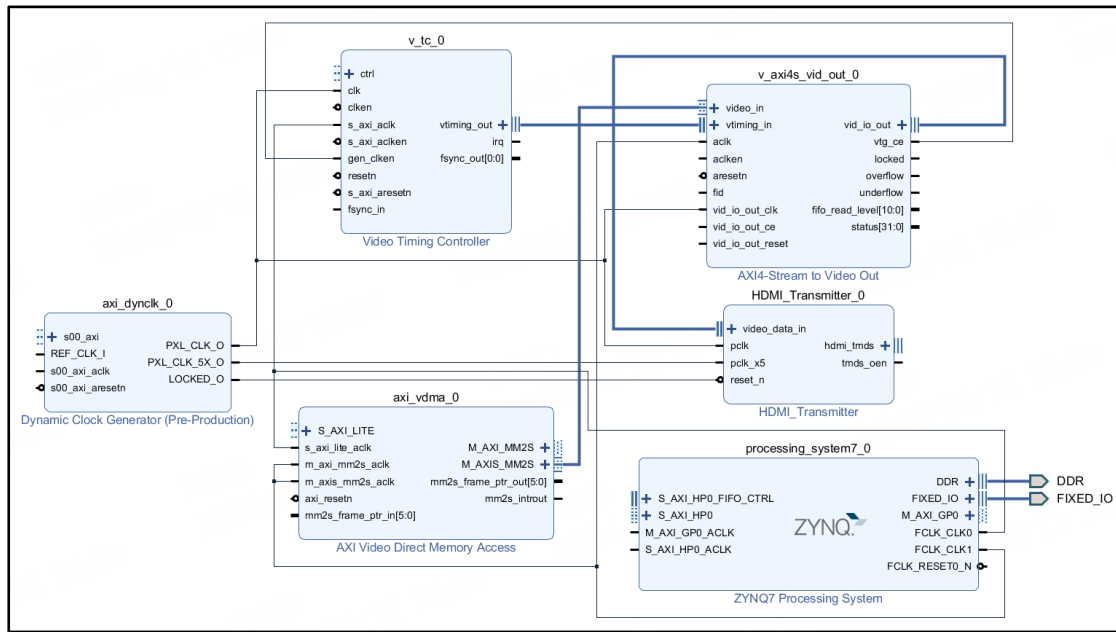


Figure 19-7 Manual connection

Then it will connect automatically. If there is a Run Block Automation prompt in the Diagram, click to automatically configure the ZYNQ IP. If not, ignore it. Then click "Run connection Automation" to connect automatically, check "All Automation" in the pop-up interface, and then click OK. We will also need to bring out the port pins later.

The final overall IP module connection diagram is shown in Figure 19-8:

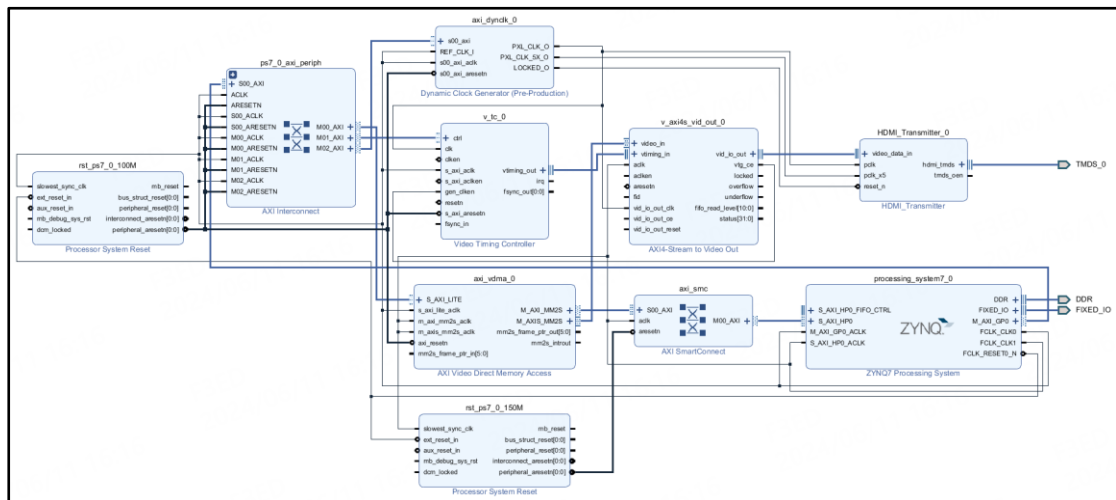


Figure 19-8 Overall IP block diagram

At this point our Block Design is completed. Press the "F6" key to perform "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally, press the shortcut key "Ctrl + S" to save the design.

Then execute "Generate Output Products" and "Create HDL Wrapper" in sequence. Then perform pin constraints on the used pins.

Code Listing 19-1 Pin Constraints (system_wrapper.xdc)

```
1 set_property PACKAGE_PIN Y18 [get_ports {TMD5_0_tmds_data_p[2]}]
```

```

2 set_property PACKAGE_PIN Y16 [get_ports {TMDS_0_tmds_data_p[1]}}]
3 set_property PACKAGE_PIN W14 [get_ports {TMDS_0_tmds_data_p[0]}}]
4 set_property PACKAGE_PIN V20 [get_ports TMDS_0_tmds_clk_p]

```

Then generate a bit stream file and export the hardware (check "Include bitstream" in the pop-up dialog box). Finally launch the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and When readers do this experiment themselves, they should copy the following files to their own project:



Figure 19-9 SDK related source code

vdma_api stores related functions for configuring VDMA. This is an official routine provided by xilinx. It has also been modified here to support three functions: read-only, write-only, and read-write.

display_ctrl is also an open source file from Digilent, and dynclk is also an open source file from Digilent. It is used to control the dynamic controller. axi_dynclk_ip is used to configure the output of different clocks to be compatible with more resolutions.

Code List 19-2 HDMI display color bar reference code (main.c)

```

1 #include <stdio.h> 2 #include <stdlib.h> 3 #include <string.h> 4 #include "xil_types.h" 5 #include "xil_cache.h"
6 #include "xparameters.h" 7 #include "xaxivdma.h" 8 #include "xaxivdma_i.h" 9 #include "vdma_api/vdma_api.h"
10 #include "display_ctrl/display_ctrl.h" 11 12 #define CLOCK_ADDR XPAR_AXI_DYNCLK_0_BASEADDR
//Clock address 13 #define VDMA_DEVICE_ID XPAR_AXIVDMA_0_DEVICE_ID //VDMA device ID 14 #define
XVTC_DEVICE_ID XAXiVdma Vdma; 23 DisplayCtrl Dispctrl; 24 VideoMode VIDEO_Mode; 25 26 /*buffer
memory base address*/ 27 unsigned int const FB_Base = (XPAR_PS7_DDR_0_S_AXI_BASEADDR + 0x100000
0); 28 29 int main(void)

```

```
30 { 31 printf("HDMI_COLORBAR TEST!\r\n"); 32 //lcd_mode.h can change the resolution 33 VIDEO_
Mode = VMODE_1024x600; 34 //Configure lcd parameters 35 run_vdma_frame_buffer(&Vdma, VDMA
_DEVICE_ID, VIDEO_Mode.width, 36 VIDEO_Mode.height, FB_Base,0, 0,ONLY_READ); 37 //Initiali
ze Display 38 DisplayInitialize(&Dispctrl, /*hdmi display color bar*/ 42 HDMI_Show(VIDEO_Mode.wi
dth,VIDEO_Mode.height); 43 return 0; 44 } 45 46 //Filling function 47 void HDMI_Show( u32 width, u3
2 height) 48 { 49 u32 count; 50 u8 *frame = (u8*)FB_Base; 51 52 for (count = 0; count < (width * height)
*3; count += 3) { 53 if (count >=0 && count < height*width) { 54 frame[count + 0] = 0x00; 55 frame[co
unt + 1] = 0x00; 56 frame[count + 2] = 0xFF; 57 } 58 else if (count >= height*width && count < height*
width*2) { 59 frame[count + 0] = 0x00; 60 frame[count + 1] = 0xFF; 61 frame[count + 2] = 0x00; 62 } 6
3 else { 64 frame[count + 0] = 0xFF; 65 frame[count + 1] = 0x00; 66 frame[count + 2] = 0x00; 67 } 68 }
69 Xil_DCacheFlush(); //Update Cache 70 }
```

In lines 1-10 of the code we introduce several new header files. The vdma files and related functions refer to the official routines provided by xilinx and make corresponding modifications.

In lines 12-14 of code, we define the base address CLOCK_ADDR of the dynamic clock IP and the device base addresses of VDMA and VTC, which can be found in xparameters.h.

In line 27 of the code, we define the address FB_Base where the image data is stored in the memory. Generally, we add 0x1000000 to the base address of DDR3. This is not fixed, it just needs to be large enough.

In the main function, we use VIDEO_Mode to configure the resolution to be displayed. This is a custom structure that can be found in the lcd_mode.h header file, as shown below:

Code Listing 19-3 LCD configuration information (lcd_mode.h)

```
_1 typedef struct {
2 char label[64];
3 u32 width;
4 u32 height;
5 u32 hp;
6 u32 hpe;
7 u32 hmax;
```

```
8 u32 hpol; 9 u32 vps; 10 u32 vpe; 11 u32 vmax; 12 u32 vpol; 13
double freq; 14 } VideoMode; 15 16 static const VideoMode VM
ODE_480x272 = { 17 .label = "480x272@60Hz", 18 .width = 48
0, 19 .height = 272, 20 .hps = 482, 21 .hpe = 523, 22 .hmax = 525
, 23 .hpol = 0, 24 .vps = 274, 25 .vpe = 284, 26 .vmax = 286, 27 .
vpol = 0, 28 .freq = 9 29 }; 30 31 32 33 static const VideoMode V
MODE_800x480 = { 34 .label = "800x480@60Hz", 35 .width =
800, 36 .height= 480, 37 .hps = 840, 38 .hpe = 968, 39 .hmax = 1
056, 40 .hpol = 0, 41 .vps = 490, 42 .vpe = 492, 43 .vmax = 525,
44 .vpol = 0, 45 .freq = 33.0 46 };
```

Since all configurations are configured using variables, if readers want to modify the resolution, they only need to modify the value of VIDEO_Mode. Width represents the image width, and height represents the image height. The following hps, hpe, hmax, hpol, vps, vpe, vmax, vpol and freq can be found online, that is, VGA display timing or VESA timing can be found.

Then use the run_vdma_frame_buffer() function to configure the size, address and operation mode of VDMA to write to the memory. Read-only mode is used here because we only need to read the color bar data from the memory. The function run_vdma_frame_buffer() is actually a function officially provided by xilinx for configuring VDMA. It has been modified here to support read-only, write-only, and read-write. The author will not analyze it in detail here. The essence is to configure the registers of VDMA to implement the corresponding functions. Readers can study it in detail if they are interested.

Line 70 of code 47~ is a function that generates color bars. By directly replacing the value of the corresponding memory address with color bar data, the color bar data can be written into the memory. Since RGB888 is 3 bytes, the code is 52 lines, and our count needs +3 each time. At the end of the function, the data cached in the DataCache is flushed to DDR3 by calling the Xil_DCacheFlush() function so that VDMA can read it.

Compile the project and verify it on the board.

19.3.4 Board verification

As shown in Figure 19-10, the development board is connected to the 12V DC power supply, XILINX downloader JTAG port and HDMI display. After the lines are connected correctly, turn on the switch to power on the board.

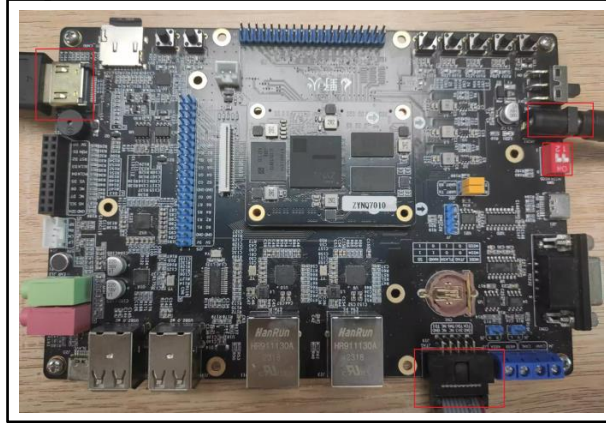


Figure 19-10 Program download connection diagram

As shown in Figure 19-11, download the program for the development board.

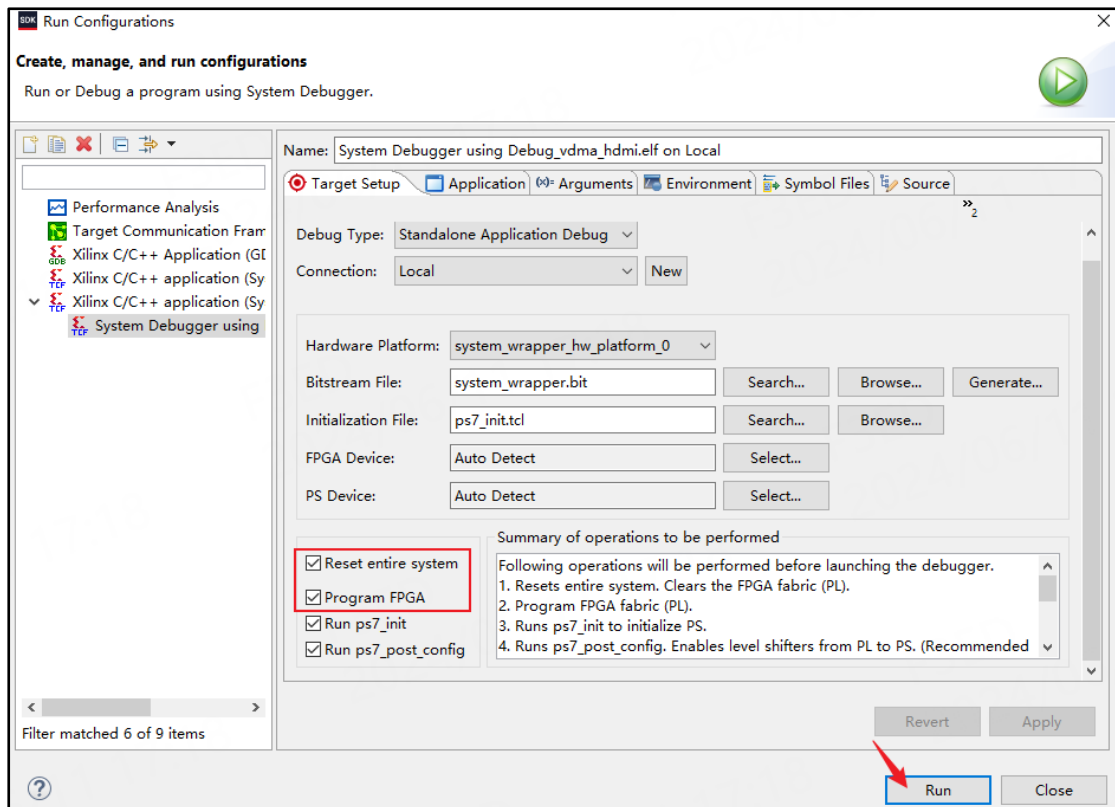


Figure 19-11 Program download diagram

After the program download is completed, color bars will be displayed on the HDMI display with an image resolution of 1024*600, as shown in Figure 19-12.



Figure 19-12 HDMI color bar image display

19.4 End of chapter summary

This chapter introduces the display of HDMI color bar images.

19.5 Development training

Change related parameters to achieve color bar display at other HDMI resolutions.

Chapter 20 SD card reading BMP display LCD

20.1 Chapter Introduction

In the previous chapter, we learned how to write data to the SD card and read the data. In this chapter, we read the BMP in the SD card and display it on the LCD screen.

20.2 Theoretical study

For VDMA related knowledge, readers can refer to the previous chapter and will not go into details in this chapter.

20.2.1 Introduction to BMP image format

I believe all readers know the common image formats, such as JPG, BMP, PNG and dynamic GIF. Among them, JPG is a lossy compressed image, PNG is a lossless compression, and BMP is not compressed, so it takes up a lot of space. This is its shortcoming, but precisely because it is not compressed, parsing its format is simpler than the other three. So this time we will read BMP images.

BMP is derived from the abbreviation of Bitmap, also known as DIB (Device-Independent Bitmap), which is a display-independent bitmap digital image file format and the standard image file format in the Windows operating system. Images are typically saved in color depths of 2 (1 bit), 16 (4 bits), 256 (8 bits), 65536 (16 bits), and approximately 16.77 million (24 bits) colors (where the bit is the number of data bits used to represent each point). The recording order of bitmap data is from left to right within a scan line, and from bottom to top between scan lines.

The data of the BMP file is divided into four parts in order starting from the file header:

Bitmap file header (bmp file header): Provides information such as file format and size.

Bitmap information header (bitmap information): Provides information such as the size of the image data, the number of bit planes, the compression method, and the color index.

Color palette: Optional. If an index is used to represent an image, the palette is a mapping table between the index and its corresponding color.

Bitmap data (bitmap data): image data area, usually RGB888, 24bit.

The figure below shows the size of each part of the data:

数据段名称	大小 (byte)	开始地址	结束地址
位图文件头(bitmap-file header)	14	0000h	000Dh
位图信息头(bitmap-information header)	40	000Eh	0035h
调色板(color table)	由biBitCount决定	0036h	未知
图片点阵数据(bitmap data)	由图片大小和颜色定	未知	未知

Figure 20-1 Introduction to data format

Only 1, 4, and 8-bit images will use palette data. 16, 24, and 32-bit images do not require palette data, that is, the palette only requires a maximum of 256 items (index 0 - 255). The size of the color table depends on the color mode used: 8 for a 2-color image

Bytes; 64 bytes for a 16-color image; 1024 bytes for a 256-color image. Among them, each 4 bytes represents a color, and is represented by B (blue), G (green), R (red), alpha (transparency value of 32-bit bitmap, generally not needed). In the absence of a palette, the bitmap information header is followed by the bitmap data. Next, we will analyze it with a 24-bit BMP picture, as shown in the figure below.

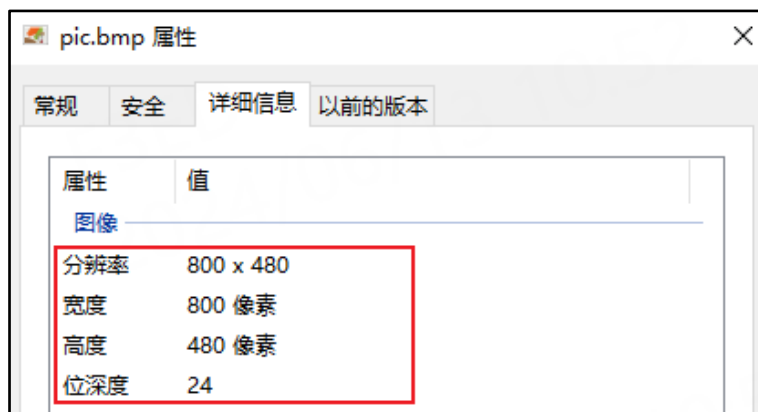


Figure 20-2 BMP information

You can see that this is an 800x480 picture with a depth of 24. We can open the picture through notepad++ and select hexadecimal display.

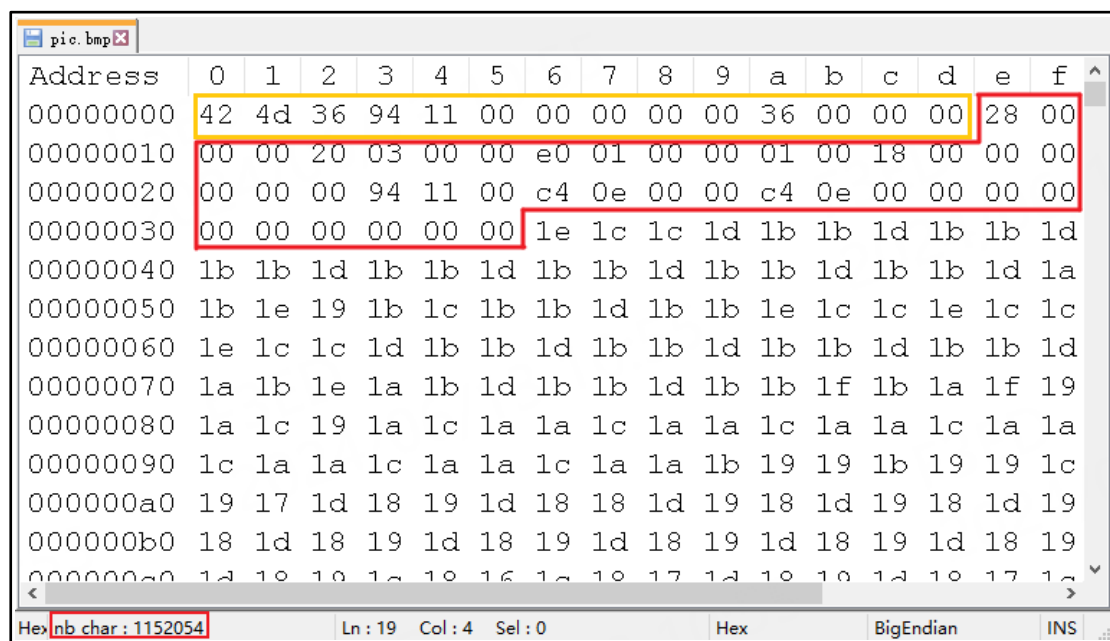


Figure 20-3 Hexadecimal display

The yellow box in the above figure contains a total of 14 bytes corresponding to the BMP file header, and the red box is the BMP information header, which is 40 bytes in total (generally 40 bytes, different versions of the information header occupy different numbers of bytes). Then you can see below that this nb char has a total of 1152054 bytes, minus the file header and information header, which total 54 bytes, leaving 1152000 bytes, which is the bitmap data. A BMP image in 800*480 RGB888 format occupies $800 \times 480 \times 3 = 1152000$ bytes of space, which is consistent with the analysis.

Figure 20-4| and Figure 20-5 are the detailed introduction of BMP file header and information header respectively.

字段名	大小 (单位: 字节)	描述
bfType	2	位图类别, 根据不同的操作系统而不同, 在Windows中, 此字段的值总为'BM'
bfSize	4	BMP图像文件的大小
bfReserved1	2	总为0
bfReserved2	2	总为0
bfOffBits	4	BMP图像数据的地址

Figure 20-4 BMP file header introduction

字段名	大小 (单位: 字节)	描述
biSize	4	本结构的大小, 根据不同的操作系统而不同, 在Windows中, 此字段的值总为28h字节=40字节
biWidth	4	BMP图像的宽度, 单位像素
biHeight	4	总为0
biPlanes	2	总为0
biBitCount	2	BMP图像的色深, 即一个像素用多少位表示, 常见有1、4、8、16、24和32, 分别对应单色、16色、256色、16位高彩色、24位真彩色和32位增强型真彩色
biCompression	4	压缩方式, 0表示不压缩, 1表示RLE8压缩, 2表示RLE4压缩, 3表示每个像素值由指定的掩码决定
biSizeImage	4	BMP图像数据大小, 必须是4的倍数, 图像数据大小不是4的倍数时用0填充补足
biXPelsPerMeter	4	水平分辨率, 单位像素/m
biYPelsPerMeter	4	垂直分辨率, 单位像素/m
biClrUsed	4	BMP图像使用的颜色, 0表示使用全部颜色, 对于256色位图来说, 此值为100h=256
biClrImportant	4	重要的颜色数, 此值为0时所有颜色都重要, 对于使用调色板的BMP图像来说, 当显卡不能够显示所有颜色时, 此值将辅助驱动程序显示颜色

Figure 20-5 BMP header format

First, let's analyze the BMP file header and compare it with the hexadecimal data in Figure 20-3. For BMP images, the bfType must be 0x4d42, indicating that this is a BMP image. The next 4 bytes represent the image size. You can see that the bfAize is 0x00119436. When converted to decimal, it is 1152054 bytes, which is consistent with what our notepad++ displays. The subsequent bfReserve1 and bfReserve2 are reserved bits fixed to 0, and bfOffBits refers to the address of our image data, that is, the offset of the image data, here it is 0x00000036. Converting to decimal is 54, which means that after 54 bytes, our image data will be obtained.

Next let's look at the BMP information header.

Address	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00000000	42	4d	36	94	11	00	00	00	00	00	36	00	00	00	28	00
00000010	00	00	20	03	00	00	e0	01	00	00	01	00	18	00	00	00
00000020	00	00	00	94	11	00	c4	0e	00	00	c4	0e	00	00	00	00

Figure 20-6 Notepad++ corresponding area value

From the analysis of Figure 20-5 and Figure 20-6, we can get:

0x00000320, which is 800 pixels. biHeight: is 0x000001e0, which is 480 pixels. biBitCount: You can see that it is 0x0018, which is 24.

biSizeImage: is 0x00119400, which is 1152000 bytes. Since the BMP does not contain a color palette, the BMP contains image data after the file header and information header. Next

start experimenting.

20.3 Practical exercises

20.3.1 Experimental objectives

PS reads the information of the BMP image in the SD card, then displays it on the LCD, and prints out the image resolution and other information.

20.3.2 Hardware resources

This experiment uses the LCD interface display on the PL side. SD card slot. UART.

20.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 20-7.

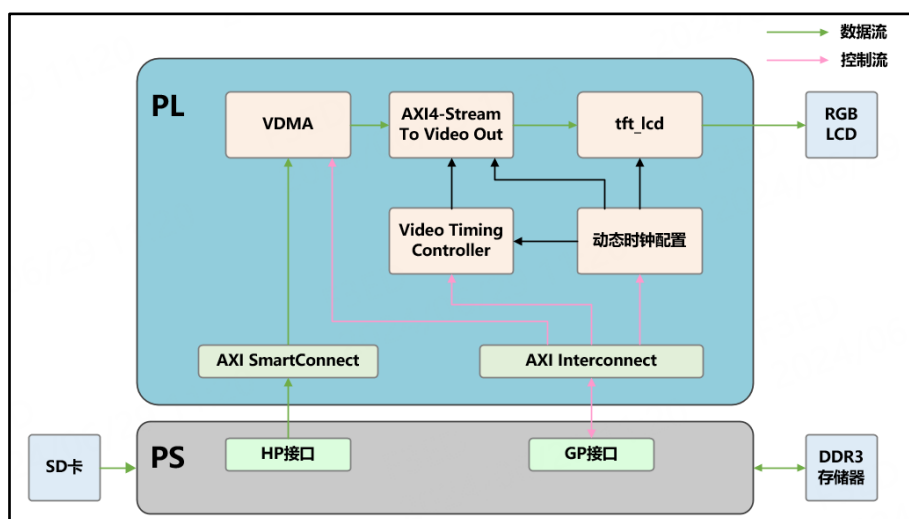


Figure 20-7 Overall block diagram

This system block diagram is basically the same as the system block diagram in the chapter "VDMA Driven LCD Display". The difference is that for VDMA driven LCD display, we generate the color bar data ourselves and wrote it into the DDR3 memory and then read it out for display. In this experiment, we wrote the BMP image data in the SD card into the DDR3 memory and then read it out for display. So remember to open the SD card controller when configuring the IP of ZYNQ PS. For details, please view the SD card reading and writing TXT experiment.

2. Hardware design

ZYNQ system ip

The configuration of the ZYNQ system IP in the chapter "VDMA Driver LCD Display" is basically the same. However, since an SD card is used, readers must remember to check the enable SD card, as shown in the figure below:

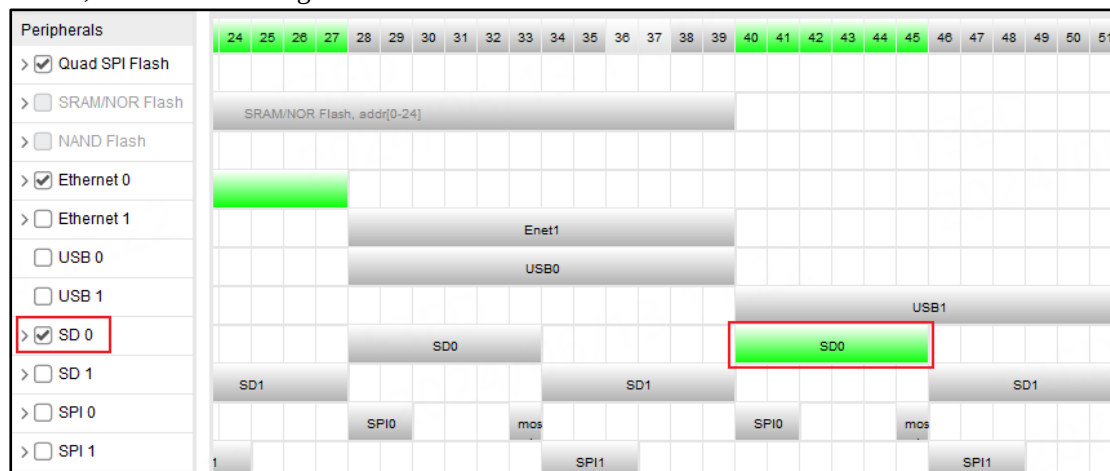


Figure 20-8 Check SD0

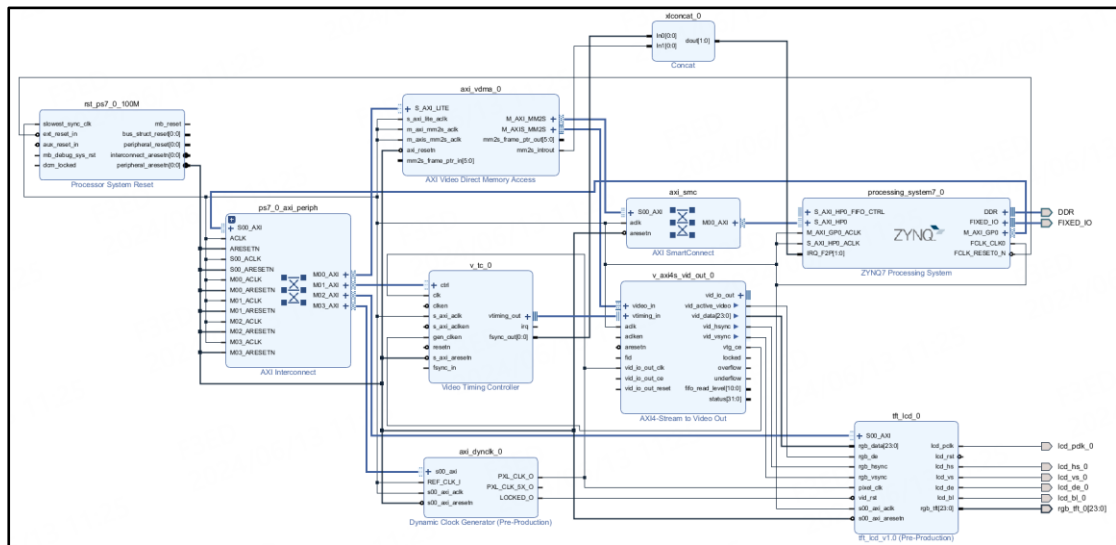


Figure 20-9 Overall IP block diagram

At this point, our Block Design is completed. Press the "F6" key to perform "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally press the shortcut key "Ctrl + S" to save the design.

Then execute "Generate Output Products" and "Create HDL Wrapper" in sequence. Then perform pin constraints on the used pins, generate bit stream files, and export the hardware (check "Include bitstream" in the pop-up dialog box). Finally start the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and

Add the following code in the main.c file:

Code List 20-1 Read SD card BMP display LCD reference code (main.c)

```
1 #include <stdio.h> 2 #include <stdlib.h> 3 #include <string.h> 4 #include "xil_types.h" 5 #include
"xil_cache.h" 6 #include "xparameters.h" 7 #include "xaxivdma.h" 8 #include "xaxivdma_i.h" 9 #incl
ude "vdma_api/vdma_api.h" 10 #include "display_ctrl/display_ctrl.h" 11 #include "xdevcfg.h" 12 #inc
lude "ff.h" 13 14 #define CLOCK_ADDR XPAR_AXI_DYNCLK_0_BASEADDR //Clock address 1
5 #define VDMA_DEVICE_ID XPAR_AXIVDMA_0_DEVICE_ID //VDMA device ID 16 #define
XVTC_DEVICE_ID XPAR_VTC_0_DEVICE_ID //VTC device ID 17 18 22 TCHAR file_name[20]
= "pic.bmp"; 23 /*buffer memory base address*/
```

```
24 unsigned int const FB_Base = (XPAR_PS7_DDR_0_S_AXI_BASEADDR + 0x1000000);
25 26 //Mount the file system 27 int Mount_Init() 28 { 29 TCHAR *path = "0:/"; 30 int err=0; 31 /*Mou
nt SD card*/ 32 if (f_mount(&fatfs, path, 1) != FR_OK) { 33 err = 1; 34 xil_printf("Mount failed , Mayb
e not fat format.\r\n"); 35 } 36 else { 37 xil_printf("SD card mount success !\n"); 38 } 39 return err; 40 }
41 42 //Read the picture in the SD card 43 void read_bmp(u8 *frame_addr) 44 { 45 u8 bmp_header[54];
46 UINT bmp_offs; 47 UINT bmp_width, bmp_height, bmp_size; 48 UINT br; 49 int err=0; 50 FIL Txt;
51 52 err = Mount_Init(); 53 if(!err) { 54 //Open the file and prepare to read the picture 55 f_open(&Txt,
file_name,FA_READ); 56 57 //Read the bmp frame, the header + information header is 54 bytes 58 f_re
ad(&Txt,bmp_header,54,&br); 59 60 xil_printf("The bmp_header is : \r\n"); 61 for (int i=0;i<54;i++) 62
xil_printf("%02x ",bmp_header[i]); 63 64 bmp_width = *(UINT *)(bmp_header + 0x12); 65 bmp_heigh
t = *(UINT *)(bmp_header + 0x16); 66 bmp_size = *(UINT *)(bmp_header + 0x22); 67 xil_printf("\n w
idth:%d height:%d size:%d\r\n",bmp_width, 68 bmp_height,bmp_size); 69 70 /*Get the image data offse
t*/ 71 bmp_offs = *(UINT *)(bmp_header + 0x0a); 72 /*Jump to the location of the image data*/ 73 f_ls
eek(&Txt, bmp_offs); 74 75 //Write the image data to ddr and read from bottom to top 76 for(int j=bmp_
height-1;j>=0;j--) 77 f_read(&Txt,frame_addr+j*3*bmp_width,bmp_width*3,&br); 78 79 f_close(&Txt
); //After reading, close the file 80 Xil_DCacheFlush(); //Refresh cache and prepare to read out the displa
y 81 xil_printf("bmp read_ok!\n\r"); 82 } 83 else { 84 xil_printf("bmp read_failed!\n\r"); 85 } 86 } 87 88
int main(void)
```

```
89 {
90     VIDEO_Mode = VMODE_800x480;
91
92     //配置 lcd 参数
93     run_vdma_frame_buffer(&Vdma, VDMA_DEVICE_ID, VIDEO_Mode.width,
94         VIDEO_Mode.height, FB_Base, 0, 0, ONLY_READ);
95
96     DisplayInitialize(&Dispctrl, XVTC_DEVICE_ID, CLOCK_ADDR);
97     DisplaySetMode(&Dispctrl, &VIDEO_Mode); //设置 lcd 参数
98     DisplayStart(&Dispctrl); //开启
99     //读取 bmp 图片显示
100     read_bmp((u8*)FB_Base);
101     return 0;
102 }
```

The basic variable definitions of this program are basically the same as those of VDMA driver LCD. It introduces SD card read and write functions. In lines 27-40 of the code, it first detects whether the SD card is mounted, and if successful, prints relevant information. In lines 43-86 of the code, use the function of the FATS library to read the BMP image. We have already analyzed that the first 54 bytes are the information header and file header, so we define a bmp_header to store this information and output it to the serial port, which reflects the image resolution and other information.

One thing to note is that in lines 76-77 of the code, our for loop is from high to low. This is because when reading BMP, the scanning method is from bottom to top and left to right, so the data must be stored here in reverse, otherwise the picture will be reversed if read in the normal order. Finally, use Xil_DCacheFlush() to flush the data in the Cache to DDR3 memory.

The main function still configures different display resolutions, VDMA and dynamic clock.

Compile the project and verify it on the board.

20.3.4 Board verification

As shown in Figure 20-10, the development board is connected to the 12V DC power supply, XILINX downloader JTAG port and LCD screen. After the lines are connected correctly, turn on the switch to power on the board.

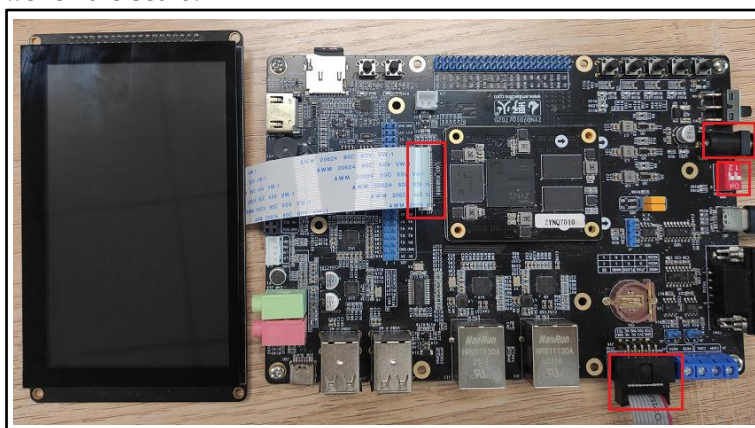


Figure 20-10 Program download connection diagram

First, store a BMP picture in the SD card, name it "pic.bmp", and then insert it into the development board. Note that the file name must be consistent with that in the program. (Be careful not to include special symbols such as underlines and spaces)

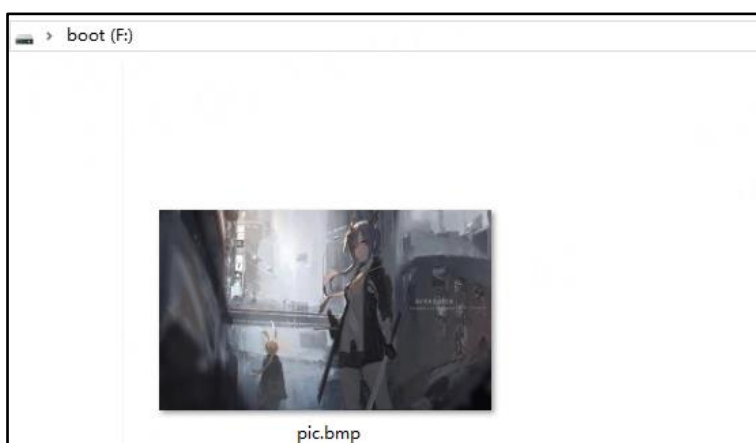


Figure 20-11 BMP pictures stored in SD card

Then start the burning process.

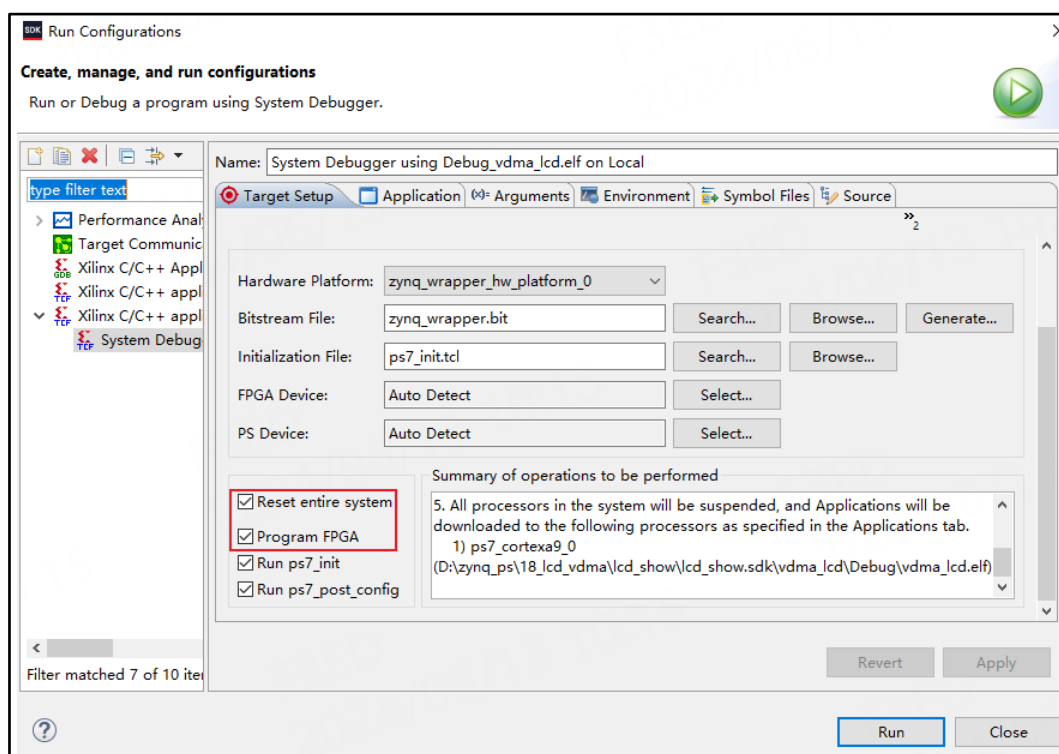


Figure 20-12 Program download diagram

The serial port prints out the file header and information header, and outputs the height, width and size of the image.

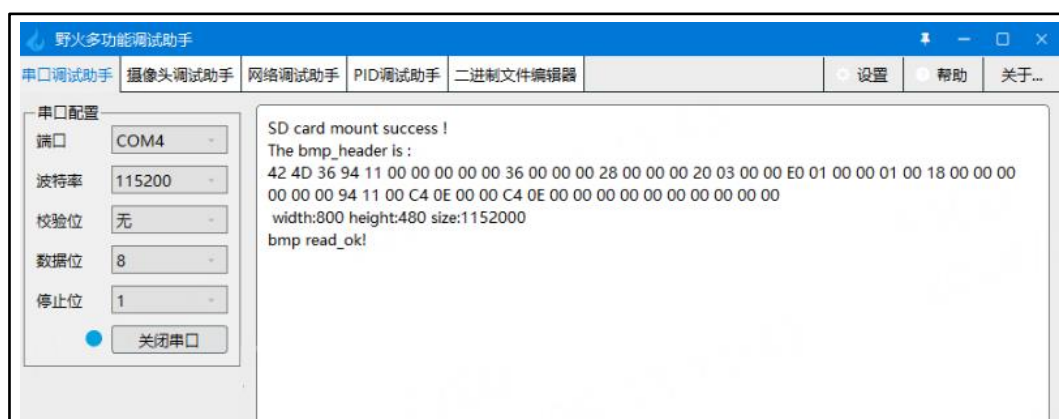


Figure 20-13 Data displayed by the serial port

As shown in Figure 20-14, the LCD displays the pictures stored in the SD card.



Figure 20-14 LCD display results

20.4 End of chapter summary

Based on the chapter "VDMA Drive LCD Display", this chapter adds an SDIO controller so that pictures can be read from the SD card and displayed on the LCD. Before the experiment, we also introduced the BMP image format, opened a BMP image in hexadecimal mode, and analyzed the data format inside.

20.5 Development training

You can try to save multiple BMP pictures to the SD card, and then cycle through these pictures on the LCD.

Chapter 21 SD card reading BMP display HDMI

21.1 Chapter Introduction

The principle of this chapter is basically the same as that of the previous chapter, except that the LCD is replaced by HDMI. I will not go into details about the relevant principles.

21.2 Theoretical study

For VDMA-related knowledge, readers can refer to the previous chapter. For HDMI display principles, you can also refer to the previous HDMI color bar experiment. This chapter will not go into details.

21.3 Practical exercises

21.3.1 Experimental objectives

PS reads the information of the BMP image in the SD card, then displays it on HDMI, and prints out the image resolution and other information at the same time.

21.3.2 Hardware resources

This experiment uses the HDMI interface display on the PL side. SD card slot. UART.

21.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 21-1.

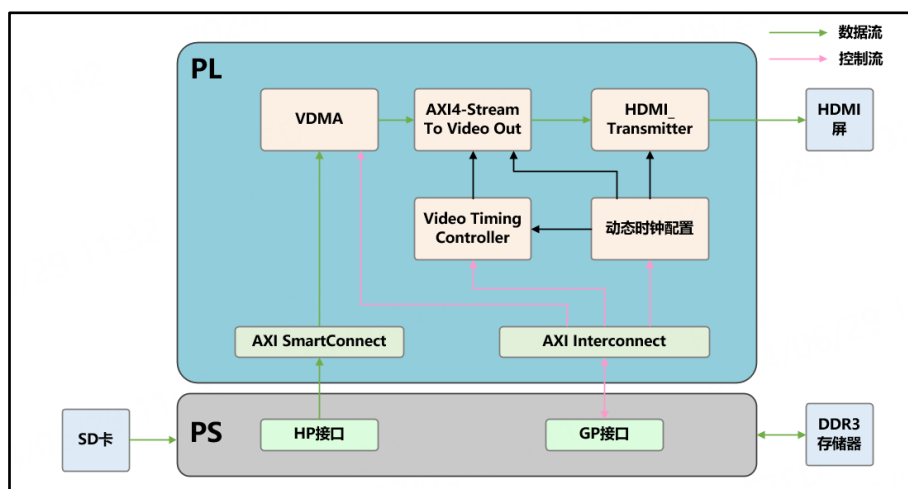


Figure 21-1 Overall block diagram

This system block diagram is basically the same as the system block diagram of VDMA driving HDMI display. The difference is that when VDMA drives HDMI display, we generate the color bar data ourselves and write it into the DDR3 memory and then read out the display. In this experiment, we write the BMP image data in the SD card into the DDR3 memory and then read out the display. So remember to turn on the SD card controller when configuring the IP of ZYNQ PS. For details, please view the SD card reading and writing TXT experiment.

2. Hardware design

ZYNQ system ip

The configuration of the ZYNQ system IP in the chapter "SD Card Reading BMP Display LCD" is basically the same, except that since an SD card is used, the reader remembers to check the SD card enablement, and at the same time, replace the LCD-related IP with an HDMI-related IP, as shown below:

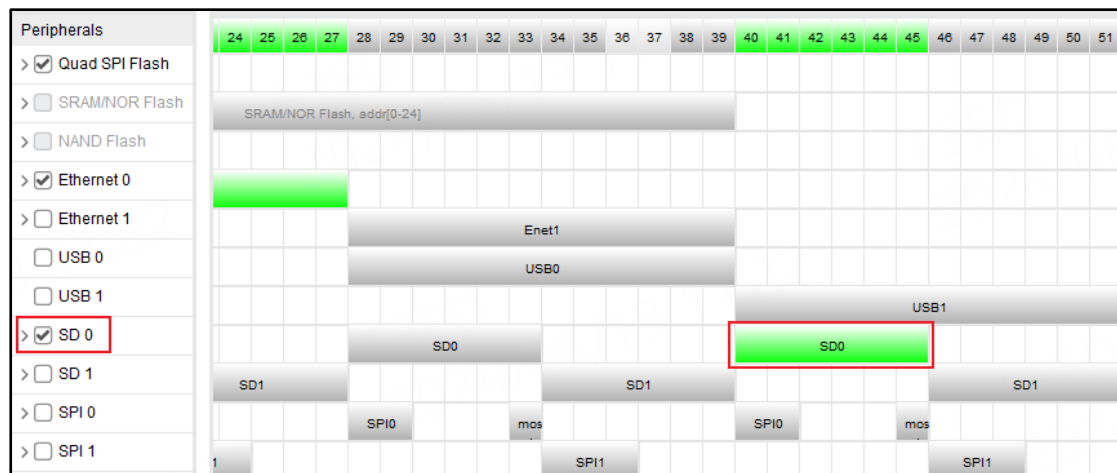


Figure 21-2 Check SD0

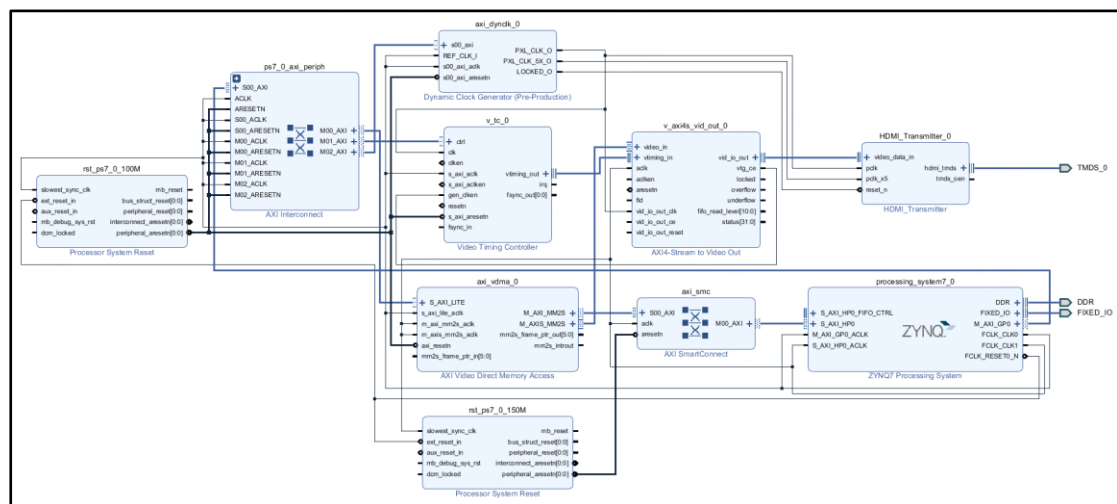


Figure 21-3 Overall IP block diagram

At this point, our Block Design is completed. Press the "F6" key to perform "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally press the shortcut key "Ctrl + S" to save the design.

Then execute "Generate Output Products" and "Create HDL Wrapper" in sequence. Then perform pin constraints on the used pins, then generate a bit stream file and export the hardware (check "Include bitstream" in the pop-up dialog box). Finally launch the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and

Code List 21-1 SD card reading BMP display HDMI reference code (main.c)

```
1 #include <stdio.h> 2 #include <stdlib.h> 3 #include <string.h> 4 #include "xil_types.h" 5 #include "xil_cache.h"
6 #include "xparameters.h" 7 #include "xaxivdma.h" 8 #include "xaxivdma_i.h" 9 #include "vdma_api/vdma_api.h"
10 #include "display_ctrl/display_ctrl.h" 11 #include "xdevcfg.h" 12 #include "ff.h" 13 14 #define CLOCK_ADD
R XPAR_AXI_DYNCLK_0_BASEADDR //Clock address 15 #define VDMA_DEVICE_ID XPAR_AXIVDMA_
0_DEVICE_ID //VDMA device ID 16 #define XVTC_DEVICE_ID XPAR_VTC_0_DEVICE_ID //VTC device I
D 17 18 22 TCHAR file_name[20] = "test1.bmp"; 23 /*Buffer memory base address*/ 24 unsigned int const FB_B
ase = (XPAR_PS7_DDR_0_S_AXI_BASEADDR + 0x1000000); 25 26 //Mount file system 27 int Mount_Init() 2
8 { 29 TCHAR *path = "0:/"; 30 int err=0; 31 /*Mount SD card*/ 32 if (f_mount(&fatfs, path, 1) != FR_OK) { 33
err = 1; 34 xil_printf("Mount failed , Maybe not fat format.\r\n"); 35 } 36 else { 37 xil_printf("SD card mount succ
ess !\n"); 38 } 39 return err; 40 } 41 42 //Read the picture in the SD card 43 void read_bmp(u8 *frame_addr) 44 { 4
5 u8 bmp_header[54]; 46 UINT bmp_offs; 47 UINT bmp_width, bmp_height, bmp_size; 48 UINT br; 49 int err=0;
50 FILE Txt; 51 52 err = Mount_Init(); 53 if(!err) {
```

```
54 //打开文件 准备读取图片
55 f_open(&Txt, file_name, FA_READ);
56
57 //读取 bmp 的帧, 头文件头+信息头一个 54 个字节
58 f_read(&Txt, bmp_header, 54, &br);
59
60 xil_printf("The bmp_header is : \r\n");
61 for (int i=0; i<54; i++)
62     xil_printf("%02x ", bmp_header[i]);
63
64 bmp_width = *(UINT *) (bmp_header + 0x12);
65 bmp_height = *(UINT *) (bmp_header + 0x16);
66 bmp_size = *(UINT *) (bmp_header + 0x22);
67 xil_printf("\n width:%d height:%d size:%d\r\n", bmp_width,
68     bmp_height, bmp_size);
69
70 /*获取图片数据偏移量*/
71 bmp_offs = *(UINT *) (bmp_header + 0x0a);
72 /*跳转到图像数据的位置*/
73 f_lseek(&Txt, bmp_offs);
74
75 //将图片数据写入 ddr, 从下而上读取
76 for (int j=bmp_height-1; j>=0; j--)
77     f_read(&Txt, frame_addr+j*3*bmp_width, bmp_width*3, &br);
78
79 f_close(&Txt); //读完 关闭 文件
80 Xil_DCacheFlush(); //刷新 cache 准备读出显示
81 xil_printf("bmp read_ok!\n\r");
82 }
83 else {
84     xil_printf("bmp read_failed!\n\r");
85 }
86 }
87
88 int main(void)
89 {
90     VIDEO_Mode = VMODE_1024x600;
91
92     run_vdma_frame_buffer(&Vdma, VDMA_DEVICE_ID, VIDEO_Mode.width,
93         VIDEO_Mode.height, FB_Base, 0, 0, ONLY_READ);
94
95     DisplayInitialize(&Dispctrl, XVTC_DEVICE_ID, CLOCK_ADDR);
96     DisplaySetMode(&Dispctrl, &VIDEO_Mode);
97     DisplayStart(&Dispctrl); //开启
98     //读取 bmp 图片显示
99     read_bmp((u8*) FB_Base);
100     return 0;
101 }
```

The code is basically the same as the previous chapter, except that the display resolution has been changed. VIDEO_Mode readers can check and modify it by themselves. At the same time, they should also pay attention to whether their monitor supports a certain resolution output. The author here uses 1024*600.

21.3.4 Board verification

As shown in Figure 21-4, the development board is connected to the 12V DC power supply, XILINX downloader JTAG port and HDMI display. After the lines are connected correctly, turn on the switch to power on the board.

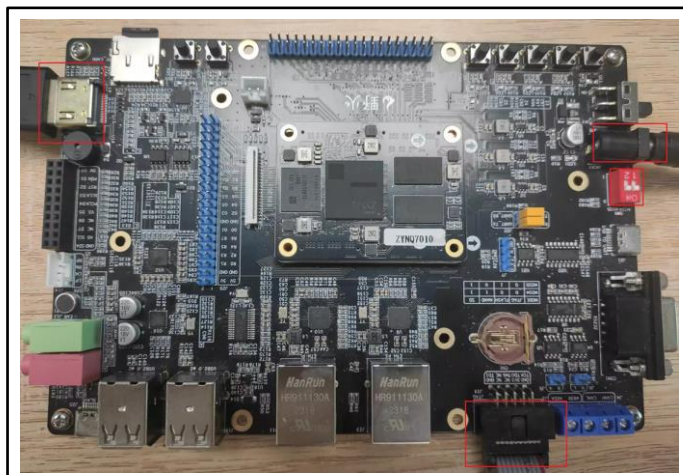


Figure 21-4 Program download connection diagram

First, store a BMP image in the SD card and then insert it into the development board. Note that the file name must be consistent with that in the program. (Be careful not to include special symbols such as underlines and spaces)

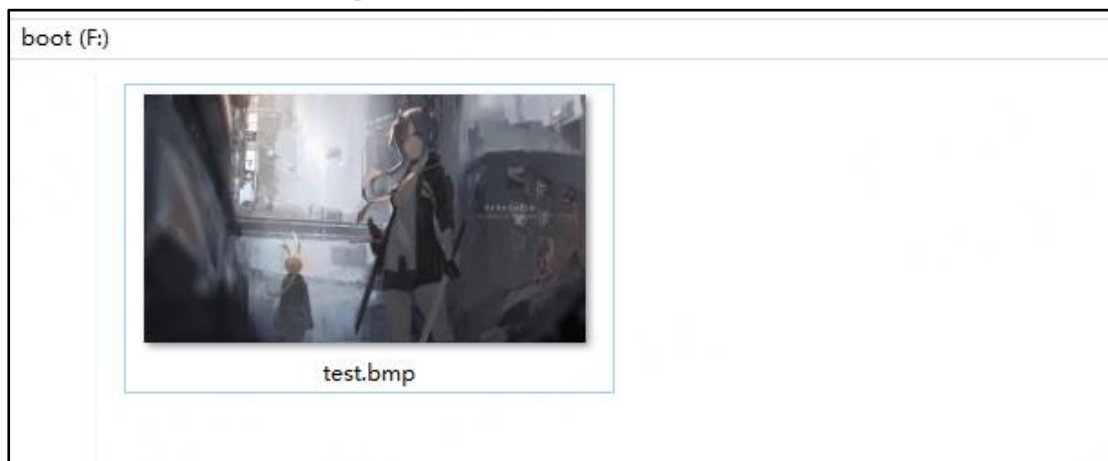


Figure 21-5 Storing BMP pictures in SD card

As shown in Figure 21-6, download the program for the development board.

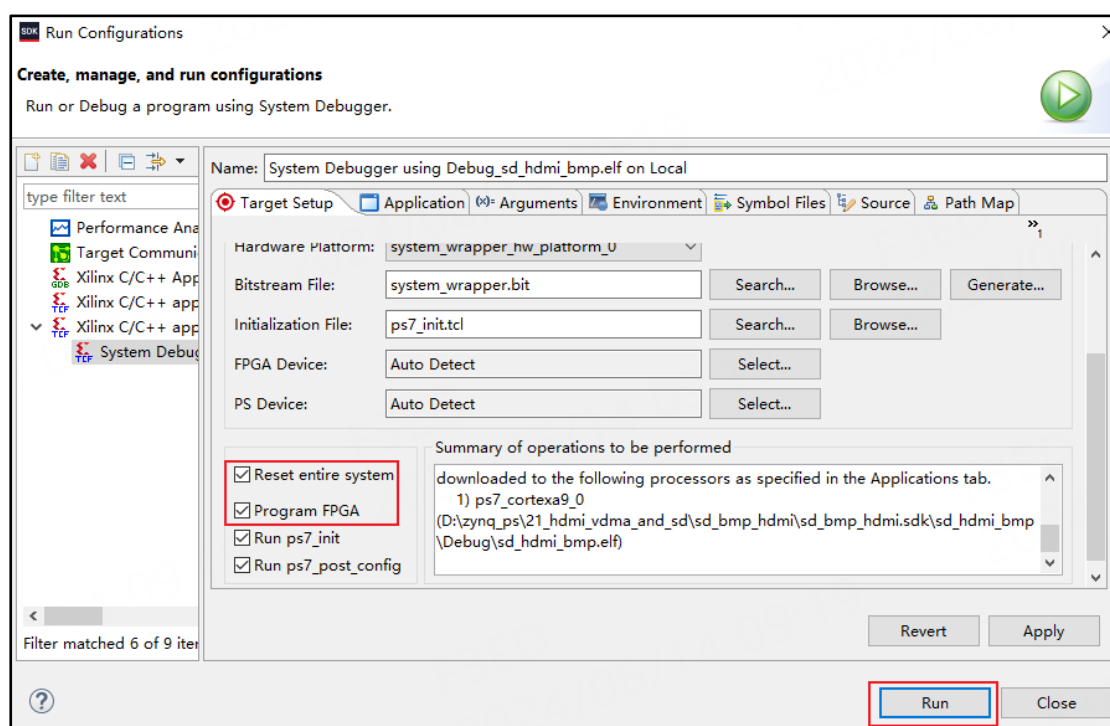


Figure 21-6 Program download diagram

After the program download is completed, the corresponding image will be displayed on the HDMI display with a resolution of 1024x600, as shown in Figure 21-7. At the same time, the serial port prints out the file header and information header, and outputs the height, width and size of the image.



Figure 21-7 HDMI display result (1024x600)

21.4 End of chapter summary

This time I learned how to read the BMP images in the SD card, and then parse and display them on the HDMI display.

21.5 Development training

Modified image display for larger resolution.

Chapter 22 OV7725 LCD Display

22.1 Chapter Introduction

In the previous chapter, we learned that the data generated and displayed by the PS is transmitted to the LCD interface of the PL side for display through VDMA. In this chapter, image data is collected through ov7725 and transmitted to the LCD for real-time display.

22.2 Theoretical study

22.2.1 Camera Introduction

1. The difference between digital cameras and analog cameras

Output signal type: The output signal of the digital camera is a digital signal, and the output signal of the analog camera is a standard analog signal. **Interface type:** Digital cameras have USB interfaces (such as common PC-side driver-free cameras), IEEE1394 FireWire interface (a high-speed transmission interface developed by a development alliance led by Apple, with a data transfer rate of up to 800Mbps), Gigabit network interface (network camera). Analog cameras mostly use AV video terminals (signal line + ground wire) or SVIDEO (i.e. Lotus head--SUPER VIDEO, which is a five-core interface consisting of two video brightness signals, two video chrominance signals and one common shielded ground wire, a total of five core wires). **Resolution:** The pixel index of the photosensitive device of the analog camera is generally maintained at about 752(H)*582(V), and the number of pixels is generally maintained at about 41W. Digital camera resolutions generally range from hundreds of thousands to millions or even tens of millions. But this does not mean that the imaging resolution of digital cameras is higher than that of analog cameras. The reason is that analog cameras output analog video signals, which are generally directly input to a TV or monitor. The resolution of the photosensitive device has a certain conversion relationship with the number of scans of the TV signal. The display medium of the image has been determined. Therefore, it is not that the resolution of the photosensitive device of the analog camera cannot be made higher, but it is not necessary to make it so high based on the actual situation.

2. The difference between CCD and CMOS

The main differences between the camera's image sensor CCD and CMOS sensor are as follows:

Imaging materials

The names of CCD and CMOS are related to the materials used in their imaging. CCD is the abbreviation of Charge Coupled Device, and CMOS is the abbreviation of Complementary Metal Oxide Semiconductor.

Power consumption

Since the pixels of the CCD are composed of MOS capacitors, when reading the charge signal, a two-phase, three-phase, or four-phase timing pulse signal with a relatively large voltage (at least 12V) is required to effectively transfer the charge. Therefore, in addition to having multiple power supplies, the CCD imaging system also consumes considerable power in its peripheral circuits. Some CCD imaging systems require

2~5W of power. The CMOS photoelectric sensor device only needs to use a single power supply of 5V or 3V, and the power consumption is very small, only 1/8~1/10 of the CCD. Some CMOS imaging systems only consume 20~50mW of power.

Image quality

CCD sensor device production technology started early and is mature. It uses PN junction or silicon dioxide (sio₂) isolation layer to isolate noise, so the noise is low and the imaging quality is good. Compared with CCD, the main disadvantages of CMOS are high noise and low sensitivity. However, with the continuous development of CMOS circuit noise reduction technology, good conditions have been provided for the production of high-density and high-quality CMOS sensing devices. Nowadays, CMOS sensors have occupied most of the market. Mainstream SLR cameras and smart phones have generally adopted CMOS sensors.

22.2.2 OV7725 Introduction

This section mainly explains the OV7725 camera used in the experimental project. The actual camera is shown in

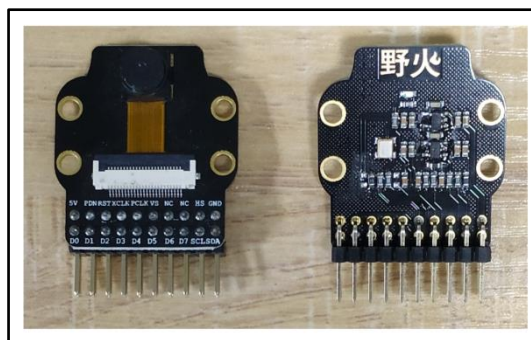


Figure 22-1 OV7725 physical picture

The camera is mainly composed of lens, image sensor, on-board circuit and signal pins below. The lens component includes a lens holder and a convex lens that can be rotated to adjust the distance. The focal length can be adjusted by rotating. During normal use, the lens holder is covered on the circuit board to block light. The light can only be transmitted to the image sensor in the center through the lens. The image data it collects is directly transmitted to the FPGA chip. The FPGA caches the received image data into DDR3 SDRAM. In the VGA image display effective area, the VGA driver reads the image data from the DDR3 SDRAM and displays it on the display.

If you disassemble the camera holder, you can see a square device on the PCB directly under the camera. It is the core component of the camera, a CMOS type digital image sensor model OV7725. The sensor supports outputting images up to 300,000 pixels (640x480 resolution). It has a small size and low operating voltage. It supports the use of VGA timing to output image data. The data format of the output image supports YUV (422/420), YCbCr422 and RGB565 formats. It can also compensate the collected images and support basic processing such as gamma curve, white balance, saturation, and chroma.

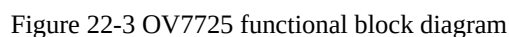
22.2.3 OV7725 pin and functional frame diagram

The OV7725 sensor is packaged in BGA. Its front end is a lighting window, and the pins are led out from the back. The pin distribution is shown in the figure below:



Table 22-1 OV7725 Pin Description

Below we explain these signal pins based on the OV7725 functional block diagram.



天貓: <https://yehuosm.tmall.com>

Marked is the control register of OV7725, which operates according to the parameters configured in these registers, and these parameters are written by the external controller through the SCL and SDA pins. The communication protocol SCCB used by SCL and SDA is very similar to I2C. In STM32, we can directly use I2C hardware peripherals to control it.

(2) Communication, control signals and clocks

The label contains the communication, control signals and external clock of OV7725, among which PCLK, HREF and VSYNC are the pixel synchronization clock, horizontal synchronization signal and frame synchronization signal respectively, which are very similar to the VGA signal in LCD screen control. When the RSTB pin is low, it is used to reset the entire sensor chip, and PWDN is used to control the chip to enter low-power mode. Pay attention to the last XCLK pin, which is completely different from PCLK. XCLK can be connected to an external crystal oscillator or provided by an external controller. To compare XCLK to OV7725, it is equivalent to the relationship between the HSE clock input pin and the STM32 chip. The PCLK pin can be compared to the SCL pin of the I2C peripheral of STM32.

(3) Photosensitive matrix

Labeled is the photosensitive matrix, where optical signals are converted into electrical signals. After various processing, these signals are stored into digital images represented by pixels.

(4) Data output signal

The label contains the DSP processing unit, which will do some basic image processing operations according to the configuration of the control register. This part also includes the image format conversion unit and compression unit. The converted data is finally output through the D0-D9 pins. Generally speaking, we use 8 data lines for transmission. At this time, only the D2-D9 pins are used.

22.2.4 SCCB Timing

The configuration parameters of the OV7725 register from the external controller are transmitted through the SCCB bus, and the SCCB bus is very similar to I2C. For the complete content of the SCCB Agreement, please view the "SCCB Agreement" document in the supporting materials. A brief introduction is given below.

SCCB start, stop signals and data validity

The start signal, stop signal and data validity of SCCB are exactly the same as I2C, see Figure 22-4 and Figure 22-5. Start signal: When SCL (SIO_C in the figure) is high level, SDA (SIO_D in the figure) appears with a falling edge, and SCCB starts transmitting. Stop signal: When SCL is high and SDA has a rising edge, SCCB stops transmission. Data validity: In addition to the start and stop states, during the data transmission process, when SCL is high level, the data on SDA must be guaranteed to be stable. That is to say, the level change on SDA can only occur when SCL is low level, and the SDA signal is collected when SCL is high level.

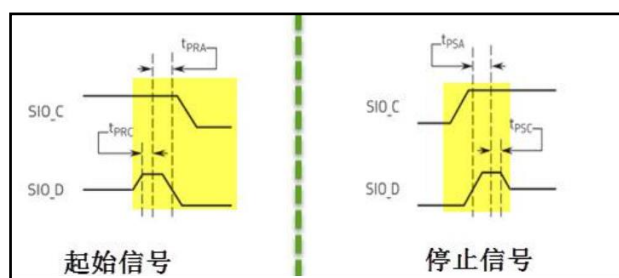


Figure 22-4 SCCB start and stop signals

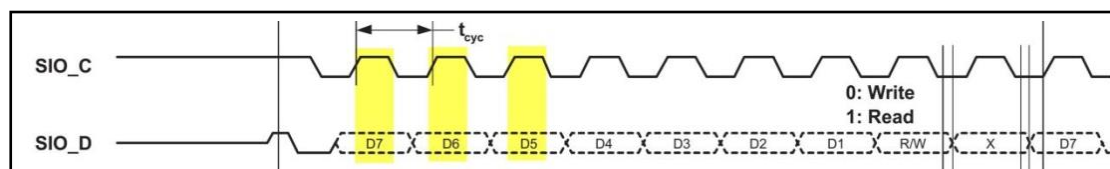


Figure 22-5 SCCB data valid signal

SCCB data reading and writing process

The read and write operations defined in the SCCB protocol are the same as those in I2C, just in a different way. It defines two types of write operations, namely three-step write operation and two-step write operation. The three-step write operation can write data to a destination register of the slave device, as shown in the figure below. In the three-step write operation, the first stage sends the ID address +W flag of the slave device (equal to the I2C device address: 7-bit device address + read and write direction flag), the second stage sends the 8-bit address of the slave device target register, and the third stage sends the 8-bit data to be written to the register. The "X" data bit in the figure can be written as 1 or 0, which has no effect on communication.

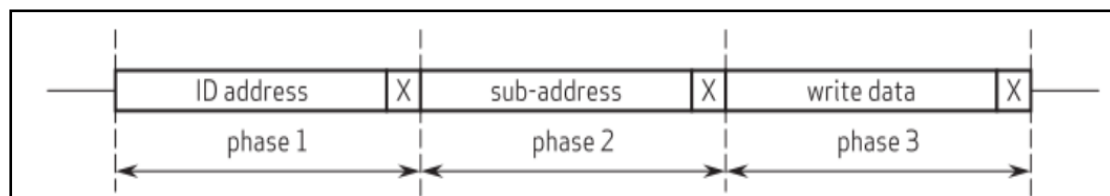


Figure 22-6 Three-step write operation of SCCB

There is no third stage in the two-step write operation, that is, only the device ID+W flag and the address of the destination register are transmitted to the slave device, as shown in the figure below. The two-step write operation is used to cooperate with the subsequent read register data operation. It is used together with the read operation to implement the I2C composite process.

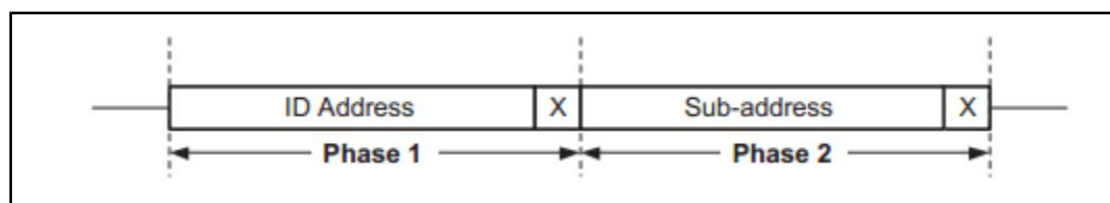


Figure 22-7 SCCB two-step write operation

Two-step read operation, which is used to read the data in the slave device's destination register, see the figure below. In the first phase, the slave device's device ID +R flag (device address + read direction flag) and free bit are sent, and in the second phase, the 8-bit data in the register is read and the NA bit (non-acknowledge signal) is written. Since the two-step read operation does not determine the address of the destination register, a two-step write operation must be performed before the read operation to provide the register address in the read operation.

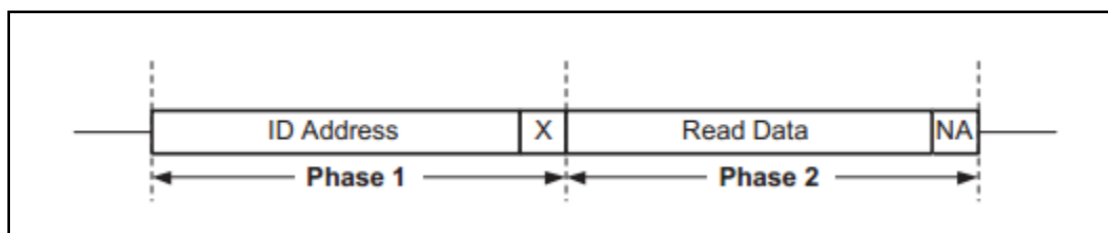


Figure 22-8 SCCB two-step read operation

It can be seen that the SCCB characteristics introduced above are no different from I2C. You can use I2C driver instead of SCCB to realize the communication between FPGA and OV7725.

22.2.5 Registers of OV7725

To drive the OV7725 camera, many internal registers need to be configured. For the configuration parameters of the relevant registers, readers can directly query the "OV7725datasheet" to learn more. Through the configuration of these registers, the resolution, image format, image processing and image direction of the camera output image can be controlled. See Figure 22-9.

Address (Hex)	Register Name	Default (Hex)	R/W	Description
10	AEC	40	RW	Exposure Value Bit[7:0]: AEC[7:0] (see register AEC for AEC[15:8])
11	CLKRC	80	RW	Internal Clock Bit[7]: Reserved Bit[6]: Use external clock directly (no clock pre-scale available) Bit[5:0]: Internal clock pre-scalar $F(\text{internal clock}) = F(\text{input clock}) / (\text{Bit}[5:0] + 1) / 2$ • Range: [0 0000] to [1 1111]
12	COM7	00	RW	Common Control 7 Bit[7]: SCCB Register Reset 0: No change 1: Resets all registers to default values Bit[6]: Resolution selection 0: VGA 1: QVGA Bit[5]: BT.656 protocol ON/OFF selection Bit[4]: Sensor RAW Bit[3:2]: RGB output format control 00: GBR4:2:2 01: RGB565 10: RGB555 11: RGB444 Bit[1:0]: Output format control 00: YUV 01: Processed Bayer RAW 10: RGB 11: Bayer RAW

Figure 22-9 OV7725 partial register configuration description

Pixel data output timing

When FPGA drives OV7725, it uses the SCCB protocol to configure its registers, and when it outputs images, it uses VGA or QVGA timing. VGA is used when the output image resolution is 640*480, and QVGA is Quarter

VGA, its output resolution is 320*240, these timings are very similar to controlling the LCD screen to output image data. When the OV7725 sensor outputs an image, it outputs it frame by frame. The data within the frame is generally output pixel by pixel from left to right, top to bottom (the direction can also be modified through the register), similar to VGA scanning display.

For example, see Figure 22-10 and Figure 22-11, if we use the D2-D9 data line and the image format is set to RGB565, when data is output, the D2-D9 data line remains stable during the rising edge of PCLK and will send a 1-byte data signal driven by 1 pixel synchronization clock PCLK, so 2 PCLK clocks can send 1 pixel data in RGB565 format. When HREF is high level, pixel data is transmitted in sequence. When each row of data is transmitted, the horizontal synchronization signal HREF will output a level jump signal to separate the data of the current row and the next row; a frame of image consists of N rows of data. When VSYNC is low level, the pixel data of each row is transmitted in sequence. When each frame of image is transmitted, VSYNC will output a level jump signal.

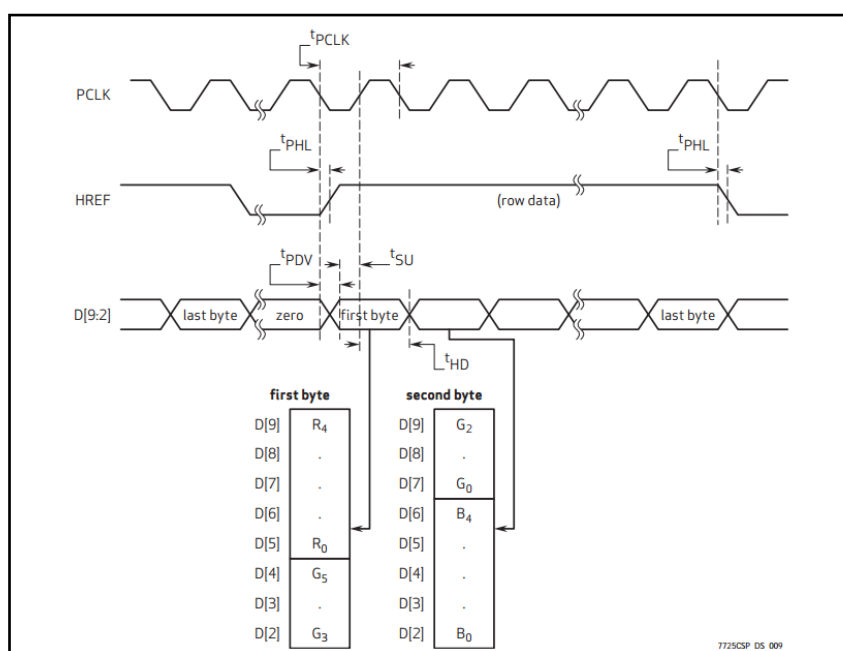


Figure 22-10 Pixel synchronization timing

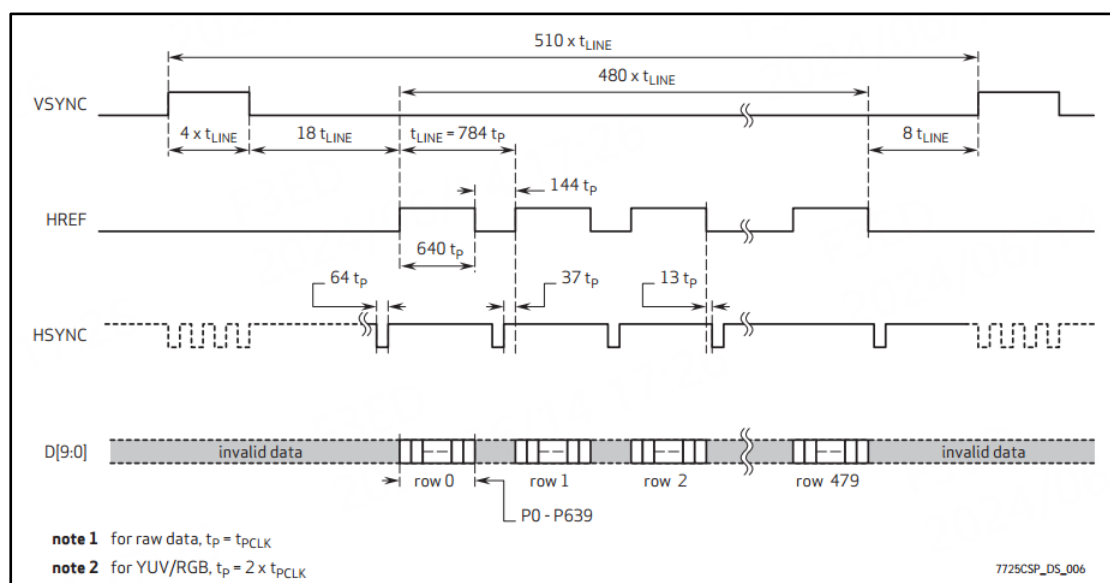


Figure 22-11 VGA frame image synchronization timing

22.3 Practical exercises

22.3.1 Experimental objectives

The development board collects image data through OV7725 and displays it on the LCD display.

22.3.2 Hardware resources

The hardware resources of the OV7725 camera have been introduced in detail in the previous part. This part explains the camera interface on the development board. The camera interface on the Haoyue development board is two rows of in-line interfaces with a width of 10. The physical diagram and schematic diagram of the camera interface are shown in Figure 22-12 and Figure 22-13

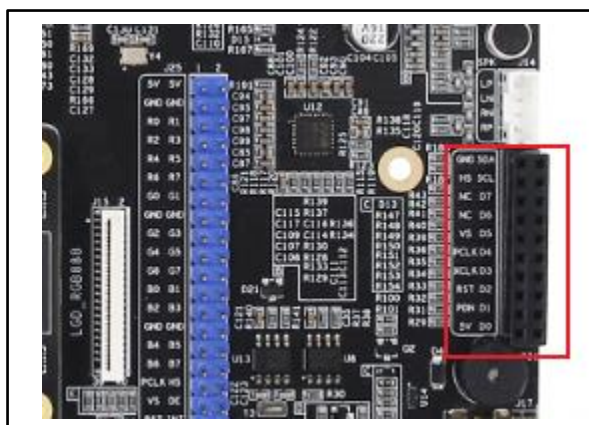


Figure 22-12 Actual picture of camera interface

摄像头接口

OV5640_I2C地址(7bit): 0x3C
OV7725_I2C地址(7bit): 0x21

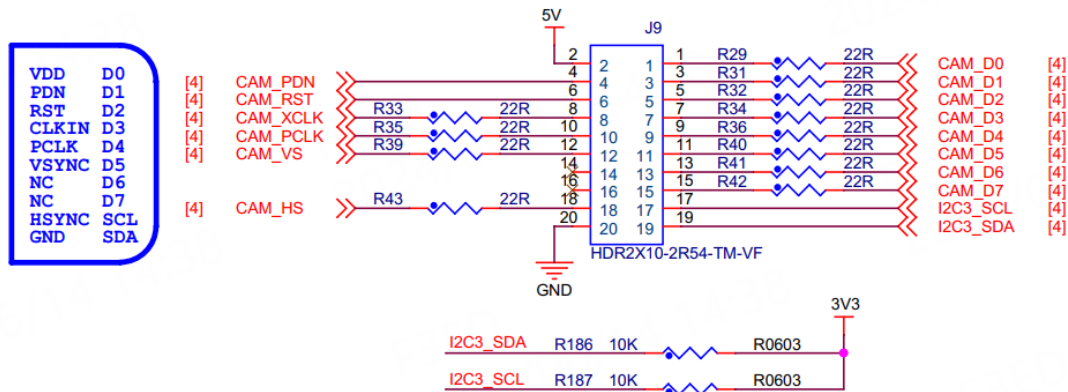


Figure 22-13 Camera interface schematic diagram

22.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 22-14.

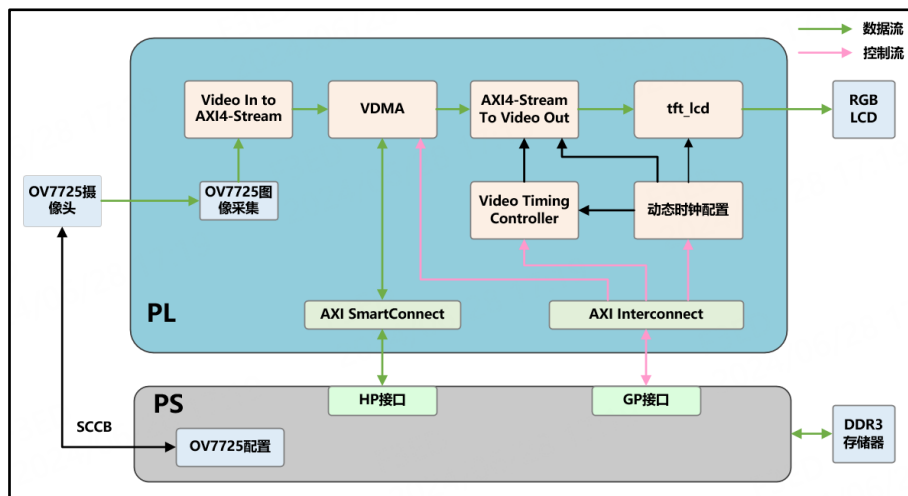


Figure 22-14 Overall block diagram

In this experiment, the camera is configured with registers through PS to make the camera work in a predetermined state. Since VDMA is a conversion between AXI-Stream format and AXI4 Memory Map format, the video stream data collected by the camera should be converted into AXI-Stream data stream format first. Here we use the Video in to AXI4-Stream IP core officially provided by Xilinx to convert the video stream format into AXI-Stream data stream format, and then send it to VDMA. VDMA caches data into DDR through the AXI-HP interface. When display is needed, VDMA reads the data from the DDR and sends it to the AXI4-Stream to Video Out IP core. The AXI4-Stream to Video Out IP core, under the control of the VTC IP core, converts the AXI4-Stream format data into the video output data format (such as RGB888), and connects the output video data stream to the input end of the tft_lcd IP core.

2. Hardware design

The hardware platform for this experiment was built based on the experiment in the chapter "VDMA Driven LCD ZYNQ system ip

Modify the ZYNQ7 processing system and enable the two pins of EMIO to connect to the S CCB interface of the camera. The configuration is as follows:

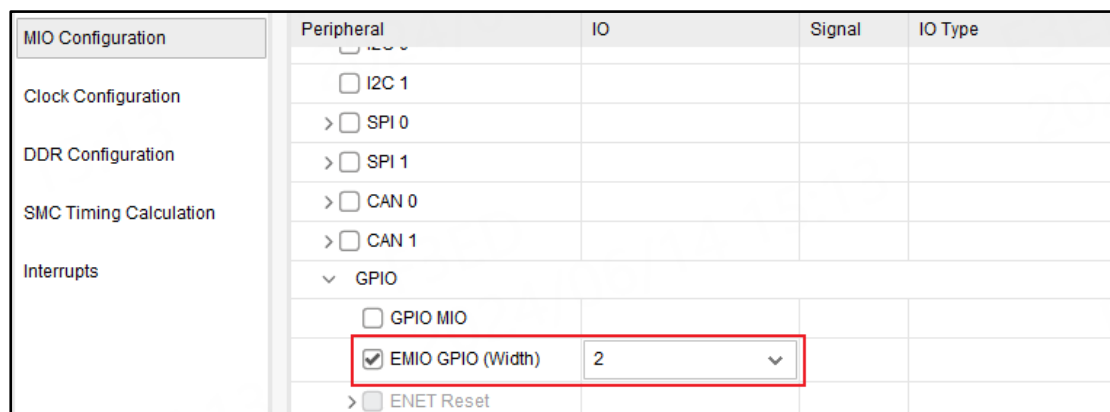


Figure 22-15 Enabling EMIO

Next, you need to export the port and name it "cam_sccb".

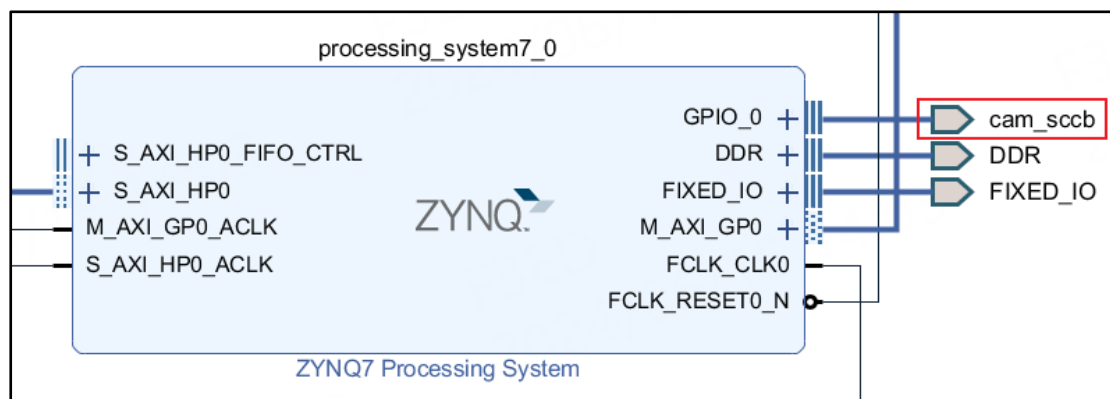


Figure 22-16 ZYNQ7 system module

Modify VDMA IP

The image shows the 'Basic' tab of the AXI VDMA configuration interface. The 'Address Width (32-64)' is set to 32 bits. 'Frame Buffers' is set to 3. Both 'Enable Write Channel' and 'Enable Read Channel' are checked. Under 'Write Channel Options', 'Memory Map Data Width' is 64, 'Write Burst Size' is 64, 'Stream Data Width (Auto)' is 32, and 'Line Buffer Depth' is 2048. Under 'Read Channel Options', 'Memory Map Data Width' is 64, 'Read Burst Size' is 64, 'Stream Data Width' is 24, and 'Line Buffer Depth' is 2048. Red boxes highlight the 'Frame Buffers' dropdown, the 'Enable Write Channel' checkbox, and the 'Write Channel Options' dropdowns.

Figure 22-17 AXI VDMA configuration interface

In the chapter "VDMA driving LCD display", we discussed the frame buffer mechanism of VDMA and its genlock mode. In order to prevent the read and write channels from accessing the same frame buffer at the same time, this experiment set the number of frame buffers to 3 and adopted the dynamic genlock mode. In addition, since the read channel needs to obtain the data written by the write channel in real time, the write channel is configured as Dynamic-Master and the read channel is configured as Dynamic-Slave in the experiment.

The genlock mode configuration of VDMA is as follows.

The image shows the 'Advanced' tab of the VDMA genlock configuration interface. 'Enable Asynchronous Mode (Auto)' is unchecked. 'Enable Single AXI4 Data Interface' is unchecked. 'Enable Vertical Flip' is checked. Under 'Write Channel Options', 'Fsync Options' is 's2mm tuser' and 'GenLock Mode' is 'Dynamic-Master'. Under 'Read Channel Options', 'Fsync Options' is 'None' and 'GenLock Mode' is 'Dynamic-Slave'. Both 'Allow Unaligned Transfers' checkboxes are unchecked. Red boxes highlight the 'GenLock Mode' dropdowns for both write and read channels.

Figure 22-18 VDMA genlock configuration

Add Video in to AXI4-Stream IP

Video in to AXI4-Stream IP core configuration interface is shown below:

Component Name		v_vid_in_axi4s_0
Pixels Per Clock	1	▼
Video Format	RGB	▼
Native Video Input Component Width	8	▼
AXI4S Video Output Component Width	8	▼
FIFO Depth	1024	▼
Clock Mode		
☐ Common ☒ Independent		

Figure 22-19 Video in to AXI4-Stream IP

Clock Mode The clock mode is used to specify the AXI4-Stream input and video output signals, whether to use a common clock or an independent clock for clock control. Here we check Independent, that is, use an independent clock for control. Other options can be kept as default.

Add custom ov7725_data IP core

Here we need to customize an IP core to implement video stream collection. The timing diagram of the video data stream flag signal format is as shown below:

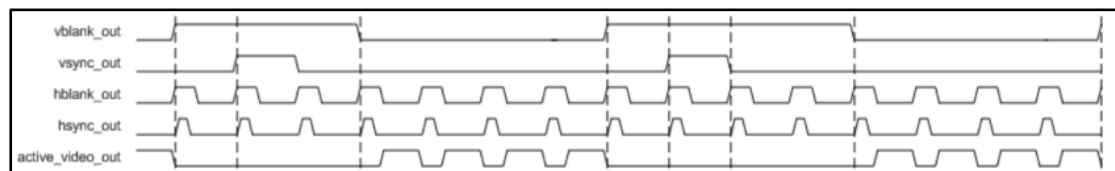


Figure 22-20 Video stream format

vblank_out (output vertical blank): vertical blank signal, when it is high level, it indicates that it is currently in the invalid data area in the vertical direction;

vsync_out (output vertical synchronization): vertical synchronization signal, used for frame synchronization;

hblank_out (output horizontal blank): horizontal blank signal, when it is high level, it indicates the invalid data area currently in the horizontal direction;

hsync_out (output horizontal synchronization): horizontal synchronization signal, used for line synchronization;

active_video_out: Image valid signal, valid at high level.

Since the camera only outputs vsync and href, two signals used to control whether the data is valid, we only output the field synchronization signal and image data valid signal of the above mentioned signals. In addition, we also need to output clock signals, clock valid signals and image data.

Code List 22-1 Video stream acquisition module reference code (ov7725_to_lcd.v)

```
1 module ov7725_to_lcd 2 (
```

```
3 input wire sys_rst_n , //Reset signal 4 // ov7725 5 input wire ov7725_pclk , //Camera pixel clock 6 input wire ov7725_href , //Camera horizontal synchronization signal 7 input wire ov7725_vsync , //Camera field synchronization signal 8 input wire [7:0] ov7725_data , //Camera image data 9 10 output cam_rst_n , //cmos reset signal 11 output cam_pwdn , 12 output xclk , 13 //RGB888 interface 14 output tft_clk , //clock signal 15 output tft_ce , //clock enable signal 16 17 output vsync , //field synchronization signal 18 output tft_de , //data valid signal 19 output [23:0] cnt_pic ; //Camera input field synchronization signal for shooting 37 reg ov7725_href_dly2 ; 38 reg [3:0] cnt_pic ; //Image frame counter 39 reg pic_valid ; //Frame valid flag signal 40 reg [7:0] pic_data_reg ; //Input 8-bit image data buffer 41 reg [15:0] data_out_reg ; //Output 16-bit image data buffer 42 reg data_flag ; //Input 8-bit image data cache 43 reg data_flag_dly1 ; //Image data splicing flag signal beat 44 45 //***** Main Code *****
// 46 //*****
// 47 //*****
// 48 49 //Do not reset the camera hardware, fixed high level 50 assign cam_rst_n = 1'b1; 51 assign cam_pwdn = 1'b0; 52 assign xclk = 1'b1; 53 assign tft_clk = ov7725_pclk; 54 55 //ov7725_vsync_dly: camera input field sync signal for shooting 56 always@(posedge ov7725_pclk or negedge sys_rst_n) 57 if(sys_rst_n == 1'b0) begin 58 ov7725_vsync_dly1 <= 1'b0; 59 ov7725_vsync_dly2 <= 1'b0; 60 61 ov7725_href_dly1 <= 1'b0; 62 ov7725_href_dly2 <= 1'b0; 63 end 64 else 65 begin
```

```

66 ov7725_vsync_dly1 <= ov7725_vsync; 67 ov7725_vsync_dly2 <= ov7725_vsync_dly1; 68
69 ov7725_href_dly1 <= ov7725_href; 70 ov7725_href_dly2 <= ov7725_href_dly1; 71 end
72 73
//pic_flag: frame image flag signal, each time it is pulled high, it represents a complete frame of i
mage
74 assign pic_flag = ((ov7725_vsync_dly1 == 1'b0) 75 && (ov7725_vsync == 1'b1)) ? 1'b
1 : 1'b0; 76
77 //cnt_pic: image frame counter
78 always@(posedge ov7725_pclk or negedge sys
_rst_n) 79 if(sys_rst_n == 1'b0) 80 cnt_pic <= 4'd0; 81 else if(cnt_pic < PIC_WAIT) 82 cnt_pic
<= cnt_pic + 1'b1; 83 else 84 cnt_pic <= cnt_pic; 85
86 //pic_valid: true valid flag
87 always@(p
osedge ov7725_pclk or negedge sys_rst_n) 88 if(sys_rst_n == 1'b0) 89 pic_valid <= 1'b0; 90 else
if((cnt_pic == PIC_WAIT) && (pic_flag == 1'b1)) 91 pic_valid <= 1'b1; 92 else 93 pic_valid
<= pic_valid; 94
95 //data_out_reg,pic_data_reg,data_flag: output 16-bit image data buffer
96 //i
nput 8-bit image data cache input 8-bit, image data cache
97 always@(posedge ov7725_pclk or n
egedge sys_rst_n) 98 if(sys_rst_n == 1'b0) 99 begin 100 data_out_reg <= 16'd0; 101 pic_data_re
g <= 8'd0; 102 data_flag <= 1'b0; 103 end 104 else if(ov7725_href == 1'b1) 105 begin 106 data_
flag <= ~data_flag; 107 pic_data_reg <= ov7725_data; 108 data_out_reg <= data_out_reg; 109 i
f(data_flag == 1'b1) 110 data_out_reg <= {pic_data_reg,ov7725_data}; 111 else 112 data_out_re
g <= data_out_reg; 113 end 114 else 115 begin 116 data_flag <= 1'b0; 117 pic_data_reg <= 8'd0;
118 data_out_reg <= data_out_reg; 119 end 120
121 //data_flag_dly1: image data cache
122 alw
ays@(posedge 127 128 //ov7725_data_out: output 16 Bit image data
129 assign tft_data = (pic_v
alid == 1'b1) ? {data_out_reg[15:11]

```

```

130 data_out_reg[10:5],2'd0,data_out_reg[4:0],
,3'd0,
3'd0} :

```

```

131         24'd0;
132
133 //ov7725_wr_en:输出 16 位图像数据使能
134 assign tft_de = (pic_valid == 1'b1) ? ov7725_href_dly2 : 1'b0;
135
136 assign tft_ce = pic_valid ? (data_flag_dly1 & tft_de)
137                        || (!tft_de) : 1'b0;
138 //输出帧有效信号
139 assign vsync = (pic_valid == 1'b1) ? ov7725_vsync_dly2 : 1'b0;
140
141 endmodule

```

We introduce the assignment of the clock enable signal `tft_ce` and the image data valid signal `tft_de`. The `tft_de` signal is always high during the row valid period, so after `pic_valid` is high, the camera's row valid signal is assigned to `tft_de`. We know that the actual output 24-bit RGB888 data is not always valid during the row validity period. This is controlled by the `tft_ce` signal and is divided into two situations: row valid and row invalid. The output image data is valid only when the `tft_ce` signal and the `tft_de` signal are high level at the same time.

After adding the custom IP core, lead out the pins that need to be connected to the top-level port:

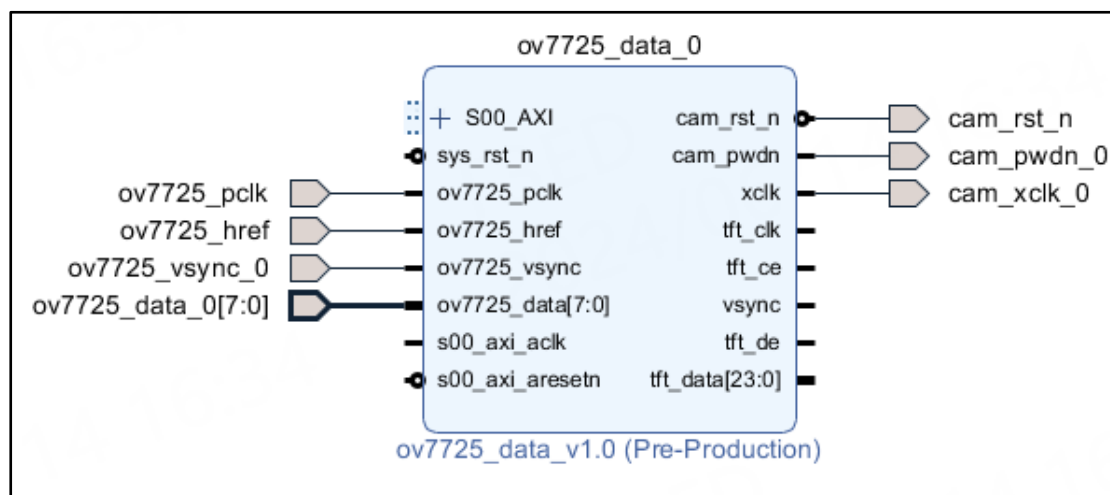


Figure 22-21 Leading out the camera external pins

The creation and configuration of all IP cores is completed here. Because Vivado's automatic wiring function cannot completely connect according to our needs, before we perform automatic wiring, we first manually connect some of the newly added IP core interfaces to avoid Vivado incorrect connections. The manual wiring is as shown in the figure below:

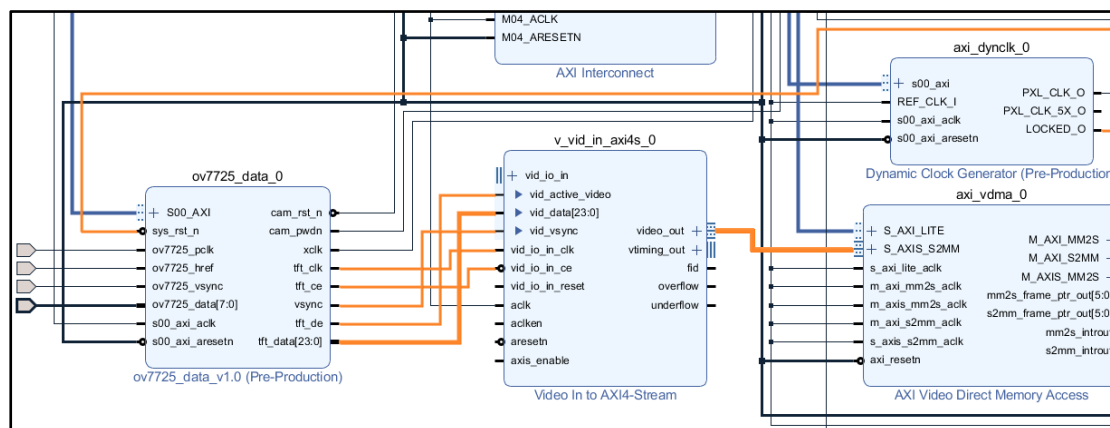


Figure 22-22 Manual connection

After the connection is completed, click "Run Connection Automation" at the top of the block diagram interface. The automatically connected modules and their interfaces are listed below, check "All Automation", and then click the "OK" button.

The final overall IP module connection diagram is shown in Figure 22-23.

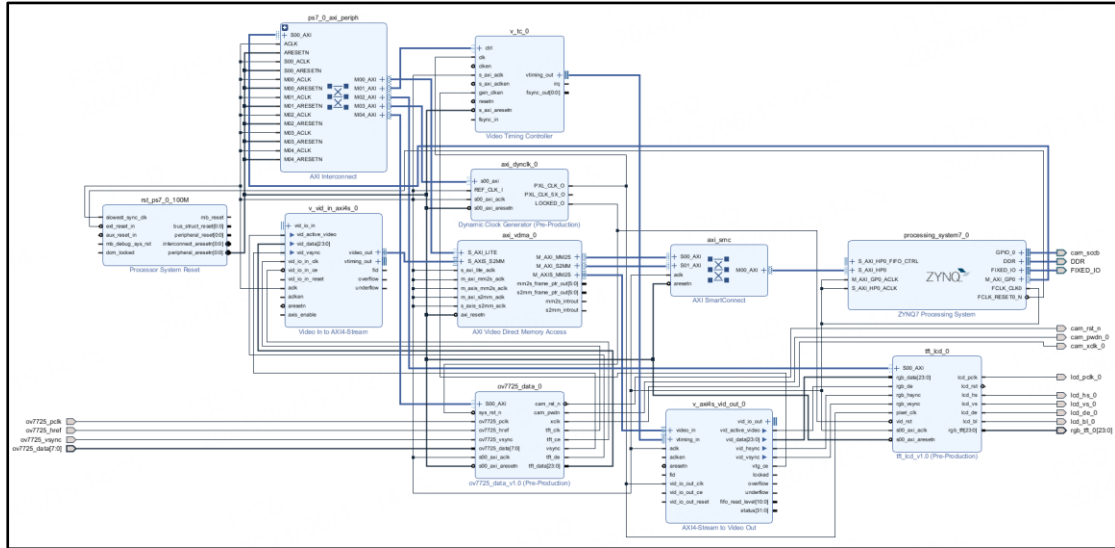


Figure 22-23 Overall IP block diagram

At this point, our Block Design is completed. Press the "F6" key to perform "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally, press the shortcut key "Ctrl + S" to save the design.

Then execute "Generate Output Products" and "Create HDL Wrapper" in sequence. Then perform pin constraints on the used pins.

Code Listing 22-2 Pin Constraints (vdma_lcd.xdc)

1	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[23]}}
2	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[22]}}
3	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[21]}}
4	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[20]}}
5	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[19]}}
6	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[18]}}
7	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[17]}}
8	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[16]}}
9	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[15]}}
10	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[14]}}
11	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[13]}}
12	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[12]}}
13	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[11]}}
14	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[10]}}
15	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[9]}}
16	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[8]}}
17	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[7]}}
18	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[6]}}
19	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[5]}}
20	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[4]}}
21	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[3]}}
22	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[2]}}
23	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[1]}}
24	set_property	IOSTANDARD	LVCOS33	[get_ports {rgb_tft_0[0]}}
25	set_property	PACKAGE_PIN	T15	[get_ports {rgb_tft_0[23]}}
26	set_property	PACKAGE_PIN	T14	[get_ports {rgb_tft_0[22]}}
27	set_property	PACKAGE_PIN	V13	[get_ports {rgb_tft_0[21]}}

```
28 set_property PACKAGE_PIN U13 [get_ports {rgb_tft_0[20]}] 29 set_property PACK
AGE_PIN U12 [get_ports {rgb_tft_0[19]}] 30 set_property PACKAGE_PIN T12 [get_po
rts {rgb_tft_0[18]}] 31 set_property PACKAGE_PIN T10 [get_ports {rgb_tft_0[17]}] 32
set_property PACKAGE_PIN T11 [get_ports {rgb_tft_0[16]}] 33 set_property PACKAG
E_PIN R14 [get_ports {rgb_tft_0[15]}] 34 set_property PACKAGE_PIN P14 [get_ports
{rgb_tft_0[14]}] 35 set_property PACKAGE_PIN P16 [get_ports {rgb_tft_0[13]}] 36 set
_property PACKAGE_PIN P15 [get_ports {rgb_tft_0[12]}] 37 set_property PACKAGE_
PIN U17 [get_ports {rgb_tft_0[11]}] 38 set_property PACKAGE_PIN T16 [get_ports {rg
b_tft_0[10]}] 39 set_property PACKAGE_PIN V18 [get_ports {rgb_tft_0[9]}] 40 set_pro
perty PACKAGE_PIN V17 [get_ports {rgb_tft_0[8]}] 41 set_property PACKAGE_PIN
R17 [get_ports {rgb_tft_0[7]}] 42 set_property PACKAGE_PIN R16 [get_ports {rgb_tft_
0[6]}] 43 set_property PACKAGE_PIN U19 [get_ports {rgb_tft_0[5]}] 44 set_property P
ACKAGE_PIN U18 [get_ports {rgb_tft_0[4]}] 45 set_property PACKAGE_PIN W19 [g
et_ports {rgb_tft_0[3]}] 46 set_property PACKAGE_PIN W18 [get_ports {rgb_tft_0[2]}]
47 set_property PACKAGE_PIN W16 [get_ports {rgb_tft_0[1]}] 48 set_property PACK
AGE_PIN V16 [get_ports {rgb_tft_0[0]}] 49 set_property PACKAGE_PIN T19 [get_por
ts lcd_bl_0] 50 set_property PACKAGE_PIN V15 [get_ports lcd_de_0] 51 set_property P
ACKAGE_PIN V12 [get_ports lcd_hs_0] 52 set_property PACKAGE_PIN W13 [get_por
ts lcd_vs_0] 53 set_property PACKAGE_PIN W15 [get_ports lcd_pclk_0] 54 set_proper
ty IOSTANDARD LVCMOS33 [get_ports lcd_bl_0] 55 set_property IOSTANDARD LVCMOS33 [get_ports l
cd_hs_0] 57 set_property IOSTANDARD LVCMOS33 [get_ports lcd_pclk_0] 58 set_pro
perty IOSTANDARD LVCMOS33 [get_ports lcd_vs_0] 59 60
```

```
61 #cam
62 set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets ov7725_pclk_IBUF]
63 set_property IOSTANDARD LVCMOS33 [get_ports ov7725_pclk] 64 set_property IOSTANDAR
D LVCMOS33 [get_ports {ov7725_data[7]}] 65 set_property IOSTANDARD LVCMOS33 [get_ports
{ov7725_data[6]}] 66 set_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[5]}] 67 set
_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[4]}] 68 set_property IOSTANDARD
LVCMOS33 [get_ports {ov7725_data[3]}] 69 set_property IOSTANDARD LVCMOS33 [get_ports {o
v7725_data[2]}] 70 set_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[1]}] 71 set_pr
operty IOSTANDARD LVCMOS33 [get_ports {ov7725_data[0]}] 72 set_property IOSTANDARD L
VCMOS33 [get_ports ov7725_href] 73 set_property IOSTANDARD LVCMOS33 [get_ports ov7725_
vsync] 74 set_property IOSTANDARD LVCMOS33 [get_ports cam_rst_n] 75 set_property PACKAG
E_PIN N15 [get_ports {ov7725_data[0]}] 76 set_property PACKAGE_PIN N16 [get_ports {ov7725_d
ata[1]}] 77 set_property PACKAGE_PIN L14 [get_ports {ov7725_data[2]}] 78 set_property PACKA
GE_PIN L15 [get_ports {ov7725_data[3]}] 79 set_property PACKAGE_PIN M14 [get_ports {ov7725
_data[4]}] 80 set_property PACKAGE_PIN M15 [get_ports {ov7725_data[5]}] 81 set_property PACK
AGE_PIN L16 [get_ports {ov7725_data[6]}] 82 set_property PACKAGE_PIN L17 [get_ports {ov772
5_data[7]}] 83 set_property PACKAGE_PIN M19 [get_ports ov7725_vsync] 84 set_property PACKA
GE_PIN M20 [get_ports ov7725_href] 85 set_property PACKAGE_PIN L19 [get_ports ov7725_pclk]
86 87 set_property PACKAGE_PIN M18 [get_ports cam_rst_n] 88 89 set_property PACKAGE_PIN T
17 [get_ports {cam_sccb_tri_io[0]}] 90 set_property PACKAGE_PIN R18 [get_ports {cam_sccb_tri_i
o[1]}] 91 set_property IOSTANDARD LVCMOS33 [get_ports {cam_sccb_tri_io[0]}] 92 set_property
IOSTANDARD LVCMOS33 [get_ports {cam_sccb_tri_io[1]}] 93
```

```
94 set_property IOSTANDARD LVCMOS33 [get_ports cam_pwdn_0] 95 set_property
PACKAGE_PIN M17 [get_ports cam_pwdn_0] 96 97 set_property IOSTANDARD LV
CMOS33 [get_ports cam_xclk_0] 98 set_property PACKAGE_PIN L20 [get_ports cam
_xclk_0]
```

Then generate a bit stream file and export the hardware (check "Include bitstream" in the pop-up dialog box). Finally launch the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and transplanted the following code into the newly created project.

display_ctrl	2024/6/15 10:35	文件夹	
dynclk	2024/6/15 10:35	文件夹	
ov7725	2024/6/15 10:35	文件夹	
sccb	2024/6/15 10:35	文件夹	
vdma_api	2024/6/15 10:35	文件夹	
lscript.ld	2024/6/15 10:34	LD 文件	7 KB
main.c	2024/6/15 11:58	C 源文件	2 KB

Figure 22-24 Schematic diagram

Transplant the above 5 folders into your own project, then create a new main.c file and add the following code

Code List 22-3 OV7725 LCD display reference code (main.c)

```
1 #include <stdio.h> 2 #include <stdlib.h> 3 #include <string.h> 4 #include "xil_types.h" 5 #include "xil_cache.h"
6 #include "xparameters.h" 7 #include "xaxivdma.h" 8 #include "xaxivdma_i.h" 9 #include "display_ctrl/display_ctr
l.h" 10 #include "ov7725/ov7725_init.h" 11 #include "vdma_api/vdma_api.h" 12 #include "sccb/sccb_init.h" 13
14 #define FB_Num 3 15 #define CLOCK_ADDR XPAR_AXI_DYNCLK_0_BASEADDR //clock base address 16
#define VDMA_DEVICE_ID XPAR_AXIVDMA_0_DEVICE_ID //Vdma id number 17 #define XVTC_DEVICE
_ID XPAR_VTC_0_DEVICE_ID //vtc id number 18 19 XAxiVdma Vdma; 20 DisplayCtrl Dispctrl; 21 VideoMod
e VIDEO_Mode; 22 23 /*Buffer memory base address*/ 24 unsigned int const FB_Base = (XPAR_PS7_DDR_0_S
_AXI_BASEADDR + 0x1000000); 25 26 int main(void) 27 { 28 /*Select lcd resolution*/ 29 VIDEO_Mode = VM
ODE_800x480; 30 31 SCCB_Init(); //Initialize sccb 32 OV7725_Init(); //Initialize ov7725 33
```



```

34  /*使能 vdma 读写模式*/
35  run_vdma_frame_buffer(&Vdma, VDMA_DEVICE_ID, VIDEO_Mode.width,
36                        VIDEO_Mode.height, FB_Base, 0, 0, BOTH);
37
38  /*清除 lcd 背景缓存*/
39  memset((unsigned int *)FB_Base, 0,
40         VIDEO_Mode.height*VIDEO_Mode.width*FB_Num*3);
41  Xil_DCacheFlush(); //更新缓冲
42  DisplayInitialize(&Dispctrl, XVTC_DEVICE_ID, CLOCK_ADDR);
43  DisplaySetMode(&Dispctrl, &VIDEO_Mode);
44  DisplayStart(&Dispctrl);
45
46  return 0;
47 }

```

In fact, the code is basically the same as before, with the addition of OV7725 initialization and SCCB initialization, and the VDMA mode is changed to support reading and writing at the same time.

In line 31 of the code, use the SCCB_Init() function to initialize the SCCB pin to communicate with the OV7725. Then call the OV7725_Init() function to configure the OV7725 register so that the camera can output the correct image format, such as resolution, frequency, etc.

Continue to use run_vdma_frame_buffer() in line 35 of the code to configure the read and write mode of vdma. The last parameter uses BOTH to indicate that it supports both reading and writing. Here we continue to set VIDEO_Mode manually, using 800x480 mode here.

Then clear the cache of the LCD background, and then use Xil_DCacheFlush() to refresh the Cache into the DDR.

Finally, use DisplaySetMode() to set the resolution of VTC, and DisplayStart() to start VTC.

Now we mainly look at the OV7725_Init() function:

Code List 22-4 OV7725 initialization reference code (ov7725_init.c)

```

1 #include "../sccb/sccb_init.h" 2 3 //OV7725 initialization 4 u8 OV77
25_Init(void) 5 { 6 u16 cam_id = 0; 7 //Read OV7725 camera ID 8 ca
m_id = sccb_read_reg8(0x0B); 9 cam_id |= sccb_read_reg8(0x0A)
<< 8; 10 xil_printf("cam_id = 0x%x\r\n", cam_id); 11 12 sccb_write_r
eg8 (0x3d, 0x03); 13 sccb_write_reg8 (0x15, 0x00); 14 sccb_write_r
eg8 (0x17, 0x23); 15 sccb_write_reg8 (0x18, 0xa0); 16 sccb_write_r
eg8 (0x19, 0x07); 17 sccb_write_reg8 (0x1a, 0xf0); 18 sccb_write_re
g8 (0x32, 0x00); 19 sccb_write_reg8 (0x29, 0xa0); 20 sccb_write_re
g8 (0x2a, 0x00); 21 sccb_write_reg8 (0x2b, 0x00); 22 sccb_write_re
g8 (0x2c, 0xf0); 23 sccb_write_reg8 24 sccb_write_reg8 (0x11, 0x00)
sccb_write_reg8 (0x4d, 0x09); 29 sccb_write_reg8 (0x63, 0xf0); 30 s
ccb_write_reg8 (0x64, 0xff);

```

```

31     sccb_write_reg8 (0x65, 0x00) ;
32     sccb_write_reg8 (0x66, 0x00) ;
33     sccb_write_reg8 (0x67, 0x00) ;
34     sccb_write_reg8 (0x13, 0xff) ;
35     sccb_write_reg8 (0x0f, 0xc5) ;
36     sccb_write_reg8 (0x14, 0x11) ;
37     sccb_write_reg8 (0x22, 0x98) ;
38     sccb_write_reg8 (0x23, 0x03) ;
39     sccb_write_reg8 (0x24, 0x40) ;
40     sccb_write_reg8 (0x25, 0x30) ;
41     sccb_write_reg8 (0x26, 0xa1) ;
42     sccb_write_reg8 (0x6b, 0xaa) ;
43     sccb_write_reg8 (0x13, 0xff) ;
44     sccb_write_reg8 (0x90, 0x0a) ;
45     sccb_write_reg8 (0x91, 0x01) ;
46     sccb_write_reg8 (0x92, 0x01) ;
47     sccb_write_reg8 (0x93, 0x01) ;
48     sccb_write_reg8 (0x94, 0x5f) ;
49     sccb_write_reg8 (0x95, 0x53) ;
50     sccb_write_reg8 (0x96, 0x11) ;
51     sccb_write_reg8 (0x97, 0x1a) ;
52     sccb_write_reg8 (0x98, 0x3d) ;
53     sccb_write_reg8 (0x99, 0x5a) ;
54     sccb_write_reg8 (0x9a, 0x1e) ;
55     sccb_write_reg8 (0x9b, 0x3f) ;
56     sccb_write_reg8 (0x9c, 0x25) ;
57     sccb_write_reg8 (0x9e, 0x81) ;
58     sccb_write_reg8 (0xa6, 0x06) ;
59     sccb_write_reg8 (0xa7, 0x65) ;
60     sccb_write_reg8 (0xa8, 0x65) ;
61     sccb_write_reg8 (0xa9, 0x80) ;
62     sccb_write_reg8 (0xaa, 0x80) ;
63     sccb_write_reg8 (0x7e, 0x0c) ;
64     sccb_write_reg8 (0x7f, 0x16) ;
65     sccb_write_reg8 (0x80, 0x2a) ;
66     sccb_write_reg8 (0x81, 0x4e) ;
67     sccb_write_reg8 (0x82, 0x61) ;
68     sccb_write_reg8 (0x83, 0x6f) ;
69     sccb_write_reg8 (0x84, 0x7b) ;
70     sccb_write_reg8 (0x85, 0x86) ;
71     sccb_write_reg8 (0x86, 0x8e) ;
72     sccb_write_reg8 (0x87, 0x97) ;
73     sccb_write_reg8 (0x88, 0xa4) ;
74     sccb_write_reg8 (0x89, 0xaf) ;
75     sccb_write_reg8 (0x8a, 0xc5) ;
76     sccb_write_reg8 (0x8b, 0xd7) ;
77     sccb_write_reg8 (0x8c, 0xe8) ;
78     sccb_write_reg8 (0x8d, 0x20) ;
79     sccb_write_reg8 (0x0e, 0x65) ;
80     sccb_write_reg8 (0x09, 0x00) ;
81     return 0;
82
83 }

```

The relevant camera configuration is given here. Readers can refer to the previous OV7725 register table to modify the corresponding registers to output different resolutions. The resolution here is fixed at 640*480.

22.3.4 Board verification

As shown in Figure 22-25, the development board is connected to the 12V DC power supply, XILINX downloader JTAG port, LCD screen and OV7725 camera. After the lines are connected correctly, turn on the switch to power on the board.

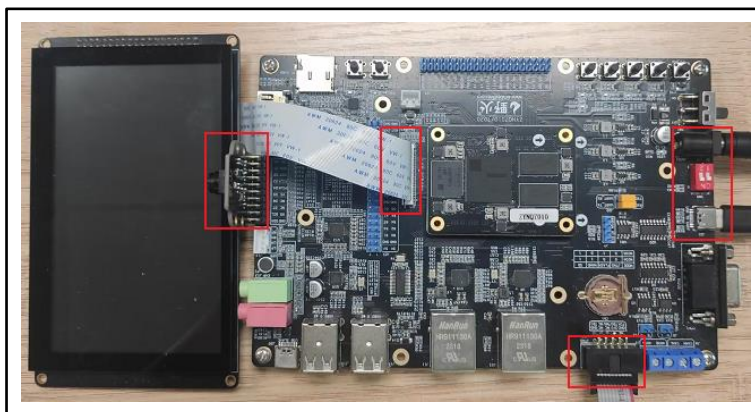


Figure 22-25 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system and Program FPGA -> Run.

After the program download is completed, the image collected by the OV7725 camera will be displayed on the LCD display. The resolution of the collected image is 640*480, as shown in Figure 22-26.

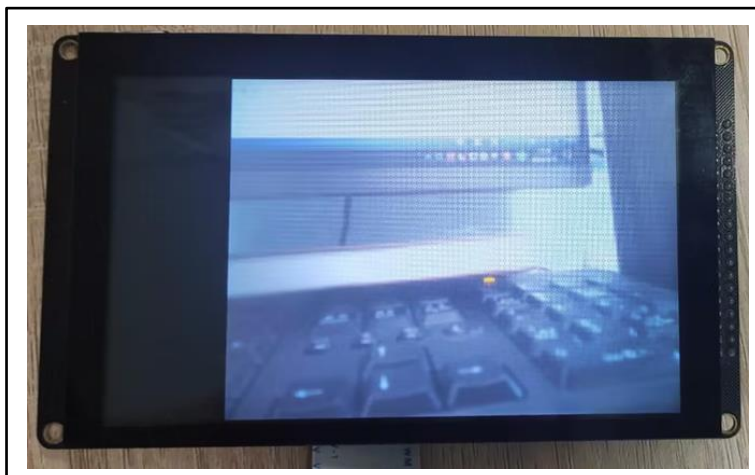


Figure 22-26 OV7725+LCD image display

22.4 End of chapter summary

This chapter introduces the relevant knowledge of the OV7725 camera in detail, and combines the OV7725 camera with the LCD to realize the real-time display of the images collected by the OV7725. This chapter is the tip of the iceberg of knowledge about the use of the OV7725. The OV7725 has more powerful functions. Readers can refer to the data manual for in-depth learning.

22.5 Development training

Consult the OV7725 camera data manual to understand the functions of related registers under different configurations, change the register configuration, and realize functions such as up and down flipping, left and right mirroring of the captured image, and display it on the LCD monitor in real time.

Chapter 23 OV5640 LCD Display

23.1 Chapter Introduction

In the previous chapters, we learned the relevant knowledge of the camera, introduced the OV7725 camera in detail, and implemented the display of the OV7725-LCD.

In this chapter, we will learn the relevant knowledge about another higher-end camera OV5640, and implement the real-time display of OV5640-LCD.

23.2 Theoretical study

23.2.1 OV5640 Introduction

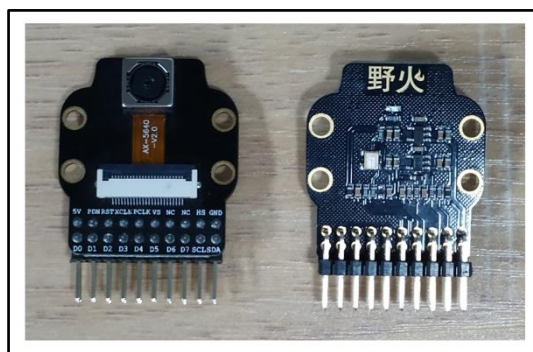


Figure 23-1 OV5640 physical picture

The camera is mainly composed of lens, image sensor, on-board circuit and signal pins below. The lens component includes a lens holder and a convex lens that can be rotated to adjust the distance. The focal length can be adjusted by rotating. During normal use, the lens holder is covered on the circuit board to block light. The light can only be transmitted to the image sensor in the center through the lens. The image data it collects is directly transmitted to the FPGA chip. The FPGA caches the received image data into DDR3 SDRAM. In the VGA image display effective area, the VGA driver reads the image data from the DDR3 SDRAM and displays it on the display.

The image sensor is the core component of the camera. The image sensor in the above camera is a CMOS type digital image sensor model OV5640. The sensor supports the output of images up to 5 million pixels (2592x1944 resolution), and supports the use of VGA timing to output image data. The data format of the output image supports YUV (422/420), YCbCr422, RGB565 and JPEG formats. Directly outputting images in JPEG format can greatly reduce the amount of data and facilitate network transmission. It can also compensate the collected images and support basic processing such as gamma curve, white balance, saturation, and chroma. According to different resolution configurations, the frame rate of the sensor output image data is adjustable from 15-60 frames, and the working power is between 150mW-200mW.

23.2.2 OV5640 pins and functional framework

The OV5640 module has an autofocus function. The pin distribution is shown in the figure below:

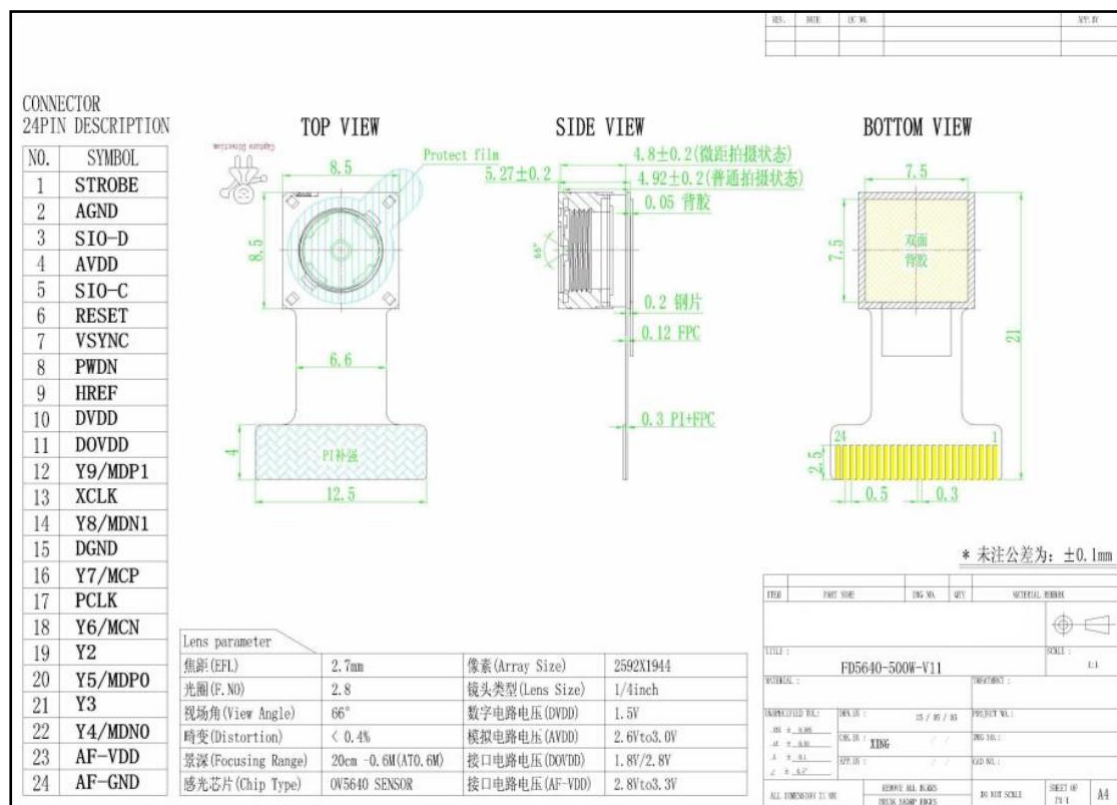


Figure 23-2 OV5640 sensor pin distribution diagram

The signal pin functions are introduced as follows in Table 23-1.

Table 23-1 OV5640 pins

管脚名称	管脚类型	管脚描述
SIO_C	输入	SCCB 总线的时钟线, 可类比 I2C 的 SCL
SIO_D	I/O	SCCB 总线的数据线, 可类比 I2C 的 SDA
RESET	输入	系统复位管脚, 低电平有效
PWDN	输入	掉电/省电模式, 高电平有效
HREF	输出	行同步信号
VSYN	输出	帧同步信号
PCLK	输出	像素同步时钟输出信号
XCLK	输入	外部时钟输入端口, 可接外部晶振
Y2,,Y9	输出	像素数据输出端口

Below we explain these signal pins in conjunction with the OV5640 functional block diagram in Figure 23-3.

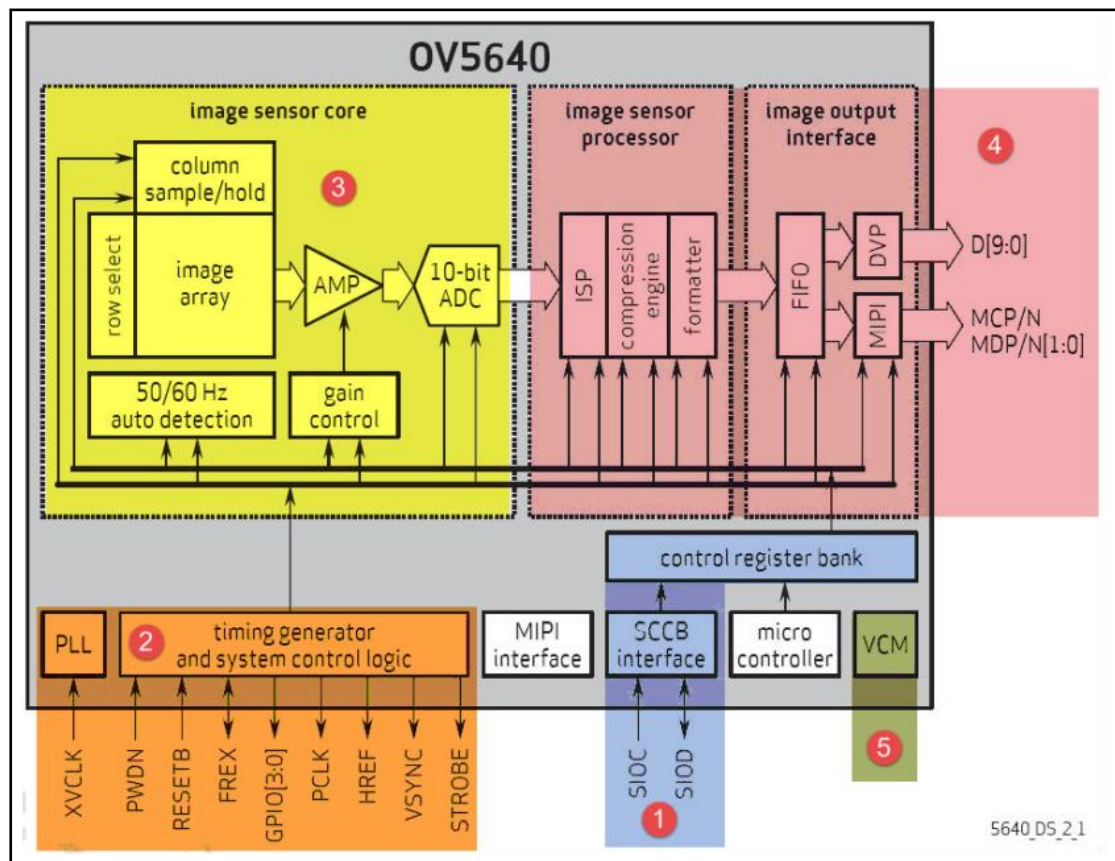


Figure 23-3 OV5640 functional block diagram

(1) Control register

Labeled 1 is the control register of OV5640, which operates according to the parameters configured in these registers, and these parameters are written by the external controller through the SIO_C and SIO_D pins. The communication protocol used by SIO_C and SIO_D is very similar to I2C.

(2) Communication, control signals and clocks

The label 2 contains the communication, control signals and external clock of OV5640. PCLK, HREF and VSYNC are the pixel synchronization clock, horizontal synchronization signal and frame synchronization signal respectively, which are very similar to the signals in LCD screen control. When the RESETB pin is low, it is used to reset the entire sensor chip, and PWDN is used to control the chip to enter low-power mode. Pay attention to the last XCLK pin, which is completely different from PCLK. XCLK can be connected to an external crystal oscillator or provided by an external controller.

(3) Photosensitive matrix

Labeled 3 is the photosensitive matrix, where optical signals are converted into electrical signals. After various processing, these signals are stored into digital images represented by pixels.

(4) Data output signal

The label 4 contains the DSP processing unit, which will do some basic image processing operations according to the configuration of the control register. This part also includes the image format conversion unit and compression unit. The converted data is finally imported through Y0-Y9.

Pin output, generally speaking, we use 8 data lines for transmission. At this time, only Y2-Y9 pins are used. The connection method between OV5640 and external devices is shown in Figure 23-4.

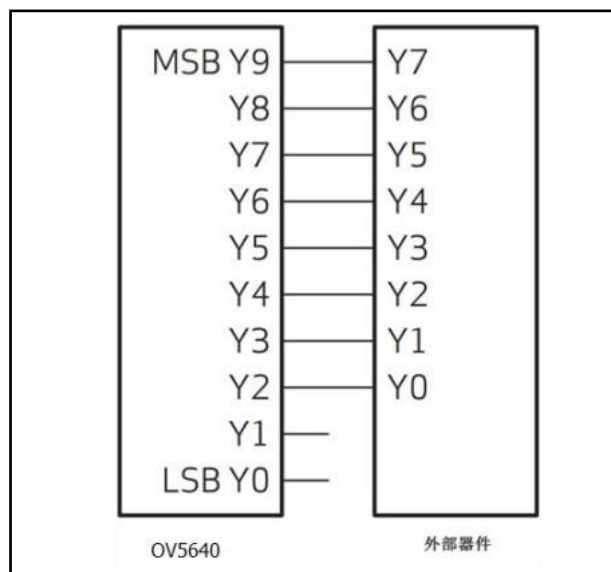


Figure 23-4 8-bit data line connection method

(5) Data output signal

Marked ⑤ is the VCM processing unit, which will realize the automatic focus function of the image through image analysis. To achieve autofocus, you also need to download autofocus firmware to the module.

23.2.3 Registers of OV5640

Controlling OV5640 involves many registers. You can directly query "ov5640datasheet" to learn about it. Through the configuration of these registers, you can control the resolution, image format and image direction of its output image, etc. It should be noted that the OV5640 register address is 16 bits.

The official also provides a document "OV5640_Autofocus Camera Module Application Guide (DVP_Interface)_R2.13C.pdf", which provides configuration examples for different configuration requirements, see Figure 23-5. Among them, write_SCCB is a function that uses SCCB to write data to the register. The first parameter is the address of the register to be written, and the second parameter is the content to be written.

4.1.2 VGA 预览

VGA 30 帧/秒

// YUV VGA 30fps, night mode 5fps

// Input Clock = 24Mhz, PCLK = 56MHz

write_i2c(0x3035, 0x11); // PLL

write_i2c(0x3036, 0x46); // PLL

write_i2c(0x3c07, 0x08); // light meter 1 threshold [7:0]

write_i2c(0x3820, 0x41); // Sensor flip off, ISP flip on

write_i2c(0x3821, 0x07); // Sensor mirror on, ISP mirror on, H binning on

Figure 23-5 Example of register configuration for adjusting frame rate

Pixel data output timing

The OV5640 is controlled using the SCCB protocol, and it uses VGA timing when outputting images (SVGA and UXGA can also be used, these timings are similar), which is very similar to controlling the LCD screen to input images. When OV5640 outputs an image, it outputs it frame by frame. The data in the frame is generally output from left to right, from top to bottom, pixel by pixel (the direction can also be modified through the register).

As shown in Figure 23-6, if we use the Y2-Y9 data line and the image format is set to RGB565, when performing data output, the Y2-Y9 data line will send a 1-byte data signal driven by 1 pixel synchronization clock PCLK, so 2 PCLK clocks can send 1 pixel data in RGB565 format. Pixel data is transmitted in sequence. When each row of data is transmitted, the horizontal synchronization signal HREF will output a level jump signal. When each frame of image is transmitted, VSYNC will output a level jump signal.

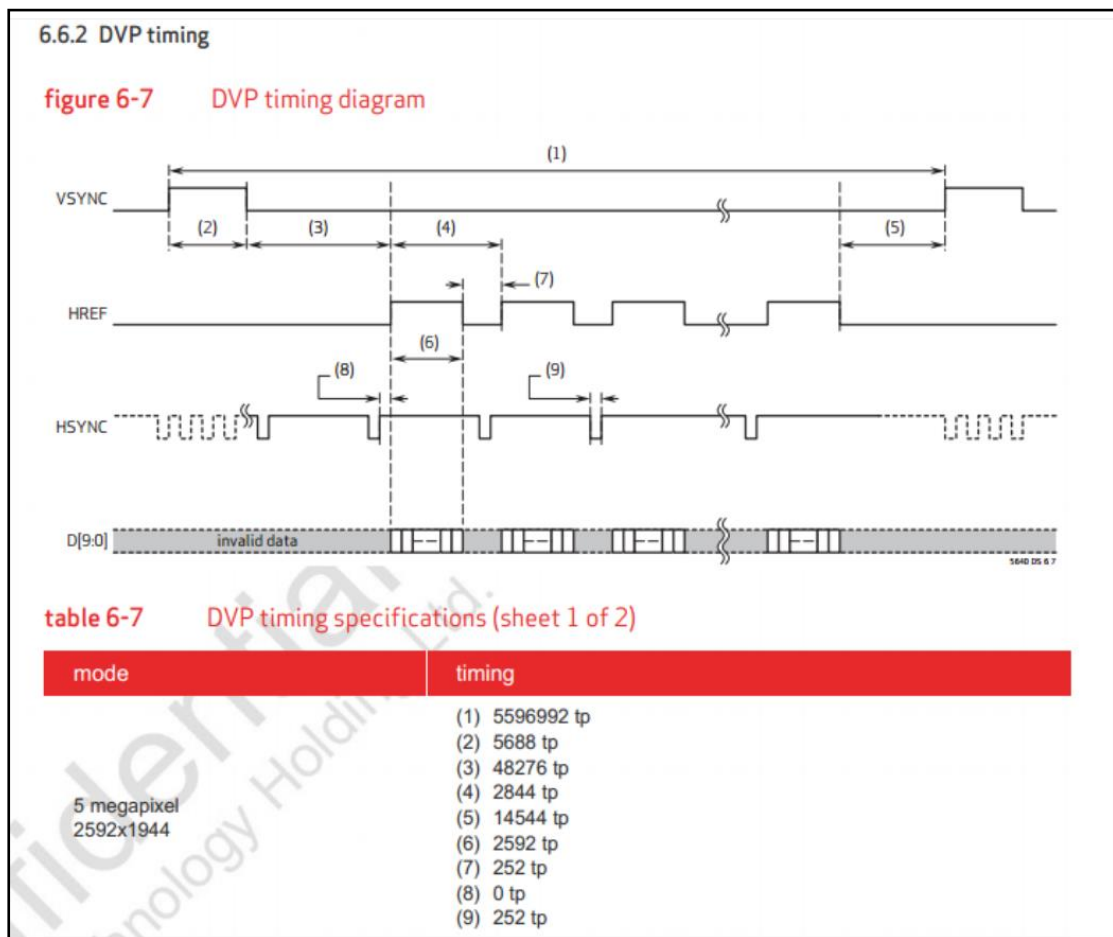


Figure 23-6 DVP interface timing sequence

23.3 Practical exercises

23.3.1 Experimental objectives

The development board collects image data through OV5640 and displays it on the LCD display.

23.3.2 Hardware resources

See the "Hardware Resources" section in the "OV7725 LCD Display" chapter.

23.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 23-7.

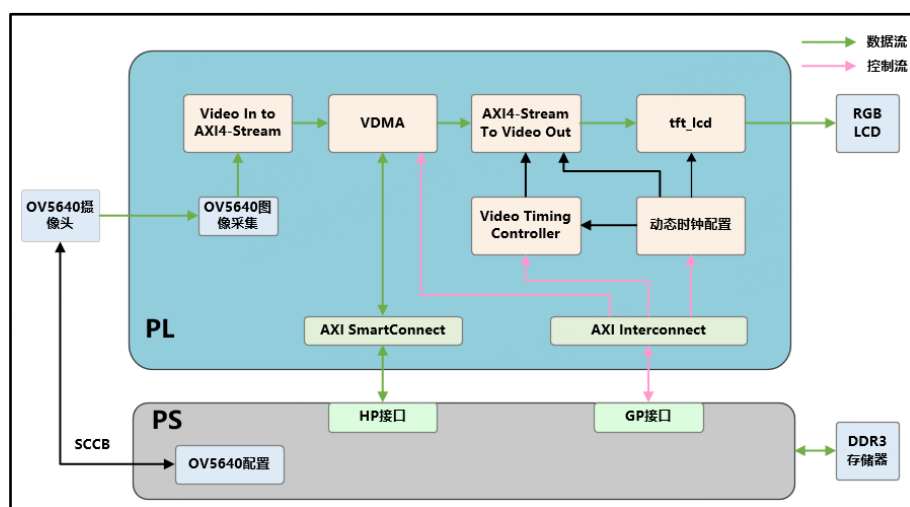


Figure 23-7 Overall block diagram

The entire framework description is the same as the experiment in the "OV7725 LCD Display" chapter, except that OV7725 is replaced by OV5640.

2. Hardware design

The hardware platform of this experiment was built on the basis of the experiment in the "OV7725 LCD Display" chapter. The custom OV5640 IP core is only different in name.

The final overall IP block diagram is shown in Figure 23-8.

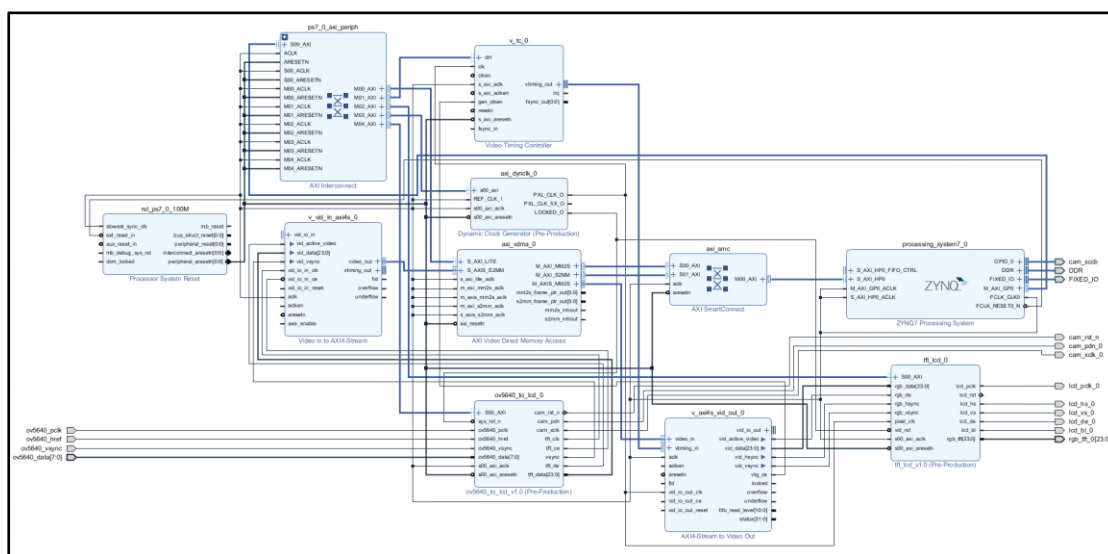


Figure 23-8 Overall IP block diagram

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and transplanted the following code into the newly created project.

名称	修改日期	类型	大小
display_ctrl	2024/6/15 16:08	文件夹	
dynclk	2024/6/15 16:08	文件夹	
ov5640	2024/6/15 16:08	文件夹	
sccb	2024/6/15 16:08	文件夹	
vdma_api	2024/6/15 16:08	文件夹	
lscript.ld	2024/6/15 16:56	LD 文件	7 KB
main.c	2024/6/15 17:15	C 源文件	2 KB

Figure 23-9 Copy the folder to your own project

Migrate the above 5 folders into your own project, then create a new main.c file and add the following code.

Code List 23-1 OV5640 LCD display reference code (main.c)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include "xil_types.h"
5 #include "xil_cache.h"
6 #include "xparameters.h"
7 #include "xaxivdma.h"
8 #include "xaxivdma_i.h"
9 #include "display_ctrl/display_ctrl.h"
10 #include "vdma_api/vdma_api.h"
11 #include "sccb/sccb_init.h"
12 #include "ov5640/ov5640_init.h"
13 #include "ov5640/ov5640_AF.h"
14
15 #define CLOCK_ADDR      XPAR_AXI_DYNCLK_0_BASEADDR
16 #define VDMA_ID         XPAR_AXIVDMA_0_DEVICE_ID
17 #define XVTC_DEVICE_ID  XPAR_VTC_0_DEVICE_ID
    
```

```
18
19 /* 缓冲内存基地址 */
20 unsigned int const FB_Base = (XPAR_PS7_DDR_0_S_AXI_BASEADDR + 0x1000000);
21
22 XAxiVdma      vdma;
23 DisplayCtrl   Dispctrl;
24 VideoMode     VIDEO_Mode;
25
26 int main(void)
27 {
28     /* 选择lcd 分辨率 */
29     VIDEO_Mode = VMODE_800x480;
30     ul6 cam_h_disp = 800; // ov5640 输出水平像素点
31     ul6 cam_v_disp = 480; // ov5640 输出垂直像素点
32
33     SCCB_Init(); //初始化 sccb
34     OV5640_Init(cam_h_disp, cam_v_disp); //初始化
35     OV5640_AUTO_FOCUS(); // 自动对焦
36
37     // 配置VDMA
38     run_vdma_frame_buffer(&vdma, VDMA_ID, VIDEO_Mode.width,
39                          VIDEO_Mode.height, FB_Base, 0, 0, BOTH);
40
41     DisplayInitialize(&Dispctrl, XVTC_DEVICE_ID, CLOCK_ADDR);
42     DisplaySetMode(&Dispctrl, &VIDEO_Mode);
43     DisplayStart(&Dispctrl);
44
45     return 0;
46 }
```

The above code is actually basically the same as "OV7725 LCD Display", except that the OV7725_Init() is replaced by the OV5640_Init() function, and the autofocus function is added. The resolution supports 800x480. Readers who need to modify the resolution also need to Modify the register configuration in OV5640_Init().

To modify the output mode of OV5640, readers can modify it in OV5640_Init() and refer to the relevant register description of OV5640 to modify it.

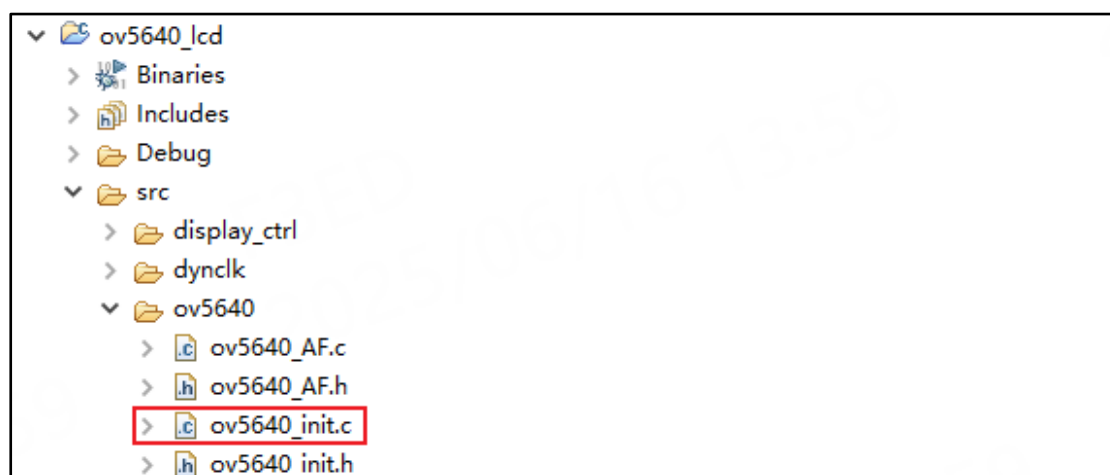


Figure 23-10 OV5640_Init()

23.3.4 Board verification

As shown in Figure 23-11, the development board is connected to the 12V DC power supply, XILINX downloader JTAG port, LCD screen and ov5640 camera. After the lines are connected correctly, turn on the switch to power on the board.

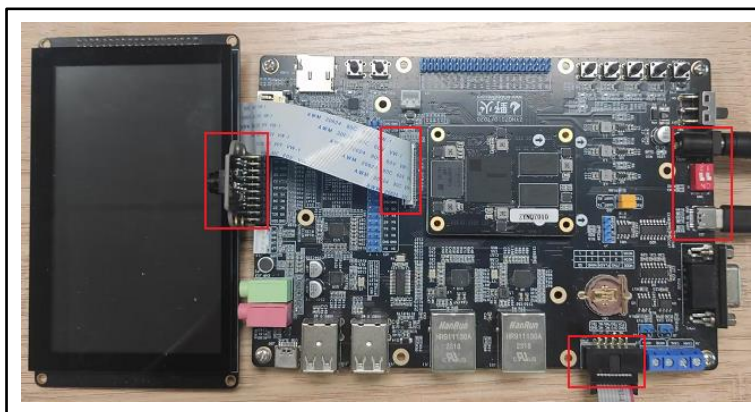


Figure 23-11 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system and Program FPGA -> Run.

After the program download is completed, the image collected by the OV5640 camera will be displayed on the LCD display with a resolution of 800*480, as shown in Figure 23-12.

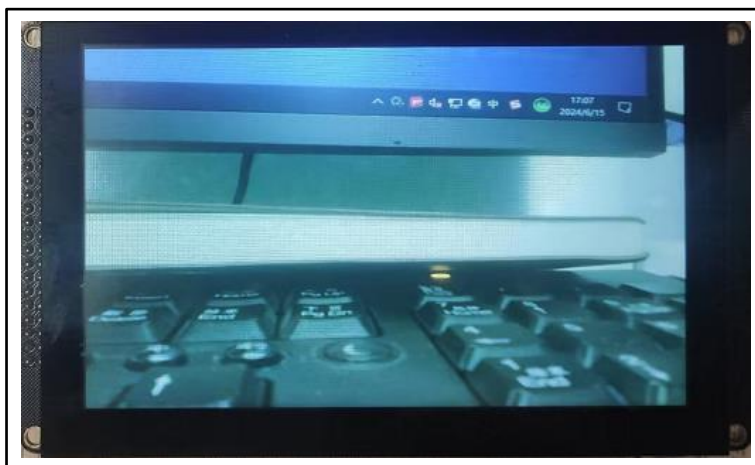


Figure 23-12 OV5640+LCD image display

23.4 End of chapter summary

This chapter introduces the relevant knowledge of the OV5640 camera, and combines the OV5640 camera with HDMI to realize the real-time display of the images collected by the OV5640. This chapter is the tip of the iceberg of knowledge about the use of the OV5640. The OV5640 has more powerful functions. Readers should refer to the data manual for in-depth learning.

23.5 Development training

Consult the OV5640 camera data manual to understand the functions of the relevant registers under different configurations, change the register configuration, and realize functions such as up and down flipping, left and right mirroring of the captured image, and display them in real time on the LCD monitor.

Chapter 24 OV7725 HDMI Display

24.1 Chapter Introduction

In the "OV7725 LCD Display" chapter, we implemented the OV7725 camera to collect images and display them on the LCD. In this chapter, we do not use the LCD display, but use a display screen with an HDMI interface.

24.2 Theoretical study

For the theoretical study of OV7725 and HDMI in this experiment, readers can refer to the previous chapters, and I will not go into details here.

24.3 Practical exercises

24.3.1 Experimental objectives

The development board collects image data through OV7725 and displays it on the HDMI display.

24.3.2 Hardware resources

This experiment used the HDMI interface, OV7725 camera interface, and UART on the PL side.

24.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 24-1.

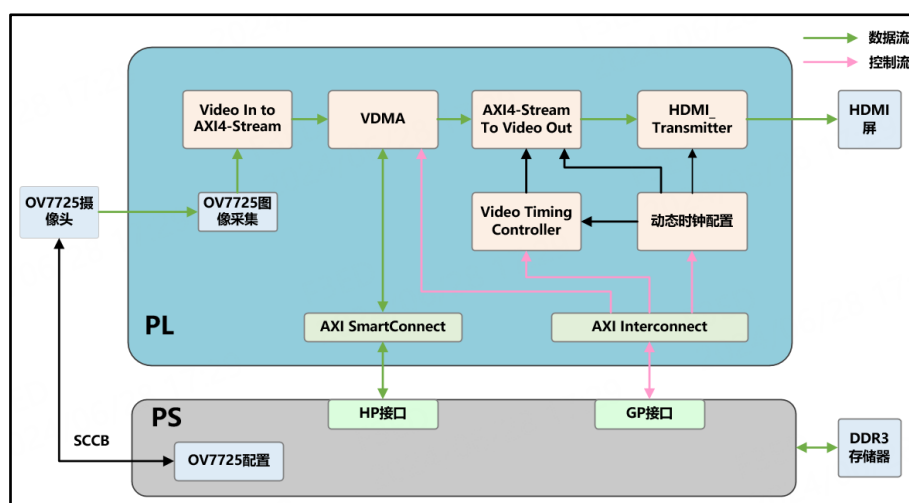


Figure 24-1 Overall block diagram

In practical applications, the overall block diagram is actually the same as "OV7725 LCD Display", except that it is replaced by HDMI display, and the tft_lcd IP core is replaced by HDMI_Transmitter IP. There are no other changes.

2. Hardware design

The hardware platform of this experiment was built on the basis of the "OV7725 LCD Display" experiment. Except for changing the tft_lcd IP core to the hdmi_transmitter IP core and modifying the dynamic clock configuration, no other changes are required.

Modify dynamic clock

ov7725_hdmi > ip_repo > axi_dynclk_v1_0 > src			
名称	修改日期	类型	大小
axi_dynclk.vhd	2024/2/28 9:54	硬盘映像文件	11 KB
axi_dynclk_S00_AXI.vhd	2024/2/28 9:54	硬盘映像文件	19 KB
mmcme2_drp.v	2024/2/28 9:54	V 文件	16 KB

Figure 24-2 Locate the path where axi_dynclk.vhd is located

Modify BUFIO to BUFG:

197	-- Add user logic here
198	BUFIO_inst : BUFG
199	port map (
200	0 => PXL_CLK_5X_0, -- 1-bit output: Clock output (connect to I/O clock loads).
201	I => bufio_in -- 1-bit input: Clock input (connect to an IBUF or BUFMR).
202	);
203	

Figure 24-3 Modify axi_dynclk.vhd

Add custom hdmi_transmitter IP core

For the files used by this IP core, please refer to the "HDMI Color Display" chapter of the ZYNQ PL side tutorial. Here we only list the top-level code:

Code List 24-1 hdmi_transmitter core top-level reference code (hdmi_transmitter.v)

```

01 module hdmi_transmitter(
02     input        pclk,           // pixel clock
03     input        pclk_x5,       // pixel clock x5
04     input        rst_n,         // reset
05
06     input [23:0] video_din,      // RGB888 video in
07     input        video_hsync,    // hsync data
08     input        video_vsync,    // vsync data
09     input        video_de,       // data enable
10
11     output        tmds_clk_p,     // TMDs 时钟通道
12     output        tmds_clk_n,
13     output [2:0]  tmds_data_p,    // TMDs 数据通道
14     output [2:0]  tmds_data_n,
15     output        tmds_oen        // TMDs 输出使能
16
17 );
18
19
20 wire [9:0]  blue_10bit;
21 wire [9:0]  green_10bit;
22 wire [9:0]  red_10bit;

```

```
23 24 25 assign tmds_oen = 1'b1; 26 27 28 29 //Encode three color channels 30 en
code encode_b ( 31 .sys_clk (pclk), 32 .sys_rst_n (rst_n), 33 .data_in (video_din[7:
0]), 34 .c0 (video_hsync), 35 .c1 (video_vsync), 36 .de (video_de), 37 38 .data_out
(blue_10bit) 39 ); 40 41 encode encode_g ( 42 .sys_clk (pclk), 43 .sys_rst_n (rst_n
), 44 .data_in (video_din[15:8]), 45 .c0 (video_hsync), 46 .c1 (video_vsync), 47 .d
e (video_de), 48 49 .data_out (green_10bit) 50 ); 51 52 encode encode_r ( 53 .sys
_clk (pclk), 54 .sys_rst_n (rst_n), 55 .data_in (video_din[23:16]), 56 .c0 (video_hs
ync), 57 .c1 (video_vsync), 58 .de (video_de), 59 60 .data_out (red_10bit) 61 ); 62
63 //Convert the encoded data to parallel to serial 64 par_to_ser par_to_ser_b 65 (
66 .clk_5x (pclk_x5), //Input system clock 67 .par_data (blue_10bit), //Input parall
el data 68 69 .ser_data_p (tmds_data_p[0]), //Output serial differential data 70 .ser
_data_n (tmds_data_n[0]) //Output serial differential data 71 ); 72 par_to_ser par_t
o_ser_g 73 ( 74 .clk_5x (pclk_x5), //Input system clock 75 .par_data (green_10bit)
, //Input parallel data 76 77 .ser_data_p (tmds_data_p[1]), //Output serial differenti
al data 78 .ser_data_n (tmds_data_n[1]) //Output serial differential data 79 ); 80 pa
r_to_ser par_to_ser_r 81 ( 82 .clk_5x (pclk_x5), //Input system clock 83 .par_data
(red_10bit), //Input parallel data 84 85 .ser_data_p (tmds_data_p[2]), //Output seri
al differential data 86 .ser_data_n (tmds_data_n[2]) //Output serial differential data
```

After adding the custom IP core, lead out the pins that need to be connected to the top-level port, and manually connect

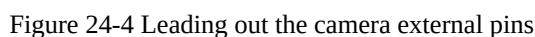


Figure 24-5 Overall IP block diagram

```
1 #cam
2 set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets ov7725_pclk_IBUF]
3 set_property IOSTANDARD LVCMOS33 [get_ports ov7725_pclk]
```

```

4 set_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[7]}]
5 set_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[6]}]
6 set_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[5]}]
7 set_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[4]}]
8 set_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[3]}]
9 set_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[2]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[1]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {ov7725_data[0]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports ov7725_href]
13 set_property IOSTANDARD LVCMOS33 [get_ports ov7725_vsync]
14 set_property IOSTANDARD LVCMOS33 [get_ports cam_rst_n]
15 set_property PACKAGE_PIN N15 [get_ports {ov7725_data[0]}]
16 set_property PACKAGE_PIN N16 [get_ports {ov7725_data[1]}]
17 set_property PACKAGE_PIN L14 [get_ports {ov7725_data[2]}]
18 set_property PACKAGE_PIN L15 [get_ports {ov7725_data[3]}]
19 set_property PACKAGE_PIN M14 [get_ports {ov7725_data[4]}]
20 set_property PACKAGE_PIN M15 [get_ports {ov7725_data[5]}]
21 set_property PACKAGE_PIN L16 [get_ports {ov7725_data[6]}]
22 set_property PACKAGE_PIN L17 [get_ports {ov7725_data[7]}]
23 set_property PACKAGE_PIN M19 [get_ports ov7725_vsync]
24 set_property PACKAGE_PIN M20 [get_ports ov7725_href]
25 set_property PACKAGE_PIN L19 [get_ports ov7725_pclk]
26
27 set_property PACKAGE_PIN M18 [get_ports cam_rst_n]
28
29 set_property PACKAGE_PIN T17 [get_ports {cam_sccb_tri_io[0]}]
30 set_property PACKAGE_PIN R18 [get_ports {cam_sccb_tri_io[1]}]
31 set_property IOSTANDARD LVCMOS33 [get_ports {cam_sccb_tri_io[0]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {cam_sccb_tri_io[1]}]
33
34 set_property IOSTANDARD LVCMOS33 [get_ports cam_pwn_0]
35 set_property PACKAGE_PIN M17 [get_ports cam_pwn_0]
36
37 set_property IOSTANDARD LVCMOS33 [get_ports cam_xclk_0]
38 set_property PACKAGE_PIN L20 [get_ports cam_xclk_0]
39
40
41 set_property PACKAGE_PIN W14 [get_ports {tmads_data_p[0]}]
42 set_property PACKAGE_PIN Y16 [get_ports {tmads_data_p[1]}]
43 set_property PACKAGE_PIN Y18 [get_ports {tmads_data_p[2]}]
44
45 set_property PACKAGE_PIN V20 [get_ports tmads_clk_p]
46 set_property PACKAGE_PIN W20 [get_ports tmads_clk_n]
47
48 set_property IOSTANDARD LVCMOS33 [get_ports tmads_oen]
49 set_property PACKAGE_PIN T20 [get_ports tmads_oen]

```

Then generate a bit stream file and export the hardware (check "Include bitstream" in the pop-up dialog box). Finally launch the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and transplanted the following code into the newly created project.

display_ctrl	2023-10-11 15:03	文件夹	
dynclk	2023-10-11 15:03	文件夹	
ov7725	2023-10-11 15:03	文件夹	
sccb	2023-10-11 15:03	文件夹	
vdma_api	2023-10-11 15:03	文件夹	
lscript.ld	2023-09-12 9:38	LD 文件	7 KB
main.c	2023-09-12 9:40	C 文件	2 KB

Figure 24-6 Schematic diagram

Migrate the above 5 folders into your own project, then create a new main.c file and add the following code.

Code List 24-3 OV7725 HDMI display reference code (main.c)

```

1 #include <stdio.h> 2 #include <stdlib.h> 3 #include <string.h> 4 #include "xil_types.h" 5 #include "xil_cache.h"
6 #include "xparameters.h" 7 #include "xaxivdma.h" 8 #include "xaxivdma_i.h" 9 #include "display_ctrl/display_ctrl.h"
10 #include "ov7725/ov7725_init.h" 11 #include "vdma_api/vdma_api.h" 12 #include "sccb/sccb_init.h" 13
14 #define CLOCK_ADDR XPAR_AXI_DYNCLK_0_BASEADDR 15 #define VDMA_DEVICE_ID XPAR_AXI_VDMA_0_DEVICE_ID
16 #define XVTC_DEVICE_ID XPAR_VTC_0_DEVICE_ID 17 18 22 /*Buffer memory base address*/
23 unsigned int const FB_Base = (XPAR_PS7_DDR_0_S_AXI_BASEADDR + 0x1000000);
24 25 int main(void) 26 { 27 28 SCCB_Init(); //Initialize EMIO 29 OV7725_Init(); //Initialize ov7725
30 31 /*Output resolution setting, ov7725 maximum is 640x480*/ 32 VIDEO_Mode = VMODE_640x480;
33 34 /*Enable vdma read and write mode*/ 35 run_vdma_frame_buffer(&Vdma, VDMA_DEVICE_ID, VIDEO_Mode.width,
36 VIDEO_Mode.height, FB_Base, 0, 0, BOTH);
37 38 DisplayInitialize(&Dispctrl, XVTC_DEVICE_ID, CLOCK_ADDR);
39 DisplaySetMode(&Dispctrl, &VIDEO_Mode);
40 DisplayStart(&Dispctrl);
41 42 return 0;
43 }

```

The code is actually basically the same as "OV7725 LCD Display", except that the output resolution is different. The author configures a resolution of 640x480 here. This is the maximum resolution of OV7725. The output video timing is 640x480@60HZ.

Some HDMI displays may not support this resolution. If the reader encounters a black screen output, you can try changing the display or changing the resolution.

24.3.4 Board verification

As shown in Figure 24-7, the development board is connected. After the lines are connected correctly, turn on the s

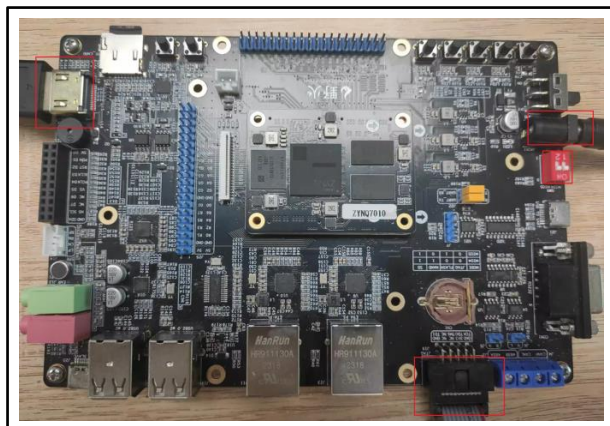


Figure 24-7 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system and Program FPGA -> Run.

After the program download is completed, the image collected by the OV7725 camera will be displayed on the HDMI display with a resolution of 640*480, as shown in Figure 24-8.



Figure 24-8 OV7725+HDMI image display

24.4 End of chapter summary

This chapter combines the OV7725 camera with HDMI to realize the real-time display of images collected by the

24.5 Development training

Consult the OV7725 camera data manual to understand the functions of related registers under different configurations, change the register configuration, and realize functions such as up and down flipping, left and right mirroring of the captured image, and real-time display on the HDMI monitor.

Chapter 25 OV5640 HDMI Display

25.1 Chapter Introduction

In the above chapter, we implemented the LCD display of the OV5640 camera, and in this chapter we implemented the HDMI display of the OV5640 camera.

25.2 Theoretical study

For the theoretical study of OV5640 and HDMI in this experiment, readers can refer to the previous chapters, and I will not go into details here.

25.3 Practical exercises

25.3.1 Experimental objectives

The development board collects image data through OV5640 and displays it on the HDMI display.

25.3.2 Hardware resources

This experiment used the HDMI interface, OV5640 camera interface, and UART on the PL side.

25.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 25-1.

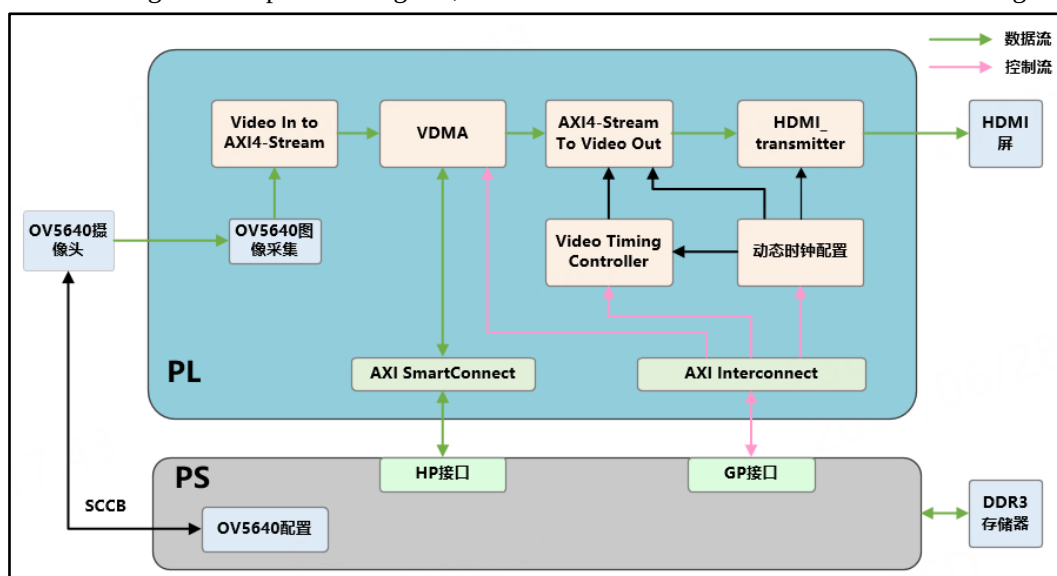


Figure 25-1 Overall block diagram

This overall block diagram is actually the same as "OV5640 LCD Display", except that it is replaced by HDMI display, and the tft_lcd IP core is replaced by HDMI_Transmitter IP. There are no other changes.

2. Hardware design

The hardware platform for this experiment was built based on the experiments in the chapter "OV5640 HDMI Display". Except that the OV5640 IP core name is different, everything else is the same.

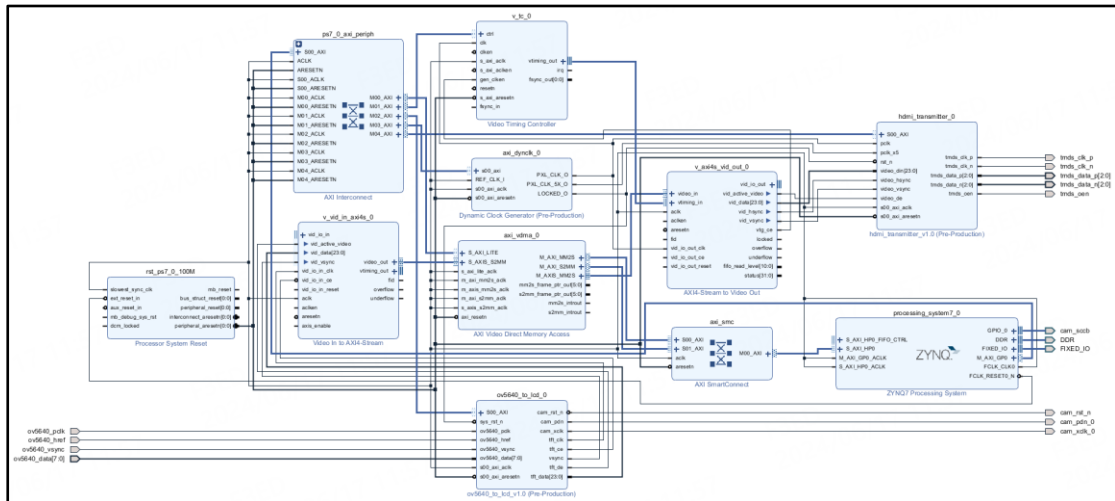


Figure 25-2 Overall IP block diagram

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and transplanted the following code into the newly created project.

display_ctrl	2024/6/17 13:33	文件夹	
dynclk	2024/6/17 13:33	文件夹	
ov5640	2024/6/18 10:01	文件夹	
sccb	2024/6/17 13:33	文件夹	
vdma_api	2024/6/17 13:33	文件夹	
lscript.ld	2024/6/17 13:32	LD 文件	7 KB
main.c	2024/6/17 14:27	C 源文件	2 KB
README.txt	2018/12/7 11:56	TXT 文件	1 KB
Xilinx.spec	2024/6/17 13:32	SPEC 文件	1 KB

Figure 25-3 Schematic diagram

Migrate the above 5 folders into your own project, then create a new main.c file and add the following code.

Code List 25-1 ov5640 HDMI display reference code (main.c)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include "xil_types.h"
5 #include "xil_cache.h"
6 #include "xparameters.h"
7 #include "xaxivdma.h"
8 #include "xaxivdma_i.h"

```

```
9 #include "display_ctrl/display_ctrl.h" 10 #include "vdma_api/vdma_api.h" 11 #include "sccb/sccb_init.  
h" 12 #include "ov5640/ov5640_init.h" 13 #include "ov5640/ov5640_AF.h" 14 15 #define FB_Num 3 //  
Number of frame buffers 16 #define CLOCK_ADDR XPAR_AXI_DYNCLK_0_BASEADDR // Dynam  
ic clock base address
```

```
17 #define VDMA_ID XPAR_AXIVDMA_0_DEVICE_ID // VDMA device ID 18 #define The base ad  
dress is enough */ 24 unsigned int const FB_Base =0x3000000; 25 26 Output horizontal pixels 33 u16 ca  
m_v_disp; // ov5640 output vertical pixels 34 35 /* Due to vivado project configuration issues, the HDM  
I frequency requirement cannot be higher than 100MHz, 36 * so the resolution of 1280*1024 and above  
needs to be reduced to @30hz, but the screen may not support it */ 37 #if display_800x480 38 44 49
```

```
    #elif display_1280x800  
50    cam_h_disp =1280;  
51    cam_v_disp =800;  
52    VIDEO_Mode = VMODE_1280x800;  
53    #elif display_1280x1024  
54    cam_h_disp =1280;  
55    cam_v_disp =1024;  
56    VIDEO_Mode = VMODE_1280x1024; 5  
7 #elif display_1920x1080 58 cam_h_disp =1920; 59 cam_v  
_disp =1080; 60 VIDEO_Mode = VMODE_1920x1080; 61  
#endif  
  
62 SCCB_Init(); // Initialize SCCB 63 OV5640_Init(cam_h_disp, cam_v_disp); // Initializ  
e 64 OV5640_AUTO_FOCUS(); // Auto focus  
  
65 66  
    /* Configure VDMA */  
67 run_vdma_frame_buffer(&vdma, VDMA_ID, VIDEO_Mode.width, 6  
8    VIDEO_Mode.height, FB_Base,0,0,BOTH);  
69  
70 /* Clear the background cache */ 71 memset((unsigned int *)FB_Base, 0, 72 VIDEO_M  
ode.height*VIDEO_Mode.width*FB_Num*3);
```

```
73
74     Xil_DCacheFlush(); //更新缓冲
75
76     DisplayInitialize(&Dispctrl, XVTC_DEVICE_ID, CLOCK_ADDR);
77     DisplaySetMode(&Dispctrl, &VIDEO_Mode); // 设置 VTC 显示参数
78     DisplayStart(&Dispctrl);                // 开启
79     return 0;
80 }
```

The code is actually basically the same as "OV7725 HDMI Display", except that the OV7725_Init() function is replaced by the OV5640_Init() function, and the auto-focus function is added. The default resolution is 1280*720. Readers who need to modify the resolution can modify line 21. The specific supported resolutions also need to be supported by the screen.

25.3.4 Board verification

As shown in Figure 25-4, the development board is connected to the 12V DC power supply, XILINX downloader JTAG port, HDMI display and OV5640 camera. After the lines are connected correctly, turn on the switch to power on the board.

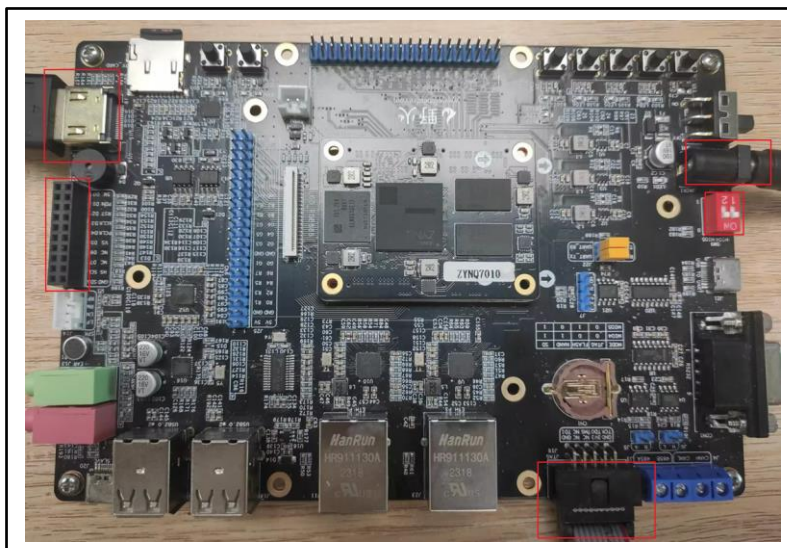


Figure 25-4 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system and Program FPGA -> Run.

After the program download is completed, the image collected by the OV5640 camera will be displayed on the HDMI display with a resolution of 1280*720, as shown in Figure 25-5.



Figure 25-5 OV5640+HDMI image display

25.4 End of chapter summary

This chapter combines the OV5640 camera with HDMI to achieve real-time display of images captured by the OV

25.5 Development training

Consult the OV5640 camera data manual to understand the functions of related registers under different configurations, change the register configuration, and realize functions such as up and down flipping, left and right mirroring of the captured image, and real-time display on the HDMI monitor.

Chapter 26 OV7725 HDMI Grayscale Display

26.1 Chapter Introduction

In the previous chapter, we learned how to drive HDMI to display camera data. Next, we will grayscale the data and then display it on the HDMI screen.

26.2 Theoretical study

For the theoretical study of OV7725 and HDMI in this experiment, readers can refer to the previous chapters, and I will not go into details here.

26.3 Practical exercises

26.3.1 Experimental objectives

The image data collected by the OV7725 camera is grayscaled and displayed on HDMI in real time.

26.3.2 Hardware resources

This experiment used the HDMI interface, OV7725 camera interface, and UART on the PL side.

26.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 26-1.

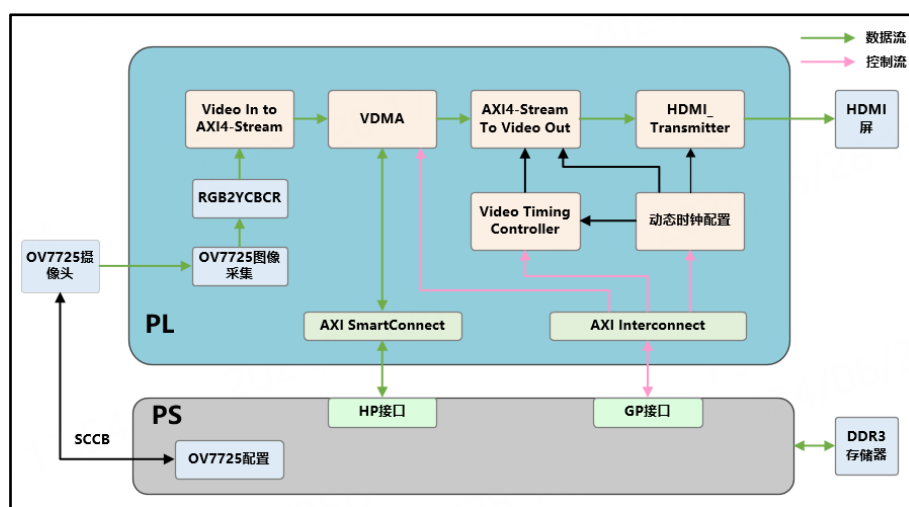


Figure 26-1 Overall block diagram

The overall block diagram is the same as the "OV7725 HDMI Display" chapter, except that after the OV7725 collects the image, RGB2YCBCR is added to grayscale the collected image.

2. Hardware design

Please refer to the previous chapter for the detailed steps of project creation. Below we only list the process of adding

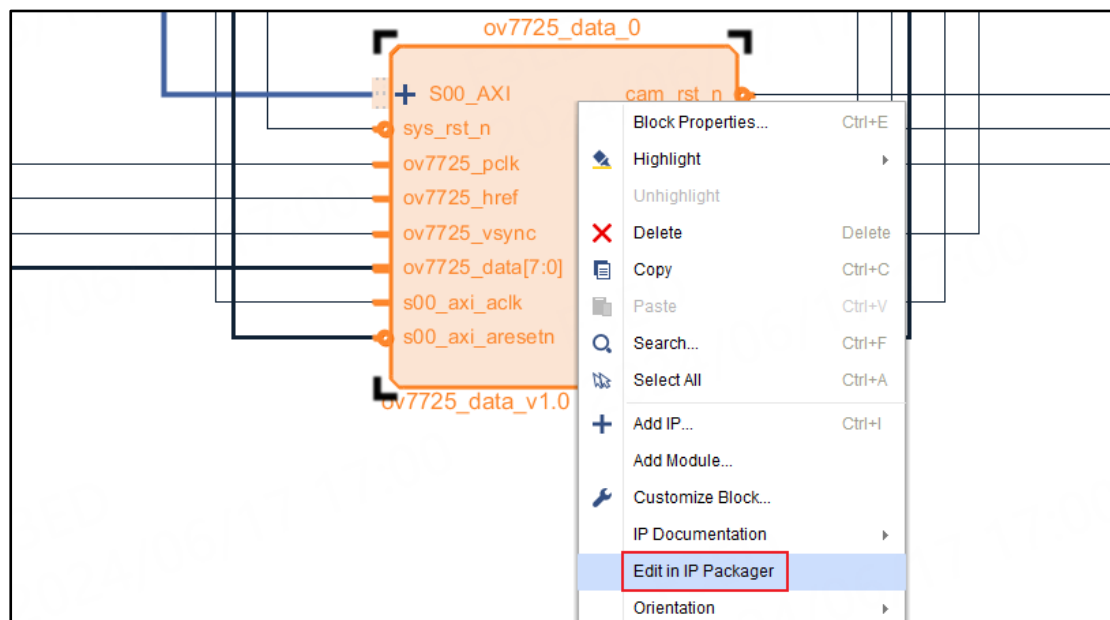


Figure 26-2 Open IP design

Open Block Design in the routine and find the IP ov7725_data, right-click and select Edit in IP Packager to edit, as shown below:

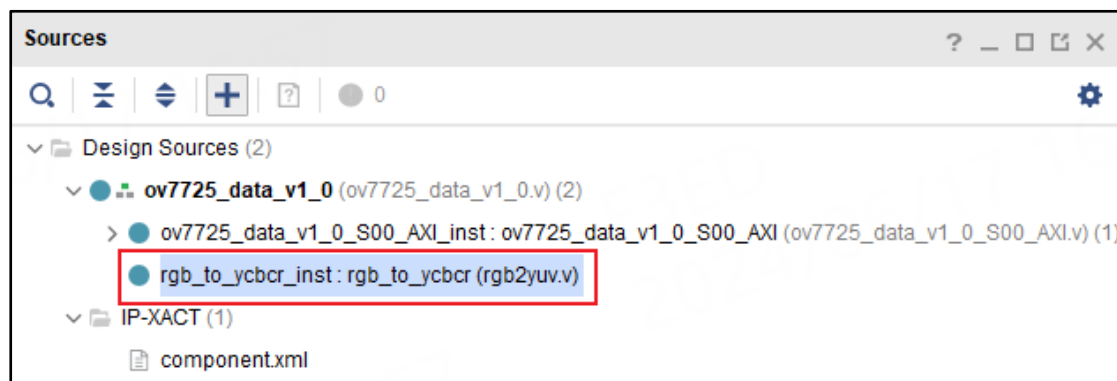


Figure 26-3 Project example after opening

You can see that there is a rgb_to_ycbcr file in the project directory, which is grayscale. We double-click the file to open it.

Code List 26-1 Image grayscale reference code (rgb2yuv.v)

```

1 module  rgb_to_ycbcr(
2     input                clk,
3     input                [7 : 0] i_r_8b,
4     input                [7 : 0] i_g_8b,
5     input                [7 : 0] i_b_8b,
6
7     input                i_h_sync,
8     input                i_v_sync,
```

```
9 input i_data_en, 10 11 output [7 : 0] o_y_8b, 12 output [7 : 0] o_cb_8b, 13 output [7 : 0] o_cr_8b, 14 15 output o_h_sync, 16 output o_v_sync, 17 output o_data_en 18 ); 19 20 /*****
****parameters**** 21 ****/ 22 //multiply 256 23 parameter para_0183_10b = 10'd47; //0.183 fixed point number 24 parameter para_0614_10b = 10'd157; 25 parameter para_0062_10b = 30 parameter para_0040_10b = 10'd10; 31 parameter para_16_18b = 18'd4096; 32 parameter para_128_18b = 18'd32768; 33 /*****
****signals**** 37 ****/ 38 wire sign_cb; 39 wire sign_cr; 40 reg[17: 0] mult_r_for_y_18b; 41 reg[17: 0] mult_r_for_cb_18b; 42 reg[17: 0] mult_r_for_cr_18b; 43 44 reg[17: 0] mult_g_for_y_18b; 45 reg[17: 0] mult_g_for_cb_18b; 46 reg[17: 0] mult_g_for_cr_18b; 47 48 reg[17: 0] mult_b_for_y_18b; 49 reg[17: 0] mult_b_for_cb_18b; 50 reg[17: 0] mult_b_for_cr_18b; 51 52 reg[17: 0] add_y_0_18b; 53 reg[17: 0] add_cb_0_18b; 54 reg[17: 0] add_cr_0_18b; 55 56 reg[17: 0] add_y_1_18b; 57 reg[17: 0] add_cb_1_18b; 58 reg[17: 0] add_cr_1_18b; 59 60 reg[17: 0] result_y_18b; 61 69 reg i_v_sync_delay_1; 70 reg i_data_en_delay_1; 71 72 reg i_h_sync_delay_2; 73 reg i_v_sync_delay_2; 74 reg i_data_en_delay_2;
```

```
75 76 reg i_h_sync_delay_3; 77 reg i_v_sync_delay_3; 78 reg i_data_en_delay_3; 79 80 reg i_h_sync_delay_4;
81 reg i_v_sync_delay_4; 82 reg i_data_en_delay_4; 83 84 /*****
***** 85 *****/ 86 87 /*****
*****initial***** 88 *****/ 89 initial 90 be
gin 91 mult_r_for_y_18b <= 18'd0; 92 mult_r_for_cb_18b <= 18'd0; 93 mult_r_for_cr_18b <= 18'd0; 94 95 mu
lt_g_for_y_18b <= 18'd0; 96 mult_g_for_cb_18b <= 18'd0; 97 mult_g_for_cr_18b <= 18'd0; 98 99 104 109 add
_cr_1_18b <= 18'd0; 110 111 result_y_18b <= 18'd0; 112 result_cb_18b <= 18'd0; 113 result_cr_18b <= 18'd0;
114 115 i_h_sync_delay_1 <= 1'd0; 116 i_v_sync_delay_1 <= 1'd0; 117 i_data_en_delay_1 <= 1'd0; 118 119 i_
h_sync_delay_2 <= 1'd0; 120 i_v_sync_delay_2 <= 1'd0; 121 i_data_en_delay_2 <= 1'd0; 122 123 end 124 /**
*****
```

```
125 *****/ 126 127 /*****arithmetic***
*****
```

```
128 *****/ 129 //LV1 pipeline: mult 130 always @ (pose
dge clk) 131 begin 132 mult_r_for_y_18b <= i_r_8b * para_0183_10b; 133 mu
lt_r_for_cb_18b <= i_r_8b * para_0101_10b; 134 mult_r_for_cr_18b <= i_r_8
b * para_0439_10b; 135 end 136 137 always @ (posedge clk) 138 begin 139 m
ult_g_for_y_18b <= i_g_8b * para_0614_10b; 140 mult_g_for_cb_18b <= i_g
_8b * para_0338_10b;
```

```
141 mult_g_for_cr_18b <= i_g_8b * para_0399_10b; 142 end 143 144 always @ (posedge clk) 145 begin 146 m
ult_b_for_y_18b <= i_b_8b * para_0062_10b; 147 mult_b_for_cb_18b <= i_b_8b * para_0439_10b; 148 mult_b
_for_cr_18b <= i_b_8b * para_0040_10b; 149 end 150 //LV2 pipeline: add 151 always @ (posedge clk) 152 begi
n 153 add_y_0_18b <= mult_r_for_y_18b + mult_g_for_y_18b; 154 add_y_1_18b <= mult_b_for_y_18b + para
_16_18b; 155 156 add_cb_0_18b <= mult_b_for_cb_18b + para_128_18b; 157 add_cb_1_18b <= mult_r_for_cb
_18b + mult_g_for_cb_18b; 158 159 add_cr_0_18b <= mult_r_for_cr_18b + para_128_18b; 160 add_cr_1_18b
<= mult_g_for_cr_18b + mult_b_for_cr_18b; 161 end 162 //LV3 pipeline: y + cb + cr 163 164 assign sign_cb = (
add_cb_0_18b >= add_cb_1_18b); 165 assign sign_cr = (add_cr_0_18b >= add_cr_1_18b); 166 always @ (pose
dge clk) 167 begin 168 result_y_18b <= add_y_0_18b + add_y_1_18b; 169 result_cb_18b <= sign_cb ? (add_cb
_0_18b - add_cb_1_18b) : 18'd0; 170 result_cr_18b <= sign_cr ? (add_cr_0_18b - add_cr_1_18b) : 18'd0; 171 en
d 172 173 always @ (posedge clk) 174 begin 175 y_tmp <= result_y_18b[17:8] + {9'd0,result_y_18b[7]}; 176 cb
_tmp <= result_cb_18b[17:8] + {9'd0,result_cb_18b[7]}; 177 cr_tmp <= result_cr_18b[17:8] + {9'd0,result_cr_1
8b[7]}; 178 end 179 //output 180 assign o_y_8b = (y_tmp[9:8] == 2'b00) ? y_tmp[7:0] : 8'hFF; 181 assign o_cb
_8b = (cb_tmp[9:8] == 2'b00) ? cb_tmp[7:0] : 8'hFF; 182 assign o_cr_8b = (cr_tmp[9:8] == 2'b00) ? cr_tmp[7:
0] : 8'hFF; 183 /***** 184 *****/
*****/ 185 /*****/timing*****/
*****/ 186 *****/ 187 always @ (posedge clk) 188 begin 189 i_h_sync_delay_1 <=
i_h_sync; 190 i_v_sync_delay_1 <= i_v_sync; 191 i_data_en_delay_1 <= i_data_en; 192 193 i_h_sync_delay_2
<= i_h_sync_delay_1; 194 i_v_sync_delay_2 <= i_v_sync_delay_1; 195 i_data_en_delay_2 <= i_data_en_delay
_1; 196 197 i_h_sync_delay_3 <= i_h_sync_delay_2; 198 i_v_sync_delay_3 <= i_v_sync_delay_2; 199 i_data_e
n_delay_3 <= i_data_en_delay_2; 200 i_h_sync_delay_4 <= i_h_sync_delay_2; 201 i_v_sync_delay_4 <= i_v_s
ync_delay_2; 202 i_data_en_delay_4 <= i_data_en_delay_2; 203 end 204 //----- 205
//timing 206 //-----
```

```

207 assign o_h_sync = i_h_sync_delay_4; 208 assign o_v_sync
= i_v_sync_delay_4; 209 assign o_data_en = i_data_en_delay_4
; 210 211 endmodule

```

This module implements RGB to YCBCR conversion, using the idea of a three-stage pipeline. The calculation formula

$$\begin{aligned}
 \text{RGB转YCBCR算法} \\
 \text{计算公式: } Y &= 0.183R + 0.614G + 0.062B + 16; \\
 CB &= -0.101R - 0.338G + 0.439B + 128; \\
 CR &= 0.439R - 0.399G - 0.040B + 128;
 \end{aligned}$$

Figure 26-4 RGB to YCBCR formula

We can indeed convert RGB to YCBCR in one clock cycle, but in practice, calculation takes time. If the combinational logic performs complex calculations, it will consume too much time and easily lead to timing violations. Therefore, we often use the pipeline method to divide it into multiple steps and use the characteristics of parallelism to realize pipeline calculation. Calculate first, then calculate in the second step, then calculate in the third step, and finally get 8-bit grayscale data. Since the three-stage pipeline delays the data by three beats, we must also delay the corresponding row and field signals and data valid signals by three clock cycles to maintain data synchronization.

The complete Block Design looks like this:

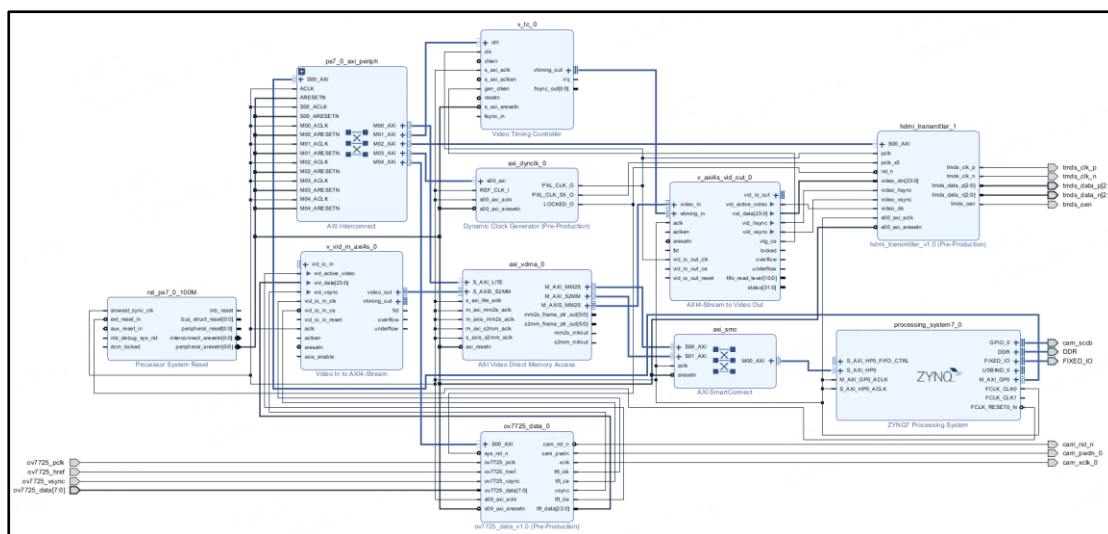


Figure 26-5 Overall IP block diagram

3. Software design

In the hardware design part, we started the SDK development environment. Since our overall design actually only changed the Block Design, that is, only the code on the PL side was changed. The code on the PS side does not need to be modified in any way. It can remain the same as the previous OV7725 HDMI display. I will not repeat the description here.

26.3.4 Board verification

As shown in Figure 26-6, the development board is connected to the 12V DC power supply, XILINX downloader JTAG port, HDMI display and OV7725 camera. After the lines are connected correctly, turn on the switch to power on the board.

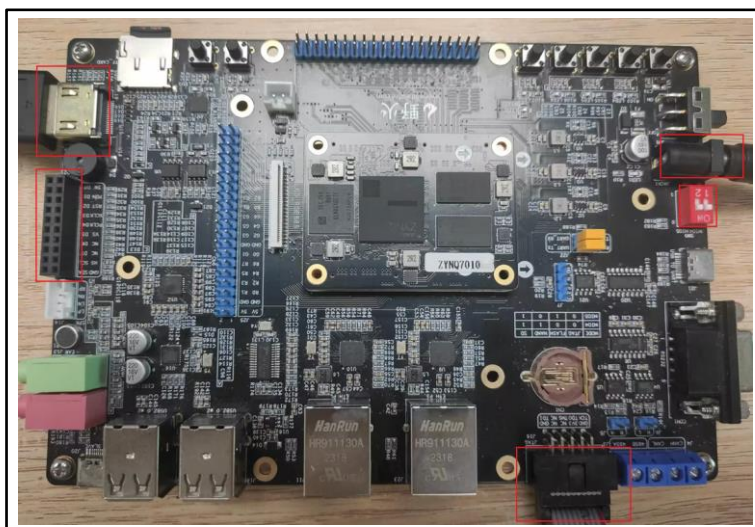


Figure 26-6 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system and Program FPGA -> Run.

After the program download is completed, the image collected by the OV7725 camera will be displayed on the HDMI display, and the image will be displayed in grayscale, as shown in Figure 26-7.



Figure 26-7 OV7725+HDMI image display

26.4 End of chapter summary

This chapter describes how to grayscale the images captured by OV7725 and display them on an HDMI. Image grayscale is a practice adopted in image processing, which facilitates subsequent image processing, because compared with YUV, pixel data in RGB format is more complex and large.

26.5 Development training

Consult the OV7725 camera data manual to understand the functions of related registers under different configurations, change the register configuration, and realize functions such as up and down flipping, left and right mirroring of the captured image, and real-time display on the HDMI monitor.

Chapter 27 OV5640 HDMI Grayscale Display

27.1 Chapter Introduction

In the previous chapter, we learned how to drive HDMI to display OV5640 camera data. Next, we will grayscale the data collected by OV5640 and then display it on the HDMI screen.

27.2 Theoretical study

For the theoretical study of OV5640 and HDMI in this experiment, readers can refer to the previous chapters, and I will not go into details here.

27.3 Practical exercises

27.3.1 Experimental objectives

The image data collected by the OV5640 camera is grayscaled and displayed on HDMI in real time.

27.3.2 Hardware resources

This experiment used the HDMI interface, OV5640 camera interface, and UART on the PL side.

27.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 27-1.

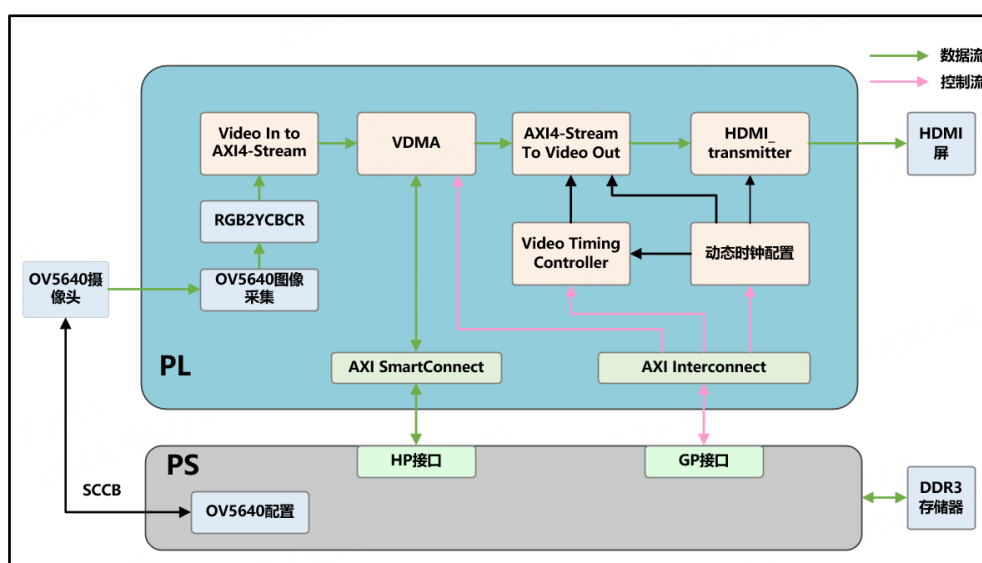


Figure 27-1 Overall block diagram

The block diagram is basically the same as "ov7725 HDMI Grayscale Display", except that the camera is changed

2. Hardware design

Please refer to the chapter "ov7725 HDMI Grayscale Display".

The complete Block Design looks like this:

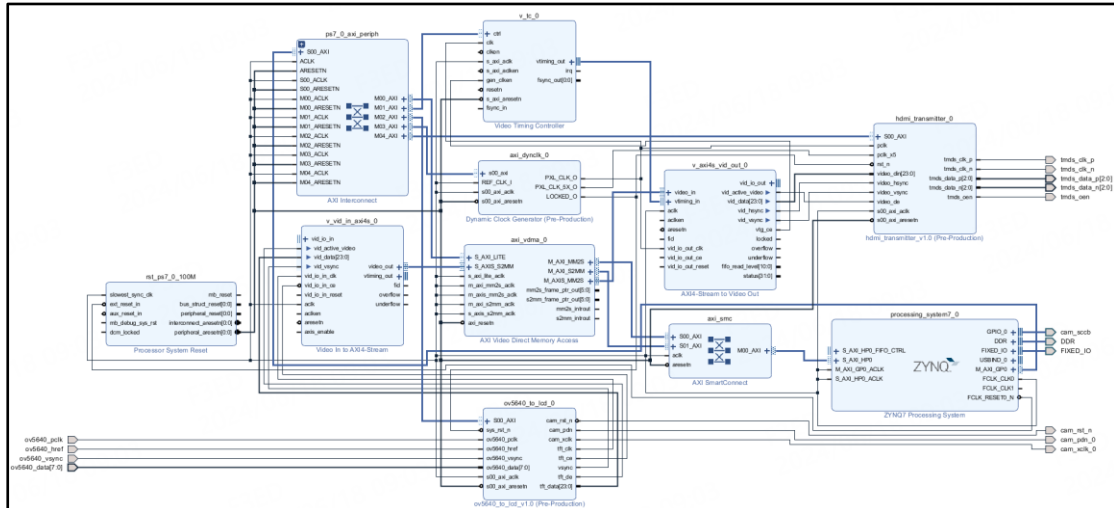


Figure 27-2 Overall IP block diagram

3. Software design

In the hardware design part, we started the SDK development environment. Since our overall design actually only changed the Block Design, that is, only the code on the PL side was changed. The code on the PS side does not need to be modified in any way. It can remain the same as the previous OV5640 HDMI display.

27.3.4 Board verification

As shown in Figure 27-3, the development board is connected to the 12V DC power supply, XILINX downloader JTAG port, HDMI display and OV5640 camera. After the lines are connected correctly, turn on the switch to power on the board.

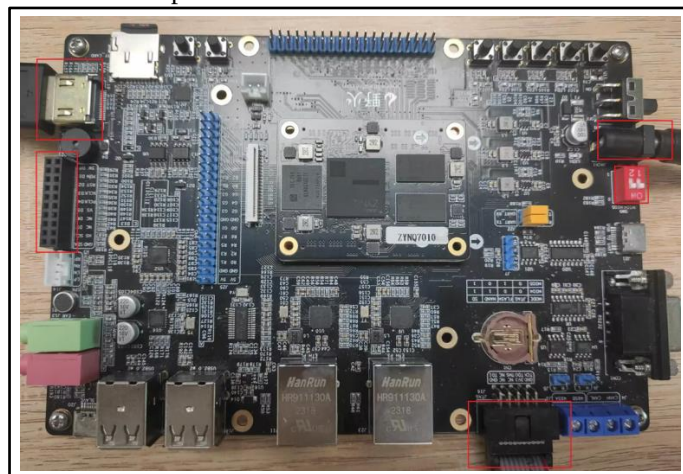


Figure 27-3 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system and Program FPGA -> Run.

After the program download is completed, the image collected by the OV5640 camera will be displayed on the HDMI display, and the image will be displayed in grayscale, as shown in Figure 27-4.



Figure 27-4 OV5640+HDMI image display

27.4 End of chapter summary

This chapter grayscales the images captured by OV5640 and displays them on HDMI.

27.5 Development training

Consult the OV5640 camera data manual to understand the functions of related registers under different configurations, change the register configuration, and realize functions such as up and down flipping, left and right mirroring of the captured image, and real-time display on the HDMI monitor.

Chapter 28 OV5640 HDMI display binarization

28.1 Chapter Introduction

In the previous chapter, we implemented grayscale conversion of the data collected by OV5640 and then displayed it on the HDMI screen. Next, we will implement the binary display of the image on this basis.

28.2 Theoretical study

Binary images consist of pixels that are either black or white (foreground or background), with no intermediate gray levels. This is usually done by applying a threshold to each pixel intensity value in the image. Pixels with a brightness value above the threshold are set to white, and pixels with a brightness value below or equal to the threshold are set to black.

Image binarization is a commonly used technology in image processing. By converting color or grayscale images into binary images with only black and white, it can simplify image information and highlight specific targets. It can improve the efficiency of image analysis and recognition, help extract targets, reduce noise and remove unnecessary details, while saving storage space and computing resources. For example, in applications such as text recognition and barcode reading, binary images can greatly simplify the image processing and recognition process.

Image binarization presents an obvious black and white effect on the entire image, thereby greatly reducing the amount of data in the binarized image and highlighting the outline of the target. To obtain a binary image, the image must first be grayscaled, and then the grayscale image with 256 brightness levels is selected through appropriate thresholds to obtain a binary image that can still reflect the overall and local characteristics of the image. All pixels whose grayscale is greater than or equal to the threshold are judged to belong to a specific object, and their grayscale value is 255. Otherwise, these pixels are excluded from the object area, and their grayscale value is 0, indicating the background or exceptional object area. Figure 28-1 shows the process of converting a color image into a binary image. The image is grayscaled before binarization.

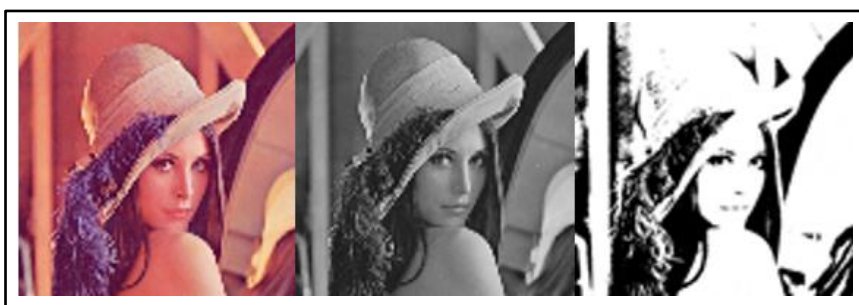


Figure 28-1 Image binarization process

According to the different threshold selection, the binarization algorithm is divided into fixed threshold and adaptive threshold. The more commonly used binarization methods include: bimodal method, P parameter method, iterative method, OTSU method, etc. This experiment uses a fixed threshold method.

28.3 Practical exercises

28.3.1 Experimental objectives

The images collected by the OV5640 camera are binarized and displayed in real time on HDMI.

28.3.2 Hardware resources

This experiment used the HDMI interface and OV5640 camera interface on the PL side.

28.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 28-2.

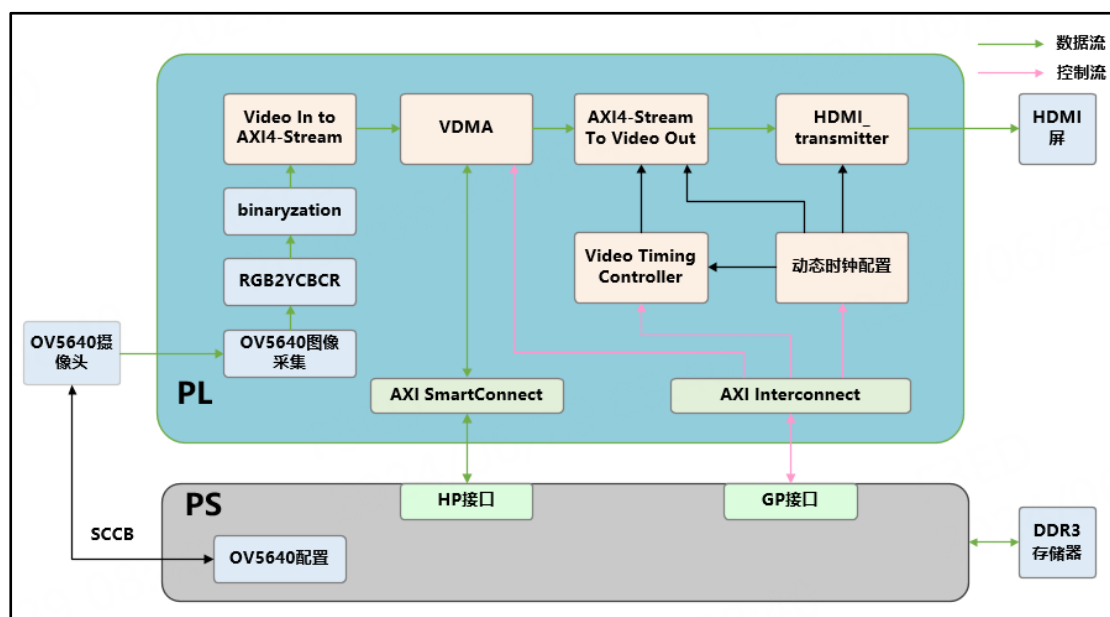


Figure 28-2 Overall block diagram

It is basically the same as the chapter "OV5640 HDMI Grayscale Display", except that an image binarization module is added.

2. Hardware design

Detailed steps for project creation can be found in the previous chapters. Below we only list the configuration process. Add image binarization module

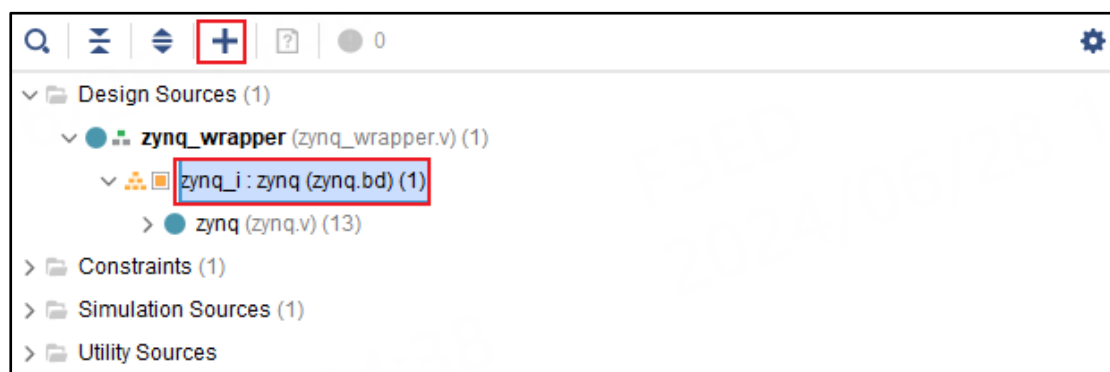


Figure 28-3 Click '+'

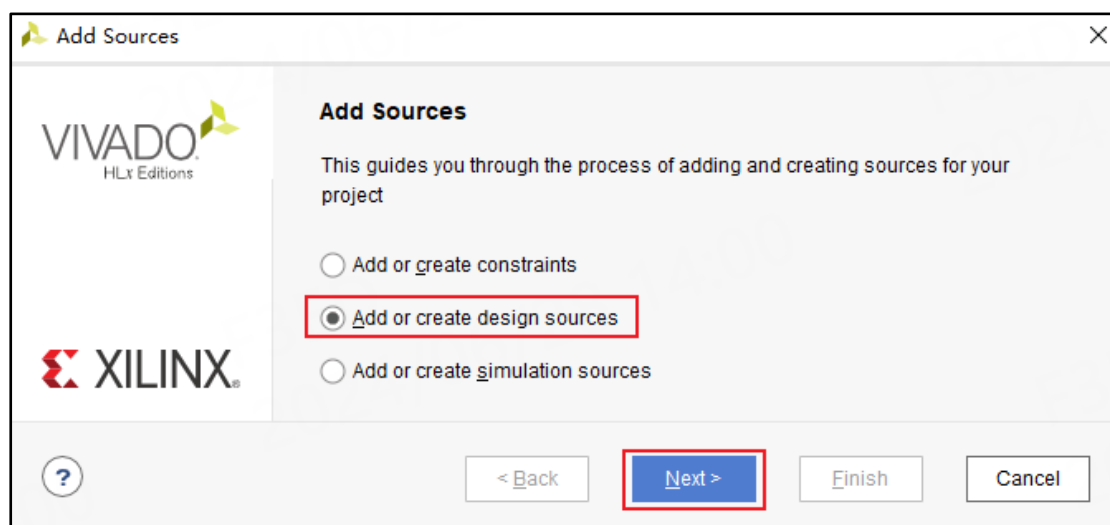


Figure 28-4 add sources

Click Next and click Add Files in the next pop-up interface.

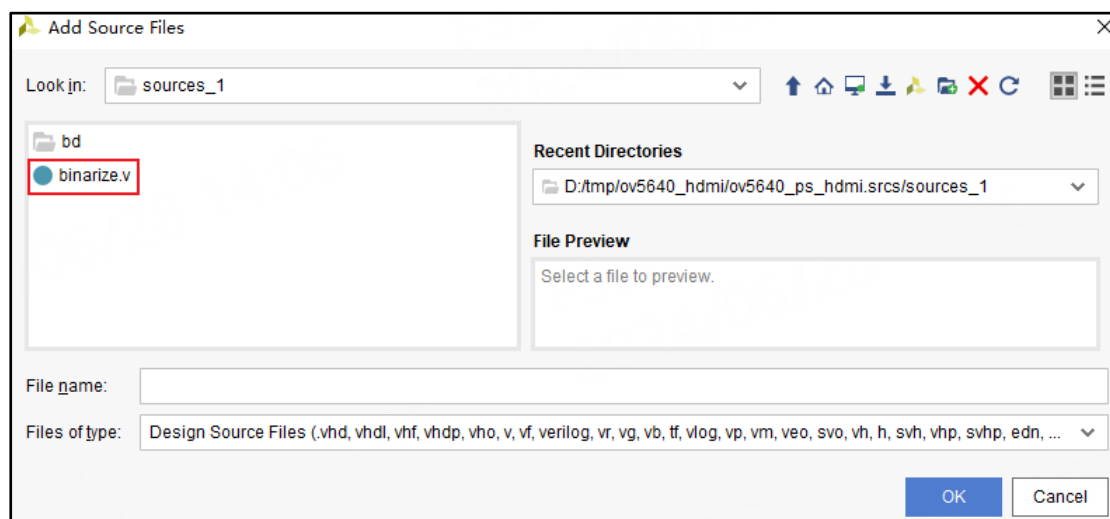


Figure 28-5 Add binarize.v

Select the binarize.v file in Figure 28-5. This file is in the ov5640_ps_hdmi.srcs/sources_1 directory in the project, which we have written in advance.

Code List 28-1 Image binarization reference code (binarize.v)

```

1 /*
2  二值化操作
3  */
4 module binarization(
5     input        clk        ,
6     input        rst_n      ,
7
8     //进来灰度数据
9     input        grey_vs    ,
10    input        grey_hs    ,
11    input        grey_de    ,
12    input [23:0]  grey_data  ,
13
14    //输出二值化后的数据
15    output        binary_vs  ,
16    output        binary_hs  ,

```

```
17 output binary_de , 18 output [23:0] binary_data 19 ); 20 21 parameter THRESHOLD
= 8'd127; //Threshold 22 23 24 //reg define 25 reg gray_vsync_d; 26 reg gray_hsync_d;
27 reg gray_data_valid_d; 28 reg 37 38 assign binary_data = {24{thre_flag}}; 39 40 //C
ompare with the threshold. Greater than the threshold is 1 and less than 0 41 always @(p
osedge clk or negedge rst_n) begin 42 if(!rst_n) 43 thre_flag <= 1'b0; 44 else if(grey_da
ta[7:0]> THRESHOLD) 45 thre_flag <= 1'b1; 46 else 47 thre_flag <= 1'b0; 48 end 49 5
0 //beat to keep in sync 51 always@(posedge clk or negedge rst_n) begin 52 if(!rst_n) be
gin 53 gray_vsync_d <= 1'd0; 54 gray_hsync_d <= 1'd0; 55 gray_data_valid_d <= 1'd0;
56 end 57 else begin 58 gray_vsync_d <= grey_vs; 59 gray_hsync_d <= grey_hs; 60 gra
y_data_valid_d <= grey_de; 61 end 62 end 63 endmodule
```

In lines 41-48 of code, when the grayscale is greater than the 127 threshold, the pixel output is 255 (white), otherwise the output is 0 (black).

Then right-click on the blank space of the Diagram interface and select Add Module. In the pop-up interface, select the binarization module we just added, and then click OK to complete the addition of the module.

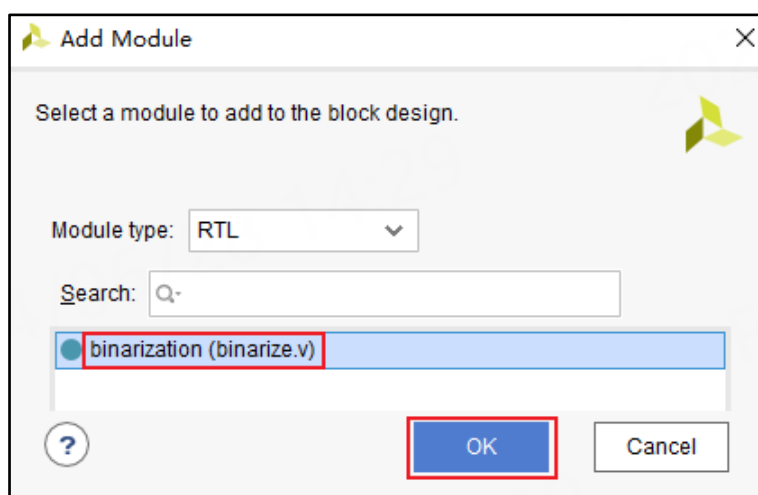


Figure 28-6 Add Module

Because Vivado's automatic connection function cannot completely connect according to our needs, we first manually connect some interfaces before automatic connection to avoid Vivado incorrect connection. The manual connection is as shown in the figure below:

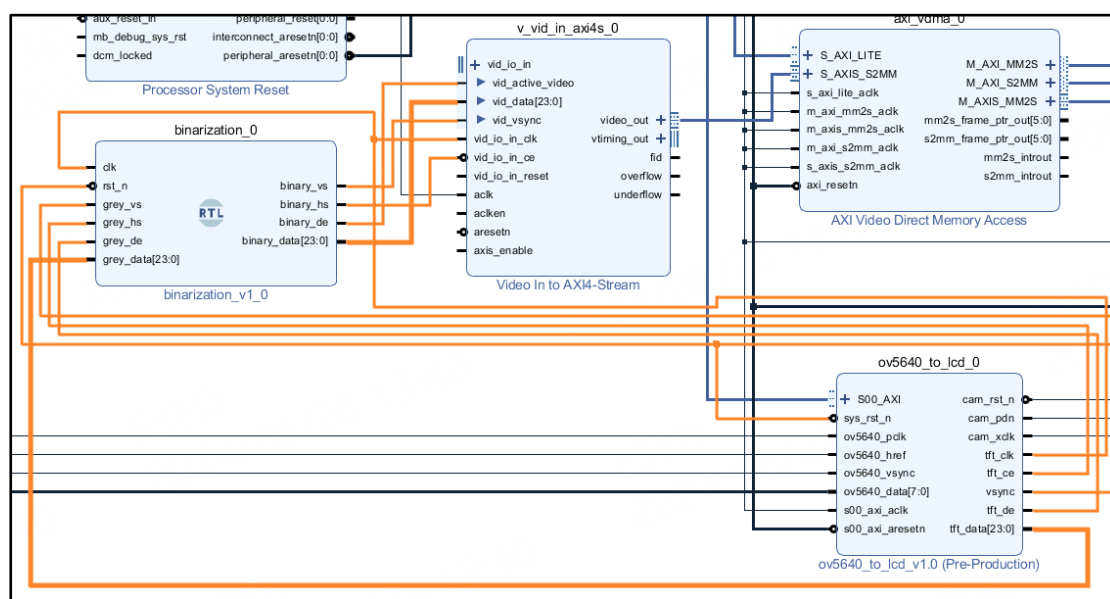


Figure 28-7 Manual connection

The final Block Design IP block diagram is shown in Figure 29-7.

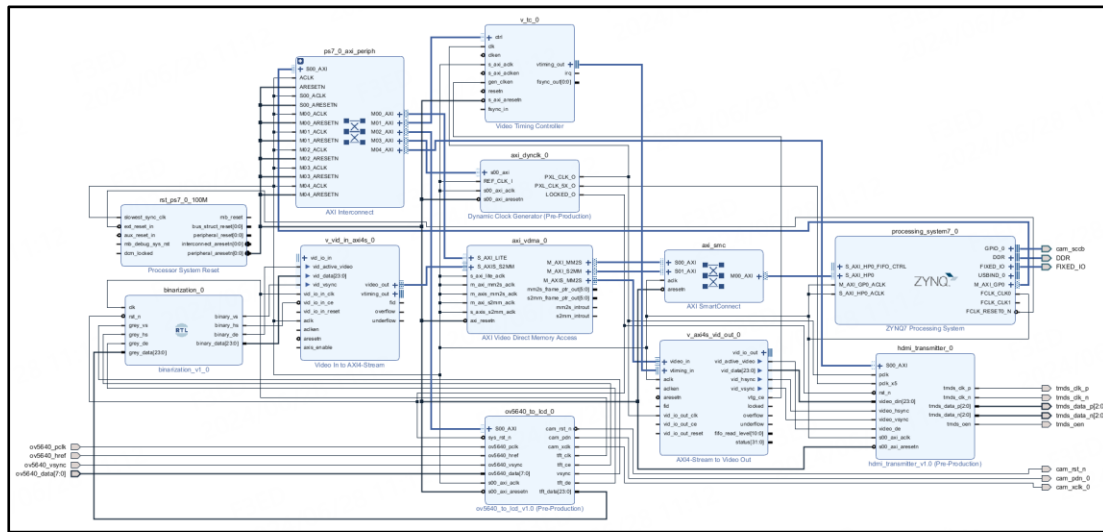


Figure 28-8 Overall IP block diagram

At this point, our Block Design is completed. Press the "F6" key to perform "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally press the shortcut key "Ctrl + S" to save the design.

Then execute "Generate Output Products" and "Create HDL Wrapper" in sequence. Then perform pin constraints on the used pins, then generate a bit stream file and export the hardware (check "Include bitstream" in the pop-up dialog box). Finally launch the SDK.

3. Software design

In the hardware design part, we started the SDK development environment. Since our overall design actually only changed the Block Design, that is, only the code on the PL side was changed. The code on the PS side does not need to be modified in any way. It can remain the same as the previous OV5640 HDMI display.

28.3.4 Board verification

As shown in Figure 28-9, the development board is connected. After the lines are connected correctly, turn on the power.

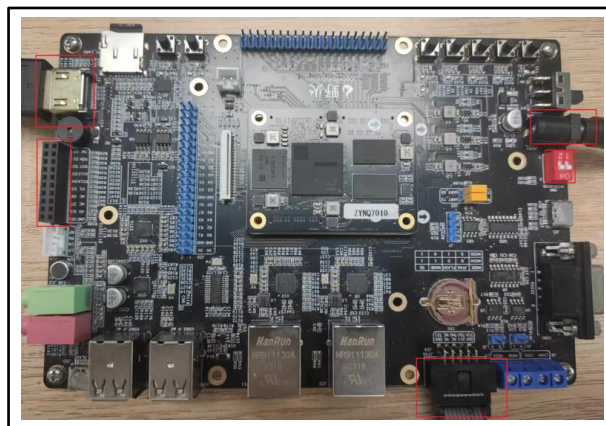


Figure 28-9 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system and Program FPGA -> Run.

After the program download is completed, the image collected by the OV5640 camera will be displayed on the HDMI display. You can see that the image is already a binary image at this time.

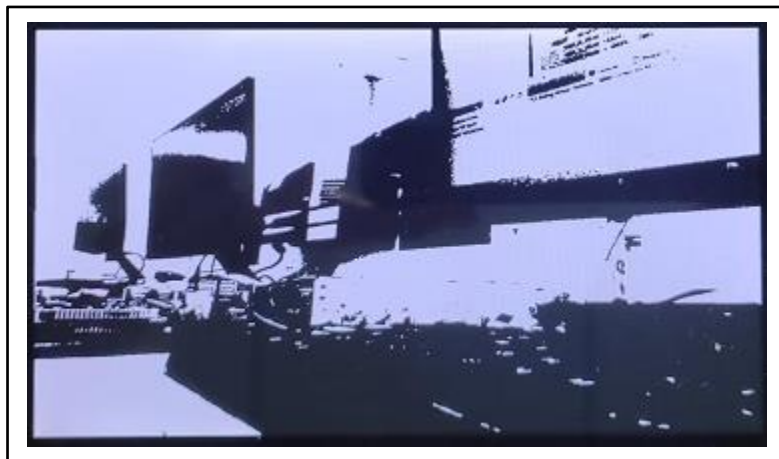


Figure 28-10 OV5640+HDMI binary image display

28.4 End of chapter summary

In this chapter, we learned about image binarization, which plays a very important role in image processing. Because binary images only contain two colors, the image processing algorithm is simpler and faster.

28.5 Development training

Consult the OV5640 camera data manual to understand the functions of related registers under different configurations, change the register configuration, and realize functions such as up and down flipping, left and right mirroring of the captured image, and real-time display on the HDMI monitor.

Chapter 29 OV7725 SD card save display

29.1 Chapter Introduction

In the previous chapter, we learned how to drive the OV7725 and display the data on HDMI. In this experiment, we will write the video data captured by the OV7725 to the SD card and save it as a BMP image.

29.2 Theoretical study

For the theoretical study of OV7725 and SD card in this experiment, readers can refer to the previous chapters, and I will not go into details here.

29.3 Practical exercises

29.3.1 Experimental objectives

Save the screen information captured by OV7725 to the SD card.

29.3.2 Hardware resources

This experiment used the HDMI interface, OV7725 camera interface, and UART on the PL side.

29.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 29-1.

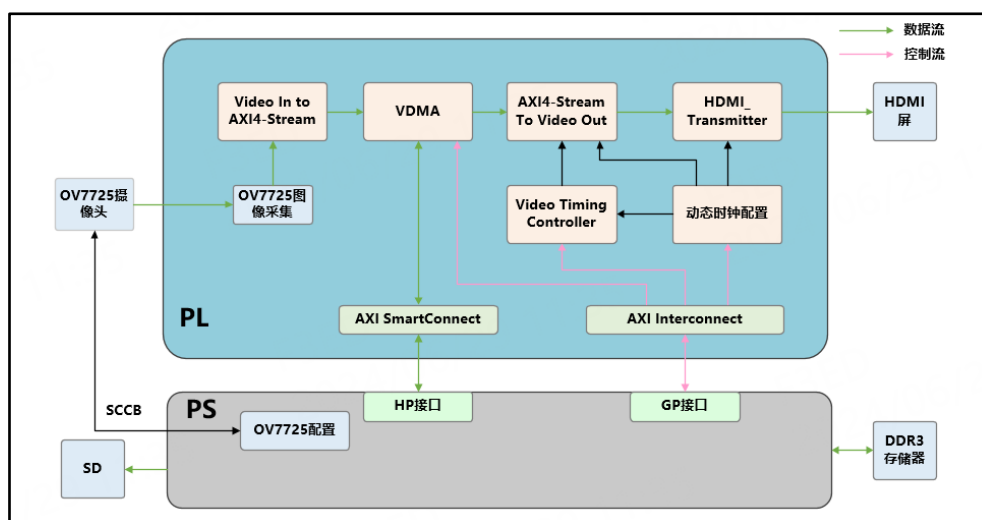


Figure 29-1 Overall block diagram

The overall block diagram is basically the same as that in the "OV7725 HDMI Display" chapter, except that an SD controller is added to save the images collected by ov7725.

2. Hardware design

Detailed steps for project creation can be found in the previous chapters. Below we only list the configuration process of the ZYNQ system IP.

The configuration of the ZYNQ system IP in the "OV7725 HDMI Display" chapter is basically the same. However, since an SD card is used, readers must remember to check SD enable, as shown in the figure below:

MIO Configuration	Peripheral	IO	Signal	IO Type	Speed
	☐ USB 0				
	☐ USB 1				
	☒ SD 0	MIO 40 .. 45			
	☐ CD				
	☐ WP				
	☐ Power				
	SD 0	MIO 40	clk	LVC MOS 1.8V	slow
	SD 0	MIO 41	cmd	LVC MOS 1.8V	slow
	SD 0	MIO 42	data[0]	LVC MOS 1.8V	slow
	SD 0	MIO 43	data[1]	LVC MOS 1.8V	slow
	SD 0	MIO 44	data[2]	LVC MOS 1.8V	slow
	SD 0	MIO 45	data[3]	LVC MOS 1.8V	slow
	> ☐ SD 1				

Figure 29-2 Enable SD

The final overall IP module connection diagram is shown in Figure 29-3:

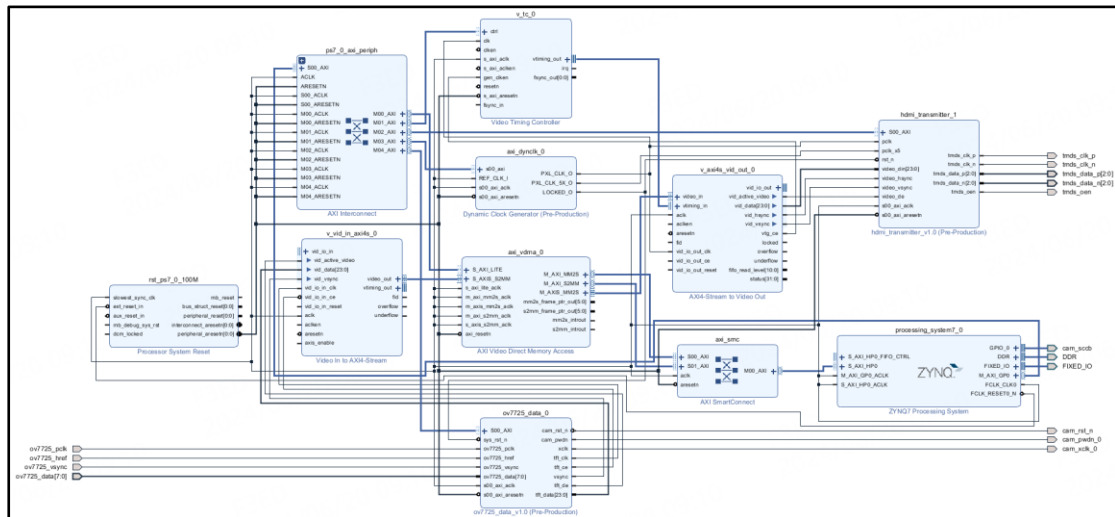


Figure 29-3 Overall IP block diagram

At this point, our Block Design is completed. Press the "F6" key to perform "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally, press the shortcut key "Ctrl + S" to save the design.

Then execute "Generate Output Products" and "Create HDL Wrapper" in sequence. Then perform pin constraints on the used pins, generate bit stream files, and export the hardware (check "Include bitstream" in the pop-up dialog box). Finally start the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and transplanted the following code into the newly created project.

display_ctrl	2023-10-11 15:03	文件夹	
dynclk	2023-10-11 15:03	文件夹	
ov7725	2023-10-11 15:03	文件夹	
sccb	2023-10-11 15:03	文件夹	
vdma_api	2023-10-11 15:03	文件夹	
lscript.ld	2023-09-12 9:38	LD 文件	7 KB
main.c	2023-09-12 9:40	C 文件	2 KB
README.txt	2018-12-07 11:56	文本文档	1 KB
Xilinx.spec	2023-09-12 9:38	SPEC 文件	1 KB

Figure 29-4 Schematic diagram

Migrate the above 5 folders into your own project, then create a new main.c file and add the following code.

Code List 29-1 SD card save captured image reference code (main.c)

```
1 #include <stdio.h> 2 #include <stdlib.h> 3 #include <string.h> 4 #include "xil
_types.h" 5 #include "xil_cache.h" 6 #include "xparameters.h" 7 #include "xaxi
vdma.h" 8 #include "xaxivdma_i.h" 9 #include "display_ctrl/display_ctrl.h" 10
#include "ov7725/ov7725_init.h" 11 #include "vdma_api/vdma_api.h" 12 #inclu
de "sccb/sccb_init.h" 13 14 #include "ff.h" 15 #include "xil_cache.h" 16 17 #def
ine CLOCK_ADDR XPAR_AXI_DYNCLK_0_BASEADDR 18 #define VDM
A_DEVICE_ID XPAR_AXIVDMA_0_DEVICE_ID 19 #define XVTC_DEVI
CE_ID XPAR_VTC_0_DEVICE_ID 20 21 #pragma pack(push, 1) // Ensure byt
e alignment is 1 byte 22 typedef struct { 23 char signature[2]; 24 unsigned int fil
eSize; 25 unsigned int reserved; 26 unsigned int dataOffset; 27 } BMPFileHead
er; 28 29 typedef struct { 30 unsigned int headerSize; 31 int width; 32 int height;
33 unsigned short int planes; 34 unsigned short int bitsPerPixel;
```

```
35 unsigned int compression; 36 unsigned int imageSize; 37 int xResolution; 38 int yResolution; 39 unsigned int colorsUsed; 40 unsigned int importantColors; 41 } BMPInfoHeader; 42 #pragma pack(pop) 43 44 unsigned int const bmpAddr = (XPAR_PS7_DDR_0_S_AXI_BASEADDR + 0x11000000);
```

```
45 unsigned int const fbBase = (XPAR_PS7_DDR_0_S_AXI_BASEADDR + 0x1000000); 46 static FATFS fatfs; 47 48 int saveBMP(const char* filename, unsigned char* imageData, int width, 49 int height) 50 { 51 BMPFileHeader fileHeader; 52 BMPInfoHeader infoHeader; 53 FILE* file; 54 FRESULT ret; 55 UINT bw; 56 printf("saving %s (%d * %d, bpp = %d)...\n", filename, width, height, 57 24); 58 ret = f_open(&file, filename, FA_CREATE_ALWAYS | FA_WRITE); 59 if (ret != FR_OK) { 60 perror("Unable to create file"); 61 return -1; 62 } 63 64 // BMP file header 65 fileHeader.signature[0] = 'B'; 66 fileHeader.signature[1] = 'M'; 67 fileHeader.fileSize = sizeof(BMPFileHeader) + sizeof(BMPInfoHeader) + 68 width * height * 3; // bpp:24bit 69 fileHeader.reserved = 0; 70 fileHeader.dataOffset = sizeof(BMPFileHeader) + sizeof(BMPInfoHeader) 71 + 0; // no palette 72 73 // BMP header 74 infoHeader.headerSize = sizeof(BMPInfoHeader); 75 infoHeader.width = width; 76 infoHeader.height = height; 77 infoHeader.planes = 1; 78 infoHeader.bitsPerPixel = 24; // 24-bit BMP 79 infoHeader.compression = 0; 80 infoHeader.imageSize = width * height * 3; // bpp:24bit 81 infoHeader.xResolution = 0; 82 infoHeader.yResolution = 0; 83 infoHeader.colorsUsed = 0; 84 infoHeader.importantColors = 0; 85 86 //Write file header and information header 87 f_write(&file, &fileHeader, sizeof(BMPFileHeader), &bw); 88 f_write(&file, &infoHeader, sizeof(BMPInfoHeader), &bw); 89 90 // Write pixel data 91 f_lseek(&file, fileHeader.dataOffset); 92 f_write(&file, imageData, width * height * 3, &bw); 93 94 f_close(&file); 95 return 0; 96 } 97 98 int Mount_Init(void) 99 { 100 TCHAR* path = "0:";
```

```
101 if (f_mount(&fatfs, path, 1) != FR_OK) { 102 xil_printf("Mount failed , Maybe not fat format.\r\n"); 103 return -1; 104 } 105 xil_printf("SD card mount success !\n"); 106 return 0; 107 } 108 109 int main(void) 110 { 111     bmpCnt = 0; 112     char picName[20]; 113     int ret; 114     SCCB_Init(); 115     OV7725_Init(); 116     videoMode = VMO_640x480; 117     run_vdma_frame_buffer(&vdma, VDMA_DEVICE_ID, videoMode.width, videoMode.height, fbBase, 0, 0, BOTH); 118     memset((void *)fbBase, 0, videoMode.height*videoMode.width*3*3); 119     Xil_DCacheFlush(); 120     DisplayInitialize(&dispCtrl, XVTC_DEVICE_ID, CLOCK_ADDR); 121     DisplaySetMode(&dispCtrl, &videoMode); 122     DisplayStart(&dispCtrl); 123     if (Mount_Init() < 0) 124         return -1; 125     for (; ) { 126         scanf("%c", &cmd); 127         if (cmd == 'a') { 128             printf("capture picture\n"); 129             // Get the current frame 130             readIndex = XAxiVdma_CurrFrameStore(&vdma, XAXIVDMA_READ); 131             readFrameAddr = fbBase + videoMode.height*videoMode.width*3*readIndex; 132             XAxiVdma_StopParking(&vdma, 133             if (ret == 0) 134                 printf("save %s successfully\n", picName); 135             else 136                 printf("Failed to save file %s\n", picName); 137         } 138     } 139     return 0; 140 }
```

Line 21~42 of the code defines the file header and information header data types of the BMP image. #pack(push, 1) sets the alignment to ensure that the structure is aligned according to 1 byte, that is, the addresses of the structure members are closely adjacent, ensuring that the layout of the data in the memory is completely consistent with the specifications of the BMP file format. #pragma pack(pop) is used to cancel the previous settings and restore the default alignment.

Line 44 of code, bmpAddr is used to save the captured image data.

Line 48~96 of code first fills in the BMP file header and information header according to the passed parameters width and height, in which the color depth defaults to 24bit, and then writes the file header, information header, and pixel data in sequence. The captured image data is saved as a BMP format picture.

Line 98~107 of code is to mount the SD card.

Line 109~159 of code is the definition and implementation of the main function. First, initialize the camera, configure VDMA and display according to the selected resolution rate. Since the maximum resolution supported by ov7725 is 640x480, the resolution set in this experiment is also 640x480. Then mount the Fatfs file system. Then an infinite loop is performed, in which the command sent by the serial port is accepted. When the a character is received, the image is captured and saved as a bmp image file.

It should be noted that in this experiment, the VDMA write channel and read channel were enabled, and three frame buffers were used. The read and write channels will cycle through the three frame buffers to read and write, and due to the synchronization phase lock mechanism, the read and write channels cannot operate on the same frame buffer at the same time. When the a character is received from the serial port, the frame number of the current read channel will be obtained first, and the position of the current frame in the video memory will be calculated based on the read index. However, before the image data is copied to memcpy, the XAxiVdma_StartParking() function will be called to prevent the read channel from switching frames and stay in the frame buffer corresponding to the specified number. At the same time, the write channel is also prohibited from writing to the frame buffer to prevent the newly collected image data from the camera from destroying the image data currently being captured. After the image data copy is completed, the XAxiVdma_StopParking() function will be called to restore the read and write channels to perform cyclic read and write operations on these three frame buffers. Through this processing, it is ensured that the captured image is a complete image.

29.3.4 Board verification

As shown in Figure 29-5, the development board is connected to the 12V DC power supply, XILINX downloader JTAG port, HDMI display and OV7725 camera, and the SD card is inserted. After the lines are connected correctly, turn on the switch to power on the board.

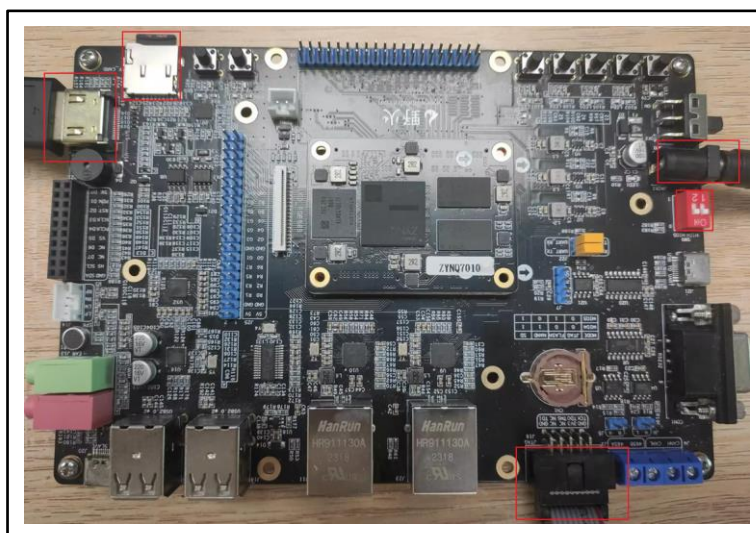


Figure 29-5 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system and Program FPGA -> Run.

After the program download is completed, open the serial port terminal, and the HDMI will display the current real-time collected image. After sending the character 'a', the SD card will be triggered to save the current image collected by the camera, as shown in Figure 29-6.

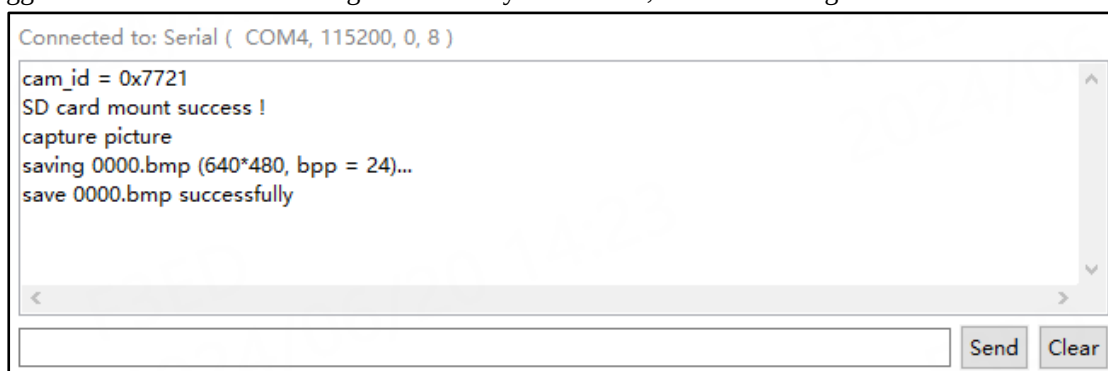


Figure 29-6 Serial port output information

Then you can use the card reader to see that the SD card saves the images collected by the camera. The format is bmp, the resolution is 640*480, and the color depth is 24bit, as shown in Figure 30-7.



Figure 29-7 Pictures saved to SD card

29.4 End of chapter summary

This chapter introduces how to save the images collected by ov7725 in the SD card in BMP image format to achieve the effect of camera photography. For related content about BMP format, please refer to the previous chapter "SD Card Reading BMP Display LCD".

29.5 Development training

You can try to achieve experimental results similar to this chapter based on the experiments in the "OV5640 HDMI

Chapter 30 WM8960 audio loopback experiment

30.1 Chapter Introduction

WM8960 is a low-power, high-quality stereo multimedia digital signal codec. It is mainly used in portable applications, such as digital cameras and camcorders. It combines a stereo differential microphone preamplifier with drivers for speakers, headphones and differential, stereo line outputs, reducing the number of external components necessary for the application, such as a separate microphone or headphone amplifier.

This chapter will take you through an audio loopback experiment to understand the driving method of WM8960.

30.2 Theoretical study

30.2.1 Basic concepts of sound

Sound is a continuous wave that propagates through a certain medium. Sound signals can be divided into two types (Nature, physics), one is digital signal (computer). The digitization process of the sound signal is shown in the figure below.

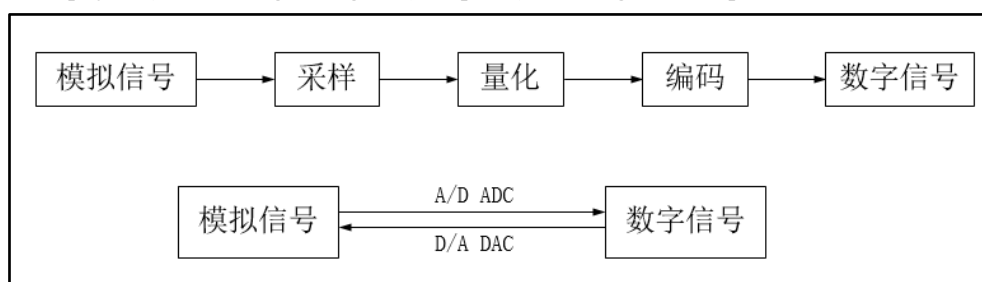


Figure 30-1 Sound digitization process diagram

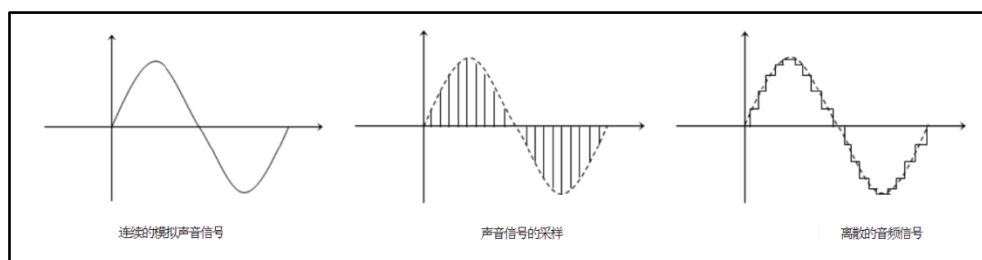


Figure 30-2 Waveform diagram of the sound digitization process

Three elements of sound digitization:

Table 30-1 Three elements of sound digitization

采样频率	量化位数	声道数
每秒钟抽取声波幅度样本的次数	每个采样点用多少二进制位表示数据范围	使用声音通道的个数
采样频率越高声音质量越好 数据量也越大	量化位数越多音质越好数据 量也越大	立体声比单声道的表现力丰 富，但数据量翻倍

11.025 KHz 22.05 KHz 44.1 KHz	8 位 = 256 16 位 = 65536	单声道 立体声
-------------------------------------	---------------------------	------------

30.2.2 I2S audio bus

The collection, processing and transmission of audio data are important components of multimedia technology. Numerous digital audio systems have entered the consumer market, such as digital audio tapes, digital sound processors, etc. For equipment and manufacturers, a standardized information transmission structure can improve the adaptability of the system. I2S (Inter-IC Sound) bus, also known as integrated circuit built-in audio bus, is a bus standard developed by Philips for audio data transmission between digital audio devices. This bus is specially used for data transmission between audio devices and is widely used in various multimedia systems.

The I2S bus interface has three main interfaces:

(1) SD (Serial Data): Serial data line, used to send or receive data on two time-division multiplexed data channels. The highest bit of data is always transmitted first. (2) SCK (continuous serial clock): Serial clock, corresponding to each bit of digital audio data, SCK has a pulse. $\text{SCK frequency} = 2 \times \text{sampling frequency} \times \text{sampling bits}$. (3) WS (Word Select): Field selection line, also called frame clock (LRC) line, indicates the channel currently transmitting data. Different standards have different definitions. The frame rate of the WS line is equal to the sampling frequency (F_s).

30.2.3 WM8960 chip introduction

The WM8960 is a low-power, high-quality stereo codec CODEC designed for portable digital audio applications. Integrated 24bit ADC and DAC. Internal stereo Class D speaker drivers allow each channel to drive a 1W 8Ω speaker. Highly flexible input configuration integrating three stereo sources with a complete microphone interface.

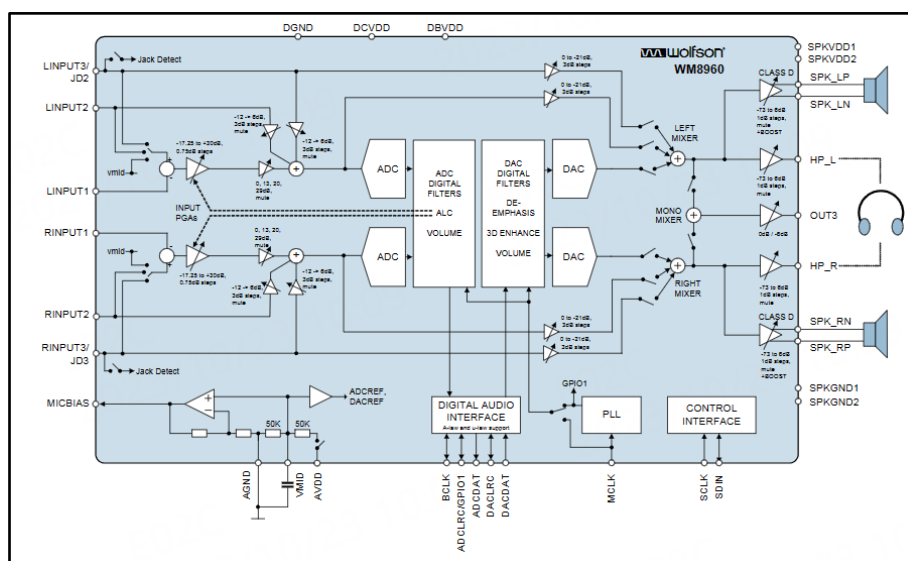


Figure 30-3 WM8960 internal structure

Data clock interface:

ADCDAT: ADC data output pin. The collected audio data is converted into digital signals and then transmitted to the main controller through this pin.

ADCLRC: ADC data alignment clock, also known as frame clock (LRCK), is used to switch left and right channel data. The frequency of this signal is the sampling rate. This pin can be configured as a GPIO function. After configuring it as a GPIO, the ADC will use the DACLRC pin as the frame clock.

DACDAT: DAC data input pin. The master controller inputs digital signals to the DAC of WM8960 through this pin.

DACLRC: DAC data alignment clock. Its function is the same as ADCLRC. It is a frame clock (LRCK). It is used to switch left and right channel data. The frequency of this signal is equal to the sampling rate.

BCLK: bit clock, used for synchronization.

MCLK: main clock, WM8960 also needs a main clock when working

A total of three audio data input channels are provided, namely LINPUT1/RINPUT1, LINPUT2/RINPUT2, and LINPUT3/RINPUT3. The microphone or line input is connected to this interface, which is fixed when drawing the board. The specific channel to use needs to be determined by looking at the schematic diagram of the Haoyue development board.

Control interface:

Standard IIC, this I2C interface is used to configure the WM8960 to work.

IIS interface: four pins are used. SCK: Serial clock signal. WS: Field (channel) selection signal. SDO: serial data signal output. SDI: Serial data signal input.

30.3 Practical exercises

30.3.1 Experimental objectives

Use the audio connection cable to input the audio from the computer or mobile phone into the ZYNQ through the WM8960, and then send the input audio data to the WM8960 to output it to the speakers or headphones to achieve the audio loopback effect.

30.3.2 Hardware resources

The schematic diagram of the WM8960 peripheral circuit on the Haoyue development board is shown in Figure 30

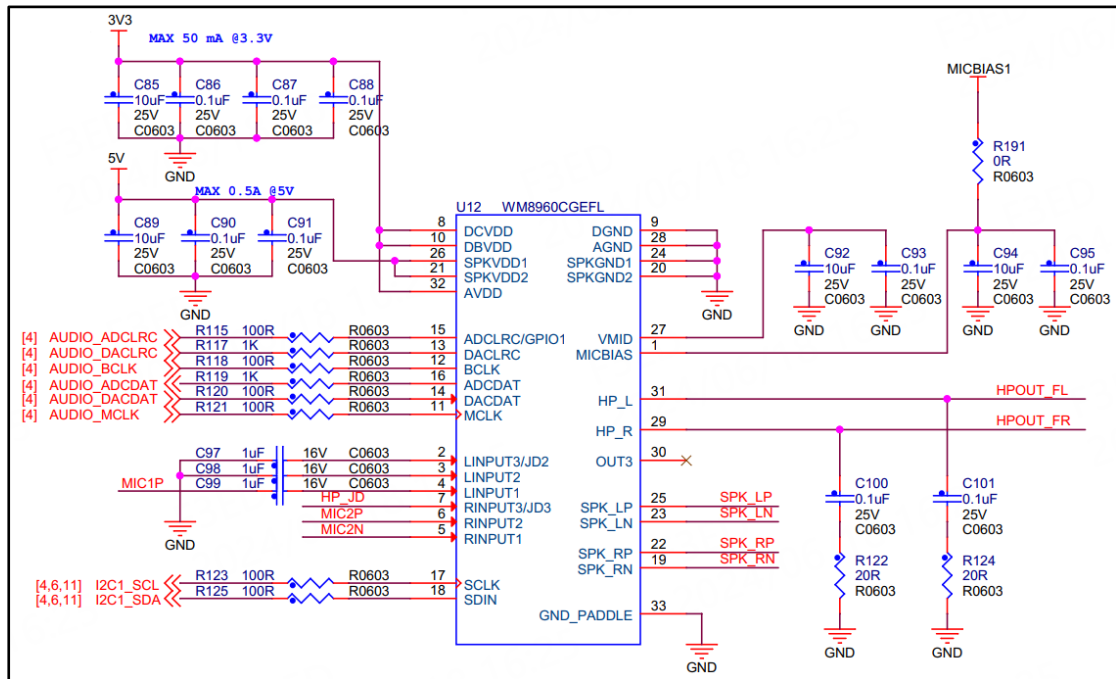


Figure 30-4 WM8960 peripheral circuit

30.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 30-5.

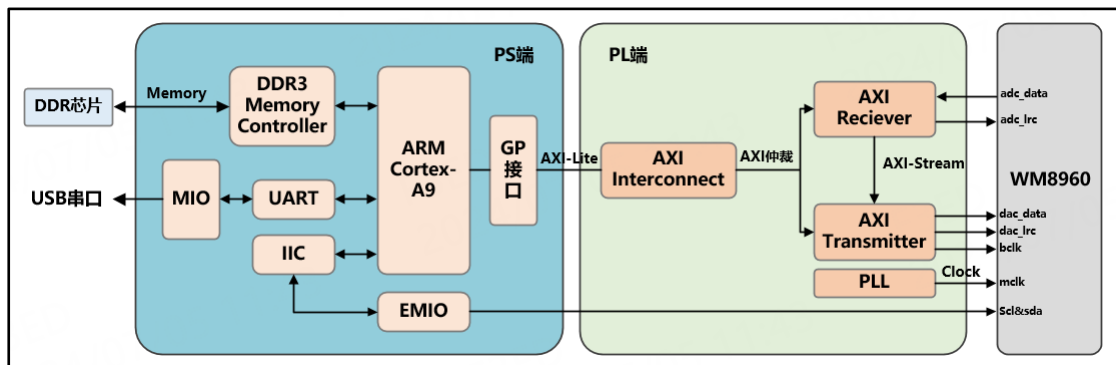


Figure 30-5 Overall block diagram

Looking at the schematic diagram, you can see that the IIC interface of WM8960 is connected to the PL side, so we need PS to use EMIO to lead out the pins for IIC configuration of the WM8960 chip, and the I2S Receiver is used to receive the audio data of the WM8960 in the I2S format, and then convert it to the AXI4-Stream format. I2S Transmitter outputs the received AXI4-Stream format data stream to the dac interface of WM8960 according to the I2S protocol format.

The PS end and the PL end conduct data exchange through the GP interface, use AXI-LITE to control the two I2S IP cores, and use AXI Interconnect for arbitration. Use PLL to output MCLK, and configure a serial port on the PS side to facilitate debugging and printing related information.

2. Hardware design

Detailed steps for project creation can be found in the previous chapters. Below we only list the configuration process of the ZYNQ system IP.

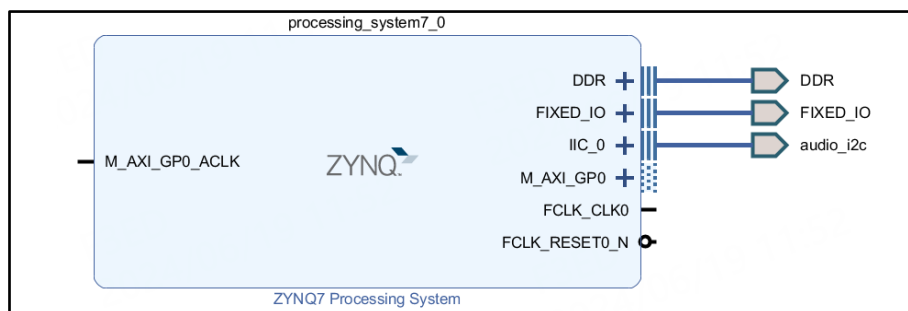


Figure 30-6 ZYNQ7 system module

It is basically the same as the experiment in the "Customized IP Core" chapter, except that an additional I2C controller is added for configuring the WM8960.

I2S Receiver IP

Right-click on the blank space of the Diagram and select Add IP. In the pop-up window, select the "I2S Receiver" IP core and double-click to add it. After the addition is completed, double-click the IP core to enter the configuration interface.

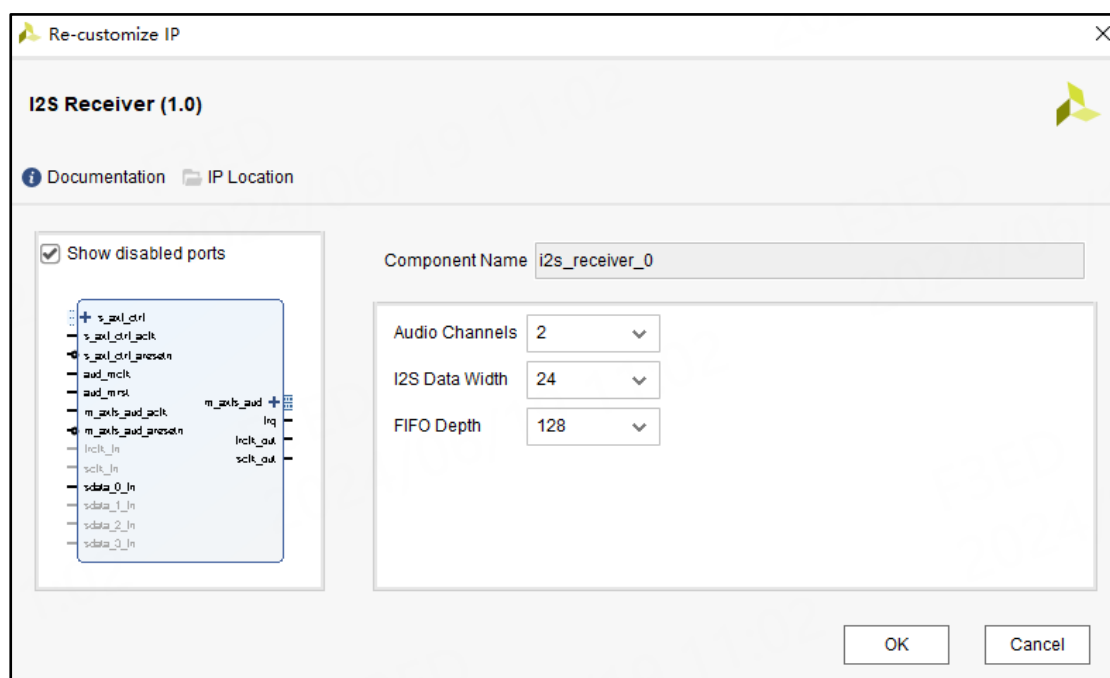


Figure 30-7 I2S Receiver configuration interface

It can be seen that there are three main parameters that can be configured. Audio Channels is the number of audio channels, I2S Data Width is the data bit width, which is the data bit width sampled by the ADC in the WM8960 chip, and FIFO Depth is the depth of the FIFO buffer used internally in the IP.

I2S Transmitter IP

Re-customize IP

I2S Transmitter (1.0)

Documentation
 IP Location

☒ Show disabled ports

Component Name

Audio Channels

I2S Data Width

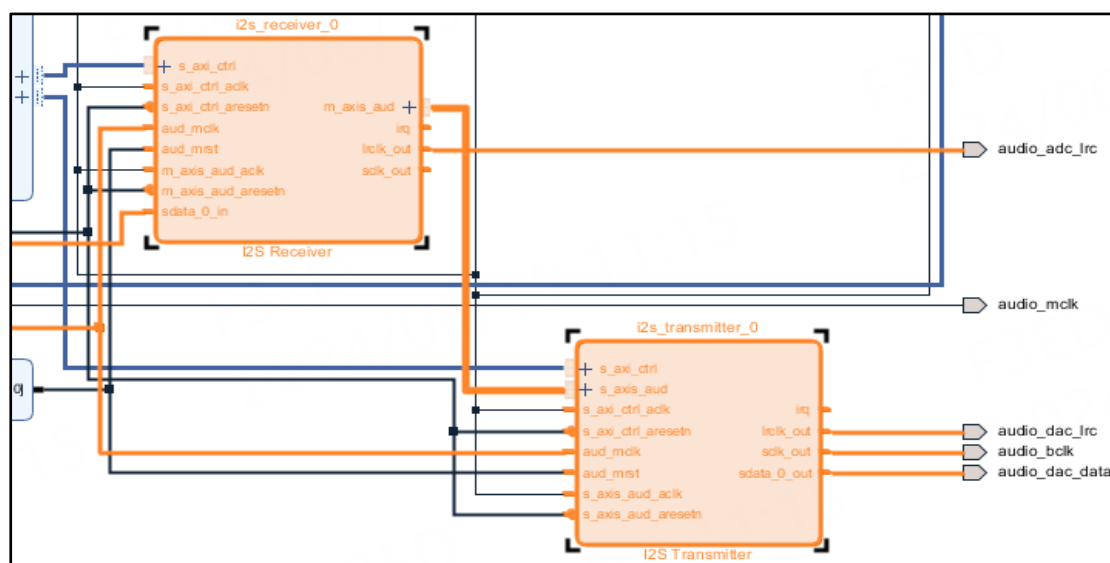
FIFO Depth

☐ Enable FIFO Data Count

OK

Cancel

It can be seen that the configuration parameters of I2S Transmitter IP and I2S Receiver are basically the same parameters and have the same meaning, but there is an additional option of Enable FIFO DATA COUNT. The function of this option is whether to turn on FIFO read and write data counting. This experiment does not require it, so uncheck it and keep the default configuration.



346 / 376

Because these two I2S IPs only support master mode, our WM8960 should be configured as slave mode, so the BCLK and LRC here are no longer output by the WM8960, but should be output by the ZYNQ terminal.

I2S Transmitter transmits audio data to WM8960, so three interfaces are introduced here: lrc_k_out: DAC data alignment clock. sdata_0_out: audio output interface. sclk_out: bit clock BCLK.

I2S_Receiver leads out lrclk_out, which is the ADC data alignment clock. sdata_0_in: It is the external input audio data, which is also extracted.

The m_axis_aud of I2S_Receiver is the received audio data, which needs to be connected to the s_axis_aud interface of I2S Transmitter, and the received audio data is given to the I2S Transmitter IP.

Next add the clock module. As shown below:

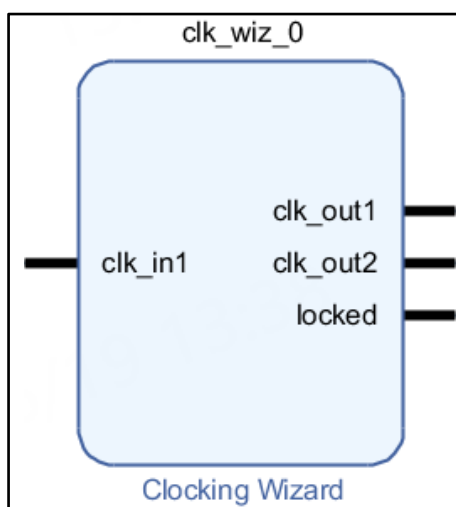


Figure 30-10 CLK module

Clocking Options									
Output Clocks									
MMCM Settings									
Summary									
The phase is calculated relative to the active input clock.									
Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)		Drives	
		Requested	Actual	Requested	Actual	Requested	Actual		
☒ clk_out1	clk_out1	12	12.000	0.000	0.000	50.000	50.0	BUFG	
☒ clk_out2	clk_out2	18.432	18.432	0.000	0.000	50.000	50.0	BUFG	
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A	BUFG	
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A	BUFG	
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A	BUFG	
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A	BUFG	
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A	BUFG	

Figure 30-11 Configuration interface

When the module inputs, the PL side system clock is 50MHZ. We must enable two clocks. clk_out1 is the driving clock for the WM8960 chip. And clk_out2 is the sampling clock for audio reception and transmission. According to the manual, the clock calculation is related to the sampling frequency. The sampling frequency of our experiment is 48KHZ. According to the formula, MCLK is an integer multiple of 128*FS. The clock here should be $\text{clk} = 48\text{KHZ} * 128 * n$ (n is an integer). Here, n=3 is taken, then MCLK is approximately equal to 18.432MHZ.

Next add a negation module as shown below:

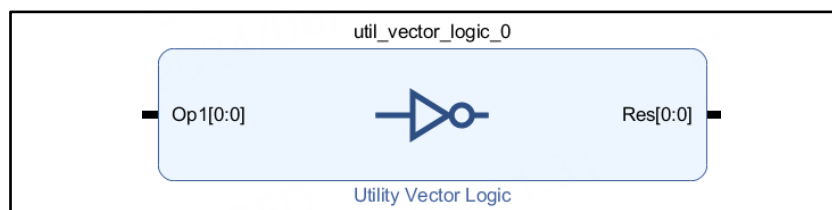


Figure 30-12 Negate IP

Since the resets of both I2S IPs are high-level resets, we need to invert the locked signal generated by CLK and then input it into the resets of these two IPs, as shown below:

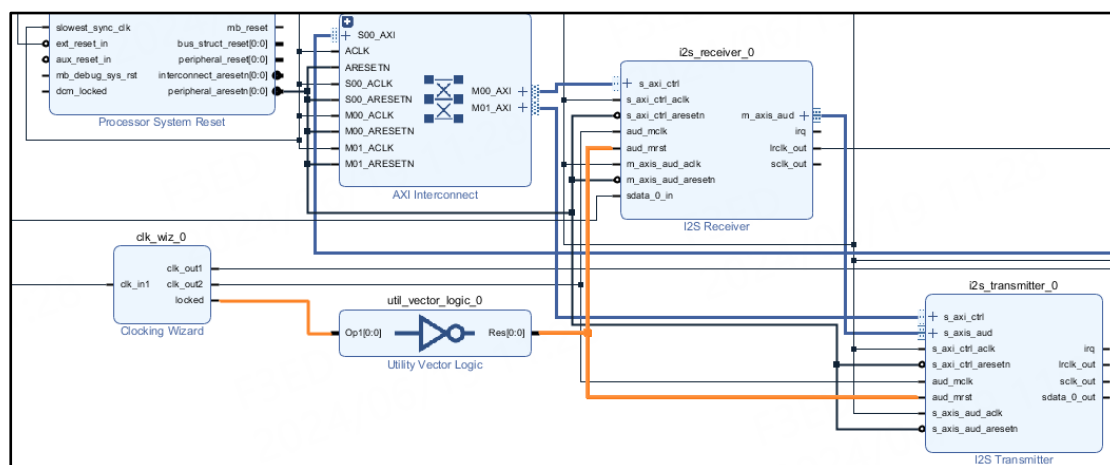


Figure 30-13 Connection diagram

The final overall IP module connection diagram is shown in Figure 30-14:

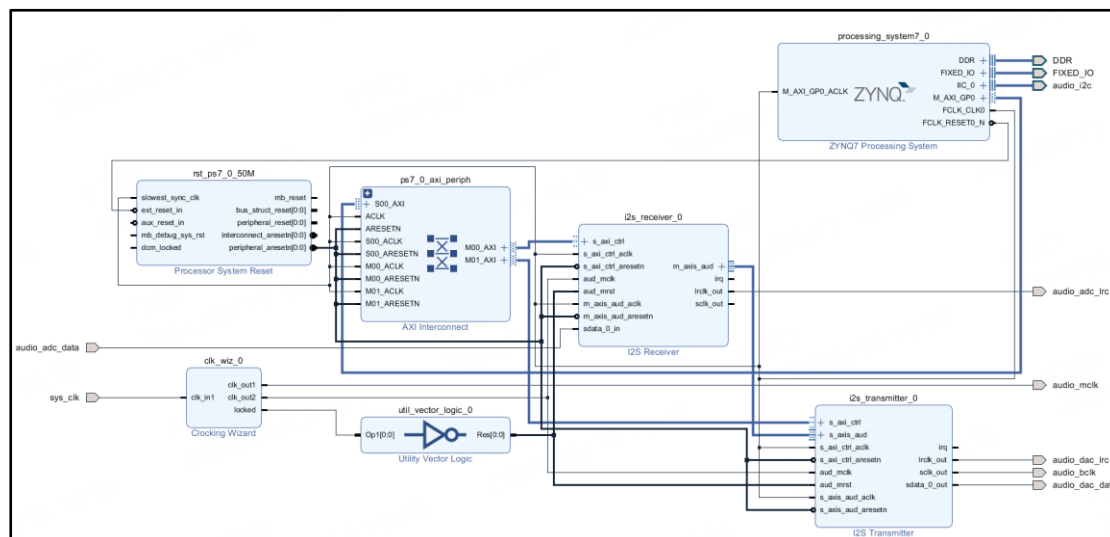


Figure 30-14 Overall IP block diagram

At this point, our Block Design is completed. Press the "F6" key to perform "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally press the shortcut key "Ctrl + S" to save the design.

Then execute "Generate Output Products" and "Create HDL Wrapper" in sequence. Then perform pin constraints on the used pins.

Code Listing 30-1 Pin Constraints (main.c)

```
1 set_property PACKAGE_PIN K19 [get_ports audio_adc_lrc]
2 set_property PACKAGE_PIN J19 [get_ports audio_dac_lrc]
3 set_property PACKAGE_PIN J18 [get_ports audio_bclk]
4 set_property PACKAGE_PIN H18 [get_ports audio_adc_data]
5 set_property PACKAGE_PIN H16 [get_ports audio_dac_data]
6 set_property PACKAGE_PIN H17 [get_ports audio_mclk]
7 set_property IOSTANDARD LVCMOS33 [get_ports audio_mclk]
8 set_property IOSTANDARD LVCMOS33 [get_ports audio_adc_data]
9 set_property IOSTANDARD LVCMOS33 [get_ports audio_adc_lrc]
10 set_property IOSTANDARD LVCMOS33 [get_ports audio_bclk]
11 set_property IOSTANDARD LVCMOS33 [get_ports audio_dac_data]
12 set_property IOSTANDARD LVCMOS33 [get_ports audio_dac_lrc]
13 set_property PACKAGE_PIN N17 [get_ports audio_i2c_scl_io]
14 set_property PACKAGE_PIN P18 [get_ports audio_i2c_sda_io]
15 set_property IOSTANDARD LVCMOS33 [get_ports audio_i2c_scl_io]
16 set_property IOSTANDARD LVCMOS33 [get_ports audio_i2c_sda_io]
17
18 set_property PACKAGE_PIN N18 [get_ports sys_clk]
19 set_property IOSTANDARD LVCMOS33 [get_ports sys_clk]
20 set_property IOSTANDARD LVCMOS33 [get_ports sys_rst_n]
21 set_property PACKAGE_PIN G15 [get_ports sys_rst_n]
```

Then generate a bit stream file and export the hardware (check "Include bitstream" in the pop-up dialog box). Finally launch the SDK.

3. Software design

In the hardware design part, we started the SDK development environment, created a new application project, and

Code Listing 30-2 Audio loopback reference code (main.c)

```
1 #include <stdio.h> 2 #include "xparameters.h" 3 #include "xi2srx.h" 4 #include "xi2stx.h" 5 #
include "xiicps.h" 6 7 #define FS 48000 8 #define WM8960_FREQ 3*128*FS //WM8960 clock
9 #define I2C_DEV_ADDR 0x1A //Device address 10 #define I2C_FREQ 100000 //i2c clock f
requency 11 12 XI2srx_Config *I2srx_ptr; 13 XI2stx_Config *I2stx_ptr; 14 XI2s_Rx I2s_Rx;
17 XI2s_Tx I2s_Tx; 18 (addr<<1) | ((u8)((date>>8) & 0x0001)); 26 //Data 27 DATE_DUFFE
R[1] = (u8)(date & 0x00FF); 28
```

```
29     XIicPs_MasterSendPolled(&IicPs, DATE_DUFFER, 2, I2C_DEV_ADDR);
30     while (XIicPs_BusIsBusy(&IicPs)) ; //等待写入
31 }
32
33 void WM8960_Init(void) {
34     /*向寄存器写入初始化参数*/
35     I2C_Write(15, 0x0000); //复位
36     I2C_Write(25, 0x00FC); //使能相应输入通道以及 ADC 通道
37     I2C_Write(26, 0x01F9); //使能 DAC 通道、输出通道、PLL
38     I2C_Write(47, 0x000C); //使能左右通道的 mixer
39     I2C_Write(34, 0x0100); //左 DAC 输出的 mixer
40     I2C_Write(37, 0x0100); //右 DAC 输出的 mixer
41     I2C_Write( 5, 0x0000);
42     I2C_Write( 2, 0x0179);
43     I2C_Write( 3, 0x0179);
44     I2C_Write(32, 0x0118);
45     I2C_Write(33, 0x0118);
46     I2C_Write(23, 0x01c4);
47     I2C_Write(51, 0x011B);
48     I2C_Write(49, 0x00F7);
49     I2C_Write(40, 0x0179);
50     I2C_Write(41, 0x0179);
51     I2C_Write(43, 0x007A); //0050
52     I2C_Write(44, 0x005E); //000A
53     I2C_Write( 7, 0x000A);
54     I2C_Write( 4, 0x0005);
55     I2C_Write(48, 0x001E); //0005
56     I2C_Write(52, 0x0028);
57 }
58
59 int main()
60 {
61     /*配置 i2s 接收端*/
62     I2srx_ptr = XI2s_Rx_LookupConfig(XPAR_XI2SRX_0_DEVICE_ID);
63     XI2s_Rx_CfgInitialize(&I2s_Rx, I2srx_ptr, I2srx_ptr->BaseAddress);
64     XI2s_Rx_SetSclkOutDiv(&I2s_Rx, WM8960_FREQ, FS);
65     XI2s_Rx_Enable(&I2s_Rx, TRUE);
66
67     /*配置 i2s 发送端*/
68     I2stx_ptr = XI2s_Tx_LookupConfig(XPAR_XI2STX_0_DEVICE_ID);
69     XI2s_Tx_CfgInitialize(&I2s_Tx, I2stx_ptr, I2stx_ptr->BaseAddress);
70     XI2s_Tx_SetSclkOutDiv(&I2s_Tx, WM8960_FREQ, FS);
71     XI2s_Tx_Enable(&I2s_Tx, TRUE);
72
73     /*初始化 i2c*/
74     Iic_ptr = XIicPs_LookupConfig(XPAR_XIICPS_0_DEVICE_ID);
75     XIicPs_CfgInitialize(&IicPs, Iic_ptr, Iic_ptr->BaseAddress);
76     XIicPs_SetSCLk(&IicPs, I2C_FREQ);
77
78     WM8960_Init();
79
80     return 0;
81 }
```

In lines 7-10 of the code, the sampling frequency and the final output MCLK clock are respectively defined, as well as the IIC communication rate of 100KHZ. Because we are using the two IPs of I2S Receiver and I2S Transmitter, two new header files are introduced. xi2srx.h and xi2stx.h respectively store functions related to I2S transceiver functions.

The I2C_Write function is defined in lines 20-31 of the code. Its function is to write data to the WM8960 through IIC. The first parameter of the configuration register is written to the address of the register we want to write, and the second parameter is written to the value to be configured.

What needs to be noted here is that looking at the register manual of WM8960, we can find that the register address is 7bit and the data is 9bit, so when we send it, we need to put the register address and the highest bit of the register data together to make it 8bit, and then wait until the WM8960 register responds and then send the remaining 8bit register data to WM8960. As you can see in lines 22-29 of the code, we define an 8-bit array DATE_BUFFER with two elements. The first element shifts the assignment input address to the left by one bit or the input register data to the right by 8 and then adds 0x0001. "|" is used to splice data, while "&" can be used to mask data bits. Here `addr<<1` changes it from 7bit to 8bit (the low bit is filled with 0), and the `8-bit date>>` puts the 9th bit of data into the first bit of data. Then "&" is added with 0x0001 to ensure that only the first bit has data, and the rest are all 0. Finally, "|" is used to realize the splicing of 8-bit data. The second element can be directly assigned the remaining register data. Finally, the data is written to WM8960 through the IIC protocol through the `XIicPs_MasterSendPolled()` function provided by `xii cps.h`. The first parameter of this function is the pointer of the I2C controller instance; the second parameter is the cache address of the data to be sent, and the third parameter needs to specify the number of bytes to be sent, that is, sending two bytes of data; and the last parameter is the address of the I2C device. Looking at the schematic diagram, it can be seen that the device address of WM8960 is the binary number 0011010, which is the hexadecimal number 1A.

Then, in lines 35-56 of the code, make relevant configurations by checking the register manual of WM8960 so that it can achieve the corresponding functions. The function of this configuration is to allow WM8960 to implement audio input, speaker output or headphone output. This enables audio loopback. Readers can check the WM8960 register manual for more register configurations.

Then in the main function, I2S reception and transmission are initialized respectively, and `XI2s_Rx_SetSclkOutDiv()` and `XI2s_Tx_SetSclkOutDiv()` are used to configure the BCLK rate. We only need to enter the clock and sampling frequency of WM8960. Finally, use `XI2s_Rx_Enable()` and `XI2s_Tx_Enable()` to enable the corresponding modules. Finally, initialize the IIC configuration and use `XIicPs_SetSclk()` to set the communication rate to 100KHZ. Finally, just initialize the register configuration of WM8960.

30.3.4 Board verification

As shown in Figure 30-15, the development board is connected to a 12V DC power supply, the JTAG port of the XILINX downloader, and an audio cable is used to connect the development board and the computer. The computer audio is used as input, and the audio output can be connected to headphones or speakers. This experiment uses headphones. After the lines are connected correctly, turn on the switch to power on the board.

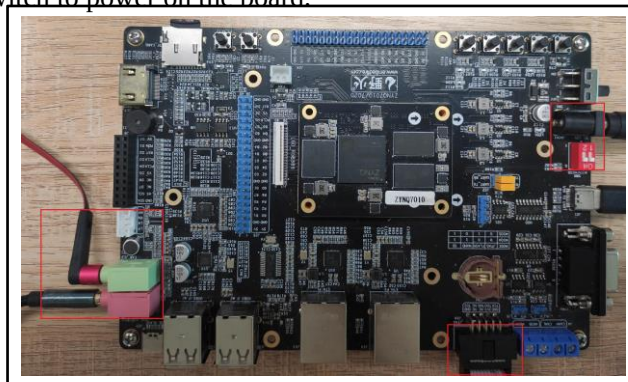


Figure 30-15 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system and Program FPGA -> Run.

After downloading the program, play a piece of music on the computer. At this time, you can hear it on the headphones.

30.4 End of chapter summary

This chapter first introduces the process of converting sound from analog signals to digital signals, and then explains the I2S bus interface and the internal structure and data clock interface of the WM8960 audio decoding chip. Finally, by implementing audio loopback on the ZYNQ platform, the entire process from analog-to-digital conversion (ADC) to digital-to-analog conversion (DAC) is completed.

30.5 Development training

Modify the relevant registers to control the volume and adjust it through buttons.

Chapter 31 TCP experiment based on lwip

31.1 Chapter Introduction

In embedded systems, Ethernet is a very important interface that allows us to transmit data to the network, such as sending it to a computer. Next, we will use LWIP to implement the TCP/IP protocol. LWIP is a lightweight IP protocol and a small open source TCP/IP protocol stack. Next we will learn how to use LWIP to implement TCP communication.

31.2 Theoretical study

31.2.1 Introduction to TCP/IP protocol

Ethernet TCP/IP is the communications protocol used on the Internet. The TCP/IP communication protocol is a description of the rules that computers must follow. Only by complying with these rules can computers communicate with each other. TCP provides reliable data transmission in the IP environment. The services it provides include data streaming, reliability, effective flow control, full-duplex operation, and multiplexing. Delivered via connection-oriented, end-to-end and reliable packets. In layman's terms, it opens up a connected channel for the data to be sent in advance, and then sends the data.

In terms of the protocol layered model, TCP/IP consists of four layers: network interface layer, network layer, transport layer, and application layer. OSI (Open System Interconnection) is an open system interconnection reference model. This model divides TCP/IP into seven layers: physical layer, data link layer, network layer, transport layer, session layer, presentation layer and application layer. As shown below:

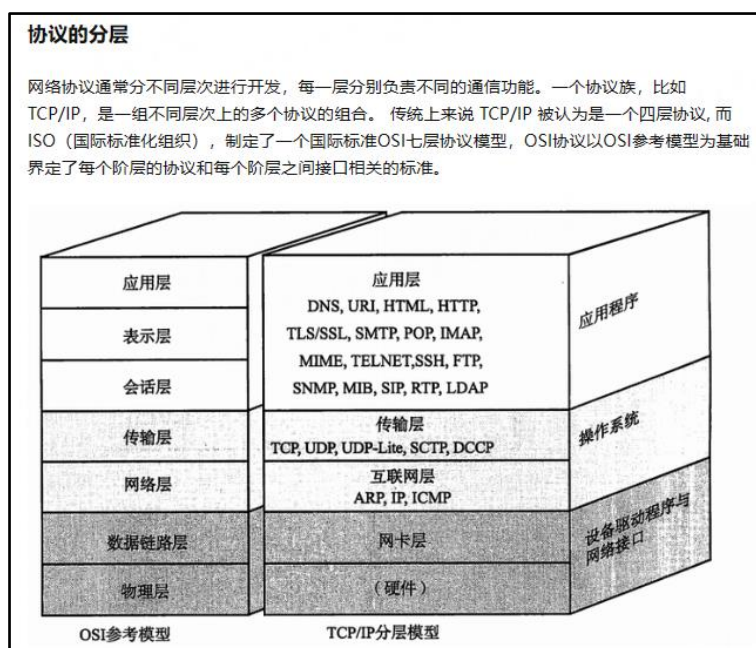


Figure 31-1 OSI model and TCP/IP model

31.2.2 Introduction to LWIP

The full name of LwIP: Light weight IP, which means lightweight TCP/IP protocol, is a small open source TCP/IP protocol stack developed by Adam Dunkels of the Swedish Institute of Computer Science (SICS). The original design intention of LwIP is to implement a relatively complete TCP/IP protocol stack with a small amount of resource consumption. "Complete" mainly refers to the integrity of the TCP protocol. The focus of the implementation is to reduce the RAM usage while maintaining the main functions of the TCP protocol. In addition, LwIP can be ported to run on an operating system or run independently without an operating system.

LwIP has key features:

Supports ARP protocol (Ethernet Address Resolution Protocol). Supports ICMP protocol (Control Message Protocol) for network debugging and maintenance. Supports IGMP protocol (Internet Group Management Protocol), which can realize the reception of multicast data. Supports UDP protocol (User Datagram Protocol). Supports TCP protocol (Transmission Control Protocol), including blocking control, RTT estimation, fast recovery and fast forwarding.

Supports PPP protocol (point-to-point communication protocol) and PPPoE.

Supports DNS (domain name resolution).

Supports DHCP protocol and dynamically assigns IP addresses. Supports IPsec protocol, supports IPv4, IPv6 protocols, IP fragmentation and reassembly functions, and multi-network interface packet forwarding.

Supports SNMP protocol (Simple Network Management Protocol). Supports AUTOIP, automatic IP address configuration. Provides a specialized internal callback interface (Raw API) to improve application performance. Provides an optional Berkeley interface API, namely Socket (used in multi-threaded situations).

At the same time, LwIP provides three programming interfaces, namely RAW/Callback API, NETCONN API, SOCKET API.

RAW/Callback API refers to the kernel callback API. In a streaming environment without operating system support, this API can only be used for development. At the same time, this API can also be used in the operating system environment.

NETCONN API is implemented based on the IPC mechanism of the operating system (ie, semaphore and mailbox mechanism). Its design separates the LwIP kernel code and network application into independent threads. In this way, the LwIP kernel thread is only responsible for TCP/IP encapsulation and decapsulation of data packets, without performing application layer processing of data, which greatly improves the system's processing efficiency of network data packets.

Socket, that is, socket, provides a high-level abstraction of network connections, allowing users to operate network connections like files.

This time we will use lwip202_v1_2, which is a library built on lwip2.0.2 provided by the SDK2018.3 version. The library provides two user programming interface methods: raw API and socket API.

This time we are not using an operating system, so we only use the raw API and not the socket API.

31.2.3 PS Gigabit Ethernet Controller

This time we need to use the Gigabit Ethernet interface on the PS side. When configuring vivado, we need to open the Gigabit Ethernet Controller (MAC) in the ZYNQ IP core. The built-in Ethernet MAC of the SOC can be connected to the Lebu PHY chip through the MII/RMII interface or GMII/RGMII interface to complete network data transmission.

MII (Media Independent Interface), or media independent interface, is an Ethernet industry standard defined by IEEE-802.3. It is used to connect the Ethernet MAC layer and PHY chip. It was later gradually expanded to: MII, RMII, SMII, GMII, RGMII, etc.

MII: 100M Ethernet interface, single edge sampling.

RMII: 100M Ethernet interface, a simplified version of MII, dual edge sampling.

GMII: Gigabit Ethernet interface, single edge sampling.

RGMII: Gigabit Ethernet interface, a simplified version of GMII, dual edge sampling.

MII communication requires 13 wires, GMII requires 20 wires, and RGMII only requires 12 communication wires. RGMII functionally implements Gigabit Ethernet, but the signal wires are greatly reduced.

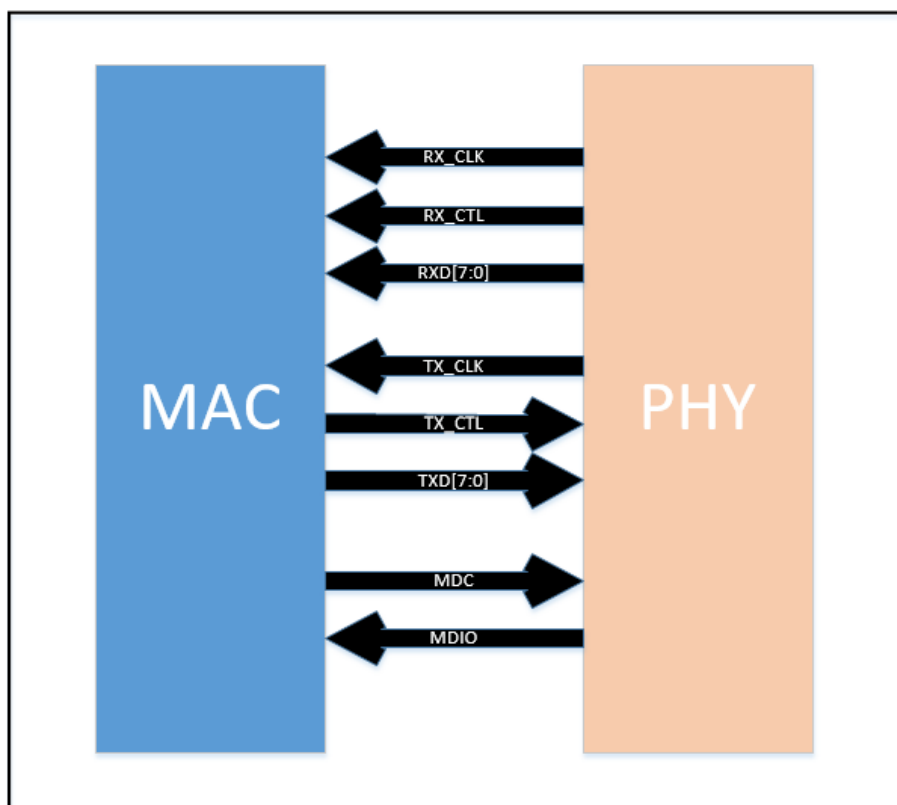


Figure 31-2 Schematic diagram of GMII interface

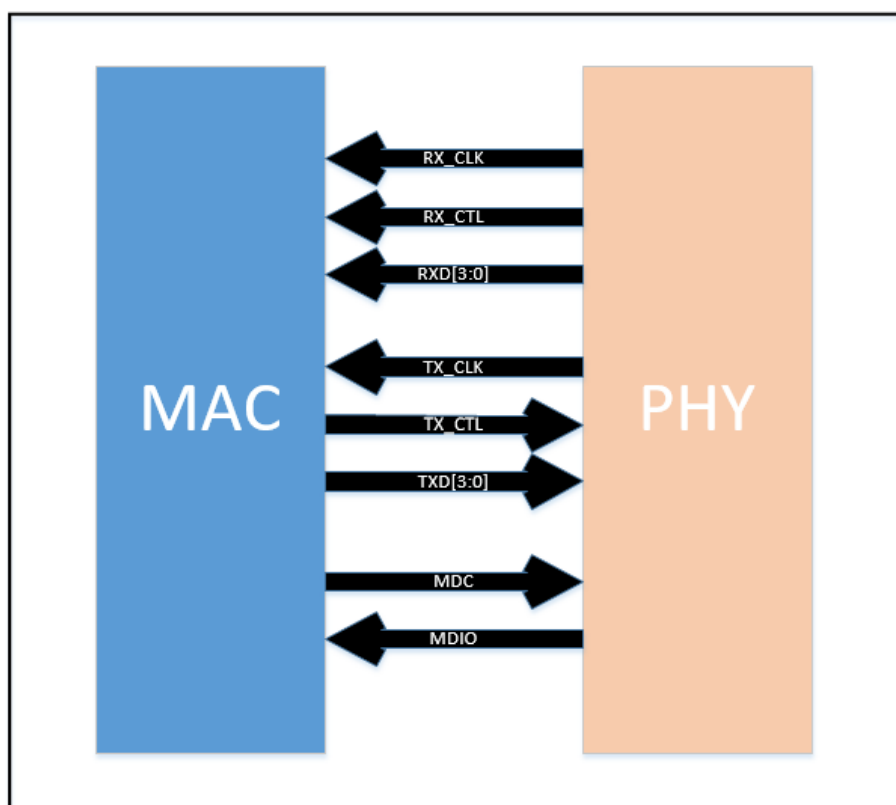


Figure 31-3 RGMII interface diagram

TX_CLK: Data transmission clock line. The nominal transmission rate is 2.5MHz when the rate is 10Mbit/s; it is 25MHz when the rate is 100Mbit/s, and it is 125M when the rate is 1000Mbit/s. **TX_CTL**: Data transmission control enable. Maintains an active level during the entire data transmission process. **TXD[7:0] or TXD[3:0]**: Data transmission data line. There are 8 bits for GMII, sampled on the rising edge of the clock, and only 4 bits for RGMII, sampled on both edges of the clock. The data line is only valid when TX_CTL is at an active level. **RX_CLK**: Data reception clock line. Driven by the PHY chip. The nominal transmission rate is 2.5MHz at 10Mbit/s; 25MHz at 100Mbit/s; and 125M at 1000Mbit/s. **RX_CTL**: Receive data valid signal, the function is similar to TX_CTL, but it is used for data reception and is driven by the PHY chip. **RXD[7:0] or RXD[3:0]**: Data receiving data line, driven by the PHY chip. There are 8 bits for GMII and only 4 bits for RGMII. The data line is only valid when RX_CTL is at an active level. In addition, there are some more important interfaces in the Ethernet interface. Although not used in this experiment, it is necessary to understand them, as follows: **RX_ER**: Receive error signal line, driven by PHY, reports to MAC that an error is detected somewhere in the frame. **CRS**: Carrier sense signal, driven by the PHY chip, enables this signal when the transmitting or receiving medium is in a non-idle state. This signal line is invalid in full-duplex mode. **COL**: Collision detection signal, driven by the PHY chip. This line is enabled after detecting a conflict on the medium and remains until the conflict is resolved. This signal line is invalid in full-duplex mode.

REF_CLK: used only for RMII interface, 50MHz reference clock provided by external clock source. Because to achieve a transmission speed of 100Mbit/s, the number of data lines of MII and RMII is different, and the design of clock lines using MII and RMII is completely different. For the MII interface, a 25MHz clock source is generally provided externally to the PHY, and then the PHY provides the TX_CLK and RX_CLK clocks. For the RMII interface, it is generally necessary to provide a 50MHz clock source directly from the outside and access the MAC and PHY at the same time.

MDC: The clock signal of the SMI serial management interface. It is an aperiodic signal. The minimum period of the signal (actually the sum of the positive level time and the negative level time) is 400ns. The minimum positive level time and negative level time are 160ns. The maximum positive and negative level time is unlimited. It has nothing to do with TX_CLK and RX_CLK.

MDIO: The data signal of the SMI serial management interface. MDIO is a bidirectional data line used to transmit MAC layer control information and physical layer status information. MDIO data is synchronized with the MDC clock and is valid on the rising edge of MDC.

Next, we will introduce ZYNQ's Ethernet controller

This time we will use the PS-GEM0 interface. As shown in Figure 31-4, this interface is connected to the external PHY chip through GMII TO RGMII. And using RGMII is also beneficial because it saves pins. At the same time, the controller provides a DMA for data buffering and transmission. The other interface, GEM1, uses the SGMII interface and can be connected to the SFP interface. You can also connect to the Ethernet interface on the PL side through EMIO, but at this time only the GMII interface can be used. Since the PL side and PS side of GEM1 share a MAC, they cannot be used at the same time. and can be connected to memory via the AXI bus

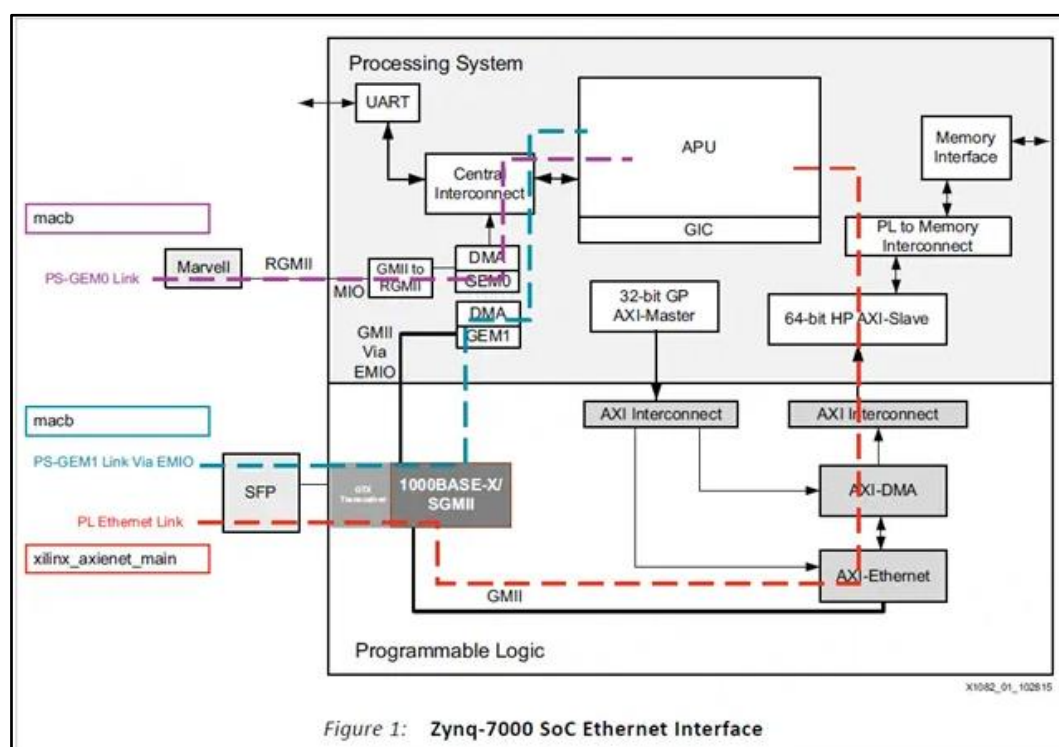


Figure 1: Zynq-7000 SoC Ethernet Interface

Figure 31-4 ZYNQ Ethernet controller

31.3 Practical exercises

31.3.1 Experimental objectives

By building PS-side Ethernet, use the lwip Echo Sever routine that comes with the SDK to implement TCP communication and understand the use of LWIP.

31.3.2 Hardware resources

This experiment uses the PS side Ethernet interface and UART.

31.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 31-5.

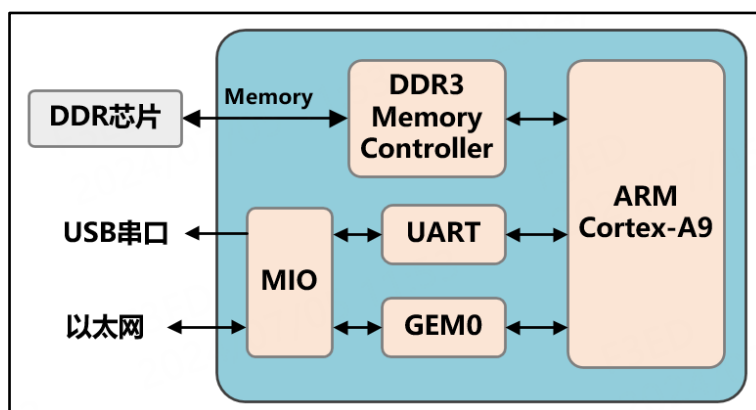


Figure 31-5 Overall block diagram

In the real hardware design where UART is used to print relevant information, the Gigabit Ethernet controller uses GEM0 to connect to the external Ethernet via MIO.

2. Hardware design

Detailed steps for project creation can be found in the previous chapters. Below we only list the configuration process for the ZYNQ system IP.

This experiment is basically the same as Hello World's Block Design. We only need to turn on the Ethernet controller. Double-click to open the ZYNQ system IP, check Ethernet0 and note that at the Bank1 voltage bit is LVC MOS 1.8V.

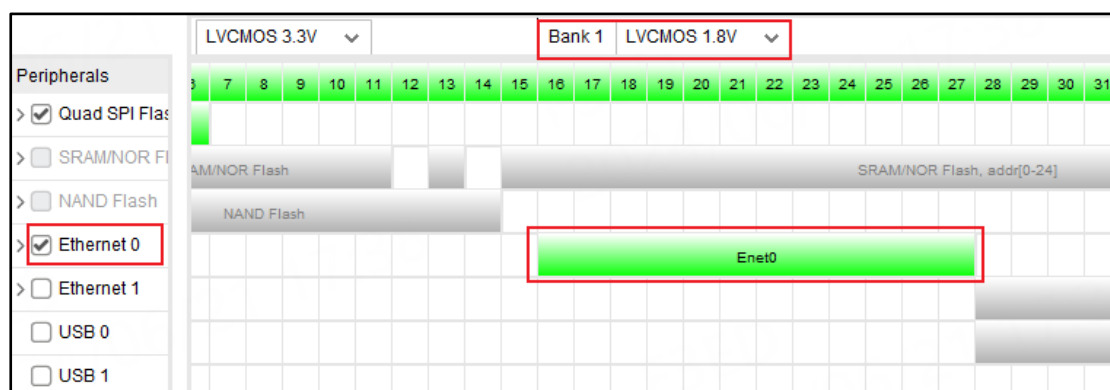


Figure 31-6 Open the Ethernet controller

The complete Block Design looks like this:

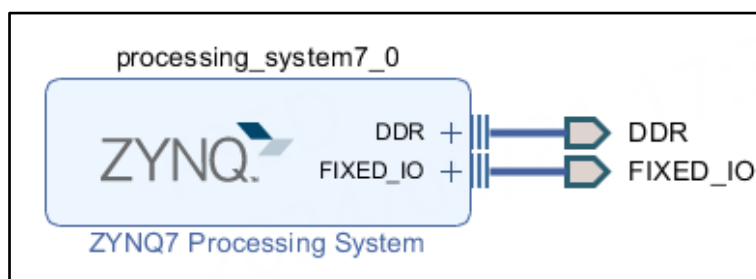


Figure 31-7 Overall IP block diagram

At this point, our Block Design is completed. Press the "F6" key to perform "Validate Design" to verify the design. After the verification is completed, a dialog box will pop up prompting "Validation Successful" indicating that the design is correct. Click "OK" to confirm. Finally press the shortcut key "Ctrl + S" to save the design.

Then execute "Generate Output Products" and "Create HDL Wrapper" in sequence. Then export the hardware. Since this experiment does not use PL resources, there is no need to generate and export bit stream files. Finally launch the SDK.

3. Software design

In the hardware design part, we started the SDK development environment and created a new application project. Instead of selecting a blank template, we selected the LWIP Echo Server template. As shown below:

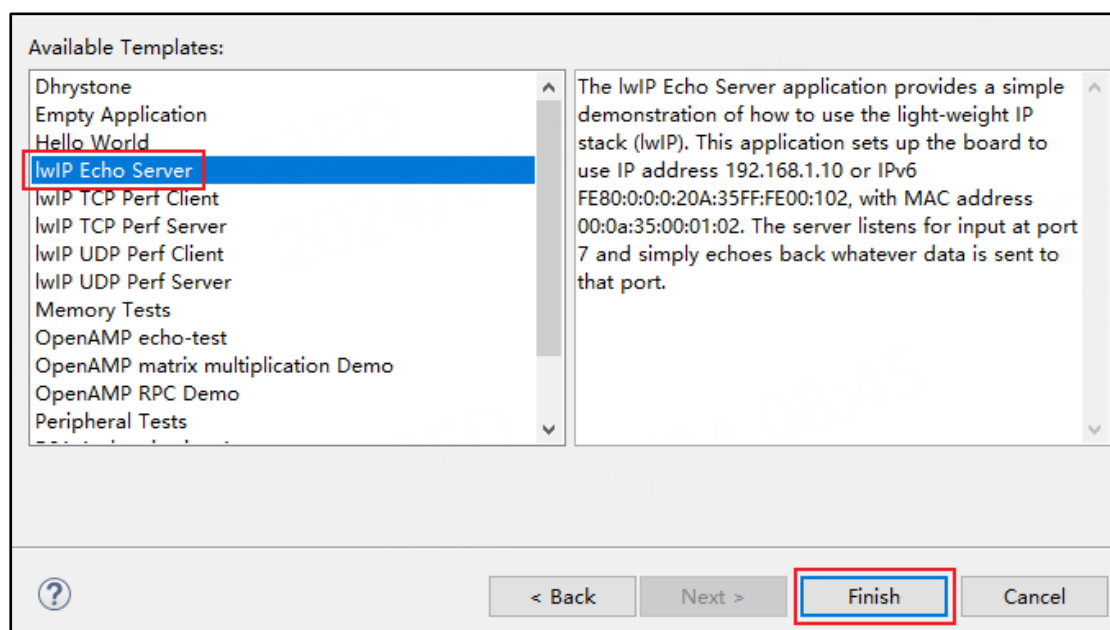


Figure 31-8 Select lwIP Echo Server

Next, open the project directory and check what files are there. This is the official routine of xilinx. For our board, some codes and files can be deleted.

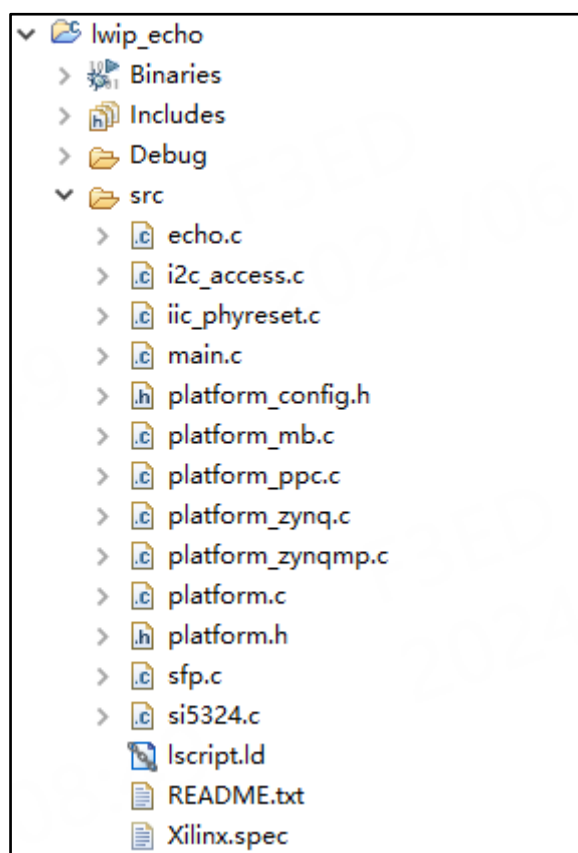
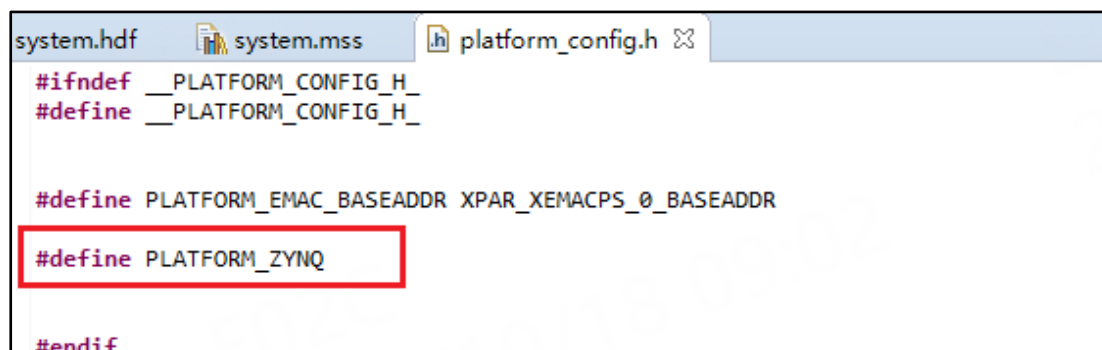


Figure 31-9 Project directory

Next, we will introduce the functions of these files:

Platform related files are configuration files of related hardware platforms. We are using ZYNQ this time, so we only need to keep the platform file platform_zynq.c. Platform_config.h defines macro definitions for all platform-related configurations. We just declare PLATFORM_ZYNQ.



```
system.hdf system.mss platform_config.h
#ifndef __PLATFORM_CONFIG_H__
#define __PLATFORM_CONFIG_H__

#define PLATFORM_EMAC_BASEADDR XPAR_XEMACPS_0_BASEADDR
#define PLATFORM_ZYNQ

#endif
```

Figure 31-10 Declare PLATFORM_ZYNQ

i2c_access.c and iic_phyreset.c are files used to access the PHY chip for access and reset through IIC. This experiment does not require the operation of the PHY chip, so it does not need to be used.

echo.c This is the file that implements the echo service.

main.c is the file that stores the main function.

The sfp.c and si5324.c files are suitable for the official xilinx development board, but our Haoyue does not have the PHY and si5324 chip of this sfp, so they can be discarded.

Next, the author will delete irrelevant files and keep key files. Right-click the file and select Delete. After deletion,

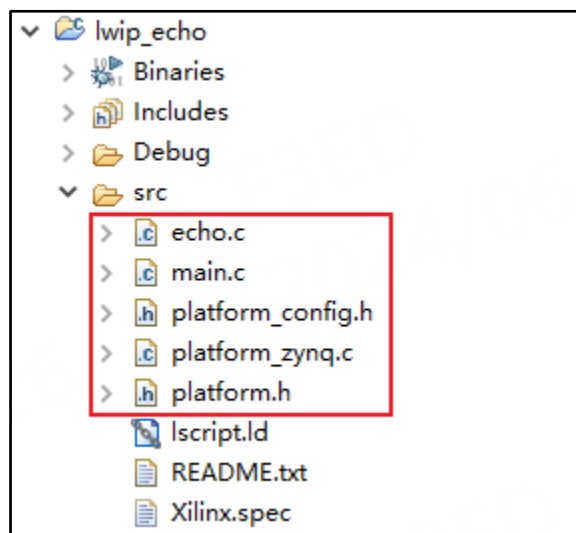


Figure 31-11 Folder after deletion

Next, open the main.c file and make relevant modifications to the file. We are using the ZYNQ series and not the ZCU102, so we deleted some precompiled instructions and added Chinese comments. Readers can look at the modified code to compare with the official code.

```
#if XPAR_GIGE_PCS_PMA_SGMII_CORE_PRESENT == 1 || XPAR_GIGE_PCS_PMA_1000BASEX_CORE_PRESENT == 1
int ProgramSi5324(void);
int ProgramSfpPhy(void);
#endif

#ifdef XPS_BOARD_ZCU102
#ifdef XPAR_XIICPS_0_DEVICE_ID
int IicPhyReset(void);
#endif
#endif
```

Figure 31-12 Delete related precompiled directives

The following is the modified code:

Code Listing 31-1 Modified code (main.c)

```
1 #include <stdio.h> 2 #include "xparameters.h" 3 #include "netif/xadapter
.h" 4 #include "platform.h" 5 #include "platform_config.h" 6 #include "xil
_printf.h" 7 #include "lwip/tcp.h" 8 #include "xil_cache.h" 9 10 #if LWIP_
IPV6==1 11 #include "lwip/ip.h" 12 #else 13 #if LWIP_DHCP==1 14 #in
clude "lwip/dhcp.h" 15 #endif 16 #endif 17 18 /* defined by each RAW m
ode application */ 19 void print_app_header(); 20 int start_application(); 2
1 int transfer_data(); 22 void tcp_fasttmr(void); 23 void tcp_slowtmr(void)
; 24 25 /* missing declaration in lwIP */ 26 void lwip_init(); 27 28 #if LWI
P_IPV6==0 29 #if LWIP_DHCP==1 30 extern volatile int dhcp_timeoutcnt
r; 31 err_t dhcp_start(struct netif *netif); 32 #endif 33 #endif 34 35 extern
volatile int TcpFastTmrFlag; 36 extern volatile int TcpSlowTmrFlag; 37 st
atic struct netif server_netif; 38 struct netif *echo_netif; 39 40 #if LWIP_I
PV6==1 41 void print_ip6(char *msg, ip_addr_t *ip) 42 { 43 print(msg); 4
4 xil_printf(" %x:%x:%x:%x:%x:%x:%x:%x\n\r", 45 IP6_ADDR_BLOC
K1(&ip->u_addr.ip6), 46 IP6_ADDR_BLOCK2(&ip->u_addr.ip6), 47 IP6
_ADDR_BLOCK3(&ip->u_addr.ip6), 48 IP6_ADDR_BLOCK4(&ip->u
_addr.ip6), 49 IP6_ADDR_BLOCK5(&ip->u_addr.ip6), 50 IP6_ADDR_B
LOCK6(&ip->u_addr.ip6),
```

```
51 IP6_ADDR_BLOCK7(&ip->u_addr.ip6), 52 IP6_ADDR_BLOCK8(&ip->u_addr.ip6)); 53 54 } 55 #else
56 /* Print ip address */ 57 void print_ip(char *msg, ip_addr_t *ip) 58 { 59 print(msg); 60 xil_printf("%d.%d
%d.%d\n\r", ip4_addr1(ip), ip4_addr2(ip), 61 ip4_addr3(ip), ip4_addr4(ip)); 62 } 63 64 /* Print IP configura
tion */ 65 void print_ip_settings(ip_addr_t *ip, ip_addr_t *mask, ip_addr_t *gw) 66 { 67 68 print_ip("Board
IP: ", ip); 69 print_ip("Netmask : ", mask); 70 print_ip("Gateway : ", gw); 71 } 72 #endif 73 74 int main() 75
{ 76 #if LWIP_IPV6==0 77 ip_addr_t ipaddr, netmask, gw; 78 79 #endif 80 /* Set the MAC address of the d
evelopment board without modification */ 81 unsigned char mac_ethernet_address[] = 82 { 0x00, 0x0a, 0x35
, 0x00, 0x01, 0x02 }; 83 84 echo_netif = &server_netif; 85 /* Initialize hardware platform */ 86 init_platform
(); 87
```

```
88 #if LWIP_IPV6==0 89 #if LWIP
_DHCP==1 90 ipaddr.addr = 0; 91 g
w.addr = 0; 92 netmask.addr = 0;
```

```
93 #else 94 /* Set the static IP address of the development board */ 95 I
P4_ADDR(&ipaddr, 192, 168, 1, 10); 96 IP4_ADDR(&netmask, 255, 2
55, 255, 0); 97 IP4_ADDR(&gw, 192, 168, 1, 102);
```

```
98 #endif 99 #endif 100 print_app_header(); 101 102 lwip_init(); 103 104 #if (LWIP_IPV6 == 0) 105 /* Add ne
twork interface to the netif_list, and set it as default *
```

```
106 if (!xemac_add(echo_netif, &ipaddr, &netmask, 107 &gw, mac_ethernet_address, 108 PLATFORM_EMA
C_BASEADDR)) { 109 xil_printf("Error adding N/W interface\n\r"); 110 return -1; 111 } 112 #else 113 /* Add
network interface to the netif_list, and set it as default *
```

```
114 if (!xemac_add(echo_netif, NULL, NULL, NULL, mac_ethernet_address, 115 PLATFORM_EMAC_BAS
EADDR)) {
```

```
116 xil_printf("Error adding N/W interface\n\r"); 117 return -1; 118 } 119 echo_netif->ip6_autoconfig_enabled = 1; 120 121 netif_create_ip6_linklocal_address(echo_netif, 1); 122 netif_ip6_addr_set_state(echo_netif, 0, IP6_ADDR_VALID); 123 124 print_ip6("\n\rBoard IPv6 address ", &echo_netif->ip6_addr[0].u_addr.ip
```

6);

```
125 126 #endif 127 netif_set_default(echo_netif); 128 129 /* Turn on interrupts */ 130 platform_enable_interrupts(); 131 132 /* Start the network */ 133 netif_set_up(echo_netif); 134 135 #if (LWIP_IPV6 == 0) 136 #if (LWIP_DHCP == 1) 137 138 139 /* Create DHCP client */ 140 dhcp_start(echo_netif); 141 /* Timeout setting */ 142 dhcp_timeoutcnt = 24; 143 144 while(((echo_netif->ip_addr.addr) == 0) && (dhcp_timeoutcnt > 0)) 145 xemacif_input(echo_netif); 146 /* If the DHCP connection times out, set the static IP directly */ 147 if (dhcp_timeoutcnt <= 0) { 148 if ((echo_netif->ip_addr.addr) == 0) { 149 dhcp_stop(echo_netif); 150 xil_printf("DHCP Timeout\n\r"); 151 xil_printf("Configuring default IP of 192.168.1.10\n\r"); 152 IP4_ADDR(&(echo_netif->ip_addr), 192, 168, 1, 10); 153 IP4_ADDR(&(echo_netif->netmask), 255, 255, 255, 0); 154 IP4_ADDR(&(echo_netif->gw), 192, 168, 1, 1); 155 } 156 } 157 158 ip_addr.addr = echo_netif->ip_addr.addr; 159 gw_addr = echo_netif->gw_addr; 160 netmask_addr = echo_netif->netmask_addr; 161 #endif 162 163 print_ip_settings(&ip_addr, &netmask, &gw); 164 165 #endif 166 /* Start the application */ 167 start_application(); 168 169 /* Loop to receive and process packets */ 170 while (1) { 171 if (TcpFastTmrFlag) { 172 tcp_fasttmr(); 173 TcpFastTmrFlag = 0; 174 } 175 if (TcpSlowTmrFlag) { 176 tcp_slowtmr(); 177 TcpSlowTmrFlag = 0; 178 } 179 xemacif_input(echo_netif); 180 /* Send data */
```



```
181 transfer_data(); 182 } 183 184 /* never  
reached */ 185 cleanup_platform(); 186 18  
7 return 0; 188 }
```

Some precompiled instructions in the code have not been deleted, such as LWIP_IPV6 and LWIP_DHCP. If readers want to try these two functions in the future, they can be used directly, so they will not be deleted here.

In line 81 of the code, the physical address of the development board is set, which can be modified. The official default physical address is used here.

In line 86 of the code, use `init_platform` to initialize the hardware platform, including configuring timers and related interrupts. The timer period is 250ms.

Use the `print_app_header()` function to print out relevant test information at line 100 of the code. Then use the `lwip_init()` function to initialize the lwip protocol stack.

In code Line 106 uses the `xemac_add()` function to add the physical address to the protocol stack.

In lines 130-133 of the code, use `platform_enable_interrupts()` to turn on interrupts, and then use the `netif_set_up()` function to start the network.

In lines 140-156 of the code, since the program turns on DHCP by default, DHCP will automatically allocate IP addresses at the beginning. If the reader does not want to turn it on, you can set the macro definition of DHCP to 0. If the dynamic IP address fails to be obtained after timeout, the default static IP address will be used. Readers can modify it at will, but they must ensure that the computer and development board are in the same network segment.

In line 167 of the code, `start_application()` is used to start the application function, which is defined in `echo.c`, creates a TCP service and sets the callback function of the service. When we establish a TCP connection and send data, the callback function resends the sent data back to the client.

In the while loop, data packets will be continuously received, and the timer interrupt will change the two flags `TcpFastTmrFlag` and `TcpSlowTmrFlag` at 250ms and 500ms. Data packet reception is processed in the `xemacif_input()` function. After receiving, the data is passed to lwIP, and then lwIP calls the relevant callback function for processing. The reader will not analyze too much here. Readers who are interested can read the code carefully.

After the writing is completed, `crtl+` is saved and compiled, and the program is ready to be downloaded.

31.3.4 Board verification

As shown in Figure 31-13, the development board is connected to the 12V DC power supply, XILINX downloader JTAG port, serial port and PS network port. After the lines are correctly connected, use the switch to power on the board.

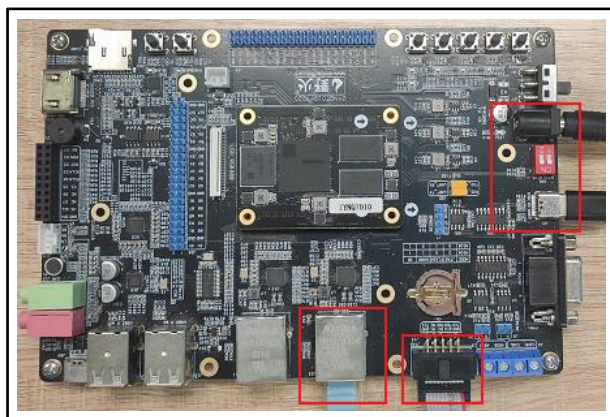


Figure 31-13 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system -> Run.

As shown in Figure 31-14, after the program is run, the serial port prints information related to the IP address and port. Since the author of this experiment used a network cable to connect to the router, the IP address was automatically assigned. If the reader is connected to a computer, you need to manually set the IP address at line 152~155 in Listing 31-1. Note that the IP address of the development board must be in the same network segment as the computer.

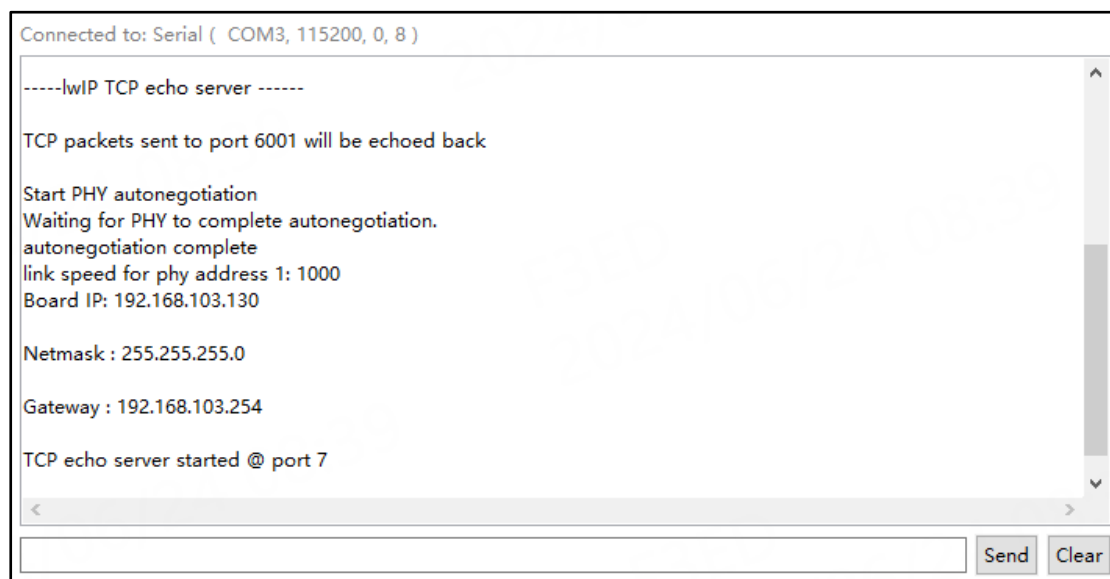


Figure 31-14 Serial port output information

Open the network debugging assistant, connect to the remote host (development board), and then send data to the development board. At this time, the network debugging assistant will also receive the data from the development board, and the data is the same, which means that lwip's TCP experiment was successful.

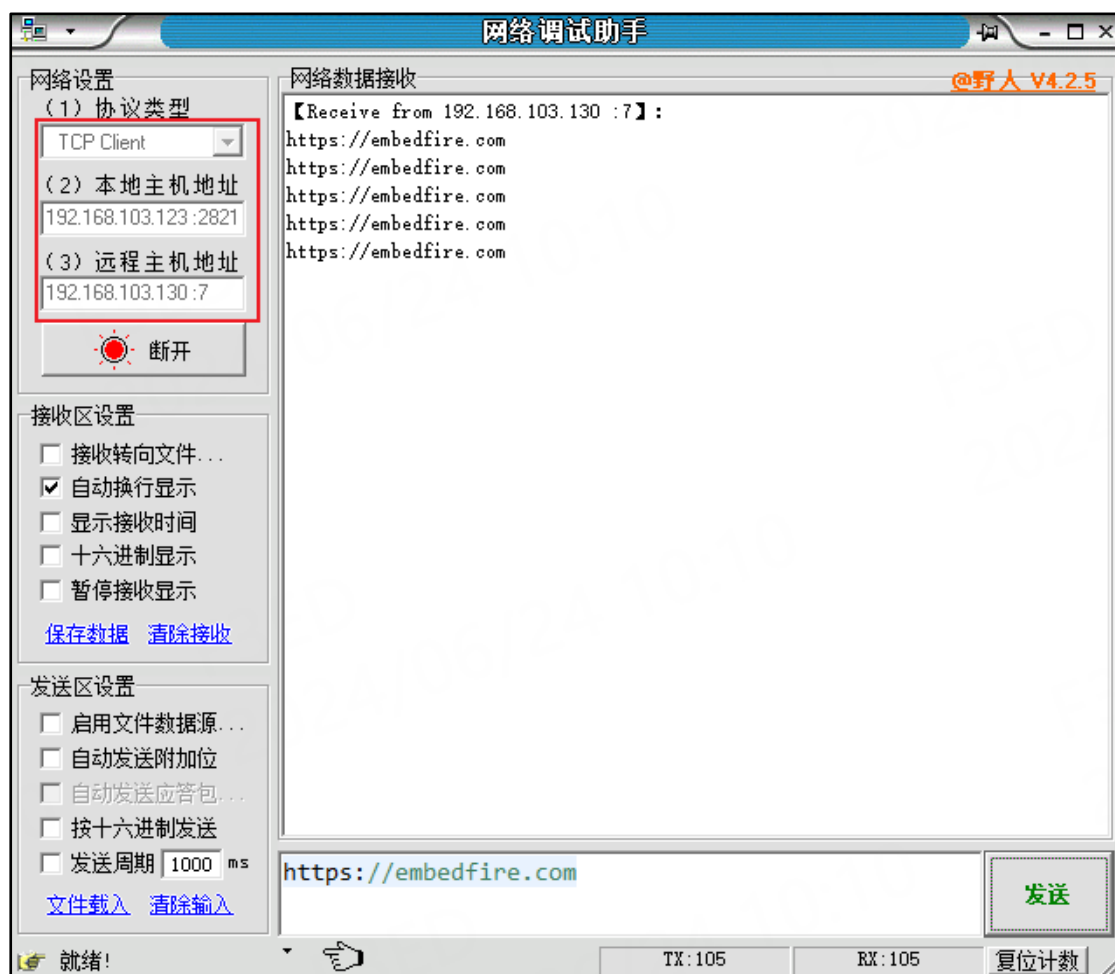


Figure 31-15 Network debugging assistant

31.4 End of chapter summary

This chapter has made some modifications to the LWIP Echo Server project template, mainly deleting files and content not used in this experiment. In this experiment, the computer and the development board communicate through TCP, in which the development board serves as the lwip data loopback server and the computer serves as a client.

31.5 Development training

You can use the telnet command in the CMD window to connect to the development board to achieve communication.

Chapter 32 UDP experiment based on lwip

32.1 Chapter Introduction

In the previous chapter, we learned how to use lwip to implement TCP communication. In this chapter, we will use the UDP protocol for communication.

32.2 Theoretical study

32.2.1 UDP packet format

After receiving the data packet, the network layer removes the IP header of the data packet and sends the remaining valid data packets to the transport layer. The transport layer provides end-to-end services between host application processes. Its basic functions are: segmentation and reorganization of data, addressing by port number, connection management, error control and flow control, and error correction functions.

There are two transmission protocols in the transport layer: TCP protocol based on byte stream, and UDP protocol based on message stream. Both protocols have their own advantages and disadvantages and are used in different scenarios. The TCP protocol is a connection-oriented streaming protocol that can ensure complete and orderly data transmission. It is a reliable protocol and is commonly used in situations that require high data integrity and correctness, such as file transfer. It occupies more resources and is slower than UDP. The UDP protocol is a connectionless transport layer protocol that provides simple and unreliable transaction-oriented information transmission services. Because no connection is required, the transmission speed is faster than TCP and it occupies less resources than TCP. It is suitable for the transmission of real-time data, such as video calls.

If the transport layer uses the UDP protocol, then the data packet transmitted to the transport layer is a UDP data packet, and the first 8 bytes are the UDP header. UDP data packet format, see Figure 32-1 for details.

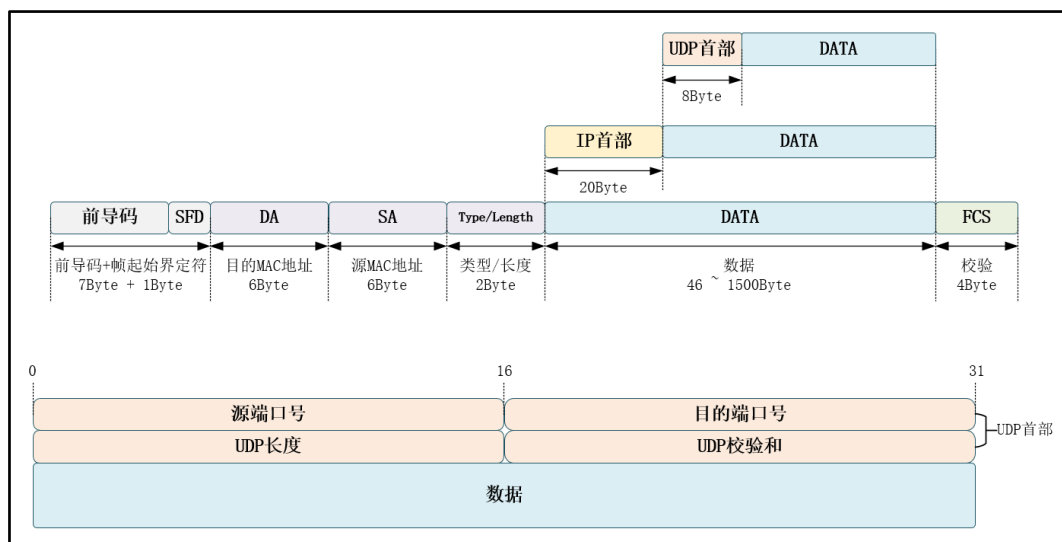


Figure 32-1 UDP packet format

UDP data packets are divided into two parts: UDP header and valid data. The UDP header consists of source port, destination port, message length and checksum. Compared with TCP, UDP has higher transmission efficiency and lower overhead, but it cannot guarantee the reliability of data transmission. The meaning of each field in the header is as follows in order:

Source port number (16Byte): The port number used by the source host application. Destination port number (16Byte): The port number used by the application program of the destination host. UDP length (16Byte): refers to the byte length of UDP header and UDP data. Because the UDP header length is 8 bytes, the minimum field value is 8. UDP checksum (16Byte): Provides the same function as the TCP checksum field; this field is optional.

32.2.2 UDP checksum

The calculation of UDP checksum requires three parts of data: UDP pseudo header, UDP header and valid data. The pseudo header contains some fields of the IP header. Its purpose is to allow UDP to check twice whether the data has arrived at the destination correctly. It is simply used for verification. Schematic diagram of parameters related to UDP verification calculation, see Figure 32-2 for details.

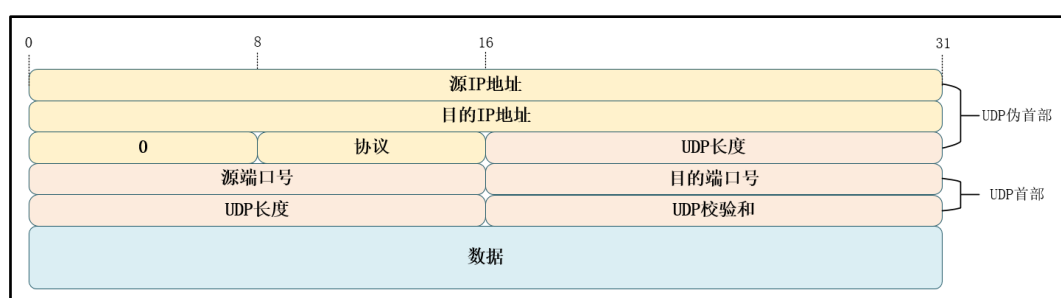


Figure 32-2 Schematic diagram of parameters related to UDP check calculation

Now that we know the relevant parameters required for UDP verification, we start to explain the UDP checksum calculation method. UDP checksum calculation steps: The check byte is forced to 0, and the three parts of data are divided into 2 bytes, that is, 16 bits, and added separately. If the result is greater than FFFF, then add the high 16 bits and the low 16 bits until the final result is 16 bits. The calculated result is inverted as the UDP checksum byte.

Example: The relevant data is shown in the figure below to calculate the UDP checksum.

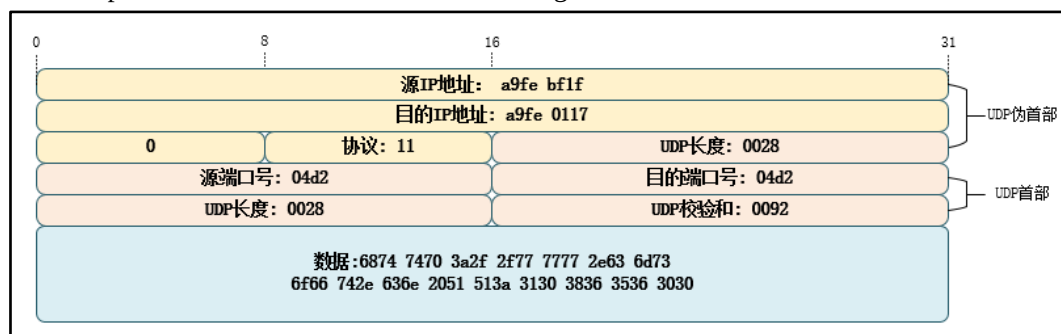


Figure 32-3 UDP checksum calculation

(1) Set checksum field 0092 to 0000:

a9 fe bf 1f a9 fe 01 17 00 11 00 28 04 d2 04 d2 00 28 00 00 68 74 74 70 3a 2f 2f 77 77 7
7 2e 63 6d 73 6f 66 74 2e 63 6e 20 51 51 3a 31 30 38 36 35 36 30 30.

(2) Using 2 bytes as a unit, sum the data in reverse code:

$a9fe + bf1f + a9fe + 0117 + 0011 + 0028 + 04d2 + 04d2 + 0028 + 0000 + 6874 + 7470 + 3a2f + 2f77 + 7777 + 2e63 + 6d73 + 6f66 + 742e + 636e + 2051 + 513a + 3130 + 3836 + 3536 + 3030 = 6\ ff67.$

(3) Add carry (6) to the lower 16 bits (ff67): $6 + ff67 = ff6d$. (4) Invert ff6d to get: c hecksum = 0092.

32.3 Practical exercises

32.3.1 Experimental objectives

Modify the Iwip Echo Sever routine that comes with the SDK by building a PS-side Ethernet to enable UDP data loopback.

32.3.2 Hardware resources

This experiment uses the PS side Ethernet interface and UART.

32.3.3 Programming

1. Overall description

According to the experimental goals, we can draw the overall structure as shown in Figure 32-4.

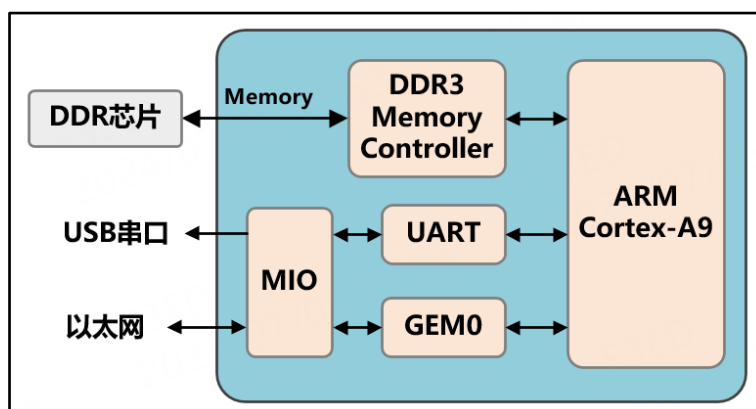


Figure 32-4 Overall block diagram

In the real hardware design where UART is used to print relevant information, the Gigabit Ethernet controller uses GEM0 to connect to the external Ethernet via MIO. It is the same as the previous chapter except that the communication protocol is changed to UDP.

Detailed steps for project creation can be found in the previous chapters. Below we only list the configuration process.

2. Hardware design

Same as "TCP Experiment Based on Iwip".

3. Software design

Make modifications in the lwip Echo Sever routine.

Code Listing 32-1 Modified code (main.c)

```
1//DEFINE STATEMENTS TO INCREASE SPEED 2 #undef LWIP_TCP 3 #undef LWIP
_DHCP 4 // #undef CHECKSUM_CHECK_UDP 5 // #undef LWIP_CHECKSUM_ON_COPY 6 // #undef CHECKSUM_GEN_UDP 7 8 #include <stdio.h> 9 10 #inc "xparameters.h" 11
12 #include "netif/xadapter.h" 13 14 #include "platform.h" 15 #include "platform_config.h"
16 #if defined (__arm__) || defined (__aarch64__) 17 #include "xil_printf.h" 18 #endif 19 20 #include "lwip/udp.h" 21 #include "xil_cache.h" 22 23 24 /* defined by each RAW mode a
pplication */ 25 void print_app_header(); 26 int start_application(); 27 //int transfer_data(); 28 void tcp_fasttmr(void); 29 void tcp_slowtmr(void); 30 31 /* missing declaration in lwIP */
32 void lwip_init(); 33 34 extern volatile int TcpFastTmrFlag; 35 extern volatile int TcpSlowTmrFlag; 36 /* set up netif stuctures */ 37 static struct netif server_netif; 38 struct netif *echo_netif; 39 40 // Global Variables to store results and handle data flow 41 int Centroid; 42 43 // Global variables for data flow 44 volatile u8 IndArrDone; 45 volatile u32 EthBytesReceived; 46 volatile u8 SendResults; 47 volatile u8 DMA_TX_Busy; 48 volatile u8 Error; 49 50 // Global Variables for Ethernet handling 51 u16_t RemotePort = 8; 52 ip_addr_t RemoteAddr; 53 struct udp_pcb send_pcb; 54 55 void print_ip(char *msg, ip_addr_t *ip) 56 { 57 print(
msg); 58 xil_printf("%d.%d.%d.%d\n", ip4_addr1(ip), ip4_addr2(ip)
```

```
59 ip4_addr3(ip), ip4_addr4(ip));
```

```
),
```

```
60 } 61 62 void print_ip_settings(ip_addr_t *ip, ip_addr_t *mask, ip_addr_t *gw) 63 { 64 65 print_ip("Board IP
: ", ip); 66 print_ip("Netmask : ", mask); 67 print_ip("Gateway : ", gw); 68 } 69 70 /* print_app_header: functi
on to print a header at start time */ 71 void print_app_header() 72 { 73 xil_printf("\n-----lwIP UDP GetCentro
id Application-----\n"); 74 } 75 76 int main() 77 { 78 ip_addr_t ipaddr, netmask, gw /*, Remotenetmask, Re
motegw*/; 79 int status = 0; 80 81 /* the mac address of the board. this should be unique per board 82 */ 83 un
signed char mac_ethernet_address[] = 84 { 0x00, 0x0a, 0x35, 0x00, 0x01, 0x10 }; 85 86 /* Use the same struct
ure for the server and the echo server */ 87 echo_netif = &server_netif; 88 init_platform(); 89 90 /* initialize I
P addresses to be used */ 91 IP4_ADDR(&ipaddr, 192, 168, 103, 122); 92 IP4_ADDR(&netmask, 255, 255, 2
55, 0); 93 IP4_ADDR(&gw, 192, 168, 103, 254); 94 IP4_ADDR(&RemoteAddr, 192, 168, 103, 114); 95 print
_app_header(); 96 97 /* Initialize the lwip for UDP */ 98 lwip_init(); 99 100 /* Add network interface to the n
etif_list, and set it as default
```

```
101 */ 102 if (!xemac_add(echo_netif, &ipaddr, &netmask, 103 &gw, mac_ethernet_address, 104 PLATFO
RM_EMAC_BASEADDR)) { 105 xil_printf("Error adding N/W interface\n"); 106 return -1; 107 } 108 neti
f_set_default(echo_netif); 109 110 /* now enable interrupts */ 111 platform_enable_interrupts(); 112 113 /*
specify that the network if is up */ 114 netif_set_up(echo_netif); 115 116 xil_printf("ZYNQ IP settings: \n")
; 117 print_ip_settings(&ipaddr, &netmask, &gw); 118 xil_printf("Remote IP settings: \n"); 119 print_ip("R
emote IP: ", &RemoteAddr); 120 121 /* start the application */ 122 udp_recv_init(&RemoteAddr); 123 if (s
tatus != 0){ 124 xil_printf("Error in start_application() with code: %d\n", 125 status);
```



```
126 goto ErrorOrDone; 127 } 128 129 /* receive and process packets */ 130 while (Error == 0) { 131 if (TcpFastTmrFlag) { 132 tcp_fasttmr(); 133 TcpFastTmrFlag = 0; 134 } 135 if (TcpSlowTmrFlag) { 136 tcp_slowtmr(); 137 TcpSlowTmrFlag = 0; 138 } 139 140 /* Enable reception */ 141 xemacif_input(echo_netif); 142 } 143 144 // Jump point for failure 145 ErrorOrDone: 146 xil_printf("Catastrophic Error! Shutting down and exiting..\n"); 147 cleanup_platform(); 148 return 0; 149 }
```

In line 83 of the code, the physical address of the development board is set. It can be modified here. The author uses the official default physical address.

In line 88 of the code, use `init_platform` to initialize the hardware platform, including configuring timers and related interrupts. The timer period is 250ms.

Set the static IP address of the development board in lines 91-93 of the code, making sure it is in the same network.

In line 94 of the code, set the IP address of the remote host, which is the IP address of the computer. It is needed when sending the received data back to the computer.

Use the `print_app_header()` function to print out relevant test information in lines 95-98 of the code. Then use the `lwip_init()` function to initialize the lwip protocol stack.

In line 102 of the code, use the `xemac_add()` function to add the physical address to the protocol stack.

In line 111 of the code, use `platform_enable_interrupts()` to turn on interrupts, and then use the `netif_set_upo()` function to start the network.

In line 122 of the code, use `udp_rcv_init()` to start the application function, which is defined in `udp_process.c`, create a UDP service and set the data acceptance callback function and connect to the remote host (computer). In the callback function, the sent data is sent back to the client.

Code Listing 32-2 UDP application (udp_process.c)

```
1 #include <stdio.h> 2 #include <string.h> 3 #include "lwip/err.h" 4 #include "lwip/udp.h" 5 #include "lwipopts.h" 6 #include "netif/xadapter.h" 7 #include "xil_printf.h" 8 #include "sleep.h" 9 10 #include "udp_process.h" 11 12 struct udp_pcb *pcb = NULL; 13 static unsigned local_port = 7; /* server port */ 14 static unsigned remote_port = 7;
```

```
15 const ip_addr_t *ipaddr; 16 17 void udp_rcv_callback(void *arg, struct udp_pcb *tpcb, struct pbuf *p,
18 ip_addr_t *addr, u16_t port) 19 { 20 err_t err; 21 22 err = udp_sendto(tpcb, p, ipaddr, remote_port); 23 i
f (err != ERR_OK) 24 { 25 xil_printf("Error on udp_send: %d\r\n", err); 26 pbuf_free(p); 27 return; 28 } 29
pbuf_free(p); 30 return; 31 } 32 33 int udp_rcv_init(const ip_addr_t *remote_addr) 34 { 35 err_t err; 36 37
/* create new UDP PCB structure */ 38 pcb = udp_new(); 39 if (!pcb) { 40 xil_printf("Error creating PCB.
Out of Memory\r\n"); 41 return -1; 42 } 43 44 /* bind to local port */ 45 err = udp_bind(pcb, IP_ADDR_A
NY, local_port); 46 if (err != ERR_OK) { 47 xil_printf("udp_rcv_init: Unable to bind to port %d: err = 48
%d\r\n", local_port, err); 49 return -2; 50 } 51 52 ipaddr = remote_addr; 53 err = udp_connect(pcb, remote_
addr, remote_port); 54 if (err != ERR_OK) 55 xil_printf("error on udp_connect: %x\r\n", err); 56 udp_rcv(
pcb, (udp_rcv_fn)udp_rcv_callback, NULL); 57 58 xil_printf("local_port: %d\r\n", local_port); 59 xil_printf("remote_p
ort: %d\r\n", remote_port); 60 61 return 0; 62 }
```

Back to the main function, in the while loop, the timer interrupt will continuously receive data packets and change the two flags `TcpFastTmrFlag` and `TcpSlowTmrFlag` at 250ms and 500ms. Data packet reception is processed in `xemacif_input()`. After receiving, the data is passed to lwIP, and then lwIP calls the relevant callback function for processing. The reader will not analyze too much here. Readers who are interested can read the code carefully.

After the writing is completed, `ctrl+` is saved and compiled, ready to download the program.

32.3.4 Board verification

As shown in Figure 32-5, the development board is connected. After the lines are connected correctly, turn on the switch.

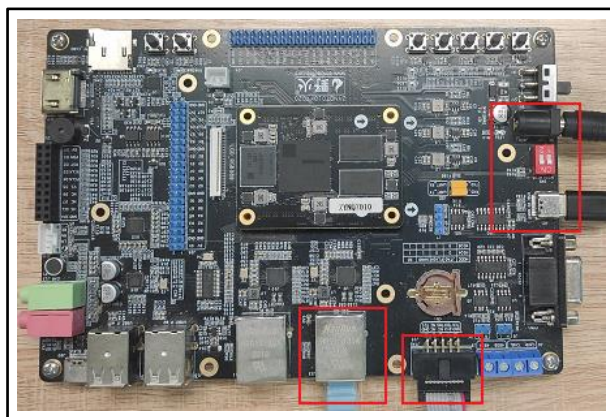


Figure 32-5 Program download connection diagram

Download the program to the development board. Right-click the application project name in the project, then select Run As -> Run Configurations -> and check Reset entire system -> Run.

As shown in Figure 32-6, after the program is run, the serial port prints IP address and port-related information. In this experiment, the local port and remote port are both set to 7.

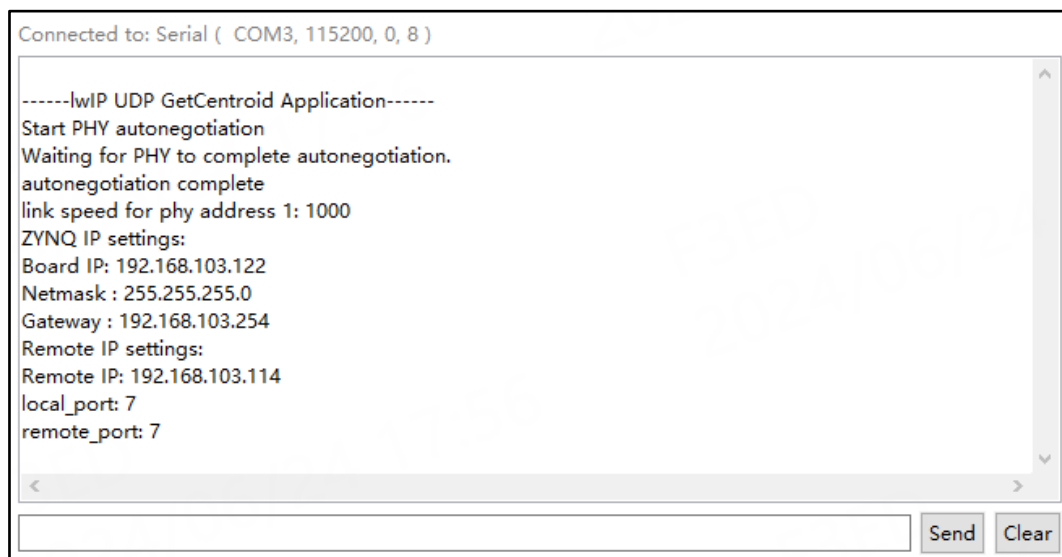


Figure 32-6 Serial port output

As shown in Figure 32-7, open the network debugging assistant, connect to the remote host (development board), and then send data to the development board. At this time, the network debugging assistant will also receive the data from the development board, and the data is the same, which means that lwip's UDP experiment was successful.

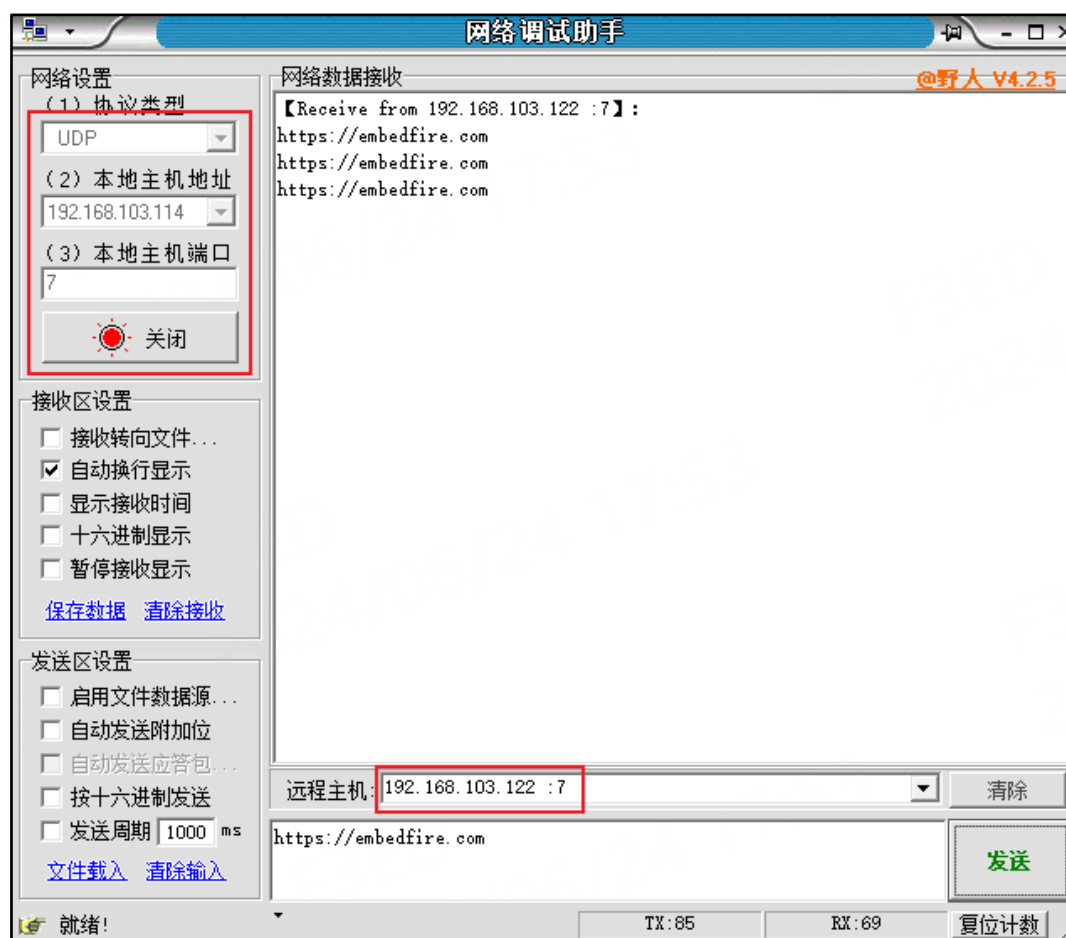


Figure 32-7 Data sending and receiving

32.4 End of chapter summary

In this experiment, we learned how to use lwip for UDP communication between the computer and the development board.

32.5 Development training

When creating a new project and selecting a template, we can see that in addition to lwip Echo Server, there are other project templates related to lwip, which you can modify to achieve communication between the computer and the development board.